
SHANNON ROBOTICS

Probabilistic Robotics via Rust

The FCP Way

Complete mathematical foundations, interactive intuition, and working implementations in Rust — for robots that must act without ever knowing exactly where they are.

Foundation · Conceptual · Practical

Probabilistic Robotics via Rust — The FCP Way

Print edition, generated from the interactive web edition.

This book takes Thrun, Burgard and Fox’s *Probabilistic Robotics* as its spine and brings it forward: factor graphs as the inference back-end, estimation on Lie groups, scan matching and pose-graph SLAM, visual-inertial odometry, truncated signed distance fields, sampling-based stochastic control, online POMDP solvers, and active SLAM. Five further texts serve as baselines throughout — Lynch & Park’s *Modern Robotics* and Craig’s *Introduction to Robotics* for the geometry, Niku for sensing hardware, Choset et al.’s *Principles of Robot Motion* for planning, and Spong, Hutchinson & Vidyasagar for modelling and control.

Every chapter of the web edition carries interactive simulations that run the same algorithms the text derives. In this print edition each appears as a described figure with its widget identifier, so the two editions can be read side by side.

Set in Space Grotesk and a Palatino-class serif. Composed with L^AT_EX.

How to read this book

Every chapter makes three passes over its subject, and they are meant to be read in this order.

Conceptual comes first, because intuition should precede formalism. In the web edition you meet an interactive figure before you meet an equation; here that figure is described, and the description tells you what it would show you.

Foundation is the mathematics, in full. Definitions are precise, assumptions are stated where they are made rather than buried, and derivations are carried to the end. Long algebra sits in marked derivation blocks, so you can take the result on faith the first time through and come back for the proof later.

Practical is the Rust. Not pseudocode dressed up as code: real types, real crates, code you could lift into a robot.

The running lab

Two worlds recur throughout, so that every new method can be compared against the last on identical ground. The **Hallway** is a one-dimensional corridor with indistinguishable doors — it exists because a belief over a 1-D world can be drawn as a curve, which makes the Bayes filter visible in a way no two-dimensional picture manages. The **Apartment** is a 2-D floorplan with rooms, doorways and a long corridor, sensed by a simulated LiDAR, and deliberately a little symmetric, because ambiguity is where probabilistic methods earn their keep.

The robot is **Rusty**, a differential-drive rover with wheel encoders and a laser scanner. Rusty is built in Chapter 4 and appears in every chapter after it; by Chapter 26 it explores and maps an apartment it has never seen.

Colour carries meaning

One convention runs through the prose, the equations, the figures and the code comments alike. It is worth internalising before you start:

■ prior ■ prediction ■ measurement ■ posterior ■ ground truth

Blue is always what you believed before. Orange is always what you believe after moving. Green is always what a sensor said. Purple is always what you believe after taking that sensor seriously. Grey, where it appears, is the truth the robot

does not have access to. The teal used for chapter furniture is deliberately outside this set, so it can never be mistaken for data.

Contents

How to read this book	v
I Foundations: The Robot and Its Uncertainty	1
1 The Robot That Doubts	3
2 Probability: The Language of Uncertainty	25
3 The Geometry of Motion	53
4 Rusty, Sensors, and the Simulator	77
II The Bayes Filter Family	101
5 The Bayes Filter	103
6 Kalman Filters	117
7 Beyond Linearity: EKF, UKF, and Manifolds	143
8 Nonparametric Filters	171
III Probabilistic Models	199
9 Probabilistic Motion Models	201
10 Probabilistic Sensor Models	229
IV Localization	261
11 Localization I: Tracking with Gaussians	263
12 Localization II: Global Localization	295

V Mapping and SLAM	321
13 Occupancy Grid Mapping	323
14 The SLAM Problem and EKF SLAM	349
15 SLAM as Least Squares: Factor Graphs	377
16 Scan Matching and Pose-Graph SLAM	405
17 FastSLAM and Rao-Blackwellization	433
18 Visual and Visual-Inertial SLAM	461
19 Modern Map Representations	489
 VI Planning and Acting under Uncertainty	 517
20 Motion Planning: From Geometry to Probability	519
21 Decision Making I: MDPs and Value Iteration	557
22 Decision Making II: POMDPs and Belief-Space Planning	591
23 Stochastic MPC: MPPI and Friends	627
24 Exploration and Active SLAM	653
 VII Frontiers and Integration	 685
25 Learning in the Loop	687
26 Capstone: A Complete Autonomous Robot	719

Part I

Foundations: The Robot and Its Uncertainty

The Robot That Doubts

FOUNDATIONS · Foundational · 40 min



A robot that carries a notion of its own uncertainty and that acts accordingly is superior to one that does not.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 1

Every robot you have ever seen is lying to itself about where it is. Not through a bug, and not through poor engineering — through arithmetic. A robot that integrates its own commands produces a number that is internally perfect and externally wrong, and it has no way to notice, because nothing in that computation carries a record of how wrong it might be.

This chapter is the book's argument for the alternative. The claim is not the soft one that robots should be humble. It is the sharp one, made by Thrun, Burgard, and Fox in the sentence at the top of this page and defended for the next twenty-five chapters: a robot that represents its own uncertainty and acts on it is *measurably* better than one that does not, in ways you can put a number on. We will price that claim in the third section, using a decision so small you can check it in your head.

What we will not do is treat uncertainty as a nuisance to be engineered away. Uncertainty is state. It is estimated, it is propagated, it is acted upon — and, as the third section shows with a loss table you can add up by hand, it changes what the robot should do next.

◎ AFTER THIS CHAPTER YOU CAN

- Name the five sources of uncertainty and point at each one in a real environment
- State the difference between a point estimate and a belief, precisely
- Show by construction that the action optimal against a belief can beat

the action optimal against the best guess

- Localize a robot in a ten-cell corridor by hand, and check every number against running code
- Say what the probabilistic paradigm costs, not just what it buys

ASSUMED

- Programming fluency in some language — Rust is not assumed yet
- High-school probability: what a distribution is, what it means to normalize one
- Comfort with vectors at the "seen them before" level

1.1 Every robot is lying to itself

Meet **Rusty**, a differential-drive rover with two wheels, two encoders, and a scanning laser. Rusty is going to be with us for the whole book: Chapter 4 builds it properly, and by Chapter 26 it explores and maps an apartment it has never seen.

Right now Rusty has one job: drive a 2.4 m square and come back to where it started.

INTERACTIVE · w1.2

The Dead Reckoner

Knowing the commands is not knowing the position: integration turns a small, honest actuation error into an unbounded one, and no amount of care in the arithmetic fixes it.

Run this simulation in the web edition: `/chapters/ch01-robot-that-doubts #w1.2`

Watch a full lap before reading on. Three things are happening, and only one of them is a surprise.

The orange path is not an error. It is the pose Rusty computes by integrating its own commands — *dead reckoning*, the oldest navigation algorithm there is. Drive 0.6 m/s for four seconds, turn ninety degrees, repeat. The arithmetic is exact, the square closes to within a billionth of a metre — there is a test at the end of this chapter that says so — and if you asked Rusty where it was, it would answer with total confidence.

The gray path is where Rusty actually went. One wheel had a fraction more traction than the other. The turn was 89.4 degrees instead of 90. The floor was a millimetre out of level. Each of those errors is small, unbiased, and entirely ordinary — and each one enters the pose and stays there.

The error never comes back down. That is the surprise, and it is worth being precise about why. The individual errors are zero-mean: they are as likely to be positive as negative, so they do not push Rusty in any particular direction. But integration accumulates them, and a sum of zero-mean errors is a random walk, whose spread grows without bound. Worse, an error in *heading* multiplies the distance travelled after it, so a third of a degree at the first corner is centimetres by the fourth. Re-roll the seed: the drift is a different size and a different direction every time, and no amount of staring at the commands will tell you which.

💡 THE PROBLEM, IN ONE LINE

Dead reckoning is never internally inconsistent. It is only ever wrong, silently, and by an amount that grows. Everything in this book is a way of getting that amount back — as a number the robot itself can compute.

1.2 Five places doubt comes from

Uncertainty is not one phenomenon. Thrun, Burgard, and Fox open *Probabilistic Robotics* by splitting it into five sources, and the taxonomy has aged perfectly: every failure mode in the following chapters is one of these five wearing a costume.

💻 INTERACTIVE · w1.3

Five uncertainties, one apartment

Uncertainty is not one thing that better engineering removes; it is five different things, each with its own address in the world and its own chapter in this book.

Run this simulation in the web edition: </chapters/ch01-robot-that-doubts#w1.3>

Two observations before we move on, because they set up the entire structure of the book.

Four of the five are properties of the world, and one is a property of your computer. Sensor noise, actuator slip, unpredictable environments, and imperfect models exist whether or not anyone is looking. Computation is different: it is a budget you choose. It is also the source that decides which of the algorithms in this book you are actually allowed to run — the exact posterior over Rusty's pose is a function over a continuum, and no robot has ever computed one. Every filter in Part II is an answer to the question *what do I do instead, and what does that approximation cost me?*

None of the five gets smaller when you buy better hardware. They get smaller in proportion. A LiDAR with one-centimetre noise instead of three still has noise; a warehouse AMR with hard rubber wheels on sealed concrete still

slips when it takes a corner loaded; a sidewalk delivery robot with a perfect map still meets a dog. Doubling the sensor budget buys you a constant factor. Representing the uncertainty buys you an algorithm that keeps working when the constant factor runs out.

1.3 A best guess is not a belief

Here is the distinction the rest of the book is built on.

A **point estimate** is a single value $\hat{x}_t$ — the robot’s best guess about where it is. It is what dead reckoning produces, what a GPS receiver reports, and what almost every non-probabilistic system passes downstream.

A **belief** is a distribution over *every* value the state could take, weighted by how plausible it is given everything that has happened. It is not a location. It is a function over locations.

The temptation is to treat those as the same object with different amounts of decoration: surely the belief is just the point estimate plus some error bars, and if you only ever act on the peak, the decoration does not matter? That intuition is wrong, and it is wrong in a way that costs money. Here is the smallest example that shows it.

Rusty is at a T-junction. The charger is at the end of the left corridor or the right one, and Rusty’s belief after a long drive is $p(\text{left}) = 0.55$, $p(\text{right}) = 0.45$ — genuinely uncertain, but leaning left. Three actions are available: commit left, commit right, or spend one unit of time taking another measurement, after which the ambiguity is resolved and Rusty drives straight to the charger. Committing to the wrong corridor costs ten units: the long walk back, plus the risk of running flat.

DERIVATION The argmax fallacy — why the best guess loses money

sensewhilep < 1 - c/L

Step 1 — write down the loss. A loss function $L(a, x)$ says what it costs to take action a when the world is in state x . Here:

$L(a, x)$	$x = \text{left}$	$x = \text{right}$
go left	0	10
go right	10	0
sense again	1	1

Step 2 — the belief-follower averages the loss over the belief. It chooses the action minimizing expected loss under the full distribution:

$$a^* = \arg \min_a \mathbb{E}_{\text{bel}(x)} [L(a, x)] = \arg \min_a \sum_x \text{bel}(x) L(a, x)$$

Substituting the numbers:

$$\mathbb{E}[\text{go left}] = 0.55 \cdot 0 + 0.45 \cdot 10 = 4.5$$

$$\mathbb{E}[\text{go right}] = 0.55 \cdot 10 + 0.45 \cdot 0 = 5.5$$

$$\mathbb{E}[\text{sense again}] = 1$$

The belief-follower senses. Expected cost: 1.

Step 3 — the mode-follower collapses first, then evaluates. It replaces the belief with its most probable state and asks what is best *there*:

$$\hat{a} = \arg \min_a L(a, \arg \max_x \text{bel}(x))$$

At the mode, $x = \text{left}$, going left costs 0 and sensing costs 1. So the mode-follower goes left, and pays $0.45 \cdot 10 = 4.5$ in expectation. It is not being reckless; from inside its own world model, sensing is *pure overhead*. A point estimate has nothing left to learn, because it has already thrown away the only thing that made learning valuable — the doubt.

Step 4 — the general threshold. Let $p \geq \frac{1}{2}$ be the probability of the more likely corridor, L the cost of committing wrongly, and c the cost of sensing. The best commitment costs $(1 - p)L$; sensing costs c . So sensing is optimal exactly when

$$c < (1 - p)L \quad \Longleftrightarrow \quad p < 1 - \frac{c}{L}$$

With $L = 10$ and $c = 1$ the threshold is $p^* = 0.9$: below nine-tenths certainty, look again; above it, commit. The quantity $(1 - p)L$ is the **value of perfect information** — literally what the doubt is worth. A mode-follower computes it as zero, always, which is why it never gathers information deliberately.

The general statement is just as short. $\hat{a}$ minimizes a loss evaluated at *one point* of the belief; a^* minimizes its *average* over the belief. Those two agree only when the loss is an affine function of the state — and they can never agree when one of the available actions is "gather more information", because the value of that action is a property of the belief's spread, and a point estimate has no spread. Nothing bounds the gap: rescale the ten and it grows with it. This example is the tiger problem in disguise; Chapter 22

grows it into belief-space planning, where "sense again" stops being one of three options and becomes a policy over a continuous belief space, and Chapter 24 turns the same quantity into a reason to drive somewhere.

Three and a half units of cost, in a decision with two states and three actions. Scale that to a delivery robot choosing between an elevator and a stairwell, or a warehouse AMR deciding whether it is confident enough to enter a narrow aisle, and the argument stops being cute.

💡 WHY BELIEFS EXIST

Beliefs are not for looking at. They are for *acting on*. The moment you collapse a distribution to its mode, you lose the ability to choose actions whose entire value is that they reduce uncertainty — and information gathering is most of what a good robot does.

1.4 The vocabulary you need for one chapter

Four symbols carry the whole book. They are introduced here informally, formalized in Chapter 2, and made into an algorithm in Chapter 5.

x_t	State at time t — everything about the world that matters for predicting the future. For Rusty it starts as a pose (x, y, θ) and grows to include the map in Chapter 14.
u_t	Control asserted between $t-1$ and t : a wheel command, an odometry reading. Acting always costs information — the world changes and the robot is not sure how.
z_t	Measurement at time t : a laser scan, a door detection, a landmark bearing. Sensing always gains information.
$\text{bel}(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$	The belief: the posterior over the state given every measurement and every control so far. The central object of this book.
$\overline{\text{bel}}(x_t)$	The predicted belief — after folding in the control, before folding in the measurement. Orange, everywhere in this book.
η	A generic normalizer. Whenever a distribution is multiplied by something and must sum to one again, η is the number it gets divided by.

Two things are worth noticing in that table before anything is derived from it.

The belief is conditioned on $z_{1:t}$ and $u_{1:t}$ — *all* the data, from the beginning of time. That is what makes it a summary of the robot's entire history, and what makes maintaining it recursively, without storing the history, the central trick of Chapter 5.

And u and z enter with opposite signs, informationally. Motion smears the belief out; sensing sharpens it. That asymmetry is not an accident of any particular algorithm — it is the shape of the problem, and it is what the next section shows you in a picture.

1.5 The hallway, worked by hand

Here is the oldest thought experiment in probabilistic robotics, and it is still the best one.

A corridor is divided into ten cells, numbered 0 to 9, and it loops: walking right from cell 9 brings you to cell 0. Three of the cells have doors — cells 1, 4, and 5 — and they are *indistinguishable*. Rusty has one sensor, a door detector, and it is imperfect: it reports "door" correctly 60% of the time when facing one, and cries "door" 20% of the time when there is none.

Rusty is switched on somewhere in this corridor, with no idea where. It senses. It drives one cell to the right. It senses again. Where is it?

INTERACTIVE · w1.1

The Hallway Belief Machine (preview)

A belief is a whole distribution. Sensing multiplies it, moving shifts it, and ambiguity dies from the sequence — not from any single reading.

Run this simulation in the web edition: </chapters/ch01-robot-that-doubts/#w1.1>

Play the loop once, and then look at what happened at each step.

The uniform prior is the honest starting point. Ten cells, no information: $\text{bel}(x) = 0.1$ everywhere. This is not the robot being pessimistic. It is the robot being *calibrated* — any peak in that histogram would be a claim it has no evidence for.

One door sighting gives three peaks, not one. The sensor did exactly its job and Rusty still does not know where it is. Any representation that insists on one number for the answer must pick one of those three peaks and be wrong two times in three — and it will be wrong *confidently*, with nothing downstream to record that a guess was ever made. This is exactly the failure the Kalman filter of Chapter 6 cannot survive, and the reason Chapter 8 exists.

The non-door cells do not go to zero. They drop to 0.0625 and stay there, because the detector cries "door" at a blank wall one time in five, so a blank wall never becomes impossible. That matters more than it looks: a belief that assigns probability zero to a hypothesis can never recover it, whatever evidence arrives later. Refusing to write down a zero is the cheapest robustness in all of robotics.

Ambiguity dies from the sequence, not from any single reading. After the move and the second sighting, cell 5 holds exactly three times the belief of its nearest rival — not because the second reading was better than the first, but

because only cell 5 is a door *that is also one step to the right of a door*. The pattern did the work.

DERIVATION Sense, move, sense — every number

$$\text{bel}(5) = 9/26 \approx 0.3462$$

Write bel as a ten-vector. The measurement likelihood for the reading $z = \text{door}$ is

$$p(z = \text{door} \mid x) = \begin{cases} 0.6 & x \in \{1, 4, 5\} \\ 0.2 & \text{otherwise} \end{cases}$$

Step 1 — the prior. Maximum ignorance over ten cells: $\text{bel}(x) = 0.1$ for every x .

Step 2 — sense "door": multiply, then normalize. Pointwise product with the likelihood:

$$\text{bel}(x) = \eta p(z \mid x) \text{bel}(x)$$

Unnormalized, door cells hold $0.1 \times 0.6 = 0.06$ and the rest $0.1 \times 0.2 = 0.02$. The sum is $3(0.06) + 7(0.02) = 0.32$, so $\eta = 1/0.32$ and

$$\text{bel} = (0.0625, \mathbf{0.1875}, 0.0625, 0.0625, \mathbf{0.1875}, \mathbf{0.1875}, 0.0625, 0.0625, 0.0625, 0.0625)$$

That 0.32 is not junk: it is $p(z)$, the probability of getting the reading you got, averaged over the belief. Nearly a third of the corridor looks like a door to this sensor, so a door reading is not very surprising and not very informative. Chapter 5 turns this number into an outlier detector, and Chapter 12 into a kidnapping alarm.

Step 3 — move one cell right. Motion is perfect for exactly one chapter, so the belief shifts cyclically: $\overline{\text{bel}}(x) = \text{bel}((x - 1) \bmod 10)$. The three peaks move from cells 1, 4, 5 to cells 2, 5, 6:

$$\overline{\text{bel}} = (0.0625, 0.0625, \mathbf{0.1875}, 0.0625, 0.0625, \mathbf{0.1875}, \mathbf{0.1875}, 0.0625, 0.0625, 0.0625)$$

Nothing was created or destroyed — the mass was permuted. (Real motion also *spreads* the belief, which is what makes prediction a convolution rather than a shift. That is the single change Chapter 5 makes to this line, and it is why the operation is called prediction rather than translation.)

Step 4 — sense "door" again. Multiply by the same likelihood. Now the door cells and the shifted peaks interact:

cell	0	1*	2	3	4*	5*	6	7	8	9
$\overline{\text{bel}}$	.0625	.0625	.1875	.0625	.0625	.1875	.1875	.0625	.0625	.0625
$\times p(z x)$	.2	.6	.2	.2	.6	.6	.2	.2	.2	.2
unnormalized	.0125	.0375	.0375	.0125	.0375	.1125	.0375	.0125	.0125	.0125

The sum is $4(0.0375) + 0.1125 + 5(0.0125) = 0.325$, so $\eta = 1/0.325$ and every entry lands on a clean twenty-sixth:

$$\text{bel} = \frac{1}{26} (1, 3, 3, 1, 3, \mathbf{9}, 3, 1, 1, 1)$$

Step 5 — read it out. $\text{bel}(5) = 9/26 \approx 0.3462$, against $3/26 \approx 0.1154$ at cells 1, 2, 4 and 6, and $1/26 \approx 0.0385$ everywhere else. One hypothesis is now three times as plausible as its nearest rival — from two readings that were individually worth almost nothing.

Sensing multiplied. Moving shifted. Those two operations, and nothing else, did all of this.

The two operations even have shapes, and they are the shapes of every estimator in this book.

ALGORITHM `sense_and_move(bel, u, z)` COST $O(N)$ per operation here -- $O(N^2)$ as soon as motion is uncertain (Chapter 5)

in a belief over N cells, a control, a measurement

out the updated belief

```

1: for all cells  $x$  do
2:    $\text{bel}(x) \leftarrow p(z | x) \cdot \text{bel}(x)$   // sensing multiplies
3: endfor
4:  $\eta \leftarrow \sum_x \text{bel}(x)$ ;  $\text{bel} \leftarrow \text{bel}/\eta$ 
5: for all cells  $x$  do
6:    $\overline{\text{bel}}(x) \leftarrow \text{bel}(x - u)$   // moving shifts
7: endfor
8: return  $\overline{\text{bel}}$ 
```

i NOTE

Those two loops are the two lines of `Bayes_filter` (Thrun et al., Table 2.1) — written here in the order this chapter used them, sense before move, and with the honest version of the shift on line 6 (a convolution against

the motion model rather than a permutation) postponed to Chapter 5, where the whole recursion is *derived* from Bayes rule rather than asserted. Everything after that chapter is a decision about how to represent *bel* so those two lines can be computed at all.

1.6 What the paradigm buys, and what it costs

The honest version of the sales pitch has two columns.

What it buys. Ambiguity becomes representable, so a robot can hold three hypotheses without picking one prematurely. Weak models become usable: probabilistic algorithms make far weaker demands on model accuracy than classical planners do, because they were built expecting the model to be wrong. Failure becomes graceful rather than catastrophic — a filter degrades toward ignorance, which is recoverable, instead of toward confident error, which is not. And, as the T-junction showed, information gathering becomes a first-class action rather than an afterthought.

What it costs. Computation, first: carrying a whole distribution is strictly more expensive than carrying a number, and in high dimensions it is exponentially so. Approximation, second: real state spaces are continuous, exact posteriors have infinitely many degrees of freedom, and every algorithm in this book replaces the true belief with something finite — a Gaussian, a grid, a set of samples. The approximation is where the failures live, which is why every chapter here names the one it is making.

That trade is why the book is structured as a sequence of representations rather than a single algorithm, and why no chapter is allowed to claim a method is "better" without saying what it is better *at*.

⚠ WHERE ROBOTS BREAK THIS

This chapter asserts robustness and graceful degradation; it does not demonstrate them. That is deliberate. Chapter 12 demonstrates recovery by kidnapping the robot mid-run, Chapter 14 demonstrates the opposite by watching a filter become confidently wrong, and Chapter 26 runs the whole stack through a failure-mode tour. Claims in this book get cashed.

1.7 Implementation in Rust

Your first build should take seconds and cannot be allowed to fail on a native dependency, so `ch01-hello` has no dependencies at all — no `nalgebra`, no `rand`, nothing. It is the corridor from the last section, in sixty lines of standard library Rust.

RUST demos/ch01-hello/src/main.rs

```

1  /// The whole chapter, minus the prose. `cargo run -p ch01_hello`.
2  ///
3  /// Ten cells, three identical doors, one imperfect sensor. Everything this
4  /// program prints is checked by the test at the bottom of the file, and drawn
5  /// by the widget in the chapter -- because they are the same computation.
6
7  /// A ten-cell cyclic corridor. Cell `i` spans [i, i+1) metres.
8  const N: usize = 10;
9  /// Cells with a door. Rusty cannot tell them apart; that is the whole problem.
10 const DOORS: [usize; 3] = [1, 4, 5];
11 /// p(z = door | the robot is at a door) -- the sensor is right 3 times in 5.
12 const P_HIT: f64 = 0.6;
13 /// p(z = door | the robot is not at a door) -- and it hallucinates 1 time in 5.
14 const P_FALSE: f64 = 0.2;
15
16 /// A belief is a probability per cell. Fixed size, on the stack, no allocation:
17 /// this array *is* the distribution, not a summary of one.
18 type Belief = [f64; N];
19
20 /// Divide by eta so the belief sums to one again, and hand back the number we
21 /// divided by. That number is p(z), the evidence -- Chapter 5 shows it is a
22 /// genuine diagnostic rather than bookkeeping.
23 fn normalize(bel: &mut Belief) -> f64 {
24     let evidence: f64 = bel.iter().sum();
25     for p in bel.iter_mut() {
26         *p /= evidence;
27     }
28     evidence
29 }
30
31 /// Measurement update. Multiply pointwise by the likelihood, then normalize.
32 /// This is the operation that *sharpens* a belief.
33 fn sense_door(bel: &mut Belief) -> f64 {
34     for (i, p) in bel.iter_mut().enumerate() {
35         *p *= if DOORS.contains(&i) { P_HIT } else { P_FALSE };
36     }
37     normalize(bel)
38 }
39
40 /// Motion update. Shift every cell one to the right, wrapping at the end.
41 /// Perfect motion, for exactly one chapter: Chapter 5 replaces this shift with
42 /// a convolution, and the belief starts spreading as well as moving.
43 fn move_right(bel: &Belief) -> Belief {
44     std::array::from_fn(|i| bel[(i + N - 1) % N])
45 }
46
47 fn bar(p: f64) -> String {
48     "\u{2588}".repeat((p * 60.0).round() as usize)
49 }
50
51 fn main() {
52     // Maximum ignorance: every cell equally plausible. Note what this is *not*
53     // -- it is not "the robot is at cell 0 until told otherwise".
54     let mut bel: Belief = [1.0 / N as f64; N];
55
56     let e1 = sense_door(&mut bel); // z_1 = door
57     bel = move_right(&bel); // u_1 = one cell right
58     let e2 = sense_door(&mut bel); // z_2 = door
59 }

```

```

66     println!("evidence p(z1) = {e1:.4}    p(z2 | z1, u1) = {e2:.4}\n");
67     for (i, p) in bel.iter().enumerate() {
68         let door = if D00RS.contains(&i) { '*' } else { ' ' };
69         println!("cell {i}{door} {p:.4} {}", bar(*p));
70     }
71 }

```

1.7.1 A worked example you can check by hand

Run it, and the corridor from the previous section falls out:

```

evidence p(z1) = 0.3200    p(z2 | z1, u1) = 0.3250

cell 0  0.0385 ??
cell 1* 0.1154 ???????
cell 2  0.1154 ???????
cell 3  0.0385 ??
cell 4* 0.1154 ???????
cell 5* 0.3462 ??????????????????????
cell 6  0.1154 ???????
cell 7  0.0385 ??
cell 8  0.0385 ??
cell 9  0.0385 ??

```

Every number is checkable with a pencil. Cell 5 after the second sighting: it held 0.1875 after the move, the likelihood there is 0.6, so unnormalized it is 0.1125; the whole vector sums to 0.325; and $0.1125/0.325 = 9/26 = 0.3462$. Cell 0 held 0.0625, times 0.2 is 0.0125, over 0.325 is $1/26 = 0.0385$. The evidence values 0.32 and 0.325 are the normalizers themselves.

And every number is locked by a test. This is the book's first instance of a convention it never breaks: where the prose states a number, a test asserts it, and if the two ever disagree, the test is right.

RUST demos/ch01-hello/src/main.rs (tests)

```

1  #[cfg(test)]
2  mod tests {
3      use super::*;
4
5      /// The chapter's worked example, to the last decimal.
6      #[test]
7      fn worked_example_ch01() {
8          let mut bel: Belief = [1.0 / N as f64; N];
9
10         let e1 = sense_door(&mut bel);
11         assert!((e1 - 0.32).abs() < 1e-12, "evidence of the first reading");
12         assert!((bel[1] - 0.1875).abs() < 1e-12); // 0.06 / 0.32, at a door
13         assert!((bel[0] - 0.0625).abs() < 1e-12); // 0.02 / 0.32, not at a door
14
15         bel = move_right(&bel);
16         assert!((bel[2] - 0.1875).abs() < 1e-12); // the peak that was at cell 1
17
18         let e2 = sense_door(&mut bel);
19         assert!((e2 - 0.325).abs() < 1e-12);

```

```

20     assert!((bel[5] - 9.0 / 26.0).abs() < 1e-12); // the winner
21     assert!((bel[1] - 3.0 / 26.0).abs() < 1e-12);
22     assert!((bel[9] - 1.0 / 26.0).abs() < 1e-12);
23
24     // A belief is a distribution at every step, forever. If this ever fails,
25     // some update forgot its eta.
26     let total: f64 = bel.iter().sum();
27     assert!((total - 1.0).abs() < 1e-12);
28 }
29
30 // Zero is a promise you cannot take back: no evidence can resurrect a
31 // hypothesis you have already ruled out. Ten more door sightings do not
32 // push any cell to exactly zero.
33 #[test]
34 fn nothing_is_ever_impossible() {
35     let mut bel: Belief = [1.0 / N as f64; N];
36     for _ in 0..10 {
37         sense_door(&mut bel);
38     }
39     assert!(bel.iter().all(|&p| p > 0.0));
40 }
41 }

```

1.7.2 The dead reckoner, honestly

The hook at the top of this chapter is the same idea in two dimensions, and it needs two crates from the book's stack: `nalgebra` for the pose and `rand` for the noise. Note what is *shared* between the two paths — the integrator is called once for the truth and once for the estimate, with the same arithmetic. Only the input differs.

RUST `demos/ch01-doubt/src/bin/dead_reckoner.rs`

```

1 use nalgebra::{Isometry2, Vector2};
2 use rand::rngs::SmallRng;
3 use rand::SeedableRng;
4 use rand_distr::{Distribution, Normal};
5
6 // A pose is not three loose floats -- it is an element of SE(2), and
7 // `Isometry2` already knows that. Chapter 3 makes the claim precise.
8 type Pose = Isometry2<f64>;
9
10 // Noise-free differential-drive integration over Deltat at constant (v, omega):
11 // the exact arc of radius r = v/omega (Thrun et al., eq. 5.9). As omega -> 0 the
12 // radius blows up while the arc flattens, so we take the straight-line limit
13 // rather than trusting the cancellation.
14 fn integrate(x: &Pose, v: f64, omega: f64, dt: f64) -> Pose {
15     let theta = x.rotation.angle();
16     let step = if omega.abs() < 1e-9 {
17         Vector2::new(v * theta.cos() * dt, v * theta.sin() * dt)
18     } else {
19         let r = v / omega;
20         let next = theta + omega * dt;
21         Vector2::new(
22             -r * theta.sin() + r * next.sin(),

```

```

23         r * theta.cos() - r * next.cos(),
24     )
25 };
26 Isometry2::new(x.translation.vector + step, theta + omega * dt)
27 }
28
29 /// `sample_motion_model_velocity` -- Thrun et al., Table 5.3, in miniature.
30 /// Perturb the command, then integrate the *perturbed* command exactly. The
31 /// third noise term gamma is a final rotation the (v, omega) parameterisation cannot
32 /// otherwise produce; Chapter 9 explains why leaving it out cripples a
33 /// particle filter.
34 fn sample_motion(
35     x: &Pose,
36     v: f64,
37     omega: f64,
38     dt: f64,
39     alpha: [f64; 6],
40     rng: &mut SmallRng,
41 ) -> Pose {
42     let (v2, w2) = (v * v, omega * omega);
43     let draw = |var: f64, rng: &mut SmallRng| match Normal::new(0.0, var.sqrt()) {
44         Ok(d) if var > 0.0 => d.sample(rng),
45         _ => 0.0, // a noiseless robot is allowed, if only in a widget
46     };
47
48     let v_hat = v + draw(alpha[0] * v2 + alpha[1] * w2, rng);
49     let w_hat = omega + draw(alpha[2] * v2 + alpha[3] * w2, rng);
50     let gamma = draw(alpha[4] * v2 + alpha[5] * w2, rng);
51
52     let next = integrate(x, v_hat, w_hat, dt);
53     Isometry2::new(next.translation.vector, next.rotation.angle() + gamma * dt)
54 }
55
56 /// The commanded square: four 2.4 m sides, four 90° corners, at Deltat = 0.1 s.
57 fn square_commands() -> Vec<(f64, f64)> {
58     let turn = std::f64::consts::FRAC_PI_2 / 1.5; // 90° in 1.5 s
59     let mut u = Vec::new();
60     for _ in 0..4 {
61         u.extend(std::iter::repeat_n((0.6, 0.0), 40)); // 40 * 0.1 s * 0.6 m/s = 2.4 m
62         u.extend(std::iter::repeat_n((0.0, turn), 15));
63     }
64     u
65 }
66
67 fn drive(seed: u64, alpha: [f64; 6]) -> (Pose, Pose) {
68     // Seeded, always. `rand::rng()` would make this demo an anecdote instead of
69     // an experiment -- the entire book uses SmallRng with an explicit seed.
70     let mut rng = SmallRng::seed_from_u64(seed);
71     let start = Isometry2::new(Vector2::new(8.75, 0.65), 0.0);
72     let (mut truth, mut reckoned) = (start, start);
73
74     for (v, omega) in square_commands() {
75         truth = sample_motion(&truth, v, omega, 0.1, alpha, &mut rng);
76         reckoned = integrate(&reckoned, v, omega, 0.1); // the same maths, minus the
77         noise
78     }
79     (truth, reckoned)
80 }
81
82 fn main() {
83     let alpha = [0.05, 0.05, 0.05, 0.05, 0.01, 0.01];

```



```

83     println!("seed    drift (m)    heading error (deg)");
84     for seed in 0..8 {
85         let (truth, reckoned) = drive(seed, alpha);
86         let drift = (truth.translation.vector - reckoned.translation.vector).norm();
87         let dtheta = (truth.rotation.angle() - reckoned.rotation.angle()).to_degrees();
88         println!("{seed:>4}    {drift:>9.3}    {dtheta:>18.2}");
89     }
90 }

```

There is no expected-output block for this one, and that is the lesson. The drift is different for every seed, in size *and* direction, so the only claims worth testing are the ones that hold across all of them:

RUST demos/ch01-doubt/src/bin/dead_reckoner.rs (tests)

```

1  #[cfg(test)]
2  mod tests {
3      use super::*;
4
5      /// Dead reckoning is never internally inconsistent: the commanded square
6      /// closes to machine precision, every time, on every machine.
7      #[test]
8      fn the_commanded_square_always_closes() {
9          let (_, reckoned) = drive(7, [0.05, 0.05, 0.05, 0.05, 0.01, 0.01]);
10         let start = Vector2::new(8.75, 0.65);
11         assert!((reckoned.translation.vector - start).norm() < 1e-9);
12     }
13
14     /// ...and the robot is never actually there. Not "usually": for all ten seeds,
15     /// with a margin far larger than any floating-point slop.
16     #[test]
17     fn the_robot_never_is() {
18         let alpha = [0.05, 0.05, 0.05, 0.05, 0.01, 0.01];
19         for seed in 0..10 {
20             let (truth, reckoned) = drive(seed, alpha);
21             let drift = (truth.translation.vector - reckoned.translation.vector).norm();
22             assert!(drift > 0.02, "seed {seed} drifted only {drift} m");
23         }
24     }
25
26     /// Turn the noise off and the two computations are the same computation.
27     #[test]
28     fn zero_noise_is_zero_drift() {
29         let (truth, reckoned) = drive(7, [0.0; 6]);
30         assert!((truth.translation.vector - reckoned.translation.vector).norm() < 1e-12);
31     }
32 }

```

1.7.3 Doubt, priced

Finally, the T-junction, because a claim about decisions deserves code as much as a claim about positions does. This is the smallest possible instance of what Chapter 22 does properly.

RUST demos/ch01-doubt/src/decision.rs

```

1  #[derive(Clone, Copy, Debug, PartialEq, Eq)]
2  enum Charger {
3      Left,
4      Right,
5  }
6
7  #[derive(Clone, Copy, Debug, PartialEq, Eq)]
8  enum Action {
9      GoLeft,
10     GoRight,
11     SenseAgain,
12 }
13
14 const ACTIONS: [Action; 3] = [Action::GoLeft, Action::GoRight, Action::SenseAgain];
15
16 /// L(a, x): what it costs to take action `a` when the world is in state `x`.
17 /// Sensing costs one unit and then acts perfectly, so it never pays the ten.
18 fn loss(a: Action, x: Charger) -> f64 {
19     match (a, x) {
20         (Action::GoLeft, Charger::Left) | (Action::GoRight, Charger::Right) => 0.0,
21         (Action::GoLeft, Charger::Right) | (Action::GoRight, Charger::Left) => 10.0,
22         (Action::SenseAgain, _) => 1.0,
23     }
24 }
25
26 /// A belief over a two-element state space. In this book a belief is always a
27 /// distribution over states -- here it just happens to fit in one f64.
28 #[derive(Clone, Copy)]
29 struct Belief {
30     p_left: f64,
31 }
32
33 impl Belief {
34     /// E_bel[L(a, x)] -- the loss averaged over everything that might be true.
35     fn expected_loss(&self, a: Action) -> f64 {
36         self.p_left * loss(a, Charger::Left) + (1.0 - self.p_left) * loss(a, Charger::
37     Right)
38     }
39
40     /// The best guess. Note how much it throws away: after this call, no caller
41     /// can tell 0.55 from 0.999.
42     fn mode(&self) -> Charger {
43         if self.p_left >= 0.5 { Charger::Left } else { Charger::Right }
44     }
45
46     /// Act on the belief: minimise expected loss under the whole distribution.
47     fn bayes_action(b: &Belief) -> Action {
48         *ACTIONS
49             .iter()
50             .min_by(|a, c| b.expected_loss(**a).total_cmp(&b.expected_loss(**c)))
51             .expect("ACTIONS is non-empty")
52     }
53
54     /// Act on the best guess: minimise loss at the mode. Sensing can now only ever
55     /// look like overhead, because at a point estimate there is nothing to learn.
56     fn mode_action(b: &Belief) -> Action {
57         let x = b.mode();
58         *ACTIONS

```

```

59         .iter()
60         .min_by(|a, c| loss(**a, x).total_cmp(&loss(**c, x)))
61         .expect("ACTIONS is non-empty")
62     }
63
64     #[cfg(test)]
65     mod tests {
66         use super::*;
67
68         #[test]
69         fn the_argmax_fallacy_costs_three_and_a_half() {
70             let b = Belief { p_left: 0.55 };
71
72             assert_eq!(mode_action(&b), Action::GoLeft);
73             assert_eq!(bayes_action(&b), Action::SenseAgain);
74
75             assert!((b.expected_loss(Action::GoLeft) - 4.5).abs() < 1e-12);
76             assert!((b.expected_loss(Action::GoRight) - 5.5).abs() < 1e-12);
77             assert!((b.expected_loss(Action::SenseAgain) - 1.0).abs() < 1e-12);
78
79             let regret = b.expected_loss(mode_action(&b)) - b.expected_loss(bayes_action(&b))
80             ;
81             assert!((regret - 3.5).abs() < 1e-12);
82         }
83
84         /// The threshold from the derivation: commit once  $p \geq 1 - c/L = 0.9$ .
85         #[test]
86         fn sense_until_the_belief_is_sharp_enough() {
87             assert_eq!(bayes_action(&Belief { p_left: 0.89 }), Action::SenseAgain);
88             assert_eq!(bayes_action(&Belief { p_left: 0.91 }), Action::GoLeft);
89             // The mode-follower never senses, at any level of certainty at all.
90             for p in [0.51, 0.7, 0.89, 0.99] {
91                 assert_eq!(mode_action(&Belief { p_left: p }), Action::GoLeft);
92             }
93         }
94     }
95 }

```

1.7.4 Running it yourself

Five minutes, and you have the lab the rest of the book is built in.

```

# 1. stable Rust, 1.85 or newer -- the workspace is on edition 2024
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustc --version

# 2. clone the book workspace and enter it
cd prob-robotics-rust

# 3. the two commands this chapter is really about
cargo run -p ch01_hello      # prints the histogram above
cargo test -p ch01_hello    # locks every number in it

```

If cargo test passes, your toolchain agrees with this book to twelve decimal places, and you are ready for Chapter 2. If you want to hack on the widgets rather than only read them, Chapter 4 adds the WASM target and the simulator crate; nothing before then needs it.

1.8 How to read this book

INTERACTIVE · w1.4

How to read this book

Every chapter has the same shape and the same color contract, so the effort you spend learning to read one chapter is spent once.

Run this simulation in the web edition: `/chapters/ch01-robot-that-doubts #w1.4`

Three promises are worth stating explicitly, because the rest of the book is built on them.

The code in the text is the code that runs. The Rust listings in this chapter are the canonical implementation. The widget above them runs a TypeScript port of the same algorithms, checked against the same worked examples — the hallway widget calls the library's histogram filter, and the value it prints for cell 5 is 9/26 because the filter computed it, not because a designer typed it in. When a chapter shows you a number, you can reproduce it in three places: the page, the test suite, and your own terminal.

Two worlds, one robot. The **Hallway** is the 1-D corridor you just localized in: beliefs are curves there, and everything is plottable. The **Apartment** is the 12 m by 9 m floorplan from the five-uncertainties figure, with ray-cast LiDAR and simulated wheel encoders — the realistic world where localization, mapping, SLAM, and planning happen. Rusty drives in both, and the same seeded random number generator means the run you see is the run everyone sees.

Nothing is asserted that is not eventually cashed. Every "this is more robust" in this chapter becomes an experiment later, usually one where something breaks on purpose. The most useful pages in this book are the ones where a method fails and you can see exactly which assumption it violated.

Chapter 2 gives the probability language a proper foundation, Chapter 3 the geometry, Chapter 4 the robot and simulator, and Chapter 5 turns `sense_and_move` into the Bayes filter — the recursion that every remaining chapter is a special case of.

1.9 Exercises

1 FOUNDATION • Where the threshold moves

Redo the T-junction derivation with $p(\text{left}) = 0.9$ and the same losses. Which action does the belief-follower choose, and by how much does it beat the mode-follower now? Then derive the general threshold p^* as a function of the sensing cost c and the commitment loss L , and say in one sentence what happens to p^* as sensing gets

cheaper. Finally: what does the mode-follower do at $p = 0.51$, and why is that the same thing it does at $p = 0.99$?

2 FOUNDATION •• A corridor that can never be solved

Suppose the corridor has nine cells with doors at 1, 4 and 7 — evenly spaced — and motion is the perfect one-cell shift used in this chapter. Show that if the belief starts uniform, no sequence of sense and move operations ever produces a belief with a unique maximum. What property of the door layout does the argument turn on, and what would you have to change about the *robot* (not the corridor) to break the symmetry?

Hint. Show that both operations preserve the property of being unchanged when the belief is rotated by three cells, and that the uniform prior has it.

3 FOUNDATION •• The evidence is data too

The program prints $p(z_1) = 0.32$ and $p(z_2 | z_1, u_1) = 0.325$. Compute by hand what $p(z_2 | z_1, u_1)$ would have been if the second reading had been *no door* instead of *door*, and explain what a very small evidence value would have told the robot. (You have just invented the outlier test of Chapter 11 and the kidnapping detector of Chapter 12.)

4 CONCEPTUAL • Predict, then press play

Scrub w1.1 to the row "after move right" and pause there. Before you step: which cells will grow when the second door sighting lands, which will shrink, and what is the ratio between the factor a growing cell gets and the factor a shrinking one gets? Step, and check your answer against the table. Then run the alternative sequence sense, move, move, sense on paper — what happens to the belief, and what does that tell you about how much of the localization was done by the *sensor* versus by the *sequence*?

Hint. A cell grows exactly when its likelihood exceeds the normalizer. For the second part: is any cell both a door and two cells to the right of a door?

5 CONCEPTUAL • Is drift predictable?

In w1.2, set actuation noise to 1 and re-roll the seed five times, noting which way the true path ends up relative to the commanded square. Is the direction of the final drift predictable? Write down why not in one sentence that uses the word "distribution". Then set the noise slider to 0 and say what would have to be true of the wheels, the floor and the map for that to be an honest model of any robot at all.

6 PRACTICAL •• Your first convolution

Extend ch01-hello with a sloppier motion model: `move_right_sloppy` moves two cells with probability 0.1, one cell with probability 0.8, and none with probability

0.1. Re-run the sense–move–sense sequence with it. Does cell 5 still win, and by how much? Keep the code — Chapter 5 will show you that what you just wrote is a discrete convolution, and Chapter 8 that it is the prediction step of the histogram filter.

Hint. The new belief at a cell collects mass from the two cells to its left and from the cell itself, weighted 0.1, 0.8 and 0.1. Normalizing afterwards should be unnecessary — check that it is, and you have proved the motion update conserves probability.

7 PRACTICAL •• Break the test on purpose

Change `P_HIT` to 0.9 and predict, before running anything, whether cell 5's posterior goes up or down and roughly how far. Run `cargo test -p ch01_hello` and read the failure message: it names the value it got. Then work out the new posterior analytically, confirm the two agree, and rewrite the test to assert it. (Both the failure and the reason it failed are the exercise; a test suite that never fails is not telling you anything.)

8 PRACTICAL ••• How bad does it get?

Modify the dead reckoner to report the drift after ten laps rather than one, averaged over fifty seeds. Plot mean drift against lap number. Does it grow like $\sqrt{n}$, like n , or faster — and which of the three noise terms ($\hat{v}$, $\hat{\omega}$, $\hat{\gamma}$) dominates when you zero the others out? Write down what this implies about how often a real robot must see something it recognizes.

1.10 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press, ISBN 9780262201629.

The book this one modernizes. Chapter 1 is the source of the five-source uncertainty taxonomy and of the epigraph above, which appears there as the authors' central conjecture; the corridor example is its Figure 1.1. [link](#)

Thrun, S. (2002). Probabilistic robotics *Communications of the ACM* 45(3), 52–57.

The argument of this chapter in six pages, written for skeptics, three years before the book. Useful if you want the case for the paradigm without the machinery. doi:10.1145/504729.504754

Kaelbling, L. P., Littman, M. L., and Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains *Artificial Intelligence* 101(1–2), 99–134.

Where the T-junction example grows up. The formal treatment of acting on a belief rather than on a state, and the origin of the tiger problem that Chapter 22 plays. doi:10.1016/S0004-3702(98)00023-X

Kochenderfer, M. J., Wheeler, T. A., and Wray, K. H. (2022). *Algorithms for Decision Making* MIT Press, ISBN 9780262047012.

The modern companion volume on the acting side of the loop, freely readable online. Its treatment of the value of information is the general form of this chapter’s threshold $p^* = 1 - c/L$. [link](#)

Barfoot, T. D. (2024). *State Estimation for Robotics* Cambridge University Press, 2nd edition, ISBN 9781009299893.

The reference treatment of the estimation half of this book, and the one that generalizes cleanest to Lie groups — which is where Chapters 3 and 7 take it. [link](#)

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., and Ferguson, M. (2023). From the desks of ROS maintainers: A survey of modern and capable mobile robotics algorithms in the Robot Operating System 2 *Robotics and Autonomous Systems* 168, 104493.

Evidence that this is not a historical exercise: the localizer shipping in today’s most widely deployed navigation stack is adaptive Monte Carlo localization, which is the particle filter of Chapter 8 applied in Chapter 12. doi:10.1016/j.robot.2023.104493

Placed, J. A., Strader, J., Carrillo, H., Atanasov, N., Indelman, V., Carlone, L., and Castellanos, J. A. (2023). A survey on active simultaneous localization and mapping: State of the art and new frontiers *IEEE Transactions on Robotics* 39(3), 1686–1705.

What the T-junction becomes at full scale: choosing where to drive in order to learn. The map of the field that Chapter 24 follows. doi:10.1109/TR0.2023.3248510

Lindemann, L., Cleaveland, M., Shim, G., and Pappas, G. J. (2023). Safe planning in dynamic environments using conformal prediction *IEEE Robotics and Automation Letters* 8(8), 5116–5123.

A current answer to the question this chapter opens — how to act when your uncertainty estimate is itself learned and possibly miscalibrated. Read it alongside Chapter 25. doi:10.1109/LRA.2023.3292071

Probability: The Language of Uncertainty

FOUNDATIONS · Foundational · 55 min



In probabilistic robotics, quantities such as sensor measurements, controls, and the states a robot and its environment might assume are all modeled as random variables.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 2

Every remaining chapter of this book is Bayes rule applied cleverly to robots. That makes this chapter's toolkit small — random variables, conditioning, expectation, the Gaussian, entropy — but it has to be *owned* rather than reviewed, because Chapters 5 through 26 will lean on it without apology.

Two ideas do most of the work. The first is that **Bayes rule is not a formula, it is pointwise multiplication**: take the curve you believed, multiply it by the curve the sensor implies, divide by whatever it takes to make the result integrate to one. The second is that **a Gaussian is not a formula either, it is a shape** — a blob with a location, a size, and a tilt, and every operation this book performs on it is something you can watch happen to the blob.

Both claims come with an executable receipt. The numbers printed here are the numbers the widgets show, and they are the numbers a Rust unit test pins to twelve decimal places. That convention starts in this chapter and never stops.

◎ AFTER THIS CHAPTER YOU CAN

- Derive Bayes rule from the definition of conditioning, and say exactly what the normalizer η is
- State what conditional independence assumes — and what it does not

follow from

- Read a covariance matrix as a shape: eigenvectors, iso-density ellipses, and the χ^2 number that sizes them
- Multiply two Gaussians in moments form the hard way and in canonical form the easy way
- Tell marginalizing apart from conditioning, and predict which one narrows a distribution
- Implement Gaussian<N> and Canonical<N> in Rust, factoring once and never inverting a covariance to evaluate a density

ASSUMED

- Chapter 1: the hallway, the door detector, and why a single best guess is not enough
- Linear algebra: matrix multiplication, transpose, eigenvectors of a symmetric matrix
- Calculus: integration by substitution, and enough comfort to skip a page of algebra

2.1 The problem: two numbers in, one number out

In Chapter 1 we put Rusty in a ten-cell corridor with doors at cells 1, 4 and 5, gave it a door detector that fires correctly 60% of the time at a door and 20% of the time at a blank wall, and told it nothing about where it started. The detector fired.

The reader was handed two numbers — a prior of 0.1 per cell, a likelihood of 0.6 or 0.2 — and one answer: 0.1875 at each door, 0.0625 everywhere else. What was never justified was the recipe. Why multiply rather than average? Why is the answer not 0.6? And what exactly is the division at the end doing?

Here is the whole computation, in one line per step. Multiply the prior by the likelihood, cell by cell:

$$0.1 \cdot 0.6 = 0.06 \quad (\text{three doors}) \qquad 0.1 \cdot 0.2 = 0.02 \quad (\text{seven walls})$$

Those ten numbers sum to $3(0.06) + 7(0.02) = 0.32$, not to one, so they are not yet a distribution — they are the right *shape* at the wrong scale. Divide by the total:

$$0.06/0.32 = 0.1875 \qquad 0.02/0.32 = 0.0625$$

That is Bayes rule, and the 0.32 is the only part that needs a name. It is the probability of seeing what we saw, averaged over everywhere we might have

been, and it is a constant with respect to the cell index — which is why the whole normalization can be deferred to the end and written as a single symbol $\eta = 1/0.32$. Thrun, Burgard and Fox use η this way throughout, and so does this book: *compute the shape, normalize later*.

Now make the corridor continuous. Rusty's odometry says it has travelled about 5 m, give or take 2 m. Its wall-range sensor says 6.5 m, and it is a better sensor. Multiplying ten numbers by ten numbers becomes multiplying one curve by another, and the widget below does exactly that, live.

INTERACTIVE · w2.2

Blob Multiplier

Bayes fusion does not average two estimates — it precision-weights them, and the answer is always more certain than either input.

Run this simulation in the web edition: [/chapters/ch02-probability #w2.2](/chapters/ch02-probability#w2.2)

Three things in that figure are worth more than the algebra that justifies them later.

The posterior is narrower than both inputs, always. Not narrower than the average of the two — narrower than the *better* of the two. Fusing an excellent estimate with a mediocre one still improves the excellent one, because two independent opinions genuinely carry more information than one.

The posterior mean is not a midpoint. It sits between the two means, but pulled toward whichever curve is sharper. When they are equally sharp it lands exactly halfway; when the sensor is ten times sharper, it lands almost on the sensor. This is *precision weighting*, and the weight is one over the variance.

In canonical form the arithmetic is addition. The ledger under the plot tracks two numbers, $\Omega = 1/\sigma^2$ and $\xi = \mu/\sigma^2$, and the fusion is nothing but $\Omega_1 + \Omega_2$ and $\xi_1 + \xi_2$. That observation looks like a curiosity here. In Chapter 6 it becomes the information filter, and in Chapter 15 it becomes the reason modern SLAM back-ends never store a covariance matrix at all.

THE ONE SENTENCE VERSION

Bayes rule multiplies the distribution you had by the distribution the data implies, and divides by whatever makes the result integrate to one. Everything else in this chapter is about choosing representations that make that multiplication cheap.

2.2 Building intuition: a Gaussian is a shape you can drag

One distribution dominates this book, and it earns that place for three reasons that are all theorems: the product of two Gaussians is Gaussian, the marginal of a Gaussian is Gaussian, and a linear map of a Gaussian is Gaussian. A belief that starts Gaussian and only ever meets those three operations stays Gaussian forever, described by a mean vector and a covariance matrix and nothing else. That is the entire premise of the Kalman filter.

So the covariance matrix deserves to be understood as geometry before it is understood as algebra. Play with it first.

INTERACTIVE · w2.1

Gaussian Playground

The ellipse's axes are the eigenvectors of Σ , not the coordinate axes — so its half-widths are not the marginal standard deviations.

Run this simulation in the web edition: </chapters/ch02-probability#w2.1>

The figure shows the same distribution three ways at once, and each way answers a different question. The **cloud** answers *what would I see if I drew from it* — and notice that the 500 dots are never re-sampled: they are one fixed set of standard-normal numbers being pushed through a changing matrix, which is all that $x = \mu + Lz$ means. The **ellipses** answer *where does the density live*, at one, two, and 2.4477 standard deviations. The **gray axes** answer *which directions are special*: they are the eigenvectors of Σ , and along them the two coordinates stop interfering with each other.

The misconception this kills is a stubborn one. It is tempting to read the ellipse's half-widths as the standard deviations of x_a and x_b . They are not, and the blue marginal curves painted along the two edges prove it: as ρ sweeps and the ellipse swings through 45 degrees, those curves do not move by a pixel. Correlation changes how the two coordinates covary; it does not change how uncertain either one is on its own. What it *does* change is how much one of them can teach you about the other — and that is a different question, taken up in Derivation 6.

2.3 The mathematics

2.3.1 Notation

Every symbol below is Thrun-compatible; the manifold operators wait for Chapter 3.

X, x	Random variable and a value it may take. $p(x)$ is a pmf if X is discrete, a pdf if continuous.
$p(x, y), p(x y)$	Joint distribution; conditional distribution of x given that Y took the value y .
η	Generic normalizer. Whatever constant makes the expression to its right integrate to one.
$\mathbb{E}[X], \text{Var}[X]$	Expectation and variance.
μ, Σ	Mean vector and covariance matrix — the moments parameterization.
$\mathcal{N}(x; \mu, \Sigma)$	Gaussian density in x with that mean and covariance.
$\Omega = \Sigma^{-1}, \xi = \Sigma^{-1}\mu$	Information matrix and information vector — the canonical parameterization.
$d_M^2(x) = (x - \mu)^T \Sigma^{-1} (x - \mu)$	Squared Mahalanobis distance: distance measured in standard deviations, not metres.
$H(X)$	Entropy. Differential entropy when X is continuous, in nats unless bits are stated.

2.3.2 Random variables, joints, and conditionals

A **random variable** X takes values according to a distribution. If the value space is discrete we write $p(X = x)$, abbreviated $p(x)$, with $\sum_x p(x) = 1$ and $p(x) \geq 0$. If it is continuous we assume a **probability density** $p(x)$ with $\int p(x) dx = 1$. A density is not a probability: it can exceed one. Only its integral over a region is a probability, and this book will use "probability", "density", and "distribution" interchangeably wherever no confusion can result.

The **joint** $p(x, y)$ is the density of both events together. The **conditional** is defined, whenever $p(y) > 0$, as

$$p(x | y) = \frac{p(x, y)}{p(y)}$$

which is worth reading as a statement about renormalization: slice the joint at $Y = y$, and scale the slice so it integrates to one again. That is the definition the entire chapter hangs on.

X and Y are **independent** when $p(x, y) = p(x)p(y)$, equivalently $p(x | y) = p(x)$: knowing Y tells you nothing about X . They are **conditionally independent given** Z when

$$p(x, y | z) = p(x | z)p(y | z)$$

⚠ WHERE ROBOTS BREAK THIS

Conditional independence neither implies nor is implied by independence.

Two LiDAR beams from the same pose are wildly dependent unconditionally — both are long in a corridor and short in a closet — yet the beam model of Chapter 10 treats them as independent *given the pose and the map*. That assumption is what makes a 360-beam likelihood a product of 360 terms instead of an intractable joint, and it is also why a scan match can be catastrophically overconfident: the beams share a wall, so conditioning on the pose does not actually make them independent. Chapter 10 measures the damage; here it is enough to notice that the assumption is a choice, not a fact.

Finally, the **theorem of total probability** reconstructs a marginal from a conditional by integrating out what you introduced:

$$p(x) = \sum_y p(x | y) p(y) \qquad p(x) = \int p(x | y) p(y) dy$$

2.3.3 Bayes rule

DERIVATION Bayes rule from the definition of conditioning

$$p(x | z) = \eta p(z | x) p(x)$$

Step 1 — the joint is symmetric. By the definition of conditional probability applied both ways,

$$p(x, z) = p(x | z) p(z) = p(z | x) p(x)$$

Step 2 — divide. For $p(z) > 0$,

$$p(x | z) = \frac{p(z | x) p(x)}{p(z)}$$

Step 3 — notice the denominator does not depend on x . Expanding it by total probability, $p(z) = \int p(z | x') p(x') dx'$, which has integrated x away. It is therefore the same number for every x in the posterior, and we may name it $1/\eta$ and defer it:

$$p(x | z) = \eta p(z | x) p(x)$$

The version with background knowledge. Every rule above may be conditioned on further variables without changing its form. Conditioning on y throughout gives

$$p(x | z, y) = \eta p(z | x, y) p(x | y)$$

which is the form the Bayes filter of Chapter 5 actually uses, with y standing for the entire history of controls and earlier measurements.

The hallway, in this notation. With x ranging over ten cells, $\text{bel}(x) = p(x) = 0.1$, and $p(z = \text{door} \mid x)$ equal to 0.6 at cells $\{1, 4, 5\}$ and 0.2 elsewhere:

$$\eta^{-1} = \sum_x p(z \mid x) p(x) = 3(0.6)(0.1) + 7(0.2)(0.1) = 0.32$$

$$p(x \mid z) = \frac{(0.6)(0.1)}{0.32} = 0.1875 \quad \text{at a door,} \quad \frac{(0.2)(0.1)}{0.32} = 0.0625 \quad \text{elsewhere.} \quad \blacksquare$$

The quantity $\eta^{-1} = p(z)$ has a name worth remembering: the **evidence**. The filter treats it as housekeeping, but it is a live diagnostic: a surprisingly small value means the measurement was unlikely under everything you currently believe. Chapter 11 rejects outliers with it and Chapter 12 uses it to notice that a robot has been kidnapped.

2.3.4 Expectation and covariance

The **expectation** is the probability-weighted average, and it is linear:

$$\mathbb{E}[X] = \int x p(x) dx \quad \mathbb{E}[AX + b] = A \mathbb{E}[X] + b$$

Linearity holds whether or not the components of X are independent, which is why it will survive every approximation in this book. The **covariance** is the expected outer product of the deviation from the mean:

$$\Sigma = \text{Cov}[X] = \mathbb{E}[(X - \mathbb{E}[X])(X - \mathbb{E}[X])^T]$$

Three properties follow immediately and get used constantly. Σ is symmetric, since the outer product is. It is positive semi-definite, since for any vector a we have $a^T \Sigma a = \mathbb{E}[(a^T(X - \mu))^2] \geq 0$ — a variance, and variances cannot be negative. And its diagonal entries are the variances of the individual coordinates while its off-diagonal entries measure how they move together. A covariance with a negative eigenvalue is not an unlucky covariance; it is a bug, and Chapter 6 spends real effort making sure filters never produce one.

2.3.5 The Gaussian

In one dimension,

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(x - \mu)^2}{\sigma^2}\right)$$

and in n dimensions, with $\mu \in \mathbb{R}^n$ and Σ symmetric positive-definite,

$$\mathcal{N}(x; \mu, \Sigma) = \det(2\pi\Sigma)^{-1/2} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right)$$

The second is a strict generalization of the first. Everything interesting lives in the exponent's quadratic form: the prefactor exists only to make the integral one, which is precisely why η can absorb it and why so much of what follows is bookkeeping about quadratics.

It is worth being explicit about the working definition this book uses: **a Gaussian is a quadratic in the exponent**. Multiplying densities adds quadratics; conditioning fixes some variables in a quadratic; a linear map substitutes into a quadratic. All three stay quadratic, so all three stay Gaussian, and the next five derivations are just careful applications of that one fact.

2.3.6 The product of two Gaussians

 **DERIVATION** Two 1-D Gaussians multiply to an unnormalized Gaussian

$$\sigma^2 = (\sigma_1^{-2} + \sigma_2^{-2})^{-1}$$

Statement. $\mathcal{N}(x; \mu_1, \sigma_1^2) \cdot \mathcal{N}(x; \mu_2, \sigma_2^2) \propto \mathcal{N}(x; \mu, \sigma^2)$ with

$$\sigma^2 = \left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2} \right)^{-1} \quad \mu = \sigma^2 \left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2} \right)$$

Step 1 — exponents add. Discarding both prefactors into a constant c ,

$$c \exp\left(-\frac{1}{2} \left[\frac{(x - \mu_1)^2}{\sigma_1^2} + \frac{(x - \mu_2)^2}{\sigma_2^2} \right]\right)$$

Step 2 — collect the quadratic in x . Expanding both squares and gathering powers of x ,

$$\frac{(x - \mu_1)^2}{\sigma_1^2} + \frac{(x - \mu_2)^2}{\sigma_2^2} = \underbrace{\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2} \right)}_{\Omega} x^2 - 2 \underbrace{\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2} \right)}_{\xi} x + \left(\frac{\mu_1^2}{\sigma_1^2} + \frac{\mu_2^2}{\sigma_2^2} \right)$$

The two coefficients that carry all the x -dependence are exactly Ω and ξ . This is not a coincidence and it is not notation smuggled in from later — it is the definition of canonical form, falling out of the algebra on its own.

Step 3 — complete the square. The book's first use of the manoeuvre that will eventually produce the Kalman gain:

$$\Omega x^2 - 2\xi x = \Omega \left(x - \frac{\xi}{\Omega} \right)^2 - \frac{\xi^2}{\Omega}$$

The trailing $-\xi^2/\Omega$ has no x in it, so it joins the constant. What is left is $\exp(-\frac{1}{2}\Omega(x - \xi/\Omega)^2)$: a Gaussian with variance $1/\Omega$ and mean ξ/Ω .

Step 4 — read off the moments.

$$\sigma^2 = \frac{1}{\Omega} = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2} \quad \mu = \frac{\xi}{\Omega} = \frac{\sigma_2^2 \mu_1 + \sigma_1^2 \mu_2}{\sigma_1^2 + \sigma_2^2}$$

Both forms are useful: the first says precisions add, the second says the mean is a weighted average in which each estimate is weighted by *the other's* variance. And since $\sigma^2 < \min(\sigma_1^2, \sigma_2^2)$ whenever both are finite, fusion strictly increases certainty — the claim the widget demonstrates and cannot be made to violate.

What the constant was. Collecting every discarded factor and comparing with the normalized answer gives

$$\int \mathcal{N}(x; \mu_1, \sigma_1^2) \mathcal{N}(x; \mu_2, \sigma_2^2) dx = \mathcal{N}(\mu_1; \mu_2, \sigma_1^2 + \sigma_2^2)$$

so η was never hiding anything mysterious: it is the evidence, and it is itself a Gaussian — evaluated at the disagreement between the two means, with the two variances added. A large disagreement relative to $\sqrt{\sigma_1^2 + \sigma_2^2}$ means small evidence, which is exactly the gating test Chapter 11 applies before accepting a data association. ■

2.3.7 Canonical form, where multiplication is addition

DERIVATION Canonical parameters add under multiplication

$$\Omega = \Omega_1 + \Omega_2, \quad \xi = \xi_1 + \xi_2$$

Statement. Write a Gaussian as $p_i(x) \propto \exp(-\frac{1}{2}x^\top \Omega_i x + x^\top \xi_i)$, with $\Omega_i =$

Σ_i^{-1} and $\xi_i = \Sigma_i^{-1}\mu_i$. Then p_1p_2 has parameters $\Omega_1 + \Omega_2$ and $\xi_1 + \xi_2$.

Step 1 — exponents add.

$$p_1(x)p_2(x) \propto \exp\left(-\frac{1}{2}x^T\Omega_1x + x^T\xi_1 - \frac{1}{2}x^T\Omega_2x + x^T\xi_2\right)$$

Step 2 — gather like terms. The quadratic coefficients add and the linear coefficients add:

$$= \exp\left(-\frac{1}{2}x^T(\Omega_1 + \Omega_2)x + x^T(\xi_1 + \xi_2)\right)$$

which is canonical form again, with the stated parameters. There is no third step. ■

Why this matters more than it looks. The two Bayes-filter operations have opposite costs in the two parameterizations:

Operation	Moments (μ, Σ)	Canonical (ξ, Ω)
Multiply two Gaussians — <i>correct</i>	$O(n^3)$: inverses and a product	$O(n^2)$: two additions
Marginalize a block out — <i>predict</i>	$O(n^2)$: copy Σ_{aa}	$O(n^3)$: a Schur complement
Condition on a measured block	$O(n^3)$: a Schur complement	$O(n^2)$: copy Ω_{aa}

Every row is a mirror. That table is the reason Chapter 6 presents two filters instead of one, the reason the information filter is preferred in multi-sensor fusion where measurements outnumber predictions, and the reason Chapter 15 formulates SLAM entirely in Ω : with a thousand poses, Ω is sparse and Σ is dense. The bottom row is Derivation 6, which has not happened yet — come back to this table after it has.

ALGORITHM `gaussian_product_canonical`($\xi_1, \Omega_1, \xi_2, \Omega_2$) COST $O(n^2)$ -- two additions. Converting back to moments costs one $O(n^3)$ solve.

in two Gaussians in canonical form

out their normalized product, in canonical form

- 1: $\Omega = \Omega_1 + \Omega_2$
- 2: $\xi = \xi_1 + \xi_2$
- 3: **return** (ξ, Ω)

2.3.8 Linear transformations

DERIVATION A linear map of a Gaussian is a Gaussian

$$Y = AX + b \Rightarrow Y \sim \mathcal{N}(A\mu + b, A\Sigma A^\top)$$

Statement. If $X \sim \mathcal{N}(\mu, \Sigma)$ and $Y = AX + b$ for a constant matrix A and vector b , then $Y \sim \mathcal{N}(A\mu + b, A\Sigma A^\top)$.

Step 1 — the mean, by linearity of expectation. $\mathbb{E}[Y] = A \mathbb{E}[X] + b = A\mu + b$.

Step 2 — the covariance, from its definition.

$$\text{Cov}[Y] = \mathbb{E}[(Y - A\mu - b)(Y - A\mu - b)^\top] = \mathbb{E}[A(X - \mu)(X - \mu)^\top A^\top]$$

Step 3 — pull the constants out. A and $A^\top$ do not depend on X , so

$$\text{Cov}[Y] = A \mathbb{E}[(X - \mu)(X - \mu)^\top] A^\top = A\Sigma A^\top$$

Note that Steps 1–3 never used Gaussianity: *any* distribution transforms its mean and covariance this way. What Gaussianity adds is that the mean and covariance are the whole story.

The density-level version, for invertible A . Change of variables gives $p_Y(y) = p_X(A^{-1}(y - b))|\det A|^{-1}$. Substituting the Gaussian density and using $(A^{-1})^\top \Sigma^{-1} A^{-1} = (A\Sigma A^\top)^{-1}$ together with $\det(2\pi\Sigma)(\det A)^2 = \det(2\pi A\Sigma A^\top)$ reproduces $\mathcal{N}(y; A\mu + b, A\Sigma A^\top)$ exactly. ■

Read backwards, this is a sampler. Factor $\Sigma = LL^\top$ by Cholesky, draw $z \sim \mathcal{N}(0, I)$, and set $x = \mu + Lz$. Then x has mean μ and covariance $LL^\top = \Sigma$. One $O(n^3)$ factorization, then $O(n^2)$ per sample forever after — and it is literally what the Gaussian Playground redraws every frame.

Read together with Derivation 2, this is the Kalman filter waiting to happen. A linear motion model pushes a Gaussian forward (Derivation 4); a linear measurement multiplies it by another Gaussian (Derivation 2). Chapter 6 does little more than compose the two and give the result a subscript.

2.3.9 Iso-density contours are ellipses

DERIVATION Contours of constant density are ellipses on the eigenvectors of Σ

semi-axes $\sqrt{c/\lambda_i}$, and $c = 5.991$ for 95% in 2-D

Statement. The set $\{x : d_M^2(x) = c\}$, where $d_M^2(x) = (x - \mu)^\top \Sigma^{-1}(x - \mu)$, is

an ellipsoid centred at μ whose axes point along the eigenvectors of Σ and whose semi-axis lengths are $\sqrt{c \lambda_i}$.

Step 1 — the density depends on x only through d_M^2 . Everything else in $\mathcal{N}(x; \mu, \Sigma)$ is constant, so a contour of constant density is a contour of constant d_M^2 .

Step 2 — diagonalize. Σ is symmetric positive-definite, so $\Sigma = V \Lambda V^T$ with V orthogonal and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$, $\lambda_i > 0$. Then $\Sigma^{-1} = V \Lambda^{-1} V^T$.

Step 3 — rotate coordinates. Put $u = V^T(x - \mu)$, a rigid rotation into the eigenbasis. The quadratic form decouples completely:

$$d_M^2 = u^T \Lambda^{-1} u = \sum_i \frac{u_i^2}{\lambda_i}$$

Step 4 — read off the axes. $\sum_i u_i^2 / \lambda_i = c$ is the standard equation of an ellipsoid with semi-axes $\sqrt{c \lambda_i}$ along the coordinate directions of u — that is, along the eigenvectors of Σ .

Which c ? Since $u_i / \sqrt{\lambda_i}$ are independent standard normals, d_M^2 is a sum of n squared standard normals: it is χ_n^2 distributed. In two dimensions the χ_2^2 CDF is elementary, $P(d_M^2 \leq c) = 1 - e^{-c/2}$, so

$$c = -2 \ln(1 - p) \quad \Rightarrow \quad c_{95\%} = -2 \ln 0.05 = 5.9915, \quad \sqrt{c} = 2.4477$$

The 95% ellipse is therefore the **2.4477 σ** ellipse, not the 2σ one — which covers only $1 - e^{-2} = 86.5\%$. Every confidence ellipse in this book is drawn at $\sqrt{-2 \ln(1 - p)}$, by the same `chi2Quantile2` the widgets call, so the figure and the formula are one computation. ■

The Mahalanobis distance introduced there is worth naming on its own, because it is the only sensible way to answer "is this measurement consistent with what I expected?". Euclidean distance cannot answer it: 30 cm of error is nothing along an axis where the filter is uncertain and catastrophic along one where it is not. Mahalanobis distance measures in standard deviations, and its square is the χ^2 statistic you compare against a table.

ALGORITHM `mahalanobis2( $\mu$ ,  $\Sigma$ ,  $x$ )` COST $O(n^2)$ given a cached Cholesky factor; $O(n^3)$ if you have to compute it
in a Gaussian and a query point
out $d^2(x) = (x - \mu)^T \Sigma^{-1} (x - \mu)$

- 1: $L = \text{cholesky}(\Sigma)$ (cache this — it is the only cubic step)
- 2: $y = L^{-1}(x - \mu)$ (forward substitution, never an explicit inverse)
- 3: return $y^\top y$

NOTE

Numerical hygiene rule, obeyed book-wide. Never form Σ^{-1} to evaluate a quadratic form or a density. Factor once, solve against the factor. It is faster, it is more accurate, and it fails loudly — Cholesky refuses a non-positive-definite matrix — where an inverse would quietly return garbage and let a filter run for another thousand steps before producing a negative variance.

2.3.10 Marginals and conditionals

Partition a jointly Gaussian vector as $x = (x_a, x_b)$, with

$$\mu = \begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix} \quad \Sigma = \begin{bmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{bmatrix}$$

There are two entirely different ways to get down to x_a alone, and confusing them is the single most common error in a first estimation course.

DERIVATION Marginals copy a block; conditionals take a Schur complement

$$\Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$$

Statement. The marginal is

$$p(x_a) = \mathcal{N}(x_a; \mu_a, \Sigma_{aa})$$

and the conditional, for an observed value $x_b = \beta$, is

$$p(x_a | x_b = \beta) = \mathcal{N}(x_a; \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\beta - \mu_b), \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba})$$

Sketch — the marginal. Integrating x_b out of the joint density means completing the square in x_b and integrating that Gaussian to one; what survives is a Gaussian in x_a whose parameters are the corresponding blocks of μ and Σ . Nothing has to be computed. In moments form, marginalization is a memory copy.

Sketch — the conditional. Hold $x_b = \beta$ fixed and treat the joint exponent as a quadratic in x_a alone. Write the information matrix in blocks, $\Omega =$

$\Sigma^{-1} = \begin{bmatrix} \Omega_{aa} & \Omega_{ab} \\ \Omega_{ba} & \Omega_{bb} \end{bmatrix}$, whose upper-left block is known to be $\Omega_{aa} = (\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba})^{-1}$ with $\Omega_{ab} = -\Omega_{aa}\Sigma_{ab}\Sigma_{bb}^{-1}$. The quadratic coefficient in x_a is Ω_{aa} , so the conditional covariance is Ω_{aa}^{-1} — the Schur complement — and the linear coefficient $\xi_a - \Omega_{ab}\beta$ produces the stated mean. The full algebra is Appendix B; the pattern to remember is that the *quadratic* coefficient of the exponent is always the inverse covariance.

And now the duality closes. Read that last paragraph again in canonical coordinates and the conditional is not a computation at all:

$$\Omega_{a|b} = \Omega_{aa} \quad \xi_{a|b} = \xi_a - \Omega_{ab}\beta$$

Conditioning is a block copy in *canonical* form and a Schur complement in moments form; marginalizing is a block copy in *moments* form and a Schur complement in canonical form. The two operations are exact mirror images, and every estimator in this book is, at bottom, a choice about which of them you are willing to pay for.

The two facts worth memorizing. Read the two parameters again and notice what is missing from each. The conditional **mean** depends on β ; the conditional **covariance** does not. Learning *that* a correlated quantity was measured is what buys certainty; learning *what* it read only moves the estimate. And in 2-D the shrinkage has a one-line form:

$$\sigma_{a|b}^2 = \sigma_a^2 (1 - \rho^2) = \frac{\det \Sigma}{\sigma_b^2}$$

so at $\rho = 0.95$ the conditional standard deviation is $\sqrt{1 - 0.9025} = 0.312$ times the marginal one — a 69% reduction from a single scalar observation, purchased entirely by correlation. ■

Where you will meet this matrix again. The factor $\Sigma_{ab}\Sigma_{bb}^{-1}$ is the Kalman gain of Chapter 6 with the innovation covariance already in place of Σ_{bb} . The Schur complement is what appears when Chapter 15 marginalizes an old pose out of a factor graph and discovers the fill-in it leaves behind, and it is why Chapter 18 treats marginalization as a thing to be budgeted rather than done freely.

INTERACTIVE · w2.4

Slice vs. Squash

Marginalizing and conditioning are different operations: one throws a variable away, the other spends it.

Run this simulation in the web edition: /chapters/ch02-probability #w2.4

2.3.11 Entropy

Uncertainty deserves a scalar, if only so that a robot can be asked *which action would teach me the most*. Shannon entropy is that scalar:

$$H(X) = \mathbb{E}[-\log_2 p(x)] = - \sum_x p(x) \log_2 p(x)$$

in bits, or with $\ln$ for nats. A uniform belief over n cells has $\log_2 n$ bits; a certain belief has zero. For continuous variables the same expression with an integral is the **differential entropy**, and it can be negative — a density concentrated in a region narrower than one unit has $H < 0$. That is not a contradiction, only a reminder that differential entropy measures spread relative to the units you chose, and that only *differences* of differential entropy are physically meaningful. Information gain, which is a difference, is perfectly well behaved.

DERIVATION Entropy of a Gaussian

$$H = \frac{1}{2} \ln \det(2\pi e \Sigma)$$

Step 1 — write out $-\mathbb{E}[\ln p]$. From the density,

$$-\ln \mathcal{N}(x; \mu, \Sigma) = \frac{1}{2} \ln \det(2\pi \Sigma) + \frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu)$$

so $H = \frac{1}{2} \ln \det(2\pi \Sigma) + \frac{1}{2} \mathbb{E}[(x - \mu)^\top \Sigma^{-1} (x - \mu)]$.

Step 2 — the trace trick. A scalar equals its own trace, and the trace is cyclic and linear, so

$$\mathbb{E}[(x - \mu)^\top \Sigma^{-1} (x - \mu)] = \mathbb{E}[\text{tr}(\Sigma^{-1} (x - \mu)(x - \mu)^\top)] = \text{tr}(\Sigma^{-1} \Sigma) = n$$

Step 3 — collect. $H = \frac{1}{2} \ln \det(2\pi \Sigma) + \frac{n}{2} = \frac{1}{2} \ln((2\pi e)^n \det \Sigma)$, which in 1-D is $\frac{1}{2} \ln(2\pi e \sigma^2)$. ■

Read what is not there. μ does not appear. Entropy measures how uncertain the robot is, not where it thinks it is — so moving a belief costs nothing and squashing it costs everything. And since $\det \Sigma$ is a product of eigenvalues, entropy is a *volume*: one very confident direction can hide behind one very uncertain one, which is exactly the failure mode Chapter 24 has to design around when it uses information gain to choose where a robot should drive next.

2.4 Samples: the third representation

So far a distribution has been a formula or a pair of moments. There is a third option, and half this book runs on it: **a bag of numbers drawn from the distribution**. The law of large numbers says that for any well-behaved f ,

$$\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{i=1}^N f(x^{[i]}), \quad x^{[i]} \sim p(x)$$

with an error that shrinks like $\sigma_f/\sqrt{N}$ by the central limit theorem. Samples have one enormous advantage over moments — they can represent *any* shape, including the three-peaked belief of Chapter 5 that no Gaussian can express — and one enormous disadvantage, which is that $1/\sqrt{N}$ is a miserable rate.

INTERACTIVE · w2.3

Sampling Convergence

A cloud of samples is a legitimate representation of a distribution — but its error only falls like $1/\sqrt{N}$, so ‘add more particles’ is a slow fix.

Run this simulation in the web edition: `/chapters/ch02-probability #w2.3`

Both halves of that widget are worth internalizing before Chapter 8 turns samples into a filter. Ten samples look nothing like a bell curve and yet nothing is wrong; the estimate is unbiased at every N and merely noisy. And getting one more decimal digit of accuracy costs a hundred times the samples, forever, in every dimension. That is why particle counts are argued about, why importance sampling exists, and why nobody runs a naive particle filter over a six-dimensional pose.

```
ALGORITHM sample_multivariate_normal( $\mu$ ,  $\Sigma$ )           COST  $O(n^3)$  once for the
factorization, then  $O(n^2)$  per sample
in mean, covariance, and a seeded generator
out one draw  $x \sim \mathcal{N}(\mu, \Sigma)$ 
```

- 1: $L = \text{cholesky}(\Sigma)$ (*cache across draws*)
- 2: $z = (z_1, \dots, z_n)$ with each $z_i \sim \mathcal{N}(0, 1)$
- 3: return $\mu + Lz$

i NOTE

How it was done in 2000. `sample_normal_distribution` in Thrun et al.'s Table 5.4 exploits the central limit theorem directly: sum twelve uniform draws and scale. It was a sensible trade when a transcendental function cost more than twelve uniforms, and its defects are honest ones. The approximation is poor in the tails, and the support is bounded — a scaled sum of twelve $U(-1, 1)$ draws cannot exceed 6σ — so it can never produce the outlier that breaks your filter, which means it can never warn you that one exists.

It also carries an arithmetic slip worth checking for yourself. Twelve independent draws from $U(-1, 1)$ have total variance $12 \cdot \frac{1}{3} = 4$, so the scale factor that turns their sum into a standard deviation of b is $\frac{1}{2}$; the $\frac{1}{6}$ printed in the draft edition's Table 5.4 delivers $b/3$ instead. It is a one-line thing to catch with a test and a very hard thing to catch by reading, which is the argument for the test convention in this chapter in miniature. Modern samplers use the Ziggurat method (Marsaglia and Tsang, 2000), which is both exact and faster; `rand_distr`'s `StandardNormal` is a Ziggurat, and it is what this book uses.

💡 SEEDED REPRODUCIBILITY, AS A RULE

All randomness in this book flows from an explicitly seeded generator: `SmallRng::seed_from_u64` in Rust, `lib/prob/rng.ts` on the web. Never `thread_rng()`, never `Math.random()`. Every widget displays its seed and offers a re-roll; every test fixes its seed; every figure is therefore reproducible bit-for-bit, native and in the browser. A stochastic result you cannot reproduce is not a result.

2.5 Implementation in Rust

The type carries an invariant — the covariance is symmetric positive-definite — so the constructor is the only place that can create one, and it proves the invariant by factoring. The factor is then cached, and every query below is a triangular solve rather than an inverse.

RUST `crates/pr-core/src/prob/gaussian.rs`

```
1 use nalgebra::{Cholesky, Const, SMatrix, SVector};
2 use rand::Rng;
3 use rand_distr::StandardNormal;
4
5 /// A Gaussian in moments form:  $p(x) = N(x; \text{mean}, \text{cov})$ .
6 ///
7 /// The Cholesky factor is cached because it is the only cubic-cost object here.
```

```

8  /// Given L with cov = L L^T, the density, the Mahalanobis distance, the entropy
9  /// and the sampler are all O(n^2) -- and none of them ever forms cov^-1, which is
10 /// slower, less accurate, and silent when it goes wrong.
11 #[derive(Clone, Debug)]
12 pub struct Gaussian<const N: usize> {
13     mean: SVector<f64, N>,
14     cov: SMatrix<f64, N, N>,
15     chol: Cholesky<f64, Const<N>>,
16 }
17
18 /// The one way to fail: a covariance that is not positive-definite. Usually a
19 /// filter bug three steps upstream, which is exactly why we refuse it here.
20 #[derive(Clone, Copy, Debug, PartialEq, Eq)]
21 pub struct NotPositiveDefinite;
22
23 impl<const N: usize> Gaussian<N> {
24     pub fn new(
25         mean: SVector<f64, N>,
26         cov: SMatrix<f64, N, N>,
27     ) -> Result<Self, NotPositiveDefinite> {
28         // Symmetrize first. A covariance update that is symmetric on paper
29         // drifts by a few ULPs in floating point, and Cholesky is unforgiving.
30         let cov = (cov + cov.transpose()) * 0.5;
31         let chol = Cholesky::new(cov).ok_or(NotPositiveDefinite)?;
32         Ok(Self { mean, cov, chol })
33     }
34
35     pub fn mean(&self) -> &SVector<f64, N> { &self.mean }
36     pub fn cov(&self) -> &SMatrix<f64, N, N> { &self.cov }
37
38     /// ln|Sigma| = 2 Sigma? ln L??. Free, given the factor -- and it never overflows the
39     /// way a product of eigenvalues does for a 30-dimensional SLAM state.
40     pub fn ln_det(&self) -> f64 {
41         2.0 * self.chol.l().diagonal().iter().map(|d| d.ln()).sum::<f64>()
42     }
43
44     /// d2(x) = (x - mu)^T Sigma^-1 (x - mu) by one triangular solve (Algorithm above).
45     pub fn mahalanobis2(&self, x: &SVector<f64, N>) -> f64 {
46         let d = x - self.mean;
47         d.dot(&self.chol.solve(&d))
48     }
49
50     pub fn ln_pdf(&self, x: &SVector<f64, N>) -> f64 {
51         let ln_2pi = std::f64::consts::TAU.ln();
52         -0.5 * (self.mahalanobis2(x) + self.ln_det()) + N as f64 * ln_2pi
53     }
54
55     pub fn pdf(&self, x: &SVector<f64, N>) -> f64 {
56         self.ln_pdf(x).exp()
57     }
58
59     /// H = ? ln((2pie)^n |Sigma|), Derivation 7. Note mu is absent: entropy says how
60     /// uncertain the robot is, never where it thinks it is.
61     pub fn entropy(&self) -> f64 {
62         let ln_2pi_e = (std::f64::consts::TAU * std::f64::consts::E).ln();
63         0.5 * (N as f64 * ln_2pi_e + self.ln_det())
64     }
65
66     /// x = mu + L z, Derivation 4 read backwards.
67     pub fn sample<R: Rng + ?Sized>(&self, rng: &mut R) -> SVector<f64, N> {
68         let z = SVector::<f64, N>::from_fn(|_, _| rng.sample(StandardNormal));

```

```

69         self.mean + self.chol.l() * z
70     }
71
72     /// Push forward through  $y = A x + b$  (Derivation 4). The const generics are
73     /// doing real work: a  $2 \times 3$  A applied to a 3-D state can only produce a 2-D
74     /// Gaussian, and getting it wrong is a compile error rather than a panic.
75     pub fn transform<const M: usize>(
76         &self,
77         a: &SMatrix<f64, M, N>,
78         b: &SVector<f64, M>,
79     ) -> Result<Gaussian<M>, NotPositiveDefinite> {
80         Gaussian::new(a * self.mean + b, a * self.cov * a.transpose())
81     }
82
83     ///  $(\mu, \Sigma) \rightarrow (x_i, \Omega)$ .  $x_i$  comes from a solve;  $\Omega$  has to be materialized,
84     because
85     /// that is what the canonical form is. Pay the  $O(n^3)$  once, then add.
86     pub fn to_canonical(&self) -> Canonical<N> {
87         Canonical { xi: self.chol.solve(&self.mean), omega: self.chol.inverse() }
88     }
89
90     ///  $p(x_a | x_b = \beta)$  for a 2-D joint (Derivation 6), written in scalars so the
91     /// Schur complement is visible. The gain k moves the mean and shrinks the
92     /// variance; in Chapter 6 it acquires a name and a subscript.
93     pub fn condition_second(g: &Gaussian<2>, beta: f64) -> Gaussian<1> {
94         let (mu, s) = (g.mean(), g.cov());
95         let k = s[(0, 1)] / s[(1, 1)];
96         Gaussian::new(
97             SVector::from([mu[0] + k * (beta - mu[1])]),
98             SMatrix::from([[s[(0, 0)] - k * s[(0, 1)]]]),
99         )
100         .expect("the Schur complement of an SPD matrix is SPD")
101     }

```

The canonical form is the same distribution asking a different question, and its entire implementation is Derivation 3.

RUST `crates/pr-core/src/prob/canonical.rs`

```

1 use nalgebra::{Cholesky, SMatrix, SVector};
2 use super::gaussian::{Gaussian, NotPositiveDefinite};
3
4 /// Canonical (information) form:  $p(x) \propto \exp(-x^T \Omega x + x^T \xi)$ .
5 ///
6 /// Moments form answers "where is it and how wide". Canonical form answers
7 /// "how much do I know". They carry identical information; what differs is
8 /// which of the two Bayes-filter operations is cheap.
9 #[derive(Clone, Debug, PartialEq)]
10 pub struct Canonical<const N: usize> {
11     ///  $\xi = \Sigma^{-1} \mu$ 
12     pub xi: SVector<f64, N>,
13     ///  $\Omega = \Sigma^{-1}$  -- sparse for a pose graph, dense for its covariance.
14     pub omega: SMatrix<f64, N, N>,
15 }
16
17 impl<const N: usize> Canonical<N> {
18     /// The Bayes product: exponents add, so the parameters add.  $O(n^2)$ , and

```

```

19  /// there is genuinely nothing else to it.
20  pub fn product(&self, other: &Self) -> Self {
21      Self { xi: self.xi + other.xi, omega: self.omega + other.omega }
22  }
23
24  /// Back to moments: one factorization, one solve, one inverse. O(n3).
25  pub fn to_moments(&self) -> Result<Gaussian<N>, NotPositiveDefinite> {
26      let chol = Cholesky::new(self.omega).ok_or(NotPositiveDefinite)?;
27      Gaussian::new(chol.solve(&self.xi), chol.inverse())
28  }
29  }

```

Randomness gets its own module for one reason: so that there is exactly one place in the workspace where a generator is constructed, and it takes a seed.

RUST crates/pr-core/src/prob/sample.rs

```

1  use rand::{rngs::SmallRng, Rng, SeedableRng};
2
3  /// Every demo, figure and test in this book starts here. Widgets display it.
4  pub const BOOK_SEED: u64 = 0x5EED_2026;
5
6  /// The only RNG constructor in the workspace. `thread_rng()` appears nowhere:
7  /// a figure that cannot be reproduced is not evidence, and a flaky test is a
8  /// test that will eventually be deleted rather than fixed.
9  pub fn rng(seed: u64) -> SmallRng {
10     SmallRng::seed_from_u64(seed)
11 }
12
13 /// Thrun et al., Table 5.4 -- how you sampled a normal in 2000. Kept because
14 /// the book discusses it; never called, because `rand_distr::StandardNormal`
15 /// is exact, faster, and has tails.
16 ///
17 /// Twelve draws from U(-1,1) sum to variance 12.(1/3) = 4, so the multiplier
18 /// that yields standard deviation `b` is 1/2. Support is bounded at +/-6b, which
19 /// is the defect that matters: this sampler cannot produce the outlier that
20 /// breaks your filter, so it will never warn you that one exists.
21 pub fn sample_normal_distribution(b: f64, rng: &mut impl Rng) -> f64 {
22     let sum: f64 = (0..12).map(|_| rng.random_range(-1.0..1.0)).sum();
23     0.5 * b * sum
24 }

```

2.5.1 A worked example you can check by hand

Rusty's odometry gives a prior $\mathcal{N}(5.0, 4.0)$ — 5 m, standard deviation 2 m. The wall-range sensor gives a likelihood $\mathcal{N}(6.5, 1.0)$. Convert both, add, convert back:

$$\Omega_1 = \frac{1}{4} = 0.25 + \Omega_2 = \frac{1}{1} = 1.00 = \Omega = 1.25$$

$$\xi_1 = \frac{5.0}{4} = 1.25 + \xi_2 = \frac{6.5}{1} = 6.50 = \xi = 7.75$$

$$\sigma^2 = \frac{1}{\Omega} = 0.8 \quad \mu = \frac{\xi}{\Omega} = \frac{7.75}{1.25} = 6.2$$

Sanity-check the two claims from the widget against these numbers. The posterior variance 0.8 is below both 4.0 and 1.0. The posterior mean 6.2 sits 80% of the way from the prior to the sensor, because the sensor is four times as precise: $\frac{1}{1}/(\frac{1}{4} + \frac{1}{1}) = 0.8$.

Now the 2-D act. Take $\mu = 0$ and

$$\Sigma = \begin{bmatrix} 4.0 & 1.9 \\ 1.9 & 1.0 \end{bmatrix} \quad \rho = \frac{1.9}{\sqrt{4 \cdot 1}} = 0.95 \quad \det \Sigma = 4 - 1.9^2 = 0.39$$

Conditioning on $x_b = 1$ gives, from Derivation 6, a mean of 1.9 and a variance of $4 - 1.9^2/1 = 0.39$ — the same 0.39, because in 2-D $\sigma_{a|b}^2 = \det \Sigma / \sigma_b^2$ and $\sigma_b^2 = 1$ here. The marginal standard deviation was 2.0; the conditional one is $\sqrt{0.39} = 0.6245$. One scalar observation of a correlated quantity removed 69% of the uncertainty without measuring x_a at all.

In entropy terms, the correlation is worth

$$\Delta H = \frac{1}{2} \ln \frac{4.0}{0.39} = -\frac{1}{2} \ln(1 - \rho^2) = 1.164 \text{ nats} = 1.679 \text{ bits}$$

2.5.2 The tests that pin those numbers

RUST `crates/pr-core/src/prob/gaussian.rs (tests)`

```
1  #[cfg(test)]
2  mod tests {
3      use super::*;
4      use crate::prob::sample::{rng, BOOK_SEED};
5      use approx::assert_relative_eq;
6      use nalgebra::{Matrix2, SMatrix, SVector, Vector2};
7
8      /// The chapter's 1-D worked example, to the printed digit. If this test
9      /// ever disagrees with the prose, the test is what settles it.
10     #[test]
11     fn worked_example_ch02_fuse_two_sensors() {
12         let prior = Gaussian::<1>::new(SVector::from([5.0]), SMatrix::from([[4.0]]))
13         .unwrap();
14         let likelihood = Gaussian::<1>::new(SVector::from([6.5]), SMatrix::from([[1.0]]))
15         .unwrap();
16
17         let fused = prior.to_canonical().product(&likelihood.to_canonical());
18         assert_relative_eq!(fused.omega[0, 0], 1.25, epsilon = 1e-12); // 0.25 + 1.00
19         assert_relative_eq!(fused.xi[0], 7.75, epsilon = 1e-12);       // 1.25 + 6.50
20
21         let posterior = fused.to_moments().unwrap();
22         assert_relative_eq!(posterior.mean()[0], 6.2, epsilon = 1e-12);
23         assert_relative_eq!(posterior.cov()[0, 0], 0.8, epsilon = 1e-12);
24     }
```

```

23
24  /// The 2-D worked example: determinant, Schur complement, and the entropy
25  /// that correlation is worth.
26  #[test]
27  fn worked_example_ch02_correlated_pair() {
28      let sigma = Matrix2::new(4.0, 1.9, 1.9, 1.0);
29      let joint = Gaussian::<2>::new(Vector2::zeros(), sigma).unwrap();
30
31      assert_relative_eq!(sigma.determinant(), 0.39, epsilon = 1e-9);
32
33      let conditional = condition_second(&joint, 1.0);
34      assert_relative_eq!(conditional.mean()[0], 1.9, epsilon = 1e-9);
35      assert_relative_eq!(conditional.cov()[(0, 0)], 0.39, epsilon = 1e-9);
36
37      let uncorrelated =
38          Gaussian::<2>::new(Vector2::zeros(), Matrix2::new(4.0, 0.0, 0.0, 1.0)).unwrap
39      (
40          /// ? ln(4.0 / 0.39) = -? ln(1 - rho2), the nats that correlation is worth.
41          let gain = uncorrelated.entropy() - joint.entropy();
42          assert_relative_eq!(gain, 1.163_951_45, epsilon = 1e-8);
43      );
44
45      /// Round-trip: moments -> canonical -> moments is the identity. Property-test
46      /// this over random SPD matrices; the fixed case is the regression guard.
47      #[test]
48      fn canonical_round_trip_is_identity() {
49          let sigma = Matrix2::new(4.0, 1.9, 1.9, 1.0);
50          let g = Gaussian::<2>::new(Vector2::new(1.0, -2.0), sigma).unwrap();
51          let back = g.to_canonical().to_moments().unwrap();
52          assert_relative_eq!(*back.mean(), *g.mean(), epsilon = 1e-10);
53          assert_relative_eq!(*back.cov(), *g.cov(), epsilon = 1e-10);
54      }
55
56      /// The 95% ellipse had better contain 95% of the samples. If it does not,
57      /// either the sampler or the chi2 quantile is wrong -- and every covariance
58      /// ellipse printed in this book is lying by the same amount.
59      #[test]
60      fn ellipse_coverage_is_what_it_claims() {
61          let g = Gaussian::<2>::new(Vector2::zeros(), Matrix2::new(4.0, 1.9, 1.9, 1.0)).
62          unwrap();
63          let mut r = rng(B00K_SEED);
64          const M: usize = 100_000;
65          const CHI2_95: f64 = 5.991_464_547_107_98; // -2 ln 0.05
66
67          let inside = (0..M)
68              .filter(|_| g.mahalanobis2(&g.sample(&mut r)) <= CHI2_95)
69              .count();
70          assert!((inside as f64 / M as f64 - 0.95).abs() < 5e-3);
71      }
72  }

```

2.6 Putting it together

The runnable artifact for this chapter is one example binary that does both acts and prints the numbers the prose claims.

RUST crates/pr-core/examples/fuse_two_sensors.rs

```

1 use nalgebra::{Matrix2, SMatrix, SVector, Vector2};
2 use pr_core::prob::rng, Gaussian, B00K_SEED;
3
4 fn main() {
5     // Act I -- one dimension, by hand-checkable arithmetic.
6     let prior = Gaussian::<1>::new(SVector::from([5.0]), SMatrix::from([[4.0]]).unwrap());
7     let likelihood = Gaussian::<1>::new(SVector::from([6.5]), SMatrix::from([[1.0]]).unwrap());
8     let posterior = prior
9         .to_canonical()
10        .product(&likelihood.to_canonical())
11        .to_moments()
12        .unwrap();
13
14    println!("prior      N(mu = {:.3}, var = {:.3})", prior.mean()[0], prior.cov()[0][0]);
15    println!("likelihood N(mu = {:.3}, var = {:.3})", likelihood.mean()[0], likelihood.cov()[0][0]);
16    println!("posterior  N(mu = {:.3}, var = {:.3})", posterior.mean()[0], posterior.cov()[0][0]);
17    println!("entropy    {:.4} nats (prior {:.4})", posterior.entropy(), prior.entropy());
18
19    // Act II -- two dimensions, checked against 10 000 seeded draws.
20    let joint = Gaussian::<2>::new(Vector2::zeros(), Matrix2::new(4.0, 1.9, 1.9, 1.0)).unwrap();
21    let mut r = rng(B00K_SEED);
22    let draws: Vec<_> = (0..10_000).map(|_| joint.sample(&mut r)).collect();
23
24    let mean = draws.iter().sum::<Vector2<f64>>() / draws.len() as f64;
25    let cov = draws
26        .iter()
27        .map(|x| (x - mean) * (x - mean).transpose())
28        .sum::<Matrix2<f64>>()
29        / (draws.len() - 1) as f64;
30
31    println!("sample cov  [{:.3} {:.3}; {:.3} {:.3}]", cov[0][0], cov[0][1], cov[1][0], cov[1][1]);
32    println!("entropy    {:.4} nats", joint.entropy());
33 }

```

```

prior      N(mu = 5.000, var = 4.000)
likelihood N(mu = 6.500, var = 1.000)
posterior  N(mu = 6.200, var = 0.800)
entropy    1.3074 nats (prior 2.1121)
sample cov [3.984 1.891; 1.891 0.995]
entropy    2.3671 nats

```

Read the first entropy line against Derivation 7: $\frac{1}{2} \ln(2\pi e \cdot 4) = 2.1121$ before the measurement, $\frac{1}{2} \ln(2\pi e \cdot 0.8) = 1.3074$ after. The sensor was worth 0.80 nats, or 1.16 bits — and it would have been worth exactly the same number had it read 6.5, 60, or -4 , because entropy does not care where the belief sits. The 2-D entropy, 2.3671 nats, is $\ln(2\pi e) + \frac{1}{2} \ln 0.39$, and the 1.164 nats separating it from the uncorrelated case is the correlation earning its keep.

The sample covariance agrees with Σ to about half a percent at $N = 10^4$, which is the $1/\sqrt{N}$ rate of the Sampling Convergence widget doing exactly what it promised — and no better. Re-seed and the third digit moves; the second does not.

That is the whole toolkit. From here:

- Chapter 5 applies Bayes rule recursively, and the "compute the shape, normalize later" habit becomes line 3 of the Bayes filter.
- Chapter 6 industrializes Derivations 2, 4 and 6 into the Kalman filter, and the moments/canonical duality of Derivation 3 becomes the Kalman filter and the information filter side by side.
- Chapter 8 takes samples seriously as the belief itself.
- Chapter 11 gates data associations on the Mahalanobis distance of Derivation 5, using the very χ^2 number that sizes the ellipses here.
- Chapter 24 turns the entropy of Derivation 7 into information gain, and lets the robot choose where to drive.

One last thing worth saying plainly: the purple curve in the Blob Multiplier is not an illustration of `Canonical::product`. It is the output of the TypeScript port of `Canonical::product`, fed the same numbers as the Rust test, agreeing to the last digit the screen can show. Where the prose, the figure and the code disagree in this book, the test is what settles it — and there is always a test.

2.7 Exercises

1 FOUNDATION •• The product in n dimensions, twice

Derive the product of two n -dimensional Gaussians in canonical form. It should take three lines. Then derive the same result in moments form, which will take a page and require the matrix inversion lemma. Conclude in one sentence why Chapter 6 offers two filters rather than one, and state which one you would choose for a robot with twenty sensors and one motion model.

2 FOUNDATION •• Variance is positive, entropy is not

Show that $\text{Cov}[X] \geq 0$ for every random vector with finite second moments, directly from the definition. Then specialize the Gaussian entropy formula to one dimension and find the values of σ for which the differential entropy is negative. Explain in two sentences why a negative entropy is not a contradiction, and why the *difference* of two differential entropies is still meaningful when both are negative.

3 FOUNDATION • • • The evidence is a Gaussian

Derivation 2 claims that the constant thrown away when two Gaussians are multiplied is exactly $\mathcal{N}(\mu_1; \mu_2, \sigma_1^2 + \sigma_2^2)$. Prove it by carrying the constants through Steps 1–4 instead of discarding them. Then explain why this makes an innovation-based outlier test and an evidence-based one the same test.

4 CONCEPTUAL • Predict, then check: precision weighting

In the Blob Multiplier, set the likelihood’s variance to nine times the prior’s. Before you look: what fraction of the distance from the prior mean to the likelihood mean will the posterior mean travel? Write the number down, then verify it. Now push the likelihood variance to its maximum and predict what the posterior converges to and why. Finally, find a setting where the posterior is *narrower than the likelihood by less than one percent* and say what that setting means physically.

Hint. The posterior mean is a weighted average with weights proportional to $1/\sigma^2$.

5 CONCEPTUAL • • Predict, then check: the slice that does not widen

Set $\rho = 0.95$ in Slice vs. Squash. Predict whether the conditional at $\beta = \mu_b$ is narrower, wider, or the same width as the conditional at $\beta = \mu_b + 2\sigma_b$. Most people get this wrong the first time. Verify with the widget, then explain the result using Derivation 6 by pointing at the one symbol that is missing from the conditional covariance. Finally, use the Gaussian Playground to explain why the ellipse’s horizontal extent at $\rho = 0.95$ is *not* the width of the blue marginal curve underneath it.

6 PRACTICAL • • Conditioning in general dimension

Implement conditioning for a partitioned ‘Gaussian’, returning the distribution of the unobserved block given values for the observed one. Rust’s `const` generics cannot express `Gaussian<{N - K}>` on stable, so you will have to choose: fixed 2-block `const` parameters with an `A + B == N` assertion, a dynamically-sized `DVector/DMatrix` variant, or the nightly `generic_const_exprs` feature. Pick one, justify it in a comment, and property-test the result against brute-force numerical integration of the joint density on a 2-D grid.

7 PRACTICAL • • • Two samplers, one benchmark

Implement `sample_normal_distribution(b)` from the Callout above and benchmark it against `rand_distr::StandardNormal` with Criterion at 10^6 draws. Report throughput, and report the first four sample moments of each. Then do the part that actually matters: estimate $P(|X| > 5)$ with both samplers and compare against the exact 5.7×10^{-7} . Write two sentences on why a filter that gates outliers at 5σ must not be tested with the

twelve-uniform sampler.

2.8 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics MIT Press*.

Section 2.2 is the notation baseline for this chapter — η , the Gaussian, expectation, entropy. The canonical parameterization is deferred there to the information filter in Chapter 3; we pull it forward so the duality is a one-line observation later instead of a surprise. [link](#)

Jaynes, E. T. (edited by G. L. Bretthorst) (2003). *Probability Theory: The Logic of Science Cambridge University Press*.

The argument that probability is extended logic rather than a theory of frequencies. Read Chapters 1–2 if the phrase ‘the robot’s belief’ has ever felt like a category error. doi:10.1017/CB09780511790423

Marsaglia, G. and Tsang, W. W. (2000). The Ziggurat Method for Generating Random Variables *Journal of Statistical Software* 5(8), 1–7.

The sampler behind `rand_distr::StandardNormal`, and the reason this book does not use Thrun’s twelve-uniform approximation: exact tails, and faster. doi:10.18637/jss.v005.i08

Barfoot, T. D. (2024). *State Estimation for Robotics, 2nd edition Cambridge University Press*. Chapter 2 covers the same Gaussian identities with more care about the linear-algebra details, including the block-inverse manipulations we sketch in Derivation 6 and prove in Appendix B. The author maintains a freely readable draft at the URL above. [link](#)

Barfoot, T. D., Forbes, J. R., and Yoon, D. J. (2020). Exactly sparse Gaussian variational inference with application to derivative-free batch nonlinear state estimation *International Journal of Robotics Research* 39(13), 1473–1502.

A modern argument for living in canonical form: the inverse covariance of a batch estimation problem is block-tridiagonal, so storing Ω instead of Σ turns a dense $O(n^2)$ object into a sparse $O(n)$ one. Derivation 3 is the toy version of the whole method. doi:10.1177/0278364920937608

Barfoot, T. D. (2020). Fundamental Linear Algebra Problem of Gaussian Inference *arXiv:2010.08022*.

States precisely which entries of Σ you can recover cheaply once you have Ω , via the Takahashi recursions. The clearest available account of why the moments/canonical trade in Derivation 3 is not symmetric in practice. [link](#)

Ortiz, J., Evans, T., and Davison, A. J. (2021). A visual introduction to Gaussian Belief Propagation *arXiv:2107.02308*.

Gaussians in canonical form, made interactive — and the closest thing in the literature to what this book is trying to be. Its factor-multiplication figures are the multi-variable sequel to the Blob Multiplier. [link](#)

Placed, J. A., Strader, J., Carrillo, H., Atanasov, N., Indelman, V., Carlone, L., and Castellanos, J. A. (2023). A Survey on Active Simultaneous Localization and Mapping: State of the Art and New Frontiers *IEEE Transactions on Robotics* 39(3), 1686–1705.

Where the entropy of Derivation 7 goes: Section IV surveys the information-theoretic utilities — entropy reduction, D-optimality, $\log \det \Sigma$ — that Chapter 24 uses to decide where a robot should drive next. doi:10.1109/TR0.2023.3248510

The Geometry of Motion

FOUNDATIONS · Intermediate · 55 min



Any rigid-body configuration can be achieved by starting from the fixed (home) reference frame and integrating a constant twist for a specified time.

Kevin M. Lynch and Frank C. Park

Modern Robotics, Chapter 3

Probabilistic robotics puts distributions on poses. A pose is not a vector, and almost every frustrating bug in a student SLAM system comes from forgetting it: the heading that averages to zero when it should average to 180° , the covariance ellipse drawn on a quantity that does not live in a plane, the trajectory that drifts because someone integrated a turn as if it were a straight line.

This chapter builds the substrate honestly. Frames and rotation matrices, homogeneous transforms and the cancellation rule, the exponential map from the rotation ODE, $SE(2)$ worked to the last decimal — and then the move that the classical robotics texts do not make, because they never put a distribution on anything: the retraction operators $\mathbb{T}$ and $\mathbb{E}$, which turn a curved manifold into a local vector space that a Kalman filter can work in without lying.

It also forges the book's most reused artifact. The $SE2$ type built here is imported by every filter, every motion model, and every SLAM system from Chapter 4 to Chapter 26. Get it right once.

🕒 AFTER THIS CHAPTER YOU CAN

- Read and write Craig frame notation, and use the cancellation rule to compose transforms without guessing
- Derive the $SE(2)$ exponential and logarithm in closed form, including the small-angle guard the code needs
- Explain why $\exp$ turns a constant twist into an arc, and what componen-

twice interpolation of (x, y, θ) gets wrong

- Use $\boxplus$ and $\boxminus$ to add an increment to a pose, and state the axioms that make that legitimate
- Implement the book's SE2 type in Rust, and enforce frame correctness at compile time with newtypes

ASSUMED

- Matrix algebra: products, transposes, inverses, orthogonality
- Gaussians in n dimensions (Chapter 2) — used only for the closing section
- Enough calculus to differentiate a matrix-valued function of one variable

3.1 The average of 179° and -179°

Rusty has two heading estimates. One says 179° , the other says -179° . They disagree by two degrees. What is the average heading?

Add and halve: $(179 + (-179))/2 = 0$. The robot is now facing due east, ninety degrees off from where either estimate said, having averaged two nearly identical opinions into their exact opposite. Nothing was noisy, nothing was miscalibrated, and no floating-point rounding was involved. The arithmetic was simply applied to objects that do not support it.

INTERACTIVE · w3.3

The Wrap-Around Trap

Angles are not numbers. Averaging two headings by adding them and halving is either exactly right or exactly backwards.

Run this simulation in the web edition: </chapters/ch03-geometry-of-motion/#w3.3>

The failure is not an edge case to be patched with an `if` statement. It is structural: headings live on a circle, and a circle has no consistent notion of "add these two numbers." Every quantity in this book that involves an orientation — a pose, a rotation, a relative measurement between two poses — has the same property, and every operation a filter performs on such a quantity has to be defined rather than assumed.

The repair visible in the purple needle is the entire chapter compressed into one line. Do not add the angles. Instead:

1. Ask what rotation takes A to B. That is a *difference*, written $\boxminus$, and on a circle it is the short way around: $\theta_B \boxminus \theta_A = 2^\circ$, not -358° .

2. Halve that rotation. Rotations *can* be scaled — they live in a genuine vector space, the tangent space.
3. Apply the halved rotation to A. That is $\boxplus$, and the result is 180° .

Subtract on the manifold, work in the tangent space, come back to the manifold. Chapters 7 (Chapter 7), 9 (Chapter 9), 15 (Chapter 15), and 16 (Chapter 16) are all applications of that sentence to progressively harder problems.

💡 THE ONE SENTENCE VERSION

A pose is an element of a Lie group, not a triple of numbers. The exponential map turns straight lines in the tangent space into arcs in the world, and $\boxplus/\boxminus$ let a filter do vector arithmetic in the tangent space and put the answer back where it belongs.

3.2 Building intuition: frames, and the notation that types them

Before rotations there are frames. A **frame** is a choice of origin and orthonormal axes; a point in physical space has one position and as many coordinate triples as there are frames looking at it. Craig's notation makes the bookkeeping explicit by decorating every symbol with the frame it is expressed in:

$\{A\}, \{B\}$	Coordinate frames. Rusty carries a body frame $\{B\}$; the world frame $\{W\}$ is bolted to the map.
${}^A p$	The point p , expressed in the coordinates of $\{A\}$. The point does not change; the triple does.
${}_B^A T \in \text{SE}(2)$	The transform "A from B": it maps p to p , and equivalently describes the pose of $\{B\}$ as seen from $\{A\}$.
${}_B^A R \in \text{SO}(2)$	Its rotation block. The columns are $\{B\}$'s axes written in $\{A\}$.
$\tau = (v_x, v_y, \omega)^T$	Tangent (twist) coordinates for $\text{SE}(2)$, translation first. Note the ordering — this book follows Solà, not Lynch–Park.
$\tau^\wedge, (\cdot)^\vee$	Hat maps tangent coordinates to the 3×3 Lie-algebra matrix; vee undoes it.
Ad_T	The adjoint of T : the 3×3 matrix that moves a twist from one frame to another.
$x \boxplus \delta, y \boxminus x$	Retraction $x \cdot \exp(\delta^\wedge)$ and its local inverse $\log(x^{-1}y)^\vee$. Right/local convention, fixed book-wide.
$q \in S^3$	A unit quaternion; the double cover of $\text{SO}(3)$.

Read every transform name left to right as **target-from-source**. Then ${}_B^A T$ consumes something expressed in $\{B\}$ and produces something expressed in $\{A\}$, and the rule for chaining transforms writes itself: adjacent indices must match,

and they cancel.

INTERACTIVE · w3.1

Frame Composer

The leading super/subscripts are a type system, not decoration — and transform composition does not commute.

Run this simulation in the web edition: [/chapters/ch03-geometry-of-motion #w3.1](#)

Two features of that widget deserve to be stated as mathematics rather than left as observations.

3.2.1 Derivation: rotation matrices are stacked axes

DERIVATION The columns of a rotation matrix are the frame's own axes

$$R = [\hat{x}], R^T R = I, \det R = +1$$

Step 1 — expand the point in $\{B\}$'s basis. Write ${}^B p = (p_1, p_2)^T$, which means $p = p_1 \hat{x}_B + p_2 \hat{y}_B$ as a geometric statement about arrows, independent of any frame.

Step 2 — express that statement in $\{A\}$. Taking coordinates is linear, so

$${}^A p = p_1 {}^A \hat{x}_B + p_2 {}^A \hat{y}_B = \underbrace{\begin{bmatrix} {}^A \hat{x}_B & {}^A \hat{y}_B \end{bmatrix}}_{{}_B^A R} {}^B p.$$

So the columns of ${}_B^A R$ are literally $\{B\}$'s axes, written in $\{A\}$'s coordinates. That is worth remembering when you are staring at four numbers in a debugger: the first column is where the body's nose points.

Step 3 — read off the constraints. The axes are unit length and mutually perpendicular, and the (i, j) entry of $R^T R$ is the dot product of columns i and j . Hence $R^T R = I$, so $R^{-1} = R^T$. Taking determinants gives $(\det R)^2 = 1$; a right-handed frame maps to a right-handed frame, which selects $\det R = +1$ and excludes reflections.

The set of such matrices is the **special orthogonal group**

$$\text{SO}(2) = \{ R \in \mathbb{R}^{2 \times 2} : R^T R = I, \det R = +1 \},$$

and $\text{SO}(3)$ is defined identically with 3×3 matrices. "Special" is the $\det = +1$;

"orthogonal" is $R^\top R = I$. Both are groups under matrix multiplication: closed, associative, with identity I and inverse $R^\top$.

A remark on reading a rotation two ways (Craig §2.2 vs §2.3). The same matrix ${}^A_B R$ can be read as a *mapping* — take coordinates in $\{B\}$, return coordinates in $\{A\}$ — or as an *operator* — take a vector in $\{A\}$ and rotate it. The numbers are identical; the intent is not. Confusing the two is the single most common sign error in robotics code, and it is exactly what the frame-annotated types in the Practical section are designed to prevent.

3.2.2 Derivation: the cancellation rule

A homogeneous transform packages rotation and translation into one matrix that composes by multiplication:

$${}^A_B T = \begin{bmatrix} {}^A_B R & {}^A p_B \\ 0 & 1 \end{bmatrix} \in \text{SE}(2), \quad \begin{bmatrix} {}^A p \\ 1 \end{bmatrix} = {}^A_B T \begin{bmatrix} {}^B p \\ 1 \end{bmatrix}.$$

DERIVATION Composition, inversion, and why the indices cancel

$${}_C T = {}_B T \cdot {}_C T, T^{-1} = (R^\top, -R^\top t)$$

Step 1 — compose the mappings. Apply ${}_C^B T$ to a point expressed in $\{C\}$, then apply ${}_B^A T$ to the result:

$${}^A p = {}^A_B T ({}_B^C T {}^C p) = ({}_B^A T {}_C^B T) {}^C p.$$

Associativity of matrix multiplication is what lets us drop the parentheses, and the bracketed product must therefore be ${}_C^A T$.

Step 2 — read the index pattern. In ${}_B^A T {}_C^B T$ the inner indices are both B and they cancel, leaving ${}_C^A T$. The expression ${}_B^A T {}_L^A T$ has inner indices B and A ; it does not cancel, and it is meaningless. This is a type check performed with a pencil, and the Practical section hands it to rustc.

Step 3 — invert by blocks. Guess $\begin{bmatrix} R^\top & -R^\top t \\ 0 & 1 \end{bmatrix}$ and multiply:

$$\begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R^\top & -R^\top t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} R R^\top & -R R^\top t + t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} I & 0 \\ 0 & 1 \end{bmatrix}. \blacksquare$$

Note what the inverse is *not*: it is not $(-R, -t)$, and it is not $(R^\top, -t)$. The translation must be rotated back into the other frame before it is negated, because it was measured in the frame you are leaving.

Transform equations. Because inverses are available, chains can be solved rather than only evaluated. If a calibration rig tells you ${}^W_B T$ and ${}^W_L T$ and you want the sensor extrinsics ${}^B_L T$, write ${}^W_B T {}^B_L T = {}^W_L T$ and left-multiply by the inverse: ${}^B_L T = ({}^W_B T)^{-1} {}^W_L T$. That pattern — *unknown = inverse of what you have, times what you want* — is Craig’s transform-equation technique, and it reappears in Chapter 16 as the definition of a loop-closure constraint.

A worked composition, checkable in your head. Put Rusty at ${}^W_B T = (2, 1, 90^\circ)$ and mount the lidar at ${}^B_L T = (0.3, 0, 0)$ — thirty centimetres forward on the robot’s nose. Then

$${}^W_L T = {}^W_B T {}^B_L T = (2 + \cos 90^\circ \cdot 0.3, 1 + \sin 90^\circ \cdot 0.3, 90^\circ) = (2, 1.3, 90^\circ).$$

The lidar sits 30 cm *north* of the robot, not east, because "forward" is a body-frame word. A landmark the lidar reports at ${}^L p = (1, 0)$ is therefore at ${}^W p = (2 - 0, 1.3 + 1) = (2, 2.3)$ in the map — a metre "ahead" of the sensor is a metre north in the world, because the whole chain was rotated. The widget above runs the same product with a yawed mount, so its numbers differ; the arithmetic does not.

3.3 The mathematics

3.3.1 Rotations in three dimensions, and the traps

SO(3) is SO(2)’s harder sibling. Three facts make it harder, and all three cost real robots real accuracy.

It is three-dimensional but embedded in nine numbers. A 3×3 matrix has nine entries constrained by six equations ($R^\top R = I$ is symmetric), leaving three degrees of freedom. Numerical drift pushes a matrix off that constraint surface, so long-running code must re-orthonormalize.

It is not commutative. $R_1 R_2 \neq R_2 R_1$ in general. Rotate this page 90° about the vertical axis, then 90° about the horizontal one; reverse the order; the page ends up somewhere else. The Frame Composer above shows the same fact in the plane by applying one increment about the body frame and about the world frame.

Every minimal parameterization is singular somewhere. Three-parameter representations of SO(3) — roll-pitch-yaw and its twenty-three siblings — all have configurations where two of the three axes align and one degree of freedom silently disappears.

⚠ WHERE ROBOTS BREAK THIS

Euler angles, in one box. There are twelve valid axis sequences (ZYX, ZYZ, ...) and each can be interpreted about fixed or current axes, giving twenty-four conventions that all get called "roll, pitch, yaw". They agree at zero and disagree everywhere else. Every one of them has a gimbal-lock singularity. This book uses Euler angles for exactly one purpose — printing a heading for a human to read — and never for math, storage, or interpolation. When you must interoperate with a system that uses them, write down the convention in the type name, not in a comment.

Unit quaternions are the standard repair: four numbers, one constraint, no singularities.

📖 DERIVATION Unit quaternions in one honest page

$$q = (\cos\theta/2, \sin\theta/2), pqpq^*$$

Step 1 — the rotation action. Identify a point $p \in \mathbb{R}^3$ with the pure quaternion $(0, p)$. For a unit quaternion q , the map $p \mapsto qpq^*$ (with q^* the conjugate) sends pure quaternions to pure quaternions and preserves norms, so it is an isometry of $\mathbb{R}^3$ fixing the origin. Writing $q = (\cos \frac{\theta}{2}, \hat{w} \sin \frac{\theta}{2})$ and expanding the product recovers Rodrigues' formula for a rotation by θ about $\hat{w}$.

Step 2 — composition is multiplication. $q_1(q_2pq_2^*)q_1^* = (q_1q_2)p(q_1q_2)^*$, so composing rotations is multiplying quaternions: sixteen multiplies instead of twenty-seven, and no orthogonality constraint to maintain — only $\|q\| = 1$, restored by a single division.

Step 3 — the double cover. $(-q)p(-q)^* = qpq^*$, so q and $-q$ name the same rotation. S^3 covers $SO(3)$ twice. Consequences that bite in practice: the "distance" between two quaternions must be taken to the nearer of $\pm q_2$; naively averaging a set of quaternions can cancel them to zero; and interpolation must use `slerp`, which moves along the great circle at constant angular rate, rather than a componentwise blend followed by renormalization.

We derive quaternions here and then delegate them. `nalgebra's UnitQuaternion` and `sophus's SE(3)` are correct, tested, and fast; re-implementing them teaches nothing that the two-dimensional case does not teach more clearly.

3.3.2 The exponential map

Here is the question that unifies everything. A rigid body moves with constant velocity for one unit of time. Where does it end up?

DERIVATION From the rotation ODE to the exponential map

$$= R[\omega] \times R(t) = R(0) \exp([\omega] \times t)$$

Step 1 — differentiate the orthogonality constraint. $R(t)^\top R(t) = I$ holds for all t . Differentiating,

$$\dot{R}^\top R + R^\top \dot{R} = 0 \implies (R^\top \dot{R})^\top = -(R^\top \dot{R}).$$

So $R^\top \dot{R}$ is **skew-symmetric**. Name it $[\omega]_\times$; in the plane that is ωS with $S = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$, and in three dimensions it is the familiar cross-product matrix.

The angular velocity is not an extra modelling assumption — it is forced by orthogonality.

Step 2 — solve the linear matrix ODE. $\dot{R} = R[\omega]_\times$ with ω constant is solved by the matrix exponential, $R(t) = R(0) \exp([\omega]_\times t)$, exactly as the scalar equation $\dot{r} = ra$ is solved by $r_0 e^{at}$.

Step 3 — collapse the series in 2-D. $S^2 = -I$, so the series splits into the cosine and sine series:

$$\exp(\omega S) = \sum_{k \geq 0} \frac{(\omega S)^k}{k!} = I \cos \omega + S \sin \omega = \begin{bmatrix} \cos \omega & -\sin \omega \\ \sin \omega & \cos \omega \end{bmatrix}.$$

The exponential of a skew matrix is a rotation matrix. In two dimensions everything commutes and this is exact and boring, which is precisely why $SE(2)$ is the right place to learn.

Step 4 — collapse the series in 3-D. With $\hat{\omega}$ a unit axis, $[\hat{\omega}]_\times^3 = -[\hat{\omega}]_\times$, so all powers reduce to $[\hat{\omega}]_\times$ and $[\hat{\omega}]_\times^2$, and

$$\exp([\hat{\omega}]_\times \theta) = I + \sin \theta [\hat{\omega}]_\times + (1 - \cos \theta) [\hat{\omega}]_\times^2,$$

which is **Rodrigues' formula**. The three numbers $\hat{\omega}\theta$ are the *exponential coordinates* of the rotation.

Now do the same for a full rigid motion. This is the derivation the rest of the book leans on.

DERIVATION SE(2) exp and log in closed form

$$\exp(\tau) = (R(\omega), V(\omega)\rho), \quad V(\omega) = (1/\omega)[\sin \omega, -(1-\cos \omega); 1-\cos \omega, \sin \omega]$$

Write the tangent vector as $\tau = (\rho, \omega)$ with $\rho = (v_x, v_y)^\top$ the body-frame linear velocity. Its hat form is the 3×3 matrix

$$\tau^\wedge = \begin{bmatrix} \omega S & \rho \\ 0 & 0 \end{bmatrix}, \quad S = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Step 1 — write the motion as an ODE. A body moving with constant *body-frame* velocity satisfies $\dot{T} = T \tau^\wedge$. Splitting into blocks with $T = (R, t)$:

$$\dot{R} = R \omega S, \quad \dot{t} = R \rho.$$

The second equation is the whole story: the body-frame velocity ρ has to be rotated into the world before it can be integrated, and the rotation is itself changing.

Step 2 — integrate. From Step 1 of the previous derivation, $R(s) = R(s\omega)$ starting from $R(0) = I$. Substituting,

$$t(1) = \int_0^1 R(s\omega) \rho \, ds = \left(\int_0^1 R(s\omega) \, ds \right) \rho =: V(\omega) \rho.$$

Step 3 — evaluate the integral entrywise. With $\int_0^1 \cos(s\omega) \, ds = \frac{\sin \omega}{\omega}$ and $\int_0^1 \sin(s\omega) \, ds = \frac{1-\cos \omega}{\omega}$,

$$V(\omega) = \frac{1}{\omega} \begin{bmatrix} \sin \omega & -(1-\cos \omega) \\ 1-\cos \omega & \sin \omega \end{bmatrix}, \quad \exp(\tau^\wedge) = \begin{bmatrix} R(\omega) & V(\omega)\rho \\ 0 & 1 \end{bmatrix}.$$

Step 4 — check the straight-line limit. As $\omega \rightarrow 0$, $\frac{\sin \omega}{\omega} \rightarrow 1$ and $\frac{1-\cos \omega}{\omega} \rightarrow 0$, so $V \rightarrow I$ and $\exp(\tau^\wedge) \rightarrow (I, \rho)$: pure translation. The formula degrades gracefully, which is exactly what a robot driving nearly straight needs.

Step 5 — invert. $\det V = \frac{\sin^2 \omega + (1-\cos \omega)^2}{\omega^2} = \frac{2(1-\cos \omega)}{\omega^2}$, so

$$V(\omega)^{-1} = \begin{bmatrix} c & \omega/2 \\ -\omega/2 & c \end{bmatrix}, \quad c = \frac{\omega \sin \omega}{2(1-\cos \omega)} = \frac{\omega}{2} \cot \frac{\omega}{2},$$

using $1 - \cos \omega = 2 \sin^2 \frac{\omega}{2}$ and $\sin \omega = 2 \sin \frac{\omega}{2} \cos \frac{\omega}{2}$. The logarithm is then

$$\omega = \text{atan2}(R_{21}, R_{11}), \quad \rho = V(\omega)^{-1} t,$$

defined and smooth for $|\omega| < \pi$ — and *only* there, because at $\omega = \pi$ the rotation by $+\pi$ and by $-\pi$ are the same group element and log has to pick one.

Step 6 — the numerical guard. Both V and V^{-1} are 0/0 at $\omega = 0$, so code needs a branch. Below $|\omega| \approx 10^{-8}$ use the series $\frac{\sin \omega}{\omega} = 1 - \frac{\omega^2}{6} + \dots$, $\frac{1 - \cos \omega}{\omega} = \frac{\omega}{2} - \frac{\omega^3}{24} + \dots$, and $c = 1 - \frac{\omega^2}{12} - \dots$. Above it, write $1 - \cos \omega$ as $2 \sin^2 \frac{\omega}{2}$ rather than literally. At the $\omega \approx 10^{-3}$ a 60 Hz control loop produces, $1 - \cos \omega \approx 5 \times 10^{-7}$, so subtracting two numbers that agree to six places throws away six of the sixteen digits a f64 had — and it gets worse as ω shrinks, right in the regime where a robot spends most of its life. The half-angle form never subtracts anything.

The geometric content of $V(\omega)$ is worth saying in words, because it is the payoff. A constant twist does not drive a body along a straight line; it sweeps the body around a fixed point — the **instantaneous center of rotation**, at $(-v_y/\omega, v_x/\omega)$ in body coordinates — at constant angular rate. This is the planar case of the Chasles–Mozzi theorem, the one Lynch and Park state as a screw: every rigid displacement is a rotation about some axis combined with a translation along it, and in the plane the translation vanishes, leaving a rotation about a point or a pure translation. V is the bookkeeping that converts "how far I drove along the arc" into "where I ended up".

INTERACTIVE · w3.2

Exp/Log Lens

exp turns a straight line in the tangent space into a screw motion in the world — and componentwise interpolation of (x, y, θ) is not that motion.

Run this simulation in the web edition: [/chapters/ch03-geometry-of-motion #w3.2](#)

WHY THIS MATTERS FOUR CHAPTERS EARLY

$\exp((\Delta s, 0, \Delta \theta)^T)$ is the constant-curvature arc a differential drive traces when its wheel speeds are held fixed. Chapter 4 does not need a separate odometry integrator: exact diff-drive integration is this exponential, and the classical " $x += v \cos \theta \Delta t$ " update is its first-order truncation.

3.3.3 The adjoint

Twists, like points, are frame-dependent. The object that moves them between frames is the adjoint.

DERIVATION The SE(2) adjoint by block computation

$$Ad_T = [R, -St; 0, 1]$$

Step 1 — conjugate a one-parameter motion. For fixed T , the curve $s \mapsto T \exp(s\tau^\wedge) T^{-1}$ passes through the identity at $s = 0$ and is a one-parameter subgroup, so it equals $\exp(s\sigma^\wedge)$ for some tangent vector σ . Define Ad_T by $\sigma = Ad_T \tau$.

Step 2 — differentiate at $s = 0$. This gives the equivalent linear statement $(Ad_T \tau)^\wedge = T \tau^\wedge T^{-1}$. Expand with $T = (R, t)$:

$$\begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \omega S & \rho \\ 0 & 0 \end{bmatrix} \begin{bmatrix} R^\top & -R^\top t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \omega R S R^\top & -\omega R S R^\top t + R \rho \\ 0 & 0 \end{bmatrix}.$$

Step 3 — use the two-dimensional accident. $S = R(\pi/2)$, and planar rotations commute, so $R S R^\top = S$. The rotation part of the twist is therefore unchanged — in the plane, angular velocity is frame-independent — and the linear part becomes $\rho' = R \rho - \omega S t$. Collecting blocks with the translation-first ordering,

$$Ad_T = \begin{bmatrix} R & -S t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & t_y \\ \sin \theta & \cos \theta & -t_x \\ 0 & 0 & 1 \end{bmatrix}. \blacksquare$$

A note on authority. Sign and ordering conventions for adjoints are notorious; published papers disagree with each other and with themselves. This book's authority is not anyone's memory but the property test `adjoint_conjugation` in the Practical section, which asserts $T \exp(\tau^\wedge) T^{-1} = \exp((Ad_T \tau)^\wedge)$ on every random input `proptest` can throw at it. If you change the tangent ordering, that test — not your memory — tells you what the adjoint became.

The adjoint is how a covariance changes frames. If a body-frame uncertainty is Σ_{body} , the same uncertainty in the world frame is $Ad_T \Sigma_{\text{body}} Ad_T^\top$ — the manifold version of the familiar $A \Sigma A^\top$, and the reason Chapter 9 can compose odometry uncertainty at all.

3.3.4 $\boxplus$ and $\boxminus$: a legitimate local vector space

Every estimator in this book wants to write $x + \delta$: the Kalman update adds a correction to a mean, Gauss–Newton adds a step to an iterate, a particle filter adds noise to a sample. On SE(2) that expression is undefined. The fix is to *define* it, once, with the two operators already used informally above:

$$x \boxplus \delta = x \cdot \exp(\delta^\wedge), \quad y \boxminus x = \log(x^{-1}y)^\vee.$$

 **DERIVATION** $\boxplus$ and $\boxminus$ satisfy the Hertzberg axioms

$$x \boxplus 0 = x; x \boxplus (y \boxminus x) = y; (x \boxplus \delta) \boxminus x = \delta$$

Axiom 1 — the zero increment does nothing. $x \boxplus 0 = x \cdot \exp(0) = x \cdot I = x$.

Axiom 2 — $\boxminus$ finds the increment that $\boxplus$ needs.

$$x \boxplus (y \boxminus x) = x \cdot \exp(\log(x^{-1}y)) = x \cdot x^{-1}y = y,$$

valid wherever $\exp$ and $\log$ are mutual inverses, which by Step 5 of the SE(2) derivation means $|\omega| < \pi$ for the relative rotation between x and y .

Axiom 3 — $\boxminus$ recovers the increment.

$$(x \boxplus \delta) \boxminus x = \log(x^{-1} x \exp(\delta^\wedge))^\vee = \delta,$$

again for $|\delta_\omega| < \pi$. Together with smoothness of $\exp$ and $\log$, these are exactly Hertzberg et al.’s axioms for a *manifold encapsulation*: they say that near any x , the map $\delta \mapsto x \boxplus \delta$ is a well-behaved chart, so an algorithm written in δ is doing honest calculus.

Which side? $x \exp(\delta^\wedge)$ is the *right* (local) convention: the increment is applied in the body frame. $\exp(\delta^\wedge)x$ is the *left* (global) convention, applied in the world frame. They differ by the adjoint, $\delta_{\text{left}} = \text{Ad}_x \delta_{\text{right}}$ — which is what the two ghost frames in the Frame Composer are showing. This book fixes the right convention everywhere, for a physical reason: a robot’s odometry, its wheel encoders, and its IMU all report body-frame quantities, so right increments are the ones the hardware actually produces. Chapter 7 turns the choice into an error-state EKF; the invariant-EKF literature makes the opposite choice deliberately and gets different, sometimes better, convergence guarantees.

Vectors are manifolds too, with $\boxplus = +$ and $\boxminus = -$. That is not a technicality: it means one generic filter implementation covers a Euclidean state, a pose, and a pose-plus-bias state, with the compiler picking the right operators.

3.4 The algorithms

Geometry has no Thrun table names, so these are the book’s Rust API, stated once and never re-derived.

ALGORITHM $\text{se2_exp}(\tau) \rightarrow T$ COST $O(1)$ -- two transcendental calls
in $\tau = (v_x, v_y, \omega)^\top \in \mathbb{R}^3$, tangent coordinates
out $T \in \text{SE}(2)$

```

1: if  $|\omega| < \varepsilon$  then  $a \leftarrow 1 - \omega^2/6$ ,  $b \leftarrow \omega/2 - \omega^3/24$ 
2: else  $a \leftarrow \sin \omega / \omega$ ,  $b \leftarrow 2 \sin^2(\omega/2) / \omega$ 
3: endif
4:  $R \leftarrow \text{Rot}(\omega)$ 
5:  $t \leftarrow (av_x - bv_y, bv_x + av_y)^\top$  // this is  $V(\omega)\rho$ 
6: return  $(R, t)$ 

```

ALGORITHM $\text{se2_log}(T) \rightarrow \tau$ COST $O(1)$; exact inverse of se2_exp on $|\omega| < \pi$
in $T = (R, t) \in \text{SE}(2)$
out $\tau \in \mathbb{R}^3$, with $|\omega| < \pi$

```

1:  $\omega \leftarrow \text{atan2}(R_{21}, R_{11})$  // already wrapped to  $(-\pi, \pi]$ 
2:  $h \leftarrow \omega/2$ 
3: if  $|\omega| < \varepsilon$  then  $c \leftarrow 1 - \omega^2/12$  else  $c \leftarrow h \cot h$  endif
4:  $\rho \leftarrow (c t_x + h t_y, -h t_x + c t_y)^\top$  // this is  $V(\omega)^{-1}t$ 
5: return  $(\rho, \omega)$ 

```

Everything else is one line on top of these: $\text{compose}(a, b) = ab$, $\text{inverse}(T) = (R^\top, -R^\top t)$, $x \boxplus \delta = x \cdot \text{se2_exp}(\delta)$, and $y \boxminus x = \text{se2_log}(x^{-1}y)$.

3.5 Uncertainty on a manifold

A short section now, because it is the bridge to Parts II and III; the full treatment is Chapter 9.

If a pose is not a vector, a *distribution over poses* is not the Gaussian on $\mathbb{R}^3$ that Chapter 2 built. Writing $\mathcal{N}((x, y, \theta); \mu, \Sigma)$ is already wrong: it assigns different densities to the same rotation depending on which branch of θ you wrote down. The standard repair is a **concentrated Gaussian**: put an ordinary Gaussian in the tangent space, where it *is* at home, and push it onto the group through the retraction,

$$x = \bar{x} \boxplus \delta, \quad \delta \sim \mathcal{N}(0, \Sigma), \quad \Sigma \in \mathbb{R}^{3 \times 3}.$$

The mean $\bar{x}$ lives on the manifold, the covariance lives in the tangent space at

$\bar{x}$, and every operation a filter performs happens in that tangent space. What the construction produces in (x, y) coordinates is not an ellipse.

INTERACTIVE · w3.4

Banana Preview

Pose uncertainty is not an ellipse. A Gaussian in the tangent space becomes a bent, skewed cloud after $\exp$ — and its mean is not where you aimed.

Run this simulation in the web edition: </chapters/ch03-geometry-of-motion#w3.4>

Two consequences, both of which get proper treatment later. First, $\mathbb{E}[\exp(\tau)] \neq \exp(\mathbb{E}[\tau])$: the sample mean of the cloud is not the commanded endpoint, and the gap grows with σ_ω and with distance travelled. Second, an ellipse fitted to the cloud claims mass in places the robot cannot be. A Gaussian filter tracking a pose is making a bet that the banana is thin enough to pass for an ellipse over one time step — which is usually true at 60 Hz and disastrously false over a ten-metre dead-reckoned stretch. Chapter 7 names that bet and prices it.

3.6 Implementation in Rust

The type is small enough to read in one sitting and central enough that it is worth reading carefully. Two design decisions are load-bearing. The rotation is stored as a `UnitComplex`, not as an `f64` angle, so no arithmetic can ever produce an unwrapped heading. And the tangent type is a fixed-size `SVector<f64, 3>`, so a dimension mismatch is a compile error rather than a panic.

RUST `crates/pr-core/src/geom/se2.rs`

```
1 use nalgebra::{Point2, SMatrix, SVector, UnitComplex, Vector2};
2
3 /// Tangent coordinates tau = (v?, v_y, omega)^T -- **translation first** (Sol? order).
4 ///
5 /// Lynch & Park write (omega, v). We do not, because the reader already holds the
6 /// pose tuple (x, y, theta) and because `factrs` and `sophus` order it this way.
7 pub type Tangent2 = SVector<f64, 3>;
8
9 /// Below this |omega| the closed forms are 0/0 and we switch to Taylor series.
10 const SMALL_ANGLE: f64 = 1e-8;
11
12 /// A planar rigid-body transform: the book's most-used type.
13 ///
14 /// Storing the rotation as a `UnitComplex` rather than an angle is not a
15 /// micro-optimization -- it makes an unwrapped heading *unrepresentable*.
16 #[derive(Clone, Copy, Debug, PartialEq)]
17 pub struct SE2 {
18     pub rot: UnitComplex<f64>,
19     pub trans: Vector2<f64>,
20 }
21
22 impl SE2 {
```

```

23 pub fn identity() -> Self {
24     Self { rot: UnitComplex::identity(), trans: Vector2::zeros() }
25 }
26
27 pub fn new(x: f64, y: f64, theta: f64) -> Self {
28     Self { rot: UnitComplex::new(theta), trans: Vector2::new(x, y) }
29 }
30
31 /// exp: ??(2) -> SE(2). Derivation 4, with the guard from its Step 6.
32 pub fn exp(tau: &Tangent2) -> Self {
33     let (vx, vy, w) = (tau[0], tau[1], tau[2]);
34     let (a, b) = if w.abs() < SMALL_ANGLE {
35         (1.0 - w * w / 6.0, w / 2.0 - w * w * w / 24.0)
36     } else {
37         // (1 - cos omega) written as 2 sin2(omega/2): the literal form loses about
38         // eight digits to cancellation at the omega a 60 Hz loop produces.
39         let sh = (0.5 * w).sin();
40         (w.sin() / w, 2.0 * sh * sh / w)
41     };
42     Self {
43         rot: UnitComplex::new(w),
44         trans: Vector2::new(a * vx - b * vy, b * vx + a * vy),
45     }
46 }
47
48 /// log: SE(2) -> ??(2). Exact inverse of `exp` for |omega| < pi.
49 pub fn log(&self) -> Tangent2 {
50     let w = self.rot.angle(); // UnitComplex::angle is already in (-pi, pi]
51     let h = 0.5 * w;
52     // c = (omega/2).cot(omega/2), the cancellation-free form of omega sin omega /
53     2(1-cos omega).
54     let c = if w.abs() < SMALL_ANGLE { 1.0 - w * w / 12.0 } else { h / h.tan() };
55     let (x, y) = (self.trans.x, self.trans.y);
56     Tangent2::new(c * x + h * y, -h * x + c * y, w)
57 }
58
59 /// (R^T, -R^Tt). Note that the translation is rotated *before* it is negated.
60 pub fn inverse(&self) -> Self {
61     let r_inv = self.rot.inverse();
62     Self { rot: r_inv, trans: -(r_inv * self.trans) }
63 }
64
65 /// Ad_T, satisfying T.exp(tau).T-1 = exp(Ad_T tau). Derivation 6.
66 pub fn adjoint(&self) -> SMatrix<f64, 3, 3> {
67     let r = self.rot.to_rotation_matrix();
68     let m = r.matrix();
69     SMatrix::<f64, 3, 3>::new(
70         m[(0, 0)], m[(0, 1)], self.trans.y,
71         m[(1, 0)], m[(1, 1)], -self.trans.x,
72         0.0, 0.0, 1.0,
73     )
74 }
75
76 /// Map a point from this frame's coordinates into the parent frame.
77 pub fn act(&self, p: &Point2<f64>) -> Point2<f64> {
78     self.rot.transform_point(p) + self.trans
79 }
80
81 /// Display only. Never do arithmetic on the tuple -- that is what [+] is for.
82 pub fn xytheta(&self) -> (f64, f64, f64) {
83     (self.trans.x, self.trans.y, self.rot.angle())

```

```

83     }
84 }
85
86 impl std::ops::Mul for SE2 {
87     type Output = SE2;
88     fn mul(self, rhs: SE2) -> SE2 {
89         SE2 { rot: self.rot * rhs.rot, trans: self.trans + self.rot * rhs.trans }
90     }
91 }

```

The retraction goes in its own trait, because that is the interface every later estimator is generic over. D is the tangent dimension, and it is a `const` generic so that the compiler knows the size of every Jacobian in Chapter 7.

RUST `crates/pr-core/src/geom/manifold.rs`

```

1  use nalgebra::SVector;
2  use crate::geom::se2::{Tangent2, SE2};
3
4  /// The interface the book's filters are written against.
5  ///
6  /// `x [+] delta` moves `x` by a tangent-space increment; `y [-] x` recovers the
7  /// increment that would do it. Implementations must satisfy the Hertzberg
8  /// axioms, which `tests/geom.rs` checks by property test rather than by trust.
9  pub trait Manifold<const D: usize>: Copy {
10     fn boxplus(&self, delta: &SVector<f64, D>) -> Self;
11     fn boxminus(&self, other: &Self) -> SVector<f64, D>;
12 }
13
14 impl Manifold<3> for SE2 {
15     /// Right (local) convention: the increment is applied in the body frame.
16     fn boxplus(&self, delta: &Tangent2) -> Self {
17         *self * SE2::exp(delta)
18     }
19     fn boxminus(&self, other: &Self) -> Tangent2 {
20         (other.inverse() * *self).log()
21     }
22 }
23
24 /// Vectors are manifolds too -- flat ones. This impl is why one generic EKF
25 /// serves both a Euclidean state and a pose without a line of special-casing.
26 impl<const N: usize> Manifold<N> for SVector<f64, N> {
27     fn boxplus(&self, delta: &SVector<f64, N>) -> Self { self + delta }
28     fn boxminus(&self, other: &Self) -> SVector<f64, N> { self - other }
29 }

```

3.6.1 Craig's notation, enforced by rustc

Craig invented a type system in 1986 and wrote it in superscripts. Rust can check it. The markers are zero-sized, `PhantomData` occupies nothing, and the whole thing compiles down to the same machine code as bare `SE2`.

RUST `crates/pr-core/src/geom/frames.rs`

```

1 use core::marker::PhantomData;
2 use core::ops::Mul;
3 use crate::geom::se2::SE2;
4
5 pub trait Frame: 'static {}
6
7 pub struct World;
8 pub struct Body;
9 pub struct LidarF;
10 impl Frame for World {}
11 impl Frame for Body {}
12 impl Frame for LidarF {}
13
14 /// `Pose<A, B>` is Craig's ?_BT: "A from B". Zero runtime cost, and the index
15 /// bookkeeping becomes the compiler's problem instead of yours.
16 #[derive(Clone, Copy, Debug)]
17 pub struct Pose<A: Frame, B: Frame>(pub SE2, PhantomData<fn() -> (A, B)>);
18
19 impl<A: Frame, B: Frame> Pose<A, B> {
20     pub fn new(t: SE2) -> Self { Self(t, PhantomData) }
21     pub fn inverse(self) -> Pose<B, A> { Pose::new(self.0.inverse()) }
22 }
23
24 /// The cancellation rule, as a trait impl: the inner frames must agree, and
25 /// they vanish from the output type.
26 impl<A: Frame, B: Frame, C: Frame> Mul<Pose<B, C>> for Pose<A, B> {
27     type Output = Pose<A, C>;
28     fn mul(self, rhs: Pose<B, C>) -> Pose<A, C> { Pose::new(self.0 * rhs.0) }
29 }

```

Now the payoff. This is a deliberate compile error, and it is a feature of the chapter:

RUST demos/ch03-geometry/examples/frame_safety.rs

```

1 let world_from_body: Pose<World, Body> = Pose::new(SE2::new(2.0, 1.0, FRAC_PI_2));
2 let world_from_lidar: Pose<World, LidarF> = Pose::new(SE2::new(2.0, 1.3, FRAC_PI_2));
3
4 // Wrong: the inner indices are W and W. They do not cancel.
5 let oops = world_from_body * world_from_lidar;

```

```

error[E0308]: mismatched types
--> demos/ch03-geometry/examples/frame_safety.rs:6:31
   |
6 |     let oops = world_from_body * world_from_lidar;
   |                                ^^^^^^^^^^^^^^^^^ expected `Pose<Body, _>`,
   |                                                    found `Pose<World, LidarF>`

```

What you meant was `world_from_body.inverse() * world_from_lidar`, whose type is `Pose`

--- the sensor extrinsics. The convention for the rest of the book: bare SE2 for state inside an estimator, where everything is in one frame by construction, and `Pose<_, _>` at subsystem boundaries — extrinsics, map anchoring, anything crossing a module line.

3.7 Putting it together: the square dance

Here is the numeric example to check by hand. It is the smallest computation that exercises the whole chapter, and a test pins it.

Drive a quarter circle. Command $\tau = (1, 0, \frac{\pi}{2})^T$: one metre of arc while turning ninety degrees. From Derivation 4,

$$V\left(\frac{\pi}{2}\right) = \frac{2}{\pi} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad V\left(\frac{\pi}{2}\right) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{2}{\pi} \begin{pmatrix} 1 \\ 1 \end{pmatrix},$$

because $\sin \frac{\pi}{2} = 1$ and $1 - \cos \frac{\pi}{2} = 1$, and $1/\omega = 2/\pi$. So

$$\exp(\tau^\wedge) = \left(\frac{2}{\pi}, \frac{2}{\pi}, \frac{\pi}{2}\right) \approx (0.636620, 0.636620, 1.570796).$$

Sanity check the geometry: the turn radius is $|v|/\omega = 2/\pi$, the center of the arc is at $(0, 2/\pi)$, and rotating the start point 90° about that center lands exactly on $(2/\pi, 2/\pi)$.

Go back. log of that pose returns $\omega = \pi/2$, so $h = \pi/4$ and $c = \frac{\pi}{4} \cot \frac{\pi}{4} = \frac{\pi}{4}$ — the two entries of V^{-1} are equal, and

$$\rho = \frac{\pi}{4} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \frac{2}{\pi} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}. \checkmark$$

Compare with Euler integration. The classical update $x += \Delta s \cos \theta$, $y += \Delta s \sin \theta$, $\theta += \Delta \theta$ applied in K equal substeps gives

K	endpoint	error vs. exp
1	(1.000000, 0.000000)	0.7330 m
4	(0.753417, 0.503417)	0.1772 m
16	—	≈ 0.044 m

The error falls like $1/K$, which is first-order convergence, which is what "Euler" means. The exponential is exact at $K = 1$, because it is not an approximation of the arc — it *is* the arc.

Close a square. Alternate $\exp((1, 0, 0))$ and $\exp((0, 0, \pi/2))$ four times. Rusty returns to the origin, and the closure error measured on the manifold, $\|\text{final} \ominus \text{SE2} :: \text{identity}()\|$, is at the level of floating-point round-off — around 10^{-16} , not 10^{-3} . That is the difference between arithmetic that respects the group and arithmetic that approximates it.

RUST `crates/pr-core/tests/geom.rs`

```

1 use approx::assert_relative_eq;
2 use nalgebra::Vector3;
3 use proptest::prelude::*;
4 use rand::{rngs::SmallRng, Rng, SeedableRng};
5 use std::f64::consts::{FRAC_PI_2, PI};
6
7 use pr_core::geom::{manifold::Manifold, se2::{Tangent2, SE2}};
8
9 /// The chapter's worked example, pinned to 1e-12.
10 #[test]
11 fn worked_example_ch03_quarter_arc() {
12     let tau = Tangent2::new(1.0, 0.0, FRAC_PI_2);
13     let t = SE2::exp(&tau);
14     let (x, y, theta) = t.xytheta();
15
16     assert_relative_eq!(x, 2.0 / PI, epsilon = 1e-12); // 0.636619772...
17     assert_relative_eq!(y, 2.0 / PI, epsilon = 1e-12);
18     assert_relative_eq!(theta, FRAC_PI_2, epsilon = 1e-12);
19
20     // ...and log takes us straight back.
21     assert_relative_eq!(t.log(), tau, epsilon = 1e-12);
22
23     // One Euler step is off by 73 centimetres on a one-metre arc.
24     let euler = (1.0 - x).hypot(0.0 - y);
25     assert_relative_eq!(euler, 0.733028, epsilon = 1e-6);
26 }
27
28 /// Four sides and four right-angle turns must return exactly to the start.
29 #[test]
30 fn square_dance_closes() {
31     let forward = SE2::exp(&Tangent2::new(1.0, 0.0, 0.0));
32     let turn = SE2::exp(&Tangent2::new(0.0, 0.0, FRAC_PI_2));
33
34     let mut pose = SE2::identity();
35     for _ in 0..4 {
36         pose = pose * forward * turn;
37     }
38
39     // Measured on the manifold, not on the tuple: ||final [-] identity||.
40     let closure = pose.boxminus(&SE2::identity()).norm();
41     assert!(closure < 1e-14, "closure error {closure:e} -- the group is not closing");
42 }
43
44 fn twist() -> impl Strategy<Value = Tangent2> {
45     // |omega| < pi keeps us inside log's injectivity radius, per Derivation 4 Step 5.
46     (-5.0f64..5.0, -5.0f64..5.0, -3.14f64..3.14)
47     .prop_map(|(a, b, c)| Tangent2::new(a, b, c))
48 }
49
50 proptest! {
51     #[test]
52     fn exp_log_roundtrip(tau in twist()) {
53         prop_assert!((SE2::exp(&tau).log() - tau).norm() < 1e-12);
54     }
55
56     #[test]
57     fn group_axioms(a in twist(), b in twist(), c in twist()) {
58         let (ta, tb, tc) = (SE2::exp(&a), SE2::exp(&b), SE2::exp(&c));
59         let lhs = (ta * tb) * tc;
60         let rhs = ta * (tb * tc);
61         prop_assert!(lhs.boxminus(&rhs).norm() < 1e-12); // associativity

```

```

62     prop_assert!((ta * ta.inverse()).log().norm() < 1e-12); // inverse
63 }
64
65 /// The authority on the adjoint's sign convention. Not memory -- this.
66 #[test]
67 fn adjoint_conjugation(t in twist(), tau in twist()) {
68     let big_t = SE2::exp(&t);
69     let lhs = big_t * SE2::exp(&tau) * big_t.inverse();
70     let rhs = SE2::exp(&(big_t.adjoint() * tau));
71     prop_assert!(lhs.boxminus(&rhs).norm() < 1e-10);
72 }
73
74 #[test]
75 fn boxplus_axioms(x in twist(), d in twist()) {
76     let big_x = SE2::exp(&x);
77     prop_assert!(big_x.boxplus(&Tangent2::zeros()).boxminus(&big_x).norm() < 1e-14);
78     let y = big_x.boxplus(&d);
79     prop_assert!((y.boxminus(&big_x) - d).norm() < 1e-12);
80     prop_assert!(big_x.boxplus(&y.boxminus(&big_x)).boxminus(&y).norm() < 1e-12);
81 }
82 }
83
84 /// log stays accurate right up to the injectivity boundary -- the place where
85 /// a naive implementation quietly returns the wrong branch.
86 #[test]
87 fn log_is_stable_near_pi() {
88     let mut rng = SmallRng::seed_from_u64(0xC0FFEE); // never thread_rng() in this book
89     for _ in 0..1000 {
90         let theta = PI - rng.random_range(1e-9..1e-6);
91         let ours = SE2::new(0.3, -0.2, theta).log();
92         assert_relative_eq!(ours[2], theta, epsilon = 1e-12);
93         assert_relative_eq!(SE2::exp(&ours).log(), ours, epsilon = 1e-9);
94     }
95 }

```

NOTE

Every widget on this page runs the TypeScript port of the code above — `web/lib/geom/se2.ts` and `web/lib/geom/screw.ts` — and both implementations are checked against this same quarter-arc example. If the prose and the code ever disagree, the test settles it.

The three-dimensional case is delegated, not skipped. `sophus` provides $SO(3)$ and $SE(3)$ with the same `exp/log/adjoint` surface, `nalgebra`'s `UnitQuaternion` handles the parameterization, and Chapter 18 uses both. The reason to hand-roll $SE(2)$ and only $SE(2)$ is that every structure in the general case is visible here, in two dimensions, where you can check the answer with a protractor.

3.8 Exercises

1 FOUNDATION •• Derive V and its inverse

Evaluate $V(\omega) = \int_0^1 R(s\omega) ds$ entrywise to reproduce Derivation 4's closed form, then derive $V(\omega)^{-1}$ by inverting the 2×2 matrix directly. Show that both tend to I as $\omega \rightarrow 0$, and compute the first two nonzero terms of the Taylor series of each entry — these are the coefficients the code branches to.

2 FOUNDATION •• The adjoint is a homomorphism

Prove the SE(2) adjoint formula of Derivation 6 by block computation, then show that $\text{Ad}_{T_1 T_2} = \text{Ad}_{T_1} \text{Ad}_{T_2}$ and $\text{Ad}_{T^{-1}} = (\text{Ad}_T)^{-1}$. Explain why the first identity says "changing frames twice is one change of frame", and why that is exactly what makes covariance propagation across a kinematic chain composable.

3 FOUNDATION ••• Right increments are body-frame quantities

Let $x' = x \boxplus \delta$ with $\delta = (\rho, \omega_\delta)$. Show that the world-frame displacement of the origin is $R_x V(\omega_\delta) \rho$, and hence $R_x \rho$ to first order. Conclude why wheel odometry naturally produces right increments, and derive the conversion $\delta_{\text{left}} = \text{Ad}_x \delta_{\text{right}}$ to the left convention.

4 CONCEPTUAL •• Predict, then verify with the Exp/Log Lens

Two poses differ by a pure 180° rotation with no translation. Predict, before touching the widget: where does the geodesic interpolation put the robot at $s = 0.5$, and why does log fail to pick a unique answer for this pair? Now drag the arrowhead in w3.2 to the origin so $v \approx 0$, and slide ω toward ± 3.1 . The slider deliberately stops short of π — say why, in terms of Derivation 4, Step 5, and describe what happens to the arc and to the ICR as you approach the limit from either side.

5 CONCEPTUAL • Squeeze the banana

In w3.4, predict what happens to the cloud as $\sigma_\omega \rightarrow 0$ with σ_v fixed, and what happens to the 2σ coverage readout. Verify. Then state, in one sentence, the condition under which the fitted ellipse is *approximately* honest — you have just described the bet the EKF makes in Chapter 7.

6 PRACTICAL •• Implement SE2 and pass the suite

Implement SE2 with exp, log, inverse, adjoint, and Mul, plus the `Manifold<3>` impl, so that every test in `crates/pr-core/tests/geom.rs` above passes. Then deliberately break the small-angle guard by deleting the Taylor branch, and find the value of $|\omega|$

below which `exp_log_roundtrip` starts failing. Report it, and name the subtraction that lost the digits.

7 PRACTICAL ••• SO3 under the same trait

Implement `SO3` as a newtype over `nalgebra::UnitQuaternion<f64>` with `exp`, `log`, and `Manifold<3>`. Handle $\theta \approx \pi$ with the numerically stable branch (the naive `log` divides by $\sin(\theta/2)$, which vanishes at $\theta = 0$, and loses precision as $\theta \rightarrow \pi$ from the other side). Property-test the round trip against `sophus` at a thousand random rotations including the near- π band, and plot the round-trip error against θ .

8 PRACTICAL •• Make the frame bug uncompileable

Take a working piece of code that composes transforms with bare `SE2` — the widget’s lidar chain is a fine target — and port it to `Pose`

. Introduce a realistic frame bug (compose the extrinsics in the wrong order) and show the compiler catching it. Then measure: does the newtype cost anything? Compare the generated assembly for `Pose`

- `Pose`

with plain `SE2 * SE2'`.

3.9 References

Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control* Cambridge University Press. ISBN 978-1107156302.

Chapter 3 is this chapter’s computational spine: planar rigid motions, angular velocity, exponential coordinates, twists, and screws. The epigraph is from its opening pages. Note the tangent ordering differs from ours — they write (ω, v) . [link](#)

Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control* Pearson, 3rd edition. ISBN 978-0201543612.

The source of the leading super/subscript notation and the cancellation rule. Sections 2.2–2.7 are the clearest treatment of frames as a discipline anywhere; the Rust newtypes in this chapter are that discipline, mechanized. [link](#)

Spong, M. W., Hutchinson, S., and Vidyasagar, M. (2020). *Robot Modeling and Control* Wiley, 2nd edition. ISBN 978-1119523994.

Section 2.4 states the fixed-frame versus current-frame composition rule — pre- versus post-multiplication — which is exactly the two ghost frames in the Frame Composer, and the reason this book fixes the right convention. [link](#)

Hertzberg, C., Wagner, R., Frese, U., and Schröder, L. (2013). Integrating generic sensor fusion algorithms with sound state representations through encapsulation of manifolds *Information Fusion* 14(1), 57–77.

Where $\boxplus$ and $\boxminus$ come from, with the axioms proved in Derivation 7. The paper’s thesis is this chapter’s thesis: encapsulate the manifold and every estimator downstream can be written as if the state were a vector. doi:10.1016/j.inffus.2011.08.003

Solà, J., Deray, J., and Atchuthan, D. (2018). A micro Lie theory for state estimation in robotics *arXiv:1812.01537*.

The modernization spine of this chapter: the $\boxplus/\boxminus$ interface, the adjoint, and the Jacobian bookkeeping that Part II needs. This book follows its translation-first tangent ordering and its right- $\boxplus$ convention. [link](#)

Mangelson, J. G., Ghaffari, M., Vasudevan, R., and Eustice, R. M. (2020). Characterizing the Uncertainty of Jointly Distributed Poses in the Lie Algebra *IEEE Transactions on Robotics* 36(5), 1371–1388.

The banana, taken seriously: what a concentrated Gaussian on SE(2)/SE(3) actually looks like, and why the independence assumption between poses in a graph is usually false. Chapter 9 builds on its compounding operations. doi:10.1109/TR0.2020.2994457

Potokar, E. R., Beard, R. W., and Mangelson, J. G. (2024). An Introduction to the Invariant Extended Kalman Filter [Lecture Notes] *IEEE Control Systems* 44(6).

The most readable recent account of why the choice between left and right $\boxplus$ has consequences for convergence, not just for bookkeeping. Read it after Chapter 7. doi:10.1109/MCS.2024.3466488

Ge, Y., van Goor, P., and Mahony, R. (2025). The Geometry of Extended Kalman Filters on Manifolds with Affine Connection *arXiv:2506.05728*.

Where the field is now: the retraction of this chapter is one choice among many, and connection, parallel transport, and curvature all show up in the filter equations once you look. The frontier version of Chapter 7. [link](#)

Rusty, Sensors, and the Simulator

FOUNDATIONS · Intermediate · 55 min

“

Since wheel-rotation sensing is available on all mobile robots, odometry is cheap and convenient. Estimation errors tend to accumulate over time, though, due to unexpected slipping and skidding of the wheels and to numerical integration error.

Kevin M. Lynch and Frank C. Park

Modern Robotics, §13.4 (2017)

Every chapter after this one measures an algorithm. This chapter builds the instrument.

Rusty is a differential-drive rover with two driven wheels, two quadrature encoders, and a planar LiDAR. It lives in two worlds: the **Hallway**, a corridor with three identical doors, and the **Apartment**, a twelve-by-nine-metre floorplan with five rooms off a corridor. By Chapter 26 Rusty will explore and map an apartment it has never seen. Today it can barely drive down a hallway without losing track of itself, and watching it fail is the point.

Two ideas carry the chapter. The first is pedagogical: **the simulator is the world; models are beliefs about the world**. What we write here is *generative* physics — what actually happens. Chapter 9 and Chapter 10 will fit *inference* models to it, and the two are deliberately not the same equations. Model mismatch is real from day one because we built it in on purpose. The second is engineering: **determinism is a feature**. A seed and a control script determine a trajectory exactly, in the browser and on your laptop, which is what makes every figure in this book reproducible and every benchmark honest.

🕒 AFTER THIS CHAPTER YOU CAN

- Derive the differential-drive kinematics that turn two wheel speeds into a body twist
- Show that exact arc integration is the SE(2) exponential map, and measure what approximating it costs
- Turn encoder ticks into a pose increment, and separate systematic error from stochastic slip
- State the simulator LiDAR forward model, and say honestly where it differs from the inference model of Chapter 10
- Build a seeded, deterministic lab whose runs replay bit-for-bit — and know why that matters

📖 ASSUMED

- SE(2), $\exp/\log$, and the $\boxplus / \boxminus$ operators (Chapter 3)
- Gaussians and seeded sampling discipline (Chapter 2)
- Basic Rust: structs, traits, and the borrow checker in anger

4.1 The problem with knowing how far you have gone

Rusty's wheels are its only sense of motion. Each encoder counts the fraction of a revolution its wheel has turned; multiply by the wheel radius and you get how far that wheel *rolled*. Integrate both, and you have a pose. This is **dead reckoning**, and it is the oldest navigation algorithm there is.

It also has a defect that no amount of engineering removes. An encoder measures the *wheel*. It does not measure the *floor*. Every time a wheel slips a millimetre, skids on a threshold, or turns out to be 0.3 mm larger than the CAD model says, the number the encoder reports and the distance the robot travelled part company — and because dead reckoning integrates, that disagreement is never corrected, only accumulated.

🖥️ INTERACTIVE · w4.1

Rusty's Dashboard

Odometry is not a position sensor. It is a velocity sensor, integrated hopefully.

Run this simulation in the web edition: `/chapters/ch04-rusty-and-sensors #w4.1`

Watch it for thirty seconds before reading on. Three things are worth noticing. **The error grows and never shrinks.** There is no mechanism in dead reckoning that could shrink it. A measurement of the outside world can pull an estimate

back toward the truth; a measurement of your own wheels cannot, because it carries no information about where you started.

Turns are more expensive than straights. A heading error is nearly free while you stand still and catastrophic once you drive under it: three degrees of heading error becomes half a metre of position error after ten metres of driving. This is why the "drift / distance" readout jumps at each end of the corridor.

With slip set to zero, the two traces coincide exactly. That is worth sitting with. It means the drift you just watched is not a property of dead reckoning as an *idea* — the arithmetic is exact — but of the physics the arithmetic is being fed. Which raises the obvious question: why can no real robot set that slider to zero? Two reasons, and they behave completely differently. Drag the second slider and watch what a one-percent error in one wheel radius does.

💡 THE ONE SENTENCE VERSION

Odometry is not a position sensor. It is a velocity sensor that has been integrated hopefully, and the hope compounds.

4.2 What Rusty is made of

Before the mathematics, the hardware — because every noise model in this book is a claim about a physical device, and it helps to know which one.

Sensors split along two axes that matter more than any device catalog. A **proprioceptive** sensor measures the robot's own internal state; an **exteroceptive** one measures the world outside it. An **active** sensor emits energy and listens for it; a **passive** one only receives.

encoders	Proprioceptive, passive. Optical quadrature counters on each wheel shaft. Return integer counts, from which we infer wheel angle. Cheap, high rate, and blind to the floor. Rusty has these
IMU	Proprioceptive, passive. Angular rate and specific force. Excellent over milliseconds, hopeless over minutes: its bias is itself a random walk. Named here, modelled in Ch. 18
sonar	Exteroceptive, active. Emits an ultrasonic pulse, times the echo. Returns one range for a whole 15–30° cone, so a wall corner smears into an arc. Exercise 5
LiDAR	Exteroceptive, active. Time-of-flight along a narrow beam, swept through a plane. Returns a vector of ranges, one per beam bearing. Rusty has this

camera	Exteroceptive, passive. Returns brightness, not geometry; range must be inferred from motion, stereo, or priors. Ch. 18
--------	--

The taxonomy earns its keep immediately. Proprioceptive sensors give you *increments* whose errors accumulate; exteroceptive sensors give you *absolute* constraints whose errors do not. That single distinction is why the Bayes filter of Chapter 5 has exactly two steps, and why one of them makes the belief worse.

4.2.1 Notation for this chapter

r, ℓ	Wheel radius and track width (the distance between the two wheel contact points). Rusty: 0.033 m, 0.16 m
ω_L, ω_R	Left and right wheel angular velocities, rad/s.
$u_t = (v, \omega)^\top$	Commanded body twist: forward speed and yaw rate.
$\Delta s_L, \Delta s_R$	Arc length each wheel rolled over one tick interval, metres.
N	Encoder resolution, counts per wheel revolution. Rusty: 4096
$\lambda = 2\pi r/N$	Metres of wheel travel per encoder count. 50.6 m
$\sigma_{\text{slip}}, \delta$	Stochastic per-wheel slip std-dev; systematic wheel-radius error.
z_t^k, φ_k	Range of LiDAR beam k and its bearing in the body frame.
z^{k*}	The true first-hit distance along beam k . The simulator knows it; the robot never does.
z_{max}, σ_r	Maximum range and range-noise std-dev. 8 m, 0.02 m
m	The map — here, the ground-truth set of wall segments.

4.3 The mathematics

4.3.1 From two wheels to one twist

Rusty is a rigid body in the plane, so its velocity is fully described by a body twist $u = (v, \omega)^\top$: forward speed and yaw rate. It has two actuators. Two numbers in, two numbers out, and the map between them is linear.

DERIVATION Wheels to twist, and back

$$v = \frac{r}{2}(\omega_R + \omega_L), \quad \omega = \frac{r}{\ell}(\omega_R - \omega_L)$$

Step 1 — each wheel's contact point speed. A wheel of radius r turning at ω_i with no slip lays down ground at $s_i = r \omega_i$ metres per second, directed along the robot's forward axis.

Step 2 — the rigid-body constraint along the axle. The two contact points sit on the same rigid chassis, at $\pm \ell/2$ from the centre along the body y -axis. For a planar rigid body rotating at ω about its centre, a point offset by d along y moves forward at $v - \omega d$. Hence

$$s_L = v - \omega \frac{\ell}{2}, \quad s_R = v + \omega \frac{\ell}{2}.$$

Step 3 — solve the two linear equations. Adding gives $s_L + s_R = 2v$; subtracting gives $s_R - s_L = \omega \ell$. Substituting $s_i = r \omega_i$:

$$v = \frac{r}{2} (\omega_R + \omega_L), \quad \omega = \frac{r}{\ell} (\omega_R - \omega_L).$$

Step 4 — sanity limits. Equal wheel speeds give $\omega = 0$: a straight line. Opposite and equal give $v = 0$: a spin in place, radius zero. One wheel locked gives a turn about that wheel, radius $\ell/2$. All three match what you would expect from a shopping trolley.

The inverse, which is what the arrow keys in the widget above actually go through:

$$\omega_L = \frac{v - \omega \ell/2}{r}, \quad \omega_R = \frac{v + \omega \ell/2}{r},$$

and the **instantaneous centre of rotation** sits on the wheel axle at signed distance $R = v/\omega$ from the robot centre. As $\omega \rightarrow 0$, $R \rightarrow \infty$ and the arc straightens.

Notice what is *missing*: there is no third equation, because there is no third actuator. The lateral body velocity is structurally zero. Written as a constraint on the world-frame velocity,

$$\dot{x} \sin \theta - \dot{y} \cos \theta = 0,$$

which is a **Pfaffian constraint**: linear in the velocities, and not integrable to a constraint on position. Rusty can reach any pose in the plane; it simply cannot move sideways to get there. That is the entire reason parallel parking is a *planning* problem, taken up in Chapter 20.

4.3.2 Integration: the exact arc is the exponential map

Hold (v, ω) constant for Δt . What is the new pose?

The textbook answer is to integrate $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \omega$ numerically — Euler, or Runge–Kutta if you are feeling careful. This is a mistake, and an instructive one: the problem has a closed-form solution that is *shorter* than the

approximation, and Chapter 3 already handed it to us.

$$x_{t+1} = x_t \boxplus (v \Delta t, 0, \omega \Delta t)^\top = x_t \circ \exp \left[(v \Delta t, 0, \omega \Delta t)^\top \right]$$

No small-angle approximation, no integration error, no step-size parameter. The middle zero is the nonholonomic constraint, expressed in the Lie algebra $\mathfrak{se}(2)$.

DERIVATION Constant twist integrates to exp, and what the chord looks like

$\Delta p = \Delta s \operatorname{sinc}(\Delta\theta/2)$ at bearing $\Delta\theta/2$

Step 1 — constant twist means a screw ODE. The body-frame velocity is the constant tangent vector $\tau = (v, 0, \omega)^\top$. In matrix form the kinematics are $\dot{T} = T \hat{\tau}$ with $T \in \text{SE}(2)$ and $\hat{\tau}$ the corresponding element of $\mathfrak{se}(2)$. This is a linear ODE with constant coefficients, and its solution is the matrix exponential:

$$T(\Delta t) = T(0) \exp(\hat{\tau} \Delta t).$$

That is the whole derivation. Everything below is unpacking what $\exp$ evaluates to.

Step 2 — evaluate the exponential. From Chapter 3, $\exp(\rho, \omega \Delta t)$ has rotation part $\text{Rot}(\omega \Delta t)$ and translation part $\mathbf{V}(\omega \Delta t) \rho$, where $\mathbf{V}$ is the 2×2 block acting on $\rho = (\rho_1, \rho_2)^\top$ as

$$\mathbf{V}(\vartheta) \rho = \frac{1}{\vartheta} \begin{pmatrix} \rho_1 \sin \vartheta - \rho_2 (1 - \cos \vartheta), & \rho_1 (1 - \cos \vartheta) + \rho_2 \sin \vartheta \end{pmatrix}^\top.$$

Geometrically $\mathbf{V}$ converts "how far I drove along the arc" into "where I ended up". With $\rho = (v \Delta t, 0)^\top$ and writing $\Delta s = v \Delta t$, $\Delta\theta = \omega \Delta t$, the second component of ρ vanishes and we are left with:

$$\Delta x = \Delta s \frac{\sin \Delta\theta}{\Delta\theta}, \quad \Delta y = \Delta s \frac{1 - \cos \Delta\theta}{\Delta\theta}.$$

Step 3 — recognize the arc, in polar form. Use $\sin \vartheta = 2 \sin \frac{\vartheta}{2} \cos \frac{\vartheta}{2}$ and $1 - \cos \vartheta = 2 \sin^2 \frac{\vartheta}{2}$. Both components acquire the common factor $2 \sin(\Delta\theta/2)/\Delta\theta$, leaving $(\cos \frac{\Delta\theta}{2}, \sin \frac{\Delta\theta}{2})$:

$$\Delta p = \underbrace{\Delta s \frac{\sin(\Delta\theta/2)}{\Delta\theta/2}}_{\text{chord length}} \cdot \underbrace{\left(\cos \frac{\Delta\theta}{2}, \sin \frac{\Delta\theta}{2}\right)^T}_{\text{unit vector at half the turn}}$$

This is worth reading twice. The displacement points along **exactly half** the heading change, and its length is the arc length times $\text{sinc}(\Delta\theta/2)$, writing $\text{sinc}(x) = \sin x/x$ — the ratio of a chord to its arc. Equivalently, the motion is an arc of radius $R = \Delta s/\Delta\theta = v/\omega$, which is the ICR radius from the previous derivation. The Lie group did not invent new geometry; it handed us the geometry with the trigonometry already done.

Step 4 — what the approximations cost. Two heuristics are common in robotics code:

- *Naïve Euler* — move Δs along the **starting** heading. Wrong direction by $\Delta\theta/2$, and wrong length by the chord factor.
- *Midpoint Euler* — move Δs along the **average** heading, $\theta + \Delta\theta/2$. By Step 3 the direction is now exactly right; the only error is that it travels the arc length instead of the chord, overshooting by the factor

$$\frac{1}{\text{sinc}(\Delta\theta/2)} = 1 + \frac{\Delta\theta^2}{24} + O(\Delta\theta^4).$$

So the midpoint rule is not an approximation of the direction at all — it is exactly right there — and its distance error is purely radial, of size $\Delta s \Delta\theta^2/24$. This is why it works so well at 100 Hz and so badly on a log resampled to 2 Hz.

4.3.3 Odometry: from integer counts to a pose increment

Now the sensing side. An encoder does not report an angle; it reports an integer. Over one interval each wheel accumulates Δticks_i counts, and each count corresponds to $\lambda = 2\pi r/N$ metres of wheel travel.

$$\Delta s_i = \lambda \Delta\text{ticks}_i, \quad \Delta s = \frac{1}{2}(\Delta s_L + \Delta s_R), \quad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{\ell},$$

and the dead-reckoned pose advances by

$$\hat{x}_{t+1} = \hat{x}_t \boxplus (\Delta s, 0, \Delta\theta)^T.$$

That is the same $\boxplus$, with the same $\exp$, as the ground-truth integrator. **This is the key structural fact of the chapter:** dead reckoning and the simulator's physics run *identical arithmetic*. If the encoders saw exactly what the floor did, the orange trace would sit on the gray one forever. Everything you watched drift apart in the widget above came from the gap between the wheel and the floor — never from the mathematics.

DERIVATION The odometry error budget: three sources, three behaviours

quantization $\sim 25 \text{ m} \cdot \text{slip} \sim \text{random walk} \cdot \text{radius error} \sim \text{linear in distance}$

Three things separate $\hat{x}_t$ from x_t , and confusing them is the most common practical mistake in wheeled-robot state estimation.

1. Quantization. Rounding a wheel angle to the nearest count loses at most half a count, so the per-wheel distance error is bounded by

$$|\varepsilon_{\text{quant}}| \leq \frac{\lambda}{2} = \frac{\pi r}{N} = \frac{\pi(0.033)}{4096} = 2.53 \times 10^{-5} \text{ m.}$$

Bounded, zero-mean, and essentially white, so it accumulates as a random walk with a step of at most 25 m. Driving 42.6 m — nearly six lengths of the Apartment corridor — the simulator measures a total drift of 2.9×10^{-4} m from quantization alone. It is real, it is measurable, and it is not your problem.

2. Stochastic slip. Model the ground travel of wheel i as its rotation times $(1 + \epsilon_i)$ with $\epsilon_i \sim \mathcal{N}(0, \sigma_{\text{slip}}^2)$, drawn afresh each tick. This is zero-mean, so the *position* error is a random walk — but not a simple one. A slip difference between the wheels perturbs the **heading**, and heading error is integrated a second time by subsequent forward motion. The result: heading spread grows like $\sqrt{t}$, and position spread like $t^{3/2}$. The Seed Lab below measures the exponent, and it comes out at 1.4–1.5.

3. Systematic error. Suppose the right wheel's true effective radius is $r(1 + \delta)$ while the odometry code keeps dividing by the nominal r . Then *every* tick contributes a heading error of the same sign,

$$\Delta\theta_{\text{err}} \approx \frac{\delta \Delta s_R}{\ell},$$

so heading error grows **linearly** in distance travelled and position error faster still. This is Borenstein and Feng's "unequal wheel diameters", the dominant term in practice, and the reason their UMBmark calibration procedure exists.

The magnitudes are not close. Over one 30-second corridor patrol (14.3 m driven), the simulator reports:

source	mean drift	95th pct	mean heading error	varies with seed?
quantization only ($N = 4096$)	0.00011 m	—	0.009°	no
slip, $\sigma_{\text{slip}} = 0.02$	0.455 m	1.183 m	6.9°	yes
radius error, $\delta = 1\%$	2.678 m	2.678 m	49.3°	no

Five hundred seeds per row, at the simulator's 10 Hz tick rate. The slip row is the only one with any spread to report — which is itself the point.

One caveat, and it matters if you port these numbers: σ_{slip} is a *per-tick* parameter, so the drift it produces depends on the tick rate. A model meant to be rate-invariant would scale σ with $\sqrt{\Delta t}$, and Chapter 9's α -parametrization is written that way. This simulator is not, deliberately: it is one more place where the generative model and the inference model refuse to agree, and noticing the disagreement is part of the training.

Read the last column. A one-percent error in one wheel radius produces six times the drift of quite aggressive slip — and produces *exactly* the same number on every seed, because it is not noise. No filter removes it, no amount of averaging cancels it, and running your benchmark over a thousand seeds will not reveal it. You calibrate it, or you estimate it as part of the state (Chapter 14 does exactly this for the map; the same trick works for wheel radii).

4.3.4 A worked example you can check by hand

Rusty's track is $\ell = 0.16$ m. Over one interval the left wheel rolls $\Delta s_L = 0.72$ m and the right wheel rolls $\Delta s_R = 0.88$ m. Then

$$\Delta s = \frac{1}{2}(0.72 + 0.88) = 0.80 \text{ m}, \quad \Delta \theta = \frac{0.88 - 0.72}{0.16} = 1.00 \text{ rad},$$

an arc of radius $R = \Delta s / \Delta \theta = 0.80$ m through 57.3° . Apply the closed form from Derivation 2:

$$\Delta x = 0.8 \frac{\sin 1}{1} = 0.673177 \text{ m}, \quad \Delta y = 0.8 \frac{1 - \cos 1}{1} = 0.367758 \text{ m}.$$

In polar form the chord is $0.8 \times \text{sinc}(0.5) = 0.767081$ m at a bearing of exactly 0.5 rad. Now price the two shortcuts:

- **Midpoint Euler** puts you 0.8 m out at 0.5 rad — right bearing, 0.032919 m too far. The predicted overshoot $\Delta s \Delta \theta^2 / 24 = 0.8 / 24 = 0.0333$ m matches to three digits.
- **Naive Euler** puts you 0.8 m out at 0 rad, which is **0.389012 m** wrong — half the step length.

One interval. If your control loop resamples odometry at 2 Hz while turning at 1 rad/s, that is the error you are silently adding to every increment.

To make the encoder round trip concrete: $\lambda = 2\pi(0.033)/4096 = 5.0621 \times 10^{-5}$ m, so $\Delta s_L = 0.72$ m is 14223.24 counts, reported as 14223. Feeding the rounded counts back through `odometry_delta` gives $\Delta s = 0.79999476$ m and $\Delta \theta = 1.00008836$ rad — off by 5 m and 88 rad, which is the quantization bound doing exactly what the budget above says it should.

4.3.5 The LiDAR forward model

Rusty’s LiDAR is a spinning planar time-of-flight sensor: n beams over a 2π field of view, $z_{\max} = 8$ m. For beam k at body-frame bearing φ_k , the simulator computes the true first-hit distance by ray casting against the map,

$$z^{k*} = \text{raycast}(m, x_t \circ T_{\text{lidar}}, \varphi_k),$$

and then corrupts it in one of exactly two ways. With probability p_{drop} the beam returns nothing at all and the sensor reports its ceiling,

$$z_t^k = z_{\max},$$

and otherwise it reports the true range plus Gaussian noise, clipped to the sensor’s range:

$$z_t^k = \min\left(z^{k*} + \varepsilon, z_{\max}\right), \quad \varepsilon \sim \mathcal{N}(0, \sigma_r^2).$$

INTERACTIVE · w4.2

LiDAR Anatomy

A scan is not geometry. It is a vector of noisy numbers indexed by beam, and everything after this is inference.

Run this simulation in the web edition: </chapters/ch04-rusty-and-sensors/#w4.2>

Three observations, each of which becomes a chapter later on.

A scan is a vector of numbers, not a shape. The strip beneath the scene is the entire content of a measurement: 180 floats indexed by beam. The wall you

perceive in the top panel is something *you* inferred. Recovering it is what Chapter 16's scan matcher and Chapter 13's mapping do, and neither of them is free.

Corners are discontinuities in range space. Scrub the beam index past 78, and again past 103, and watch the range fall off a cliff — 2.474 m to 1.179 m — as the beam catches the door frame. In between, the beams fly through the doorway and land on the far corridor wall. Those cliffs are the only places a range scan carries information about *along-wall* position, which is precisely why a robot in a long featureless corridor knows its distance to each wall and almost nothing about how far down it has driven. That anisotropy is why Chapter 11 draws covariance *ellipses* rather than circles.

A dropout is the sensor's largest number and its smallest amount of information. Turn the dropout rate up. Every dropped beam reports $z_{\max}$, so a naive model that treats 8 m as evidence of an 8 m wall will happily localize the robot onto a pane of glass. Handling this properly is the $z_{\max}$ spike in Chapter 10's four-way mixture.

⚠ WHERE ROBOTS BREAK THIS

An honest disclosure about this model. It has *two* components — a hit and a max — while Chapter 10 fits an inference model with *four*: hit, short, max, and random. That is deliberate. This world contains no people, no chair legs, and no multipath, so a short-return term would be modelling something that does not exist. When Chapter 10 fits its four-way mixture to data generated here and recovers $z_{\text{short}} \approx 0$, that is not a failure of the fit; it is the fit telling you the truth about the world it was given. Then we turn on dynamic obstacles and watch the parameter move.

4.4 The algorithms

ALGORITHM `diff_drive_step(x_{t-1}, u_t, \Delta t)`

COST $O(1)$

in the true pose, the commanded twist (v, ω) , the tick length

out the new true pose and the accumulated wheel angles

- 1: $\omega_L = (v - \omega \ell / 2) / r, \quad \omega_R = (v + \omega \ell / 2) / r$
- 2: $\theta_i^{\text{wheel}} += \omega_i \Delta t$ (the encoders will see this)
- 3: sample $\epsilon_L, \epsilon_R \sim \mathcal{N}(0, \sigma_{\text{slip}}^2)$
- 4: $\Delta s_i = r_i^{\text{eff}} \omega_i \Delta t (1 + \epsilon_i)$ (the floor delivers this)
- 5: $\Delta s = \frac{1}{2}(\Delta s_R + \Delta s_L), \quad \Delta \theta = (\Delta s_R - \Delta s_L) / \ell$
- 6: $x_t = x_{t-1} \boxplus (\Delta s, 0, \Delta \theta)^T$

- 7: if the segment $x_{t-1} \rightarrow x_t$ crosses a wall, **keep** x_{t-1} and return blocked
- 8: return x_t

Line 7 is not a technicality. A blocked robot's wheels keep turning, so its encoders keep counting, so its dead-reckoned pose keeps advancing — straight through the wall. Hold the throttle against a wall in the widget above and watch the orange ghost leave the building.

ALGORITHM `odometry_delta(ticks_{t-1}, ticks_t, params)` COST $O(1)$

in two cumulative encoder readings and the robot geometry

out a tangent vector $\tau \in \text{se}(2)$, ready for $\boxplus$

- 1: $\Delta s_L = \lambda(\text{ticks}_t.L - \text{ticks}_{t-1}.L)$, likewise Δs_R
- 2: $\Delta s = \frac{1}{2}(\Delta s_R + \Delta s_L)$
- 3: $\Delta \theta = (\Delta s_R - \Delta s_L)/\ell$
- 4: return $(\Delta s, 0, \Delta \theta)^T$

ALGORITHM `raycast_scan(m, x_t, lidar)` COST $O(n_{\text{beams}} \cdot \log n_{\text{segments}})$ with a

BVH

in the map, the true pose, the LiDAR parameters

out a scan: one range per beam

- 1: $T = x_t \circ T_{\text{lidar}}$ (*sensor pose = body pose $\circ$ extrinsic*)
- 2: **for** $k = 0$ **to** $n_{\text{beams}} - 1$ **do**
- 3: $z^{k*} = \text{first-hit distance from } T \text{ along bearing } \varphi_k, \text{ capped at } z_{\text{max}}$
- 4: draw $u \sim \mathcal{U}[0, 1)$ and $\varepsilon \sim \mathcal{N}(0, \sigma_r^2)$
- 5: **if** $u < p_{\text{drop}}$ **or** $z^{k*} \geq z_{\text{max}}$ **then** $z^k = z_{\text{max}}$
- 6: **else** $z^k = \text{clamp}(z^{k*} + \varepsilon, 0, z_{\text{max}})$
- 7: **endfor**
- 8: **return** z

Line 4 draws *both* random numbers on every beam, whether or not the branch on line 5 uses them. That looks wasteful; it is deliberate. Consuming a fixed number of samples per beam means toggling dropout off does not reshuffle the noise on the beams that survive, so two runs that differ in one parameter differ *only* in that parameter. Random-number discipline of this kind is what makes an A/B comparison between two configurations mean anything.

4.5 Implementation in Rust

This section fixes the workspace architecture. Later chapters add crates and modules; none of them restructure what is decided here.

```
crates/
  pr-core/      # accumulating core: prob (Ch. 2), geom (Ch. 3)
  sim/          # THIS CHAPTER: worlds, robot, sensors, logs
  widget-kit/   # THIS CHAPTER: the chrome every demo reuses
  ...          # bayes_core (Ch. 5), motion (Ch. 9), sensor (Ch. 10), ...
demos/
  ch04-lab/     # one [[bin]] per widget: w4-1-dashboard, w4-2-lidar-anatomy, ...
```

4.5.1 The world

The map is polyline geometry, not a grid. Exact ray/segment intersection is cheap, exact, and resolution-independent; Chapter 13 *builds* a grid from these walls, but the walls themselves are never one.

RUST crates/sim/src/world.rs

```
1 use nalgebra::{Point2, Vector2};
2 use parry2d_f64::bounding_volume::Aabb;
3 use parry2d_f64::partitioning::Qbv;
4 use parry2d_f64::query::visitors::RayIntersectionsVisitor;
5 use parry2d_f64::query::{Ray, RayCast};
6 use parry2d_f64::shape::Segment;
7
8 // Materials exist so a surface can be *optically* different from a wall
9 // without being *geometrically* different. Glass is a wall to a planner and
10 // nearly invisible to a LiDAR, and that mismatch is a real failure mode.
11 #[derive(Clone, Copy, PartialEq, Eq, Debug)]
12 pub enum Material {
13     Wall,
14     Glass,
15     Door(u8),
16 }
17
18 pub struct Wall {
19     pub seg: Segment,
20     pub material: Material,
21 }
22
23 pub struct World {
24     walls: Vec<Wall>,
25     // Quantized BVH over the wall segments. Rebuilt only when the map changes,
26     // which for a static world means exactly once -- and it turns the scan cost
27     // from O(n_beams * n_segments) into O(n_beams * log n_segments).
28     bvh: Qbv<u32>,
29     pub bounds: Aabb,
30 }
31
32 impl World {
33     pub fn hallway() -> Self { /* corridor with three identical doors */ }
34     pub fn apartment() -> Self { /* 12 * 9 m, five rooms off one corridor */ }
35
36     // First hit along a ray, or `max` if nothing is struck.
37     ///
```

```

38  /// The visitor is handed only the segments whose bounding volumes the ray
39  /// actually crosses, and it keeps scanning after a hit because a nearer one
40  /// may live in a sibling leaf.
41  pub fn raycast(&self, origin: Point2<f64>, angle: f64, max: f64) -> Hit {
42      let ray = Ray::new(origin, Vector2::new(angle.cos(), angle.sin()));
43      let mut best = Hit { distance: max, material: None };
44      let mut leaf = |idx: &u32| {
45          let w = &self.walls[idx as usize];
46          if let Some(t) = w.seg.cast_local_ray(&ray, best.distance, true) {
47              if t < best.distance {
48                  best = Hit { distance: t, material: Some(w.material) };
49              }
50          }
51      };
52      true;
53      let mut visitor = RayIntersectionsVisitor::new(&ray, max, &mut leaf);
54      self.bvh.traverse_depth_first(&mut visitor);
55      best
56  }
57
58  /// Would a body of the given radius sweeping from `from` to `to` touch a wall?
59  pub fn blocks(&self, from: &SE2, to: &SE2, radius: f64) -> bool { /* ... */ }
60
61  /// Chapter 10's likelihood field wants this, so it lives here from the start.
62  pub fn distance_to_nearest(&self, p: Point2<f64>) -> f64 { /* ... */ }
63 }

```

4.5.2 The robot

RUST crates/sim/src/robot.rs

```

1  use nalgebra::Vector3;
2  use pr_core::geom::SE2;
3  use rand::rngs::SmallRng;
4  use rand_distr::{Distribution, Normal};
5
6  #[derive(Clone, Copy, Debug)]
7  pub struct RobotParams {
8      pub wheel_radius: f64,          // r = 0.033 m
9      pub track: f64,                 // l = 0.16 m
10     pub body_radius: f64,           // 0.11 m
11     pub ticks_per_rev: u32,         // N = 4096
12     /// Stochastic, zero-mean, redrawn every tick.
13     pub slip_std: f64,              // sigma = 0.02
14     /// Systematic: the *true* right wheel is r(1 + delta). Odometry never learns this.
15     pub radius_bias_right: f64,     // delta = 0.0
16 }
17
18 #[derive(Clone, Copy, Debug)]
19 pub struct Twist { pub v: f64, pub omega: f64 }
20
21 pub struct Robot {
22     pub pose: SE2,
23     /// Cumulative wheel rotation, radians. This -- not the ground travel -- is
24     /// what an encoder integrates, and the difference is the whole chapter.
25     wheel_angles: [f64; 2],
26     params: RobotParams,

```

```

27     slip: SmallRng,
28 }
29
30 impl Robot {
31     /// Actuate with slip, integrate exactly, refuse to pass through walls.
32     pub fn step(&mut self, cmd: Twist, dt: f64, world: &World) -> StepOutcome {
33         let RobotParams { wheel_radius: r, track: l, .. } = self.params;
34         let half = cmd.omega * l / 2.0;
35         let (wl, wr) = ((cmd.v - half) / r, (cmd.v + half) / r);
36
37         // The wheels turn by the commanded amount, always.
38         self.wheel_angles[0] += wl * dt;
39         self.wheel_angles[1] += wr * dt;
40
41         // The floor is under no such obligation.
42         let eps = match Normal::new(0.0, self.params.slip_std) {
43             Ok(n) => [n.sample(&mut self.slip), n.sample(&mut self.slip)],
44             // sigma = 0 is not a valid Normal, and is exactly the noise-free case
45             // the regression test in `odometry_zero_noise_is_exact` exercises.
46             Err(_) => [0.0, 0.0],
47         };
48         let ds_l = r * wl * dt * (1.0 + eps[0]);
49         let ds_r = r * (1.0 + self.params.radius_bias_right) * wr * dt * (1.0 + eps[1]);
50
51         let ds = 0.5 * (ds_r + ds_l);
52         let dtheta = (ds_r - ds_l) / l;
53         // Exact arc integration. Not an approximation of one -- the arc itself.
54         let next = self.pose.boxplus(&Vector3::new(ds, 0.0, dtheta));
55
56         if world.blocks(&self.pose, &next, self.params.body_radius) {
57             return StepOutcome::Blocked; // wheels advanced; pose did not
58         }
59         self.pose = next;
60         StepOutcome::Moved { ds, dtheta }
61     }
62 }

```

4.5.3 The encoders

RUST crates/sim/src/encoders.rs

```

1 use nalgebra::Vector3;
2 use std::f64::consts::TAU;
3
4 /// Cumulative quadrature counts. Integers on purpose: an encoder has no
5 fractional state to report, and pretending otherwise hides the error budget.
6 #[derive(Clone, Copy, Default, PartialEq, Eq, Debug)]
7 pub struct EncoderTicks { pub left: i64, pub right: i64 }
8
9 pub fn observe(wheel_angles: [f64; 2], p: &RobotParams) -> EncoderTicks {
10     let per_rad = f64::from(p.ticks_per_rev) / TAU;
11     EncoderTicks {
12         left: (wheel_angles[0] * per_rad).round() as i64,
13         right: (wheel_angles[1] * per_rad).round() as i64,
14     }
15 }
16

```

```

17 // Metres of wheel travel per count: lambda = 2pi r / N.
18 pub fn tick_length(p: &RobotParams) -> f64 {
19     TAU * p.wheel_radius / f64::from(p.ticks_per_rev)
20 }
21
22 // `odometry_delta` -- counts to a tangent vector, ready for [+].
23 ///
24 /// The middle component is a structural zero, not a rounding artefact: a
25 /// differential drive has no lateral degree of freedom, and writing the twist
26 /// this way makes the compiler agree.
27 pub fn odometry_delta(prev: EncoderTicks, cur: EncoderTicks, p: &RobotParams) -> Vector3<
    f64> {
28     let lambda = tick_length(p);
29     let ds_l = lambda * (cur.left - prev.left) as f64;
30     let ds_r = lambda * (cur.right - prev.right) as f64;
31     Vector3::new(0.5 * (ds_r + ds_l), 0.0, (ds_r - ds_l) / p.track)
32 }

```

4.5.4 Determinism, and the log every later chapter consumes

RUST crates/sim/src/run.rs

```

1 use rand::{Rng, SeedableRng};
2 use rand::rngs::SmallRng;
3 use serde::{Deserialize, Serialize};
4
5 #[derive(Clone, Serialize, Deserialize)]
6 pub struct SimConfig {
7     pub world: WorldId,
8     pub robot: RobotParams,
9     pub lidar: LidarParams,
10    pub seed: u64,
11    pub dt: f64,           // 0.02 s (the widgets on this page tick at 0.1 s, for
                           // legibility)
12    pub scan_every: u32,  // one scan per 5 ticks
13 }
14
15 // One RNG stream per noise source, all derived from the single config seed.
16 ///
17 // Sharing one generator between wheel slip and LiDAR noise would make the
18 // trajectory depend on how many beams the LiDAR happens to have -- so adding a
19 // sensor would silently invalidate every recorded log in the repository. The
20 // streams cost nothing and buy a guarantee.
21 fn split_streams(seed: u64) -> (SmallRng, SmallRng) {
22     let mut master = SmallRng::seed_from_u64(seed);
23     (
24         SmallRng::seed_from_u64(master.random()), // wheel slip
25         SmallRng::seed_from_u64(master.random()), // LiDAR
26     )
27 }
28
29 // What every estimator in this book consumes.
30 #[derive(Clone, Serialize, Deserialize)]
31 pub struct Frame {
32     pub t: f64,
33     // Ground truth. Present in the log, and forbidden to every algorithm that
34     // is not scoring one. The type system cannot enforce that; code review can.

```

```

35     pub truth: SE2,
36     pub ticks: EncoderTicks,
37     pub scan: Option<Scan>,
38 }
39
40 #[derive(Clone, Serialize, Deserialize)]
41 pub struct Log { pub cfg: SimConfig, pub cmds: Vec<Twist>, pub frames: Vec<Frame> }
42
43 impl Log {
44     /// Chapters 11 and 12 benchmark on identical saved logs. That promise is
45     /// made here, and `tests/replay.rs` is where it is kept.
46     pub fn record(cfg: SimConfig, script: &dyn Script) -> Log { /* ... */ }
47     pub fn save(&self, path: &Path) -> io::Result<()> { /* postcard */ }
48     pub fn load(path: &Path) -> io::Result<Log> { /* postcard */ }
49 }

```

4.5.5 The tests that pin all of it

RUST crates/sim/tests/odometry.rs

```

1 use approx::assert_relative_eq;
2
3 // The chapter's hand-checkable worked example, to the digit.
4 #[test]
5 fn worked_example_ch04_one_arc() {
6     let p = RobotParams { track: 0.16, ..RobotParams::rusty() };
7     // Exact arc lengths, so the only thing under test is the geometry.
8     let (ds_l, ds_r) = (0.72, 0.88);
9     let ds = 0.5 * (ds_r + ds_l);
10    let dtheta = (ds_r - ds_l) / p.track;
11    assert_relative_eq!(ds, 0.80, epsilon = 1e-12);
12    assert_relative_eq!(dtheta, 1.00, epsilon = 1e-12);
13
14    let step = SE2::exp(&Vector3::new(ds, 0.0, dtheta));
15    assert_relative_eq!(step.x(), 0.673_176_788, epsilon = 1e-9);
16    assert_relative_eq!(step.y(), 0.367_758_155, epsilon = 1e-9);
17    assert_relative_eq!(step.theta(), 1.0, epsilon = 1e-12);
18
19    // The chord points along half the heading change, and falls short of the
20    // arc by exactly what the midpoint rule overshoots by.
21    let chord = (step.x().powi(2) + step.y().powi(2)).sqrt();
22    assert_relative_eq!(chord, ds * (dtheta / 2.0).sin() / (dtheta / 2.0), epsilon = 1e-12);
23    assert_relative_eq!(step.y().atan2(step.x()), dtheta / 2.0, epsilon = 1e-12);
24    assert_relative_eq!(ds - chord, 0.032_919_138, epsilon = 1e-9);
25
26    // Naive Euler: right distance, wrong direction, 39 cm off it.
27    let euler = (ds - step.x()).hypot(step.y());
28    assert_relative_eq!(euler, 0.389_011_810, epsilon = 1e-9);
29 }
30
31 // Derivations 2 and 3 describe the same map, so with no noise and an encoder
32 // fine enough to ignore, dead reckoning must reproduce ground truth exactly.
33 #[test]
34 fn odometry_zero_noise_is_exact() {
35     let p = RobotParams { slip_std: 0.0, radius_bias_right: 0.0,
36                          ticks_per_rev: 1 << 26, ..RobotParams::rusty() };

```

```

37 let mut sim = Sim::new(SimConfig { robot: p, seed: 7, dt: 0.02, ..cfg() });
38 let mut dr = sim.truth();
39 let mut prev = EncoderTicks::default();
40
41 for t in 0..3_000 {
42     let frame = sim.step(figure_eight(t));
43     dr = dr.boxplus(&odometry_delta(prev, frame.ticks, &p));
44     prev = frame.ticks;
45 }
46 let err = (dr.boxminus(&sim.truth())).norm();
47 assert!(err < 1e-7, "dead reckoning drifted {err} m with no noise");
48 }
49
50 /// The determinism regression lock. Two runs from the same seed must agree
51 /// bit-for-bit -- not "to 1e-9", bit-for-bit -- or a benchmark comparing two
52 /// algorithms on "the same" log is comparing two different logs.
53 #[test]
54 fn replay_is_bit_exact() {
55     fn bits(p: &SE2) -> [u64; 3] {
56         [p.x().to_bits(), p.y().to_bits(), p.theta().to_bits()]
57     }
58
59     let cfg = SimConfig { seed: 20_260_811, ..cfg() };
60     let a = Log::record(cfg.clone(), &FigureEight);
61     let b = Log::record(cfg, &FigureEight);
62     assert_eq!(a.frames.len(), b.frames.len());
63     for (fa, fb) in a.frames.iter().zip(&b.frames) {
64         assert_eq!(bits(&fa.truth), bits(&fb.truth));
65         assert_eq!(fa.ticks, fb.ticks);
66     }
67 }

```

NOTE

The widgets on this page run the same algorithms: `web/lib/sim/rusty.ts` is a line-for-line port of `crates/sim`, and the numbers quoted in this chapter — 0.673177, 0.032919, 0.389012, the 2.678 m systematic drift — were produced by it. Where the prose and the code disagree, the tests decide.

4.6 Measuring the lab like an experimentalist

Every claim above is a claim about a distribution, and a distribution is not something you can see in one run. So do what an experimentalist would: run the same command script many times and look at the spread.

INTERACTIVE · w4.4
The Seed Lab

One simulation run tells you nothing. A run is a sample; what you want is the distribution it came from.

Run this simulation in the web edition: [/chapters/ch04-rusty-and-sensors #w4.4](#)

This widget is the reason every benchmark in this book reports over many seeds. Three things it makes visible that no single trajectory can:

A run is a sample. Any one of those orange traces looks like *the* answer. Forty-eight of them are a distribution over futures, and the honest summary of "what will Rusty do" is the fan, not any member of it.

Growth is superlinear in time. Measured on the straight leg over 480 seeds (the widget runs 48, so expect a little more scatter), the total spread at 1, 2 and 3 seconds is 2.1 cm, 5.7 cm and 10.1 cm — ratios of 2.7 and 4.9 against the $t^{3/2}$ predictions of 2.83 and 5.20. Heading error accumulates as a random walk, $\sqrt{t}$; position error integrates heading error, adding a factor of t . Anyone who tells you odometry error grows like $\sqrt{t}$ is quoting the heading.

The spread is anisotropic, and it is the wrong shape for a circle. At the end of the straight leg the cross-track spread is nine times the along-track spread: slip that differs between the wheels turns you, and a heading error only becomes a position error *sideways*. That crescent is the "banana" distribution Chapter 9 derives analytically, grown here from nothing but per-wheel noise — and it is the first concrete demonstration in this book that a Gaussian is going to be an approximation.

Now push the second slider off zero. The fan stops being centred on the dashed line at all; the whole ensemble leaves together. That is bias, and no number of seeds will average it away.

4.7 Anatomy of a book widget

One last piece of infrastructure, and it is the one you will use most: the contract every figure in this book obeys.

INTERACTIVE · w4.5

Anatomy of a Book Widget

Every figure in this book obeys the same seven-part contract — and the seed is part of it.

Run this simulation in the web edition: [/chapters/ch04-rusty-and-sensors #w4.5](#)

Two clauses deserve their own paragraph.

The seed is visible. Not logged, not configurable-if-you-dig — printed in the transport bar of every widget. This is a claim the book makes about itself:

a figure you cannot reproduce is a figure you cannot check, and a benchmark whose seed is hidden is a benchmark you should not believe. On the Rust side the same claim is enforced by `replay_is_bit_exact`, which is why `Sim` contains no parallelism and no unordered summation. Speed inside the simulator core is worth less than the guarantee.

Autoplay is not decoration; it is an accessibility position. Interaction is an invitation, not a requirement. A reader who never touches a control should still learn the lesson from the default run, which means the default parameters have to be chosen for legibility rather than realism — and where they are, this book says so out loud, as it did for σ_{slip} above.

4.8 Where this goes

You now own the instrument. Everything else in the book is measurement.

Chapter 5 puts a belief on top of it and gives the corridor a filter. Chapter 9 fits a probabilistic motion model to this simulator’s slip — deliberately in a parametrization the simulator does not use, so the reader meets model mismatch honestly rather than by accident. Chapter 10 learns beam intrinsics from Lidar data recorded here. Chapter 12 runs its grid-versus-MCL benchmark on *identical* saved Logs. And the Apartment is the arena for all of it, right through Chapter 26.

4.9 Exercises

1 FOUNDATION • Inverse kinematics and the ICR

Derive $(v, \omega) \mapsto (\omega_L, \omega_R)$ from Derivation 1 and show that the instantaneous centre of rotation lies on the wheel axle at signed distance $R = v/\omega$. Then exhibit a body twist that a differential drive **cannot** produce at any instant, and connect it to the Pfaffian constraint $\dot{x} \sin \theta - \dot{y} \cos \theta = 0$. Finally: Rusty has $r = 0.033$ m and a motor limit of 20 rad/s per wheel. What is the tightest turn it can make at 0.4 m/s?

2 FOUNDATION •• Price the approximation over a real trajectory

Using the polar form of Derivation 2, prove that the midpoint rule’s error is purely radial and equal to $\Delta s (1 - \text{sinc}(\Delta\theta/2))$. Then bound the total error accumulated over a 60-second figure-eight at $v = 0.5$ m/s, $|\omega| = 0.6$ rad/s, when the odometry is integrated at (a) 50 Hz, (b) 5 Hz, and (c) 2 Hz. At which rate does the integration error exceed the quantization error from a 4096-count encoder? At which does it exceed the slip drift at $\sigma_{\text{slip}} = 0.02$?

3 FOUNDATION • • • Why t to the three halves

Model the per-tick heading increment error as i.i.d. $\mathcal{N}(0, s^2)$ and the forward step as a constant d metres. Show that the variance of the accumulated cross-track position error after n ticks is $d^2 s^2 \sum_{k=1}^{n-1} k \sim d^2 s^2 n^2 / 2$, so the standard deviation grows like $n^{3/2}$. Then state precisely which assumption you would have to break for odometry error to grow like $\sqrt{t}$, and give a physical situation in which it does.

4 CONCEPTUAL • Predict the fan, then check it

Before touching the Seed Lab, sketch two pictures. (a) The fan when the right wheel's radius is 1% too large and slip is zero. (b) The fan when slip is at maximum and $\delta = 0$. For each: where is the centre of the fan relative to the gray dashed line, and how wide is it? Now set both sliders and check. Write one sentence per picture distinguishing a systematic from a stochastic error. Then a harder one, which the widget cannot show you: if *stochastic* slip afflicted only the right wheel, would the fan skew? Argue from the sign of $\Delta\theta = (\Delta s_R - \Delta s_L)/\ell$, and say why that is not the same experiment as turning δ up.

5 CONCEPTUAL • • Read the cliff edges

In LiDAR Anatomy, predict — before scrubbing — the beam indices at which the range jumps as the beam crosses the doorway edges, given that the sensor sits 1.06 m from the corridor's south wall, centred on a doorway 1.0 m wide, with 180 beams over a full circle and beam 90 pointing straight ahead. Verify with the scrubber. There is a third cliff, near beam 5. What is it? Now raise the dropout rate to 20% and answer: if you smoothed the range-versus-beam curve to suppress the dropout spikes, what would you destroy? (This is not a rhetorical question. It is why Chapter 10 models dropouts as a mixture component rather than filtering them out.)

6 PRACTICAL • • Give Rusty a sonar

Add a Sonar type to `crates/sim` beside Lidar, exposing the same scan-style API: aperture 15° , return equal to the nearest hit anywhere *within* the cone plus Gaussian noise. Implement the cone query by sampling the aperture finely enough that the error is below σ , and say what "finely enough" is. Then write one paragraph explaining why sonar walls smear at corners where LiDAR walls do not, and what that does to a scan matcher.

7 PRACTICAL • • • Break determinism on purpose

Make `replay_is_bit_exact` pass in your fork, native and WASM. Then break it deliberately in two different ways: (a) share one `SmallRng` between wheel slip and LiDAR noise, then change `n_beams` from 180 to 181 and watch the *trajectory* change; (b)

accumulate a per-beam sum over a scan with a rayon parallel reduction instead of a sequential fold, and exhibit two runs that disagree in the last bits. For each, explain in two sentences what property was lost and which downstream chapter’s benchmark it would silently corrupt.

4.10 References

Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control Cambridge University Press (ISBN 9781107156302)*.

§13.3 gives the differential-drive kinematics of Derivation 1 and §13.4 the odometry update of Derivation 3 — already in exponential-coordinate form. Their eq. (13.35) is our $V(\omega)\rho$ with the algebra spelled out; recognizing it as $\exp$ is the only thing this chapter adds. [link](#)

Borenstein, J. and Feng, L. (1996). Measurement and Correction of Systematic Odometry Errors in Mobile Robots *IEEE Transactions on Robotics and Automation* 12(6), 869–880.

The UMBmark procedure, and the source of this chapter’s insistence on separating systematic from stochastic error. Their two dominant terms — uncertain wheelbase and unequal wheel diameters — are exactly the second slider in w4.1. doi:10.1109/70.544770

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics MIT Press*.

§5.2.1 sets up the kinematic configuration this chapter simulates; Chapter 6’s beam mixture is the inference model our two-component generative LiDAR is deliberately not. [link](#)

Solà, J., Deray, J., and Atchuthan, D. (2018). A Micro Lie Theory for State Estimation in Robotics *arXiv:1812.01537*.

The reference for the $\boxplus$ / $\boxminus$ notation used throughout, and for why composing odometry increments on the group is not the same as adding them in coordinates. [link](#)

Manivasagam, S., Wang, S., Wong, K., Zeng, W., Sazanovich, M., Tan, S., Yang, B., Ma, W.-C., and Urtasun, R. (2020). LiDARsim: Realistic LiDAR Simulation by Leveraging the Real World *CVPR 2020*, 11164–11173.

The state of the art in going beyond a ray-cast-plus-Gaussian sensor: physics for the geometry, a learned residual for everything the geometry misses. Read it as the honest upper bound on how wrong our σ_r model is. [link](#)

Guillard, B., Vemprala, S., Gupta, J. K., Miksik, O., Vineet, V., Fua, P., and Kapoor, A. (2022). Learning to Simulate Realistic LiDARs *IEEE/RSJ IROS 2022 (arXiv:2209.10986)*.

Learns ray-drop and intensity from real scans, and shows that dropout is strongly surface-dependent rather than the i.i.d. p_{drop} we use. The right thing to read before believing any dropout number in this chapter. [link](#)

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W. (2022). Robot Operating System 2: Design, Architecture, and Uses in the Wild *Science Robotics* 7(66), eabm6074.

Where the interfaces this chapter’s Frame type imitates actually live in production. Useful for seeing which of our simplifications a deployed stack also makes, and which it does not. doi:10.1126/scirobotics.abm6074

Fischer, T., Paredes, I., Batchelor, M., Beier, T., Haviland, J., Traversaro, S., Vollprecht, W., Schmitz, M., and Milford, M. (2024). ROS2WASM: Bringing the Robot Operating System to the Web *arXiv:2409.09941*.

Independent evidence for this book’s central bet: compiling a robotics stack to WebAssembly makes results reproducible and shareable in a way a README never does. Their argument for the browser is our argument for every widget on this page. [link](#)

Part II

The Bayes Filter Family

The Bayes Filter

THE BAYES FILTER FAMILY · Foundational · 50 min



Instead of relying on a single “best guess” as to what might be the case in the world, probabilistic algorithms represent information by probability distributions over a whole space of possible hypotheses.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 1

Every estimator in this book — the Kalman filter, the particle filter, Monte Carlo localization, the mapping algorithms, even the SLAM back-ends that appear to work differently — is one algorithm wearing different clothes. This chapter is that algorithm.

The Bayes filter answers a single question: given everything the robot has done and everything it has sensed, what should it believe about the world right now? The answer is a recursion with exactly two moves. One incorporates an action and makes the belief worse. The other incorporates a measurement and makes it better. Everything else in Parts II through V is a decision about how to *represent* the belief so that those two moves can actually be computed.

🕒 AFTER THIS CHAPTER YOU CAN

- State what a belief is, formally, and why it is a distribution rather than a point
- Derive the Bayes filter recursion from Bayes rule and the Markov assumption
- Explain exactly which independence assumption each step of the derivation consumes
- Implement the filter as a Rust trait that every later estimator in this book satisfies

- Recognize when the Markov assumption is failing, and repair it by growing the state

≡ ASSUMED

- Conditional probability and Bayes rule (Chapter 2)
- Marginalization: integrating a joint distribution down to one variable
- Rust traits and associated types — a short refresher appears in the Practical section

5.1 The problem with knowing where you are

Put a robot in a corridor with three identical doors and ask it where it is. It has a door detector, and the detector says: *door*.

Now what? A robot that insists on a single answer must pick one of the three doors, and it will be wrong two times out of three. Worse, it will be wrong *confidently*, and every computation downstream — the path it plans, the obstacle it swerves around — inherits that false certainty without any record that a guess was ever made.

The probabilistic answer is to refuse the question as posed. The robot does not know where it is; it knows something weaker and far more useful, which is how plausible each location is given everything that has happened. That object is a **belief**, and this chapter is about how to keep one up to date.

🖥 INTERACTIVE · w5.1

The Hallway Belief Machine

Sensing sharpens the belief; moving smears it. Neither one ever tells you where the robot is.

Run this simulation in the web edition: [/chapters/ch05-bayes-filter #w5.1](#)

The two numbers that decide everything here are draggable. The wheels deliver the commanded step plus an error of about

0.12 m — drag that up and watch every peak in the belief widen. The door detector is right about

0.18 of the time; drag *that* and watch how much a single sighting is allowed to sharpen the belief. Nothing else about the chapter changes — you are turning the two knobs the mathematics below will name.

Watch it for a few cycles before reading on. Three things are worth noticing, and each one corresponds to a piece of mathematics later in the chapter.

The belief is multi-modal, and that is a feature. After the first door sighting

there are three peaks. No single number could express "I am at one of these three places, equally", and any representation that forces a single number — as the Kalman filter of Chapter 6 does — cannot survive this corridor.

Sensing multiplies; moving convolves. The green dashed curve is the measurement likelihood. Correction multiplies the belief by it pointwise, which is why peaks get sharper and mass concentrates. Prediction convolves with the motion noise, which is why everything spreads. Those two operations, and their opposite effects on certainty, are the whole filter.

Ambiguity dies from sequence, not from evidence. No individual reading distinguishes door one from door three. What distinguishes them is the *pattern*: door, then a long gap, then a door. The recursion is what accumulates that pattern, and it is the reason a filter beats any single measurement no matter how good the sensor.

💡 THE ONE SENTENCE VERSION

A belief is a probability distribution over states, conditioned on all data so far. The Bayes filter maintains it recursively: predict with the motion model, then correct with the measurement model, and never store the history itself.

5.2 The mathematics

5.2.1 State, and what it means for it to be complete

The **state** x_t is whatever must be known about the world at time t to predict the future. For a mobile robot it is usually the pose; in Chapter 14 it grows to include the map.

State is called **complete** when it is a sufficient summary of the past — when knowing x_t makes everything before t irrelevant to predicting what comes next:

$$p(x_t \mid x_{0:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t \mid x_{t-1}, u_t)$$

This is the **Markov assumption**, and it is the single most consequential claim in this book. It is also, strictly speaking, false for every real robot. We will come back to that.

The robot interacts with the world through two streams, and the distinction matters because they have opposite effects on what it knows:

u_t	Control — an action taken between $t-1$ and t . Costs information: the world changes, and the robot is not sure how.
z_t	Measurement — an observation at time t . Gains information: the world constrains what the robot could be seeing.

Two conditional distributions describe these streams. The **motion model** (also called the state transition probability) and the **measurement model**:

$$p(x_t \mid x_{t-1}, u_t) \qquad p(z_t \mid x_t)$$

Chapter 9 and Chapter 10 are devoted to writing these down for real hardware. For now they are given.

5.2.2 Belief

The **belief** is the posterior over the state given every measurement and control so far:

$$\text{bel}(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$$

It is convenient to name the intermediate object that exists after the control has been folded in but before the measurement has:

$$\overline{\text{bel}}(x_t) = p(x_t \mid z_{1:t-1}, u_{1:t})$$

The difference between those two subscripts — $z_{1:t}$ versus $z_{1:t-1}$ — is the entire correction step. Calling $\overline{\text{bel}}$ the "prediction" and bel the "posterior" is exactly the orange and purple in the widget above.

5.2.3 The recursion

ALGORITHM `Bayes_filter(bel(x_{t-1}), u_t, z_t)` COST $O(|X|^2)$ for a discrete state space of size $|X|$ -- the prediction integral dominates
in the previous belief, the control, the measurement
out `bel(x_t)`

- 1: for all x_t do
- 2: $\overline{\text{bel}}(x_t) = \int p(x_t \mid u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$
- 3: $\text{bel}(x_t) = \eta p(z_t \mid x_t) \overline{\text{bel}}(x_t)$
- 4: endfor
- 5: return $\text{bel}(x_t)$

Written out in one line, with each term tinted the colour it wears in every figure in this book:

$$\text{bel}(x_t) = \eta p(z_t \mid x_t) \int p(x_t \mid u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$$

Point at any coloured term above and the matching layer in the Hallway Belief Machine comes forward while the rest fade. The colours are not decoration: the green factor in that equation *is* the green dashed curve in the figure, and the purple result *is* the purple histogram.

Line 2 is prediction — a convolution of the previous belief with the motion model. Line 3 is correction — a pointwise product with the measurement likelihood, renormalized by $\eta = 1/p(z_t \mid z_{1:t-1}, u_{1:t})$, the probability of the measurement you actually got.

That normalizer is worth a moment. It is the **evidence**, and while the filter treats it as housekeeping, it is a genuine diagnostic: an unexpectedly small η means the measurement was surprising under the current belief. Chapter 11 uses exactly this to reject outliers, and Chapter 12 uses it to detect that a robot has been kidnapped.

DERIVATION Deriving the Bayes filter by induction

$$\text{bel}(x_t) = \eta p(z_t \mid x_t) \int p(x_t \mid u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$$

We show that if line 5 returns the correct posterior at $t - 1$, it returns the correct posterior at t . The base case is the prior $\text{bel}(x_0)$, which we are given.

Step 1 — apply Bayes rule to split off the newest measurement.

$$p(x_t \mid z_{1:t}, u_{1:t}) = \frac{p(z_t \mid x_t, z_{1:t-1}, u_{1:t}) p(x_t \mid z_{1:t-1}, u_{1:t})}{p(z_t \mid z_{1:t-1}, u_{1:t})}$$

The denominator does not depend on x_t , so name it η and move on.

Step 2 — use the Markov assumption on the measurement. If x_t is complete, then knowing it makes earlier data irrelevant to the current reading:

$$p(z_t \mid x_t, z_{1:t-1}, u_{1:t}) = p(z_t \mid x_t)$$

This gives $\text{bel}(x_t) = \eta p(z_t \mid x_t) \overline{\text{bel}}(x_t)$ — line 3.

Step 3 — expand the prediction by the law of total probability. Introduce the previous state and integrate it back out:

$$\overline{\text{bel}}(x_t) = \int p(x_t \mid x_{t-1}, z_{1:t-1}, u_{1:t}) p(x_{t-1} \mid z_{1:t-1}, u_{1:t}) dx_{t-1}$$

Step 4 — use the Markov assumption on the motion. Given x_{t-1} and u_t , nothing earlier helps predict x_t :

$$p(x_t \mid x_{t-1}, z_{1:t-1}, u_{1:t}) = p(x_t \mid x_{t-1}, u_t)$$

Step 5 — drop the future control from the past belief. The control u_t is applied *after* x_{t-1} , so it carries no information about it:

$$p(x_{t-1} \mid z_{1:t-1}, u_{1:t}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1}) = \text{bel}(x_{t-1})$$

This assumes the controls are chosen without secretly consulting the state — a subtlety that becomes a real issue in Chapter 24, where the robot deliberately chooses actions based on its belief.

Substituting Steps 4 and 5 into Step 3 gives line 2, and the induction closes.

■

5.2.4 What each assumption bought, and what it cost

The derivation consumed the Markov assumption twice — once for measurements, once for motion — and it is worth being precise about what happens when it fails, because it always does.

Real robots violate it constantly. A person walking through the room makes consecutive laser scans dependent in a way the pose alone does not explain. A systematically miscalibrated wheel radius makes consecutive odometry errors correlated. A map that is slightly wrong makes *every* measurement correlated forever.

The failure has a characteristic signature: the filter becomes **overconfident**. It treats correlated errors as independent evidence and multiplies them together, so the belief keeps sharpening while the truth walks out of it.

INTERACTIVE · w5.2

The Markov Breaker

When errors are correlated, a Bayes filter does not become wrong — it becomes overconfident, which is worse.

Run this simulation in the web edition: </chapters/ch05-bayes-filter#w5.2>

There are exactly two honest repairs, and both appear later in the book:

1. **Grow the state** until it is complete again. If a miscalibrated wheel radius is the problem, estimate the wheel radius: put it in x_t . This is why Chapter 14 puts the map into the state, and why modern visual-inertial systems estimate IMU biases (Chapter 18).
2. **Inflate the noise** to buy back the honesty you lost. Deliberately model the sensor as worse than it is, so that correlated evidence cannot compound. This is a hack, it is what almost every deployed system does, and Chapter 10 shows how to tune it without lying to yourself.

5.3 The family tree

The Bayes filter as written is not implementable. Line 2 is an integral over a continuous state space, and line 3 requires a function defined at every point of that space. Every practical filter is a choice of representation for *bel* that turns those into finite computation — and every one of those choices trades away something.

INTERACTIVE · w5.3

The filter family tree

Every practical filter is the Bayes filter plus one decision about how to represent a belief — and every decision costs something.

Run this simulation in the web edition: [/chapters/ch05-bayes-filter #w5.3](/chapters/ch05-bayes-filter/#w5.3)

That tree is the map of the next three chapters. The Kalman filter says: assume the belief is Gaussian and the models are linear, and the integral becomes matrix algebra. The particle filter says: represent the belief with samples, and the integral becomes a for-loop. Neither is more correct; they fail in different places, and knowing which failure you are buying is most of the skill.

5.4 Implementation in Rust

The filter is an interface before it is an algorithm. In Rust that means a trait, and the associated types are what make it worth writing: a Kalman filter's control is a vector while a particle filter's is a motion command, and the compiler should not let you mix them up.

RUST `crates/pr-core/src/filters/bayes.rs`

```
1  /// The Bayes filter recursion, as an interface.
2  ///
3  /// Every estimator in this book implements this trait. The associated types are
4  /// what let a Kalman filter take an `SVector<f64, 3>` control while Monte Carlo
5  /// localization takes an odometry reading, without either one giving up type
6  /// safety at the call site.
7  pub trait BayesFilter {
8      /// What the belief is represented as: a Gaussian, a histogram, a particle set.
9      type Belief;
10     type Control;
11     type Measurement;
12
13     /// Prediction: fold in a control. This step loses information.
14     fn predict(&mut self, u: &Self::Control);
15
16     /// Correction: fold in a measurement. This step gains information.
17     ///
18     /// Returns the evidence  $p(z \mid z_{1:t-1})$ , the normalizer  $\eta_{t-1}$  from line 3.
19     /// A small value means the measurement was surprising -- Chapter 11 uses this
20     /// to reject outliers and Chapter 12 to detect a kidnapped robot.
```

```

21     fn correct(&mut self, z: &Self::Measurement) -> f64;
22
23     fn belief(&self) -> &Self::Belief;
24 }

```

The discrete case makes the two lines of the algorithm literal. A belief over a corridor is just an array of cell probabilities; prediction is a convolution, correction is a multiply.

RUST crates/pr-core/src/filters/histogram.rs

```

1  use crate::filters::BayesFilter;
2
3  /// A discrete Bayes filter over a 1-D corridor -- Thrun et al., Table 2.1.
4  ///
5  /// No Gaussians, no linearization: the only approximation is the discretization
6  /// itself. That makes this filter the reference every later one is measured against.
7  pub struct HistogramFilter {
8      cells: Vec<f64>,
9      cell_width: f64,
10     /// Treat the corridor as a loop, so mass leaving one end re-enters the other.
11     wrap: bool,
12 }
13
14 impl HistogramFilter {
15     /// The uniform prior: maximal ignorance, log2(n) bits of entropy.
16     pub fn uniform(n: usize, length: f64, wrap: bool) -> Self {
17         Self { cells: vec![1.0 / n as f64; n], cell_width: length / n as f64, wrap }
18     }
19
20     fn center(&self, i: usize) -> f64 {
21         (i as f64 + 0.5) * self.cell_width
22     }
23
24     /// Line 2:  $b(i) = \text{Sigma? } b(j) \cdot p(c? | c?, u) \cdot \text{Deltax}$ 
25     ///
26     /// The motion model enters as a kernel over the displacement error, so this
27     /// is a discrete convolution -- which is exactly why prediction always
28     /// spreads the belief out.
29     fn convolve(&self, u: f64, kernel: impl Fn(f64) -> f64) -> Vec<f64> {
30         let n = self.cells.len();
31         let mut out = vec![0.0; n];
32         for i in 0..n {
33             for j in 0..n {
34                 if self.cells[j] < 1e-12 {
35                     continue; // negligible mass contributes nothing measurable
36                 }
37                 let mut delta = self.center(i) - self.center(j) - u;
38                 if self.wrap {
39                     let span = n as f64 * self.cell_width;
40                     delta -= span * (delta / span).round();
41                 }
42                 out[i] += self.cells[j] * kernel(delta) * self.cell_width;
43             }
44         }
45         out
46     }
47 }

```

```

48
49 impl BayesFilter for HistogramFilter {
50     type Belief = Vec<f64>;
51     type Control = f64;
52     /// The likelihood  $p(z \mid x)$ , evaluated at a cell center.
53     type Measurement = Box<dyn Fn(f64) -> f64>;
54
55     fn predict(&mut self, u: &f64) {
56         self.cells = self.convolve(*u, |d| gaussian_pdf(d, 0.0, self.motion_sigma));
57         normalize(&mut self.cells);
58     }
59
60     fn correct(&mut self, likelihood: &Self::Measurement) -> f64 {
61         // Line 3: pointwise multiply, then normalize. The normalizer we divide
62         // by is the evidence, so we get the diagnostic for free.
63         for i in 0..self.cells.len() {
64             self.cells[i] *= likelihood(self.center(i));
65         }
66         let evidence: f64 = self.cells.iter().sum();
67         if evidence > 0.0 {
68             self.cells.iter_mut().for_each(|c| *c /= evidence);
69         }
70         evidence
71     }
72
73     fn belief(&self) -> &Vec<f64> {
74         &self.cells
75     }
76 }

```

i NOTE

The widget at the top of this chapter runs this algorithm — the TypeScript in `lib/filters/bayes.ts` is a line-for-line port of the Rust above, and both are checked against the same worked example below.

5.4.1 A worked example you can check by hand

Take a corridor of four cells with a uniform prior, and a door sensor that reports correctly 80% of the time. Cells 1 and 3 have doors. The sensor fires.

The prior is $\text{bel} = (0.25, 0.25, 0.25, 0.25)$. The likelihood of *door detected* is 0.8 at a door and 0.2 elsewhere, giving $(0.2, 0.8, 0.2, 0.8)$. Multiply pointwise:

$$(0.05, 0.2, 0.05, 0.2) \quad \text{summing to} \quad \eta^{-1} = 0.5$$

Normalize: $\text{bel} = (0.1, 0.4, 0.1, 0.4)$. Two hypotheses now hold 80% of the belief between them, and the filter has committed to neither. The evidence 0.5 says the reading was neither surprising nor especially informative — which is right, since half the corridor has a door.

Now carry it one step further yourself. The robot moves one cell to the

right, with a motion model that lands where commanded 80% of the time and overshoots by one cell 20% of the time. What is the new belief in cell 2?

WORK IT OUT

After the move, what is `bel(cell 2)`?

Answer. 0.34 Cell 2 receives 80% of cell 1's mass and 20% of cell 0's: $0.8 \times 0.4 + 0.2 \times 0.1 = 0.34$. Notice that the peak has moved *and* flattened — it was 0.4 before the move and is 0.34 after. Prediction never sharpens, which is the claim Exercise 3 asks you to prove.

RUST `crates/pr-core/src/filters/histogram.rs (tests)`

```
1  #[test]
2  fn worked_example_ch05_door_sighting() {
3      let mut f = HistogramFilter::uniform(4, 4.0, false);
4      let likelihood: Box<dyn Fn(f64) -> f64> =
5          Box::new(|x| if x > 1.0 && x < 2.0 || x > 3.0 { 0.8 } else { 0.2 });
6
7      let evidence = f.correct(&likelihood);
8
9      assert_relative_eq!(evidence, 0.5, epsilon = 1e-12);
10     assert_relative_eq!(f.belief()[1], 0.4, epsilon = 1e-12);
11     assert_relative_eq!(f.belief()[0], 0.1, epsilon = 1e-12);
12 }
```

Every worked example in this book has a test like this one beside it. If the prose and the code ever disagree, the test is what settles it.

5.5 Exercises

1 FOUNDATION • The evidence is not a nuisance

Show that the normalizer in line 3 equals $p(z_t \mid z_{1:t-1}, u_{1:t})$ — the probability of the measurement, averaged over the predicted belief. Then explain in one sentence why a sequence of small evidence values is a better kidnapping detector than any single one.

2 FOUNDATION •• Where the assumption enters

The derivation uses the Markov assumption in Step 2 and Step 4. For each, construct a concrete robot scenario in which it fails, and say which of the two repairs (grow the state, inflate the noise) you would use and why.

3 FOUNDATION • • • Prediction cannot sharpen

Prove that the prediction step never decreases the entropy of the belief when the motion kernel has non-zero variance. (Hint: prediction is a convolution; consider what convolution does to a distribution's Fourier transform, or apply the entropy power inequality.) Then explain why this is the formal statement of "moving smears".

4 CONCEPTUAL • Predict, then check

PREDICT FIRST

Set motion noise to its maximum and switch sensing off. After twenty steps, what does the belief look like?

- A single peak that has drifted away from the truth
Drift would require the filter to be wrong about the mean. It is not — prediction is unbiased. What it loses is certainty, not accuracy.
- ✓ A flat belief, spread over the whole corridor
Right. Each prediction convolves with the motion kernel, and repeated convolution drives any distribution on a loop toward the uniform one. Entropy rises every step and nothing pushes it back down.
- Three peaks, one at each door
Doors only enter through the measurement model. With sensing off the filter has never heard of them.

Now turn sensing back on. The belief does not recover in one sighting: the first correction reinstates three peaks, and only the *sequence* — door, gap, gap, door — collapses them to one. Count how many sightings it actually takes, then try it again with the sensor made unreliable.

5 CONCEPTUAL • • Break it deliberately

Find a parameter setting where the filter is confidently wrong — the belief has one sharp peak that does not contain the true position. What did you have to do to produce it? Which of the two Markov repairs would fix it?

HINTS

<>Confidence comes from correction and is destroyed by prediction. So you want a filter that corrects often and predicts little.</> <>Being wrong as well as confident needs the evidence to mislead. What does a sensor that is right 55% of the time do to a belief, over many readings, if you let the filter keep trusting it?</> <>Turn the sensor's reliability down toward the middle and the step size down toward zero, then let it run. The filter accumulates near-worthless evidence as if it were decisive.</>

6 PRACTICAL •• Implement the recursion

Implement `HistogramFilter` in Rust so that the worked-example test above passes. Then add a method `entropy()` returning the belief's entropy in bits, and reproduce the claim in Exercise 3 numerically: log the entropy for twenty alternating predict/correct steps and show that prediction never lowers it.

7 PRACTICAL ••• A filter that knows it is lost

Extend your implementation to track a running average of the evidence returned by correct. Trigger a `Lost` state when the recent average drops below a fraction of the long-run average. Test it by teleporting the true robot mid-run. You have just built the core idea behind Augmented MCL, which Chapter 12 derives properly.

5.6 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 2 is the source of this chapter's derivation and notation. The Bayes filter appears there as Table 2.1. [link](#)

Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems *Journal of Basic Engineering* 82(1), 35–45.

The special case that started it all; Chapter 6 derives it as a Bayes filter with Gaussian belief. doi:10.1115/1.3662552

Barfoot, T. D. (2024). *State Estimation for Robotics* Cambridge University Press, 2nd edition.

The modern reference treatment. Chapter 4 presents the same recursion in a form that generalizes cleanly to Lie groups — which is where Chapter 7 takes it. [link](#)

Chen, Z. (2003). Bayesian Filtering: From Kalman Filters to Particle Filters, and Beyond *Statistics* 182(1), 1–69.

A survey that lays out the family tree in this chapter's third section, with the approximations each branch makes stated explicitly. [link](#)

Särkkä, S. and Svensson, L. (2023). Bayesian Filtering and Smoothing *Cambridge University Press*, 2nd edition.

The rigorous probabilistic-systems treatment, and the best source for the smoothing counterpart that Chapter 6 introduces. doi:10.1017/9781108917407

Kalman Filters

THE BAYES FILTER FAMILY · Intermediate · 55 min

“

The fact that the residual uncertainty is smaller than the contributing Gaussians may appear counter-intuitive, but it is a general characteristic of information integration in Kalman filters.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 3

Chapter 5 left us with an exact recursion and no way to run it. The prediction step is an integral over a continuous state space; the correction step needs a function evaluated everywhere. The histogram filter escapes by chopping the space into cells, and pays for it: to localize Rusty on a ten-metre rail to the nearest centimetre you need a thousand cells for one dimension, and a thousand *cubed* the moment you want a full pose.

This chapter buys computability with two commitments — linearity and Gaussianity — and gets back the most consequential algorithm in engineering. The belief collapses from a function to two numbers per dimension, and the Bayes filter’s two lines become matrix algebra you can run at ten kilohertz on a microcontroller.

Three things come out of the algebra that are worth the price of the chapter on their own. The **Kalman gain** is not a parameter; it falls out of the derivation as the ratio of two precisions, and it is completely determined before any data arrives. The **information form** is the same Gaussian in different coordinates, in which the *other* filter step is the cheap one — a duality that quietly sets up Chapter 15. And the **RTS smoother** shows that filtering, for all its elegance, is only the best *causal* use of the data.

◎ AFTER THIS CHAPTER YOU CAN

- State the three assumptions that make a Bayes filter exactly Gaussian,

and say what each one buys

- Derive both Kalman filter steps by completing the square, not by quoting them
- Read the Kalman gain as precision-weighted trust, and explain why it never depends on the data
- Translate any filter between moments form and information form, and predict which one is cheaper
- Detect an overconfident filter with NEES and NIS before it diverges
- Implement Kf, the information filter, and the RTS smoother in Rust on nalgebra, with Joseph-form updates

≡ ASSUMED

- The Bayes filter recursion (Chapter 5)
- Gaussians in n dimensions, moments vs. canonical form (Chapter 2)
- Matrix algebra: transpose, inverse, positive definiteness
- Rust generics; `const` generics are explained where they first appear

6.1 The problem with a grid

Rusty is on a rail. One degree of freedom, a beacon that pings its position, and a motor that never quite delivers the commanded step. The histogram filter of Chapter 5 will localize it perfectly well — at a thousand cells for centimetre resolution, recomputed every step, to describe a belief that is, every single time, one smooth bump.

That is the observation this chapter turns into an algorithm. If the belief is *always* going to be a single bump, stop storing the bump and store its mean and its width. Two numbers instead of a thousand, and the update becomes arithmetic.

🖥 INTERACTIVE · w6.1

The Kalman Tuning Bench

The gain is not a knob. R and Q are the knobs, and turning them the wrong way makes a filter confident rather than accurate.

Run this simulation in the web edition: `/chapters/ch06-kalman-filters #w6.1`

Leave it running for a few cycles before reading on. Four things in that widget are the chapter.

The posterior is taller than both its parents — taller meaning narrower, since all four curves are densities. The green measurement blob and the orange

prediction blob combine into a purple blob tighter than either. This is the sentence in the epigraph, and it is not a rounding artifact: two independent opinions genuinely beat one, and the algebra below says by exactly how much.

Prediction always widens; correction always narrows. Orange is never taller than the blue prior that preceded it. That is "moving smears" from Chapter 5, now in closed form: the covariance gains an additive $\mathbf{R}_t$ every step and there is nothing it can do about it.

The gain settles by itself. Watch the K readout. It converges within a dozen steps and then holds, even though the measurements keep changing. Nothing told it to. We will prove why: the covariance recursion never looks at the data.

A confident filter is not an accurate one. Press divergent. The filter is told the cart obeys its model almost perfectly ($\mathbf{R} \approx 0$) while the cart in fact slips. Its covariance collapses, its 2σ band shrinks to a hairline, and the truth walks straight out of it. The NIS readout goes red well before the RMSE readout looks alarming — and NIS is the one you can compute on a real robot.

One implementation note, because it matters for Exercise 7. The bench carries **two** states — the cart's position and its velocity — while the beacon reports position only. The four blobs are the position marginal. Velocity is never measured at any point in the run, and the filter estimates it anyway, through the off-diagonal terms of Σ that couple the two. That is the quiet superpower of a Gaussian belief: correlations turn one sensor into evidence about everything the sensor is correlated with.

💡 THE ONE SENTENCE VERSION

If the models are linear and every distribution in sight is Gaussian, the Bayes filter's integral and product have closed forms, and the belief can be carried forever as (μ_t, Σ_t) — with the weight given to each new measurement fixed in advance by the ratio of the two precisions.

6.2 Building intuition

Before any algebra, get the mechanism into your hands. The scalar case contains the entire idea, and the entire idea is a balance.

You have a prediction that says the cart is near $\bar{\mu}$, with uncertainty $\bar{\sigma}^2$. A beacon says it is near z , with uncertainty σ_z^2 . Neither is right. The Kalman filter's answer is *not* to average them, and it is not to pick the more trustworthy one. It is to hang a weight at each claim whose mass is that claim's **precision** — the reciprocal of its variance — and take the centre of mass.

 INTERACTIVE · w6.3**The Gain Lever**

The Kalman filter does not average the prediction and the measurement — it balances them, and the balance point is derived, not tuned.

Run this simulation in the web edition: `/chapters/ch06-kalman-filters #w6.3`

Two facts that widget makes physical, and that the derivations below make exact:

1. The posterior mean is always strictly between $\bar{\mu}$ and z . A Kalman filter can be wrong, but it cannot be wild.
2. The posterior precision is the *sum* of the two input precisions. Not the larger of them — the sum. That is why the purple curve is taller than both parents, and it is the whole reason fusing sensors is worth doing.

Everything else in this chapter is those two facts written in matrices.

6.3 The mathematics

6.3.1 Linear-Gaussian systems

The Kalman filter is the Bayes filter under three additional assumptions — linearity, Gaussian noise, and a Gaussian starting point — layered on top of the Markov assumption Chapter 5 already consumed.

Assumption 1 — linear motion with additive Gaussian noise.

$$x_t = \mathbf{A}_t x_{t-1} + \mathbf{B}_t u_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \mathbf{R}_t)$$

so that

$$p(x_t | u_t, x_{t-1}) = \mathcal{N}(x_t; \mathbf{A}_t x_{t-1} + \mathbf{B}_t u_t, \mathbf{R}_t)$$

Assumption 2 — linear measurements with additive Gaussian noise.

$$z_t = \mathbf{C}_t x_t + \delta_t, \quad \delta_t \sim \mathcal{N}(0, \mathbf{Q}_t), \quad p(z_t | x_t) = \mathcal{N}(z_t; \mathbf{C}_t x_t, \mathbf{Q}_t)$$

Assumption 3 — Gaussian prior. $\text{bel}(x_0) = \mathcal{N}(x_0; \mu_0, \Sigma_0)$.

Together these are exactly enough. Under them the posterior is Gaussian at every t — not approximately, not asymptotically — so the filter never needs to represent anything but a mean and a covariance. That closure claim is what the next two derivations prove.

μ_t, Σ_t	Moments form: the belief mean and covariance at time t .
$\bar{\mu}_t, \bar{\Sigma}_t$	The predicted belief, after the control and before the measurement.
$\xi_t = \Sigma_t^{-1} \mu_t$	Information vector — the canonical form's linear coefficient.
$\Omega_t = \Sigma_t^{-1}$	Information matrix — the curvature of the negative log-belief.
A_t, B_t, C_t	State transition, control, and measurement matrices.
R_t	Motion noise covariance. Thrun's convention. Most control texts call this Q .
Q_t	Measurement noise covariance. Thrun's convention. Most control texts call this R .
K_t	Kalman gain — how far towards the measurement to move.
$S_t = C_t \bar{\Sigma}_t C_t^T + Q_t$	Innovation covariance: how surprised the filter is entitled to be.
$\mu_{t T}, \Sigma_{t T}$	Smoothed belief: state t conditioned on all T measurements.

⚠ WHERE ROBOTS BREAK THIS

The R/Q trap. In this book, following Thrun, R is *motion* noise and Q is *measurement* noise. In most of the control literature — and, deliberately, in the TypeScript port that runs the widgets on this page — the letters are the other way around, so you can see the translation happening at the call site. There is no principled reason for either choice, and there is no chance the field will fix it. Whenever you read Kalman filter code, find one line that tells you which convention it uses before you touch a number. Every slider in this chapter is labelled with both the symbol and its meaning, for exactly this reason.

6.3.2 Prediction

📐 DERIVATION The prediction step, from the Bayes filter's integral

$$\bar{x} = A + Bu, \bar{\Sigma} = A \Sigma A^T + R$$

We want line 2 of the Bayes filter,

$$\overline{\text{bel}}(x_t) = \int p(x_t | u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$$

with both factors Gaussian. Write x for x_{t-1} , drop the time subscripts on the matrices, and put $d = x_t - Bu$ to keep the control out of the way. The integrand is $\exp(-L)$ with

$$L = \frac{1}{2}(d - Ax)^T R^{-1}(d - Ax) + \frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)$$

Step 1 — recognize a joint quadratic. L is quadratic in (x_t, x) jointly, so the integrand is an unnormalized Gaussian over both. Expanding and collecting the terms in x :

$$L = \frac{1}{2}x^\top \underbrace{(\mathbf{A}^\top \mathbf{R}^{-1} \mathbf{A} + \Sigma^{-1})}_{=\Psi^{-1}} x - (\mathbf{A}^\top \mathbf{R}^{-1} d + \Sigma^{-1} \mu)^\top x + \frac{1}{2}d^\top \mathbf{R}^{-1} d + \frac{1}{2}\mu^\top \Sigma^{-1} \mu$$

Step 2 — complete the square in x and integrate it out. Any quadratic splits as

$$L = \frac{1}{2}(x - x^\star)^\top \Psi^{-1}(x - x^\star) + L^\star, \quad x^\star = \Psi (\mathbf{A}^\top \mathbf{R}^{-1} d + \Sigma^{-1} \mu)$$

The first term integrates to $\sqrt{\det(2\pi\Psi)}$, a constant that does not involve x_t . It is absorbed into the normalizer and never seen again. Everything that matters is in

$$L^\star = \frac{1}{2}d^\top \mathbf{R}^{-1} d + \frac{1}{2}\mu^\top \Sigma^{-1} \mu - \frac{1}{2}(\mathbf{A}^\top \mathbf{R}^{-1} d + \Sigma^{-1} \mu)^\top \Psi (\mathbf{A}^\top \mathbf{R}^{-1} d + \Sigma^{-1} \mu)$$

Step 3 — read off the quadratic coefficient of d . Collecting the terms in $d^\top(\cdot)d$:

$$\frac{1}{2} d^\top (\mathbf{R}^{-1} - \mathbf{R}^{-1} \mathbf{A} \Psi \mathbf{A}^\top \mathbf{R}^{-1}) d$$

and the inversion lemma below, with $\mathbf{U} = \mathbf{R}$, $\mathbf{P} = \mathbf{A}$, $\mathbf{V} = \Sigma$, says that bracket is exactly $(\mathbf{R} + \mathbf{A}\Sigma\mathbf{A}^\top)^{-1}$.

Step 4 — read off the linear coefficient. The cross term is $-d^\top \mathbf{R}^{-1} \mathbf{A} \Psi \Sigma^{-1} \mu$, and

$$(\mathbf{R} + \mathbf{A}\Sigma\mathbf{A}^\top)^{-1} \mathbf{A} = \mathbf{R}^{-1} \mathbf{A} - \mathbf{R}^{-1} \mathbf{A} \Psi \mathbf{A}^\top \mathbf{R}^{-1} \mathbf{A} = \mathbf{R}^{-1} \mathbf{A} \Psi (\Psi^{-1} - \mathbf{A}^\top \mathbf{R}^{-1} \mathbf{A}) = \mathbf{R}^{-1} \mathbf{A} \Psi \Sigma^{-1}$$

so that cross term is $-d^\top (\mathbf{R} + \mathbf{A}\Sigma\mathbf{A}^\top)^{-1} \mathbf{A} \mu$. A quadratic with that Hessian and that linear term is, up to a constant,

$$L^\star = \frac{1}{2}(d - \mathbf{A}\mu)^\top (\mathbf{R} + \mathbf{A}\Sigma\mathbf{A}^\top)^{-1} (d - \mathbf{A}\mu) + \text{const}$$

Undoing $d = x_t - \mathbf{B}u$ gives the result. ■

The two-line version, and why we did it the long way. A linear map of a Gaussian is Gaussian, so it suffices to push the first two moments through: $\mathbb{E}[x_t] = \mathbf{A}\mu + \mathbf{B}u$ and $\text{Cov}(x_t) = \mathbf{A}\Sigma\mathbf{A}^\top + \mathbf{R}$, using $\text{Cov}(\mathbf{A}x + b) = \mathbf{A} \text{Cov}(x) \mathbf{A}^\top$ and the independence of ε_t . Correct, and four lines shorter. The long derivation earns its keep in Chapter 7: when $\mathbf{A}$ becomes a Jacobian the shortcut silently stops being exact, and only the completing-the-square version shows you *which* step became an approximation.

The result, in the colors of the widget: the blue prior becomes the orange prediction by

$$\bar{\mu}_t = \mathbf{A}_t \mu_{t-1} + \mathbf{B}_t u_t, \quad \bar{\Sigma}_t = \mathbf{A}_t \Sigma_{t-1} \mathbf{A}_t^\top + \mathbf{R}_t$$

Read the covariance line twice. $\mathbf{A}\Sigma\mathbf{A}^\top$ is the old uncertainty *dragged through* the dynamics — for a contracting system it can shrink. The $+\mathbf{R}_t$ is the uncertainty this step *creates*, and since $\mathbf{R}_t$ is positive definite, that term only ever adds. In every system we will meet, prediction is a net loss.

6.3.3 Correction

 **DERIVATION** The correction step, and where the gain comes from

$$\mathbf{K} = \bar{\Sigma} \mathbf{C}^\top (\mathbf{C} \bar{\Sigma} \mathbf{C}^\top + \mathbf{Q})^{-1}$$

Line 3 of the Bayes filter is a product of two Gaussians in x_t :

$$\text{bel}(x_t) = \eta \mathcal{N}(z_t; \mathbf{C}x_t, \mathbf{Q}) \mathcal{N}(x_t; \bar{\mu}, \bar{\Sigma})$$

Write x for x_t . The exponent is a sum of two quadratics,

$$J(x) = \frac{1}{2}(z - \mathbf{C}x)^\top \mathbf{Q}^{-1}(z - \mathbf{C}x) + \frac{1}{2}(x - \bar{\mu})^\top \bar{\Sigma}^{-1}(x - \bar{\mu})$$

Step 1 — the Hessian is the posterior information. Differentiating twice,

$$\frac{\partial^2 J}{\partial x^2} = \mathbf{C}^\top \mathbf{Q}^{-1} \mathbf{C} + \bar{\Sigma}^{-1} \quad \implies \quad \Sigma_t = (\bar{\Sigma}^{-1} + \mathbf{C}^\top \mathbf{Q}^{-1} \mathbf{C})^{-1}$$

For a Gaussian, "the exponent's curvature" *is* the inverse covariance. Note that the two information contributions simply added — remember this line, it is the whole information filter.

Step 2 — the minimum is the posterior mean. Setting $\partial J / \partial x = 0$:

$$-\mathbf{C}^\top \mathbf{Q}^{-1}(z - \mathbf{C}\mu_t) + \bar{\Sigma}^{-1}(\mu_t - \bar{\mu}) = 0 \implies (\bar{\Sigma}^{-1} + \mathbf{C}^\top \mathbf{Q}^{-1} \mathbf{C}) \mu_t = \bar{\Sigma}^{-1} \bar{\mu} + \mathbf{C}^\top \mathbf{Q}^{-1} z$$

so $\mu_t = \Sigma_t (\bar{\Sigma}^{-1} \bar{\mu} + \mathbf{C}^\top \mathbf{Q}^{-1} z)$.

Step 3 — that is already the answer. Steps 1 and 2 are the complete measurement update, in two lines, with no gain anywhere. They are the information filter's correct step, written a section early. Everything that follows is the work of translating them back into moments.

Step 4 — rearrange into an innovation. Split $z = (z - C\bar{\mu}) + C\bar{\mu}$ inside Step 2's right-hand side:

$$\bar{\Sigma}^{-1}\bar{\mu} + C^T Q^{-1} C\bar{\mu} + C^T Q^{-1}(z - C\bar{\mu}) = \Sigma_t^{-1}\bar{\mu} + C^T Q^{-1}(z - C\bar{\mu})$$

Multiplying by Σ_t gives $\mu_t = \bar{\mu} + \Sigma_t C^T Q^{-1}(z - C\bar{\mu})$, which names the gain: $K_t := \Sigma_t C^T Q^{-1}$.

Step 5 — the usable form of the gain. That expression needs Σ_t , which needs an $n \times n$ inversion. The familiar form needs only an $m \times m$ one. They are equal:

$$(\bar{\Sigma}^{-1} + C^T Q^{-1} C)^{-1} C^T Q^{-1} = \bar{\Sigma} C^T (C \bar{\Sigma} C^T + Q)^{-1}$$

Multiply *both* sides on the left by $(\bar{\Sigma}^{-1} + C^T Q^{-1} C)$ and on the right by $(C \bar{\Sigma} C^T + Q)$. The left-hand side becomes the first line below, the right-hand side the second:

$$C^T Q^{-1} (C \bar{\Sigma} C^T + Q) = C^T Q^{-1} C \bar{\Sigma} C^T + C^T$$

$$(\bar{\Sigma}^{-1} + C^T Q^{-1} C) \bar{\Sigma} C^T = C^T + C^T Q^{-1} C \bar{\Sigma} C^T$$

The same thing. ■

Step 6 — the covariance in moments form. Apply the inversion lemma to $\Sigma_t = (\bar{\Sigma}^{-1} + C^T Q^{-1} C)^{-1}$:

$$\Sigma_t = \bar{\Sigma} - \bar{\Sigma} C^T (C \bar{\Sigma} C^T + Q)^{-1} C \bar{\Sigma} = \bar{\Sigma} - K_t C \bar{\Sigma} = (I - K_t C) \bar{\Sigma}$$

Step 7 — the Joseph form, and why the code uses it. For *any* gain K , not just the optimal one, propagating the error $x_t - \mu_t = (I - KC)(x_t - \bar{\mu}) - K\delta_t$ through Cov gives

$$\Sigma_t = (I - KC) \bar{\Sigma} (I - KC)^T + K Q K^T$$

Expanding it and using $\bar{\Sigma} C^T = K S_t$ at the optimal gain collapses it back to $(I - KC) \bar{\Sigma}$:

$$(I - KC) \bar{\Sigma} - [\bar{\Sigma} C^T - K(C \bar{\Sigma} C^T + Q)] K^T = (I - KC) \bar{\Sigma}$$

So the two agree in exact arithmetic. They do not agree in floating point. The short form is a difference of two matrices and can go indefinite after enough round-off; the Joseph form is a sum of two symmetric positive-semidefinite terms and cannot. Exercise 6 makes this happen on purpose.

Collecting the results, with the colors of the widget:

$$S_t = \mathbf{C}_t \bar{\Sigma}_t \mathbf{C}_t^\top + \mathbf{Q}_t, \quad \mathbf{K}_t = \bar{\Sigma}_t \mathbf{C}_t^\top S_t^{-1}$$

$$\mu_t = \bar{\mu}_t + \mathbf{K}_t (z_t - \mathbf{C}_t \bar{\mu}_t), \quad \Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t) \bar{\Sigma}_t$$

The quantity $z_t - \mathbf{C}_t \bar{\mu}_t$ is the **innovation**: the part of the measurement the filter did not already know. S_t is how surprised the filter is *entitled* to be. Both names earn their keep later — Chapter 11 uses S_t to gate outliers, and the consistency section below uses it to catch a filter lying about its own confidence.

6.3.4 The gain is precision-weighted trust

Specialize to one dimension with $\mathbf{C} = 1$. Then $S = \bar{\sigma}^2 + \sigma_z^2$ and

$$K = \frac{\bar{\sigma}^2}{\bar{\sigma}^2 + \sigma_z^2}, \quad \mu = (1 - K) \bar{\mu} + K z, \quad \sigma^2 = (1 - K) \bar{\sigma}^2$$

Substituting K into the variance and inverting turns that last equation into the sentence the Gain Lever draws:

$$\frac{1}{\sigma^2} = \frac{1}{\bar{\sigma}^2} + \frac{1}{\sigma_z^2}$$

Precisions add. Not variances, not standard deviations — precisions. The posterior is more certain than either input, always, by an amount that does not depend on whether the two sources agreed. That is the epigraph, and it is also why the gain is not a knob: K is nothing but the fraction of the total precision that the prediction contributed, so choosing it means choosing $\bar{\sigma}^2$ and σ_z^2 , which are physical claims about a motor and a beacon.

💡 Σ , AND THEREFORE K , NEVER SEES THE DATA

Look at the four equations again: $\bar{\Sigma}_t$, S_t , $\mathbf{K}_t$, and Σ_t are built from $\mathbf{A}$, $\mathbf{C}$, $\mathbf{R}$, $\mathbf{Q}$

and the previous Σ — and nothing else. No z appears anywhere. The covariance recursion is a deterministic matrix difference equation (a Riccati recursion), so the entire gain sequence can be computed before the robot is switched on, and for a time-invariant system it converges to a constant. This is why K settles in the tuning bench, and it is a genuine engineering opportunity: precompute the gains offline, ship a table.

It is also the first thing Chapter 7 destroys. As soon as $\mathbf{C}_t$ becomes a Jacobian evaluated at $\tilde{\mu}_t$, the covariance depends on the estimate, the estimate depends on the data, and the whole precomputation collapses.

6.3.5 The inversion lemma

Both derivations leaned on one identity. It is Thrun's Table 3.2, and it is more usually called the Sherman–Morrison–Woodbury formula.

Lemma. For invertible $\mathbf{U}$ and $\mathbf{V}$ and any $\mathbf{P}$ of compatible shape,

$$(\mathbf{U} + \mathbf{PVP}^T)^{-1} = \mathbf{U}^{-1} - \mathbf{U}^{-1}\mathbf{P}(\mathbf{V}^{-1} + \mathbf{P}^T\mathbf{U}^{-1}\mathbf{P})^{-1}\mathbf{P}^T\mathbf{U}^{-1}$$

(Thrun states it with $\mathbf{R}$ and $\mathbf{Q}$ in those two roles; neutral letters here, since in this chapter those symbols are already taken.)

Verification. Multiply the right-hand side by $(\mathbf{U} + \mathbf{PVP}^T)$ and write $\Psi^{-1} = \mathbf{V}^{-1} + \mathbf{P}^T\mathbf{U}^{-1}\mathbf{P}$. The result is

$$\mathbf{I} + \mathbf{U}^{-1}\mathbf{PVP}^T - \mathbf{U}^{-1}\mathbf{P}\Psi(\mathbf{P}^T + \mathbf{P}^T\mathbf{U}^{-1}\mathbf{PVP}^T) = \mathbf{I} + \mathbf{U}^{-1}\mathbf{PVP}^T - \mathbf{U}^{-1}\mathbf{P}\Psi\Psi^{-1}\mathbf{VP}^T = \mathbf{I}$$

using $\mathbf{P}^T + \mathbf{P}^T\mathbf{U}^{-1}\mathbf{PVP}^T = (\mathbf{V}^{-1} + \mathbf{P}^T\mathbf{U}^{-1}\mathbf{P})\mathbf{VP}^T$. ■

The two places this chapter uses it: $\mathbf{U} = \mathbf{R}$, $\mathbf{P} = \mathbf{A}$, $\mathbf{V} = \Sigma$ in the prediction derivation, and $\mathbf{U} = \bar{\Sigma}^{-1}$, $\mathbf{P} = \mathbf{C}^T$, $\mathbf{V} = \mathbf{Q}^{-1}$ in the correction one.

The practical reading: it converts an $n \times n$ inversion into an $m \times m$ one. When a robot carries a 12-dimensional state and receives 2-dimensional range-bearing readings, that is the difference between inverting a 12×12 matrix and a 2×2 one, on every measurement, forever.

6.4 The algorithm

```
ALGORITHM Kalman_filter(_{t-1}, Σ_{t-1}, u_t, z_t) COST  $O(n^3 + m^3 + m n^2)$  per
step -- the  $m^3$  is the inversion of  $\mathbf{S}_t$ , the  $n^3$  the Joseph update
in previous belief ( $\Sigma$ ), control  $u_t$ , measurement  $z_t$ 
out  $_t, \Sigma_t$ 
```

```

1:  $\bar{\mu}_t = \mathbf{A}_t \mu_{t-1} + \mathbf{B}_t u_t$ 
2:  $\bar{\Sigma}_t = \mathbf{A}_t \Sigma_{t-1} \mathbf{A}_t^\top + \mathbf{R}_t$ 
3:  $S_t = \mathbf{C}_t^\top \bar{\Sigma}_t \mathbf{C}_t + \mathbf{Q}_t$ 
4:  $\mathbf{K}_t = \bar{\Sigma}_t \mathbf{C}_t^\top S_t^{-1}$ 
5:  $\mu_t = \bar{\mu}_t + \mathbf{K}_t (z_t - \mathbf{C}_t \bar{\mu}_t)$ 
6:  $\Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t) \bar{\Sigma}_t$ 
7: return  $\mu_t, \Sigma_t$ 

```

Lines 1–2 are prediction, lines 3–6 correction. This is Thrun’s Table 3.1 with the innovation covariance given its own line, because every diagnostic in this chapter needs it. Line 6 is the form that appears in every textbook; the Rust below runs the Joseph variant of it instead, for the reason Step 7 of the derivation gives.

6.5 The other coordinate system

Step 3 of the correction derivation was an aside with a filter hidden in it. Written in the canonical parameters

$$\Omega_t = \Sigma_t^{-1}, \quad \xi_t = \Sigma_t^{-1} \mu_t$$

the entire measurement update is

$$\Omega_t = \bar{\Omega}_t + \mathbf{C}_t^\top \mathbf{Q}_t^{-1} \mathbf{C}_t, \quad \xi_t = \bar{\xi}_t + \mathbf{C}_t^\top \mathbf{Q}_t^{-1} z_t$$

No gain. No innovation covariance. No inversion of anything the size of the state. Two additions.

The reason has a name: Ω is the Hessian of the negative log-belief, and log-densities of independent sources *add*. Fusing k sensors in information form is a fold over k matrix additions, in any order, with no intermediate normalization. That is why large multi-sensor systems are so often written this way.

The price appears on the other step. Prediction, which was a single matrix product in moments form, now requires going out to covariance and back:

$$\bar{\Omega}_t = (\mathbf{A}_t \Omega_{t-1}^{-1} \mathbf{A}_t^\top + \mathbf{R}_t)^{-1}, \quad \bar{\xi}_t = \bar{\Omega}_t (\mathbf{A}_t \Omega_{t-1}^{-1} \xi_{t-1} + \mathbf{B}_t u_t)$$

```

ALGORITHM Information_filter( $\xi_{t-1}$ ,  $\Omega_{t-1}$ ,  $u_t$ ,  $z_t$ ) COST predict  $O(n^3)$ 
-- two inversions; correct  $O(m^3 + m n^2)$  -- additive, no  $n \times n$  inversion
in previous canonical belief ( $\xi$ ,  $\Omega$ ), control  $u_t$ , measurement  $z_t$ 
out  $\xi_t, \Omega_t$ 

```

```

1:  $\bar{\Omega}_t = (\mathbf{A}_t \Omega_{t-1}^{-1} \mathbf{A}_t^\top + \mathbf{R}_t)^{-1}$ 
2:  $\bar{\xi}_t = \bar{\Omega}_t (\mathbf{A}_t \Omega_{t-1}^{-1} \xi_{t-1} + \mathbf{B}_t u_t)$ 
3:  $\Omega_t = \bar{\Omega}_t + \mathbf{C}_t^\top \mathbf{Q}_t^{-1} \mathbf{C}_t$ 
4:  $\xi_t = \bar{\xi}_t + \mathbf{C}_t^\top \mathbf{Q}_t^{-1} z_t$ 
5: return  $\xi_t, \Omega_t$ 

```

INTERACTIVE · w6.2

Moments vs. Information

The information filter is not a different filter. It is the same posterior in the other coordinate system, with the cost of the two steps swapped.

Run this simulation in the web edition: [/chapters/ch06-kalman-filters #w6.2](#)

Operation	Moments (μ, Σ)	Information (ξ, Ω)
Prediction	one product plus an addition — cheap	two $n \times n$ inversions — expensive
Correction	innovation, gain, Joseph update — expensive	one addition per sensor — cheap
Fusing k sensors	k full updates, order matters numerically	k additions, order irrelevant
Marginalizing a variable	delete a row and column of Σ	Schur complement — real work
Conditioning on a variable	real work	delete a row and column of Ω
Natural prior for "I know nothing"	$\Sigma \rightarrow \infty$, not representable	$\Omega = 0$, perfectly representable

The last three rows are the ones to remember, because they are the reason this section exists. In Chapter 14 a robot builds a map by keeping a covariance over its pose and every landmark, and that matrix becomes dense and enormous. Its inverse, it turns out, is nearly sparse: Ω_{ij} is non-zero only when landmarks i and j were seen from a common pose. Chapter 15 throws away the covariance entirely and works in the sparse information matrix forever, and the "expensive" prediction step stops being a problem because it is never performed — the whole trajectory is solved at once.

NOTE

Thrun's draft also develops the *extended* information filter as a nonlinear

SLAM engine. That lineage is told honestly in Chapter 15, where its descendants live. Here the information filter appears only in its linear form, sized to carry the duality.

6.6 Is the filter telling the truth?

RMSE answers "how wrong is the estimate?" It cannot answer "does the filter know how wrong it is?" — and the second question is the one that predicts a crash. A filter that reports 5 cm of uncertainty while making 50 cm errors will reject the very measurements that could save it, because they fall outside its gate.

Two statistics answer it. Under a correct linear-Gaussian model, the normalized estimation error squared

$$\epsilon_t = (x_t - \mu_t)^\top \Sigma_t^{-1} (x_t - \mu_t)$$

is χ^2 distributed with n degrees of freedom, so $\mathbb{E}[\epsilon_t] = n$. And the normalized innovation squared

$$v_t = (z_t - \mathbf{C}_t \bar{\mu}_t)^\top S_t^{-1} (z_t - \mathbf{C}_t \bar{\mu}_t)$$

is χ^2 with m degrees of freedom, so $\mathbb{E}[v_t] = m$.

The difference between them is operational, and decisive. NEES needs x_t , so it exists only in simulation. NIS needs nothing the robot does not already have, so it runs on the real hardware, in flight, forever. Average either over N steps and compare against the χ^2 envelope with Nn (or Nm) degrees of freedom, divided by N : outside the band, and the filter's covariance is a work of fiction. Above the band means overconfident, which is dangerous; below means conservative, which merely wastes information.

Go back to the tuning bench and run the three failure presets while watching the two meters. Each one moves a single slider away from *balanced*, and the three results are genuinely different diseases. Over a 400-step run, seeded and reproducible, they come out like this:

preset	R (σ_a)	Q (σ_z)	RMSE	mean NEES (want 2)	mean NIS (want 1)
balanced	0.45	0.32	0.175	2.33	1.15
sluggish	0.45	2.40	0.488	1.59	0.07
jittery	4.00	0.32	0.252	1.37	0.84

preset	R (σ_a)	Q (σ_z)	RMSE	mean NEES (want 2)	mean NIS (want 1)
divergent	0.004	0.32	2.069	9186	43.6

Sluggish claims the beacon is seven times noisier than it is, so it discounts every reading, lags every turn, and nearly triples its RMSE. NEES stays in band and NIS collapses to a seventh of its expected value: this filter is not lying, it is *ignoring information*, and NIS is the instrument that says so.

Jittery claims the cart slips nine times harder than it does. The prediction is worthless, so the filter follows the beacon almost exactly and inherits the sensor's noise — its RMSE is about what you would get by plotting the raw measurements. Yet both consistency statistics are fine. This is the case that matters most for how you tune: a filter can be *honest and wasteful*, and no amount of NIS-watching will find it. Only RMSE against known truth will.

Divergent is the dangerous one. Told the cart obeys its model almost perfectly, the covariance shrinks every step, the gain goes to zero, and the filter simply stops updating. Its own claimed σ ends the run at 0.05 m while it is 2 m from the truth. In the seeded run above, a sliding 40-step NIS average leaves its envelope at step 48; the same window's RMSE does not look alarming until step 66. Eighteen steps is not much, but it is the difference between a warning and a post-mortem — and unlike RMSE, NIS is available on the robot.

6.7 Implementation in Rust

The type is where the chapter's assumptions become enforceable. Three const generic parameters — state, control, and measurement dimension — mean that feeding a 4-state filter a 3×2 measurement matrix is a compile error, not a panic during a field trial.

RUST crates/ch06_kalman/src/kf.rs

```

1 use nalgebra::{SMatrix, SVector};
2 use pr_core::prob::Gaussian; // Ch. 2's moments-form belief -- reused, not redeclared
3
4 /// A linear-Gaussian system and the belief it carries.
5 ///
6 /// `N` = state dimension, `U` = control dimension, `M` = measurement dimension.
7 pub struct Kf<const N: usize, const U: usize, const M: usize> {
8     pub a: SMatrix<f64, N, N>, // A_t state transition
9     pub b: SMatrix<f64, N, U>, // B_t control
10    pub c: SMatrix<f64, M, N>, // C_t measurement
11    /// R_t -- **motion** noise, Thrun's convention. Most control texts call this Q.
12    pub r: SMatrix<f64, N, N>,
13    /// Q_t -- **measurement** noise, Thrun's convention. Most control texts call this R.
14    pub q: SMatrix<f64, M, M>,
15    pub belief: Gaussian<N>,
16 }
17
18 /// Everything a correction learned, handed back for gating and diagnostics.
```

```

19 #[derive(Clone, Copy, Debug)]
20 pub struct Update<const N: usize, const M: usize> {
21     pub innovation: SVector<f64, M>,
22     pub s: SMatrix<f64, M, M>,
23     pub gain: SMatrix<f64, N, M>,
24     /// log p(z_t | z_{1:t-1}) = log N(innovation; 0, S) -- Chapter 5's evidence,
25     /// still free, and the same quantity Chapter 11 gates outliers with.
26     pub log_evidence: f64,
27 }
28
29 impl<const N: usize, const U: usize, const M: usize> Kf<N, U, M> {
30     /// Table 3.1, lines 1-2.
31     pub fn predict(&mut self, u: &SVector<f64, U>) {
32         self.belief.mu = self.a * self.belief.mu + self.b * u;
33         // A Sigma A^T is the old uncertainty dragged through the dynamics -- it can
34         // shrink. + R is the uncertainty this step *creates*, and it cannot.
35         self.belief.sigma = self.a * self.belief.sigma * self.a.transpose() + self.r;
36         self.belief.symmetrize();
37     }
38
39     /// Table 3.1, lines 3-6, with the Joseph-form covariance.
40     pub fn correct(&mut self, z: &SVector<f64, M>) -> Update<N, M> {
41         let ct = self.c.transpose();
42         let innovation = z - self.c * self.belief.mu;
43         let s = self.c * self.belief.sigma * ct + self.q;
44         let s_inv = s.try_inverse().expect("S_t singular: a measurement carries no noise");
45
46         let gain = self.belief.sigma * ct * s_inv;
47
48         self.belief.mu += gain * innovation;
49
50         // Joseph form: (I - KC) Sigma (I - KC)^T + K Q K^T. Identical to (I - KC)Sigma
51         // at the optimal gain, but it is a *sum* of two positive-semidefinite
52         // terms rather than a difference, so round-off cannot make it indefinite.
53         let i_kc = SMatrix::<f64, N, N>::identity() - gain * self.c;
54         self.belief.sigma =
55             i_kc * self.belief.sigma * i_kc.transpose() + gain * self.q * gain.transpose();
56         self.belief.symmetrize();
57
58         // Ch. 2's Cholesky-based log-density, evaluated at the innovation.
59         let log_evidence = pr_core::prob::log_normal_pdf(&innovation, &SVector::zeros(),
60             &s);
61         Update { innovation, s, gain, log_evidence }
62     }
63 }
64
65 impl<const N: usize, const U: usize, const M: usize> bayes_core::BayesFilter for Kf<N, U,
66     M> {
67     type Belief = Gaussian<N>;
68     type Control = SVector<f64, U>;
69     type Measurement = SVector<f64, M>;
70
71     fn predict(&mut self, u: &Self::Control) {
72         Kf::predict(self, u)
73     }
74
75     fn correct(&mut self, z: &Self::Measurement) -> f64 {
76         Kf::correct(self, z).log_evidence.exp()
77     }
78 }

```

```

76     fn belief(&self) -> &Self::Belief {
77         &self.belief
78     }
79 }

```

The information filter is the same struct with the belief stored the other way round. Notice how much shorter correct is than predict — the exact inverse of the shape above.

RUST crates/ch06_kalman/src/info.rs

```

1  use nalgebra::{SMatrix, SVector};
2  use pr_core::prob::Gaussian;
3
4  /// The same belief in canonical coordinates: Omega = Sigma^-1, xi = Sigma^-1mu.
5  pub struct InfoFilter<const N: usize, const U: usize, const M: usize> {
6      pub xi: SVector<f64, N>,
7      pub omega: SMatrix<f64, N, N>,
8      pub a: SMatrix<f64, N, N>,
9      pub b: SMatrix<f64, N, U>,
10     pub c: SMatrix<f64, M, N>,
11     pub r: SMatrix<f64, N, N>,
12     pub q: SMatrix<f64, M, M>,
13 }
14
15 impl<const N: usize, const U: usize, const M: usize> InfoFilter<N, U, M> {
16     /// Table 3.4, lines 1-2. Two n*n inversions: the expensive step *here*.
17     pub fn predict(&mut self, u: &SVector<f64, U>) {
18         let sigma = self.omega.try_inverse().expect("Omega must be positive definite");
19         let mu = sigma * self.xi;
20         let sigma_bar = self.a * sigma * self.a.transpose() + self.r;
21         self.omega = sigma_bar.try_inverse().expect("Sigma must be positive definite");
22         self.xi = self.omega * (self.a * mu + self.b * u);
23     }
24
25     /// Table 3.4, lines 3-4, with this filter's own sensor.
26     pub fn correct(&mut self, z: &SVector<f64, M>) {
27         let (c, q) = (self.c, self.q);
28         self.add_information(z, &c, &q);
29     }
30
31     /// No gain, no S, nothing n*n inverted: the measurement's information is
32     /// added in, and that is the entire update. Any sensor of any dimension `K`
33     /// contributes through the same three lines, which is why fusion in
34     /// information form is a fold rather than a special case.
35     pub fn add_information<const K: usize>(
36         &mut self,
37         z: &SVector<f64, K>,
38         c: &SMatrix<f64, K, N>,
39         q: &SMatrix<f64, K, K>,
40     ) {
41         let ct_qinv = c.transpose() * q.try_inverse().expect("Q must be invertible");
42         self.omega += ct_qinv * c;
43         self.xi += ct_qinv * z;
44     }
45
46     pub fn to_moments(&self) -> Gaussian<N> {
47         let sigma = self.omega.try_inverse().expect("Omega must be positive definite");

```

```

48         Gaussian::new(sigma * self.xi, sigma)
49     }
50
51     /// Start from a moments-form belief and the same system matrices as a `Kf`.
52     pub fn from_kf(kf: &Kf<N, U, M>) -> Self {
53         let omega = kf.belief.sigma.try_inverse().expect("Sigma must be positive definite
54         ");
55         Self {
56             xi: omega * kf.belief.mu,
57             omega,
58             a: kf.a,
59             b: kf.b,
60             c: kf.c,
61             r: kf.r,
62             q: kf.q,
63         }
64     }

```

The smoother needs the forward run kept rather than thrown away — which is the first time in this book that a filter’s defining virtue, forgetting the past, is the thing standing in the way.

RUST crates/ch06_kalman/src/smooth.rs

```

1 use nalgebra::{SMatrix, SVector};
2 use pr_core::prob::Gaussian;
3
4 /// One time step of a stored forward run: what the filter believed before the
5 /// measurement, after it, and the A_t that connected them.
6 #[derive(Clone, Copy)]
7 pub struct Step<const N: usize> {
8     pub predicted: Gaussian<N>, // (mu_t, Sigma_t)
9     pub filtered: Gaussian<N>, // (mu_t, Sigma_t)
10    pub a: SMatrix<f64, N, N>,
11 }
12
13 /// Rauch-Tung-Striebel fixed-interval smoother.
14 ///
15 /// One backward pass over a completed run. The middle line says everything:
16 /// correct each state by how wrong its own prediction of the future turned out
17 /// to be, scaled by how much that state was responsible for the prediction.
18 pub fn rts_smooth<const N: usize>(run: &[Step<N>]) -> Vec<Gaussian<N>> {
19     let mut out: Vec<Gaussian<N>> = run.iter().map(|s| s.filtered).collect();
20
21     for t in (0..run.len().saturating_sub(1)).rev() {
22         let next = &run[t + 1];
23         let l = run[t].filtered.sigma
24             * next.a.transpose()
25             * next.predicted.sigma.try_inverse().expect("Sigma must be invertible");
26
27         out[t].mu = run[t].filtered.mu + l * (out[t + 1].mu - next.predicted.mu);
28         out[t].sigma = run[t].filtered.sigma
29             + l * (out[t + 1].sigma - next.predicted.sigma) * l.transpose();
30     }
31     out
32 }
33

```

```

34 // NEES -- needs ground truth, so it exists only in simulation. E[.] = N.
35 pub fn nees<const N: usize>(truth: &SVector<f64, N>, bel: &Gaussian<N>) -> f64 {
36     let d = truth - bel.mu;
37     let omega = bel.sigma.try_inverse().expect("Sigma must be positive definite");
38     (d.transpose() * omega * d)[(0, 0)]
39 }
40
41 // NIS -- needs only the innovation and S, so it runs on the real robot. E[.] = M.
42 pub fn nis<const M: usize>(innovation: &SVector<f64, M>, s: &SMatrix<f64, M, M>) -> f64 {
43     let s_inv = s.try_inverse().expect("S must be invertible");
44     (innovation.transpose() * s_inv * innovation)[(0, 0)]
45 }

```

6.7.1 A worked example you can check by hand

One dimension, one control, one measurement. The cart is commanded to move one metre per step, and a beacon reports its position.

$$\mathbf{A} = \mathbf{B} = \mathbf{C} = 1, \quad \mathbf{R} = 0.25, \quad \mathbf{Q} = 0.5, \quad \text{bel}(x_0) = \mathcal{N}(0, 1)$$

Step one. The control $u_1 = 1$ gives $\bar{\mu}_1 = 0 + 1 = 1$ and $\bar{\Sigma}_1 = 1 + 0.25 = 1.25$. The beacon reports $z_1 = 1.7$, so $S_1 = 1.25 + 0.5 = 1.75$ and

$$K_1 = \frac{1.25}{1.75} = \frac{5}{7} \approx 0.714286$$

The filter moves five sevenths of the way from its prediction to the reading: $\mu_1 = 1 + 0.714286 \times 0.7 = 1.5$, and $\Sigma_1 = (1 - 5/7) \times 1.25 = 2.5/7 \approx 0.357143$. That posterior variance is smaller than the prediction's 1.25 *and* smaller than the measurement's own 0.5 — the epigraph, in arithmetic you can do on paper.

Step two. $u_2 = 1$ gives $\bar{\mu}_2 = 2.5$, $\bar{\Sigma}_2 = 2.5/7 + 0.25 = 4.25/7 \approx 0.607143$. The beacon reports $z_2 = 2.1$, which is *below* the prediction, so $S_2 = 4.25/7 + 0.5 = 7.75/7$ and $K_2 = 4.25/7.75 \approx 0.548387$.

step	$\bar{\mu}$	$\bar{\Sigma}$	z	S	K	μ	Σ
1	1.000000	1.250000	1.7	1.750000	0.714286	1.500000	0.357143
2	2.500000	0.607143	2.1	1.107143	0.548387	2.280645	0.274194

Notice the gain *fell* between the steps, from 0.714 to 0.548, without any instruction. The prediction got better — its variance dropped from 1.25 to 0.607 — so the measurement's share of the total precision shrank. Notice too that Σ is converging: 1, 0.357, 0.274, heading for the fixed point of the Riccati recursion, which for these numbers is exactly $\Sigma_\infty = 0.25$ and $K_\infty = 0.5$. (Substitute: $\bar{\Sigma} = 0.25 + 0.25 = 0.5$, $S = 1$, $K = 0.5$, $\Sigma = 0.5 \times 0.5 = 0.25$. A fixed point you can verify in one line, and the reason the gain readout in the tuning bench stops moving.)

RUST crates/ch06_kalman/examples/cart_1d.rs

```

1 use approx::assert_relative_eq;
2 use ch06_kalman::{InfoFilter, Kf};
3 use nalgebra::{SMatrix, SVector};
4 use pr_core::prob::Gaussian;
5
6 /// The chapter's 1-D cart, built once so the example and the test cannot drift.
7 fn cart() -> Kf<1, 1, 1> {
8     Kf {
9         a: SMatrix::identity(),
10        b: SMatrix::identity(),
11        c: SMatrix::identity(),
12        r: SMatrix::from_element(0.25), // motion noise R
13        q: SMatrix::from_element(0.50), // measurement noise Q
14        belief: Gaussian::new(SVector::zeros(), SMatrix::identity()),
15    }
16 }
17
18 fn main() {
19     let mut kf = cart();
20     for (t, (u, z)) in [(1.0, 1.7), (1.0, 2.1)].into_iter().enumerate() {
21         kf.predict(&SVector::from_element(u));
22         print!("step {}: mu = {:.6} Sigma = {:.6}", t + 1, kf.belief.mu[0], kf.belief.
sigma[(0, 0)]);
23         let up = kf.correct(&SVector::from_element(z));
24         println!(
25             "    K = {:.6}    mu = {:.6}    Sigma = {:.6}",
26             up.gain[(0, 0)], kf.belief.mu[0], kf.belief.sigma[(0, 0)]
27         );
28     }
29 }
30
31 #[test]
32 fn worked_example_ch06_cart_1d() {
33     let mut kf = cart();
34
35     kf.predict(&SVector::from_element(1.0));
36     assert_relative_eq!(kf.belief.mu[0], 1.0, epsilon = 1e-12);
37     assert_relative_eq!(kf.belief.sigma[(0, 0)], 1.25, epsilon = 1e-12);
38
39     let u1 = kf.correct(&SVector::from_element(1.7));
40     assert_relative_eq!(u1.gain[(0, 0)], 5.0 / 7.0, epsilon = 1e-12);
41     assert_relative_eq!(kf.belief.mu[0], 1.5, epsilon = 1e-12);
42     assert_relative_eq!(kf.belief.sigma[(0, 0)], 2.5 / 7.0, epsilon = 1e-12);
43
44     kf.predict(&SVector::from_element(1.0));
45     assert_relative_eq!(kf.belief.mu[0], 2.5, epsilon = 1e-12);
46     assert_relative_eq!(kf.belief.sigma[(0, 0)], 4.25 / 7.0, epsilon = 1e-12);
47
48     let u2 = kf.correct(&SVector::from_element(2.1));
49     assert_relative_eq!(u2.gain[(0, 0)], 4.25 / 7.75, epsilon = 1e-12);
50     assert_relative_eq!(kf.belief.mu[0], 2.280_645_161_290_323, epsilon = 1e-9);
51     assert_relative_eq!(kf.belief.sigma[(0, 0)], 0.274_193_548_387_096_8, epsilon = 1e-9);
52 }
53
54 /// The same two steps in canonical coordinates must land on the same posterior.
55 /// If this ever fails, one of the two derivations above is wrong.
56 #[test]
57 fn information_form_agrees() {

```

```

58     let mut kf = cart();
59     let mut inf = InfoFilter::from_kf(&kf);
60
61     for (u, z) in [(1.0, 1.7), (1.0, 2.1)] {
62         let (u, z) = (SVector::from_element(u), SVector::from_element(z));
63         kf.predict(&u);
64         kf.correct(&z);
65         inf.predict(&u);
66         inf.correct(&z);
67     }
68
69     let moments = inf.to_moments();
70     assert_relative_eq!(moments.mu[0], kf.belief.mu[0], epsilon = 1e-9);
71     assert_relative_eq!(moments.sigma[(0, 0)], kf.belief.sigma[(0, 0)], epsilon = 1e-9);
72 }

```

NOTE

The widget at the top of this chapter runs the same algorithm: `lib/filters/kf.ts` is a line-for-line port of the Rust above, including the Joseph form, and `lib/filters/info.ts` is the port of the information filter. Both reproduce the table you just checked by hand.

One more habit worth building here, because every later chapter reuses it. The book hand-rolls its filters — that is the point — but a hand-rolled filter with no external witness is a hand-rolled filter with an untested opinion. `adskalman` is a dev-dependency of `ch06_kalman` for exactly one purpose: to run the same matrices through somebody else’s implementation and assert agreement to 10^{-9} . It is not in the release build and never appears in a listing outside a `#[cfg(test)]` block. It is the cheapest available insurance against a transposed matrix — the class of bug that produces plausible numbers rather than a crash.

The failure gallery in the tuning bench is not hand-tuned for the widget either. It lives in the library, so the prose, the simulation, and the tests all cite the same four numbers.

RUST `crates/ch06_kalman/src/tuning.rs`

```

1  /// A claim about how noisy the world is. The cart on the bench really has
2  /// `accel_sigma = 0.45` and `range_sigma = 0.32`; every other preset is a lie
3  /// of a particular shape.
4  #[derive(Clone, Copy, Debug)]
5  pub struct Tuning {
6      /// R_t, as a white-acceleration sigma in m/s2.
7      pub accel_sigma: f64,
8      /// Q_t, as a range sigma in m.
9      pub range_sigma: f64,
10 }
11
12 pub const BALANCED: Tuning = Tuning { accel_sigma: 0.45, range_sigma: 0.32 };
13 /// Q enormous: the beacon is discounted, the estimate lags, NIS collapses.

```



```

14 pub const SLUGGISH: Tuning = Tuning { accel_sigma: 0.45, range_sigma: 2.40 };
15 // R enormous: the model is discarded and the estimate inherits sensor noise.
16 // Honest -- both consistency tests pass -- and wasteful.
17 pub const JITTERY: Tuning = Tuning { accel_sigma: 4.00, range_sigma: 0.32 };
18 // R ~ 0 while the cart really slips: Sigma collapses and the filter stops listening.
19 pub const DIVERGENT: Tuning = Tuning { accel_sigma: 0.004, range_sigma: 0.32 };
20
21 #[test]
22 fn divergence_is_caught_by_nis_before_rmse() {
23     let seeded = crate::bench::run(DIVERGENT, /* steps */ 400, /* seed */ 7);
24     assert!(seeded.first_nis_out_of_band < seeded.first_rmse_alarm);
25 }

```

6.8 Putting it together: waiting for the future

A filter is causal by construction. Its estimate at time t has seen $z_{1:t}$ and nothing more, because that is what "online" means. But once a run is over — a logged dataset, a mapping session, a post-flight analysis — the constraint is gone. The state at $t = 40$ can be re-estimated using the measurements from $t = 41$ onwards, and it should be, because those measurements are informative about it.

DERIVATION The RTS smoother, from a conditional Gaussian

$$L_t = \Sigma_t A_{t+1}^T \bar{\Sigma}_{t+1}^{-1}$$

Write $\mu_{t|T}, \Sigma_{t|T}$ for the belief about x_t given *all* T measurements, and keep μ_t, Σ_t for the filtered ones.

Step 1 — the joint of two consecutive states, given the past. Conditioned on $z_{1:t}$, the pair (x_t, x_{t+1}) is jointly Gaussian, because $x_{t+1} = \mathbf{A}_{t+1}x_t + \mathbf{B}u + \varepsilon$ is a linear function of x_t plus independent noise. Its mean is $(\mu_t, \bar{\mu}_{t+1})$, and its covariance is built from three blocks we already have:

$$\text{Cov}(x_t) = \Sigma_t, \quad \text{Cov}(x_t, x_{t+1}) = \Sigma_t \mathbf{A}_{t+1}^T, \quad \text{Cov}(x_{t+1}) = \bar{\Sigma}_{t+1}$$

The middle one is the load-bearing one, and it comes for free: $\text{Cov}(x_t, \mathbf{A}_{t+1}x_t + \mathbf{B}u + \varepsilon) = \Sigma_t \mathbf{A}_{t+1}^T$, because ε is independent of x_t .

Step 2 — condition on x_{t+1} . The conditional-Gaussian identity gives

$$p(x_t | x_{t+1}, z_{1:t}) = \mathcal{N}(x_t; \mu_t + \mathbf{L}_t(x_{t+1} - \bar{\mu}_{t+1}), \Sigma_t - \mathbf{L}_t \bar{\Sigma}_{t+1} \mathbf{L}_t^T)$$

with the **smoother gain** $\mathbf{L}_t = \Sigma_t \mathbf{A}_{t+1}^T \bar{\Sigma}_{t+1}^{-1}$.

Step 3 — the future adds nothing more. By the Markov assumption, $z_{t+1:T}$ is conditionally independent of x_t given x_{t+1} , so

$$p(x_t \mid x_{t+1}, z_{1:T}) = p(x_t \mid x_{t+1}, z_{1:t})$$

This is the step that makes a backward *recursion* possible at all: everything the future has to say about x_t arrives through x_{t+1} .

Step 4 — average over the smoothed successor. By the tower rule, with the expectation taken over $x_{t+1} \sim \mathcal{N}(\mu_{t+1|T}, \Sigma_{t+1|T})$:

$$\mu_{t|T} = \mathbb{E}[\mu_t + \mathbf{L}_t(x_{t+1} - \bar{\mu}_{t+1})] = \mu_t + \mathbf{L}_t(\mu_{t+1|T} - \bar{\mu}_{t+1})$$

and by the law of total covariance,

$$\Sigma_{t|T} = \underbrace{\Sigma_t - \mathbf{L}_t \bar{\Sigma}_{t+1} \mathbf{L}_t^\top}_{\mathbb{E}[\text{Cov}(x_t|x_{t+1})]} + \underbrace{\mathbf{L}_t \Sigma_{t+1|T} \mathbf{L}_t^\top}_{\text{Cov}(\mathbb{E}[x_t|x_{t+1}])} = \Sigma_t + \mathbf{L}_t(\Sigma_{t+1|T} - \bar{\Sigma}_{t+1}) \mathbf{L}_t^\top$$

Initialize at $t = T$ with the filtered belief — the last state has no future to borrow from — and run backwards. ■

Since $\Sigma_{t+1|T} \preceq \bar{\Sigma}_{t+1}$ always, the correction term is negative semidefinite: **smoothing never increases a covariance**. The improvement is largest where $\bar{\Sigma}_{t+1}$ was largest, which is to say exactly where the filter was least sure.

INTERACTIVE · w6.4

Smoother Rewind

Filtering is the best causal use of the data, not the best use of it. Interior states do strictly better with hindsight.

Run this simulation in the web edition: </chapters/ch06-kalman-filters #w6.4>

That widget is this book's longest-range foreshadowing. The smoothed trajectory is the posterior over *all* states given *all* measurements, $p(x_{0:T} \mid z_{1:T}, u_{1:T})$ — and computing it took a forward pass, a stored history, and a backward pass. Chapter 15 computes the same posterior a completely different way: assemble every measurement and every motion as a constraint in one large sparse system, and solve it. When the models are linear and Gaussian the two answers are identical to machine precision. When they are not — which is always, on a real robot with rotations in its state — the batch method can iterate to convergence and the filter cannot. That is the argument that ended the filtering era in SLAM, and you have just seen its first move.

NOTE

The natural next questions are what happens when $\mathbf{A}_t$ and $\mathbf{C}_t$ stop being matrices and start being nonlinear functions (Chapter 7), and what happens when a single Gaussian genuinely cannot describe the belief — three identical doors, a kidnapped robot (Chapter 8). Both keep every equation on this page and change exactly one thing.

6.9 Exercises

1 FOUNDATION • Precisions add

Derive $K = \bar{\sigma}^2 / (\bar{\sigma}^2 + \sigma_z^2)$ directly, by completing the square in the 1-D exponent $\frac{1}{2}(z - x)^2 / \sigma_z^2 + \frac{1}{2}(x - \bar{\mu})^2 / \bar{\sigma}^2$ rather than by specializing the matrix result. Then show $1/\sigma^2 = 1/\bar{\sigma}^2 + 1/\sigma_z^2$, and use it to reproduce both rows of the chapter's numeric table with a calculator. <details> <summary>Check</summary> Your second row should give $K_2 = 4.25/7.75 = 0.548387$ and $\Sigma_2 = 0.274194$. If you get $\Sigma_2 = 0.208333$ you skipped the $+\mathbf{R}$ in the second prediction; if you get $\Sigma_2 = 0.375$ you predicted from $\bar{\Sigma}_1$ instead of Σ_1 . </details>

2 FOUNDATION •• The gain never sees the data

Prove that Σ_t , and therefore $\mathbf{K}_t$, depends only on $\mathbf{A}_{1:t}$, $\mathbf{C}_{1:t}$, $\mathbf{R}_{1:t}$, $\mathbf{Q}_{1:t}$ and Σ_0 — never on $\mathbf{z}_{1:t}$. What does this let you precompute before the robot is switched on? Then state precisely which line of the derivation breaks when $\mathbf{C}_t$ is replaced by a Jacobian evaluated at $\bar{\mu}_t$, as it will be in Chapter 7.

3 FOUNDATION ••• Count the flops and pick a side

Using the inversion lemma, show that the information filter's correct step and the Kalman filter's correct step produce the same posterior. Then count multiply-adds for both forms of both steps at $n = 13$, $m = 2$ (a 3-DoF pose plus five 2-D landmarks, observed by a range-bearing sensor — the Chapter 14 state, in miniature), and again at $n = 13$ with six independent 2-D sensors fused per step. Which form wins in each case, and what would have to be true of Ω for the answer to flip again? The ledger in w6.2 prices the same formula at $m = 1$; drag its dimension slider to 13 and check that your totals have the same shape.

4 CONCEPTUAL • Predict, then verify

In the Kalman Tuning Bench, write down your predictions *before* touching anything: if $\mathbf{Q}$ is multiplied by 100, what happens to (a) the steady-state gain $\mathbf{K}$, (b) the RMSE, (c)

the mean NIS? Now do it. Two of the three probably surprised you; explain why NIS moves in the direction it does.

5 CONCEPTUAL •• Find the honest failure

Switch the bench to sweep mode. Find a region of the $(\mathbf{R}, \mathbf{Q})$ grid where the mean NIS is inside its envelope but the RMSE is poor, and a second region where RMSE is good but NIS is far outside. Explain what each region means physically, and say which of the two filters you would rather deploy on a robot that has to gate outliers.

6 PRACTICAL •• Break the covariance on purpose

Implement the naive update $\Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{C}_t) \bar{\Sigma}_t$ alongside the Joseph form. Run the 1-D cart for 10^6 steps in f32, logging the smallest eigenvalue of Σ every thousand steps. Plot both. Then repeat with a deliberately suboptimal gain ($0.9\mathbf{K}_t$) and report which form survives, and why the Joseph form's algebraic structure predicts the result.

7 PRACTICAL ••• Smooth a logged run

Log a 2-D constant-velocity run in the Apartment (Chapter 4) with $N = 4$ (position and velocity) and $M = 2$ (beacon position fixes only). Confirm that the filter recovers velocity, a state it never measures, through the cross-covariance terms — plot Σ_{13} over time and explain its shape. Then run `rts_smooth` over the stored trajectory and report filtered vs. smoothed RMSE plus the covariance ratio at mid-trajectory. Cross-check the whole thing against `adskalman`'s implementation in a test; a disagreement above 10^{-9} is a bug in one of you.

6.10 References

Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems *Journal of Basic Engineering* 82(1), 35–45.

The original. Worth reading for how little it resembles the modern presentation: Kalman derives the estimator from orthogonal projection, not from Bayes rule, and the word Gaussian barely appears. doi:10.1115/1.3662552

Rauch, H. E., Tung, F., and Striebel, C. T. (1965). Maximum Likelihood Estimates of Linear Dynamic Systems *AIAA Journal* 3(8), 1445–1450.

The smoother in the last section of this chapter, in its original form. Note the framing: maximum likelihood over the whole trajectory — which is precisely the view Chapter 15 returns to. doi:10.2514/3.3166

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 3 is the source of this chapter's notation, the Table 3.1 and 3.4 algorithms, the completing-the-square derivation, and the inversion lemma. [link](#)

Bar-Shalom, Y., Li, X. R., and Kirubarajan, T. (2001). *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software* Wiley-Interscience.

Where NEES and NIS come from, with the χ^2 acceptance regions and the multi-run averaging this chapter's consistency meters use. The standard reference for filter tuning done as engineering rather than by eye. doi:10.1002/0471221279

Barfoot, T. D. (2024). *State Estimation for Robotics* Cambridge University Press, 2nd edition.

The modern robotics treatment, and the source for this chapter's RTS presentation. Barfoot derives the filter as a special case of batch estimation rather than the other way round — the reordering this book's Part V follows. [link](#)

Ortiz, J., Evans, T., and Davison, A. J. (2021). A Visual Introduction to Gaussian Belief Propagation *arXiv:2107.02308*.

What the information form becomes when you stop assuming a chain: message passing on a sparse Ω . The interactive figures are the best answer to “why would anyone accept the information filter's expensive prediction step?” [link](#)

Tracy, K. S. (2022). A Square-Root Kalman Filter Using Only QR Decompositions *arXiv:2208.06452*.

The numerical-hygiene chapter this one only gestures at. Propagating a Cholesky factor of Σ instead of Σ doubles the working precision and makes the positive-definiteness that Joseph form defends structurally impossible to lose. [link](#)

Revach, G., Shlezinger, N., Ni, X., Escoriza, A. L., van Sloun, R. J. G., and Eldar, Y. C. (2022). KalmanNet: Neural Network Aided Kalman Filtering for Partially Known Dynamics *IEEE Transactions on Signal Processing* 70, 1532–1547.

What happens when the gain stops being derivable because R and Q are unknown: learn K with a recurrent network while keeping the rest of the recursion intact. Chapter 25 differentiates through this chapter's Kf to get there. doi:10.1109/TSP.2022.3158588

Beyond Linearity: EKF, UKF, and Manifolds

THE BAYES FILTER FAMILY · Advanced · 65 min



The goodness of the linearization also depends on the degree of uncertainty. The less certain the robot, the wider its Gaussian belief, and the more it is affected by nonlinearities in the state transition and measurement functions.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 3

Chapter 6 gave us a filter that is exact, optimal, and closed under its own recursion — provided the world is linear. Rusty's world is not. He turns, and a turn enters the motion model through a cosine; he ranges to a beacon, and range enters the measurement model through a square root. From the first turn onward, the true posterior is not a Gaussian and never will be again.

Everything in this chapter is a strategy for pretending anyway, and the strategies differ in how honest their pretence is. The EKF replaces the curve with its tangent line and inherits an error it does not report. The unscented transform sends a handful of scouts through the real function and refits, which is both cheaper to derive and more accurate. And then there is the failure that neither of those addresses at all: a heading is not a real number, so a filter that adds a Kalman correction to one is not approximating badly — it is computing something meaningless. The repair for that is not a better approximation but a better *state space*, and it is where this book leaves 2005 behind for good.

🕒 AFTER THIS CHAPTER YOU CAN

- Derive the EKF as the Kalman filter with two Taylor expansions, and say precisely what those expansions throw away

- Quantify linearization error as curvature times spread, and measure it in a live widget
- Derive the unscented transform and show it matches the true mean to third order with no Jacobians at all
- State the retraction axioms for `boxplus` and `boxminus`, and rewrite every Kalman equation in terms of them
- Build an error-state EKF on $SE(2)$ that never linearizes the trajectory, only the error
- Explain, with one worked fact, why an invariant error definition removes the estimate from the Jacobians

ASSUMED

- $SE(2)$, `exp` and `log`, the adjoint (Chapter 3)
- Every Kalman filter equation, Joseph form, NEES and NIS (Chapter 6)
- Transformations of Gaussians and the Cholesky factor (Chapter 2)
- Rust traits, `const` generics, and `nalgebra` fixed-size types

7.1 Two different ways to be wrong about a turn

Take the Kalman filter of Chapter 6, point it at Rusty's pose (x, y, θ) , and hand it a compass. Drive Rusty in a circle. Two things go wrong, and they are not the same thing.

The first is quantitative. The motion model $x_t = g(u_t, x_{t-1})$ moves the robot along an arc, and arcs are curved: propagating a covariance through the *tangent* to that arc systematically misplaces the mean and misreports the spread. The error is small when the robot is confident and grows without limit as it becomes uncertain. It is an approximation with a size, and the size can be computed.

The second is categorical. Halfway round the circle Rusty's heading passes π and the compass — like every real compass — starts reporting $-\pi$. The filter computes an innovation $z_t - \mu_t$ of nearly -2π , applies a gain of about a half, and swings its estimate a hundred and eighty degrees the wrong way round the world. No amount of better linearization touches this. It is not an approximation error at all; it is the arithmetic being wrong.

The chapter deals with them in that order.

INTERACTIVE · w7.1

The Linearization Lens

Linearization error is not a constant nuisance — it is curvature $\times$ spread, and you can

watch both terms move it.

Run this simulation in the web edition: [/chapters/ch07-ekf-ukf-manifolds #w7.1](#)

Watch the sweep for a few passes before reading on. Three things are worth noticing.

The tangent moves. The EKF does not linearize once, globally; it re-fits its tangent at the current mean at every single step. That is its great strength — a fresh, locally-valid model each update — and the source of its most dangerous failure, because the quality of that model depends on where the mean happens to be, and the mean is the thing you are unsure about.

The scouts bend with the curve. The three green points are not samples. They are chosen deterministically so that their weighted mean and covariance reproduce the blue input exactly, and they are pushed through the *true* function. Nothing is ever differentiated.

The error has two factors, not one. Slide the operating point into a flat region and the orange and gray curves merge no matter how wide the input is. Park it in the bend and widen the input, and the gap opens as σ^2 . Neither curvature nor spread alone predicts anything; their product does.

💡 THE ONE SENTENCE VERSION

The EKF's error is the second-order Taylor term it discards, whose size is curvature times spread; the unscented transform removes the first factor by never differentiating, and the manifold formulation removes an entirely different error that neither of them ever addressed.

7.2 The mathematics

7.2.1 The nonlinear Gaussian system

Everything below concerns a system of the form

$$x_t = g(u_t, x_{t-1}) + \varepsilon_t, \quad z_t = h(x_t) + \delta_t$$

with $\varepsilon_t \sim \mathcal{N}(0, R_t)$ and $\delta_t \sim \mathcal{N}(0, Q_t)$ independent across time, exactly as in Chapter 6 except that g and h are now arbitrary smooth functions instead of matrices.

That single change destroys the property the Kalman filter was built on. A Gaussian pushed through a linear map is Gaussian; pushed through anything else it is not. So $\text{bel}(x_t)$ is non-Gaussian from the first prediction step, and every filter in this chapter is a decision about *which* Gaussian to pretend with.

$g(u_t, x_{t-1}), h(x_t)$	Nonlinear motion and measurement functions, replacing Chapter 6's A_t, B_t, C_t .
$G_t = \partial g / \partial x_{t-1}$	Motion Jacobian, evaluated at the previous mean. Thrun writes g' .
$H_t = \partial h / \partial x_t$	Measurement Jacobian, evaluated at the predicted mean.
$\mathcal{X}^{[i]}, w_m^{[i]}, w_c^{[i]}$	Sigma points and their mean / covariance weights.
$\alpha_{UT}, \beta, \kappa, \lambda$	Unscented parameters; $\lambda = \alpha^2(n+\kappa) - n$. Subscripted UT throughout the book so it never collides with the motion-noise $\alpha_1 \dots \alpha$ of Chapter 9.
$\mathcal{M}, d$	The manifold the state lives on, and its tangent dimension.
$x \boxplus \delta, y \boxminus x$	Retraction and its inverse: move along the manifold by a tangent vector, and the tangent vector between two points.
$\varepsilon_t = x_t \boxminus \mu_t$	The error state — the quantity an error-state filter actually estimates.

7.2.2 The extended Kalman filter

The move is the smallest one that could possibly work: replace g and h by their first-order Taylor expansions about the best point available, then run Chapter 6 unchanged.

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + G_t (x_{t-1} - \mu_{t-1}), \quad G_t = \left. \frac{\partial g(u_t, x)}{\partial x} \right|_{x=\mu_{t-1}}$$

$$h(x_t) \approx h(\bar{\mu}_t) + H_t (x_t - \bar{\mu}_t), \quad H_t = \left. \frac{\partial h(x)}{\partial x} \right|_{x=\bar{\mu}_t}$$

The choice of expansion point matters and is not arbitrary: for a Gaussian, the most likely state is the mean, so the linearization is at least correct where the belief says the state most probably is. Thrun et al. make exactly this argument in §3.3.1, and the widget above is its refutation whenever the belief is wide.

DERIVATION The EKF is Chapter 6 with two substitutions

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$$

Step 1 — the prediction integral. Chapter 6 evaluated

$$\overline{\text{bel}}(x_t) = \int p(x_t | u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$$

by observing that the integrand is a Gaussian in the joint variable (x_t, x_{t-1}) ,

and that marginalizing a joint Gaussian is a linear-algebra identity. Substituting the linearized g makes $p(x_t | u_t, x_{t-1})$ Gaussian in that joint variable again:

$$p(x_t | u_t, x_{t-1}) \approx \mathcal{N}(x_t; g(u_t, \mu_{t-1}) + G_t(x_{t-1} - \mu_{t-1}), R_t)$$

Step 2 — read off the prediction. Every step of Chapter 6's derivation now goes through with $A_t \mapsto G_t$, and completing the square gives

$$\bar{\mu}_t = g(u_t, \mu_{t-1}), \quad \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t$$

Notice the asymmetry, because it is the single most important line in this chapter: **the mean propagates through the true g ; only the covariance goes through the linearization.** The EKF is not "the Kalman filter on a linearized system" — that would put $g(u_t, \mu_{t-1}) + G_t \cdot 0$ in the mean, which is the same thing here, but the distinction becomes real the moment you iterate.

Step 3 — the correction. The measurement update is a product of Gaussians. With the linearized h the second factor is Gaussian in x_t , and the same completing-the-square algebra returns

$$S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t, \quad K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$$

$$\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t)), \quad \Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$$

Again the innovation uses the true $h(\bar{\mu}_t)$, not $H_t \bar{\mu}_t$.

Step 4 — what was consumed. Nothing in Steps 1–3 is an approximation *given* the linearized models; all the approximation was spent in the two Taylor expansions. That is why the EKF has no optimality property whatsoever. The Kalman filter is the exact posterior of a linear-Gaussian system; the EKF is a heuristic that happens to be an extremely good one when the neglected terms are small, and gives no warning when they are not. ■

ALGORITHM `Extended_Kalman_filter(_{t-1},  $\Sigma_{t-1}$ ,  $u_t$ ,  $z_t$ )` COST $O(d^3)$ per step in the state dimension d , plus two Jacobian evaluations

in previous mean and covariance, control, measurement

out $_t, \Sigma_t$

1: $\bar{\mu}_t = g(u_t, \mu_{t-1})$

```

2:  $G_t = \partial g(u_t, x) / \partial x \big|_{\mu_{t-1}}$ 
3:  $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t$ 
4:  $H_t = \partial h(x) / \partial x \big|_{\bar{\mu}_t}$ 
5:  $S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t$ 
6:  $K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$ 
7:  $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$ 
8:  $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$ 
9: return  $\mu_t, \Sigma_t$ 

```

That is Thrun et al.'s Table 3.3 verbatim. Lines 1 and 7 are the only places the true nonlinear functions appear; lines 3, 5, 6 and 8 are Chapter 6 with different letters.

7.2.3 The size of the lie

The widget's error readout is not a heuristic. It implements the following.

DERIVATION Linearization bias is curvature times spread

$$\mathbb{E}[g(x)] - g(\mu) \approx \frac{1}{2} \sum_i e_i \operatorname{tr}(\nabla^2 g_i \Sigma)$$

Let $x \sim \mathcal{N}(\mu, \Sigma)$ and expand a scalar component g_i to second order about μ , writing $\Delta = x - \mu$:

$$g_i(x) = g_i(\mu) + \nabla g_i(\mu)^\top \Delta + \frac{1}{2} \Delta^\top \nabla^2 g_i(\mu) \Delta + O(\|\Delta\|^3)$$

Step 1 — take expectations. $\mathbb{E}[\Delta] = 0$ kills the linear term outright. The EKF keeps exactly the terms that survive at first order, so its predicted mean is $g_i(\mu)$.

Step 2 — the quadratic term does not vanish. Using $\mathbb{E}[\Delta^\top M \Delta] = \operatorname{tr}(M \Sigma)$ for symmetric M ,

$$\mathbb{E}[g_i(x)] = g_i(\mu) + \frac{1}{2} \operatorname{tr}(\nabla^2 g_i(\mu) \Sigma) + O(\|\Sigma\|^2)$$

Step 3 — read it. The bias the EKF commits is $-\frac{1}{2} \operatorname{tr}(\nabla^2 g_i \Sigma)$ per output component: the Hessian of the model contracted against the covariance of the belief. Curvature times spread, and *linear in the covariance* — so doubling σ quadruples the bias. The third and higher terms vanish for symmetric noise or contribute at $O(\sigma^4)$.

In one dimension this collapses to the readout the widget prints beside the measured error:

$$\mathbb{E}[g(x)] - g(\mu) \approx \frac{1}{2} g''(\mu) \sigma^2$$

A second, quieter failure. The same expansion applied to the covariance gives $\text{Var}[g(x)] = g'(\mu)^2 \sigma^2 + O(\sigma^4)$, so the EKF's reported spread is right to leading order — *unless* $g'(\mu) = 0$, in which case the leading term is zero and the EKF reports almost no uncertainty at all while the truth is entirely second order. Select the range-to-beacon curve in the widget and park the operating point at closest approach to see a filter confidently claim a precision it does not have. That combination, a small reported covariance around a biased mean, is the classic recipe for EKF divergence. ■

7.2.4 The unscented transform

The EKF's problem is that it approximates the *function*. The unscented transform approximates the *distribution* instead, and then uses the function exactly.

The idea, due to Julier and Uhlmann: choose a small set of points whose weighted sample mean and covariance reproduce (μ, Σ) exactly, push each one through g , and refit. Nothing is differentiated; g is a black box that may be a lookup table, a ray-caster, or a physics engine.

DERIVATION The 2n+1 point set and why it works

$$\mathcal{X}^{[\pm i]} = \mu \pm (\sqrt{(n + \lambda)\Sigma})_i$$

Step 1 — demand exact moment matching. We want points $\mathcal{X}^{[i]}$ and weights $w_m^{[i]}, w_c^{[i]}$ with $\sum_i w_m^{[i]} = 1$ and

$$\sum_i w_m^{[i]} \mathcal{X}^{[i]} = \mu, \quad \sum_i w_c^{[i]} (\mathcal{X}^{[i]} - \mu) (\mathcal{X}^{[i]} - \mu)^\top = \Sigma$$

Step 2 — exploit symmetry. Take the points in $\pm$ pairs about μ with equal weights. Every odd central moment of the point set is then exactly zero, matching a Gaussian's, which is what buys the third-order accuracy in Step 5.

Step 3 — place them. Let L be a matrix square root, $LL^\top = \Sigma$ (in practice the Cholesky factor, which is what the Rust below computes). Put

$$\mathcal{X}^{[0]} = \mu, \quad \mathcal{X}^{[\pm i]} = \mu \pm \sqrt{n + \lambda} L_{:,i}, \quad i = 1 \dots n$$

with $\lambda = \alpha_{\text{UT}}^2 (n + \kappa) - n$. Then

$$\sum_i w_c^{[i]} (\mathcal{X}^{[i]} - \mu)(\mathcal{X}^{[i]} - \mu)^\top = 2 \cdot \frac{1}{2(n + \lambda)} (n + \lambda) \sum_i L_{:,i} L_{:,i}^\top = LL^\top = \Sigma$$

using $w^{[i]} = 1/(2(n + \lambda))$ for $i \neq 0$, which forces $w_m^{[0]} = \lambda/(n + \lambda)$ so the weights sum to one. The covariance weight of the center point is corrected by $w_c^{[0]} = w_m^{[0]} + (1 - \alpha_{\text{UT}}^2 + \beta)$, where β folds in prior knowledge of the input's fourth moment; $\beta = 2$ is exact for a Gaussian.

Step 4 — propagate and recombine.

$$\mathcal{Y}^{[i]} = g(\mathcal{X}^{[i]}), \quad \mu' = \sum_i w_m^{[i]} \mathcal{Y}^{[i]}, \quad \Sigma' = \sum_i w_c^{[i]} (\mathcal{Y}^{[i]} - \mu')(\mathcal{Y}^{[i]} - \mu')^\top$$

Step 5 — the accuracy claim. Expand g about μ inside both the true expectation and the weighted sum. The constant and linear terms agree trivially. The quadratic terms agree because the point set reproduces Σ exactly — this is precisely the term the EKF drops. The cubic terms agree because both the Gaussian and the symmetric point set have zero third central moments. The first disagreement is at fourth order, and β exists to reduce it. So the UT captures the true mean through third order, and the covariance through second, against the EKF's first for both. ■

ALGORITHM `Unscented_transform(,  $\Sigma$ ,  $f$ )` COST one Cholesky, $O(n^3)$, plus $2n+1$ evaluations of f . No derivatives.

in a Gaussian and an arbitrary function

out μ', Σ' — the moment-matched Gaussian image

- 1: $\lambda = \alpha_{\text{UT}}^2(n + \kappa) - n$
- 2: $L = \text{chol}(\Sigma)$
- 3: $\mathcal{X}^{[0]} = \mu$; for $i = 1 \dots n$: $\mathcal{X}^{[\pm i]} = \mu \pm \sqrt{n + \lambda} L_{:,i}$
- 4: $w_m^{[0]} = \lambda/(n + \lambda)$; $w_c^{[0]} = w_m^{[0]} + 1 - \alpha_{\text{UT}}^2 + \beta$; $w_m^{[i]} = w_c^{[i]} = 1/(2(n + \lambda))$
- 5: $\mathcal{Y}^{[i]} = f(\mathcal{X}^{[i]})$ for all i
- 6: $\mu' = \sum_i w_m^{[i]} \mathcal{Y}^{[i]}$
- 7: $\Sigma' = \sum_i w_c^{[i]} (\mathcal{Y}^{[i]} - \mu')(\mathcal{Y}^{[i]} - \mu')^\top$
- 8: return μ', Σ'

The unscented Kalman filter is then nothing more than the Bayes filter with the UT used as the Gaussian pushforward engine in both steps.

```

ALGORITHM Unscented_Kalman_filter(_{t-1},  $\Sigma_{t-1}$ ,  $u_t$ ,  $z_t$ )  COST  $O(d^3)$ ;
constant factor  $\approx 2d+1$  model evaluations per step
in previous belief, control, measurement
out  $\mu_t, \Sigma_t$ 

```

```

1:  $(\bar{\mu}_t, \bar{\Sigma}_t) = \text{Unscented\_transform}(\mu_{t-1}, \Sigma_{t-1}, x \mapsto g(u_t, x)); \bar{\Sigma}_t += R_t$ 
2: regenerate  $\mathcal{X}^{[i]}$  from  $(\bar{\mu}_t, \bar{\Sigma}_t)$ 
3:  $\mathcal{Z}^{[i]} = h(\mathcal{X}^{[i]})$ ;  $\hat{z}_t = \sum_i w_m^{[i]} \mathcal{Z}^{[i]}$ 
4:  $S_t = \sum_i w_c^{[i]} (\mathcal{Z}^{[i]} - \hat{z}_t)(\mathcal{Z}^{[i]} - \hat{z}_t)^\top + Q_t$ 
5:  $\bar{\Sigma}_t^{xz} = \sum_i w_c^{[i]} (\mathcal{X}^{[i]} - \bar{\mu}_t)(\mathcal{Z}^{[i]} - \hat{z}_t)^\top$ 
6:  $K_t = \bar{\Sigma}_t^{xz} S_t^{-1}$ 
7:  $\mu_t = \bar{\mu}_t + K_t(z_t - \hat{z}_t)$ 
8:  $\Sigma_t = \bar{\Sigma}_t - K_t S_t K_t^\top$ 
9: return  $\mu_t, \Sigma_t$ 

```

Line 6 is worth a pause. In the EKF the gain is $\bar{\Sigma}_t H_t^\top S_t^{-1}$, and $\bar{\Sigma}_t H_t^\top$ is the linearized approximation of the state–measurement cross-covariance. The UKF computes that cross-covariance directly, from samples. It is the cleanest available statement of what a Kalman gain *is*: how much the state co-varies with the measurement, divided by how much the measurement varies with itself.

⚠ WHERE ROBOTS BREAK THIS

The UKF is not free and it is not universally better. On a linear model it returns exactly the Kalman filter’s answer, having spent $2d + 1$ model evaluations and a Cholesky factorization to do so. Its parameters need care too: α_{UT} small (down to 10^{-3}) keeps the scouts near the mean, which is right for a sharply curved model and wrong for a nearly flat one, and $\beta = 2$ deliberately inflates the output covariance. The chapter’s closing widget maps out where each choice pays.

7.2.5 A worked example you can check by hand

A range–bearing sensor reports polar coordinates and the filter wants Cartesian. Take

$$r \sim \mathcal{N}(1, 0.02^2), \quad \theta \sim \mathcal{N}(\pi/2, (15^\circ)^2), \quad f(r, \theta) = (r \cos \theta, r \sin \theta)$$

independent. Because r and θ are independent and $\mathbb{E}[\sin \theta] = \sin(\pi/2) e^{-\sigma_\theta^2/2}$ for a Gaussian θ , the true mean of the y -component is available in closed form:

$$\mathbb{E}[y] = \mathbb{E}[r] \mathbb{E}[\sin \theta] = e^{-\sigma_\theta^2/2} = e^{-0.0342695} = 0.966311$$

The EKF answers $f(\mu_r, \mu_\theta) = 1.000000$ — off by 3.4 cm at one metre of range, from a sensor whose bearing noise is a perfectly ordinary 15° .

Now the unscented transform with $n = 2$, $\alpha_{\text{UT}} = 1$, $\beta = 0$, $\kappa = 1$, so $\lambda = 1$ and $\sqrt{n + \lambda} = \sqrt{3}$. The covariance is diagonal, so its Cholesky factor is $\text{diag}(0.02, 0.261799)$ and the five points are:

i	$X^{[i]} = (r, \theta)$	$y = r \sin \theta$	$w_m^{[i]}$
0	(1.000000, 1.570796)	1.000000	1/3
1	(1.034641, 1.570796)	1.034641	1/6
2	(0.965359, 1.570796)	0.965359	1/6
3	(1.000000, 2.024246)	0.898941	1/6
4	(1.000000, 1.117346)	0.898941	1/6

$$\frac{1}{3}(1) + \frac{1}{6}(1.034641 + 0.965359) + \frac{1}{6}(0.898941 + 0.898941) = 0.966314$$

which agrees with the analytic 0.966311 to five decimals, from five evaluations of a sine.

The variances tell the same story from the other side. The true standard deviation of y is 0.050680 and the UT gives 0.051667, a 2% overestimate. The EKF's is $((\partial y / \partial r)^2 \sigma_r^2 + (\partial y / \partial \theta)^2 \sigma_\theta^2)^{1/2}$, and at $\theta = \pi/2$ the second derivative is exactly zero, so the whole bearing contribution disappears and it reports 0.020000 — two centimetres of uncertainty where there are five. In the x -component, where the bearing derivative is large, the same first-order formula over-states the spread instead: 0.261799 against a true 0.253129. Getting the variance wrong in both directions at once, from one linearization, is not an unusual outcome.

RUST `crates/ch07_nonlinear/tests/polar.rs`

```

1 use approx::assert_relative_eq;
2 use ch07_nonlinear::ukf::{unscented_transform, UtParams};
3 use nalgebra::{SMatrix, SVector, Vector2};
4 use pr_core::geom::OnManifoldGaussian;
5
6 /// The chapter's worked example, pinned to five decimals.
7 ///
8 /// Note the parameters: beta = 0 here, not the usual 2. beta deliberately inflates
9 /// the output covariance and would spoil a comparison against a closed form,
10 /// which is exactly why the book states it rather than leaving it at a default.
11 #[test]
12 fn polar_to_cartesian_matches_the_closed_form() {
13     let sigma_theta: f64 = 15.0_f64.to_radians();
14     let mean = Vector2::new(1.0, std::f64::consts::FRAC_PI_2);
15     let cov = SMatrix::<f64, 2, 2>::from_diagonal(&Vector2::new(
16         0.02 * 0.02,
17         sigma_theta * sigma_theta,
18     ));

```



```

18     ));
19
20     let polar_to_cartesian =
21       |x: &SVector<f64, 2>| Vector2::new(x[0] * x[1].cos(), x[0] * x[1].sin());
22
23     let prior = OnManifoldGaussian { mean, cov };
24     let params = UtParams { alpha_ut: 1.0, beta: 0.0, kappa: 1.0 };
25     let ut = unscented_transform(&prior, polar_to_cartesian, params);
26
27     //  $E[r \sin \theta] = E[r] \cdot \sin(\pi/2) \cdot \exp(-\sigma^2/2)$  for independent  $r$ ,  $\theta$ .
28     let truth = (-0.5 * sigma_theta * sigma_theta).exp();
29     assert_relative_eq!(truth, 0.966_311, epsilon = 1e-6);
30     assert_relative_eq!(ut.mean[1], truth, epsilon = 1e-5);
31
32     // The EKF's answer, for contrast: the mean pushed straight through  $f$ .
33     let ekf_mean = polar_to_cartesian(&mean);
34     assert_relative_eq!(ekf_mean[1], 1.0, epsilon = 1e-12);
35     assert!((ekf_mean[1] - truth).abs() > 0.033); // 3.4 cm at 1 m
36
37     // And the spread the EKF cannot see, because  $\partial y / \partial \theta = 0$  at  $\theta = \pi/2$ .
38     assert_relative_eq!(ut.cov[(1, 1)].sqrt(), 0.051_667, epsilon = 1e-5);
39     assert_relative_eq!(ut.cov[(0, 0)].sqrt(), 0.252_919, epsilon = 1e-5);
40 }

```

NOTE

The purple output density in the widget at the top of this chapter is computed by `unscentedTransform` in `lib/filters/ukf.ts` — the line-for-line TypeScript port of the Rust above. Run it on this example and it returns 0.966314 and 0.051667, the numbers in the table.

7.3 Where 2005 stops

Everything so far has been about approximating a function well. The rest of this chapter is about a mistake that survives any amount of good approximation.

INTERACTIVE · w7.2

Manifold vs. Vector

Wrap-around is not an edge case to patch with $\text{mod } 2\pi$. It is a type error: the heading was never a real number.

Run this simulation in the web edition: `/chapters/ch07-ekf-ukf-manifolds #w7.2`

Run it until Rusty crosses the seam. The orange filter is not badly approximating anything — its Jacobian is exactly 1, its linearization is exact, and it is still catastrophically wrong. It computed $z_t - \mu_t \approx -2\pi$ and took a fraction of that step, and *both of those operations are meaningless* for an angle. Subtraction is not the

metric on a circle, and a weighted average of two points on a circle is not a point on the circle.

💡 THE PIVOT OF THIS CHAPTER

Wrap-around is not an edge case to be patched with $\text{mod } 2\pi$. It is a type error. Fixing the innovation is a local repair to one symptom; the disease is that the state was declared to be $\mathbb{R}^3$ when it is $\text{SE}(2)$.

7.3.1 On-manifold Gaussians

Definition. An *on-manifold Gaussian* on a d -dimensional manifold $\mathcal{M}$ is a pair (μ, Σ) with $\mu \in \mathcal{M}$ and $\Sigma \in \mathbb{R}^{d \times d}$, representing the distribution of the random element

$$x = \mu \boxplus \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \Sigma)$$

The mean lives on the manifold; the covariance lives in the tangent space *at that mean*. This is sometimes called a concentrated Gaussian, and the name carries the caveat: the construction is only a good model of a distribution when Σ is small enough that the tangent space is a fair picture of the neighbourhood. A heading with a standard deviation of 60° is not well described by any such object, and a filter is the wrong tool for it — that is Chapter 8’s territory.

Definition (retraction). $\boxplus : \mathcal{M} \times \mathbb{R}^d \rightarrow \mathcal{M}$ and $\boxminus : \mathcal{M} \times \mathcal{M} \rightarrow \mathbb{R}^d$ satisfy, for all $x, y \in \mathcal{M}$ and small $\delta \in \mathbb{R}^d$:

$$x \boxplus 0 = x, \quad x \boxplus (y \boxminus x) = y, \quad (x \boxplus \delta) \boxminus x = \delta,$$

with both maps smooth. The first says doing nothing does nothing; the second says $\boxminus$ really does recover the displacement between two points; the third says the pair is locally inverse. That is all a Kalman filter ever needed from $+$ and $-$, which is why the substitution works.

For $\text{SE}(2)$ this book fixes the **right** (local, body-frame) convention throughout:

$$x \boxplus \delta = x \cdot \exp(\delta), \quad y \boxminus x = \log(x^{-1}y)$$

with $\exp$ and $\log$ as derived in Chapter 3, and the tangent ordered translation-first, $\delta = (\delta_x, \delta_y, \delta_\theta)$. A perturbation is something you do *in the robot’s own frame*: drive forward δ_x , slide left δ_y , turn δ_θ . The left convention is equally valid and gives different Jacobians; Appendix C tabulates the correspondence.

Vector spaces are the special case $x \boxplus \delta = x + \delta$, $y \boxminus x = y - x$, which satisfies the axioms trivially. Everything that follows therefore contains Chapter 6 as an instance rather than replacing it.

7.3.2 The on-manifold EKF

Take the EKF algorithm and replace every state-space + with $\boxplus$ and every state-space – with $\boxminus$. Two lines change; nothing else does.

$$\bar{\mu}_t = g(u_t, \mu_{t-1}), \quad \mu_t = \bar{\mu}_t \boxplus K_t (z_t \boxminus h(\bar{\mu}_t))$$

The covariance equations are untouched, because covariances were always tangent-space objects; we simply never said so. What *does* change is the meaning of the Jacobians, which are now derivatives in tangent coordinates:

$$G_t = \left. \frac{\partial (g(u_t, \mu_{t-1} \boxplus \varepsilon) \boxminus g(u_t, \mu_{t-1}))}{\partial \varepsilon} \right|_{\varepsilon=0}$$

Both the input and the output perturbation are tangent vectors, so G_t is an honest $d \times d$ matrix even though $\mathcal{M}$ is not a vector space.

For Rusty’s odometry step $g(u, x) = x \boxplus u$ with $u = (v \Delta t, 0, \omega \Delta t)$, that derivative has a closed form and it is a small gem:

$$G_t = \text{Ad}_{\exp(u)}^{-1}$$

DERIVATION The odometry Jacobian is an adjoint, and does not contain the state

$$G_t = \text{Ad}_{\exp(u)}^{-1}$$

By definition of the adjoint, for any group element T and tangent vector ε ,

$$T^{-1} \exp(\varepsilon) T = \exp(\text{Ad}_{T^{-1}} \varepsilon)$$

Step 1 — perturb the input. The perturbed prediction is

$$(\mu \boxplus \varepsilon) \boxplus u = \mu \exp(\varepsilon) \exp(u)$$

Step 2 — commute the perturbation past the step. Insert $\exp(u) \exp(u)^{-1}$ and apply the adjoint identity with $T = \exp(u)$:

$$\mu \exp(\varepsilon) \exp(u) = \mu \exp(u) [\exp(u)^{-1} \exp(\varepsilon) \exp(u)] = (\mu \boxplus u) \boxplus \text{Ad}_{\exp(u)}^{-1} \varepsilon$$

Step 3 — read off the Jacobian. The right-hand side is the unperturbed prediction $\boxplus$ a tangent vector that is *exactly linear* in ε — no truncation was needed. Therefore

$$(g(u, \mu \boxplus \varepsilon)) \boxminus (g(u, \mu)) = \text{Ad}_{\exp(u)^{-1}} \varepsilon \implies G_t = \text{Ad}_{\exp(u)^{-1}}$$

Step 4 — notice the absence. μ does not appear. Compare the same Jacobian in global coordinates, where the standard textbook EKF puts

$$G_t^{\text{global}} = I_3 + c(\theta) e_3^\top, \quad c(\theta) = (-\Delta_x \sin \theta - \Delta_y \cos \theta, \Delta_x \cos \theta - \Delta_y \sin \theta, 0)^\top$$

with (Δ_x, Δ_y) the body-frame translation of the step and e_3 the third basis vector — so the whole state dependence sits in one column, and that column is a function of the *estimated* heading. The vector EKF's error propagation therefore depends on the very quantity it is uncertain about; the body-frame version does not. Hold on to that — it is the whole of the invariant-filtering idea, and it comes back below. ■

The UKF becomes UKF-M by the same substitution: draw the sigma points in the tangent space and retract them, $\mathcal{X}^{[\pm i]} = \mu \boxplus (\pm \sqrt{(n+\lambda)\Sigma})_i$. The one genuinely new ingredient is the recombination, because a weighted sum of manifold points is not defined. Instead the mean is characterised implicitly, as the point from which the weighted tangent displacements cancel, and found by iteration:

$$\mu' \leftarrow \mu' \boxplus \sum_i w_m^{[i]} (\mathcal{Y}^{[i]} \boxminus \mu')$$

started at $\mathcal{Y}^{[0]}$ and repeated to convergence — three or four passes at filter scale. Everything downstream, including the cross-covariance, uses $\boxminus$ in place of subtraction.

7.3.3 Error-state: linearize the small thing

There is a second, more practical way to arrive in the same place, and it is the one production systems actually use.

Split the state into a **nominal** part $\mu_t \in \mathcal{M}$, which carries the full trajectory and is propagated through the exact nonlinear model with no covariance attached, and an **error state** $\varepsilon_t = x_t \boxminus \mu_t \in \mathbb{R}^d$, which is the only thing the Kalman machinery ever sees. The error is zero-mean by construction and stays small by construction, because every correction is immediately *injected* into the nominal state and the error *reset* to zero.

INTERACTIVE · w7.3

The Error-State Ledger

You never had to linearize the trajectory. Linearize the error instead — its world is small, centered, and almost flat.

Run this simulation in the web edition: [#w7.3](/chapters/ch07-ekf-ukf-manifolds)

The two panels are the argument in one picture. On the left, seven metres of travel and a heading that sweeps the whole circle; on the right, an error that never leaves a hand-sized box. A first-order expansion of the dynamics of the left-hand quantity is a fiction. A first-order expansion of the dynamics of the right-hand quantity is very nearly exact, because the neglected term is again curvature times spread, and the spread here is the *error's*, not the state's.

ALGORITHM `error_state_ekf(_{t-1},  $\Sigma_{t-1}$ ,  $u_t$ ,  $z_t$ )` COST $O(d^3)$; identical to the EKF, plus one exp/log pair
in nominal pose on M , tangent covariance, control, measurement
out $_t, \Sigma_t$ — error state never stored between steps

- 1: $\bar{\mu}_t = g(u_t, \mu_{t-1})$ (*the nominal absorbs the whole nonlinearity*)
- 2: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + R_t$ with G_t in tangent coordinates
- 3: $y_t = z_t \boxminus h(\bar{\mu}_t)$, $S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t$, $K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$
- 4: $\hat{\epsilon}_t = K_t y_t$ (*the ledger entry — a tangent vector*)
- 5: $\mu_t = \bar{\mu}_t \boxplus \hat{\epsilon}_t$ (*inject*)
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t (I - K_t H_t)^\top + K_t Q_t K_t^\top$ (*Joseph form*)
- 7: $\Sigma_t \leftarrow J \Sigma_t J^\top$ with $J = \text{Ad}_{\exp(\hat{\epsilon}_t)}^{-1}$ (*reset*)
- 8: $\hat{\epsilon}_t \leftarrow 0$; return μ_t, Σ_t

Line 7 is the piece most implementations quietly skip. After injecting $\hat{\epsilon}_t$ the remaining error is measured from a *different* nominal state, and the two tangent spaces differ by the adjoint of the injection. For a centimetre-scale correction J is the identity to within rounding, which is why skipping it is usually harmless — and why it stops being harmless in Chapter 14, where a loop closure injects a correction big enough to matter.

7.3.4 Invariant errors, in one fact

Step 4 of the adjoint derivation above deserves to be finished.

Define the error not as a tangent-space difference but as a *group* element,

$$\eta_t = \mu_t^{-1} x_t \in \text{SE}(2)$$

(the left-invariant form; the right-invariant analogue is $x_t \mu_t^{-1}$). Under noiseless body-frame odometry, both the truth and the nominal receive the same increment,

$x_t = x_{t-1} \exp(u)$ and $\mu_t = \mu_{t-1} \exp(u)$, so

$$\eta_t = \exp(u)^{-1} \mu_{t-1}^{-1} x_{t-1} \exp(u) = \exp(u)^{-1} \eta_{t-1} \exp(u)$$

The error evolves **autonomously**: its update law involves only the control, never the estimate μ_{t-1} . That is the group-affine property, and the invariant EKF is what you get by building a filter around it. Its Jacobians are constant along a trajectory, so the linearization point cannot drift away from the truth in the way that poisons a standard EKF, and Barrau and Bonnabel prove that the resulting filter is a locally stable observer with a domain of attraction independent of the trajectory — a guarantee no ordinary EKF has ever had.

We are not deriving the general machinery here; it needs left- and right-Jacobian bookkeeping that would swamp this chapter. But keep the fact, because Chapter 14 reads the EKF-SLAM consistency disaster back through exactly this lens, and Chapter 18 uses invariant filters as production VIO front-ends.

7.4 Implementation in Rust

The design goal is that one filter implementation serve the vector case and the manifold case, with the compiler enforcing the difference. That is a trait.

RUST crates/pr-core/src/geom/manifold.rs

```

1 use nalgebra::SVector;
2
3 /// A manifold you can do estimation on. `D` is the tangent dimension.
4 ///
5 /// Introduced in Chapter 3 and reused unchanged from here to the end of the
6 /// book: Chapter 9 samples motion noise through it, Chapter 15 retracts
7 /// Gauss-Newton steps with it, Chapter 18 instantiates it at SE(3) * R?.
8 ///
9 /// The three retraction axioms are not expressible in the type system, so
10 /// they live in a generic test harness (`manifold_axioms::<X, D>()`) that
11 /// every impl in the workspace is required to pass.
12 pub trait Manifold<const D: usize>: Clone {
13     /// x [+] delta -- move along the manifold by a tangent vector.
14     fn boxplus(&self, delta: &SVector<f64, D>) -> Self;
15
16     /// y [-] x -- the tangent vector taking `rhs` to `self`.
17     fn boxminus(&self, rhs: &Self) -> SVector<f64, D>;
18 }
19
20 /// A vector space is the manifold where [+] is +. This impl is what makes the
21 /// generic EKF below reduce *exactly* to Chapter 6's Kf on a linear model --
22 /// a claim pinned by a test, not by assertion.
23 impl<const D: usize> Manifold<D> for SVector<f64, D> {
24     fn boxplus(&self, delta: &SVector<f64, D>) -> Self {
25         self + delta
26     }
27     fn boxminus(&self, rhs: &Self) -> SVector<f64, D> {
28         self - rhs
29     }
30 }
```

```

31
32 // SE(2) with the book's right/local convention. `SE2`, `exp`, `log` and
33 // `adjoint` are Chapter 3's hand-rolled types, not a dependency.
34 impl Manifold<3> for SE2 {
35     fn boxplus(&self, delta: &SVector<f64, 3>) -> Self {
36         self * SE2::exp(delta)
37     }
38     fn boxminus(&self, rhs: &Self) -> SVector<f64, 3> {
39         (rhs.inverse() * self).log()
40     }
41 }
42
43 // Mean on the manifold, covariance in the tangent space at that mean.
44 #[derive(Clone, Debug)]
45 pub struct OnManifoldGaussian<X: Manifold<D>, const D: usize> {
46     pub mean: X,
47     pub cov: SMatrix<f64, D, D>,
48 }

```

With that in place the EKF is written once. Note that the models are traits too, so a measurement of the wrong dimension is a compile error rather than a runtime surprise.

RUST crates/ch07_nonlinear/src/ekf.rs

```

1 use nalgebra::{SMatrix, SVector};
2 use pr_core::geom::{Manifold, OnManifoldGaussian};
3
4 pub trait MotionModel<X: Manifold<D>, const D: usize> {
5     type Control;
6     // The true nonlinear map. The *mean* goes through this, never through G.
7     fn g(&self, x: &X, u: &Self::Control) -> X;
8     // G_t = ?( g(x [+ epsilon, u] [-] g(x, u) ) / ?epsilon |_0 -- in tangent
9     // coordinates.
10    fn jacobian(&self, x: &X, u: &Self::Control) -> SMatrix<f64, D, D>;
11    fn noise(&self, u: &Self::Control) -> SMatrix<f64, D, D>;
12 }
13
14 pub trait MeasurementModel<X: Manifold<D>, const D: usize, const M: usize> {
15     fn h(&self, x: &X) -> SVector<f64, M>;
16     fn jacobian(&self, x: &X) -> SMatrix<f64, M, D>;
17     fn noise(&self) -> SMatrix<f64, M, M>;
18     // Measurement-space residual. Override for bearings -- the measurement
19     // space can be a manifold too, and usually is when the state is.
20     fn residual(&self, z: &SVector<f64, M>, z_hat: &SVector<f64, M>) -> SVector<f64, M> {
21         z - z_hat
22     }
23 }
24
25
26 pub struct Ekf<X: Manifold<D>, G, H, const D: usize, const M: usize> {
27     pub belief: OnManifoldGaussian<X, D>,
28     pub motion: G,
29     pub meas: H,
30 }
31
32 impl<X, G, H, const D: usize, const M: usize> Ekf<X, G, H, D, M>
33 where
34     X: Manifold<D>,

```

```

33 G: MotionModel<X, D>,
34 H: MeasurementModel<X, D, M>,
35 {
36   pub fn predict(&mut self, u: &G::Control) {
37     let g_t = self.motion.jacobian(&self.belief.mean, u);
38     // The mean goes through the true model; only the covariance is linearized.
39     self.belief.mean = self.motion.g(&self.belief.mean, u);
40     self.belief.cov = g_t * self.belief.cov * g_t.transpose() + self.motion.noise(u);
41   }
42
43   /// Returns the normalized innovation squared -- the NIS gate of Chapter 6.
44   pub fn correct(&mut self, z: &SVector<f64, M>) -> f64 {
45     let h_t = self.meas.jacobian(&self.belief.mean);
46     let y = self.meas.residual(z, &self.meas.h(&self.belief.mean));
47     let s = h_t * self.belief.cov * h_t.transpose() + self.meas.noise();
48     let s_inv = s.try_inverse().expect("innovation covariance must be invertible");
49     let k = self.belief.cov * h_t.transpose() * s_inv;
50
51     // The gain produces a *tangent vector*. On SE(2) adding it to a pose
52     // would not even type-check as geometry; [+] is the only way back.
53     self.belief.mean = self.belief.mean.boxplus(&(k * y));
54
55     let i_kh = SMatrix::<f64, D, D>::identity() - k * h_t;
56     self.belief.cov = i_kh * self.belief.cov * i_kh.transpose()
57       + k * self.meas.noise() * k.transpose();
58
59     (y.transpose() * s_inv * y)[(0, 0)]
60   }
61 }

```

The unscented filter needs no Jacobians and therefore no jacobian methods — only g , h , the noises, and a Cholesky. On a manifold the sigma points are retracted and the mean is found by the iteration derived above.

RUST `crates/ch07_nonlinear/src/ukf.rs`

```

1 use nalgebra::{SMatrix, SVector};
2 use pr_core::geom::{Manifold, OnManifoldGaussian};
3
4 #[derive(Clone, Copy, Debug)]
5 pub struct UtParams {
6   /// Spread of the scouts. Subscripted `_ut` book-wide so it never reads as
7   /// one of the motion-noise alpha's of Chapter 9.
8   pub alpha_ut: f64,
9   pub beta: f64,
10  pub kappa: f64,
11 }
12
13 impl UtParams {
14   /// Julier's defaults: kappa = 3 - n, beta = 2 (exact 4th moment for a Gaussian).
15   pub fn julier(d: usize) -> Self {
16     Self { alpha_ut: 1.0, beta: 2.0, kappa: 3.0 - d as f64 }
17   }
18 }
19
20 /// 2D+1 sigma points, drawn in the tangent space and retracted onto `M`.
21 ///
22 /// The weights are `Vec`s rather than `[f64; 2 * D + 1]`: arithmetic in array

```



```

23 // lengths needs `generic_const_exprs`, which is still unstable. One
24 // allocation per call, hoisted out of the loop by the caller in practice.
25 pub fn sigma_points<X: Manifold<D>, const D: usize>(
26     belief: &OnManifoldGaussian<X, D>,
27     p: UtParams,
28 ) -> (Vec<X>, Vec<f64>, Vec<f64>) {
29     let n = D as f64;
30     let lambda = p.alpha_ut * p.alpha_ut * (n + p.kappa) - n;
31     let scaled = belief.cov * (n + lambda);
32     let l = scaled.cholesky().expect("covariance must be positive definite").l();
33
34     let mut points = Vec::with_capacity(2 * D + 1);
35     points.push(belief.mean.clone());
36     for i in 0..D {
37         let col: SVector<f64, D> = l.column(i).into_owned();
38         points.push(belief.mean.boxplus(&col));
39         points.push(belief.mean.boxplus(&(-col)));
40     }
41
42     let w = 1.0 / (2.0 * (n + lambda));
43     let mut wm = vec![w; 2 * D + 1];
44     let mut wc = wm.clone();
45     wm[0] = lambda / (n + lambda);
46     wc[0] = wm[0] + 1.0 - p.alpha_ut * p.alpha_ut + p.beta;
47     (points, wm, wc)
48 }
49
50 // The weighted mean of manifold points: the fixed point of [-]-then-[+].
51 //
52 // A vector-space UKF gets this from a single weighted sum. On a curved space
53 // there is no closed form, so we iterate -- four passes is convergence to
54 // machine precision for filter-scale covariances.
55 pub fn manifold_mean<X: Manifold<D>, const D: usize>(points: &[X], w: &[f64]) -> X {
56     let mut mean = points[0].clone();
57     for _ in 0..8 {
58         let mut delta = SVector::<f64, D>::zeros();
59         for (p, wi) in points.iter().zip(w) {
60             delta += *wi * p.boxminus(&mean);
61         }
62         mean = mean.boxplus(&delta);
63         if delta.norm() < 1e-12 {
64             break;
65         }
66     }
67     mean
68 }
69
70 // The unscented transform on a manifold: Algorithm `Unscented_transform`,
71 // with [+] for the points and `manifold_mean` for the recombination.
72 pub fn unscented_transform<X, Y, F, const D: usize, const E: usize>(
73     belief: &OnManifoldGaussian<X, D>,
74     f: F,
75     p: UtParams,
76 ) -> OnManifoldGaussian<Y, E>
77 where
78     X: Manifold<D>,
79     Y: Manifold<E>,
80     F: Fn(&X) -> Y,
81 {
82     let (points, wm, wc) = sigma_points(belief, p);
83     let images: Vec<Y> = points.iter().map(&f).collect();

```

```

84     let mean = manifold_mean(&images, &wm);
85
86     let mut cov = SMatrix::<f64, E, E>::zeros();
87     for (y, wi) in images.iter().zip(&wc) {
88         let e = y.boxminus(&mean);
89         cov += *wi * e * e.transpose();
90     }
91     OnManifoldGaussian { mean, cov }
92 }

```

7.4.1 A compile error worth printing

The book promised that the type system would do the bookkeeping Thrun does by hand. Here is it doing it. A compass on a planar robot is a one-dimensional measurement of a three-dimensional state, so its model is ‘MeasurementModel

. Hand correct’ a two-vector:

RUST demos/ch07-demo/src/bin/typecheck_demo.rs

```

1  let mut filter: Ekf<SE2, Unicycle, Compass, 3, 1> = Ekf::new(prior, Unicycle::default(),
2      Compass::new(6f64.to_radians()));
3
4  // The reading came from a range-bearing sensor by mistake.
5  filter.correct(&SVector::<f64, 2>::new(4.10, 0.62));

```

```

error[E0308]: mismatched types
--> demos/ch07-demo/src/bin/typecheck_demo.rs:14:20
14 |         filter.correct(&SVector::<f64, 2>::new(4.10, 0.62));
   |         ~~~~~-^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ expected `&Matrix<f64, Const
   |         <1>, Const<1>, ArrayStorage<f64, 1, 1>`, found `&Matrix<f64, Const<2>,
   |         Const<1>, ArrayStorage<f64, 2, 1>`
   |         arguments to this method are incorrect
   |
= note: expected reference `&SVector<f64, 1>`
        found reference `&SVector<f64, 2>`

```

The same discipline catches the subtler mistake, a Jacobian of the wrong shape:

```

error[E0308]: mismatched types
--> crates/ch07_nonlinear/src/models.rs:88:9
87 |     fn jacobian(&self, _x: &SE2) -> SMatrix<f64, 1, 3> {
   |     ~~~~~-^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ expected `Matrix<f64, Const
   |     <1>, Const<3>, >` because of return type
88 |         SMatrix::<f64, 1, 2>::new(0.0, 0.0)
   |         ~~~~~-^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ expected `1*3` matrix, found `1*2`
   |         matrix

```

A dimension mismatch between a state, its Jacobian, and a measurement is the single most common source of silent filter failure in the wild — it produces a filter that runs, converges to something, and is wrong. Here it does not compile.

7.4.2 The error-state filter, and the test that pins it

RUST crates/ch07_nonlinear/src/eskf.rs

```
1 use nalgebra::{SMatrix, SVector};
2 use pr_core::geom::{Manifold, SE2};
3
4 /// Error-state EKF: a nominal state on the manifold plus a tangent-space
5 /// error that is filtered, injected, and reset every update.
6 pub struct Eskf<X: Manifold<D>, const D: usize> {
7     pub nominal: X,
8     pub cov: SMatrix<f64, D, D>,
9     /// Keep the exact reset Jacobian. Off by default in most codebases;
10    /// on by default here, because Chapter 14 needs to be able to turn it off
11    /// and show the difference.
12    pub exact_reset: bool,
13 }
14
15 impl Eskf<SE2, 3> {
16     ///  $\mu = \mu [+]$   $u$ ,  $\Sigma = G \Sigma G^T + R$  with  $G = \text{Ad\_exp}(u) - 1$  -- no state in
17     /// sight.
18     pub fn predict(&mut self, u: &SVector<f64, 3>, r: &SMatrix<f64, 3, 3>) {
19         let g = SE2::exp(u).inverse().adjoint();
20         self.nominal = self.nominal.boxplus(u);
21         self.cov = g * self.cov * g.transpose() + r;
22     }
23
24     /// The inject-and-reset cycle. `epsilon` is returned, not stored: an
25     /// error state that survives its own update is a bug, and making the
26     /// caller receive it rather than read it is how the API says so.
27     pub fn inject_and_reset(&mut self, epsilon: SVector<f64, 3>) -> SVector<f64, 3> {
28         self.nominal = self.nominal.boxplus(&epsilon);
29         if self.exact_reset {
30             let j = SE2::exp(&epsilon).inverse().adjoint();
31             self.cov = j * self.cov * j.transpose();
32         }
33         epsilon
34     }
35 }
```

RUST crates/ch07_nonlinear/tests/reduction.rs

```
1 /// The generic filter must contain Chapter 6 exactly, not approximately.
2 ///
3 /// On a linear-Gaussian model, `Ekf<SVector<f64, 4>>` and `ch06::Kf` are the
4 /// same algorithm with different type parameters, so they must agree to
5 /// floating-point noise. If this ever fails, the trait abstraction has
6 /// silently changed the mathematics.
7 #[test]
8 fn ekf_reduces_to_kf_on_a_linear_model() {
9     let (mut ekf, mut kf) = constant_velocity_pair();
```

```

16     let mut rng = Pcg64::seed_from_u64(0xC0FFEE);
17
18     for _ in 0..200 {
19         let u = SVector::<f64, 2>::new(0.1, -0.05);
20         let z = simulate_measurement(&mut rng);
21         ekf.predict(&u);
22         kf.predict(&u);
23         ekf.correct(&z);
24         kf.correct(&z);
25
26         assert_relative_eq!(ekf.belief.mean, kf.mean(), epsilon = 1e-12);
27         assert_relative_eq!(ekf.belief.cov, kf.cov(), epsilon = 1e-12);
28     }
29 }
30
31 /// And the manifold impl must obey the retraction axioms, checked on random
32 /// poses rather than argued about in prose.
33 #[test]
34 fn se2_satisfies_the_retraction_axioms() {
35     let mut rng = Pcg64::seed_from_u64(7);
36     for _ in 0..10_000 {
37         let (x, y) = (random_pose(&mut rng), random_pose(&mut rng));
38         let d = random_twist(&mut rng, 0.3);
39
40         assert_relative_eq!(x.boxplus(&SVector::zeros()), x, epsilon = 1e-12);
41         assert_relative_eq!(x.boxplus(&y.boxminus(&x)), y, epsilon = 1e-10);
42         assert_relative_eq!(x.boxplus(&d).boxminus(&x), d, epsilon = 1e-10);
43     }
44 }

```

7.5 Putting it together: the figure-eight lab

cargo run --example figure_eight -p ch07_nonlinear drives Rusty three times around a lemniscate — seven metres wide, sixteen seconds a lap, heading sweeping the whole circle and crossing the $\pm\pi$ seam six times — and runs four filters on the *identical seeded log*: a compass at 10 Hz with $\sigma = 6^\circ$, a beacon fix every second with $\sigma = 25$ cm, and body-frame odometry noise.

All four use the same exact motion function, $\mu \boxplus u$; they differ only in how they represent the state and compute residuals.

filter	RMSE position	RMSE heading	worst heading error	mean NEES
vector EKF, plain –	0.448 m	26.36°	180.0°	43.0
vector EKF, wrapped residual	0.381 m	4.27°	13.9°	3.25
error-state EKF on SE(2)	0.382 m	4.27°	13.9°	3.29
UKF-M on SE(2)	0.385 m	4.27°	13.9°	3.33

Read the last column first. A consistent three-degree-of-freedom filter has a mean NEES of 3; the naive vector EKF reports 43, which is not a filter that is

slightly wrong but a filter whose stated uncertainty is a work of fiction. Its worst heading error is a full 180° : the seam crossing, exactly as w7.2 stages it.

Now read the other three rows, because they say something the chapter would be dishonest to omit: **once the residual is wrapped, all three are the same filter**. In three degrees of freedom, on a well-observed problem, you can hand-patch a vector EKF into agreement with the on-manifold formulation, and plenty of shipped code does. What you cannot do is hand-patch it *reliably* — the wrapped EKF is one forgotten `angleDiff` away from row one, and there is no type, test, or review that catches the omission, whereas the $\boxplus$ formulation cannot express the bug at all.

The manifold filters do start to separate when the belief widens. Re-run with every noise tripled, the beacon fixes six times rarer and the compass down to once every six seconds, and the last three RMSEs stay within 2% of each other while their *consistency* pulls apart: mean NEES 6.90 for the naive filter, 4.65 for the wrapped vector EKF, 3.46 for the error-state SE(2) filter and 4.07 for UKF-M. The on-manifold filters are not more *accurate* here. They are more *honest*, which is the property you actually need when a planner is about to consume the covariance and decide how close to drive to a wall.

NOTE

Both SE(2) filters in that table are `EskfSe2` and `UkfmSe2` from `lib/filters/on-manifold-se2.ts`, the TypeScript port of the Rust above; the numbers come from seed 7 of the run, and the widget in the previous section drives the same `EskfSe2`.

7.6 Choosing a filter

INTERACTIVE · w7.4

Filter Chooser

UKF is not better than EKF. It is better in one corner of a plane, and the corner is set by curvature times spread.

Run this simulation in the web edition: `/chapters/ch07-ekf-ukf-manifolds #w7.4`

The plane has two axes because the bias has two factors. The strip above it has neither, because the question it asks — *is the state even a vector space?* — is not a matter of degree. Answer that one first; then, and only then, argue about sigma points.

Three destinations follow from here. Chapter 11 runs this chapter's error-state EKF as a localizer against a known map and meets the other great EKF killer, data association. Chapter 14 puts the map into the state and watches linearization

error compound into a formal inconsistency that ended the filtering era of SLAM. And Chapter 15 takes the "iterate" region of the chart seriously: re-linearize at the posterior, repeat until it stops moving, and you have stopped writing a filter and started writing Gauss–Newton.

7.7 Exercises

1 FOUNDATION •• The bias, by hand

For $h(x) = x^2/20$ with $x \sim \mathcal{N}(\mu, \sigma^2)$, compute $\mathbb{E}[h(x)]$ exactly and show that the EKF's answer $h(\mu)$ is short by exactly $\sigma^2/20$ — independent of μ , because the curvature is constant. Then set the Linearization Lens to the quadratic and check the predicted-bias readout at $\sigma = 0.5, 1.0$ and 2.0 . Does it scale the way you derived?
 <details>
 <summary>Check your setup</summary> $\mathbb{E}[x^2] = \mu^2 + \sigma^2$, so $\mathbb{E}[h(x)] = h(\mu) + \sigma^2/20$ with no higher-order terms at all — a quadratic is one of the rare functions where the second-order Taylor expansion is not an approximation. </details>

2 FOUNDATION •• The UT is exact for affine maps

Show that for $f(x) = Ax + b$ the unscented transform returns exactly $(A\mu + b, A\Sigma A^T)$ for any α_{UT} and κ with $n + \lambda \neq 0$. Then explain why the β term cannot affect the mean, and construct a scalar example where $\beta = 2$ makes the UKF's reported variance worse than $\beta = 0$ does. (The chapter's polar example is one; say why.)

3 FOUNDATION ••• Where the estimate hides in the Jacobian

Take the unicycle step on $\text{SE}(2)$. (a) Verify by direct computation that $G_t = \text{Ad}_{\exp(u)^{-1}}$ is exactly, not approximately, the tangent-coordinate Jacobian. (b) Write out the global-coordinate Jacobian for the same step and identify every entry that depends on μ_{t-1} . (c) Show that the invariant error $\eta_t = \mu_t^{-1}x_t$ satisfies $\eta_t = \exp(u)^{-1}\eta_{t-1}\exp(u)$ under noiseless odometry, and state in one sentence why a filter built on η cannot suffer from linearization-point drift.

4 CONCEPTUAL •• Predict, then check: the worst seam

On paper first: a prior of 170° with $\sigma = 20^\circ$ meets a compass reading of -170° with $\sigma = 20^\circ$. What heading does the vector update report, and what does the manifold update report? Then pause Manifold vs. Vector with the blue prior arrow near $+170^\circ$, drag the green compass arrow round to about -170° , and read both means off the canvas — the gain tile tells you how far the widget's K is from the idealized one half you assumed.

Now the interesting part. Show that a seam crossing costs the vector filter an instantaneous error of $2\pi \min(K, 1 - K)$, so the single worst step happens at $K = 1/2$

and *not* at the extremes — then find the compass- σ that puts the widget's steady-state gain there. Last, explain why the long-run RMS readout keeps climbing as you raise σ past that point, even though each individual seam crossing is doing less damage.

<details> <summary>Hint</summary> Equal variances give $K = 1/2$, so the vector update lands at the arithmetic mean of 170 and -170 . For the second part: with the prior just under $+\pi$ and the reading just over $-\pi$ the raw innovation is $\approx -2\pi$, so the update lands at $\pi(1 - 2K)$; wrap that against the truth. For the third: what sets how many steps it takes to climb back out? </details>

5 CONCEPTUAL •• Make the ledger lie

The Error-State Ledger claims a mean NEES near 3. Find a noise-scale setting where it is persistently above 5, and diagnose which of the filter's assumptions you broke to get there. Then predict what happens to the *left* panel before you look — does the estimate visibly degrade, or does only the honesty degrade?

6 PRACTICAL •• Add S^1 and reproduce the failure numerically

Implement `Manifold<1>` for a circle newtype `S1(f64)` with $x \boxplus \delta$ wrapping to $(-\pi, \pi]$ and $y \boxminus x$ returning the shortest signed difference. Then write the test the widget is a picture of: 1000 seeded seam crossings, fusing a prior and a measurement of equal variance, comparing `'Ekf`

against `Ekf`

' . Assert that the manifold version's RMS error is bounded by the measurement noise while the vector version's is not.

7 PRACTICAL ••• The iterated EKF, and a first look at Gauss–Newton

Implement `iterated_correct` on the same traits: re-linearize h at the current posterior mean and repeat until the update falls below a tolerance, with the innovation always measured against the *original* prior (this is the detail everyone gets wrong — write down why). Show on the range-to-beacon model with a wide prior that it beats the single-step EKF, and that its fixed point is the maximum-a-posteriori estimate, i.e. one Gauss–Newton solve of the two-factor problem. Then read the opening of Chapter 15 with that in mind.

7.8 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 3 §3.3 is the source of this chapter’s EKF derivation, Table 3.3, and the epigraph. Its §3.3.4 practical considerations are what the Linearization Lens turns from prose warnings into measurable claims. [link](#)

Julier, S. J. and Uhlmann, J. K. (2004). Unscented Filtering and Nonlinear Estimation *Proceedings of the IEEE* 92(3), 401–422.

The definitive statement of the unscented transform, including the scaled point set and the accuracy-order argument this chapter’s third derivation follows. doi:10.1109/JPROC.2003.823141

Hertzberg, C., Wagner, R., Frese, U., and Schröder, L. (2013). Integrating generic sensor fusion algorithms with sound state representations through encapsulation of manifolds *Information Fusion* 14(1), 57–77.

Where $\boxplus$ and $\boxminus$ come from, with the retraction axioms stated exactly as used here. The paper that made on-manifold estimation a matter of interface design rather than special-casing. doi:10.1016/j.inffus.2011.08.003

Solà, J., Dera, J., and Atchuthan, D. (2018). A micro Lie theory for state estimation in robotics *arXiv:1812.01537*.

The best short reference for the Jacobian bookkeeping this chapter deliberately keeps light, including the left/right convention tables that Appendix C follows. [link](#)

Barrau, A. and Bonnabel, S. (2017). The Invariant Extended Kalman Filter as a Stable Observer *IEEE Transactions on Automatic Control* 62(4), 1797–1812.

The convergence result cited but not proved in this chapter: for group-affine systems the invariant error is autonomous, and the filter is a locally stable observer with a trajectory-independent domain of attraction. doi:10.1109/TAC.2016.2594085

Brossard, M., Barrau, A., and Bonnabel, S. (2020). A Code for Unscented Kalman Filtering on Manifolds (UKF-M) *IEEE International Conference on Robotics and Automation (ICRA)*, 5701–5708.

UKF-M: the sigma-point filter with $\boxplus$ retraction and the $\boxminus$ -mean iteration, exactly as implemented in this chapter’s `UkfMSe2` and its Rust counterpart. doi:10.1109/ICRA40945.2020.9197489

Barrau, A. and Bonnabel, S. (2023). The Geometry of Navigation Problems *IEEE Transactions on Automatic Control* 68(2), 689–704.

The modern synthesis of which state spaces admit an autonomous error, extending the group-affine condition to the two-frames systems that describe most real navigation problems. doi:10.1109/TAC.2022.3144328

Xu, W., Cai, Y., He, D., Lin, J., and Zhang, F. (2022). FAST-LIO2: Fast Direct LiDAR-Inertial Odometry *IEEE Transactions on Robotics* 38(4), 2053–2073.

Proof that this chapter’s material is production practice, not theory: an iterated error-state Kalman filter on a manifold, running at 100 Hz on a real vehicle. Chapter 16 builds a small version of it. doi:10.1109/TR0.2022.3141876

Nonparametric Filters

THE BAYES FILTER FAMILY · Intermediate · 60 min



Each particle is a concrete instantiation of the state at time t , that is, a hypothesis as to what the true world state may be at time t .

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 4

Every filter so far has bought speed by betting on a shape. The Kalman filter bets the posterior is a Gaussian; the EKF bets it stays one after a nonlinearity; the UKF bets a handful of sigma points can carry it through. Those bets pay in microseconds — and they are unpayable in the corridor this book opened with, because a robot that has just seen *a* door genuinely believes it is at one of three places, and no single ellipse can say that.

This chapter drops the parametric bet entirely. Two representations do it. The first chops the state space into cells and stores one number per cell: exact on finite spaces, honest about its error on continuous ones, and hopelessly expensive past three dimensions. The second lets the samples go where the probability is — a bag of weighted hypotheses that costs $O(M)$ per step and does not care about dimension at all. That second one, the particle filter, is the algorithm that still ships in production localization stacks in 2026, and by the end of this chapter you will have derived it, broken it on purpose, and written it in Rust.

Two smaller things get planted here and harvested later. A five-line special case — the binary static-state filter in log odds — grows into occupancy grid mapping in Chapter 13. And the trick of sampling from the wrong distribution on purpose reappears as a *control* algorithm in Chapter 23.

🕒 AFTER THIS CHAPTER YOU CAN

- Instantiate the Bayes filter on a finite decomposition and say exactly where the approximation enters

- Derive the binary static-state filter in log odds, and recognize it as the occupancy-grid update
- Derive importance sampling from first principles, and the particle filter from importance sampling
- Explain why the low-variance comb beats the roulette wheel without being any less unbiased
- Diagnose degeneracy with the effective sample size, and particle deprivation from its symptoms
- Adapt the particle count to the belief with the KLD bound, and implement all of it in Rust

ASSUMED

- The Bayes filter recursion and the Markov assumption (Chapter 5)
- Sampling, expectation, and seeded RNG discipline (Chapter 2)
- Why Gaussian filters are unimodal by construction (Chapters 6–7)

8.1 The problem with one ellipse

Rusty is in the Hallway. Three identical doors, one binary door detector, no idea where it started. The detector fires.

The true posterior is trimodal: three peaks of equal mass, and a lot of nearly-zero corridor between them. Hand that posterior to a Gaussian filter and it must answer with a mean and a covariance. Moment-match it honestly and the mean lands at the centroid of three separated hypotheses — a location the evidence never argued for, and one the robot is at most a third likely to be near — while the covariance inflates until it spans the corridor, reporting an uncertainty that is technically correct and operationally useless. Let the filter track a single mode instead, as an EKF started near one door will, and it becomes *confidently wrong* two times out of three with nothing in its state to record that a choice was ever made.

This is not a tuning problem. A Gaussian has one mode; the posterior has three; the mismatch is structural. So represent the belief with something that can hold three answers at once.

INTERACTIVE · w8.1

Particle Survival Arena

A particle set is a distribution, not a list of guesses — and resampling is selection pressure, with all the side effects that implies.

Run this simulation in the web edition: `/chapters/ch08-nonparametric-filters #w8.1`

Watch a few full cycles before reading on. Three things are happening, and each is a section of this chapter.

The cloud is a distribution, not a list of guesses. The dashed purple line is the weighted mean, and during the multimodal phase it wanders across the corridor tracking the *centroid* of the surviving hypotheses rather than any one of them. That is not a bug in the readout — it is the correct expectation of a genuinely multimodal belief, and it is exactly the number a Gaussian filter would have been forced to report. The particles are the answer; the mean is a lossy summary of them, and in this corridor it is the worst summary available.

Weighting and resampling are different operations, and only one of them destroys anything. During the green phase nothing moves: the ticks change size because their weights changed. During the purple phase nothing changes size: the ticks *move*, because low-weight hypotheses were deleted and high-weight ones were cloned. Weighting is reversible bookkeeping. Resampling is selection, and selection is irreversible.

Diversity is a finite resource. Press *Deprivation preset* — 30 particles, a razor-sharp sensor — and re-roll the seed a few times. In a sizeable fraction of runs every particle near Rusty dies in one unlucky weighting, and from then on the filter converges tightly and confidently onto the wrong door and never recovers. It cannot recover: resampling only ever redistributes hypotheses that already exist, and the motion noise is far too small to walk one back.

💡 THE ONE SENTENCE VERSION

A weighted set of samples is a legitimate representation of a posterior; importance sampling is the license to draw those samples from the wrong distribution and fix it with a weight; and resampling is the price you pay for that license, in units of diversity.

8.2 Building intuition

Both representations in this chapter answer the same question — *how do you write down a distribution you cannot write down?* — and they answer it in opposite ways.

The **histogram filter** partitions the state space in advance and stores the probability mass of each region. It commits to a resolution before seeing any data. Everything the belief could ever want to say must be expressible as "this much mass in this box", so a fine grid can say a lot and costs a lot. Nothing is adaptive; the same arithmetic runs whether the robot is lost or perfectly localized.

The **particle filter** stores nothing in advance. It carries M hypotheses, each a complete state vector, each with a weight, and the *hypotheses themselves move* —

they follow the mass rather than waiting for it. Resolution is not a parameter; it is wherever the particles happen to be. That is the whole trade: the grid pays for the region it might need, the particle set pays for the region it currently believes in.

The particle filter's rhythm has three beats, and it is worth naming them before formalizing them.

propagate	Push every particle through the motion model, sampling its noise independently. The cloud spreads. Nothing about the measurement has been used. Cost: M motion samples.
weight	Multiply each particle by the likelihood of the actual measurement at that particle. The cloud does not move; the particles change importance. Cost: M likelihood evaluations.
resample	Draw a new population of M particles from the old one with probability proportional to weight. Heavy particles are cloned; light ones vanish. Cost: $O(M)$, one random number.

Thrun's metaphor for the third beat is a roulette wheel whose arcs are the weights, spun M times. It is the right picture and the wrong algorithm, and the next few pages are about why.

8.3 The mathematics

8.3.1 Notation this chapter adds

$\mathbf{x}_k, \hat{\mathbf{x}}_k$	Cell k of a decomposition of the state space, and its representative point (usually its centroid).
$p_{k,t}$	Probability mass the grid belief assigns to cell k at time t .
$\ell_t = \log \frac{p}{1-p}$	Log odds of a binary state. Chapter 13 writes the per-cell form as $\ell_{\{t,i\}}$.
$\mathcal{X}_t = \{\mathbf{x}_t^{[i]}, w_t^{[i]}\}_{i=1}^M$	The particle set: M hypotheses with their importance weights.
f, g	Target and proposal density in importance sampling. Weights are the ratio f/g .
M_{eff}	Effective sample size, $1/\Sigma(w^{[i]})^2$. The resampling trigger.
(ϵ, δ, k)	KLD bound parameters: tolerated divergence, failure probability, number of occupied bins.

8.3.2 Chopping the space

Start with the easy half. If the state space is *finite* — the robot is in one of K rooms, the door is open or closed — then Chapter 5's integral is a sum, and the Bayes filter is implementable exactly as written, with no approximation of any kind.

ALGORITHM `Discrete_Bayes_filter({p_{k,t-1}}, u_t, z_t)` COST
 $O(K^2)$ for prediction in general, $O(K \cdot W)$ when the motion kernel has bandwidth W ; $O(K)$
 for correction
in the previous discrete belief, the control, the measurement
out $\{p_{k,t}\}$

```

1: for all  $k$  do
2:    $\bar{p}_{k,t} = \sum_i p(\mathbf{x}_k | u_t, \mathbf{x}_i) p_{i,t-1}$ 
3:    $p_{k,t} = \eta p(z_t | \mathbf{x}_k) \bar{p}_{k,t}$ 
4: endfor
5: return  $\{p_{k,t}\}$ 

```

This is Thrun's Table 4.1, and it is the same two lines as the Bayes filter with $\int$ replaced by $\sum$. Line 2 is $O(K^2)$ because in principle every cell can reach every other; in practice a robot moving a bounded distance per step has a *banded* kernel — only W neighbours are reachable — and the cost drops to $O(K \cdot W)$. Line 3 is $O(K)$ always.

The interesting case is a *continuous* state space chopped into cells, which is where the word "histogram filter" applies and where approximation finally enters.

 **DERIVATION** Why the grid version is an approximation, and how big the error is

$$p(z_t | \mathbf{x}_k) \approx p(z_t | \hat{\mathbf{x}}_k)$$

Decompose $\text{range}(X_t) = \mathbf{x}_1 \cup \dots \cup \mathbf{x}_K$ into disjoint convex regions. The grid belief stores one number $p_{k,t}$ per region, which means it is committed to a **piecewise-constant** density:

$$p(\mathbf{x}_t) = \frac{p_{k,t}}{|\mathbf{x}_k|} \quad \text{for } \mathbf{x}_t \in \mathbf{x}_k$$

where $|\mathbf{x}_k|$ is the region's volume. Everything else follows from that one commitment.

Step 1 — the region-conditioned likelihood is an average, exactly. Under the piecewise-uniform model,

$$p(z_t | \mathbf{x}_k) = \frac{\int_{\mathbf{x}_k} p(z_t | x_t) p(x_t) dx_t}{\int_{\mathbf{x}_k} p(x_t) dx_t} = |\mathbf{x}_k|^{-1} \int_{\mathbf{x}_k} p(z_t | x_t) dx_t$$

because the constant $p_{k,t}/|\mathbf{x}_k|$ cancels top and bottom. No approximation yet: this is the *exact* likelihood of the region under the model the grid has assumed.

Step 2 — replace the average by a probe. Evaluating that integral for every cell at every step is exactly the computation we were trying to avoid. So substitute the representative point $\hat{x}_k$ (Thrun's eq. 4.3 takes the centroid):

$$p(z_t | \mathbf{x}_k) \approx p(z_t | \hat{x}_k), \quad p(\mathbf{x}_k | u_t, \mathbf{x}_i) \approx \eta p(\hat{x}_k | u_t, \hat{x}_i) |\mathbf{x}_k|$$

Step 3 — bound the damage. Expanding $p(z_t | x_t)$ about $\hat{x}_k$, the first-order term integrates to zero over a centroid-probed region, so the leading error is second order in the cell diameter h :

$$|p(z_t | \mathbf{x}_k) - p(z_t | \hat{x}_k)| = O(h^2) \sup_{\mathbf{x}_k} \|\nabla_x^2 p(z_t | x)\|$$

So halving the cell size quarters the modelling error — and, in d dimensions, multiplies the cell count by 2^d . That exchange rate is the entire argument of the next widget.

Step 4 — note what is *not* approximated. The recursion itself is still exact for the piecewise-constant model. The histogram filter's error is a *representation* error, not an inference error, which is what separates it from the EKF: the EKF is exactly right about an approximate distribution *and* approximately right about the update. ■

INTERACTIVE · w8.3

Grid Resolution Ladder

Finer is not free: the same belief costs r cells in 1-D and r^3 in 3-D, and the naive prediction step squares that.

Run this simulation in the web edition: [/chapters/ch08-nonparametric-filters #w8.3](#)

The ladder is where the curse of dimensionality stops being a slogan. A 1-D corridor at 2 cm resolution is 500 cells and a rounding error of a computation. The *same* resolution over (x, y, θ) — the state that Chapter 12 actually needs — is

$500^3 \approx 1.3 \times 10^8$ cells, and the naive prediction sum over ordered pairs of cells is 10^{16} operations per step. Banding rescues the exponent on the *pair* count but not on the cell count itself: you still have to store, touch, and normalize r^d numbers. Grids survive in robotics exactly where $d \leq 2$ and the map is static — which is to say, in Chapter 13, and essentially nowhere else.

8.3.3 The binary Bayes filter, and why it deserves its own box

Before leaving grids, one special case earns disproportionate attention: a state that is **binary** and **static**. Is this cell occupied? Is that door open? The state does not change, so there is no prediction step at all — the filter is pure evidence accumulation.

Write the belief in **log odds**:

$$\ell_t = \log \frac{\text{bel}_t(x)}{1 - \text{bel}_t(x)} \iff \text{bel}_t(x) = 1 - \frac{1}{1 + \exp \ell_t}$$

Log odds run from $-\infty$ to $+\infty$, which means no amount of confidence can numerically round to exactly 0 or 1 — the truncation problem that kills naive product-of-probabilities updates.

ALGORITHM `binary_Bayes_filter( $\ell_{t-1}$ ,  $z_t$ )` COST 0(1) -- one logarithm and two additions

in previous log odds, measurement

out ℓ_t

```
1:  $\ell_t = \ell_{t-1} + \log \frac{p(x | z_t)}{1 - p(x | z_t)} - \ell_0$ 
2: return  $\ell_t$ 
```

Two features of that line are worth staring at. First, the update is an **addition** — the entire filter is $+=$. Second, the measurement enters through the **inverse** model $p(x | z_t)$, not the familiar forward model $p(z_t | x)$. That inversion is not laziness: when the state is one bit and the measurement is a 1080p image or a 1080-beam scan, it is far easier to write down "how likely is occupancy given this reading" than to describe the distribution over all readings that a wall produces.

 **DERIVATION** Deriving the log-odds recursion, and where η goes

$$\ell_t = \ell_{t-1} + \text{logit } p(x | z_t) - \ell_0$$

Step 1 — Bayes rule with the normalizer made explicit. For a static x ,

$$p(x | z_{1:t}) = \frac{p(z_t | x) p(x | z_{1:t-1})}{p(z_t | z_{1:t-1})}$$

Step 2 — flip the measurement model. Apply Bayes rule once more to $p(z_t | x)$ so that the inverse model appears:

$$p(z_t | x) = \frac{p(x | z_t) p(z_t)}{p(x)} \implies p(x | z_{1:t}) = \frac{p(x | z_t) p(z_t) p(x | z_{1:t-1})}{p(x) p(z_t | z_{1:t-1})}$$

Step 3 — write the same thing for $\neg x$ and divide. This is the trick the whole derivation turns on. The opposite event gives

$$p(\neg x | z_{1:t}) = \frac{p(\neg x | z_t) p(z_t) p(\neg x | z_{1:t-1})}{p(\neg x) p(z_t | z_{1:t-1})}$$

and dividing the two kills every term that does not depend on the hypothesis — $p(z_t)$, and the evidence $p(z_t | z_{1:t-1})$, which is precisely the η that a probability-space implementation has to compute:

$$\frac{p(x | z_{1:t})}{p(\neg x | z_{1:t})} = \frac{p(x | z_t)}{1 - p(x | z_t)} \cdot \frac{p(x | z_{1:t-1})}{1 - p(x | z_{1:t-1})} \cdot \frac{1 - p(x)}{p(x)}$$

Step 4 — take logarithms. Products become sums, and with $\ell_0 = \log \frac{p(x)}{1-p(x)}$ denoting the prior in log odds,

$$\ell_t = \underbrace{\log \frac{p(x | z_t)}{1 - p(x | z_t)}}_{\text{what this reading says}} + \underbrace{\ell_{t-1}}_{\text{what we already believed}} - \underbrace{\ell_0}_{\text{don't count the prior twice}}$$

The $-\ell_0$ correction is the step readers most often drop, and there is a clean test for whether it belongs. Feed the filter a completely uninformative reading — one for which $p(x | z_t) = p(x)$ — and the increment must be zero. With the correction it is exactly zero. Without it, every uninformative reading would add the prior again, and a robot with a slightly pessimistic occupancy prior would map an empty room as solid just by staring at it.

The inverse model *is already a posterior*; subtracting ℓ_0 is what strips the prior back out and leaves only the evidence that reading actually contributed.

Consequence — order does not matter. Telescoping the recursion gives

$$\ell_t = \ell_0 + \sum_{s=1}^t \left(\log \frac{p(x | z_s)}{1 - p(x | z_s)} - \ell_0 \right)$$

a sum, and sums commute. For a genuinely static state, the order the evidence arrived in is irrelevant. Exercise 3 asks what breaks the moment the state can change. ■

INTERACTIVE · w8.4

Log-Odds Flip-Flop

A static state still needs a filter — and in log odds that filter is one addition, which is why occupancy mapping is cheap.

Run this simulation in the web edition: </chapters/ch08-nonparametric-filters#w8.4>

NOTE

Run one of these per grid cell and you have occupancy grid mapping. Chapter 13 is this box, tiled — including the clamping guard, which exists for exactly the reason the widget demonstrates: an unclamped cell that has seen a hundred consistent readings needs a hundred contradicting ones to change its mind, and a door that opens does not get a hundred.

8.3.4 Importance sampling, from first principles

Now the other branch. We want samples from the posterior f . We cannot draw them, because we cannot even evaluate f without the normalizer. What we *can* do is draw from something else.

Let g be any density we can sample, with the single condition that it covers f 's support: $f(x) > 0 \Rightarrow g(x) > 0$. Then for any statistic ϕ ,

$$\mathbb{E}_f[\phi(x)] = \int \phi(x) f(x) dx = \int \phi(x) \frac{f(x)}{g(x)} g(x) dx = \mathbb{E}_g[w(x) \phi(x)], \quad w(x) = \frac{f(x)}{g(x)}$$

Multiply and divide by g ; recognize an expectation under g . That is the entire idea, and it is worth appreciating how much it buys: *you may sample from the wrong distribution on purpose, provided you carry a weight that records how wrong it was.*

 DERIVATION Why weighted g-samples converge to f , and what the weights cost

$$\hat{\mathbb{E}}_f[\phi] = \sum_i \tilde{w}^{[i]} \phi(x^{[i]}) \rightarrow \mathbb{E}_f[\phi]$$

Step 1 — the self-normalized estimator. In practice f is known only up to a constant (it is a posterior; the normalizer is the thing we cannot compute). So use unnormalized w and divide by their sum:

$$\hat{\mathbb{E}}_f[\phi] = \frac{\sum_{i=1}^M w^{[i]} \phi(x^{[i]})}{\sum_{i=1}^M w^{[i]}} = \sum_{i=1}^M \tilde{w}^{[i]} \phi(x^{[i]}), \quad x^{[i]} \sim g$$

Numerator and denominator are each ordinary Monte Carlo averages under g , converging by the law of large numbers to $c \mathbb{E}_f[\phi]$ and c respectively, where c is the unknown constant. The ratio converges to $\mathbb{E}_f[\phi]$ and the constant never has to be known. Thrun's eq. 4.27 states this for indicator functions $\phi = \mathbf{1}_A$, which is the statement that the weighted empirical CDF converges to F .

Step 2 — the support condition is not a technicality. If $g(x) = 0$ somewhere $f(x) > 0$, no sample ever lands there and the estimator converges — confidently — to the wrong answer. In a particle filter this is *particle deprivation*, and it is the failure mode of the whole chapter.

Step 3 — the price of a bad proposal. The estimator's variance is governed by the variance of the weights. Write $\tilde{w}$ scaled to mean 1; then the classic result (Kong, Liu and Wong) is that the estimator behaves like an unweighted sample of size

$$M_{\text{eff}} = \frac{M}{1 + \text{Var}_g[\tilde{w}]}$$

Substituting $\tilde{w}^{[i]} = Mw^{[i]}$ for normalized weights summing to one,

$$\text{Var}[\tilde{w}] = \frac{1}{M} \sum_i (Mw^{[i]} - 1)^2 = M \sum_i (w^{[i]})^2 - 1 \implies M_{\text{eff}} = \frac{1}{\sum_i (w^{[i]})^2}$$

which is the number every practical implementation watches. It is M when the weights are uniform and 1 when a single particle owns all the mass. Convergence of the estimator itself is $O(1/\sqrt{M})$ — *independent of the dimension of x* , which is the property that makes particles beat grids the moment $d > 2$. The constant, however, depends on how badly g mismatches f , and that constant is where all the engineering lives. ■

8.3.5 The particle filter targets the posterior

Now specialize. Take the target to be the posterior over the whole trajectory, $\text{bel}(x_{0:t}) = p(x_{0:t} \mid z_{1:t}, u_{1:t})$ — not because we want trajectories, but because in that space the algebra has no integrals in it.

DERIVATION The particle filter as importance sampling on trajectories

$$w_t^{[i]} = \eta p(z_t \mid x_t^{[i]})$$

Step 1 — factor the target. Applying Bayes rule and the Markov assumption exactly as in Chapter 5, but keeping every state instead of marginalizing:

$$p(x_{0:t} \mid z_{1:t}, u_{1:t}) = \eta \, p(z_t \mid x_t) \, p(x_t \mid x_{t-1}, u_t) \, p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t-1})$$

Note the absence of integral signs. That is the payoff of working in trajectory space.

Step 2 — name the proposal. Assume inductively that the particles at $t - 1$ are distributed according to $\text{bel}(x_{0:t-1})$. Line 4 of the algorithm draws $x_t^{[i]} \sim p(x_t \mid x_{t-1}^{[i]}, u_t)$, so the density the new particles actually follow is

$$g = p(x_t \mid x_{t-1}, u_t) \, \text{bel}(x_{0:t-1})$$

This is the **proposal distribution**: the motion model applied to the previous belief. It is the easiest thing in the world to sample and it completely ignores z_t .

Step 3 — take the ratio and watch it collapse. The target from Step 1 has the proposal from Step 2 sitting inside it as a factor, so the division is almost total:

$$w_t^{[i]} = \frac{\text{target}}{\text{proposal}} = \frac{\eta \, p(z_t \mid x_t) \, \overbrace{p(x_t \mid x_{t-1}, u_t) \, \text{bel}(x_{0:t-1})}^{\text{exactly the proposal}}}{p(x_t \mid x_{t-1}, u_t) \, \text{bel}(x_{0:t-1})} = \eta \, p(z_t \mid x_t^{[i]})$$

Everything cancels except the measurement likelihood. *That* is why a particle filter is fifteen lines: choosing the motion model as the proposal makes the importance weight equal to the thing you were going to compute anyway. And η never has to be evaluated, because resampling only needs weights up to a constant — normalize after the fact and it disappears.

Step 4 — resample, then marginalize. Drawing with probability proportional to $w_t^{[i]}$ produces particles distributed as $\text{proposal} \times \text{weight} = \text{bel}(x_{0:t})$.

And if $x_{0:t}^{[i]}$ is distributed according to $\text{bel}(x_{0:t})$, then its last component $x_t^{[i]}$ is trivially distributed according to $\text{bel}(x_t)$ — marginalization of a sample set is *deleting a column*.

The honest caveats. This argument is exact only as $M \rightarrow \infty$. For finite M the self-normalization in Step 1 of the previous derivation introduces a bias of order $1/M$: the weights are drawn in an M -dimensional space but live, after normalization, in an $(M-1)$ -dimensional one. With $M = 1$ the pathology is total — the single weight normalizes to 1 regardless of z_t , and the "filter" ignores its sensor completely. In practice the bias is negligible for $M \geq 100$; the *variance* discussed next is what actually hurts. ■

ALGORITHM Particle_filter($\{t-1\}$, u_t , z_t) COST $O(M)$, given $O(1)$ motion sampling and likelihood evaluation

in the previous particle set, the control, the measurement

out $_t$

```

1:  $\bar{X}_t = X_t = \emptyset$ 
2: for  $m = 1$  to  $M$  do
3:   sample  $x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]})$ 
4:    $w_t^{[m]} = p(z_t \mid x_t^{[m]})$ 
5:    $\bar{X}_t = \bar{X}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$ 
6: endfor
7: for  $m = 1$  to  $M$  do
8:   draw  $i$  with probability  $\propto w_t^{[i]}$ 
9:   add  $x_t^{[i]}$  to  $X_t$ 
10: endfor
11: return  $X_t$ 
```

That is Thrun's Table 4.3, unchanged since 1999, and it is still the core of the localizer that ships as the ROS 2 navigation default in 2026. What has changed is lines 7–10, which modern implementations do not run every step and do not run this way.

8.3.6 Resampling, and the variance nobody mentions

Lines 8–9 as written are **multinomial resampling**: M independent draws from the categorical distribution defined by the weights. Thrun's roulette wheel, spun M times.

It is unbiased. It is also needlessly noisy, and the noise is not free — it is

added directly to the estimator the filter is trying to compute.

DERIVATION Offspring variance: roulette versus comb

$$\text{Var}[N_i] = Mw_i(1 - w_i) \text{ vs. } f_i(1 - f_i)$$

Let N_i be the number of offspring particle i receives.

Step 1 — both schemes are unbiased. For multinomial resampling, $N_i \sim \text{Bin}(M, w_i)$ directly, so $\mathbb{E}[N_i] = Mw_i$. For the comb, the M pointers $u_m = r + (m - 1)/M$ form a lattice of spacing $1/M$ with a uniformly random offset $r \sim U[0, 1/M)$; the expected number of lattice points falling in an interval of length w_i is Mw_i regardless of where the interval sits. Same expectation. Unbiasedness is not what distinguishes them.

Step 2 — multinomial variance. From the binomial,

$$\text{Var}[N_i] = Mw_i(1 - w_i)$$

For a particle carrying its fair share $w_i = 1/M$, this is ≈ 1 : the standard deviation of its offspring count is as large as the count itself. Concretely, $P(N_i = 0) = (1 - 1/M)^M \rightarrow e^{-1} \approx 0.37$. **A perfectly healthy particle has a 37% chance of being deleted, every step.**

Step 3 — comb variance. Write $Mw_i = n_i + f_i$ with $n_i = \lfloor Mw_i \rfloor$ integer and $f_i \in [0, 1)$. An interval of length w_i contains either n_i or $n_i + 1$ points of a spacing- $1/M$ lattice, and by Step 1 the expectation must be $n_i + f_i$, so

$$N_i = \begin{cases} n_i + 1 & \text{with probability } f_i \\ n_i & \text{with probability } 1 - f_i \end{cases} \implies \text{Var}[N_i] = f_i(1 - f_i) \leq \frac{1}{4}$$

Every offspring count is within one of its expectation, always. When Mw_i is an integer the count is *deterministic*. A particle with $w_i \geq 1/M$ cannot be deleted at all.

Step 4 — and it is cheaper. Multinomial resampling needs M random numbers and, done naively, an $O(\log M)$ search per draw: $O(M \log M)$. The comb needs one random number and one monotone sweep: $O(M)$, with a memory access pattern that is purely sequential. It is faster, quieter, and strictly easier to write.

The intermediate scheme. Stratified resampling draws one uniform *per* comb interval — $u_m \sim U[(m - 1)/M, m/M)$ — trading the comb's determinism for a little independence. Its variance sits between the two, and Exercise 6 asks you to measure it. ■

ALGORITHM `Low_variance_sampler(_t, _t)` COST $O(M)$ time, exactly one random number
in the weighted particle set
out a resampled set of M particles with uniform weights

```

1:  $\bar{\mathcal{X}}_t = \emptyset$ 
2:  $r = \text{rand}(0; M^{-1})$ 
3:  $c = w_t^{[1]}$ 
4:  $i = 1$ 
5: for  $m = 1$  to  $M$  do
6:    $u = r + (m - 1) \cdot M^{-1}$ 
7:   while  $u > c$ 
8:      $i = i + 1$ 
9:      $c = c + w_t^{[i]}$ 
10:  endwhile
11:  add  $x_t^{[i]}$  to  $\bar{\mathcal{X}}_t$ 
12: endfor
13: return  $\bar{\mathcal{X}}_t$ 
```

INTERACTIVE · w8.2

Resampling Wheel: roulette vs. comb

Both resamplers have the same expectation. Only the variance differs — and variance was the enemy all along.

Run this simulation in the web edition: [#w8.2](/chapters/ch08-nonparametric-filters)

8.3.7 Degeneracy, deprivation, and when not to resample

Two failure modes wear similar names and have opposite cures.

Weight degeneracy is what happens if you *never* resample. The weights are products of likelihoods, and a product of many terms concentrates: after a few dozen steps one particle holds essentially all the mass, $M_{\text{eff}} \rightarrow 1$, and the other $M - 1$ particles are consuming CPU to represent nothing. The cure is to resample.

Particle deprivation is what happens if you resample *too much*. Every resample deletes hypotheses, and only the motion model's noise creates new ones. Thrun's thought experiment makes it vivid: a robot that is not moving and has no sensors. The state transition is deterministic, so no new states are ever introduced; the resampling step is pure random deletion. With probability one, the population collapses to M identical copies of a single state — and to an outside observer the robot appears to have determined its position exactly, despite having no sensors

at all. The cure is to resample less.

The standard reconciliation is to make resampling conditional on the diagnostic we already derived:

$$\text{resample} \iff M_{\text{eff}} = \frac{1}{\sum_i (w_t^{[i]})^2} < \frac{M}{2}$$

with the weights carried multiplicatively across steps when no resample happens ($w_t^{[i]} = p(z_t | x_t^{[i]}) w_{t-1}^{[i]}$, resetting to $1/M$ when one does). The threshold $M/2$ is a convention, not a theorem; Exercise 7 sweeps it and reports where the optimum actually sits for the Hallway.

Three further defences, in increasing order of honesty:

1. **Use the low-variance sampler.** Free, and it removes the single largest source of unnecessary deletion — the 37% figure above.
2. **More particles.** Effective, and expensive, and it treats the symptom: if the proposal is bad, more samples from it are still bad samples.
3. **Inject fresh hypotheses when the evidence says you are lost.** This is Augmented MCL, and Chapter 12 derives it properly, using the average measurement likelihood as the "am I lost?" detector.

💡 DEGENERACY AND DEPRIVATION ARE THE SAME KNOB AT TWO ENDS

Resampling converts weight variance into sample diversity loss. Do it too rarely and the weights degenerate; do it too often and the population does. The effective sample size is the instrument that tells you which side of the knob you are on, and it costs one pass over the weights.

8.3.8 KLD-adaptive sample size

There is a better answer than a fixed M , and it comes from asking what M is *for*.

During global localization the belief covers the whole map and needs thousands of particles to represent it. Ten seconds later it is a 20 cm blob and thirty particles would do. A fixed M must be sized for the worst case and then wastes 99% of its work for the rest of the run. Fox's KLD-sampling makes M a function of the belief's *spread*, measured as the number of histogram bins the samples actually occupy.

📐 DERIVATION The KLD bound, via Wilson–Hilferty

$$M = \chi_{k-1, 1-\delta}^2 / 2\epsilon$$

Suppose the true posterior is a discrete distribution over k bins, and we

draw M samples from it. Let $\hat{p}$ be the resulting maximum-likelihood estimate — the empirical bin frequencies.

Step 1 — the likelihood ratio statistic. The quantity $2M \text{KL}(\hat{p} \| p)$ is the log-likelihood-ratio statistic for the multinomial, and it converges in distribution to χ_{k-1}^2 as M grows. So

$$P(\text{KL}(\hat{p} \| p) \leq \varepsilon) \approx P(\chi_{k-1}^2 \leq 2M\varepsilon)$$

Step 2 — impose the confidence. We want that probability to be $1 - \delta$, so we need $2M\varepsilon$ to be the $(1 - \delta)$ quantile of χ_{k-1}^2 :

$$M = \frac{1}{2\varepsilon} \chi_{k-1, 1-\delta}^2$$

Step 3 — approximate the quantile. The Wilson–Hilferty transformation says that $(\chi_{k-1}^2 / (k-1))^{1/3}$ is approximately normal with mean $1 - \frac{2}{9(k-1)}$ and variance $\frac{2}{9(k-1)}$. Inverting,

$$\chi_{k-1, 1-\delta}^2 \approx (k-1) \left[1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta} \right]^3$$

Step 4 — read off M . Substituting into Step 2,

$$M \geq \frac{k-1}{2\varepsilon} \left[1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta} \right]^3$$

Everything on the right is $O(1)$ to evaluate, and the only thing that depends on the belief is k . **Crucially, k counts occupied bins, not grid bins** — an empty bin costs nothing — so the bound falls automatically as the cloud condenses. ■

At $\varepsilon = 0.05$ and $\delta = 0.01$ (so $z_{0.99} = 2.3263$), the bound reads:

occupied bins k	required M
3	93
10	217
100	1,347
500	5,755

ALGORITHM `KLD_sample_size(k,  $\varepsilon$ ,  $\delta$ )` COST $O(1)$; folded into the resampling loop as an adaptive stopping rule

in number of occupied bins, tolerated KL divergence, failure probability

out the number of particles that suffices

```

1: if  $k \leq 1$  then return  $M_{\min}$ 
2:  $z = \Phi^{-1}(1 - \delta)$ 
3:  $a = \frac{2}{9(k-1)}$ 
4:  $M = \frac{k-1}{2\varepsilon} (1 - a + \sqrt{a} z)^3$ 
5: return  $\max(M_{\min}, \min(M_{\max}, \lceil M \rceil))$ 

```

In practice this is not called once per step but *inside* the resampling loop: draw a particle, check whether it landed in a bin nothing has landed in yet, and if so recompute the bound. Stop as soon as the number drawn meets it. The loop discovers how many particles it needs while it is filling the set. Toggle **KLD-adaptive M** in the arena at the top of this chapter and watch the population fall from the ceiling to a few dozen as the three clouds become one.

8.4 A worked example you can check by hand

Five particles, weights already normalized:

$$w = (0.10, 0.30, 0.05, 0.40, 0.15), \quad M = 5$$

Effective sample size. $\sum_i w_i^2 = 0.01 + 0.09 + 0.0025 + 0.16 + 0.0225 = 0.285$, so

$$M_{\text{eff}} = \frac{1}{0.285} = 3.5088$$

That is above the $M/2 = 2.5$ threshold, so a well-behaved filter would **not** resample here. We will do it anyway, to have something to check.

The comb. Take $r = 0.15$, which is a legal draw from $U[0, 1/5)$. The pointers are $u_m = r + (m-1)/5 = 0.15, 0.35, 0.55, 0.75, 0.95$, and we walk them against the cumulative weights:

i	w_i	cumulative	Mw_i	pointers landing in i	offspring N_i
1	0.10	0.10	0.50	—	0
2	0.30	0.40	1.50	0.15, 0.35	2
3	0.05	0.45	0.25	—	0
4	0.40	0.85	2.00	0.55, 0.75	2
5	0.15	1.00	0.75	0.95	1

Offspring counts (0, 2, 0, 2, 1). Particles 1 and 3 die; nobody is drawn more than $\lceil Mw_i \rceil$ times; the total is 5, as it must be. Notice particle 4: $Mw_4 = 2$ exactly, and it gets exactly 2 offspring for *every* legal value of r — the comb is deterministic wherever the expectation is a whole number.

The variance gap. Multinomial gives $\text{Var}[N_i] = Mw_i(1 - w_i)$; the comb gives $f_i(1 - f_i)$ with $f_i = Mw_i - \lfloor Mw_i \rfloor$:

i	multinomial Var	comb f_i	comb Var
1	0.4500	0.50	0.2500
2	1.0500	0.50	0.2500
3	0.2375	0.25	0.1875
4	1.2000	0.00	0.0000
5	0.6375	0.75	0.1875
Σ	3.5750		0.8750

Same expectation, one quarter the variance. Set the wheel widget above back to its defaults, press *Spin* $\times 1000$, and the measured bars converge on exactly these two columns.

8.5 Implementation in Rust

The library is `crates/ch08_particles`. It depends on `bayes_core` (the `BayesFilter` trait from Chapter 5) and `sim` (the `Hallway` from Chapter 4), and it is consumed later by `localize` (Ch. 12), `ch13_occgrid`, and `ch17_fastslam`. Two design decisions drive the whole module.

Weights live in log space. A likelihood is a product over beams; in a 360-beam scan that product underflows `f64` long before it becomes uninteresting. Every weight in this crate is a log weight, normalized by `log-sum-exp`, and exponentiated only at the boundary where a resampler needs actual probabilities.

The proposal and the likelihood are traits, not functions. Chapter 9’s motion samplers and Chapter 10’s sensor models plug into these two slots unchanged, which is what makes Chapter 12’s MCL a fifty-line file rather than a rewrite.

RUST `crates/ch08_particles/src/set.rs`

```

1 use rand::rngs::SmallRng;
2
3 /// A weighted particle set. Weights are log weights, always.
4 ///
5 /// `S` is deliberately unconstrained: the Hallway filter instantiates it with
6 /// `f64`, Chapter 12 with `Se2`, Chapter 17 with a pose and a map. Nothing in
7 /// this file cares.
8 pub struct ParticleSet<S> {
9     pub states: Vec<S>,
10    pub log_w: Vec<f64>,
11 }
12
13 impl<S> ParticleSet<S> {
14     /// A fresh set with uniform weights:  $\log(1/M)$  each.
15     pub fn uniform(states: Vec<S>) -> Self {
16         let m = states.len();
17         Self { log_w: vec![-(m as f64).ln(); m], states }
18     }
19 }
```

```

19
20     pub fn len(&self) -> usize {
21         self.states.len()
22     }
23
24     /// Subtract the log-sum-exp so the weights sum to one in probability space.
25     ///
26     /// Shifting by the maximum first is the whole trick: it makes the largest
27     /// exponential exactly 1, so nothing overflows and the smallest terms
28     /// underflow to 0 harmlessly instead of poisoning the sum with NaN.
29     pub fn normalize(&mut self) {
30         let max = self.log_w.iter().copied().fold(f64::NEG_INFINITY, f64::max);
31         if !max.is_finite() {
32             // Every particle is impossible. Refuse to invent information:
33             // fall back to ignorance rather than propagate NaN.
34             let uniform = -(self.len() as f64).ln();
35             self.log_w.iter_mut().for_each(|l| *l = uniform);
36             return;
37         }
38         let log_z = max + self.log_w.iter().map(|l| (l - max).exp()).sum::() .ln();
39         self.log_w.iter_mut().for_each(|l| *l -= log_z);
40     }
41
42     /// Weights in probability space. Assumes `normalize` has been called.
43     pub fn weights(&self) -> Vec<f64> {
44         self.log_w.iter().map(|l| l.exp()).collect()
45     }
46
47     /// M_eff = 1 / Sigma w^2. M when uniform, 1 when one particle owns everything.
48     pub fn ess(&self) -> f64 {
49         let s: f64 = self.log_w.iter().map(|l| (2.0 * l).exp()).sum();
50         if s > 0.0 { 1.0 / s } else { 0.0 }
51     }
52 }
53
54 /// The proposal slot. Chapter 9's `sample_motion_model_odometry` implements it.
55 pub trait Proposal<S> {
56     type Control;
57     fn sample(&self, x: &S, u: &Self::Control, rng: &mut SmallRng) -> S;
58 }
59
60 /// The likelihood slot. Chapter 10's beam and likelihood-field models implement it.
61 pub trait Likelihood<S> {
62     type Measurement;
63     /// Log p(z | x). Log, because a 360-beam product is not representable otherwise.
64     fn log_lik(&self, z: &Self::Measurement, x: &S) -> f64;
65 }
66
67 /// The occupancy seed: one static binary cell, in log odds. Chapter 13 tiles this.
68 #[derive(Clone, Copy, Debug, PartialEq)]
69 pub struct LogOdds(pub f64);
70
71 impl LogOdds {
72     /// l = log(p / (1 - p)); l = 0 is "no idea", which is p = 0.5.
73     pub fn from_prob(p: f64) -> Self {
74         LogOdds((p / (1.0 - p)).ln())
75     }
76
77     pub fn prob(self) -> f64 {
78         1.0 - 1.0 / (1.0 + self.0.exp())
79     }

```

```

86  /// `binary_Bayes_filter` -- Thrun et al., Table 4.2.
87  ///
88  /// `inverse_model` is  $p(x | z)$ , *not*  $p(z | x)$ : with a one-bit state and a
89  /// megapixel measurement, the inverse is the only one anyone can write down.
90  /// Subtracting the prior is what stops repeated readings from double-counting it.
91  pub fn update(&mut self, inverse_model: f64, prior: LogOdds) {
92      self.0 += (inverse_model / (1.0 - inverse_model)).ln() - prior.0;
93  }
94
95  /// The guard Chapter 13 inherits: bound how certain a cell may become, so
96  /// that a world which changes can still change the robot's mind.
97  pub fn clamp_to(&mut self, bound: f64) {
98      self.0 = self.0.clamp(-bound, bound);
99  }
100 }

```

The resampler is the fifteen-line listing the chapter has been building toward, and it is a direct transcription of Table 4.4.

RUST crates/ch08_particles/src/resample.rs

```

1  use rand::{Rng, rngs::SmallRng};
2
3  /// `Low_variance_sampler(?_t, ?_t)` -- Thrun et al., Table 4.4.
4  ///
5  /// One random number and one monotone sweep. Two consequences make this the
6  /// default everywhere: any particle with  $w_i \geq 1/M$  is guaranteed at least one
7  /// offspring, and the pass is  $O(M)$  with a purely sequential access pattern
8  /// instead of  $O(M \log M)$  with a binary search per draw.
9  pub fn low_variance_resample<S: Clone>(
10     states: &[S],
11     weights: &[f64],
12     rng: &mut SmallRng,
13 ) -> Vec<S> {
14     let m = states.len();
15     let step = 1.0 / m as f64;
16     let r: f64 = rng.random_range(0.0..step); // the *only* randomness in here
17     let mut out = Vec::with_capacity(m);
18     let mut c = weights[0];
19     let mut i = 0usize;
20     for k in 0..m {
21         let u = r + k as f64 * step;
22         // The comb tooth walks forward; `i` never goes back, which is why the
23         // whole sweep is linear rather than  $M$  independent searches.
24         while u > c && i + 1 < m {
25             i += 1;
26             c += weights[i];
27         }
28         out.push(states[i].clone());
29     }
30     out
31 }
32
33 /// The same sweep, reporting offspring counts instead of particles.
34 ///
35 /// This is what the chapter's worked example asserts on and what the in-page
36 /// wheel widget draws -- the test and the figure share one implementation.

```

```

37 pub fn offspring_counts(weights: &[f64], r: f64) -> Vec<usize> {
38     let m = weights.len();
39     let step = 1.0 / m as f64;
40     let mut counts = vec![0; m];
41     let mut c = weights[0];
42     let mut i = 0;
43     for k in 0..m {
44         let u = r + k as f64 * step;
45         while u > c && i + 1 < m {
46             i += 1;
47             c += weights[i];
48         }
49         counts[i] += 1;
50     }
51     counts
52 }
53
54 /// Multinomial resampling -- M independent draws. Kept for the comparison in
55 /// sec."Resampling, and the variance nobody mentions", never used in anger.
56 pub fn multinomial_resample<S: Clone>(
57     states: &[S],
58     weights: &[f64],
59     rng: &mut SmallRng,
60 ) -> Vec<S> {
61     let cdf: Vec<f64> = weights
62         .iter()
63         .scan(0.0, |acc, w| {
64             *acc += w;
65             Some(*acc)
66         })
67         .collect();
68     (0..states.len())
69         .map(|_| {
70             let u: f64 = rng.random_range(0.0..*cdf.last().unwrap());
71             let i = cdf.partition_point(|&c| c < u);
72             states[i.min(states.len() - 1)].clone()
73         })
74         .collect()
75 }

```

The filter itself is then almost anticlimactic: propagate, add log-likelihoods, normalize, conditionally resample.

RUST crates/ch08_particles/src/filter.rs

```

1 use bayes_core::BayesFilter;
2 use rand::{Rng, SeedableRng, rngs::SmallRng};
3 #[cfg(not(target_arch = "wasm32"))]
4 use rayon::prelude::*;
5
6 use crate::set::{Likelihood, ParticleSet, Proposal};
7 use crate::resample::low_variance_resample;
8
9 pub struct ParticleFilter<S, P: Proposal<S>, L: Likelihood<S>> {
10     pub set: ParticleSet<S>,
11     pub proposal: P,
12     pub likelihood: L,
13     /// Resample iff M_eff < threshold . M. 0.5 is the usual convention.

```

```

14     pub resample_threshold: f64,
15     pub rng: SmallRng,
16 }
17
18 impl<S, P, L> BayesFilter for ParticleFilter<S, P, L>
19 where
20     S: Clone + Send + Sync,
21     P: Proposal<S> + Sync,
22     L: Likelihood<S> + Sync,
23 {
24     type Belief = ParticleSet<S>;
25     type Control = P::Control;
26     type Measurement = L::Measurement;
27
28     fn predict(&mut self, u: &Self::Control) {
29         let Self { set, proposal, rng, .. } = self;
30         // Reborrow immutably: a `&mut P` captured by a closure is a unique
31         // borrow and would not be shareable across rayon's threads.
32         let proposal: &P = proposal;
33         // One seed per particle, drawn *serially* from the filter's generator.
34         // This is what buys reproducibility under rayon: the seeds are a
35         // function of the filter's seed alone, so thread scheduling cannot
36         // change the result. `thread_rng()` here would silently destroy it.
37         let seeds: Vec<u64> = (0..set.len()).map(|_| rng.random()).collect();
38
39         #[cfg(not(target_arch = "wasm32"))]
40         let states = set.states.par_iter_mut().zip(seeds.par_iter());
41         #[cfg(target_arch = "wasm32")]
42         let states = set.states.iter_mut().zip(seeds.iter());
43
44         states.for_each(|(x, &seed)| {
45             let mut r = SmallRng::seed_from_u64(seed);
46             *x = proposal.sample(x, u, &mut r);
47         });
48     }
49
50     fn correct(&mut self, z: &Self::Measurement) -> f64 {
51         let Self { set, likelihood, .. } = self;
52         //  $w \leftarrow w \cdot p(z | x)$ , in logs: the importance weight is the likelihood,
53         // because the proposal was the motion model. See the derivation above.
54         for (w, x) in set.log_w.iter_mut().zip(set.states.iter()) {
55             *w += likelihood.log_lik(z, x);
56         }
57         // Before normalizing, the sum of weights is the evidence  $p(z | z_{1:t-1})$ .
58         // Chapter 12 uses a running average of it to notice it has been kidnapped.
59         let evidence = set.log_w.iter().map(|l| l.exp()).sum::<f64>();
60         set.normalize();
61
62         if set.ess() < self.resample_threshold * set.len() as f64 {
63             let w = set.weights();
64             set.states = low_variance_resample(&set.states, &w, &mut self.rng);
65             let uniform = -(set.len() as f64).ln();
66             set.log_w.iter_mut().for_each(|l| *l = uniform);
67         }
68         evidence
69     }
70
71     fn belief(&self) -> &ParticleSet<S> {
72         &self.set
73     }
74 }

```


⚠ WHERE ROBOTS BREAK THIS

The `#[cfg]` pair is not decoration. `rayon` does not build for `wasm32-unknown-unknown` without threads, so the WASM demos on this page compile the *same source* single-threaded. Because the per-particle seeds are drawn serially, a run with seed 8 produces byte-identical particles in the browser and on a 64-core workstation — which is the only reason the numbers in this chapter can be trusted at all.

Finally, the KLD bound and the tests that pin the chapter’s worked example.

RUST `crates/ch08_particles/src/kld.rs`

```

1 use stats::distribution::{ContinuousCDF, Normal};
2
3 /// `KLD_sample_size(k, epsilon, delta)` -- Fox (2003), eq. 7.
4 ///
5 /// `k_bins` counts the bins the particles *occupy*, not the bins in the grid.
6 /// An empty bin costs nothing, which is precisely why this number collapses as
7 /// the belief condenses.
8 ///
9 /// This returns the raw bound. Clamping it to `[M_min, M_max]` is the caller's
10 /// job -- the draw loop in `filter.rs` owns those limits because they are a
11 /// budget, not a statistical claim.
12 pub fn kld_sample_size(k_bins: usize, epsilon: f64, delta: f64) -> usize {
13     if k_bins <= 1 {
14         return 0;
15     }
16     let z = Normal::standard().inverse_cdf(1.0 - delta);
17     let k1 = (k_bins - 1) as f64;
18     let a = 2.0 / (9.0 * k1);
19     let inner = 1.0 - a + a.sqrt() * z;
20     ((k1 / (2.0 * epsilon)) * inner.powi(3)).ceil() as usize
21 }
22
23 #[cfg(test)]
24 mod tests {
25     use super::*;
26     use crate::resample::offspring_counts;
27     use approx::assert_relative_eq;
28     use rand::{Rng, SeedableRng, rngs::SmallRng};
29
30     /// The chapter's worked example, exactly:  $r = 0.15$  on the five weights.
31     #[test]
32     fn worked_example_offspring_counts() {
33         let w = [0.10, 0.30, 0.05, 0.40, 0.15];
34         assert_eq!(offspring_counts(&w, 0.15), vec![0, 2, 0, 2, 1]);
35     }
36
37     #[test]
38     fn worked_example_effective_sample_size() {
39         let w = [0.10, 0.30, 0.05, 0.40, 0.15];
40         let ess = 1.0 / w.iter().map(|x| x * x).sum::<f64>();
41         assert_relative_eq!(ess, 3.508_771_929_824_56, epsilon = 1e-9);

```

```

42     assert!(ess > 5.0 / 2.0, "above M/2 -- a sane filter would not resample here");
43 }
44
45 /// Unbiasedness is a claim about a mean, so test it as one -- and tightness
46 /// is a claim about every single draw, so test that per draw.
47 #[test]
48 fn comb_is_unbiased_and_within_one() {
49     let w = [0.10, 0.30, 0.05, 0.40, 0.15];
50     let expected: Vec<f64> = w.iter().map(|x| 5.0 * x).collect();
51     let mut rng = SmallRng::seed_from_u64(8);
52     let trials = 100_000;
53     let mut sums = [0.0f64; 5];
54
55     for _ in 0..trials {
56         let r: f64 = rng.random_range(0.0..0.2);
57         let counts = offspring_counts(&w, r);
58         for i in 0..5 {
59             // Never more than one away from M.w? -- the comb's whole point.
60             assert!((counts[i] as f64 - expected[i]).abs() < 1.0);
61             sums[i] += counts[i] as f64;
62         }
63     }
64
65     for i in 0..5 {
66         assert_relative_eq!(sums[i] / trials as f64, expected[i], epsilon = 0.02);
67     }
68 }
69
70 /// Three rows of the table printed in the KLD section.
71 #[test]
72 fn kld_sample_size_matches_the_chapter_table() {
73     assert_eq!(kld_sample_size(3, 0.05, 0.01), 93);
74     assert_eq!(kld_sample_size(10, 0.05, 0.01), 217);
75     assert_eq!(kld_sample_size(100, 0.05, 0.01), 1347);
76 }
77 }

```

8.6 Putting it together: the Hallway duel

The crate ships one example that settles the chapter's argument empirically:

```

$ cargo run --release --example hallway_duel -p ch08_particles

seed 8 . hallway 10.0 m . 3 doors . sensor p(hit) = 0.90 . motion sigma = 0.14 m

t   histogram(K=128)      particles(M=1000)      M_eff      resampled
1   modes=3  MAP=2.02      modes=3  MAP=1.98      612.4      no
2   modes=3  MAP=4.51      modes=3  MAP=4.55      318.7      yes
...
12  modes=1  MAP=7.48      modes=1  MAP=7.51      41.2       yes
    |MAP - truth| = 0.04 m          0.01 m

predict cost/step:  16 384 mul-add          1 000 motion samples
weights/step:      128 evals                1 000 evals
wall clock/step:   0.31 ms                  0.12 ms (8 threads)
                                   0.74 ms (wasm32, 1 thread)

```

```
deprivation sweep, M = 50, 10 seeds: 3 / 10 runs converged to the wrong door
M = 200, 10 seeds: 0 / 10
```

Three things in that output are worth more than the rest of this chapter's prose.

Both filters agree. The histogram filter is the reference — its only approximation is the grid — and the particle filter tracks it to within a cell. That agreement is not decoration; it is how you know a stochastic implementation is correct.

The particle filter is cheaper and gets cheaper faster. 1,000 particles beat 128 cells in a *one*-dimensional problem, and the gap becomes absurd in three. The histogram's cost is fixed by the grid whether the robot is lost or not; the particle filter's is fixed by M , and KLD sampling drops M by an order of magnitude the moment the belief collapses to a single mode.

The failure rate is not zero and the honest number is printed. At $M = 50$, three runs in ten converge to the wrong door and stay there. This is particle deprivation, measured. It is the same phenomenon the arena's preset dramatizes, and the reason Chapter 12 adds recovery particles rather than trusting the filter to notice on its own.

Where this goes next: Chapter 12 turns this machinery into Monte Carlo localization by plugging in Chapter 9's samplers and Chapter 10's likelihood fields; Chapter 13 tiles the log-odds box across a map; Chapter 17 shows that a particle can carry a *map* as well as a pose if you Rao-Blackwellize the rest; Chapter 22 plans directly over particle beliefs; and Chapter 23 runs importance sampling over control sequences instead of states, which is the same derivation with the arrows turned around.

8.7 Exercises

1 FOUNDATION •• The effective sample size, derived

Starting from $M_{\text{eff}} = M / (1 + \text{Var}_g[\tilde{w}])$ with $\tilde{w}$ the weights scaled to mean one, show that for normalized weights summing to one, $M_{\text{eff}} = 1 / \sum_i (w^{[i]})^2$. Then verify the two extremes analytically: it equals M exactly when all weights are $1/M$, and equals 1 exactly when one particle holds all the mass. Finally compute it for the chapter's five weights and confirm the value 3.5088.

2 FOUNDATION ••• Both schemes are unbiased; only one is tight

Prove that $\mathbb{E}[N_i] = Mw_i$ for both multinomial and comb resampling. Then prove that the comb's offspring count is always $\lfloor Mw_i \rfloor$ or $\lceil Mw_i \rceil$, and use that to derive $\text{Var}[N_i] = f_i(1 - f_i)$ where f_i is the fractional part of Mw_i . Compare with the multinomial's $Mw_i(1 - w_i)$, and state the condition under which the comb's variance is exactly zero. Which of the two facts — same mean, smaller variance — is the reason

the comb is preferred?

3 FOUNDATION •• Order does not matter, until it does

Telescope the log-odds recursion to show that ℓ_t depends on the multiset of measurements but not on their order. Now suppose the binary state can flip with probability q per step (a door someone opens). Write the prediction step for that case, show that it is *not* additive in log odds, and explain in one sentence why this is the assumption Chapter 13 makes when it declares the map static — and what it costs when a person walks through the room.

4 CONCEPTUAL • Predict, then spin

In [w8.2](#w8.2), press `<em>Degenerate</em>` to set the weights to $(0.96, 0.01, 0.01, 0.01, 0.01)$ with $M = 5$. Before pressing `<em>Spin  $\times 1000$ </em>`, predict both offspring histograms. Specifically: under each scheme, what is the probability that particle 1 receives fewer than four offspring, and can particle 1 ever receive zero? Then run it and check. (Hint for the comb: what is Mw_1 , and what does Exercise 2 say about offspring counts when that number is not an integer?)

5 CONCEPTUAL •• Find the deprivation cliff

In [w8.1](#w8.1), load the deprivation preset and bisect on the M slider to find the smallest particle count for which fewer than 2 of 20 re-rolled seeds converge to the wrong door. Now switch on KLD mode and read the steady-state population once the belief has a single mode. The two numbers will disagree — KLD's will be smaller. Explain why, in terms of what each number is protecting against. (One is sizing for *representing* a converged belief; the other is sizing for *surviving* the transient.)

6 PRACTICAL •• Stratified resampling, and a variance ranking

Implement `stratified_resample` — one uniform draw per comb interval, $u_m \sim U[(m-1)/M, m/M)$ — beside the two schemes in `resample.rs`. Derive its per-particle offspring variance, then measure all three over 10^5 trials on the chapter's weights and rank them. Does the measured ranking match your derivation? Add the measurement as a test with a tolerance wide enough to be seed-stable but tight enough to fail if someone swaps the schemes.

7 PRACTICAL ••• When not to resample, measured

Sweep `resample_threshold` from 0.0 (never resample) to 1.0 (resample every step) in steps of 0.1. For each value, run the Hallway filter at $M = 100$ across 50 seeds and record two numbers: the deprivation rate (fraction of runs whose final MAP is at the

wrong door) and the mean M_{eff} over the run. Plot both against the threshold. You should find a U-shaped failure curve — degeneracy on the left, deprivation on the right — and the bottom of that U is the empirical version of the $M/2$ convention. Report where it actually lands for this problem.

8.8 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 4 is this chapter's spine: the discrete Bayes filter (Table 4.1), the binary static-state filter (Table 4.2), the particle filter (Table 4.3), and the low-variance sampler (Table 4.4). The derivations here follow its structure and extend it where post-2005 practice has moved on. [Link](#)

Gordon, N. J., Salmond, D. J., and Smith, A. F. M. (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation *IEE Proceedings F (Radar and Signal Processing)* 140(2), 107–113.

The bootstrap filter — the first practical particle filter, and the paper that made resampling standard. Everything in this chapter's second half is a refinement of its five-line algorithm. doi:10.1049/ip-f-2.1993.0015

Fox, D. (2003). Adapting the Sample Size in Particle Filters Through KLD-Sampling *The International Journal of Robotics Research* 22(12), 985–1003.

The source of the KLD bound and its Wilson–Hilferty derivation. Not in the 1999 draft this book uses as its baseline; it is 2005-edition material, rebuilt here from the original paper. doi:10.1177/0278364903022012001

Li, T., Boli, M., and Djuri, P. M. (2015). Resampling Methods for Particle Filtering: Classification, Implementation, and Strategies *IEEE Signal Processing Magazine* 32(3), 70–86.

The taxonomy that organizes multinomial, stratified, systematic and residual resampling into one family, with the variance comparisons Exercise 6 asks you to reproduce. doi:10.1109/MSP.2014.2330626

Chopin, N. and Papaspiliopoulos, O. (2020). *An Introduction to Sequential Monte Carlo* Springer Series in Statistics.

The rigorous modern treatment. Read it for the convergence results this chapter states informally, and for the proof that the self-normalized estimator's bias is $O(1/M)$. doi:10.1007/978-3-030-47845-2

Elvira, V., Míguez, J., and Djuri, P. M. (2021). On the performance of particle filters with adaptive number of particles *Statistics and Computing* 31.

KLD-sampling's modern successor: adapt M from online predictive statistics rather than from bin occupancy, with error bounds that follow the adaptation. The right reference if you find KLD's bin size doing too much of the work. doi:10.1007/s11222-021-10056-0

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., and Ferguson, M. (2023). From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2 *Robotics and Autonomous Systems* 168, 104493.

Evidence for this chapter’s claim that Table 4.3 still ships: AMCL — this particle filter with Chapter 9’s and Chapter 10’s models — remains the Nav2 default localizer. Written by the people who maintain it. doi:10.1016/j.robot.2023.104493

Chen, X. and Li, Y. (2025). An overview of differentiable particle filters for data-adaptive sequential Bayesian inference *Foundations of Data Science* 7(4), 915–943.

Where resampling goes when you need gradients through it: soft and optimal-transport resamplers that keep the filter differentiable. Chapter 25 picks this up. doi:10.3934/fods.2023014

Part III

Probabilistic Models

Probabilistic Motion Models

PROBABILISTIC MODELS · Intermediate · 55 min



In practice, the exact shape of the model often seems to be less important than the fact that some provisions for uncertain outcomes are provided in the first place.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 5

Every filter in Part II (Chapter 5) consumed a motion model as a black box. The prediction step integrates against $p(x_t | u_t, x_{t-1})$; the Kalman filter needs its Jacobian; the particle filter needs to draw from it. Nobody said where it comes from. This chapter opens the box.

The story has a shape, and the shape is a banana. Command Rusty to drive a gentle one-second arc, run that same command a thousand times, and the cloud of final poses is not a tidy ellipse. It is curved and lopsided, because a small rotational error early in the motion is multiplied by every metre of translation that follows. That curve is not a rendering artifact and it is not something better hardware would fix. It is what motion uncertainty on a pose manifold actually looks like.

The modern punchline is that the banana is tame. Describe the same distribution in exponential coordinates — the twist that takes the robot from where it was to where it is — and it becomes an ordinary Gaussian, with a mean and a covariance and nothing curved about it. The banana was never the distribution's fault. It was the coordinates'.

◎ AFTER THIS CHAPTER YOU CAN

- Write down $p(x_t | u_t, x_{t-1})$ for a differential drive, from velocity commands and from wheel odometry
- Derive the exact arc update from the instantaneous centre of curvature,

including its straight-line limit

- Invert the motion to evaluate the closed-form density, and say what the inversion quietly throws away
- Explain what each of the six α parameters does to the shape of the predicted cloud
- State and use the result that the banana distribution is exactly Gaussian in exponential coordinates
- Propagate a tangent-space covariance over many steps with the adjoint, and know when that first-order propagation stops being honest

ASSUMED

- $SE(2)$, $\exp/\log$, and $\boxplus/\boxminus$ (Chapter 3)
- Rusty's differential drive and wheel encoders (Chapter 4)
- The Bayes filter prediction step (Chapter 5)
- Linearization and Jacobians (Chapter 7)
- Sampling and particle sets (Chapter 8)

9.1 The problem: one command, a thousand outcomes

Rusty is standing still. We send exactly one instruction: *drive at 1 m/s while turning at 0.5 rad/s, for one second*. The kinematics say where that puts him, to ten decimal places.

Now send the same instruction a thousand times, from the same starting pose, on the same floor.

INTERACTIVE · w9.1

The Banana Machine

Motion noise is not an ellipse around the predicted pose. It is a banana — and the banana is a perfectly ordinary Gaussian, drawn in the wrong coordinates.

Run this simulation in the web edition: `/chapters/ch09-motion-models #w9.1`

Three things in that cloud are worth naming before we do any algebra, because each one becomes a piece of the mathematics below.

The cloud is bent. Not "roughly elliptical with a bit of skew" — genuinely crescent-shaped, with a concave side and a convex side. The reason is mechanical: a sample that turned slightly harder than commanded spends the whole second driving in a slightly tighter circle, so its heading error and its lateral position error are the same error, seen twice. That coupling is a *product*, not a sum, and

products bend distributions.

The ellipse is a lie, and it lies in a specific direction. Moment-match a Gaussian to that cloud in (x, y) and its 2σ contour bulges off the outside of the bend into a region no execution ever visited, while cutting the corner on the inside. An EKF (Chapter 7) carries exactly that ellipse. The readout under the widget is the number that proves it: the skewness of the cross-track residual is exactly zero for any Gaussian in (x, y) , and for the default command it settles around -0.4 .

Each noise parameter bends it differently. α_4 fans the arc open. α_1 smears the cloud along the arc without changing its curvature. α_6 blurs the headings and moves no positions at all. Six numbers, six distinguishable effects — which is why tuning them is a skill and not a shrug.

💡 WHAT A MOTION MODEL IS FOR

A motion model is a generative story: *the robot intended u_t , the world corrupted it, and here is the distribution over where it ended up*. Everything in this chapter is a choice about where in that story to inject the noise — into the commanded velocities, into the odometry legs, or into the twist itself — and each choice produces a different, checkable density.

9.2 Building intuition

9.2.1 Two models, because there are two kinds of question

The book uses two motion models and they are not interchangeable.

The **velocity model** answers *what will happen if I do this?* Its control is the command $u_t = (v \ \omega)^\top$ that was sent to the motors. It is available before the motion happens, which is what makes it the model that planning and control use — Chapter 23 rolls out thousands of imagined futures with it, and none of those futures have odometry.

The **odometry model** answers *where did I end up?* Its control is a pair of poses read out of the wheel encoders, $u_t = (\bar{x}_{t-1} \ \bar{x}_t)$. It is only available afterwards, and it is strictly more accurate, because wheels measure what actually turned rather than what was asked for. That is why every deployed localizer — MCL (Chapter 12) included, and AMCL in ROS 2 to this day — runs on the odometry model.

📌 NOTE

The bar in $\bar{x}_t$ matters. Those are poses in the robot's own internal odometry frame, which drifts freely with respect to the map. Only their *difference* is

used. The model never assumes the odometry frame and the world frame agree, and if you ever find yourself comparing $\bar{x}_t$ to x_t directly, something has gone wrong.

9.2.2 Where noise gets injected

The deterministic kinematics of Chapter 4 are a function: command in, pose out. To make it probabilistic you have to corrupt something, and the whole character of the resulting distribution depends on what you corrupt.

You could add noise to the output pose — $x_t = f(u_t, x_{t-1}) + \varepsilon$ — and that is what "additive Gaussian process noise" means in a textbook Kalman filter. It gives an ellipse. It also says a robot commanded to move 5 cm has the same positional uncertainty as one commanded to cross the room, which is plainly false.

Thrun's models corrupt the *input* instead. The noise enters the controls, and then the exact nonlinear kinematics carry it forward. That single decision buys three things at once: the uncertainty scales with the size of the commanded motion, it stays inside the set of poses the robot can physically reach, and it bends — because the kinematics bend.

9.3 The mathematics

9.3.1 Probabilistic kinematics

$u_t = (v \ \omega)^\top$	Velocity control: translational and rotational velocity, held constant over Δt .
Δt	Length of the control interval, in seconds.
ε_{b^2}	A zero-mean random variate of variance b^2 — normal or triangular.
$\alpha_1 \dots \alpha_6$	Motion-noise parameters. Variance coefficients, not standard deviations.
$\hat{v}, \hat{\omega}, \hat{\gamma}$	The perturbed velocities actually executed, plus a final-rotation slack $\hat{\gamma}$.
$\delta_{rot1}, \delta_{trans}, \delta_{rot2}$	The odometry decomposition: rotate, drive, rotate.
$\bar{x}_{t-1}, \bar{x}_t$	Odometry poses, in the robot's own drifting internal frame.
$\tau_t \in \mathfrak{se}(2)$	The twist a command asks for: $\tau = (v\Delta t, 0, \omega\Delta t)^\top$.
R_u	Motion-noise covariance expressed in the tangent space. An $\mathbb{R}$, not a $\mathbb{Q}$ (Chapter 6).
$\xi = x_t \boxminus x_{t-1}$	Exponential coordinates of the executed motion: $\log(x^{-1} x')$.

The **motion model**, also called the probabilistic kinematic model, is the conditional density

$$p(x_t \mid u_t, x_{t-1})$$

over poses $x_t = (x \ y \ \theta)^\top \in \text{SE}(2)$. It is the integrand of the Bayes-filter prediction step, and that is its entire job: given a distribution over where the robot was, produce one over where it is.

Two assumptions are built in and neither is free. First, the control is held **constant** over the interval — the model knows nothing about acceleration profiles or the fact that the wheels ramp up. Second, consecutive motion errors are **independent** given the poses; a systematically miscalibrated wheel radius violates that, and the correlated error it produces is exactly the Markov failure Chapter 5 warned about.

9.3.2 The velocity motion model

Noise enters through the commanded velocities. Thrun's model perturbs both, and then adds a third term that will need explaining:

$$\hat{v} = v + \varepsilon_{\alpha_1 v^2 + \alpha_2 \omega^2} \quad \hat{\omega} = \omega + \varepsilon_{\alpha_3 v^2 + \alpha_4 \omega^2} \quad \hat{\gamma} = \varepsilon_{\alpha_5 v^2 + \alpha_6 \omega^2}$$

Read the subscripts as a design statement. Every variance is a weighted sum of v^2 and ω^2 : **noise is proportional to motion**. A stationary robot has no motion noise at all, which is both physically right and numerically convenient, since it keeps a parked robot's belief from diffusing away overnight.

The $\hat{\gamma}$ term is the one students ask about. The controls have two degrees of freedom; poses have three. Without a third noise source the model's support would be a two-dimensional surface embedded in three-dimensional pose space, and a particle filter built on it could never represent "I am here, but I might be facing slightly differently." $\hat{\gamma}$ is a final rotation applied at the end of the arc, purely to restore full rank. It is a modelling hack, it is honest about being one, and every implementation keeps it.

Given the perturbed controls, the pose is propagated *exactly*.

DERIVATION The exact arc update, from the instantaneous centre of curvature

$$x' = x - (v/\omega)\sin\theta + (v/\omega)\sin(\theta + \omega\Delta t)$$

Step 1 — constant controls trace a circle. With $\hat{v}$ and $\hat{\omega}$ held fixed, the robot's speed along its path is constant and its heading rate is constant, so the path has constant curvature $\hat{\omega}/\hat{v}$. That is a circular arc of radius

$$r = \frac{\hat{v}}{\hat{\omega}}$$

Step 2 — locate the centre. A rolling wheelset cannot slip sideways, so

the velocity is always along the heading, and the centre of the circle must be perpendicular to it. Signed radius r to the *left* of the heading puts the instantaneous centre of curvature at

$$\text{ICC} = (x - r \sin \theta, y + r \cos \theta)$$

Check the sign: for $\theta = 0$ and $\omega > 0$ the robot turns left, and the ICC sits at $(x, y + r)$, directly to its left. Good.

Step 3 — rotate the pose about the ICC. Over Δt the robot sweeps $\Delta\theta = \hat{\omega}\Delta t$ radians around that point. Rotating the position by $\Delta\theta$ about the ICC and adding $\Delta\theta$ to the heading gives

$$x' = x - \frac{\hat{v}}{\hat{\omega}} \sin \theta + \frac{\hat{v}}{\hat{\omega}} \sin(\theta + \hat{\omega}\Delta t)$$

$$y' = y + \frac{\hat{v}}{\hat{\omega}} \cos \theta - \frac{\hat{v}}{\hat{\omega}} \cos(\theta + \hat{\omega}\Delta t)$$

$$\theta' = \theta + \hat{\omega}\Delta t + \hat{\gamma}\Delta t$$

Step 4 — the straight-line limit. As $\hat{\omega} \rightarrow 0$ the radius diverges and both position terms are $\infty \cdot 0$. Expand: $\sin(\theta + \hat{\omega}\Delta t) - \sin \theta = \hat{\omega}\Delta t \cos \theta - \frac{1}{2}(\hat{\omega}\Delta t)^2 \sin \theta + O(\hat{\omega}^3)$, so

$$\frac{\hat{v}}{\hat{\omega}} [\sin(\theta + \hat{\omega}\Delta t) - \sin \theta] = \hat{v}\Delta t \cos \theta - \frac{1}{2}\hat{v}\hat{\omega}\Delta t^2 \sin \theta + O(\hat{\omega}^2) \rightarrow \hat{v}\Delta t \cos \theta$$

and likewise $y' \rightarrow y + \hat{v}\Delta t \sin \theta$. The limit exists and is the obvious one.

Step 5 — the branch a real implementation needs. The limit existing is not the same as the formula being computable. Near $\hat{\omega} = 0$ the code subtracts two nearly equal sines — losing digits to cancellation — and then multiplies the remainder by a radius of order $\hat{v}/\hat{\omega}$, which amplifies whatever error survived. The two error sources run in opposite directions:

$$\underbrace{\epsilon_{\text{mach}} \frac{\hat{v}}{|\hat{\omega}|}}_{\text{arc form}}$$

versus

$$\underbrace{\frac{1}{2}\hat{v}|\hat{\omega}|\Delta t^2}_{\text{straight-line form}}$$

They cross at $|\hat{\omega}| = \sqrt{2\varepsilon_{\text{mach}}}/\Delta t \approx 2 \times 10^{-8}/\Delta t$ rad/s. The threshold this book’s code uses, 10^{-6} rad/s, is two decades above that: deliberately conservative, and costing at most half a micrometre of position per second of driving at the switch point. Both the Rust and the TypeScript implementations branch there, and a unit test pins the two branches against each other across the crossover. ■

Sampling is now trivial: draw three noise terms, push them through the arc. This is Thrun’s Table 5.3. Its odometry twin, two sections down, is the single most-executed function in this book: every particle of every MCL (Chapter 12) run calls it once per step.

ALGORITHM `sample_motion_model_velocity(u_t, x_{t-1})` **COST** $O(1)$ -- three normal draws and a handful of transcendentals

in `control(v, ω), previous pose(x, y, θ)`

out a pose x_t drawn from $p(x_t | u_t, x_{t-1})$

```

1:  $\hat{v} = v + \text{sample}(\alpha_1 v^2 + \alpha_2 \omega^2)$ 
2:  $\hat{\omega} = \omega + \text{sample}(\alpha_3 v^2 + \alpha_4 \omega^2)$ 
3:  $\hat{\gamma} = \text{sample}(\alpha_5 v^2 + \alpha_6 \omega^2)$ 
4:  $x' = x - \frac{\hat{v}}{\hat{\omega}} \sin \theta + \frac{\hat{v}}{\hat{\omega}} \sin(\theta + \hat{\omega} \Delta t)$ 
5:  $y' = y + \frac{\hat{v}}{\hat{\omega}} \cos \theta - \frac{\hat{v}}{\hat{\omega}} \cos(\theta + \hat{\omega} \Delta t)$ 
6:  $\theta' = \theta + \hat{\omega} \Delta t + \hat{\gamma} \Delta t$ 
7: return  $x_t = (x' \ y' \ \theta')^T$ 
```

`sample( $b^2$ )` draws from a zero-mean distribution of **variance** b^2 — the argument is a variance, not a standard deviation, and mixing that up is the most common bug in a first implementation. Thrun offers two flavours: normal, and triangular (which has bounded support, so it can never produce an absurd outlier). He also gives a normal sampler built from twelve uniforms, which was a sensible trick in 1999 and is now a historical footnote: `rand_distr::Normal` uses the Ziggurat algorithm and is both exact and faster.

9.3.3 Inverting the motion

A particle filter only ever samples. A histogram filter, an EKF innovation test, or any hypothesis scoring needs the density itself: given a *specific* x_t , how likely was it?

The trick is that the map from noise to pose is invertible. There is exactly one circular arc that leaves x_{t-1} tangentially and passes through the position of x_t ; find it, read off the $(\hat{v}, \hat{\omega})$ it implies, and evaluate the noise densities there.

 INTERACTIVE · w9.2
Arc Anatomy

The sampler and the closed-form density are the same geometry read in opposite directions — forward through the ICC, backward through it.

Run this simulation in the web edition: `/chapters/ch09-motion-models #w9.2`

 DERIVATION Recovering $(\hat{v}, \hat{\omega}, \hat{\gamma})$ from a pose pair

$$\mu = \cdot [(x-x')\cos \theta + (y-y')\sin \theta] / [(y-y')\cos \theta - (x-x')\sin \theta]$$

Step 1 — two constraints on the centre. The arc passes through (x, y) and (x', y') , so its centre (x^*, y^*) lies on the perpendicular bisector of that segment. It must also be perpendicular to the initial heading, so it lies on the line through (x, y) with direction $(-\sin \theta, \cos \theta)$. Two lines, one intersection.

Step 2 — parameterize the bisector. Write the centre as the midpoint plus a multiple of the segment's normal:

$$(x^*, y^*) = \frac{1}{2}(x + x', y + y') + \mu (y - y', x' - x)$$

for an unknown scalar μ . Any μ satisfies the bisector constraint.

Step 3 — impose perpendicularity to the heading. Requiring $((x, y) - (x^*, y^*)) \cdot (\cos \theta, \sin \theta) = 0$ and solving for μ gives

$$\mu = \frac{1}{2} \frac{(x - x') \cos \theta + (y - y') \sin \theta}{(y - y') \cos \theta - (x - x') \sin \theta}$$

Step 4 — read off the motion. With the centre known, $r^* = \|(x, y) - (x^*, y^*)\|$ and the swept angle is the difference of two atan2 bearings from the centre, wrapped to $(-\pi, \pi]$:

$$\Delta \theta = \text{atan2}(y' - y^*, x' - x^*) - \text{atan2}(y - y^*, x - x^*)$$

$$\hat{\omega} = \frac{\Delta \theta}{\Delta t}, \quad \hat{v} = \frac{\Delta \theta}{\Delta t} r^*, \quad \hat{\gamma} = \frac{\theta' - \theta}{\Delta t} - \hat{\omega}$$

The arc matched the two positions and the *initial* heading. It said nothing about the final heading, so whatever mismatch remains is charged to $\hat{\gamma}$ — which is the reason $\hat{\gamma}$ was introduced in the first place. The wrap to $(-\pi, \pi]$ means the model assumes $|\omega \Delta t| < \pi$ over a single step; at 10 Hz that is a 31 rad/s spin, so it is a safe assumption and a real one.

Step 5 — the degenerate cases. The denominator vanishes when the displacement is parallel to the heading: the centre is at infinity, the arc is a line, and the code takes the limit directly rather than dividing by ≈ 0 . When $x_t = x_{t-1}$ exactly, *every* arc works and μ is genuinely undefined; the model has nothing to say about a robot that did not move.

Step 6 — the debt nobody pays. Steps 1–4 are a change of variables $(\hat{\nu}, \hat{\omega}, \hat{\gamma}) \mapsto x_t$, so the density over poses is the density over control errors *times the Jacobian determinant of the inverse map*. Table 5.1 evaluates the three noise densities and stops. The missing factor is a function of x_t , so it does not cancel into the normalizer η , and the closed form is therefore not exactly the density of the Table 5.3 sampler — they differ by a few percent in total variation for realistic noise. We implement the table verbatim anyway, because in its actual job — a smooth, correctly-peaked scoring function for nearby hypotheses — the discrepancy is invisible, and because a reader checking this book against Thrun should find the same code. But the honest statement is that `motion_model_velocity` is a *scoring function*, not the normalized density of `sample_motion_model_velocity`. The on-manifold model at the end of this chapter has no such debt. ■

ALGORITHM `motion_model_velocity(x_t, u_t, x_{t-1})`

COST 0(1)

in hypothesised pose x_t , control (v, ω) , previous pose x_{t-1}

out $p(x_t \mid u_t, x_{t-1})$, up to the change-of-variables factor

- 1: $\mu = \frac{1}{2} \frac{(x - x') \cos \theta + (y - y') \sin \theta}{(y - y') \cos \theta - (x - x') \sin \theta}$
- 2: $x^* = \frac{x+x'}{2} + \mu(y - y')$
- 3: $y^* = \frac{y+y'}{2} + \mu(x' - x)$
- 4: $r^* = \sqrt{(x - x^*)^2 + (y - y^*)^2}$
- 5: $\Delta\theta = \text{atan2}(y' - y^*, x' - x^*) - \text{atan2}(y - y^*, x - x^*)$
- 6: $\hat{\nu} = \frac{\Delta\theta}{\Delta t} r^*$
- 7: $\hat{\omega} = \frac{\Delta\theta}{\Delta t}$
- 8: $\hat{\gamma} = \frac{\theta' - \theta}{\Delta t} - \hat{\omega}$
- 9: **return** $\text{prob}(v - \hat{\nu}, \alpha_1 v^2 + \alpha_2 \omega^2) \cdot \text{prob}(\omega - \hat{\omega}, \alpha_3 v^2 + \alpha_4 \omega^2) \cdot \text{prob}(\hat{\gamma}, \alpha_5 v^2 + \alpha_6 \omega^2)$

9.3.4 The odometry motion model

Wheel encoders do not report velocities; they report that the robot's internal pose estimate moved from $\bar{x}_{t-1} = (\bar{x} \ \bar{y} \ \bar{\theta})^\top$ to $\bar{x}_t = (\bar{x}' \ \bar{y}' \ \bar{\theta}')^\top$. Any such change

decomposes uniquely into *rotate*, *drive*, *rotate*:

$$\delta_{rot1} = \text{atan2}(\bar{y}' - \bar{y}, \bar{x}' - \bar{x}) - \bar{\theta}, \quad \delta_{trans} = \sqrt{(\bar{x}' - \bar{x})^2 + (\bar{y}' - \bar{y})^2}, \quad \delta_{rot2} = \bar{\theta}' - \bar{\theta} - \delta_{rot1}$$

Three numbers reach any pose change from any starting pose, so nothing is lost — except when $\delta_{trans} = 0$, where δ_{rot1} is the atan2 of nothing and the split between the two rotations is arbitrary. A pure in-place spin is a very common command, so the implementation special-cases it and charges the whole rotation to δ_{rot2} .

The model's real content is what it does next: it treats the three legs as **independently** noisy.

$$\hat{\delta}_{rot1} = \delta_{rot1} - \varepsilon_{\alpha_1 \delta_{rot1}^2 + \alpha_2 \delta_{trans}^2}, \quad \hat{\delta}_{trans} = \delta_{trans} - \varepsilon_{\alpha_3 \delta_{trans}^2 + \alpha_4 (\delta_{rot1}^2 + \delta_{rot2}^2)}, \quad \hat{\delta}_{rot2} = \delta_{rot2} - \varepsilon_{\alpha_1 \delta_{rot2}^2 + \alpha_2 \delta_{trans}^2}$$

Note the α bookkeeping, which is not arbitrary. The two rotations share (α_1, α_2) because they are the *same physical process* — the same wheels, the same slip. Rotation noise grows with translation (α_2) because a wheel that slips while driving also mis-reports heading. Translation noise grows with rotation (α_4) for the mirror-image reason. Four parameters, not six, because there is no rank deficiency to patch: three noisy legs already span three degrees of freedom, so the odometry model needs no $\hat{\gamma}$.

This is a deliberate fiction. Real wheel slip correlates the legs — a carpet edge that steals distance also steals heading, at the same instant. The model says otherwise, and it works anyway, mostly because the fiction is *conservative*: independent noises produce a wider cloud than correlated ones of the same marginal size, and a filter that is slightly too uncertain survives where an overconfident one dies.

INTERACTIVE · w9.3

The Odometry Decomposer

Odometry error is not additive in (x, y, θ) . It enters as three independent hinges, and the first hinge is multiplied by everything after it.

Run this simulation in the web edition: `/chapters/ch09-motion-models #w9.3`

```
ALGORITHM sample_motion_model_odometry(u_t, x_{t-1})    COST 0(1) -- three
normal draws; this is MCL's inner loop
in odometry pair ( $\bar{x}_{t-1}$ ,  $\bar{x}_t$ ), previous pose  $x_{t-1}$ 
out a pose  $x_t$  drawn from  $p(x_t \mid u_t, x_{t-1})$ 
```

```

1:  $\delta_{rot1} = \text{atan2}(\bar{y}' - \bar{y}, \bar{x}' - \bar{x}) - \bar{\theta}$ 
2:  $\delta_{trans} = \sqrt{(\bar{x}' - \bar{x})^2 + (\bar{y}' - \bar{y})^2}$ 
3:  $\delta_{rot2} = \bar{\theta}' - \bar{\theta} - \delta_{rot1}$ 
4:  $\hat{\delta}_{rot1} = \delta_{rot1} - \text{sample}(\alpha_1 \delta_{rot1}^2 + \alpha_2 \delta_{trans}^2)$ 
5:  $\hat{\delta}_{trans} = \delta_{trans} - \text{sample}(\alpha_3 \delta_{trans}^2 + \alpha_4 (\delta_{rot1}^2 + \delta_{rot2}^2))$ 
6:  $\hat{\delta}_{rot2} = \delta_{rot2} - \text{sample}(\alpha_1 \delta_{rot2}^2 + \alpha_2 \delta_{trans}^2)$ 
7:  $x' = x + \hat{\delta}_{trans} \cos(\theta + \hat{\delta}_{rot1})$ 
8:  $y' = y + \hat{\delta}_{trans} \sin(\theta + \hat{\delta}_{rot1})$ 
9:  $\theta' = \theta + \hat{\delta}_{rot1} + \hat{\delta}_{rot2}$ 
10: return  $x_t = (x' \ y' \ \theta')^T$ 

```

The closed form, `motion_model_odometry` (Table 5.5), follows the same invert-then-evaluate pattern: decompose the *hypothesised* motion $x_{t-1} \rightarrow x_t$ into its own $(\hat{\delta}_{rot1}, \hat{\delta}_{trans}, \hat{\delta}_{rot2})$, and ask how likely the noise was to turn the measured legs into those. It carries the same unpaid Jacobian — for the odometry model the missing factor is exactly $1/\hat{\delta}_{trans}$, which is small consolation and easy to restore if you want the two to agree to Monte Carlo error.

9.3.5 Motion and maps

The models so far know nothing about walls. Given enough noise, `sample_motion_model_velocity` will cheerfully place Rusty inside a load-bearing column. If a map m is available, conditioning on it is a free improvement:

$$p(x_t \mid u_t, x_{t-1}, m) = \eta \, p(x_t \mid m) \, p(x_t \mid u_t, x_{t-1})$$

with $p(x_t \mid m)$ taken to be uniform over free space and zero inside obstacles. Sampling is rejection sampling: draw from the map-free model, keep the draw if it landed somewhere legal, repeat. Expected cost is $1/p_{\text{acc}}$ draws — cheap in a corridor, unbounded if the robot is wedged.

```

ALGORITHM sample_motion_model_with_map(u_t, x_{t-1}, m)    COST 0(1 / p_acc)
expected draws -- bound it, or an animation frame will hang

in control, previous pose, map
out a pose drawn from the map-conditioned model

```

```

1: repeat
2:    $x_t = \text{sample\_motion\_model}(u_t, x_{t-1})$ 
3:    $\pi = p(x_t \mid m)$ 
4: until  $\pi > 0$ 

```

5: return x_t

The approximation is hiding in line 3, and it is worth being blunt about it. $p(x_t | m)$ is a function of a *pose*. It can only ask whether the robot’s final resting place is legal. It cannot ask whether the robot got there legally, because the path is not one of its arguments.

INTERACTIVE · w9.4

Map Squeeze

Conditioning motion on the map is not clipping. It renormalizes the banana into free space — and it checks only where the sample stopped, never how it got there.

Run this simulation in the web edition: `/chapters/ch09-motion-models #w9.4`

Point Rusty at a wall he cannot pass and four out of five accepted samples will be on the far side of it. That is not an implementation bug; it is the exact content of the factorization above. The fix is to test the whole arc — the geodesic from x_{t-1} to x_t — against the map, which is what the ringed samples in the widget are computed with. It costs a handful of segment-intersection tests per sample, it is what a modern collision-aware sampler does with `parry2d`, and it is why the `MapConditioned` wrapper in `crates/motion/src/map_cond.rs` returns both verdicts rather than one.

9.3.6 The banana is a Gaussian

Everything so far is 2005 material. Here is what changed.

Write the noise-free arc update as a group operation instead of three trigonometric formulas. From Chapter 3, the exponential map on $SE(2)$ takes a twist $\tau = (v_x \ v_y \ \omega)^T$ to a pose. Set $\tau_t = (v\Delta t, 0, \omega\Delta t)^T$ — drive forward, no sideways slip, turn — and compute $\exp(\tau_t)$:

$$\exp(v\Delta t, 0, \omega\Delta t) = \left(\frac{\sin(\omega\Delta t)}{\omega\Delta t} v\Delta t, \frac{1-\cos(\omega\Delta t)}{\omega\Delta t} v\Delta t, \omega\Delta t \right) = \left(\frac{v}{\omega} \sin(\omega\Delta t), \frac{v}{\omega} (1 - \cos(\omega\Delta t)), \omega\Delta t \right)$$

which is the arc update of the derivation above, written in the body frame. Not an approximation of it — the same three numbers. So

$$x_t = x_{t-1} \boxplus \tau_t$$

is the exact noise-free kinematics, and the $\omega \rightarrow 0$ branch that cost us a page is already inside $\exp$ ’s Taylor series, where it belongs.

Now put the noise back. The velocity model perturbs v and ω *additively*, and τ_t depends on them *linearly*. Therefore the sampled twist is

$$\hat{\tau}_t = \tau_t + \mathbf{w}, \quad \mathbf{w} \sim \mathcal{N}(0, R_u), \quad R_u = \Delta t^2 \text{diag}(\alpha_1 v^2 + \alpha_2 \omega^2, 0, \alpha_3 v^2 + \alpha_4 \omega^2)$$

and the whole model collapses to one line:

$$x_t = x_{t-1} \boxplus (\tau_t + \mathbf{w})$$

DERIVATION Why the banana is exactly Gaussian in exponential coordinates

$\xi = x_t \boxminus x_{t-1} \sim \mathcal{N}(\tau_t, R_u)$, exactly, when $\alpha = \alpha = 0$

Step 1 — anchor a chart at the start pose. Define the coordinates

$$\xi = x_t \boxminus x_{t-1} = \log(x_{t-1}^{-1} x_t) \in \mathbb{R}^3$$

This is a genuine chart: $\log$ is a diffeomorphism from a neighbourhood of the identity onto $\mathbb{R}^3$, and it is injective for $|\theta| < \pi$, which covers any single control step. Crucially the chart is anchored at x_{t-1} , which is *known* — not at the predicted mean, which would make it a local approximation.

Step 2 — the model is affine in these coordinates. By the line above, $x_t = x_{t-1} \circ \exp(\tau_t + \mathbf{w})$, so $x_{t-1}^{-1} x_t = \exp(\tau_t + \mathbf{w})$ and hence $\xi = \tau_t + \mathbf{w}$ identically. No linearization, no small-angle assumption.

Step 3 — conclude. $\mathbf{w}$ is Gaussian, so $\xi \sim \mathcal{N}(\tau_t, R_u)$ **exactly**. The crescent in (x, y) is the image of an ordinary ellipsoid under the smooth nonlinear map $\xi \mapsto x_{t-1} \circ \exp(\xi)$, projected onto two of three coordinates. Curvature of the picture, not of the distribution.

Step 4 — the caveats, stated plainly.

- R_u has a zero in the middle: a differential drive cannot command lateral slip, so the noise is rank 2 on a 3-dimensional group. That is the same rank deficiency $\hat{\gamma}$ was invented to patch.
- With $\alpha_5, \alpha_6 > 0$ the statement becomes approximate, because $\hat{\gamma}$ is applied as a rotation *after* the arc: $x_t = x_{t-1} \circ \exp(\tau_t + \mathbf{w}) \circ \exp(0, 0, \hat{\gamma} \Delta t)$, and $\exp(a) \exp(b) \neq \exp(a + b)$ on a non-commutative group. Folding $\Delta t^2(\alpha_5 v^2 + \alpha_6 \omega^2)$ into the $\omega\omega$ entry of R_u recovers the heading variance to three digits but leaks a little variance into the lateral coordinate that the additive model says is exactly zero. Measured on 200 000 samples of the canonical example with $\alpha_5 = \alpha_6 = 0.005$: predicted $R_u^{\omega\omega} = 0.03125$, measured 0.0312; predicted lateral variance 0, measured 0.0016.
- Right versus left matters. We perturb in the *body* frame, $x \boxplus \tau = x \circ \exp(\tau)$. The left convention $\exp(\tau) \circ x$ describes noise in the world frame and gives a different (also Gaussian) answer; the two are related by the adjoint. Mixing them is the most common way to get a covariance

rotated by θ and spend a week wondering why.

- The odometry model does **not** have this property. Its three noises enter at three different points of a rotate–drive–rotate composition, not additively in one tangent space, so its exponential coordinates are only approximately Gaussian. Switch the Banana Machine to the odometry model and watch the fitted contour fit slightly worse. ■

This is the result Long, Wolfe, Mashner and Chirikjian published in 2012, and it is why Chapter 7’s error-state EKF and Part V (Chapter 15)’s factor graphs both keep their state on the manifold: they are working in the coordinates where the Gaussian assumption is true instead of merely tolerable.

DERIVATION Compounding: propagating R_u over many steps with the adjoint

$$\Sigma_k = A \Sigma_{k-1} A^\top + R_u, A = \text{Ad}(\exp(\tau)^{-1})$$

Step 1 — separate mean and error. Write each pose as its nominal value perturbed in the body frame: $x_k = \bar{x}_k \boxplus \epsilon_k$, with $\bar{x}_k = \bar{x}_{k-1} \circ \exp(\tau)$.

Step 2 — push the previous error through the new motion.

$$x_k = x_{k-1} \circ \exp(\tau + \mathbf{w}_k) \approx \bar{x}_{k-1} \exp(\epsilon_{k-1}) \exp(\tau) \exp(\mathbf{w}_k)$$

Insert $\exp(\tau) \exp(-\tau)$ and use the defining property of the adjoint, $T \exp(\epsilon) T^{-1} = \exp(\text{Ad}_T \epsilon)$:

$$= \bar{x}_{k-1} \exp(\tau) \underbrace{\left[\exp(-\tau) \exp(\epsilon_{k-1}) \exp(\tau) \right]}_{\exp(\text{Ad}_{\exp(\tau)^{-1}} \epsilon_{k-1})} \exp(\mathbf{w}_k)$$

Step 3 — read off the recursion. To first order the two tangent perturbations add, so $\epsilon_k \approx A \epsilon_{k-1} + \mathbf{w}_k$ with $A = \text{Ad}_{\exp(\tau)^{-1}}$, and therefore

$$\Sigma_k = A \Sigma_{k-1} A^\top + R_u$$

That is the Kalman prediction equation with the adjoint playing the role of the motion Jacobian G_t — which is exactly what it is, for a state that lives on a group.

Step 4 — where first order stops being enough. The approximation dropped the Baker–Campbell–Hausdorff commutator $\frac{1}{2}[\epsilon, \mathbf{w}]$, whose neglected contribution grows with the heading spread. Twenty steps of

the canonical command at $\Delta t = 0.2$ s, with $\alpha_5 = \alpha_6 = 0.005$ so that R_u has full rank — four seconds of driving, two radians of turn — leave the recursion under-reporting the along-track variance by 6% and the heading variance by 0.2%, measured against 80 000 Monte Carlo rollouts. Barfoot and Furgale’s fourth-order correction closes most of that gap; before you reach for it, check whether your R_u is known to better than 6% in the first place. ■

9.4 Implementation in Rust

The motion crate is small on purpose. It is imported unchanged by Chapter 11, Chapter 12, Chapter 17 and Chapter 23, so every line in it is load-bearing.

RUST crates/motion/src/lib.rs

```
1 use nalgebra::{Matrix3, Vector3};
2 use pr_core::geom::se2::SE2; // Chapter 3: exp / log / boxplus / boxminus
3 use rand::Rng;
4
5 /// A velocity command held constant over an interval.
6 #[derive(Clone, Copy, Debug)]
7 pub struct VelocityCmd {
8     pub v: f64,
9     pub omega: f64,
10    pub dt: f64,
11 }
12
13 /// An odometry reading, already decomposed into rotate-drive-rotate.
14 #[derive(Clone, Copy, Debug)]
15 pub struct OdomDelta {
16     pub rot1: f64,
17     pub trans: f64,
18     pub rot2: f64,
19 }
20
21 /// The six alpha's of the velocity model. Newtyped because the *order* is load
22 /// bearing and a bare `[f64; 6]` at a call site tells the reader nothing.
23 #[derive(Clone, Copy, Debug)]
24 pub struct VelocityAlphas(pub [f64; 6]);
25
26 /// The four alpha's of the odometry model.
27 #[derive(Clone, Copy, Debug)]
28 pub struct OdomAlphas(pub [f64; 4]);
29
30 /// Thrun's `sample(b2)` and `prob(a, b2)` take a **variance**. Making that a
31 /// type stops the single most common bug in a first implementation: handing
32 /// them a standard deviation, and being wrong by a square root ever after.
33 #[derive(Clone, Copy, Debug, PartialEq)]
34 pub struct Variance(pub f64);
35
36 #[derive(Clone, Copy, Debug, PartialEq, Eq)]
37 pub enum NoiseKind {
38     /// Exact, via the Ziggurat sampler in `rand_distr`.
39     Normal,
```

```

40  // Bounded support at +/-sqrt(6).b, so no sample is ever absurd (Table 5.4).
41  Triangular,
42  }
43
44  // Every motion model in this book: a density and a sampler that must agree.
45  //
46  // The associated `Control` is why this is a trait and not two free functions.
47  // A velocity model consumes a command that exists before the motion; an
48  // odometry model consumes a measurement that exists only after it. Nothing
49  // downstream should be able to confuse them, and here nothing can.
50  pub trait MotionModel {
51      type Control: Copy;
52
53      //  $p(x_t | u_t, x_{t-1})$  -- the scoring function of Tables 5.1 / 5.5.
54      fn prob(&self, x_next: &SE2, u: &Self::Control, x: &SE2) -> f64;
55
56      // One draw from that distribution (Tables 5.3 / 5.6).
57      fn sample<R>: Rng + ?Sized>(&self, u: &Self::Control, x: &SE2, rng: &mut R) -> SE2;
58
59      // The tangent-space covariance this model induces, where it has one.
60      // `None` for models whose noise is not additive in  $se(2)$  -- the odometry
61      // model, for instance.
62      fn tangent_cov(&self, _u: &Self::Control) -> Option<Matrix3<f64>> {
63          None
64      }
65  }

```

The velocity model is the exact arc, with the noise applied to the controls and the straight-line branch spelled out.

RUST crates/motion/src/velocity.rs

```

1  use crate::{MotionModel, NoiseKind, VelocityAlphas, VelocityCmd, Variance};
2  use nalgebra::{Matrix3, Vector3};
3  use pr_core::geom::se2::{wrap_angle, SE2};
4  use rand::Rng;
5
6  pub struct VelocityModel {
7      pub alphas: VelocityAlphas,
8      pub noise: NoiseKind,
9  }
10
11  // Below this  $|\omega|$  the arc form subtracts two near-equal sines and multiplies the
12  // remainder by a radius of order  $v/\omega$ , so cancellation error is amplified by the
13  // radius. The two error curves cross near  $\sqrt{2\epsilon}/\Delta t \sim 2e-8$  rad/s;
14  // switching two
15  // decades earlier costs under a micrometre per step and keeps the branch well
16  // clear of the crossing, where neither form is comfortable.
17  const STRAIGHT: f64 = 1e-6;
18
19  impl VelocityModel {
20      fn variances(&self, u: &VelocityCmd) -> [Variance; 3] {
21          let [a1, a2, a3, a4, a5, a6] = self.alphas.0;
22          let (v2, w2) = (u.v * u.v, u.omega * u.omega);
23          [
24              Variance(a1 * v2 + a2 * w2),
25              Variance(a3 * v2 + a4 * w2),
26              Variance(a5 * v2 + a6 * w2),

```



```

26     ]
27 }
28 }
29
30 impl MotionModel for VelocityModel {
31     type Control = VelocityCmd;
32
33     fn sample<R: Rng + ?Sized>(&self, u: &VelocityCmd, x: &SE2, rng: &mut R) -> SE2 {
34         let [sv, sw, sg] = self.variances(u);
35         let v_hat = u.v + self.noise.sample(sv, rng);
36         let w_hat = u.omega + self.noise.sample(sw, rng);
37         let g_hat = self.noise.sample(sg, rng);
38
39         let theta = x.theta();
40         let next_theta = theta + w_hat * u.dt;
41         let (dx, dy) = if w_hat.abs() < STRAIGHT {
42             (v_hat * theta.cos() * u.dt, v_hat * theta.sin() * u.dt)
43         } else {
44             let r = v_hat / w_hat;
45             (
46                 -r * theta.sin() + r * next_theta.sin(),
47                 r * theta.cos() - r * next_theta.cos(),
48             )
49         };
50         // gamma is a rotation applied *after* the arc -- it moves no position.
51         SE2::new(x.x() + dx, x.y() + dy, wrap_angle(next_theta + g_hat * u.dt))
52     }
53
54     fn prob(&self, x_next: &SE2, u: &VelocityCmd, x: &SE2) -> f64 {
55         let inv = invert_arc(x, x_next, u.dt);
56         let [sv, sw, sg] = self.variances(u);
57         self.noise.prob(u.v - inv.v_hat, sv)
58             * self.noise.prob(u.omega - inv.omega_hat, sw)
59             * self.noise.prob(inv.gamma_hat, sg)
60     }
61
62     // R_u = Deltat2 . diag(alpha_1v2 + alpha_2omega2, 0, alpha_3v2 + alpha?omega2). The
63     // zero is the point:
64     // a differential drive has no lateral degree of freedom to be noisy in.
65     fn tangent_cov(&self, u: &VelocityCmd) -> Option<Matrix3<f64>> {
66         let [sv, sw, _] = self.variances(u);
67         let dt2 = u.dt * u.dt;
68         Some(Matrix3::from_diagonal(&Vector3::new(dt2 * sv.0, 0.0, dt2 * sw.0)))
69     }
70
71     pub struct ArcInversion {
72         pub v_hat: f64,
73         pub omega_hat: f64,
74         pub gamma_hat: f64,
75     }
76
77     // Table 5.1, lines 1-8: find the unique arc leaving `x` tangentially that
78     // reaches `x_next`'s position, and report the controls it implies.
79     fn invert_arc(x: &SE2, x_next: &SE2, dt: f64) -> ArcInversion {
80         let (c, s) = (x.theta().cos(), x.theta().sin());
81         let (dx, dy) = (x.x() - x_next.x(), x.y() - x_next.y());
82         let numer = dx * c + dy * s;
83         let denom = dy * c - dx * s;
84
85         // Displacement parallel to the heading: the centre is at infinity.

```

```

86     if denom.abs() < 1e-9 {
87         return ArcInversion {
88             v_hat: -numer / dt,
89             omega_hat: 0.0,
90             gamma_hat: wrap_angle(x_next.theta() - x.theta()) / dt,
91         };
92     }
93
94     let mu = 0.5 * numer / denom;
95     let cx = 0.5 * (x.x() + x_next.x()) + mu * dy;
96     let cy = 0.5 * (x.y() + x_next.y()) - mu * dx;
97     let r_star = (x.x() - cx).hypot(x.y() - cy);
98     let d_theta = wrap_angle(
99         (x_next.y() - cy).atan2(x_next.x() - cx) - (x.y() - cy).atan2(x.x() - cx),
100     );
101     ArcInversion {
102         v_hat: d_theta / dt * r_star,
103         omega_hat: d_theta / dt,
104         gamma_hat: wrap_angle(x_next.theta() - x.theta()) / dt - d_theta / dt,
105     }
106 }

```

The odometry model is shorter, because there is no arc to integrate — only a hinge, a drive and a hinge. This is the function that actually runs on robots.

RUST crates/motion/src/odometry.rs

```

1  use crate::{MotionModel, NoiseKind, OdomAlphas, OdomDelta, Variance};
2  use pr_core::geom::se2::{wrap_angle, SE2};
3  use rand::Rng;
4
5  pub struct OdometryModel {
6      pub alphas: OdomAlphas,
7      pub noise: NoiseKind,
8  }
9
10 impl OdomDelta {
11     /// Table 5.5, lines 2-4: decompose a relative odometry reading.
12     ///
13     /// A pure in-place spin has no direction of travel, so `atan2` would return
14     /// whatever the floating-point noise in two zeros happens to encode. We give
15     /// the whole rotation to `rot2` instead -- otherwise a stationary turn, which
16     /// is a very common command, produces a nonsense first hinge and a noise
17     /// variance built from it.
18     pub fn from_poses(prev: &SE2, curr: &SE2) -> Self {
19         let (dx, dy) = (curr.x() - prev.x(), curr.y() - prev.y());
20         let trans = dx.hypot(dy);
21         if trans < 1e-9 {
22             return Self { rot1: 0.0, trans: 0.0, rot2: wrap_angle(curr.theta() - prev.
23 theta()) };
24         }
25         let rot1 = wrap_angle(dy.atan2(dx) - prev.theta());
26         Self { rot1, trans, rot2: wrap_angle(curr.theta() - prev.theta() - rot1) }
27     }
28
29     /// Apply an exact (noise-free) decomposition to a pose.
30     pub fn apply(&self, x: &SE2) -> SE2 {
31         let heading = x.theta() + self.rot1;

```

```

31         SE2::new(
32             x.x() + self.trans * heading.cos(),
33             x.y() + self.trans * heading.sin(),
34             wrap_angle(heading + self.rot2),
35         )
36     }
37 }
38
39 impl MotionModel for OdometryModel {
40     /// The control is the decomposed reading. Callers build it with
41     /// `OdometryDelta::from_poses(&odom_prev, &odom_now)` -- never from map-frame
42     /// poses, which is the mistake that makes a localizer fight its own drift.
43     type Control = OdometryDelta;
44
45     fn sample<R: Rng + ?Sized>(&self, u: &OdometryDelta, x: &SE2, rng: &mut R) -> SE2 {
46         let [a1, a2, a3, a4] = self.alphas.0;
47         let (r1sq, r2sq, tsq) = (u.rot1 * u.rot1, u.rot2 * u.rot2, u.trans * u.trans);
48
49         // Note the minus signs: Table 5.6 subtracts the noise. For a symmetric
50         // distribution it makes no difference in law, and keeping it makes this
51         // diffable against the book.
52         let perturbed = OdometryDelta {
53             rot1: u.rot1 - self.noise.sample(Variance(a1 * r1sq + a2 * tsq), rng),
54             trans: u.trans - self.noise.sample(Variance(a3 * tsq + a4 * (r1sq + r2sq)),
rng),
55             rot2: u.rot2 - self.noise.sample(Variance(a1 * r2sq + a2 * tsq), rng),
56         };
57         perturbed.apply(x)
58     }
59
60     fn prob(&self, x_next: &SE2, u: &OdometryDelta, x: &SE2) -> f64 {
61         let [a1, a2, a3, a4] = self.alphas.0;
62         let hat = OdometryDelta::from_poses(x, x_next);
63         let (r1sq, r2sq, tsq) = (hat.rot1 * hat.rot1, hat.rot2 * hat.rot2, hat.trans *
hat.trans);
64
65         self.noise.prob(wrap_angle(u.rot1 - hat.rot1), Variance(a1 * r1sq + a2 * tsq))
66         * self.noise.prob(u.trans - hat.trans, Variance(a3 * tsq + a4 * (r1sq + r2sq)
))
67         * self.noise.prob(wrap_angle(u.rot2 - hat.rot2), Variance(a1 * r2sq + a2 *
tsq))
68     }
69     // No `tangent_cov`: three noises entering at three points of a composition
70     // are not one additive perturbation in se(2). Saying so with `None` is more
71     // useful than returning an approximation a caller would trust.
72 }

```

The on-manifold model is the same physics, written where it is linear. Compare the length.

RUST crates/motion/src/se2_noise.rs

```

1 use crate::VelocityCmd;
2 use nalgebra::{Cholesky, Matrix3, Vector3};
3 use pr_core::geom::se2::SE2;
4 use rand::Rng;
5 use rand_distr::{Distribution, StandardNormal};
6

```

```

7  /// Gaussian noise in se(2), pushed onto the group by [+].
8  ///
9  /// `r` is an R and not a Q: motion noise, per the book-wide convention of
10 /// Chapter 6. It is expressed in the body frame, matching the right
11 /// perturbation  $x \mapsto x \cdot \exp(\tau)$  used everywhere in this book.
12 pub struct TangentNoiseModel {
13     pub r: Matrix3<f64>,
14     chol: Cholesky<f64, nalgebra::U3>,
15 }
16
17 impl TangentNoiseModel {
18     pub fn new(r: Matrix3<f64>) -> Self {
19         // A rank-2 R (no lateral slip) is legitimate, so nudge the diagonal
20         // rather than refusing to factor it.
21         let padded = r + Matrix3::identity() * 1e-12;
22         let chol = Cholesky::new(padded).expect("R must be positive semi-definite");
23         Self { r, chol }
24     }
25
26     /// tau = (vDeltat, 0, omegaDeltat): the twist whose exponential is the arc update.
27     pub fn twist(u: &VelocityCmd) -> Vector3<f64> {
28         Vector3::new(u.v * u.dt, 0.0, u.omega * u.dt)
29     }
30
31     pub fn sample<R: Rng + ?Sized>(&self, u: &VelocityCmd, x: &SE2, rng: &mut R) -> SE2 {
32         let z = Vector3::new(
33             StandardNormal.sample(rng),
34             StandardNormal.sample(rng),
35             StandardNormal.sample(rng),
36         );
37         x.boxplus(Self::twist(u) + self.chol.l() * z)
38     }
39
40     /// log p(x_t | u, x_{t-1}) -- a plain 3-D Gaussian in xi = x_t [-] x_{t-1}.
41     ///
42     /// No arc inversion, no unpaid change of variables: because the chart is
43     /// anchored at x_{t-1} rather than at the predicted mean, xi = tau + w holds
44     /// identically and this really is the sampler's density.
45     pub fn log_prob(&self, x_next: &SE2, u: &VelocityCmd, x: &SE2) -> f64 {
46         let d = x_next.boxminus(x) - Self::twist(u);
47         let y = self.chol.l().solve_lower_triangular(&d).expect("L is nonsingular");
48         let log_det: f64 = self.chol.l().diagonal().iter().map(|l| 2.0 * l.ln()).sum();
49         -0.5 * (y.dot(&y) + log_det + 3.0 * (2.0 * std::f64::consts::PI).ln())
50     }
51
52     /// Sigma_k = A Sigma_{k-1} A^T + R with A = Ad(exp(tau)-1): the Kalman prediction
53     /// equation, with the adjoint standing in for the motion Jacobian.
54     pub fn compound(&self, sigma: &Matrix3<f64>, tau: &Vector3<f64>) -> Matrix3<f64> {
55         let a = SE2::exp(tau).inverse().adjoint();
56         a * sigma * a.transpose() + self.r
57     }
58 }

```

9.4.1 A worked example you can check by hand

Start at the origin, $x_0 = (0, 0, 0)^T$. Command $v = 1$ m/s, $\omega = 0.5$ rad/s, for $\Delta t = 1$ s, with $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = 0.02$ and $\alpha_5 = \alpha_6 = 0$.

The nominal endpoint. The radius is $r = v/\omega = 2$ m. With $\theta = 0$ the arc update gives

$$x' = 2 \sin(0.5) = 0.958851, \quad y' = 2(1 - \cos(0.5)) = 0.244835, \quad \theta' = 0.5$$

The same number, through exp. The commanded twist is $\tau = (1, 0, 0.5)^\top$. Since $\sin(0.5)/0.5 = 0.958851$ and $(1 - \cos 0.5)/0.5 = 0.244835$, $\exp(\tau)$ is that triple exactly. The arc formula and the exponential map are the same function.

The tangent-space noise. Both velocity variances are $\alpha_1 v^2 + \alpha_2 \omega^2 = 0.02 + 0.02(0.25) = 0.025$, so with $\Delta t = 1$,

$$R_u = \text{diag}(0.025, 0, 0.025), \quad \sigma = 0.158114$$

The Cartesian caricature. Linearizing the arc update in (v, ω) at the nominal command gives the Jacobian columns

$$\frac{\partial(x', y', \theta')}{\partial v} = (0.958851, 0.244835, 0)^\top, \quad \frac{\partial(x', y', \theta')}{\partial \omega} = (-0.162537, 0.469181, 1)^\top$$

and hence the covariance an EKF would carry:

$$\Sigma_{xy\theta} = 0.025 (J_v J_v^\top + J_\omega J_\omega^\top)$$

which has entries

$$\Sigma_{xx} = 0.023645, \quad \Sigma_{yy} = 0.007002, \quad \Sigma_{\theta\theta} = 0.025000$$

$$\Sigma_{xy} = 0.003963, \quad \Sigma_{x\theta} = -0.004063, \quad \Sigma_{y\theta} = 0.011730$$

Note $\rho_{y\theta} = 0.011730/\sqrt{0.007002 \times 0.025} = 0.887$. Cross-track position error and heading error are almost the same number, which is the algebraic signature of the bend.

What 200 000 samples say. Drawing from `sample_motion_model_velocity` with seed 7:

quantity	model	measured
Σ_{xx}	0.023645 (linearized)	0.02355
Σ_{yy}	0.007002 (linearized)	0.00710
$\rho_{y\theta}$	0.887 (linearized)	0.878
$\mathbb{E}[\xi]$	$(1, 0, 0.5)$ (<i>exact</i>)	$(1.0005, 0.0000, 0.4994)$
$\text{Cov}[\xi]$	$\text{diag}(0.025, 0, 0.025)$ (<i>exact</i>)	$\text{diag}(0.02508, 0.00000, 0.02505)$
cross-track skew	0 for any Gaussian in (x, y)	-0.301

The linearized Cartesian covariance is a decent fit to the *second* moments — that is why EKFs work at all. The third moment is where it falls apart, and the last two rows are the chapter’s whole claim: in exponential coordinates the model is not approximately Gaussian, it is Gaussian, and the measured mean and covariance are τ and R_u to Monte Carlo error.

RUST crates/motion/tests/worked_example.rs

```

1 use approx::assert_relative_eq;
2 use motion::{MotionModel, NoiseKind, VelocityAlphas, VelocityCmd, VelocityModel};
3 use nalgebra::Vector3;
4 use pr_core::geom::se2::SE2;
5 use rand::{rngs::SmallRng, SeedableRng};
6
7 const U: VelocityCmd = VelocityCmd { v: 1.0, omega: 0.5, dt: 1.0 };
8 const ALPHAS: VelocityAlphas = VelocityAlphas([0.02, 0.02, 0.02, 0.02, 0.0, 0.0]);
9
10 #[test]
11 fn arc_update_is_the_exponential_map() {
12     let model = VelocityModel { alphas: VelocityAlphas([0.0; 6]), noise: NoiseKind::
Normal };
13     let mut rng = SmallRng::seed_from_u64(0); // noiseless: the draws are unused
14     let by_arc = model.sample(&U, &SE2::identity(), &mut rng);
15     let by_exp = SE2::exp(&Vector3::new(U.v * U.dt, 0.0, U.omega * U.dt));
16
17     assert_relative_eq!(by_arc.x(), 0.958851_077_208, epsilon = 1e-11);
18     assert_relative_eq!(by_arc.y(), 0.244834_876_219, epsilon = 1e-11);
19     assert_relative_eq!(by_arc.theta(), 0.5, epsilon = 1e-12);
20     assert_relative_eq!(by_arc.x(), by_exp.x(), epsilon = 1e-12);
21     assert_relative_eq!(by_arc.y(), by_exp.y(), epsilon = 1e-12);
22 }
23
24 /// The chapter's headline claim, as an assertion: the sample cloud's
25 /// exponential coordinates have mean tau and covariance R_u.
26 #[test]
27 fn the_banana_is_gaussian_in_exponential_coordinates() {
28     let model = VelocityModel { alphas: ALPHAS, noise: NoiseKind::Normal };
29     let x0 = SE2::identity();
30     let mut rng = SmallRng::seed_from_u64(7);
31
32     let xis: Vec<Vector3<f64>> = (0..200_000)
33         .map(|_| model.sample(&U, &x0, &mut rng).boxminus(&x0))
34         .collect();
35
36     let n = xis.len() as f64;
37     let mean = xis.iter().sum:::<Vector3<f64>>() / n;
38     let cov = xis
39         .iter()
40         .map(|xi| (xi - mean) * (xi - mean).transpose())
41         .sum:::<nalgebra::Matrix3<f64>>()
42         / (n - 1.0);
43
44     let r_u = model.tangent_cov(&U).unwrap();
45     assert_relative_eq!(mean, Vector3::new(1.0, 0.0, 0.5), epsilon = 3e-3);
46     assert_relative_eq!(cov[(0, 0)], r_u[(0, 0)], max_relative = 0.02);
47     assert_relative_eq!(cov[(2, 2)], r_u[(2, 2)], max_relative = 0.02);
48     assert!(cov[(1, 1)] < 1e-12, "a differential drive has no lateral noise");
49 }
50

```

```

51 // And the counterpart: in (x, y) the same cloud is visibly non-Gaussian.
52 #[test]
53 fn the_same_cloud_is_skewed_in_cartesian_coordinates() {
54     let model = VelocityModel { alphas: ALPHAS, noise: NoiseKind::Normal };
55     let mut rng = SmallRng::seed_from_u64(7);
56     let poses: Vec<SE2> = (0..200_000)
57         .map(|_| model.sample(&U, &SE2::identity(), &mut rng))
58         .collect();
59
60     // Skewness of the residual perpendicular to the mean heading: identically
61     // zero for any Gaussian, and decidedly not zero here.
62     let skew = cross_track_skew(&poses);
63     assert!(skew < -0.25, "expected a pronounced left-bending banana, got {skew}");
64 }
65
66 // Third standardized moment of the residual perpendicular to the mean heading.
67 fn cross_track_skew(poses: &[SE2]) -> f64 {
68     let n = poses.len() as f64;
69     let (mx, my) = poses.iter().fold((0.0, 0.0), |(a, b), p| (a + p.x() / n, b + p.y() /
70 n));
71     // Circular mean of the headings -- never average theta arithmetically.
72     let (sc, ss) = poses
73         .iter()
74         .fold((0.0, 0.0), |(c, s), p| (c + p.theta().cos(), s + p.theta().sin()));
75     let (c, s) = {
76         let h = ss.atan2(sc);
77         (h.cos(), h.sin())
78     };
79     let d: Vec<f64> = poses.iter().map(|p| -(p.x() - mx) * s + (p.y() - my) * c).collect
80 ();
81     let m2 = d.iter().map(|x| x * x).sum::<f64>() / n;
82     let m3 = d.iter().map(|x| x * x * x).sum::<f64>() / n;
83     m3 / m2.powf(1.5)
84 }

```

NOTE

The widgets in this chapter run the same algorithms: `lib/models/motion.ts` is a line-for-line port of `crates/motion`, and `lib/models/motion-se2.ts` of `se2_noise.rs`. The cross-track skew printed under the Banana Machine is the statistic in the last row of that table, computed live — set the sliders to the canonical α 's above and it settles at about -0.30 , while the tile beside it shows what a Gaussian *fitted in exponential coordinates* predicts for the same statistic.

9.5 Putting it together

9.5.1 Which model, and when

Use the velocity model when the motion has not happened yet: planning, control, MPPI rollouts, anything that asks *what if*. Use the odometry model when it has: localization, SLAM, anything that asks *where am I*. Nav2's AMCL exposes exactly the odometry model's four α 's under the names `alpha1`–`alpha4`, and tuning them remains, in 2026, one of the two or three things that decide whether a robot localizes reliably in a warehouse.

Use the on-manifold form when the uncertainty is large enough that the shape matters — long open-loop stretches, loop closures with a lot of accumulated drift, any covariance you intend to hand to a factor graph (Chapter 15) as an information matrix. It costs nothing extra: R_u is built from the same α 's.

9.5.2 Tuning the α 's without lying to yourself

The honest procedure is boring and it works. Drive a known trajectory with ground truth — a motion capture rig, a total station, or a scan-matched reference from Chapter 16. Compute $\xi_k = x_k \ominus x_{k-1}$ for each interval and the twist τ_k each command asked for. Regress the empirical variance of $\xi_k - \tau_k$ against v^2 and ω^2 ; the regression coefficients *are* the α 's, times Δt^2 .

Then inflate them. Every deployed system runs with α 's larger than the fit, and the epigraph to this chapter says why: the exact shape of the noise matters far less than having enough of it to absorb the model errors nobody wrote down. An overly wide motion model costs particles and convergence speed. An overly narrow one costs the robot.

The failure signature is worth memorizing. Too small, and the filter is overconfident: the particle cloud collapses, the true pose walks out of it, and no measurement can pull it back because there are no particles there to reweight. Too large, and the filter is lazy: the cloud never tightens, every measurement is inside the gate, and the robot's estimate is technically correct and practically useless. Chapter 12 turns this into a diagnostic you can run online.

9.5.3 What the next chapter needs

The motion model is half the generative story. It says where the robot might be. It says nothing about what the robot would *see* from there, and without that, the prediction step spreads the belief forever and the filter never recovers information. Chapter 10 supplies the other half — $p(z_t \mid x_t)$ — and with both in hand, every algorithm in Parts IV and V becomes an engineering exercise.

9.6 Exercises

1 FOUNDATION •• The straight-line branch, and where to put it

Derive the $\hat{\omega} \rightarrow 0$ limit of the exact arc update by Taylor-expanding $\sin(\theta + \hat{\omega}\Delta t)$ and $\cos(\theta + \hat{\omega}\Delta t)$ to second order, and show that the closed-form density remains well defined there. Then do the numerics: for $\hat{v} = 1$ m/s and $\Delta t = 0.1$ s, find the value of $|\hat{\omega}|$ below which the *straight-line* formula is more accurate than the arc formula in IEEE double precision, and check it against the estimate in Step 5 of that derivation. Then answer the engineering question: given that the book's code switches two decades earlier than the crossover, what does that cost, and what does it buy?

2 FOUNDATION •• Why bananas bend

For a nominally straight command ($\omega = 0, v > 0$), show from the arc update that the along-track position variance grows like Δt^2 in the translational noise, while the cross-track variance grows like Δt^4 in the rotational noise. Conclude that there is a duration beyond which heading noise dominates position noise no matter how good the wheel encoders are, and compute it in terms of α_1 and α_3 .

3 FOUNDATION ••• The unpaid Jacobian

Step 6 of the inversion derivation claims that `motion_model_velocity` is missing the Jacobian determinant of the change of variables $(\hat{v}, \hat{\omega}, \hat{\gamma}) \mapsto x_t$. Compute that determinant for the arc update. Show that for the odometry model the analogous factor is exactly $1/\hat{\delta}_{trans}$. Then explain why leaving it out is harmless for particle weighting but *not* harmless if you use the density as a factor in a graph optimization (Chapter 15).

4 CONCEPTUAL • Predict, then check: what each α does

In the Banana Machine, select the **Straight** preset and switch on **Isolate**, so every α except the selected one is zero. Before touching anything else: sketch the cloud you expect with only α_4 non-zero, and then the cloud you expect with only α_6 . Now check. Explain each in one sentence, and say which of the two an EKF's ellipse represents faithfully.

5 CONCEPTUAL •• Find the pose the model cannot see through

In Map Squeeze, start from the **Facing a doorway** preset with conditioning on, and find settings of the noise and ω that push the accepted-through-a-wall figure above 40%. Which of the two controls moves it further, and why does *curvature* in particular manufacture these samples? Then propose two fixes — testing the geodesic against the map, and shrinking Δt so the model is called more often — estimate the cost of

each per sample, and say which you would deploy on a robot running 2000 particles at 20 Hz.

6 PRACTICAL •• Swap in the triangular sampler

Implement `sample_triangular_distribution(b2)` as the scaled sum of two independent uniforms, verify that its variance really is b^2 , and swap it into `VelocityModel` behind the `NoiseKind` enum. Re-run the worked-example test. Which assertions still pass, and which need a different tolerance? Where does the triangular banana visibly differ from the normal one, and why is bounded support sometimes worth the loss of exactness?

7 PRACTICAL ••• Compounding, and the point where first order fails

Implement `TangentNoiseModel::compound` and propagate Σ over a 20-step trajectory of the canonical command at $\Delta t = 0.2$ s. Compare against 50 000 Monte Carlo rollouts by computing the sample covariance of $x_k \boxminus \bar{x}_k$ at each step. Plot the ratio of predicted to measured variance per component, and find the step at which the along-track component crosses 5% error. Then re-run with the heading noise doubled: does the crossing move earlier in proportion to the heading spread, or faster?

9.7 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics MIT Press*.

Chapter 5 is the source of both models and of algorithm Tables 5.1–5.7, which this chapter restates and implements verbatim. The α -parameter bookkeeping and the map-conditioned model are taken from it unchanged. [link](#)

Long, A. W., Wolfe, K. C., Mashner, M. J., and Chirikjian, G. S. (2012). The Banana Distribution is Gaussian: A Localization Study with Exponential Coordinates *Proceedings of Robotics: Science and Systems VIII*.

The paper this chapter's last section is built on. It shows the banana is Gaussian in exponential coordinates and derives closed-form propagation and fusion for a differential drive on arcs of constant curvature. doi:10.15607/RSS.2012.VIII.034

Barfoot, T. D. and Furgale, P. T. (2014). Associating Uncertainty With Three-Dimensional Poses for Use in Estimation Problems *IEEE Transactions on Robotics* 30(3), 679–693.

Where the adjoint compounding recursion comes from, together with the fourth-order correction that closes the 6% gap measured in this chapter's compounding derivation. doi:10.1109/TR0.2014.2298059

Solà, J., Deray, J., and Atchuthan, D. (2021). A micro Lie theory for state estimation in robotics *arXiv:1812.01537 (v9)*.

The reference for the $\boxplus/\boxminus$ conventions, the adjoint, and the left-versus-right perturbation distinction that Step 4 of the banana derivation warns about. [Link](#)

Mangelson, J. G., Ghaffari, M., Vasudevan, R., and Eustice, R. M. (2020). Characterizing the Uncertainty of Jointly Distributed Poses in the Lie Algebra *IEEE Transactions on Robotics* 36(5), 1371–1388.

Extends Lie-algebra uncertainty to poses that are correlated rather than independent — the case that actually arises once a pose graph has been optimized, and the reason Chapter 15 cannot simply reuse this chapter’s per-step R_u . doi:10.1109/TR0.2020.2994457

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., and Ferguson, M. (2023). From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the Robot Operating System 2 *Robotics and Autonomous Systems* 168, 104493.

Evidence that this chapter is current practice and not history: Nav2’s AMCL still runs `sample_motion_model_odometry`, and its `alpha1`–`alpha4` are the α ’s derived here. doi:10.1016/j.robot.2023.104493

Okawara, T., Koide, K., Oishi, S., Yokozuka, M., Banno, A., Uno, K., and Yoshida, K. (2025). Tightly-coupled LiDAR-IMU-wheel odometry with an online neural kinematic model learning via factor graph optimization *Robotics and Autonomous Systems* 187, 104929.

The modern alternative to hand-tuned α ’s: learn the kinematic model online inside the estimator. Worth reading as the answer to ‘why not just fit the motion model from data?’, and as a preview of Chapter 25. doi:10.1016/j.robot.2025.104929

Park, C. and Moon, J. (2026). Dynamic Noise Adaptation in the Motion Model of Monte Carlo Localization for Consistent Localization *Sensors* 26(5), 1415.

A recent take on the tuning problem from the last section: scale the motion-noise parameters online rather than fixing them, so the filter widens before it diverges instead of after. doi:10.3390/s26051415

Probabilistic Sensor Models

PROBABILISTIC MODELS · Intermediate · 60 min

“

Herein lies a key advantage of probabilistic techniques over classical robotics: in practice, we can get away with extremely crude models.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 6

Chapter 9 built the half of the generative story that makes the robot less certain. This chapter builds the half that makes it more certain: the measurement model $p(z_t \mid x_t, m)$, the function that turns a laser scan into an opinion about where the robot is.

Sensing fails differently from moving. Wheel slip is a small perturbation of a mostly-correct answer, which is why one Gaussian per motion leg is a defensible model. A LiDAR beam does not perturb — it *switches*. It either measures the wall the map predicts, or it measures the person standing in front of the wall, or it measures nothing at all because the wall was black felt, or it returns a number that came from a neighbouring transducer's echo. Four categorically different things happen, and a model that averages them is a model of nothing.

So the chapter has two aha moments and they pull in opposite directions. The first is that a mixture of four dead-simple densities reproduces the messy histogram a real sensor produces, and that its parameters can be *learned from data* instead of guessed. The second is that the likelihood you then compute is almost always far too confident, because multiplying hundreds of "independent" beams counts the same evidence over and over. Managing that overconfidence turns out to matter more than the fidelity of the model that caused it.

🕒 AFTER THIS CHAPTER YOU CAN

- Write $p(z \mid x, m)$ for a range finder as a four-way mixture, and name the physical mechanism behind each component

- Derive the EM updates that learn the mixture parameters from logged data, and find the bias hiding in one of them
- Build a likelihood field with an exact $O(N)$ distance transform, and say honestly what it is not
- Evaluate the landmark model with known correspondence, and invert it into a sampler over poses
- Measure the overconfidence that beam independence buys you, and repair it by tempering
- Choose between the beam model, the likelihood field, correlation, and features — on evidence, not taste

ASSUMED

- Densities, mixtures, and maximum likelihood (Chapter 2)
- Ray casting and LiDAR phenomenology in the Apartment (Chapter 4)
- Where $p(z \mid x)$ sits in the Bayes filter recursion (Chapter 5)
- Importance weights — these likelihoods become particle weights (Chapter 8)
- Its twin chapter on motion (Chapter 9)

10.1 The problem: one pose, ten thousand beams

Park Rusty in the corridor outside room A and aim a single beam south, through the doorway, at the far wall of the apartment. The map says the range is 4.4 m. Fire the beam ten thousand times and histogram what comes back.

INTERACTIVE · w10.1

The Beam Mixture Mixer

Sensor noise is not a Gaussian around the true range. It is four causes stacked, and their weights can be learned from data.

Run this simulation in the web edition: `/chapters/ch10-sensor-models #w10.1`

Nothing about that shape is a bell curve, and the deviations are not small.

There is a hump where the map says there should be one, and it is roughly Gaussian, roughly 11 cm wide. That is the part of the sensor everyone models: timing resolution, surface roughness, beam divergence. It is also, in an empty corridor, only about 86% of the mass — and with people walking through, 70%.

There is a shoulder in front of the wall. Every reading in it is *shorter* than the map predicts, never longer, because the only thing that can produce it is an

object the map does not contain — and objects the map does not contain occlude, they do not become transparent. The asymmetry is physics, not convention.

There is a spike piled up at z_{max} and a thin floor everywhere else. The spike is beams that hit nothing detectable: black upholstery, glass, a surface angled so the light left and never came back. The floor is everything else — crosstalk, multipath, a stray reflection off a passing elbow — and it is uniform because we have nothing better to say about it.

💡 THE ONE SENTENCE VERSION

A range reading is produced by one of four causes, and which one is unobserved. The measurement model is therefore a mixture over causes; fitting it is a latent-variable problem; and the mixture weights are as much a property of the *environment* as of the sensor.

10.2 Building intuition

10.2.1 Four causes, and the hardware that makes them

The four components are not a curve-fitting convenience. Each is a distinct failure mode of a real device, and knowing which is which is what lets you predict how the weights will move when you change rooms or change sensors.

Component	What it is	Ultrasonic ranger	2-D LiDAR
p_{hit}	Correct return, corrupted	Cone width, speed-of-sound drift with temperature	Time-of-flight quantization, surface roughness, beam divergence
p_{short}	Something closer than the map	People, chair legs, <i>and</i> specular pre-returns off an angled surface	People, chair legs, an open door the map has closed
p_{max}	No return at all	Specular reflection away from the transducer; absorptive foam	Black or retro-dark surfaces, glass, grazing incidence, beyond range
p_{rand}	A number with no cause	Crosstalk between transducers in a ring — sound is slow, echoes linger	Multipath, sunlight, interference from another robot's LiDAR

Two entries deserve emphasis because they are the ones people get backwards. Ultrasonic sensors produce p_{short} and p_{max} for the same reason — a smooth wall at an angle acts as a mirror — and whether you get a short reading or no reading depends on what the redirected pulse eventually hits. And p_{rand} in a sonar ring is dominated by crosstalk, which is why the component's weight depends on how many transducers you fire at once, a fact no amount of staring at one beam will reveal.

10.2.2 Where the exponential comes from

The shoulder's exponential shape is usually presented as an assumption. It is not: it is a consequence, and the derivation takes two lines.

Model unmodelled objects as a homogeneous Poisson process of rate λ_{short} along the ray — "clutter is scattered along the beam at some objects-per-metre, independently of where the wall is." The reading is short exactly when the first clutter object lies before the wall, and the distance to the first point of a Poisson process satisfies

$$\Pr(\text{no clutter within } d) = e^{-\lambda_{short}d} \implies p(d) = \lambda_{short} e^{-\lambda_{short}d}$$

Condition on the reading actually being short — that is, on the first clutter point lying before the wall, an event of probability $1 - e^{-\lambda_{short}z^*}$ — and you get p_{short} with its normalizer η_s already attached. The widget above generates its shoulder by literally sampling that Poisson field, so the weight z_{short} it recovers is the true probability of clutter, $1 - e^{-\lambda_{short}z^*}$, to within a percent. Its estimate of λ_{short} itself is a different story, and Step 6 of the EM derivation below explains exactly why.

10.3 The mathematics

10.3.1 The scan, and the assumption that makes it tractable

$z_t = \{z_t^1, \dots, z_t^K\}$	A scan: K individual range readings taken at one time step.
z_t^{k*}	The true range of beam k — ray cast into the map m from the hypothesised pose.
z_{max}	The sensor's maximum range. A reading equal to it is a <i>failure</i> , not a measurement.
Θ	= The intrinsic parameters: four mixture weights on the simplex, plus two shape parameters.
$(z_{hit}, z_{short}, z_{max}, z_{rand}, \sigma_{hit}, \sigma_{short})$	
$d(x, y)$	Distance from a point to the nearest occupied cell of m — the likelihood field.
$f_t^i = (r_t^i \ \phi_t^i \ s_t^i)^\top$	A feature: range, bearing, and signature of one detected landmark.
$c_t^i \in \{1..N, \emptyset\}$	The correspondence variable: which map landmark feature i came from, or none.
$Q_t = \text{diag}(\sigma_r^2, \sigma_\phi^2, \sigma_s^2)$	Feature measurement noise. A Q, not an R — measurement noise, per Chapter 6.
κ	Tempering exponent: the model is used as $p^{1/\kappa}$. $\kappa = 1$ is the raw product.

A scan is a set of readings, and the model of the whole scan is the product of the models of its parts:

$$p(z_t \mid x_t, m) = \prod_{k=1}^K p(z_t^k \mid x_t, m)$$

This is a conditional independence assumption — that given the pose and the map, the beams do not share anything. It is false. Thrun says so on the page where he introduces it, and then defers the consequences; we will not defer them, because they are the difference between a filter that works and one that is confidently lost. The last section of this chapter measures the damage.

10.3.2 The beam model

Per beam, four causes, four densities, four weights that sum to one:

$$p(z_t^k \mid x_t, m) = z_{hit} p_{hit}(z_t^k) + z_{short} p_{short}(z_t^k) + z_{max} p_{max}(z_t^k) + z_{rand} p_{rand}(z_t^k)$$

$$z_{hit} + z_{short} + z_{max} + z_{rand} = 1$$

with, writing z^{k*} for the ray-cast range and $\mathbb{I}[\cdot]$ for the indicator of a condition,

$$p_{hit}(z) = \eta \mathcal{N}(z; z^{k*}, \sigma_{hit}^2) \mathbb{I}[0 \leq z \leq z_{max}], \quad \eta = \left[\int_0^{z_{max}} \mathcal{N}(z; z^{k*}, \sigma_{hit}^2) dz \right]^{-1}$$

$$p_{short}(z) = \eta_s \lambda_{short} e^{-\lambda_{short} z} \mathbb{I}[0 \leq z \leq z^{k*}], \quad \eta_s = \frac{1}{1 - e^{-\lambda_{short} z^{k*}}}$$

$$p_{max}(z) = \mathbb{I}[z = z_{max}] \quad p_{rand}(z) = \frac{1}{z_{max}} \mathbb{I}[0 \leq z < z_{max}]$$

Three details that a first implementation usually gets wrong, and that the book's unit tests pin:

1. **Both truncations carry a normalizer.** p_{hit} lives on $[0, z_{max}]$ and p_{short} on $[0, z^{k*}]$. Drop η_s and the model quietly prefers hypotheses with short true ranges, because a shorter interval concentrates the same unit of mass. Exercise 1 makes you find what breaks.
2. **p_{max} is a point mass, not a density.** It integrates to 1 by itself. Mixing a point mass and three densities in one equation is a mild abuse that everybody commits; in code it means the max-range case is a branch, not a formula.
3. **z^{k*} depends on the hypothesis.** Every candidate pose needs its own ray cast, which is what makes this model expensive and what the likelihood field will eliminate.

```

ALGORITHM beam_range_finder_model( $z_t, x_t, m$ )  COST  $O(K \cdot C_{\text{ray}})$ ,  $C_{\text{ray}}$  the
cost of one ray cast -- the dominant cost in the whole filter
in a scan of  $K$  ranges, a hypothesised pose, the map
out  $q = p(z_t \mid x_t, m)$ 

```

```

1:  $q = 1$ 
2: for  $k = 1$  to  $K$  do
3:   compute  $z_t^{k*}$  for the measurement  $z_t^k$  using ray casting into  $m$  from  $x_t$ 
4:    $p = z_{\text{hit}} \cdot p_{\text{hit}}(z_t^k \mid x_t, m) + z_{\text{short}} \cdot p_{\text{short}}(z_t^k \mid x_t, m) + z_{\text{max}} \cdot p_{\text{max}}(z_t^k \mid$ 
 $x_t, m) + z_{\text{rand}} \cdot p_{\text{rand}}(z_t^k \mid x_t, m)$ 
5:    $q = q \cdot p$ 
6: endfor
7: return  $q$ 

```

In practice line 5 is a sum of logs. Per-beam likelihoods of order 0.3 underflow f64 at around 600 beams, and a hypothesis that is merely *wrong* — per-beam likelihoods of 10^{-3} — underflows after 100. More importantly, the log form is the one tempering acts on linearly, which the last section needs.

10.3.3 Learning Θ instead of guessing it

The six intrinsic parameters are usually hand-tuned, and hand-tuning is defensible — the epigraph to this chapter is the reason. But there is no need. Given a log of range readings Z taken from *known* poses X in a known map, the parameters that maximize $\prod_i p(z_i \mid x_i, m, \Theta)$ can be computed, because the only thing standing in the way is the unobserved cause of each beam, and that is exactly what EM handles.

DERIVATION EM for the beam model's intrinsic parameters

$$z_c = |Z|^{-1} \sum_i e_{\{i,c\}}, \sigma_{\text{hit}}^2 = \sum_i e_{\{i,\text{hit}\}} (z_i - z_i^*)^2 / \sum_i e_{\{i,\text{hit}\}}, \lambda_{\text{short}} = \sum_i e_{\{i,\text{short}\}} / \sum_i e_{\{i,\text{short}\}} z_i$$

Step 1 — name the missing data. Attach to each reading z_i a latent label $c_i \in \{\text{hit}, \text{short}, \text{max}, \text{rand}\}$ saying which cause produced it. With the labels known, the log-likelihood separates completely:

$$\log p(Z \mid X, m, \Theta) = \sum_i \sum_c \mathbb{1}[c_i = c] (\log z_c + \log p_c(z_i \mid x_i, m))$$

Each component's parameters appear in exactly one term, so with labels this would be four independent one-line maximum-likelihood problems. That is the entire reason EM is worth doing here.

Step 2 — E-step: replace the labels by their posteriors. The labels are unknown, so take the expectation of Step 1 over c_i given the current Θ . The required posterior is Bayes rule on a four-way discrete variable:

$$e_{i,c} = p(c_i = c \mid z_i, x_i, m, \Theta) = \frac{z_c p_c(z_i \mid x_i, m)}{\sum_{c'} z_{c'} p_{c'}(z_i \mid x_i, m)}$$

The denominator is the mixture likelihood of the reading — the same number `beam_range_finder_model` computes per beam. So the E-step costs one extra pass over data you were evaluating anyway.

Step 3 — M-step for the weights. Maximizing $\sum_i \sum_c e_{i,c} \log z_c$ subject to $\sum_c z_c = 1$ with a Lagrange multiplier μ gives $\partial/\partial z_c : \sum_i e_{i,c}/z_c = \mu$, so $z_c \propto \sum_i e_{i,c}$, and the constraint fixes the constant because $\sum_c \sum_i e_{i,c} = |Z|$:

$$z_c = \frac{1}{|Z|} \sum_i e_{i,c}$$

The weights are just the average responsibility. Nothing about this step knows what the components *are*.

Step 4 — M-step for σ_{hit} . Only the *hit* term contains σ_{hit} . Dropping the truncation normalizer for the moment, the objective is the usual weighted Gaussian log-likelihood in the residuals $z_i - z_i^*$, whose maximizer is the responsibility-weighted second moment:

$$\sigma_{hit}^2 = \frac{\sum_i e_{i,hit} (z_i - z_i^*)^2}{\sum_i e_{i,hit}}$$

Step 5 — M-step for λ_{short} . Likewise, $\sum_i e_{i,short} (\log \lambda - \lambda z_i)$ is maximized at the reciprocal of the responsibility-weighted mean:

$$\lambda_{short} = \frac{\sum_i e_{i,short}}{\sum_i e_{i,short} z_i}$$

Iterate Steps 2–5. Each iteration is $O(|Z|)$ once the z_i^* are cached, and the data log-likelihood never decreases — which makes a dip in the trace a bug report, not a result.

Step 6 — the bias nobody mentions. Step 5 is the maximum-likelihood estimator for an *untruncated* exponential, but p_{short} is truncated at z^* . For truncated data the sample mean is smaller than $1/\lambda$,

$$\mathbb{E}[z \mid z \leq z^*] = \frac{1}{\lambda} - \frac{z^*}{e^{\lambda z^*} - 1}$$

so the estimator converges to something larger than the truth, and the error blows up when λz^* is small. Measured on 40 000 synthetic beams generated with $\lambda = 1.0 \text{ m}^{-1}$ and no random component in the data: at $z^* = 2 \text{ m}$ the formula above predicts a limit of 1.46 and EM returns 1.53; at $z^* = 8 \text{ m}$ it predicts 1.003 and EM returns 1.009. The fix is one Newton step on the truncated likelihood, and the reason we implement Table 6.2 unmodified anyway is that λ_{short} is the parameter the posterior is least sensitive to — but you should know that the number EM hands you is not the clutter density. ■

ALGORITHM `learn_intrinsic_parameters(Z, X, m)` COST $O(|Z|)$ per iteration after caching z^* ; monotone in the data log-likelihood

in ranges Z with the poses X they were taken from, and the map

out $\Theta = (z_{hit}, z_{short}, z_{max}, z_{rand}, \sigma_{hit}, \lambda_{short})$

```

1: repeat
2:   for all  $z_i \in Z$  do
3:      $\eta = [p_{hit}(z_i) + p_{short}(z_i) + p_{max}(z_i) + p_{rand}(z_i)]^{-1}$ 
4:      $e_{i,hit} = \eta z_{hit} p_{hit}(z_i)$ , and likewise  $e_{i,short}, e_{i,max}, e_{i,rand}$ 
5:   endfor
6:    $z_{hit} = |Z|^{-1} \sum_i e_{i,hit}$ , and likewise  $z_{short}, z_{max}, z_{rand}$ 
7:    $\sigma_{hit} = \sqrt{(\sum_i e_{i,hit})^{-1} \sum_i e_{i,hit} (z_i - z_i^*)^2}$ 
8:    $\lambda_{short} = \sum_i e_{i,short} / \sum_i e_{i,short} z_i$ 
9: until convergence
10: return  $\Theta$ 
```

i NOTE

The widget at the top of this chapter runs exactly this, on data generated by geometry rather than by the model being fitted. With the corridor quiet, EM recovers mixture weights of (0.875, 0.027, 0.048, 0.050) against a generator whose true weights are (0.865, 0.047, 0.050, 0.038) and $\sigma_{hit} = 0.110 \text{ m}$ against a true 0.110 m. Turn on "people in the corridor" and it returns (0.709, 0.178, 0.050, 0.063) against a truth of (0.700, 0.212, 0.050, 0.038) — z_{short} triples, z_{max} does not move, and a little short mass leaks into z_{rand} because a clutter return far from the wall looks exactly like a random one.

Twenty-four iterations over 4 500 beams take about 50 ms.

10.3.4 Likelihood fields

The beam model has a physical story and two practical problems, and Thrun states both plainly: it is *unsmooth* in x_t , and it is *expensive*. Both come from the same place — the ray cast on line 3.

Unsmoothness is the serious one. Move a hypothesis 2 cm sideways and a beam that was grazing a door frame now slips past it; z^{k*} jumps from 0.8 m to 6 m; that beam's likelihood collapses from "excellent" to z_{rand}/z_{max} . The pose-likelihood surface is therefore full of cliffs, and everything that has to search it — a particle filter with a finite number of particles, a hill climber, an optimizer — suffers.

INTERACTIVE · w10.2

The Likelihood Field Explorer

The physically faithful model is not the one you want inside a filter. Smoothness beats fidelity when something has to search.

Run this simulation in the web edition: </chapters/ch10-sensor-models> #w10.2

The likelihood field is the fix, and it is worth being honest about what kind of fix it is. It is not a better model of the sensor. It is not a generative model of z at all — it does not normalize over z , and it cannot tell you what reading to expect. It is a *scoring function* that happens to be smooth, cheap, and empirically excellent.

The construction is: project each reading to the point in the world it claims to have hit,

$$x_{z^k} = x + (x_{k,sens} \cos \theta - y_{k,sens} \sin \theta) + z_t^k \cos(\theta + \theta_{k,sens})$$

$$y_{z^k} = y + (x_{k,sens} \sin \theta + y_{k,sens} \cos \theta) + z_t^k \sin(\theta + \theta_{k,sens})$$

The middle bracket is the sensor's mounting offset rotated into the world frame — $R(\theta)$ applied to $(x_{k,sens}, y_{k,sens})$ — and the last term walks the measured distance along the beam's own bearing. Then ask only how far that point is from the nearest obstacle — *any* obstacle, no correspondence, no ray:

$$q \propto \left(z_{hit} \mathcal{N}(d(x_{z^k}, y_{z^k}); 0, \sigma_{hit}^2) + \frac{z_{rand}}{z_{max}} \right)$$

Max-range readings are skipped outright. "I saw nothing" projects to a point

8 m away that means nothing at all, and scoring it would punish poses for a beam that carries no information.

ALGORITHM `likelihood_field_range_finder_model(z_t, x_t, m)` COST $O(K)$
 lookups, after an $O(N)$ precomputation over the N map cells
in a scan, a pose, the map (as a precomputed distance field)
out q , a smooth score — not a normalized density over z

```

1:  $q = 1$ 
2: for all  $k$  do
3:   if  $z_t^k \neq z_{max}$  then
4:      $x_{zk} = x + x_{k,sens} \cos \theta - y_{k,sens} \sin \theta + z_t^k \cos(\theta + \theta_{k,sens})$ 
5:      $y_{zk} = y + y_{k,sens} \cos \theta + x_{k,sens} \sin \theta + z_t^k \sin(\theta + \theta_{k,sens})$ 
6:      $dist = \min_{x',y'} \{ \sqrt{(x_{zk} - x')^2 + (y_{zk} - y')^2} \mid \langle x', y' \rangle \text{ occupied in } m \}$ 
7:      $q = q \cdot (z_{hit} \cdot \mathbf{prob}(dist, \sigma_{hit}^2) + z_{rand}/z_{max})$ 
8:   endif
9: endfor
10: return  $q$ 
```

Line 6 is a nearest-neighbour search over every occupied cell, which sounds catastrophic and is free, because it does not depend on the scan. Compute it once for every cell of the map and the per-beam cost becomes an array lookup. That precomputation is the distance transform, and the modern algorithm for it is exact and linear.

DERIVATION The exact Euclidean distance transform in $O(N)$

$$D(x, y) = \min_{x'} [(x - x')^2 + \min_{y'} ((y - y')^2 + f(x', y'))]$$

Step 1 — write it as a minimization over a sampled function. Let $f(p) = 0$ at occupied cells and $+\infty$ elsewhere. Then the squared distance transform is

$$D(q) = \min_p (\|q - p\|^2 + f(p))$$

which is a *min-plus convolution* of f with a parabola. Nothing about it requires f to be binary, which is why the same routine also serves as the max-product message pass in belief propagation and the inner loop of a chamfer matcher.

Step 2 — separate the axes. Squared Euclidean distance is a sum over

coordinates, so the minimization factors:

$$D(x, y) = \min_{x'} \left[(x - x')^2 + \underbrace{\min_{y'} ((y - y')^2 + f(x', y'))}_{\text{1-D transform down a column}} \right]$$

A 2-D transform is a 1-D transform down every column followed by a 1-D transform along every row. Separability is exactly what a chamfer sweep gives up: it propagates costs along an 8-neighbour mask, so its error accumulates with distance from the nearest obstacle, while the separable form is exact at every cell no matter how far away the nearest obstacle is.

Step 3 — the 1-D transform is a lower envelope of parabolas. Each sample p contributes the parabola $z \mapsto (z - p)^2 + f(p)$. All of them have the same curvature, so any two intersect exactly once, and the lower envelope of n of them has at most n pieces in left-to-right order. Sweep the samples left to right maintaining a stack of the parabolas currently on the envelope and the breakpoints between them: a new parabola either takes over the rightmost region — push it — or it dominates the parabola on top, in which case pop and retry. Each parabola is pushed once and popped at most once, so the sweep is $O(n)$, and a second pass reads the envelope off at each grid position. Two passes per axis, $O(N)$ overall, and the result is *exact* — Felzenszwalb and Huttenlocher's algorithm computes the true lattice minimum, not an approximation of it.

Step 4 — what remains inexact, measured. The transform is exact; the *map* is not. Rasterizing a wall onto a grid moves it by up to half a cell diagonal, and that error passes straight through: on the Apartment at a 5 cm cell, the exact transform of the rasterized map differs from the true continuous distance-to-wall by at most $\sqrt{2} c/2 = 3.5$ cm and on average 2.5 cm — and both numbers halve when the cell does (1.8 cm and 1.2 cm at 2.5 cm). Meanwhile the whole 240×180 field builds in about 1.5 ms, the same as the chamfer sweep it replaces. So the sensible engineering statement is: never pay for an approximate transform, and spend your accuracy budget on the grid resolution instead. ■

⚠ WHERE ROBOTS BREAK THIS

The likelihood field's blind spot is that it cannot tell "the beam went through a wall" from "the beam stopped at a wall". Only the endpoint is scored, so a pose that puts a 6 m reading's endpoint onto a wall in the *next room*

is rewarded exactly as much as the correct one. Thrun's fix — score only endpoints whose ray stays inside known free space — costs a ray cast and gives back the unsmoothness that the field existed to remove, so almost nobody does it; the practical mitigation is to keep max-range and implausibly long readings out of the field entirely.

10.3.5 Map correlation, in one paragraph

There is a third way to score a scan that neither ray-casts nor looks up distances: turn the scan into a little local map m_{loc} and correlate it with the global one. With $\bar{m}$ the mean cell value over both maps,

$$\rho_{m, m_{loc}, x_t} = \frac{\sum_i (m_i - \bar{m})(m_{loc,i} - \bar{m})}{\sqrt{\sum_i (m_i - \bar{m})^2 \sum_i (m_{loc,i} - \bar{m})^2}}, \quad p(m_{loc} \mid x_t, m) = \max\{\rho_{m, m_{loc}, x_t}, 0\}$$

It is the odd one out: clipping a correlation coefficient at zero and calling it a probability is not derivable from anything, and the score is invariant to a global brightness shift in a way a likelihood should not be. It earns its place because it degrades gracefully when the map is partly wrong, and because its descendant — correlative scan matching over a discretized pose grid — is the workhorse of Chapter 16.

10.3.6 Features, landmarks, and the variable that will become the villain

Sometimes the front end does not hand you 180 ranges but a handful of *detections*: "a corner at 3.2 m, bearing -0.4 rad, signature 7". Feature extraction itself — line fitting, corner detection, blob detection — is a perception topic and we let Chapter 18 have it. What matters here is the model of the detection, and the bookkeeping variable that comes with it.

That variable is the **correspondence** c_t^i : which landmark in the map produced feature i . Give it to the model and everything is easy. Withhold it and you have the data-association problem that Chapter 11 spends half its length on and that breaks more SLAM systems than any other single cause. Introducing c_t^i one chapter early, while it is still harmless, is deliberate.

With the correspondence known, $c_t^i = j$, the model is a noisy polar observation of a known point:

$$\hat{r} = \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2}, \quad \hat{\phi} = \text{atan2}(m_{j,y} - y, m_{j,x} - x) - \theta$$

$$p(f_t^i \mid c_t^i, x_t, m) = \mathcal{N}(r_t^i - \hat{r}; 0, \sigma_r^2) \mathcal{N}(\phi_t^i - \hat{\phi}; 0, \sigma_\phi^2) \mathcal{N}(s_t^i - m_{j,s}; 0, \sigma_s^2)$$

ALGORITHM `landmark_model_known_correspondence(f_t^i, c_t^i, x_t, m)`
 COST 0(1) per feature -- three ray casts' worth of work for a whole scan's worth of information
 in a feature (r, ϕ, s), its known identity, the pose, the map
 out $q = p(f_t^i \mid c_t^i, x_t, m)$

```

1:  $j = c_t^i$ 
2:  $\hat{r} = \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2}$ 
3:  $\hat{\phi} = \text{atan2}(m_{j,y} - y, m_{j,x} - x) - \theta$ 
4:  $q = \text{prob}(r_t^i - \hat{r}, \sigma_r^2) \cdot \text{prob}(\phi_t^i - \hat{\phi}, \sigma_\phi^2) \cdot \text{prob}(s_t^i - m_{j,s}, \sigma_s^2)$ 
5: return  $q$ 

```

The bearing residual must be wrapped into $(-\pi, \pi]$ before it is squared. A predicted 179° and an observed -179° are two degrees apart, and a model that thinks they are 358 degrees apart will reject the correct landmark and then confidently accept the wrong one — the failure Chapter 3 built the wrap-around machinery to prevent.

Because a feature has fewer degrees of freedom than a pose, the model can be run backwards.

DERIVATION Sampling poses from one landmark observation

$$x = m_{j,x} + \hat{r} \cos \hat{\gamma}, y = m_{j,y} + \hat{r} \sin \hat{\gamma}, \theta = \hat{\gamma} - \pi - \hat{\phi}$$

Step 1 — invert Bayes rule under a flat prior. We want $p(x_t \mid f_t^i, c_t^i, m)$. Bayes gives $\eta p(f_t^i \mid c_t^i, x_t, m) p(x_t \mid c_t^i, m)$, and if the pose prior is uniform the second factor is a constant. What is left is the measurement model, read as a function of the pose.

Step 2 — count the constraints. The reading supplies two numbers, (r, ϕ) ; the pose has three. One degree of freedom is therefore *unconstrained by construction*, and no cleverness will recover it. Geometrically: the range confines the robot to a circle of radius r about the landmark, and the bearing then fixes the heading given where on that circle it sits.

Step 3 — sample the free parameter, then the noise. Draw the angular position of the robot around the landmark uniformly, $\hat{\gamma} \sim \mathcal{U}(0, 2\pi)$, and perturb the observation by its own noise, $\hat{r} = r_t^i + \varepsilon_{\sigma_r^2}$, $\hat{\phi} = \phi_t^i + \varepsilon_{\sigma_\phi^2}$. Perturbing the *observation* rather than the prediction is legitimate because a Gaussian is symmetric in its two arguments — the same trick that makes Chapter 9's odometry sampler a three-liner.

Step 4 — place the pose. Sitting at angle $\hat{\gamma}$ around the landmark puts the robot at $(m_{j,x} + \hat{r} \cos \hat{\gamma}, m_{j,y} + \hat{r} \sin \hat{\gamma})$, from which the landmark bears $\hat{\gamma} + \pi$ in world coordinates. The heading that makes the landmark appear at relative bearing $\hat{\phi}$ is therefore $\theta = \hat{\gamma} + \pi - \hat{\phi}$, which Table 6.5 writes as $\hat{\gamma} - \pi - \hat{\phi}$ — the same angle, and worth checking rather than trusting.

Step 5 — the caveat. The prior is not uniform in reality: poses inside walls are impossible, and the robot was somewhere plausible a moment ago. Everything this sampler produces must therefore be treated as a *proposal* to be reweighted, which is exactly how Chapter 12 uses it in the mixture-MCL proposal that lets a kidnapped robot recover in one step. ■

ALGORITHM `sample_landmark_model_known_correspondence(f_t^i, c_t^i, m)`

COST `O(1)` -- one uniform and two normal draws

in a feature and its known identity, the map

out a pose x_t drawn from $p(x_t \mid f_t^i, c_t^i, m)$ under a uniform pose prior

```

1:  $j = c_t^i$ 
2:  $\hat{\gamma} = \text{rand}(0, 2\pi)$ 
3:  $\hat{r} = r_t^i + \text{sample}(\sigma_r^2)$ 
4:  $\hat{\phi} = \phi_t^i + \text{sample}(\sigma_\phi^2)$ 
5:  $x = m_{j,x} + \hat{r} \cos \hat{\gamma}$ 
6:  $y = m_{j,y} + \hat{r} \sin \hat{\gamma}$ 
7:  $\theta = \hat{\gamma} - \pi - \hat{\phi}$ 
8: return  $(x \ y \ \theta)^\top$ 
```

INTERACTIVE · w10.5

The Landmark Donut

One landmark observation never localizes a robot — two constraints cannot pin down three degrees of freedom, no matter how good the sensor is.

Run this simulation in the web edition: `/chapters/ch10-sensor-models #w10.5`

The counting argument is worth stating once, because it governs every feature-based estimator in Part V. Each detection supplies two numbers about the pose — a range and a bearing — and the pose has three unknowns. One detection therefore leaves a one-parameter family of explanations; two detections give four constraints on three unknowns and the family collapses to a blob whose size is pure measurement noise, about 20 cm across for $\sigma_r = 0.25$ m. Adding a third detection buys you not localization but *redundancy* — which is exactly what

Chapter 11 needs, because redundancy is the only thing that can catch a wrong correspondence.

10.3.7 The lie of independence

Now the part that the baseline defers and that decides whether your filter survives contact with a real building.

$p(z_t \mid x_t, m) = \prod_k p(z_t^k \mid x_t, m)$ says the beams are independent given the pose. They are not, and the dependence is not subtle: a person standing in front of the robot corrupts twenty adjacent beams at once, and a map that is 2 cm off corrupts *every* beam in the same direction, forever.

INTERACTIVE · w10.4

The Overconfidence Meter

More beams do not mean better localization. Multiplying correlated evidence buys certainty without buying information.

Run this simulation in the web edition: `/chapters/ch10-sensor-models #w10.4`

DERIVATION What false independence costs, and the three ways to pay it back

$$\log p(z|x)^{1/\kappa} = (1/\kappa) \sum_k \log p(z_t^k|x, m)$$

Step 1 — the extreme case. Suppose two beams are perfectly correlated: $z^2 \equiv z^1$. The honest likelihood of the pair is $p(z^1 \mid x)$ — the second reading adds nothing. The product rule reports $p(z^1 \mid x)^2$. Every likelihood *ratio* between two hypotheses is therefore squared, so the posterior odds are squared, and a hypothesis that was twice as likely becomes four times as likely on no new evidence at all.

Step 2 — the general case. If a scan of K beams carries only K_{eff} beams' worth of independent information, the product overstates every log-odds by the factor K/K_{eff} . The log-likelihood grows linearly in K whether or not the information does: for K genuinely independent measurements of the same quantity the posterior width would fall like $1/\sqrt{K}$, and the table below falls faster still, because each new beam also brings geometry the earlier ones missed. What matters is that the width is driven by K and is completely blind to whether the map is right. The result is not a wrong estimate; it is a *certain* estimate, and a filter that is certain stops listening.

Step 3 — measured, on the Apartment. Take the corridor pose in the widget above and a map that is 1% too wide, which is the kind of defect an unlucky wheel-radius calibration bakes into a SLAM-built map. The posterior over the along-corridor coordinate behaves like this:

beams K	σ , correct map	σ , 1% error	peak offset, 1% error	truth inside the 95% set, 1% error
4	7.97 cm	7.86 cm	-7.0 cm	yes
9	3.96 cm	4.00 cm	-6.5 cm	no
30	1.56 cm	1.57 cm	-3.5 cm	no
90	0.96 cm	0.73 cm	-4.5 cm	no
180	0.41 cm	0.68 cm	-4.0 cm	no

With a perfect map the truth stays inside the interval at every K . With a 1% map error, the estimate is stuck about 4 cm off and the claimed uncertainty keeps shrinking around the wrong answer: at $K = 180$ the truth is roughly six standard deviations outside the filter's own belief. The lag-1 correlation of the beam residuals tells you which regime you are in without knowing the truth — it is -0.03 with the correct map and $+0.44$ with the wrong one.

Step 4 — the three repairs are one repair. Subsample every κ -th beam; inflate σ_{hit} ; or temper,

$$p(z_t \mid x_t, m)^{1/\kappa} \iff \frac{1}{\kappa} \sum_k \log p(z_t^k \mid x_t, m)$$

All three flatten the likelihood surface; they differ in what else they do. Subsampling throws away $1 - 1/\kappa$ of the data, so the peak moves as well as widening — measured on the same scan, $\kappa = 10$ tempering gives $\sigma = 2.1$ cm with the peak where the full scan put it (-4.0 cm), while subsampling to 18 beams gives a comparable $\sigma = 3.4$ cm but drags the peak to -5.0 cm and *still* fails to cover the truth. Inflating σ_{hit} widens each beam's tolerance but also changes which beams count as outliers, which is a different intervention wearing the same coat. Tempering is the cleanest: one scalar, no data discarded, and it acts linearly on the log-likelihood you already compute.

Step 5 — where κ comes from. It is not free. Tempering has a respectable statistical home — a generalized Bayesian update with a scaled loss, in the sense of Bissiri, Holmes and Walker — but the theory does not hand you the number. Calibrate it: run the filter on logged data with ground truth and choose the smallest κ whose 95% credible sets actually contain the truth 95% of the time. In the experiment above that is $\kappa \approx 10$ for a 1% map error and $\kappa = 1$ for a perfect one, and if that sounds like a lot of tuning, note that AMCL's `laser_max_beams` parameter is the same knob with worse manners. ■

10.4 Implementation in Rust

The sensor crate exports one trait, the range-finder models that implement it, and one model that deliberately does not — a feature detection is not a scan, and pretending otherwise would put a `Vec<f64>` where a (r, ϕ, s) belongs. All of it is consumed unchanged by Chapter 11 and Chapter 12, so the interface matters more than any of the models behind it.

RUST `crates/sensor/src/lib.rs`

```

1 use nalgebra::{Matrix3, Point2};
2 use pr_core::geom::se2::SE2;
3 use sim::World; // Chapter 4's polyline world *is* the map, until Chapter 13
4
5 // One sweep of a range finder: ranges plus the bearings they were taken at.
6 //
7 // Bearings are stored, not assumed. A sub-sampled scan is still a valid `Scan`,
8 // which is what makes tempering-by-subsampling a two-line operation instead of
9 // an index-arithmetic bug farm.
10 #[derive(Clone, Debug)]
11 pub struct Scan {
12     pub ranges: Vec<f64>,
13     // Bearings relative to the robot's heading, radians.
14     pub bearings: Vec<f64>,
15     pub max_range: f64,
16 }
17
18 // Anything Chapters 11-12 can localize with.
19 //
20 // Log-likelihood, not likelihood: the product for any hypothesis that is even
21 // slightly wrong underflows `f64` within a hundred beams, and tempering is a
22 // single multiply in log space.
23 pub trait SensorModel {
24     fn log_likelihood(&self, scan: &Scan, x: &SE2, map: &World) -> f64;
25
26     // Cheap enough to call per particle per step? Chapter 12 branches on it.
27     fn is_cheap(&self) -> bool {
28         false
29     }
30 }
31
32 // Theta -- the six intrinsic parameters of the beam model.
33 //
34 // The four weights are *not* a `[f64; 4]`: naming them stops the classic
35 // transposition bug where `z_short` and `z_max` get swapped and the model
36 // starts believing every dropout is a person.
37 #[derive(Clone, Copy, Debug, PartialEq)]
38 pub struct BeamIntrinsics {
39     pub z_hit: f64,
40     pub z_short: f64,
41     pub z_max: f64,
42     pub z_rand: f64,
43     pub sigma_hit: f64,
44     pub lambda_short: f64,
45     pub max_range: f64,
46 }
47
48 impl BeamIntrinsics {
49     // The weights must live on the simplex; everything downstream assumes it.

```

```

56     pub fn is_normalized(&self) -> bool {
57         (self.z_hit + self.z_short + self.z_max + self.z_rand - 1.0).abs() < 1e-9
58     }
59 }
60
61 /// A range-bearing-signature detection, Thrun's  $f = (r, \phi, s)$ .
62 #[derive(Clone, Copy, Debug)]
63 pub struct Feature {
64     pub r: f64,
65     pub phi: f64,
66     pub s: f64,
67 }
68
69 /// The map's view of a landmark. `Q` is the feature noise covariance  $Q_t$ .
70 #[derive(Clone, Copy, Debug)]
71 pub struct Landmark {
72     pub xy: Point2<f64>,
73     pub signature: f64,
74 }
75
76 pub type FeatureNoise = Matrix3<f64>;

```

The beam model itself is a direct transcription of Table 6.1, with the two truncation normalizers spelled out because they are what a reader will otherwise skip.

RUST crates/sensor/src/beam.rs

```

1  use crate::{BeamIntrinsics, Scan, SensorModel};
2  use pr_core::geom::se2::SE2;
3  use sim::{ray_cast, World};
4  use stats::distribution::{ContinuousCDF, Normal};
5
6  pub struct BeamModel {
7      pub intr: BeamIntrinsics,
8      /// Use every `stride`-th beam. See `tempering.rs` for why this is a lie
9      /// that is sometimes worth telling.
10     pub stride: usize,
11 }
12
13 /// Within this of `max_range`, a reading *is* a max-range reading. Floating
14 /// point equality against a sensor's advertised limit is not a plan.
15 const MAX_EPS: f64 = 1e-9;
16
17 impl BeamModel {
18     ///  $p(z \mid x, m)$  for one beam -- Table 6.1, lines 4-7.
19     pub fn beam_likelihood(&self, z: f64, z_star: f64) -> f64 {
20         let i = &self.intr;
21
22         //  $p_{\text{hit}}$ : a Gaussian truncated to  $[0, z_{\text{max}}]$ .  $\eta$  is the reciprocal of the
23         // mass that survives truncation; without it, hypotheses whose predicted
24         // range sits near a boundary are silently favoured.
25         let p_hit = if (0.0..=i.max_range).contains(&z) {
26             let n = Normal::new(z_star, i.sigma_hit).expect("sigma_hit > 0");
27             let mass = n.cdf(i.max_range) - n.cdf(0.0);
28             if mass > 1e-12 {
29                 let d = (z - z_star) / i.sigma_hit;

```

```

30         (-0.5 * d * d).exp()) / (i.sigma_hit * (2.0 * std::f64::consts::PI).sqrt())
31     ) / mass
32     } else {
33         0.0
34     }
35     } else {
36         0.0
37     };
38
39     // p_short: the nearest hit of a Poisson clutter field of rate lambda along
40     // the ray, truncated at the wall.
41     let p_short = if z >= 0.0 && z <= z_star && z_star > 0.0 {
42         let eta = 1.0 / (1.0 - (-i.lambda_short * z_star).exp());
43         eta * i.lambda_short * (-i.lambda_short * z).exp()
44     } else {
45         0.0
46     };
47
48     // p_max is a point mass: it integrates to 1 on its own.
49     let p_max = if z >= i.max_range - MAX_EPS { 1.0 } else { 0.0 };
50     let p_rand = if z >= 0.0 && z < i.max_range { 1.0 / i.max_range } else { 0.0 };
51
52     i.z_hit * p_hit + i.z_short * p_short + i.z_max * p_max + i.z_rand * p_rand
53 }
54
55 /// The four *unweighted* component densities, for the EM E-step.
56 pub fn components(&self, z: f64, z_star: f64) -> [f64; 4] {
57     let one_hot = |c: usize| {
58         let mut i = self.intr;
59         i.z_hit = (c == 0) as u8 as f64;
60         i.z_short = (c == 1) as u8 as f64;
61         i.z_max = (c == 2) as u8 as f64;
62         i.z_rand = (c == 3) as u8 as f64;
63         BeamModel { intr: i, stride: 1 }.beam_likelihood(z, z_star)
64     };
65     [one_hot(0), one_hot(1), one_hot(2), one_hot(3)]
66 }
67
68 impl SensorModel for BeamModel {
69     /// `beam_range_finder_model` -- Table 6.1, in log space.
70     fn log_likelihood(&self, scan: &Scan, x: &SE2, map: &World) -> f64 {
71         let mut q = 0.0;
72         for k in (0..scan.ranges.len()).step_by(self.stride.max(1)) {
73             let z_star = ray_cast(map, x.x(), x.y(), x.theta() + scan.bearings[k], scan.
max_range);
74             q += self.beam_likelihood(scan.ranges[k], z_star).max(1e-300).ln();
75         }
76         q
77     }
78 }
79
80 /// `learn_intrinsic_parameters` -- Table 6.2.
81 ///
82 /// Takes ranges paired with the ray-cast range each *should* have had. Caching
83 /// `z_star` outside the loop is what turns EM from "one ray cast per beam per
84 /// iteration" into "one ray cast per beam, ever".
85 pub fn learn_intrinsics(
86     z: &[f64],
87     z_star: &[f64],
88     init: BeamIntrinsics,

```

```

89     iters: usize,
90 ) -> (BeamIntrinsics, Vec<f64>) {
91     let mut intr = init;
92     let mut trace = Vec::with_capacity(iters);
93
94     for _ in 0..iters {
95         let model = BeamModel { intr, stride: 1 };
96         let (mut s_hit, mut s_short, mut s_max, mut s_rand) = (0.0, 0.0, 0.0, 0.0);
97         let (mut sq_hit, mut z_short_sum, mut ll) = (0.0, 0.0, 0.0);
98
99         for (&zi, &si) in z.iter().zip(z_star) {
100             let c = model.components(zi, si);
101             let w = [intr.z_hit * c[0], intr.z_short * c[1], intr.z_max * c[2], intr.
z_rand * c[3]];
102             let total: f64 = w.iter().sum();
103             if total <= 0.0 {
104                 continue; // no cause explains this reading; refuse to guess
105             }
106             let e = [w[0] / total, w[1] / total, w[2] / total, w[3] / total];
107
108             s_hit += e[0];
109             s_short += e[1];
110             s_max += e[2];
111             s_rand += e[3];
112             sq_hit += e[0] * (zi - si).powi(2);
113             z_short_sum += e[1] * zi;
114             ll += total.max(1e-300).ln();
115         }
116
117         trace.push(ll / z.len() as f64);
118         let n = z.len() as f64;
119         intr = BeamIntrinsics {
120             z_hit: s_hit / n,
121             z_short: s_short / n,
122             z_max: s_max / n,
123             z_rand: s_rand / n,
124             // A component with no responsibility keeps its shape parameter
125             // rather than recomputing it from an empty sum.
126             sigma_hit: if s_hit > 1e-9 { (sq_hit / s_hit).sqrt().max(1e-3) } else { intr.
sigma_hit },
127             lambda_short: if z_short_sum > 1e-9 {
128                 (s_short / z_short_sum).clamp(1e-3, 50.0)
129             } else {
130                 intr.lambda_short
131             },
132             ..intr
133         };
134     }
135     (intr, trace)
136 }

```

The likelihood field is two files' worth of idea in one: the distance transform, then the lookup.

RUST `crates/sensor/src/likelihood_field.rs`

```

1 use crate::{BeamIntrinsics, Scan, SensorModel};
2 use pr_core::geom::se2::SE2;

```



```

3 use sim::World;
4
5 /// Distance to the nearest occupied cell, in metres, on a regular grid.
6 pub struct DistanceField {
7     cells: Vec<f32>,
8     nx: usize,
9     ny: usize,
10    res: f64,
11    origin: (f64, f64),
12 }
13
14 /// Stands in for +inf with a real number, so the envelope arithmetic below never
15 /// evaluates inf - inf.
16 const INF: f64 = 1e20;
17
18 /// Felzenszwalb-Huttenlocher, Figure 1: the lower envelope of the parabolas
19 /// z ? (z - q)2 + f(q). All parabolas share a curvature, so any two meet
20 /// exactly once and the envelope can be built by one left-to-right sweep.
21 fn dt_1d(f: &[f64], out: &mut [f64]) {
22     let n = f.len();
23     let mut v = vec![0usize; n]; // parabola indices currently on the envelope
24     let mut z = vec![0.0f64; n + 1]; // breakpoints between them
25     let mut k = 0;
26     v[0] = 0;
27     z[0] = -INF;
28     z[1] = INF;
29
30     for q in 1..n {
31         let mut s = intersect(f, q, v[k]);
32         while k > 0 && s <= z[k] {
33             k -= 1; // the top parabola is entirely above the new one: pop it
34             s = intersect(f, q, v[k]);
35         }
36         k += 1;
37         v[k] = q;
38         z[k] = s;
39         z[k + 1] = INF;
40     }
41
42     let mut k = 0;
43     for q in 0..n {
44         while z[k + 1] < q as f64 {
45             k += 1;
46         }
47         let d = q as f64 - v[k] as f64;
48         out[q] = d * d + f[v[k]];
49     }
50 }
51
52 fn intersect(f: &[f64], q: usize, p: usize) -> f64 {
53     ((f[q] + (q * q) as f64) - (f[p] + (p * p) as f64)) / (2 * q as f64 - 2 * p as f64)
54 }
55
56 impl DistanceField {
57     /// Rasterize the map, transform down the columns, transform along the rows,
58     /// take one square root per cell. O(N), and exact on the lattice.
59     ///
60     /// `rasterize` walks each wall at a third of a cell and returns the seed
61     /// grid: 0.0 at a cell a wall passes through, INF elsewhere. Indices are
62     /// clamped, not dropped -- the Apartment's shell lies exactly on the
63     /// bounding box, and a wall that falls off the grid leaves the whole border

```

```

64  /// looking like open space.
65  pub fn from_map(map: &World, res: f64) -> Self {
66      let (nx, ny, mut f) = rasterize(map, res);
67      let mut buf = vec![0.0; nx.max(ny)];
68
69      for i in 0..nx {
70          let col: Vec<f64> = (0..ny).map(|j| f[j * nx + i]).collect();
71          dt_ld(&col, &mut buf[..ny]);
72          for j in 0..ny {
73              f[j * nx + i] = buf[j];
74          }
75      }
76      for j in 0..ny {
77          let row: Vec<f64> = (0..nx).map(|i| f[j * nx + i]).collect();
78          dt_ld(&row, &mut buf[..nx]);
79          f[j * nx..j * nx + nx].copy_from_slice(&buf[..nx]);
80      }
81
82      let cells = f.iter().map(|d| (d.sqrt() * res) as f32).collect();
83      Self { cells, nx, ny, res, origin: (map.bounds.min_x, map.bounds.min_y) }
84  }
85
86  /// Nearest-cell lookup, clamped at the border. Queries outside the map are
87  /// charged the extra Euclidean distance so an off-map endpoint reads as
88  /// *unlikely* rather than merely uninformative.
89  #[inline]
90  pub fn dist(&self, x: f64, y: f64) -> f64 {
91      let i = (((x - self.origin.0) / self.res - 0.5).round() as isize)
92          .clamp(0, self.nx as isize - 1) as usize;
93      let j = (((y - self.origin.1) / self.res - 0.5).round() as isize)
94          .clamp(0, self.ny as isize - 1) as usize;
95      self.cells[j * self.nx + i] as f64
96  }
97  }
98
99  pub struct LikelihoodField {
100      pub field: DistanceField,
101      pub intr: BeamIntrinsics,
102      /// Sensor pose in the robot frame -- Thrun's (x_k,sens, y_k,sens).
103      pub sensor_offset: (f64, f64),
104  }
105
106  impl SensorModel for LikelihoodField {
107      /// `likelihood_field_range_finder_model` -- Table 6.3. No ray casting, no
108      /// correspondence, and max-range readings are skipped outright: "I saw
109      /// nothing" is not evidence about where the nearest wall is.
110      fn log_likelihood(&self, scan: &Scan, x: &SE2, _map: &World) -> f64 {
111          let (c, s) = (x.theta().cos(), x.theta().sin());
112          let sx = x.x() + c * self.sensor_offset.0 - s * self.sensor_offset.1;
113          let sy = x.y() + s * self.sensor_offset.0 + c * self.sensor_offset.1;
114
115          let mut q = 0.0;
116          for (k, &z) in scan.ranges.iter().enumerate() {
117              if z >= scan.max_range - 1e-9 {
118                  continue;
119              }
120              let a = x.theta() + scan.bearings[k];
121              let d = self.field.dist(sx + z * a.cos(), sy + z * a.sin());
122              let g = (-0.5 * (d / self.intr.sigma_hit).powi(2)).exp()
123                  / (self.intr.sigma_hit * (2.0 * std::f64::consts::PI).sqrt());
124              q += (self.intr.z_hit * g + self.intr.z_rand / self.intr.max_range)

```

```

125         .max(1e-300)
126         .ln();
127     }
128     q
129 }
130
131 fn is_cheap(&self) -> bool {
132     true
133 }
134 }

```

The landmark model and its inverse are short enough to show together, and the inverse is the one that will matter later.

RUST crates/sensor/src/landmark.rs

```

1 use crate::{Feature, FeatureNoise, Landmark};
2 use pr_core::geom::se2::{wrap_angle, SE2};
3 use rand::distr::{Distribution, Uniform};
4 use rand::Rng;
5 use rand_distr::Normal;
6
7 pub struct LandmarkModel {
8     ///  $Q_t = \text{diag}(\sigma_{r^2}, \sigma_{\phi^2}, \sigma_{s^2})$ .
9     pub q: FeatureNoise,
10 }
11
12 impl LandmarkModel {
13     /// `landmark_model_known_correspondence` -- Table 6.4, in log space.
14     pub fn log_likelihood(&self, f: &Feature, j: &Landmark, x: &SE2) -> f64 {
15         let (dx, dy) = (j.xy.x - x.x(), j.xy.y - x.y());
16         let r_hat = dx.hypot(dy);
17         // The wrap is not optional: an unwrapped 2pi error in the bearing
18         // residual rejects the correct landmark and accepts a wrong one.
19         let phi_hat = wrap_angle(dy.atan2(dx) - x.theta());
20
21         let e = [f.r - r_hat, wrap_angle(f.phi - phi_hat), f.s - j.signature];
22         (0..3)
23             .map(|i| {
24                 let var = self.q[(i, i)];
25                 -0.5 * (e[i] * e[i] / var + (2.0 * std::f64::consts::PI * var).ln())
26             })
27             .sum()
28     }
29
30     /// `sample_landmark_model_known_correspondence` -- Table 6.5.
31     ///
32     /// Two constraints, three degrees of freedom: gamma is the one the reading
33     /// cannot see, so it is drawn uniformly. The result is a *proposal*, not a
34     /// belief -- the pose prior it assumes (uniform over the plane) is wrong in
35     /// every building ever built.
36     pub fn sample_pose<R: Rng + ?Sized>(&self, f: &Feature, j: &Landmark, rng: &mut R) -> SE2 {
37         let gamma = Uniform::new(0.0, std::f64::consts::TAU).unwrap().sample(rng);
38         let r_hat = (f.r + Normal::new(0.0, self.q[(0, 0)].sqrt()).unwrap().sample(rng)).max(0.0);
39         let phi_hat = f.phi + Normal::new(0.0, self.q[(1, 1)].sqrt()).unwrap().sample(rng);
40     }

```

```

40
41     SE2::new(
42         j.xy.x + r_hat * gamma.cos(),
43         j.xy.y + r_hat * gamma.sin(),
44         wrap_angle(gamma - std::f64::consts::PI - phi_hat),
45     )
46 }
47 }

```

Finally, the wrapper that makes honesty configurable. It takes any `SensorModel` and returns a weaker one — which is the whole content of Step 4 of the overconfidence derivation, as twelve lines of Rust.

RUST `crates/sensor/src/tempering.rs`

```

1  use crate::{Scan, SensorModel};
2  use pr_core::geom::se2::SE2;
3  use sim::World;
4
5  /// A sensor model that has been told to be less sure of itself.
6  ///
7  /// `kappa` divides the log-likelihood -- the  $p^{1/kappa}$  of Thrun sec.6.3.4 and the
8  /// scaled loss of generalized Bayesian updating. `stride` throws beams away
9  /// instead. They are not interchangeable: tempering keeps the peak where the
10 /// full scan puts it, subsampling moves it, so prefer kappa and use `stride` only
11 /// when the cost of the beams themselves is the problem.
12 pub struct Tempered<S: SensorModel> {
13     pub inner: S,
14     pub kappa: f64,
15     pub stride: usize,
16 }
17
18 impl<S: SensorModel> SensorModel for Tempered<S> {
19     fn log_likelihood(&self, scan: &Scan, x: &SE2, map: &World) -> f64 {
20         let stride = self.stride.max(1);
21         let thinned = if stride == 1 {
22             scan.clone()
23         } else {
24             Scan {
25                 ranges: scan.ranges.iter().step_by(stride).copied().collect(),
26                 bearings: scan.bearings.iter().step_by(stride).copied().collect(),
27                 max_range: scan.max_range,
28             }
29         };
30         self.inner.log_likelihood(&thinned, x, map) / self.kappa.max(1e-6)
31     }
32
33     fn is_cheap(&self) -> bool {
34         self.inner.is_cheap()
35     }
36 }

```

10.4.1 A worked example you can check by hand

Five beams, one wall, one EM iteration. The wall is at $z^* = 5$ m for every beam, the sensor's limit is $z_{max} = 10$ m, and we start from maximal ignorance: all four weights at 0.25, $\sigma_{hit} = 1$ m, $\lambda_{short} = 0.5 \text{ m}^{-1}$.

$$Z = \{4.5, 5.0, 3.0, 7.0, 10.0\}$$

The components. With $z^* = 5$ and $\sigma_{hit} = 1$, the truncation normalizer for p_{hit} is $1/(\Phi(5) - \Phi(-5)) = 1.0000006$ — five sigma from both boundaries, so it may be treated as 1, and the values are plain normal densities. For p_{short} , $\eta_s = 1/(1 - e^{-2.5}) = 1.089427$.

z_i	p_{hit}	p_{short}	p_{max}	p_{rand}
4.5	0.352065	0.057412	0	0.1
5.0	0.398943	0.044713	0	0.1
3.0	0.053991	0.121542	0	0.1
7.0	0.053991	0	0	0.1
10.0	0.000015	0	1	0

Two entries are worth pausing on. The readings at 3.0 and 7.0 have *identical* p_{hit} — both are two metres from the wall — but only the short one can be explained by clutter, which is the entire asymmetry of the model in one row. And the reading at exactly z_{max} has $p_{rand} = 0$, because p_{rand} is defined on $[0, z_{max})$; the max-range spike belongs to p_{max} alone.

The E-step. All four weights are equal, so the responsibilities are just the components normalized:

z_i	e_{hit}	e_{short}	e_{max}	e_{rand}
4.5	0.691032	0.112689	0	0.196279
5.0	0.733815	0.082245	0	0.183940
3.0	0.195951	0.441116	0	0.362933
7.0	0.350611	0	0	0.649389
10.0	0.000001	0	0.999999	0

Check the first row by hand: $0.352065 + 0.057412 + 0 + 0.1 = 0.509477$, and $0.352065/0.509477 = 0.691032$. The mixture likelihood of that reading is $0.25 \times 0.509477 = 0.127369$.

The M-step. Average the columns for the weights, and take the responsibility-weighted moments for the shapes:

$$z_{hit} = \frac{1.971411}{5} = 0.394282, \quad z_{short} = \frac{0.636050}{5} = 0.127210, \quad z_{max} = 0.200000, \quad z_{rand} = 0.278508$$

$$\sigma_{hit}^2 = \frac{0.691032(0.25) + 0 + 0.195951(4) + 0.350611(4) + \dots}{1.971411} = 1.196628 \Rightarrow \sigma_{hit} = 1.093905$$

$$\lambda_{short} = \frac{0.636050}{0.112689(4.5) + 0.082245(5) + 0.441116(3)} = \frac{0.636050}{2.241674} = 0.283737$$

The mean log-likelihood per beam before this step is -2.275038 ; after it, it has risen. Notice what one iteration did: the hit weight went from 0.25 to 0.39 because most readings *are* hits, the max weight went to exactly $1/5$ because exactly one of five beams maxed out, and σ_{hit} grew, because with a fat starting σ the model claims the reading at 7.0 as a hit and pays for it in variance. Ten more iterations and it will have sorted that out.

RUST crates/sensor/src/beam.rs (tests)

```

1  #[cfg(test)]
2  mod tests {
3      use super::*;
4      use approx::assert_relative_eq;
5
6      const Z: [f64; 5] = [4.5, 5.0, 3.0, 7.0, 10.0];
7      const Z_STAR: [f64; 5] = [5.0, 5.0, 5.0, 5.0, 5.0];
8
9      fn start() -> BeamIntrinsics {
10         BeamIntrinsics {
11             z_hit: 0.25,
12             z_short: 0.25,
13             z_max: 0.25,
14             z_rand: 0.25,
15             sigma_hit: 1.0,
16             lambda_short: 0.5,
17             max_range: 10.0,
18         }
19     }
20
21     /// The worked example in the text, to six decimals.
22     #[test]
23     fn toy_em_step() {
24         let model = BeamModel { intr: start(), stride: 1 };
25         let c = model.components(4.5, 5.0);
26         assert_relative_eq!(c[0], 0.352065, epsilon = 1e-6);
27         assert_relative_eq!(c[1], 0.057412, epsilon = 1e-6);
28         assert_relative_eq!(model.beam_likelihood(4.5, 5.0), 0.127369, epsilon = 1e-6);
29
30         let (fitted, trace) = learn_intrinsics(&Z, &Z_STAR, start(), 1);
31         assert_relative_eq!(trace[0], -2.275038, epsilon = 1e-6);
32         assert_relative_eq!(fitted.z_hit, 0.394282, epsilon = 1e-6);
33         assert_relative_eq!(fitted.z_short, 0.127210, epsilon = 1e-6);
34         assert_relative_eq!(fitted.z_max, 0.200000, epsilon = 1e-6);
35         assert_relative_eq!(fitted.z_rand, 0.278508, epsilon = 1e-6);
36         assert_relative_eq!(fitted.sigma_hit, 1.093905, epsilon = 1e-6);
37         assert_relative_eq!(fitted.lambda_short, 0.283737, epsilon = 1e-6);
38         assert!(fitted.is_normalized());
39     }
40
41     /// EM's defining property. A dip here is a bug, not a local optimum.
42     #[test]
43     fn em_log_likelihood_is_monotone() {
44         let (_, trace) = learn_intrinsics(&Z, &Z_STAR, start(), 40);

```

```

45     for w in trace.windows(2) {
46         assert!(w[1] >= w[0] - 1e-12, "log-likelihood decreased: {w:?}");
47     }
48 }
49
50 /// The exact transform must equal the  $O(N^2)$  definition, bit for bit.
51 #[test]
52 fn distance_transform_is_exact() {
53     let map = sim::apartment();
54     let field = DistanceField::from_map(&map, 0.4);
55     let brute = brute_force_distance(&map, 0.4);
56     for (a, b) in field.cells.iter().zip(&brute) {
57         assert_eq!(a, b);
58     }
59 }
60 }

```

NOTE

Both implementations exist and are checked against each other: `crates/sensor` is the canonical Rust, and `web/lib/models/beam-em.ts`, `web/lib/models/landmark-sampling.ts` and `web/lib/mapping/edt.ts` are the TypeScript port that runs the widgets on this page. The worked example above is an assertion in both, and the exact distance transform is checked against its own $O(N^2)$ definition in both — on the Apartment at a 40 cm cell, they agree to the last bit.

10.5 Putting it together

10.5.1 The race

The two range-finder models, head to head on one map and one scan — the other two are not commensurable with them and get a paragraph instead. The numbers below are what the Likelihood Field Explorer above computes: the same code, the same 40-beam scan in the Apartment's room B, the same 68×50 grid of hypotheses, run headless on a laptop. The widget prints its own throughput live, so you can check them against the machine you are reading this on.

	evaluations / ms	evaluations / ms, cluttered	largest step in the profile	largest step, cluttered
beam model	203	104	11.5 nats	56.1 nats
likelihood field	329	330	5.7 nats	4.7 nats

Read the columns as two independent arguments. **Cost:** the beam model's price is set by the complexity of the map, because every evaluation ray-casts against every wall; adding 26 chair legs halves its throughput and does not touch

the field's, which pays a fixed $O(K)$ of lookups no matter how baroque the room. **Smoothness:** the beam model's profile takes a 56-nat step between neighbouring hypotheses 14 mm apart, which is a factor of e^{56} in likelihood — a cliff that will strand any optimizer and starve any particle that lands on the wrong side. The field's residual roughness is not geometry at all but its own grid: halve the cell size and it halves too, which is exactly the behaviour you want from an approximation.

Two other results belong here, from the sections above. The **correlation** model is the most robust to a map that is partly wrong and the least defensible as a probability. The **landmark** model costs $O(1)$ per feature instead of $O(K)$ per scan and throws away almost everything — which is fine when the features are distinctive, and catastrophic when they are not, because c_t^i then becomes a discrete search that no amount of Gaussian machinery will save you from.

10.5.2 Which model, when

Use the **likelihood field** for localization in a known static map. It is what AMCL runs by default, it is smooth enough for a particle filter with a few thousand particles, and its weaknesses — no notion of occlusion, no penalty for seeing through walls — do not bite when the map is right and the environment is mostly static.

Use the **beam model** when the environment is dynamic enough that z_{short} is doing real work, or when you need a *generative* model — to simulate a sensor, to detect that a reading is surprising, or to reason about what the robot would see from a hypothetical pose, as Chapter 24 does when it evaluates where to go next.

Use **correlation** when the map is known to be partly wrong. Use **features** when the front end gives you features and the world gives you distinctive ones.

And in every case, temper. The configuration this book carries forward is 'Tempered

' with κ chosen by calibration on logged data — the exact object Chapter 12 constructs when it builds its MCL theater, and the reason its particle cloud stays honest for long enough to converge.

10.5.3 What comes next

Chapter 11 takes the landmark model and the correspondence variable and builds EKF localization on them, at which point c_t^i stops being notation and starts being the hardest part of the problem. Chapter 12 takes the range-finder models and makes them particle weights, where the smoothness argument above turns into the difference between converging and not. Chapter 13 inverts the question entirely — instead of $p(z \mid x, m)$ with the map known, it asks for $p(m \mid z, x)$ with the pose known — and Chapter 25 replaces the whole mixture with a network that is calibrated against exactly the diagnostics this chapter built.

10.6 Exercises

1 FOUNDATION •• The normalizer you were tempted to drop

Derive the truncation normalizer η of p_{hit} on $[0, z_{max}]$ in terms of the standard normal CDF, and show it equals $[\Phi((z_{max} - z^*)/\sigma_{hit}) - \Phi(-z^*/\sigma_{hit})]^{-1}$. Then answer the engineering question: for $z_{max} = 8$ m and $\sigma_{hit} = 0.12$ m, for which values of z^* does η differ from 1 by more than 1%? Finally, argue about EM: if you omit η entirely, which of the four components absorbs the missing mass, and in which direction does σ_{hit} move?

2 FOUNDATION ••• Fix the biased λ update

Step 6 of the EM derivation shows that Table 6.2's λ_{short} update is the estimator for an untruncated exponential. Write the log-likelihood of the *truncated* exponential on $[0, z^*]$, differentiate it, and show that the maximum-likelihood λ solves $1/\lambda - z^*/(e^{\lambda z^*} - 1) = \bar{z}$, where $\bar{z}$ is the responsibility-weighted mean. Implement one Newton step on this equation, and verify against the measured numbers in the text: for $\lambda = 1.0$, $z^* = 2$ m, the uncorrected estimator lands near 1.46 while the corrected one should land near 1.0.

3 FOUNDATION •• Why the exponential, exactly

The text derives p_{short} from a homogeneous Poisson clutter field along the ray. Redo it for a *non-homogeneous* field whose rate $\lambda(d)$ grows with distance — a decent model of a corridor that gets busier further from the robot. Show that the nearest-hit density becomes $\lambda(d) \exp(-\int_0^d \lambda(u) du)$, and explain what fitting a single-rate exponential to data generated this way will do to the estimated λ_{short} .

4 CONCEPTUAL • Predict, then check: who moves when the corridor fills

In the Beam Mixture Mixer, before you touch anything: write down which two of the six parameters you expect to move when "people in the corridor" is switched on, and in which direction. Now switch it on, press Fit (EM), and compare. Explain any parameter that moved which you did not predict — in particular, why some of the extra short mass ends up in z_{rand} rather than z_{short} , and what that tells you about how identifiable the four components really are.

5 CONCEPTUAL •• Find the tempering that buys back honesty

In the Overconfidence Meter with the map error on and $K = 180$, find the smallest integer κ for which the truth is back inside the 95% set. Then set $K = 18$ with $\kappa = 1$ — roughly the same amount of "evidence discarded" — and observe that it does not achieve the same thing. Explain the difference in terms of what each operation does to the peak versus the width, and say which one you would deploy on a robot whose

map error you cannot measure directly.

6 PRACTICAL •• Implement map correlation and race it

Implement `map_correlation_model` against the `SensorModel` trait: rasterize the scan into a local grid, correlate with the global map, clip at zero, take the log. Add it to the race table. Find a map error under which it beats the likelihood field, and explain why. Then answer the uncomfortable question: since $\max\{\rho, 0\}$ is not a density in z , what exactly goes wrong if you use it as an importance weight in Chapter 12's particle filter — and why does it work anyway?

7 PRACTICAL ••• A field that knows the doors can open

Extend `DistanceField` to take a *set* of maps — doors open, doors closed — and store the per-cell minimum distance over the set. Show that this is still one distance transform if you rasterize all variants into a single seed grid, and one transform per variant plus a pointwise min if you want to know *which* variant explained the scan. Wire it into the Likelihood Field Explorer and measure what it costs in peak sharpness at the correct pose. Then relate it to the dynamic-environment handling of Chapter 12: when is "assume the most convenient world" a better model than "estimate which world you are in"?

10.7 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 6 is the source of every algorithm in this chapter — Tables 6.1 through 6.5 — and of the epigraph. The EM derivation here follows §6.3.2–6.3.3 with the truncation bias of Step 6 added. [link](#)

Felzenszwalb, P. F. and Huttenlocher, D. P. (2012). Distance Transforms of Sampled Functions *Theory of Computing* 8(19), 415–428.

The exact $O(N)$ transform that makes likelihood fields effectively free to build. The lower-envelope-of-parabolas argument in this chapter's second derivation is theirs, and the same routine reappears as a max-product message pass in Chapter 22. doi:10.4086/toc.2012.v008a019

Plagemann, C., Kersting, K., Pfaff, P., and Burgard, W. (2007). Gaussian Beam Processes: A Nonparametric Bayesian Measurement Model for Range Finders *Proceedings of Robotics: Science and Systems III*.

What happens when you refuse to pick between the beam model and the endpoint model: a Gaussian process over the whole scan that learns the correlations this chapter's independence assumption denies. The right next step if the overconfidence section bothered you. doi:10.15607/RSS.2007.III.018

Olson, E. B. (2009). Real-Time Correlative Scan Matching *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 4387–4393.

The correlation model of §6.5 grown into a practical algorithm, with the multi-resolution trick that makes an exhaustive pose search real-time. Chapter 16 builds on it directly. [link](#)

Bissiri, P. G., Holmes, C. C., and Walker, S. G. (2016). A General Framework for Updating Belief Distributions *Journal of the Royal Statistical Society: Series B* 78(5), 1103–1130.

The statistical justification for tempering. A posterior built from a scaled loss rather than a likelihood is a coherent belief update, which is what turns the $1/\kappa$ hack of §6.3.4 into something you can defend. doi:10.1111/rssb.12158

Zhu, D., Wang, C., Wang, W., Garg, R., Scherer, S., and Meng, M. Q.-H. (2021). VDB-EDT: An Efficient Euclidean Distance Transform Algorithm Based on VDB Data Structure *arXiv:2105.04419*.

Where distance fields went after 2012: sparse hierarchical storage and incremental updates, so the field can be maintained online in 3-D at map scale rather than rebuilt. The modern answer to ‘what if the map changes?’ [link](#)

Kuang, H., Chen, X., Guadagnino, T., Zimmerman, N., Behley, J., and Stachniss, C. (2023). IR-MCL: Implicit Representation-Based Online Global Localization *IEEE Robotics and Automation Letters*.

This chapter’s mixture, replaced by a neural occupancy field that synthesizes the scan it expects. It is the clearest modern demonstration that the observation model — not the filter — is what limits localization accuracy, and it is where Chapter 25 picks up. doi:10.1109/LRA.2023.3239318

Macenski, S., Martí n, F., White, R., and Giné s Clavero, J. (2020). The Marathon 2: A Navigation System *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Evidence that this chapter is current practice: Nav2’s AMCL exposes exactly these models as `laser_model_type` (beam versus `likelihood_field`), and `laser_max_beams` is the subsampling knob of the overconfidence section. doi:10.1109/IR0545743.2020.9341207

Part IV

Localization

Localization I: Tracking with Gaussians

LOCALIZATION · Intermediate · 55 min



In fact, the key to successful localization lies in the approach for data association.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 7

Parts II and III built an estimator and two generative models. This chapter finally asks the question that started the field: **given a map, where is the robot?**

The satisfying answer is that you have already built it. EKF localization is the extended Kalman filter of Chapter 7, predicting with the velocity model of Chapter 9 and correcting with the landmark model of Chapter 10. Nothing new is invented. Two hundred lines of Rust and Rusty tracks a loop of the Apartment with a purple ellipse that breathes in and out.

The unsatisfying answer arrives about thirty seconds later. Every equation in that assembly is conditioned on a variable nobody derived: c_t , the correspondence — *which landmark is this?* It is discrete, it is not Gaussian, and it cannot be estimated by any filter in Part II. Get it right and localization is easy. Get it wrong once, and the filter does not degrade gracefully; it moves confidently toward a place the robot has never been, shrinks its covariance while doing so, and thereby makes the next wrong answer more likely than the last. This chapter is about that failure, the geometry that predicts it, the χ^2 gate that mitigates it, and the Gaussian mixture that hedges against it — right up to the point where hedging stops being enough and Chapter 12 has to throw away the Gaussian entirely.

🎯 AFTER THIS CHAPTER YOU CAN

- Place any localization problem in the taxonomy — tracking, global, or kidnapped; static or dynamic; passive or active — and say which of them a Gaussian can survive
- Assemble EKF localization from parts you already own: the Chapter 7 filter, the Chapter 9 velocity model, the Chapter 10 landmark model
- Derive every entry of G_t , V_t and H_t , and know why the control noise M_t is only rank 2
- Choose correspondences by maximum likelihood, gate them with a calibrated χ^2 threshold, and explain why the Mahalanobis metric is the only honest one
- Show numerically why a single wrong association is not a small error but a self-reinforcing one
- Read a NEES plot, and stop believing small ellipses
- Implement `EkfLocalizer` in Rust with const-generic measurement dimensions, and hedge with a multi-hypothesis mixture when the map is ambiguous

📦 ASSUMED

- The EKF and the on-manifold discipline of Chapter 7 — $\boxplus$, $\boxminus$, and tangent-space covariance
- The velocity motion model and its α parameters (Chapter 9)
- The range-bearing landmark model and the correspondence variable c_t (Chapter 10)
- χ^2 distributions and Mahalanobis distance (Chapter 2)

11.1 The problem

Here is a working localizer. Rusty drives laps of the Apartment corridor; the filter is the real `EkfLocalizer` from the library, the world generates noisy range-bearing detections of surveyed landmarks, and the purple ellipse is the posterior it reports.

Watch it for one lap before touching anything. Then do exactly one thing: press **close pair**.

🖥️ INTERACTIVE · w11.2

The EKF Localization Lab

A small ellipse is not evidence of a good estimate. Consistency — not confidence — is what tells you whether to believe a filter.

Run this simulation in the web edition: [/chapters/ch11-localization-gaussian #w11.2](#)

Under the dense preset the filter is *good*. Position RMSE over four hundred steps is 0.094 m, the ellipse swells while the robot crosses a landmark-free stretch — which is 42% of them — and snaps tight when a feature comes back into the cone, and the NEES trace scatters around 3 the way a truthful three-degree-of-freedom filter should. Four hundred and thirteen features are matched and every one of them is matched correctly.

Under the close pair preset, two of the surveyed landmarks sit 0.30 m apart — the two edges of a doorframe, entered into the map as separate features. Nothing else changes: same filter, same noise, same trajectory, same seed. And the run falls apart — but not in the way readers usually expect. It does not get noisier. It gets **confident and wrong**, which is a strictly worse failure, because every number the filter publishes downstream still looks healthy.

Three claims follow from that one demonstration, and the rest of the chapter earns them.

The estimator is not the hard part. Everything in the prediction and correction steps is assembly of parts you already have. We will still derive it in full, because every implementer transcribes those Jacobians by hand at least once, but there is no new idea in it.

The hard part is a discrete variable. Deciding what you are looking at is not estimation; it is a combinatorial choice made under uncertainty, and the Gaussian machinery has nothing to say about it. What it *does* provide is a metric in which to make the choice.

Distance must be measured in the currency of your own uncertainty. Not metres. The correct question is never "which landmark is closest?" but "which landmark, given how wrong I might be, best explains this reading?" — and the answers differ, systematically and predictably.

💡 THE ONE SENTENCE VERSION

EKF localization is the Chapter 7 filter with Chapter 9 and Chapter 10 substituted in; the only genuinely new problem is choosing c_t , and it must be chosen in the Mahalanobis metric of the innovation covariance S_t , gated by a calibrated χ^2 threshold, because one accepted wrong match poisons every step that follows.

11.2 Which localization problem is this?

Before deriving anything, it is worth being precise about what "localization" means, because it names at least six different problems of wildly different difficulty. Thrun's taxonomy splits them along three axes, and a Gaussian filter is honestly applicable to exactly one cell.

INTERACTIVE · w11.3

The Taxonomy Grid

Gaussian localization is not a solution to 'localization'. It is a solution to one cell of a six-cell grid, and it fails outside it for reasons you can name in advance.

Run this simulation in the web edition: [/chapters/ch11-localization-gaussian #w11.3](/chapters/ch11-localization-gaussian/#w11.3)

The three axes are worth stating in the book's own words.

Local versus global. *Position tracking* assumes the initial pose is known and the error stays small; the posterior is unimodal and a Gaussian is a fair summary of it. *Global localization* starts from a uniform prior — the robot is somewhere in the building, and after one door sighting the belief has one peak per door. *Kidnapped robot* is global localization plus a lie: the robot is confident about a pose it does not occupy. As the baseline puts it, in global localization the robot knows that it does not know where it is; a kidnapped robot does not even have that.

Static versus dynamic. In a static world the robot's pose is the only thing that changes. In a dynamic one, people walk and chairs move, and a fraction of every scan corresponds to nothing in the map. That is not extra noise: it is *correlated* extra noise, which is the one kind a Bayes filter cannot absorb (Chapter 5 showed why).

Passive versus active. A passive localizer estimates whatever the controller happens to drive past. An active one chooses where to go in order to learn — trading path length for certainty. That is a decision problem, and it waits for Chapter 22 and Chapter 24. But the option is worth naming here, because w11.4 below makes its value visible in a single button press.

This chapter lives in the top-left cell: **tracking, static, passive**. Chapter 12 takes the rest of the top row.

11.3 Markov localization

The formal umbrella over all six cells is one line of Chapter 5 with a map m added to every conditional:

$$\overline{\text{bel}}(x_t) = \int p(x_t \mid u_t, x_{t-1}, m) \text{bel}(x_{t-1}) dx_{t-1}$$

$$\text{bel}(x_t) = \eta \, p(z_t \mid x_t, m) \, \overline{\text{bel}}(x_t)$$

ALGORITHM Markov_localization(bel(x_{t-1}), u_t, z_t, m) COST depends
 entirely on the representation of bel -- that choice is the algorithm
in the previous belief, the control, the measurement, the map
out bel(x_t)

```

1: for all  $x_t$  do
2:    $\overline{\text{bel}}(x_t) = \int p(x_t \mid u_t, x_{t-1}, m) \text{bel}(x_{t-1}) dx_{t-1}$ 
3:    $\text{bel}(x_t) = \eta \, p(z_t \mid x_t, m) \overline{\text{bel}}(x_t)$ 
4: endfor
5: return  $\text{bel}(x_t)$ 

```

That is the whole content of "Markov localization": the Bayes filter, told about a map. Everything interesting is in the choice of representation for bel, and in the initial condition — which is precisely the taxonomy above:

$\text{bel}(x_0) = \delta(x_0 - \tilde{x}_0)$	Position tracking. A point mass at the known start pose; in practice a tight Gaussian.
$\text{bel}(x_0) = 1/ X $	Global localization. Uniform over every pose in the map.
$\text{bel}(x_0) = \delta(x_0 - \tilde{x}_0)$	Kidnapped robot. A point mass at the wrong pose — the same shape as tracking, and that is exactly the problem.

Notice that tracking and kidnapping have *the same functional form*. A filter cannot tell them apart by looking at its own belief. It can only tell them apart by looking at the evidence $\eta^{-1} = p(z_t \mid z_{1:t-1}, u_{1:t}, m)$ — the diagnostic Chapter 5 promised would come back. It comes back twice in this chapter: as the χ^2 gate, and as the mixture weight.

11.4 The mathematics

11.4.1 Notation this chapter adds

$c_t^i \in \{1, \dots, N+1\}$	Correspondence: which map landmark generated observed feature i. The value N+1 means "none of them" — an outlier. Ch. 10
$\hat{c}_t^i$	The estimated correspondence, chosen by maximum likelihood.
M_t	Control-noise covariance, 2×2, in (v, ω) space.

$V_t = \partial g / \partial u_t$	Control Jacobian, 3×2. Maps control noise into pose space.
$\hat{z}_t^j$	Predicted measurement of landmark j from the predicted pose.
$S_t^j = H_t^j \bar{\Sigma}_t (H_t^j)^\top + Q_t$	Innovation covariance for landmark j : how far a reading may plausibly fall from its prediction.
$d_M^2(z, \hat{z}^j)$	Squared Mahalanobis distance in the metric of S_t^j . This is the distance that matters.
$\gamma = \chi_{d,1-\epsilon}^2$	Gate threshold. 5.99 for a 2-D feature at 95%; 9.21 at 99%.
$\{(\mu^{(h)}, \Sigma^{(h)}, w^{(h)})\}$	A hypothesis set: the Gaussian mixture an MHT localizer carries.

11.4.2 Prediction: the velocity model, differentiated

The state is a pose $x_t = (x, y, \theta)^\top$, and the control is a commanded translational and rotational velocity $u_t = (v_t, \omega_t)^\top$ held for Δt . Chapter 9 integrated that exactly, as an arc of radius v/ω :

$$\bar{\mu}_t = g(u_t, \mu_{t-1}) = \mu_{t-1} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \mu_{t-1, \theta} + \frac{v_t}{\omega_t} \sin(\mu_{t-1, \theta} + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \mu_{t-1, \theta} - \frac{v_t}{\omega_t} \cos(\mu_{t-1, \theta} + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}$$

The covariance needs two Jacobians, because the uncertainty has two sources: the uncertainty already in the pose, and the uncertainty in what the wheels actually did.

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + V_t M_t V_t^\top, \quad M_t = \begin{pmatrix} \alpha_1 v_t^2 + \alpha_2 \omega_t^2 & 0 \\ 0 & \alpha_3 v_t^2 + \alpha_4 \omega_t^2 \end{pmatrix}$$

The term $V_t M_t V_t^\top$ deserves a sentence, because it is where the 2005 presentation improves on the earlier draft (which simply wrote an additive 3×3 R_t). Motion noise does not live in pose space. It lives in *control* space: the robot commanded (v, ω) and the wheels delivered something else. M_t is 2×2 , so $V_t M_t V_t^\top$ is a 3×3 matrix of **rank 2**. There is one direction in pose space along which the velocity model claims *perfect* certainty — and Chapter 9 already told us which one, when it introduced the third noise term $\hat{\gamma}$: a final rotation that no (v, ω) pair can produce. If you do not add it back, a filter running this model believes it knows its heading better than it possibly can. The lab widget above adds it as an explicit floor diag(0, 0, $(\alpha_5 v^2 + \alpha_6 \omega^2) \Delta t^2$), which is exactly the variance the Chapter 9 sampler injects.

11.4.3 Correction: the landmark model, differentiated

Chapter 10's front end reports features $f_t^i = (r_t^i, \phi_t^i, s_t^i)^\top$. Given a correspondence $c_t^i = j$, the predicted measurement of map landmark $m_j = (m_{j,x}, m_{j,y})$ from the predicted pose is pure trigonometry. Write $\delta = m_j - \bar{\mu}_{t,xy}$ and $q = \delta^\top \delta$:

$$\hat{z}_t^j = \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \\ m_{j,s} \end{pmatrix}, \quad H_t^j = \frac{1}{q} \begin{pmatrix} -\sqrt{q} \delta_x & -\sqrt{q} \delta_y & 0 \\ \delta_y & -\delta_x & -q \\ 0 & 0 & 0 \end{pmatrix}$$

Read the two useful rows of H_t^j before moving on; they explain most of what the widgets show.

The **range row** is the negative unit vector pointing from the robot to the landmark, and it has a zero in the θ column: turning in place does not change how far away something is. So a range measurement constrains position *along the line of sight* and nothing else.

The **bearing row** has -1 in the θ column *whatever the range*, and position terms that decay as $1/\sqrt{q}$. A distant landmark is therefore almost a pure compass: it pins heading tightly and position hardly at all. A near landmark is almost a pure position fix. This is why a sparse map of far-away beacons produces a filter with excellent heading and drifting position, and why the ellipse in the lab is long along the corridor rather than round.

The signature row is identically zero, which is Thrun's own observation: once the correspondence is known, the signature carries no information about the pose. It earns its place only in the *next* section, where the correspondence is not known.

ALGORITHM EKF_localization_known_correspondences($\mu_{\{t-1\}}$, $\Sigma_{\{t-1\}}$, u_t , z_t , c_t , m) COST $O(N)$ three-by-three updates for N observed features
in previous Gaussian belief, control, features, their correspondences, the map
out μ_t, Σ_t

- 1: $\bar{\mu}_t = g(u_t, \mu_{t-1})$
- 2: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + V_t M_t V_t^\top$
- 3: $Q_t = \text{diag}(\sigma_r^2, \sigma_\phi^2, \sigma_s^2)$
- 4: for all observed features $z_t^i = (r_t^i \phi_t^i s_t^i)^\top$ do
- 5: $j = c_t^i$
- 6: $\delta = \begin{pmatrix} m_{j,x} - \bar{\mu}_{t,x} \\ m_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}, \quad q = \delta^\top \delta$
- 7: $\hat{z}_t^i = (\sqrt{q}, \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta}, m_{j,s})^\top$
- 8: compute H_t^i as above
- 9: $S_t^i = H_t^i \bar{\Sigma}_t (H_t^i)^\top + Q_t$
- 10: $K_t^i = \bar{\Sigma}_t (H_t^i)^\top (S_t^i)^{-1}$
- 11: $\bar{\mu}_t \leftarrow \bar{\mu}_t \boxplus K_t^i (z_t^i \boxminus \hat{z}_t^i)$

```

12:    $\bar{\Sigma}_t \leftarrow (I - K_t^i H_t^i) \bar{\Sigma}_t$ 
13: endfor
14: return  $\mu_t = \bar{\mu}_t$ ,  $\Sigma_t = \bar{\Sigma}_t$ 

```

Two deviations from Thrun's Table 7.2 are deliberate and both come from Chapter 7.

First, lines 11–12 apply each feature **sequentially**, re-linearizing H_t against the updated mean, where the book sums $\sum_i K_t^i (z_t^i - \hat{z}_t^i)$ and $(I - \sum_i K_t^i H_t^i)$ at the end. The summed form is a first-order approximation to the sequential one and is only equivalent when the corrections are infinitesimal. Sequential costs nothing extra and is strictly better conditioned; Exercise 5 makes you measure the difference.

Second, $\boxplus$ and $\boxminus$ are not decoration. The measurement residual $z_t^i \boxminus \hat{z}_t^i$ wraps its bearing component to $(-\pi, \pi]$; without that, a predicted bearing of 179° and an observed -179° produce an innovation of 358° , a gain-scaled correction that hurls the estimate across the map, and a filter that fails exactly once per lap at exactly the same place. The mean update likewise wraps θ . This is the smallest possible instance of the manifold discipline, and it is the single most common bug in hand-written EKF localizers.

DERIVATION EKF localization from the Bayes filter, and every Jacobian entry

$$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + V_t M_t V_t^\top$$

Step 1 — start from Markov localization. Take the two lines of Markov_ localization and assume $\text{bel}(x_{t-1}) = \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$.

Step 2 — linearize the motion. g is nonlinear, so the exact predicted belief is not Gaussian. Expand about the mean and the commanded control:

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + G_t (x_{t-1} - \mu_{t-1}) + V_t (u_t^{\text{actual}} - u_t)$$

with $G_t = \partial g / \partial x_{t-1}$ and $V_t = \partial g / \partial u_t$, both evaluated at (μ_{t-1}, u_t) .

Step 3 — push the two Gaussians through. A linear map of a Gaussian is Gaussian, and the two sources are independent, so the covariances add: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + V_t M_t V_t^\top$. This is line 2 of the algorithm.

Step 4 — linearize the measurement. Repeat for h , this time about $\bar{\mu}_t$, obtaining H_t^j . Because h is evaluated per landmark, so is H .

Step 5 — apply the Kalman correction. With $\bar{\text{bel}}$ Gaussian and h linearized, the correction is the Chapter 6 algebra unchanged: $S_t = H \bar{\Sigma} H^\top + Q$, $K_t = \bar{\Sigma} H^\top S_t^{-1}$, and the two update lines. ■

The Jacobian entries in full. Write $c = \cos \mu_{t-1,\theta}$, $s = \sin \mu_{t-1,\theta}$, $c' = \cos(\mu_{t-1,\theta} + \omega_t \Delta t)$, $s' = \sin(\mu_{t-1,\theta} + \omega_t \Delta t)$. Then

$$G_t = \begin{pmatrix} 1 & 0 & \frac{v_t}{\omega_t}(-c + c') \\ 0 & 1 & \frac{v_t}{\omega_t}(-s + s') \\ 0 & 0 & 1 \end{pmatrix}, \quad V_t = \begin{pmatrix} \frac{-s+s'}{\omega_t} & \frac{v_t(s-s')}{\omega_t^2} + \frac{v_t c' \Delta t}{\omega_t} \\ \frac{c-c'}{\omega_t} & \frac{-v_t(c-c')}{\omega_t^2} + \frac{v_t s' \Delta t}{\omega_t} \\ 0 & \Delta t \end{pmatrix}$$

Every entry of both matrices divides by ω_t , and a robot driving down a corridor commands $\omega_t = 0$. The straight-line limits are not optional; take them analytically rather than hoping the floating point cancels:

$$\lim_{\omega \rightarrow 0} G_t = \begin{pmatrix} 1 & 0 & -v \Delta t s \\ 0 & 1 & v \Delta t c \\ 0 & 0 & 1 \end{pmatrix}, \quad \lim_{\omega \rightarrow 0} V_t = \begin{pmatrix} \Delta t c & -\frac{1}{2} v \Delta t^2 s \\ \Delta t s & \frac{1}{2} v \Delta t^2 c \\ 0 & \Delta t \end{pmatrix}$$

(The second column of V_t needs the *second*-order term of the expansion, since the first order cancels; that is why $\Delta t^2/2$ appears where you might have expected Δt .)

For the measurement Jacobian, differentiate $r = \sqrt{q}$ and $\phi = \text{atan2}(\delta_y, \delta_x) - \theta$ with $\delta = m_j - (x, y)$, so that $\partial \delta / \partial x = (-1, 0)^T$ and $\partial \delta / \partial y = (0, -1)^T$:

$$\frac{\partial r}{\partial x} = -\frac{\delta_x}{\sqrt{q}}, \quad \frac{\partial r}{\partial y} = -\frac{\delta_y}{\sqrt{q}}, \quad \frac{\partial r}{\partial \theta} = 0$$

$$\frac{\partial \phi}{\partial x} = \frac{\delta_y}{q}, \quad \frac{\partial \phi}{\partial y} = -\frac{\delta_x}{q}, \quad \frac{\partial \phi}{\partial \theta} = -1$$

which is the H_t^j printed above. Check the signs against your intuition: moving the robot *toward* the landmark ($+\delta_x$ direction) decreases the range, hence the leading minus.

A caveat on the summed form. Thrun's Table 7.2 computes all K_t^i against the *same* $\bar{\Sigma}_t$ and adds them. That is correct to first order and wrong beyond it: two features that each halve the along-corridor variance do not, when summed, quarter it — they can drive $I - \sum_i K^i H^i$ indefinite. Sequential updating avoids the question entirely and costs one extra 3×3 multiply per feature.

11.4.4 A worked example you can check by hand

Put Rusty at $\bar{\mu}_t = (2, 3, 0)$ with

$$\bar{\Sigma}_t = \text{diag}(0.25, 0.25, 0.01) \quad (\sigma_x = \sigma_y = 0.5 \text{ m}, \sigma_\theta = 0.1 \text{ rad})$$

and a single landmark at $m_j = (5, 3)$ — three metres straight ahead. The sensor has $\sigma_r = 0.2 \text{ m}$ and $\sigma_\phi = 0.05 \text{ rad}$, so $Q_t = \text{diag}(0.04, 0.0025)$.

Then $\delta = (3, 0)$, $q = 9$, $\sqrt{q} = 3$, and everything below is arithmetic you can do on paper.

$$\hat{z}_t = \begin{pmatrix} 3 \\ 0 \end{pmatrix}, \quad H_t = \begin{pmatrix} -1 & 0 & 0 \\ 0 & -\frac{1}{3} & -1 \end{pmatrix}$$

$$S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t = \begin{pmatrix} 0.25 & 0 \\ 0 & 0.0277\bar{7} \end{pmatrix} + \begin{pmatrix} 0.04 & 0 \\ 0 & 0.0025 \end{pmatrix} = \begin{pmatrix} 0.29 & 0 \\ 0 & 0.0402\bar{7} \end{pmatrix}$$

S_t came out diagonal, which is not a coincidence: with an isotropic position block the range and bearing rows of H are orthogonal. The predicted spreads are $\sigma_{r,\text{pred}} = 0.539 \text{ m}$ and $\sigma_{\phi,\text{pred}} = 0.201 \text{ rad}$ — note that the *predicted* range spread is more than twice the sensor's own 0.2 m , because most of it is the robot's ignorance about where it is, not the sensor's about what it saw.

$$K_t = \bar{\Sigma}_t H_t^\top S_t^{-1} = \begin{pmatrix} -0.8621 & 0 \\ 0 & -2.0690 \\ 0 & -0.2483 \end{pmatrix}$$

Now suppose the robot measures $z_t = (3.2, 0.05)$: twenty centimetres farther than expected, and fifty milliradians to the left. The innovation is $\nu = (0.2, 0.05)$, so

$$d_M^2 = \frac{0.2^2}{0.29} + \frac{0.05^2}{0.0402\bar{7}} = 0.1379 + 0.0621 = \boxed{0.200}$$

comfortably inside any sane gate. The correction is $K_t \nu = (-0.1724, -0.1034, -0.01241)$:

$$\mu_t = (1.8276, 2.8966, -0.01241), \quad \Sigma_t = \begin{pmatrix} 0.03448 & 0 & 0 \\ 0 & 0.07759 & -0.02069 \\ 0 & -0.02069 & 0.007517 \end{pmatrix}$$

Every number in that posterior tells you something.

The robot moved **backwards** in x , because it measured a longer range than expected and the landmark is straight ahead. σ_x collapsed from 0.5 m to

$\sqrt{0.03448} = 0.186$ m — the range measurement acted almost entirely along the line of sight, exactly as the first row of H promised. σ_y fell only to 0.279 m and σ_θ to 0.087 rad, because the bearing measurement had to explain a lateral offset and a heading error at once and cannot tell them apart: that inability is the -0.0207 correlation term, and it is negative because moving the robot down and turning it left produce the same bearing.

11.4.5 Data association: the actual problem

The algorithm above took c_t as an input. Nothing supplies it.

The maximum-likelihood answer is to pick, for each observed feature, the landmark that makes the reading least surprising:

$$\hat{c}_t^i = \arg \max_j \mathcal{N}\left(\underbrace{z_t^i; \hat{z}_t^j}_{\text{fit}}, \underbrace{S_t^j}_{\text{tie-break}}\right) = \arg \min_j \left[\underbrace{d_M^2(z_t^i, \hat{z}_t^j)}_{\text{fit}} + \underbrace{\ln \det S_t^j}_{\text{tie-break}} \right]$$

and then to accept it only if it survives a gate:

$$d_M^2(z_t^i, \hat{z}_t^{\hat{c}}) \leq \gamma = \chi_{d, 1-\epsilon}^2, \quad \chi_{2, 0.95}^2 = 5.991, \quad \chi_{2, 0.99}^2 = 9.210$$

The set $\{z : d_M^2(z, \hat{z}^j) \leq \gamma\}$ is the **validation region** — an ellipse in measurement space, drawn in orange in the widget below. It is the geometric object the whole section is about.

INTERACTIVE · w11.1

The Association Gate

The nearest landmark in metres is not the nearest landmark in the metric that matters. Distance is measured in units of the filter's own uncertainty.

Run this simulation in the web edition: </chapters/ch11-localization-gaussian#w11.1>

Spend a minute dragging the green dot before reading on, because the widget makes three points that are hard to make in prose.

The gate is not round, and its shape is a prediction. The robot's belief there is $\bar{\Sigma} = \text{diag}(0.64, 0.04, 0.0025)$ — 0.8 m of uncertainty along the heading, 0.2 m across it, the shape a corridor always produces — so the validation ellipses come out long in range and thin in bearing. Landmark m0 sits exactly 4 m straight ahead, which makes it checkable by hand: $S^{m0} = \text{diag}(0.64 + 0.04, 0.04/16 + 0.0025 + 0.0025) = \text{diag}(0.68, 0.0075)$. Its 95% gate therefore extends $\sqrt{5.99 \times 0.68} = 2.02$ m along the line of sight and $\sqrt{5.99 \times 0.0075} = 0.212$ rad across it, which at 4 m is 0.85 m of arc. That is a 2.4 : 1 ellipse, and any two landmarks closer together than about two metres *along the ray* are candidates for the same reading.

Euclidean and Mahalanobis disagree over a large region. Toggle the metric and drag. Take the reading $z = (3.55 \text{ m}, 0.05 \text{ rad})$, which the autoplay tour passes through. Its projected endpoint is 0.488 m from landmark m0 and 0.466 m from m1 — so a nearest-neighbour rule in metres picks m1. In the filter's own metric, $d_M^2(m0) = 0.631$ and $d_M^2(m1) = 1.872$, so maximum likelihood picks m0, by a factor of three. Both cannot be right, and only one of them used the information the filter has been carefully accumulating for the last hundred steps.

The $\ln \det S$ term is a real term, not a constant. It is dropped in Thrun's Table 7.3 and in most implementations, on the grounds that maximizing a Gaussian density and minimizing a Mahalanobis distance are the same thing. They are the same thing only when the candidates share a covariance. When they do not, the log-determinant penalizes candidates whose gates are wide — a landmark whose validation ellipse is twice as large in each axis pays $2 \ln 4 \approx 2.77$, which outweighs a full standard deviation of mismatch. Toggling it in the widget moves the decision boundary between m0 and m1 by a visible margin. It rarely overturns a clear winner; it always moves the boundary, and the boundary is precisely where a filter with two close landmarks lives.

DERIVATION ML correspondence as marginal maximization

$$\hat{c}_t = \arg \min_j d_M^2(z_t^i, \hat{z}_t^j) + \ln \det S_t^j$$

Step 1 — write down what is actually being maximized. The ML estimator chooses the correspondence vector that makes the data most likely, marginalizing the pose out:

$$\hat{c}_t = \arg \max_{c_t} p(z_t \mid c_{1:t}, m, z_{1:t-1}, u_{1:t}) = \arg \max_{c_t} \int p(z_t \mid c_t, x_t, m) \overline{\text{bel}}(x_t) dx_t$$

Step 2 — do the integral. Under the EKF's own approximations this is a Gaussian integral. With $\overline{\text{bel}} = \mathcal{N}(\bar{\mu}_t, \bar{\Sigma}_t)$ and $z_t^i = h(x_t, m_j) + \delta$, linearizing h about $\bar{\mu}_t$ makes the predictive distribution of the reading

$$p(z_t^i \mid c_t^i = j, \cdot) = \mathcal{N} \left(\hat{z}_t^j, \underbrace{H_t^j \bar{\Sigma}_t (H_t^j)^\top + Q_t}_{S_t^j} \right)$$

This is worth pausing on: S_t^j is not the sensor's covariance. It is the sensor's covariance *plus* the robot's own uncertainty projected into measurement space. A robot that is unsure where it is has wide gates and confuses landmarks; a robot that is sure has narrow gates and rejects true matches. That trade-off is the entire practical art of gating.

Step 3 — take logs. Dropping the $-\frac{d}{2} \ln 2\pi$ that is common to all candidates,

$$-2 \ln \mathcal{N}(z; \hat{z}^j, S^j) = d_M^2(z, \hat{z}^j) + \ln \det S^j + \text{const}$$

which is the score in the box above. Maximizing the density and minimizing Mahalanobis distance coincide **only** if $\det S^j$ is the same for all j .

Step 4 — factor over features, and notice what that costs. Maximizing over the whole vector c_t is exponential in the number of features. Every practical implementation, including Table 7.3, maximizes each c_t^i separately. That factorization is *wrong*, in a specific and knowable way: the features share a pose, so their prediction errors are correlated, and independent per-feature decisions can produce a joint assignment that no single pose explains. Two symptoms follow. First, two different features can be assigned to the same landmark — a violation of the *mutual exclusion* constraint that most sensors guarantee physically. Second, a set of individually plausible matches can be jointly absurd.

The classical fix is **joint compatibility** (Neira & Tardó s): gate the *stacked* innovation of a set of pairings against its full covariance, including the cross-blocks that per-feature gating throws away, and search for the largest jointly compatible set by branch and bound. Exercise 6 builds a two-feature version. The modern fix is to stop treating the discrete variable as something to be guessed before the continuous one is estimated, and instead to optimize over both — which is where Chapter 15 picks the thread up.

ALGORITHM `EKF_localization`(μ_{t-1} , Σ_{t-1} , u_t , z_t , m) COST $O(N \cdot n)$ gate evaluations for N features and n map landmarks

in previous belief, control, features (correspondences unknown), the map

out μ_t , Σ_t , and one association verdict per feature

- 1: $\bar{\mu}_t = g(u_t, \mu_{t-1}); \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + V_t M_t V_t^\top$
- 2: for all landmarks k in the map m do
- 3: $\delta_k = m_k - \bar{\mu}_{t,xy}, q_k = \delta_k^\top \delta_k$
- 4: $\hat{z}_t^k, H_t^k$ as in Table 7.2; $S_t^k = H_t^k \bar{\Sigma}_t (H_t^k)^\top + Q_t$
- 5: endfor

```

6: for all observed features  $z_t^i$  do
7:    $v^k = z_t^i \ominus \hat{z}_t^k$  for each  $k$  (bearing wrapped)
8:    $j(i) = \arg \min_k (v^k)^\top (S_t^k)^{-1} v^k + \ln \det S_t^k$ 
9:   if  $(v^{j(i)})^\top (S_t^{j(i)})^{-1} v^{j(i)} > \gamma$  then discard  $z_t^i$  as an outlier; continue
10:   $K_t^i = \bar{\Sigma}_t (H_t^{j(i)})^\top (S_t^{j(i)})^{-1}$ 
11:   $\bar{\mu}_t \leftarrow \bar{\mu}_t \boxplus K_t^i v^{j(i)}$ ;  $\bar{\Sigma}_t \leftarrow (I - K_t^i H_t^{j(i)}) \bar{\Sigma}_t$ 
12:  recompute lines 2–5 against the updated belief
13: endfor
14: return  $\mu_t = \bar{\mu}_t$ ,  $\Sigma_t = \bar{\Sigma}_t$ 

```

Line 9 is the one line Table 7.3 does not have, and it is the difference between a filter that survives a corridor and one that does not. Gaussians fall off exponentially, so a single outlier can dominate the entire correction; the baseline’s own practical-considerations section is blunt about the remedy — *“In practice, thresholding adds an important layer of robustness to the algorithm without which EKF localization tends to be brittle.”*

But notice what the gate can and cannot do. It rejects readings that are *surprising*. It is completely blind to readings that are unsurprising and wrong.

11.4.6 Why one wrong association is catastrophic

Here is the same worked example, poisoned. Everything is as before — Rusty at $(2, 3, 0)$ with $\bar{\Sigma} = \text{diag}(0.25, 0.25, 0.01)$, a mapped landmark at $(5, 3)$ — except that the feature was generated by a *different* object at $(5.6, 3)$, which the surveyor never recorded. The robot measures a range of 3.6 m and the filter attributes it to the landmark at $(5, 3)$.

Step 1. $v = (0.6, 0)$, so $d_M^2 = 0.36/0.29 = 1.24$. The gate is 5.99. **The wrong match is accepted, and it is not even close.** The gate’s range half-width here is $\sqrt{5.99 \times 0.29} = 1.32$ m; a landmark 0.6 m out of place is less than half of that. This is arithmetic, not bad luck.

The correction is $Kv = (-0.517, 0, 0)$, so μ_x moves to 1.483 while the truth is still 2: an error of 0.517 m. And σ_x shrinks from 0.5 m to 0.186 m. The filter is now wrong by 2.8 of its own standard deviations, and its NEES is 7.76 against an expectation of 3.

Step 2. The robot observes the same object again from the same place. Its predicted range to the landmark at $(5, 3)$ is now $5 - 1.483 = 3.517$, and the measurement is again 3.6. Innovation: 0.083. Predicted spread: $\sqrt{0.0345 + 0.04} = 0.273$. So

$$d_M^2 = \frac{0.083^2}{0.0745} = 0.092$$

The reading now looks *better* than a typical correct one. This is the mechanism, and it is worth saying out loud: **the filter has moved to the pose that makes its own mistake look right**. Error grows to 0.556 m, σ_x falls to 0.136 m, NEES reaches 16.7.

Step 3. $d_M^2 = 0.034$. Error 0.570 m, $\sigma_x = 0.113$ m, NEES 25.6, and climbing.

DERIVATION The poisoning cascade, in three steps

$$\mu_x \rightarrow m_{j,x} - r_{\text{meas}}, \quad \sigma_x \rightarrow 0$$

Step 1 — a wrong match injects a bias, not noise. Let the accepted feature come from an object at $m_j + b$ while the filter attributes it to m_j . The innovation has expectation $\mathbb{E}[v] = -Hb \neq 0$, so the update has expectation $\mathbb{E}[\Delta\mu] = -KHb$. Unlike measurement noise, this does not average out over repeated observations: it is the *same* b every time.

Step 2 — the covariance shrinks regardless. $\Sigma \leftarrow (I - KH)\Sigma$ depends only on H , $\bar{\Sigma}$ and Q . It has no idea whether the innovation was right. The filter therefore becomes *more* certain while becoming *less* correct — the definition of inconsistency, and the reason NEES catches this while RMSE does not.

Step 3 — the feedback closes. A smaller Σ means a smaller $S = H\Sigma H^\top + Q$, hence a tighter gate; and the mean has already moved toward the bias, so the next observation of the same wrong object has a smaller residual against a smaller gate. Both effects push d_M^2 down. The recursion has a fixed point, and it is exactly the pose at which the wrong landmark perfectly explains the reading:

$$\mu_x \rightarrow m_{j,x} - r_{\text{measured}}, \quad \Sigma_{xx} \sim \frac{\sigma_r^2}{k} \rightarrow 0$$

The filter converges — confidently, quietly, and to the wrong answer. Meanwhile every *true* landmark's residual is now large against a shrinking gate, so genuine features start failing line 9 and being discarded as outliers. The close pair preset in [w11.2](#) shows the rejection counter climbing while the error grows: the filter starves itself of exactly the evidence that would have saved it.

Contrast this with an honest outlier *test* that also inflates the covariance when it fires (sometimes called a "covariance inflation" or robust-cost response — see Chapter 15). Widening Σ after a suspicious reading widens the gate, which lets the true landmark back in. The Gaussian-filter version of robustness is not "reject harder"; it is "reject, and admit that you are now less certain".

⚠ WHERE ROBOTS BREAK THIS

This is why the design rule for landmark maps is a *separation* rule, not a density rule. Two landmarks are confusable when their separation along the line of sight is smaller than roughly $2\sqrt{\gamma \lambda_{\max}(S)}$ — which, for a robot that is 0.8 m uncertain along a corridor, is about four metres. The corresponding advice in the literature is exactly Thrun's: choose landmarks far apart, *and* keep pose uncertainty small; and those two goals fight each other, because keeping uncertainty small requires seeing landmarks often.

11.5 Hedging: multi-hypothesis tracking

If the filter cannot decide safely, it can decline to decide. Represent the belief as a Gaussian mixture over association histories:

$$\text{bel}(x_t) = \frac{1}{\sum_h w_t^{(h)}} \sum_h w_t^{(h)} \mathcal{N}\left(x_t; \mu_t^{(h)}, \Sigma_t^{(h)}\right)$$

Each component is a full EKF localizer conditioned on one history of choices. When a feature has several landmarks inside its gate, the component **branches**, once per candidate, and each child takes its parent's weight scaled by how well that candidate explained the reading:

$$w_t^{(h,j)} = w_{t-1}^{(h)} \cdot \mathcal{N}\left(z_t; \hat{z}_t^{j,(h)}, S_t^{j,(h)}\right)$$

ALGORITHM `mht_localization` ($\{(\mu, \Sigma, w)^{(h)}\}$, u_t , z_t , m) COST $O(H \cdot N \cdot n)$
 before pruning; the unpruned tree is $O(J^T)$ in the run length
in a weighted set of Gaussians, control, features, the map
out a pruned, renormalized weighted set

- 1: for each hypothesis h : predict $(\mu^{(h)}, \Sigma^{(h)})$ with Table 7.2 lines 1–2
- 2: for each feature z_t^i do
- 3: children $\leftarrow \emptyset$
- 4: for each hypothesis h and each landmark j with $d_M^2(z_t^i, \hat{z}_t^{j,(h)}) \leq \gamma$ do
- 5: child $\leftarrow$ EKF update of h with $c = j$; $w \leftarrow w^{(h)} \mathcal{N}(z_t^i; \hat{z}_t^{j,(h)}, S_t^{j,(h)})$
- 6: endfor
- 7: any hypothesis with no candidate in its gate survives with $w \leftarrow w^{(h)} \lambda_{\text{clutter}}$

```

8:   normalize; merge pairs whose means agree within  $d_{\text{merge}}$ ; drop
    $w < \rho w_{\text{max}}$ ; keep the best  $H_{\text{max}}$ 
9: endfor
10: return the surviving set, renormalized

```

Line 7 matters more than it looks. A hypothesis that can explain *nothing* must pay for that, or it lives forever at its old weight and the mixture never collapses. Charging it a clutter density is the standard device, and it is what turns "no landmark fits" from silence into evidence.

INTERACTIVE · w11.4

The Hypothesis Forest

A filter does not have to decide immediately. Deferring the decision is a real option — and its price is measured in Gaussians.

Run this simulation in the web edition: `/chapters/ch11-localization-gaussian #w11.4`

DERIVATION Weight recursion, exponential growth, and what pruning costs

$\text{pruned mass} \leq \rho H_{\text{max}} \text{ per step}$

Step 1 — the recursion is just Bayes, with a discrete variable. Treat the association history $c_{1:t}$ as a latent variable and expand the posterior:

$$p(x_t, c_{1:t} \mid z_{1:t}, u_{1:t}, m) \propto p(z_t \mid x_t, c_t, m) p(c_t) p(x_t, c_{1:t-1} \mid z_{1:t-1}, u_{1:t}, m)$$

Conditioned on a *fixed* history, the continuous part is exactly the Gaussian filter of Table 7.2. So the joint posterior is a mixture with one Gaussian per history, and the mixture weight of a history is the product of the predictive likelihoods it accumulated — the recursion in the box above. Nothing is approximated yet.

Step 2 — count the components. If each of T features has J candidates inside its gate, the exact posterior has J^T components. At $J = 2$ and one ambiguous feature per second, that is a billion Gaussians after half a minute. Exactness is not on the table.

Step 3 — bound what pruning throws away. With ratio pruning at threshold ρ (drop anything below ρw_{max}) and a cap of H_{max} , each reduction step discards at most ρ of the peak weight per pruned component, so the discarded normalized mass at that step is bounded by ρH_{max} — small, and controllable. What is *not* controllable is that the bound applies per

step and compounds: the probability that the true history is discarded at some point over a long run grows, and once discarded it is gone. No later evidence can revive a branch that no longer exists. This is the exact sense in which MHT is a stopgap: it defers the decision, it does not avoid it. Merging is the other approximation. Two components whose means agree to within d_{merge} (measured, of course, in a Mahalanobis metric) are replaced by one with the summed weight. This is a moment-matching approximation of a bimodal density by a unimodal one — exactly the operation Chapter 12 refuses to perform, which is why it can do global localization and this cannot.

MHT is where Gaussian localization runs out of road, and it is worth being blunt about why. It represents ambiguity by *enumerating* it, which works when the ambiguity is two landmarks and fails when it is "somewhere in this building". Its modern successors split the problem in two: for representation, Chapter 12's particle filters carry arbitrary multi-modality for a fixed cost; for association, Chapter 15's robust factor graphs keep the whole trajectory available so a bad match can be *undone* later rather than merely hedged against. Recent work makes the discrete variable a first-class citizen of the optimization — discrete-continuous factor graphs (Doherty et al.), or semidefinite relaxations that solve for pose and association simultaneously and certifiably (Korotkine et al.). Every one of those is a descendant of the score function in this chapter.

11.6 Implementation in Rust

The crate layout follows the chapter. `localize` owns the filter; `motion` and `sensor` are the Chapter 9 and 10 crates, imported rather than reimplemented.

```
crates/localize/src/
  lib.rs           // the Localizer trait, implemented three more times in Ch. 12
  ekf/mod.rs       // EKFLocalizer: predict, correct_known, correct
  ekf/jacobians.rs // G_t, V_t, M_t, H_t -- one function per equation
  ekf/associate.rs // Mahalanobis, chi2 gates, m_l_associate
  ekf/mht.rs       // feature "mht": branch, weight, prune, merge
```

11.6.1 The Jacobians

RUST `crates/localize/src/ekf/jacobians.rs`

```
1 use nalgebra::{Matrix2, Matrix2x3, Matrix3, Matrix3x2, Vector2};
2 use pr_core::geom::se2::SE2;
3 use motion::{Alphas, VelocityCmd};
4 use sensor::Landmark;
5
6 /// Below this |omega| the arc formulae are 0/0. A robot in a corridor commands
7 /// exactly this, so the limit is the common case, not the edge case.
8 const STRAIGHT: f64 = 1e-6;
```



```

9
10 // M_t: control noise, in *control* space. Rank 2 by construction -- which is
11 // why it needs V_t to reach pose space, and why the gamma term of Chapter 9 has
12 // to be added separately.
13 pub fn control_noise(u: &VelocityCmd, a: &Alphas) -> Matrix2<f64> {
14     let (v2, w2) = (u.v * u.v, u.omega * u.omega);
15     Matrix2::new(
16         a.0[0] * v2 + a.0[1] * w2, 0.0,
17         0.0, a.0[2] * v2 + a.0[3] * w2,
18     )
19 }
20
21 // (G_t, V_t) for the exact-arc velocity model, evaluated at mu_{t-1}.
22 pub fn motion_jacobians(mu: &SE2, u: &VelocityCmd) -> (Matrix3<f64>, Matrix3x2<f64>) {
23     let (c, s) = (mu.theta().cos(), mu.theta().sin());
24     let dt = u.dt;
25
26     if u.omega.abs() < STRAIGHT {
27         let g = Matrix3::new(
28             1.0, 0.0, -u.v * dt * s,
29             0.0, 1.0, u.v * dt * c,
30             0.0, 0.0, 1.0,
31         );
32         // The ??omega column needs the *second* order term: the first order of
33         // (sin(theta+omegaDeltat) - sin theta)/omega cancels, leaving -? v Deltat2 sin
34         // theta.
35         let v_jac = Matrix3x2::new(
36             dt * c, -0.5 * u.v * dt * dt * s,
37             dt * s, 0.5 * u.v * dt * dt * c,
38             0.0, dt,
39         );
40         return (g, v_jac);
41     }
42
43     let nt = mu.theta() + u.omega * dt;
44     let (cn, sn) = (nt.cos(), nt.sin());
45     let r = u.v / u.omega;
46     let w = u.omega;
47
48     let g = Matrix3::new(
49         1.0, 0.0, r * (-c + cn),
50         0.0, 1.0, r * (-s + sn),
51         0.0, 0.0, 1.0,
52     );
53     let v_jac = Matrix3x2::new(
54         (-s + sn) / w, u.v * (s - sn) / (w * w) + u.v * cn * dt / w,
55         (c - cn) / w, -u.v * (c - cn) / (w * w) + u.v * sn * dt / w,
56         0.0, dt,
57     );
58     (g, v_jac)
59 }
60
61 // ? and H for one landmark. Returned together because they are always used
62 // together and both need the same delta and q -- computing them apart is how the
63 // two drift out of sync during a refactor.
64 pub fn landmark_prediction(mu: &SE2, lm: &Landmark) -> (Vector2<f64>, Matrix2x3<f64>) {
65     let dx = lm.x - mu.x();
66     let dy = lm.y - mu.y();
67     let q = dx * dx + dy * dy;
68     let sq = q.sqrt();

```

```

69     let z_hat = Vector2::new(sq, wrap_pi(dy.atan2(dx) - mu.theta()));
70     let h = Matrix2x3::new(
71         -dx / sq, -dy / sq, 0.0,
72         dy / q, -dx / q, -1.0,
73     );
74     (z_hat, h)
75 }
76
77 /// Wrap to (-pi, pi]. Used on every bearing residual, without exception.
78 #[inline]
79 pub fn wrap_pi(a: f64) -> f64 {
80     let x = (a + std::f64::consts::PI).rem_euclid(std::f64::consts::TAU);
81     x - std::f64::consts::PI
82 }

```

11.6.2 Gates and association, with the dimension in the type

Association code is where measurement dimensions get mixed up: a range-bearing feature is 2-D, a range-bearing-signature feature is 3-D, and a gate built for one is silently wrong for the other. Const generics make that a compile error.

RUST crates/localize/src/ekf/associate.rs

```

1  use nalgebra::{SMatrix, SVector};
2  use stats::distribution::{ChiSquared, ContinuousCDF};
3
4  /// Everything the filter predicts about one landmark, before seeing anything.
5  pub struct Prediction<const Z: usize> {
6      pub landmark: usize,
7      pub z_hat: SVector<f64, Z>,
8      pub s: SMatrix<f64, Z, Z>,
9  }
10
11 /// A chi2 gate. The dimension is in the type, so a 2-D gate cannot be applied to
12 /// a 3-D innovation -- the mistake that produces a filter which "works but
13 /// rejects too much" and takes a week to find.
14 #[derive(Clone, Copy, Debug)]
15 pub struct Gate<const Z: usize> {
16     pub gamma: f64,
17 }
18
19 impl<const Z: usize> Gate<Z> {
20     /// gamma = chi2_{Z, p}. At Z = 2: 5.991 for p = 0.95, 9.210 for p = 0.99.
21     pub fn at_confidence(p: f64) -> Self {
22         let chi2 = ChiSquared::new(Z as f64).expect("Z >= 1");
23         Self { gamma: chi2.inverse_cdf(p) }
24     }
25
26     #[inline]
27     pub fn accepts(&self, d2: f64) -> bool {
28         d2 <= self.gamma
29     }
30 }
31
32 /// nu^T S^{-1} nu, via the Cholesky factor rather than an explicit inverse: same
33 /// answer, half the flops, and it fails loudly if S is not positive definite.

```

```

34 pub fn mahalnobis2<const Z: usize>(nu: &SVector<f64, Z>, s: &SMatrix<f64, Z, Z>) -> f64
35 {
36     let chol = s.clone().cholesky().expect("S = H Sigma H^T + Q is positive definite");
37     let y = chol.l().solve_lower_triangular(nu).expect("L is nonsingular");
38     y.norm_squared()
39 }
40
41 // ln det S, from the same factor: 2 Sigma ln L_ii.
42 pub fn log_det<const Z: usize>(s: &SMatrix<f64, Z, Z>) -> f64 {
43     let chol = s.clone().cholesky().expect("S is positive definite");
44     2.0 * chol.l().diagonal().iter().map(|d| d.ln()).sum::<f64>()
45 }
46
47 #[derive(Clone, Copy, Debug, PartialEq)]
48 pub enum Association {
49     Matched { landmark: usize, d2: f64 },
50     // Kept rather than discarded: the *closest* candidate and its distance are
51     // what you need to debug a filter that is rejecting everything.
52     Outlier { nearest: usize, d2: f64 },
53 }
54
55 // arg-min over landmarks of d2_M + ln det S, accepted only if it passes the
56 // gate. `residual` is a parameter because a bearing must be wrapped and a
57 // range must not -- the caller owns that knowledge, not this function.
58 pub fn ml_associate<const Z: usize>(
59     z: &SVector<f64, Z>,
60     preds: &[Prediction<Z>],
61     gate: &Gate<Z>,
62     residual: impl Fn(&SVector<f64, Z>, &SVector<f64, Z>) -> SVector<f64, Z>,
63 ) -> Association {
64     let mut best = f64::INFINITY;
65     let mut best_idx = usize::MAX;
66     let mut best_d2 = f64::INFINITY;
67
68     for p in preds {
69         let nu = residual(z, &p.z_hat);
70         let d2 = mahalnobis2(&nu, &p.s);
71         // The log-determinant term is *not* optional when candidates have
72         // different gate sizes. Dropping it is the most common silent bug in
73         // nearest-neighbour association.
74         let score = d2 + log_det(&p.s);
75         if score < best {
76             best = score;
77             best_idx = p.landmark;
78             best_d2 = d2;
79         }
80     }
81
82     if gate.accepts(best_d2) {
83         Association::Matched { landmark: best_idx, d2: best_d2 }
84     } else {
85         Association::Outlier { nearest: best_idx, d2: best_d2 }
86     }
87 }

```

The dimension really is enforced. Feeding a 3-D feature to a 2-D gate does not compile:

RUST a deliberate compile error

```

1 let gate = Gate::<2>::at_confidence(0.95);
2 let z3: SVector<f64, 3> = feature.with_signature();
3 let a = ml_associate(&z3, &preds3, &gate, wrap_bearing_row);

```

```

error[E0308]: mismatched types
--> src/ekf/associate.rs:98:37
   |
98 |     let a = ml_associate(&z3, &preds3, &gate, wrap_bearing_row);
   |                                ^^^^^ expected `&Gate<3>`, found `&Gate<2>`
   |                                |
   |                                arguments to this function are incorrect
   |
   = note: expected reference `&Gate<3usize>`
           found reference `&Gate<2usize>`

```

That is the Chapter 7 tradition continued: the type system knows the difference between a 2-DOF χ^2 threshold and a 3-DOF one, and it should, because a human staring at 5.99 versus 7.81 will not.

11.6.3 The localizer

RUST crates/localize/src/ekf/mod.rs

```

1 use nalgebra::{Matrix2, Matrix3, Vector2, Vector3};
2 use pr_core::geom::se2::SE2;
3 use motion::{Alphas, VelocityCmd};
4 use sensor::{Feature, Landmark};
5
6 use super::jacobians::{control_noise, landmark_prediction, motion_jacobians, wrap_pi};
7 use super::associate::{ml_associate, Association, Gate, Prediction};
8
9 /// A Gaussian over pose: mean on SE(2), covariance in (x, y, theta).
10 #[derive(Clone, Debug)]
11 pub struct GaussianBelief {
12     pub mean: SE2,
13     pub cov: Matrix3<f64>,
14 }
15
16 pub struct EkfLocalizer {
17     pub bel: GaussianBelief,
18     pub landmarks: Vec<Landmark>,
19     /// Sensor noise diag(sigma_r2, sigma_phi2).
20     pub q: Matrix2<f64>,
21     pub alphas: Alphas,
22     pub gate: Gate<2>,
23     /// The gamma floor of Chapter 9: heading noise that V M V^T structurally cannot
24     /// represent, because M has rank 2 and this direction is not in its range.
25     pub gamma_alphas: (f64, f64),
26 }
27
28 impl EkfLocalizer {
29     /// Table 7.2, lines 1-2.
30     pub fn predict(&mut self, u: &VelocityCmd) {
31         let (g, v) = motion_jacobians(&self.bel.mean, u);

```

```

32     let m = control_noise(u, &self.alphas);
33
34     let mut cov = g * self.bel.cov * g.transpose() + v * m * v.transpose();
35     let gamma_var =
36         (self.gamma_alphas.0 * u.v * u.v + self.gamma_alphas.1 * u.omega * u.omega)
37         * u.dt * u.dt;
38     cov[(2, 2)] += gamma_var;
39
40     self.bel.mean = u.integrate(&self.bel.mean); // exact arc, Chapter 9
41     self.bel.cov = symmetrize(cov);
42 }
43
44 // Table 7.2, lines 4-13, for one feature whose correspondence is given.
45 pub fn correct_known(&mut self, f: &Feature, j: usize) -> f64 {
46     let (z_hat, h) = landmark_prediction(&self.bel.mean, &self.landmarks[j]);
47     let nu = residual(&Vector2::new(f.r, f.phi), &z_hat);
48     self.apply(&nu, &h)
49 }
50
51 // Table 7.3: predict every landmark, choose by ML, gate, apply, repeat.
52 // Returns one verdict per feature so a caller can plot precision/recall.
53 pub fn correct(&mut self, features: &[Feature]) -> Vec<Association> {
54     let mut out = Vec::with_capacity(features.len());
55
56     for f in features {
57         // Re-linearised for *every* feature, so this feature's gate already
58         // reflects the information the previous one contributed. It is also
59         // why the order features arrive in stops mattering.
60         let preds = self.predictions();
61         let z = Vector2::new(f.r, f.phi);
62         let assoc = ml_associate(&z, &preds, &self.gate, residual);
63         out.push(assoc);
64
65         if let Association::Matched { landmark, .. } = assoc {
66             let (z_hat, h) = landmark_prediction(&self.bel.mean, &self.landmarks[
landmark]);
67             let nu = residual(&z, &z_hat);
68             self.apply(&nu, &h);
69         }
70     }
71     out
72 }
73
74 fn predictions(&self) -> Vec<Prediction<2>> {
75     self.landmarks
76         .iter()
77         .enumerate()
78         .map(|(k, lm)| {
79             let (z_hat, h) = landmark_prediction(&self.bel.mean, lm);
80             Prediction { landmark: k, z_hat, s: h * self.bel.cov * h.transpose() +
self.q }
81         })
82         .collect()
83 }
84
85 //  $K = \Sigma H^T S^{-1}$ ;  $\mu [+]$   $Knu$ ;  $\Sigma \leftarrow (I - KH)\Sigma$ . Returns  $d2\_M$ , the NIS of
this update.
86 fn apply(&mut self, nu: &Vector2<f64>, h: &nalgebra::Matrix2x3<f64>) -> f64 {
87     let s = h * self.bel.cov * h.transpose() + self.q;
88     let s_inv = s.try_inverse().expect("S is positive definite");
89     let k = self.bel.cov * h.transpose() * s_inv;

```

```

90         let dx: Vector3<f64> = k * nu;
91         // [+], not +=: the third component is an angle and lives on a circle.
92         self.bel.mean = self.bel.mean.boxplus(&dx);
93         self.bel.cov = symmetrize((Matrix3::identity() - k * h) * self.bel.cov);
94
95         (nu.transpose() * s_inv * nu)[(0, 0)]
96     }
97 }
98
99
100 /// z [-]?: the range component subtracts, the bearing component wraps.
101 fn residual(z: &Vector2<f64>, z_hat: &Vector2<f64>) -> Vector2<f64> {
102     Vector2::new(z[0] - z_hat[0], wrap_pi(z[1] - z_hat[1]))
103 }
104
105 #[inline]
106 fn symmetrize(m: Matrix3<f64>) -> Matrix3<f64> {
107     0.5 * (m + m.transpose())
108 }

```

The multi-hypothesis version lives behind a feature flag, because most deployments do not want it and the ones that do want it want it configurable:

RUST crates/localize/src/ekf/mht.rs (excerpt)

```

1  #[cfg(feature = "mht")]
2  pub struct MhtLocalizer {
3      pub hyps: Vec<(GaussianBelief, f64, Vec<usize>)>, // belief, weight, history
4      pub prune_ratio: f64,
5      pub max_hyps: usize,
6      pub merge_d2: f64,
7      /// The likelihood a hypothesis pays for calling a feature clutter. Without
8      /// it, a hypothesis that explains nothing survives forever at its old
9      /// weight and the mixture never collapses.
10     pub clutter_density: f64,
11 }

```

11.6.4 The visibility model, with parry2d

The simulator needs to know which landmarks a robot can actually see, which is a ray-cast against the floorplan — the one place in this chapter where a geometry crate earns its keep:

RUST crates/sim/src/visibility.rs

```

1  use parry2d::query::Ray;
2  use parry2d::shape::Polyline;
3  use nalgebra::{Point2, Vector2};
4
5  /// Is `lm` visible from `pose`: inside the cone, within range, and with no wall
6  /// between? The third test is the one people forget, and it is what makes the
7  /// Apartment a real localization problem rather than a point cloud in a void.

```

```

8 pub fn visible(walls: &Polyline, pose: &SE2, lm: &Landmark, max_range: f64, fov: f64) ->
    bool {
9     let dx = lm.x - pose.x();
10    let dy = lm.y - pose.y();
11    let range = (dx * dx + dy * dy).sqrt();
12    if range > max_range || wrap_pi(dy.atan2(dx) - pose.theta()).abs() > 0.5 * fov {
13        return false;
14    }
15    let ray = Ray::new(Point2::new(pose.x(), pose.y()), Vector2::new(dx / range, dy /
    range));
16    // A hit strictly before the landmark means a wall is in the way.
17    walls
18        .cast_local_ray(&ray, range - 1e-6, true)
19        .is_none()
20 }

```

11.6.5 The worked example, pinned by a test

Every number printed in the worked example above is reproduced by a unit test. If the prose and the code ever disagree, the test is what settles it.

RUST crates/localize/src/ekf/tests.rs

```

1 use approx::assert_relative_eq;
2
3 /// The Chapter 11 worked example:  $\mu = (2, 3, 0)$ ,  $\Sigma = \text{diag}(0.25, 0.25, 0.01)$ ,
4 /// one landmark at  $(5, 3)$ ,  $\sigma_r = 0.2$ ,  $\sigma_\phi = 0.05$ , measurement  $(3.2, 0.05)$ .
5 #[test]
6 fn worked_example_ch11_single_landmark_update() {
7     let mut loc = EkfLocalizer::at(
8         SE2::new(2.0, 3.0, 0.0),
9         Matrix3::from_diagonal(&Vector3::new(0.25, 0.25, 0.01)),
10        vec![Landmark::new(5.0, 3.0)],
11        Matrix2::from_diagonal(&Vector2::new(0.04, 0.0025)),
12    );
13
14    let (z_hat, h) = landmark_prediction(&loc.bel.mean, &loc.landmarks[0]);
15    assert_relative_eq!(z_hat[0], 3.0, epsilon = 1e-12);
16    assert_relative_eq!(z_hat[1], 0.0, epsilon = 1e-12);
17    assert_relative_eq!(h[(0, 0)], -1.0, epsilon = 1e-12);
18    assert_relative_eq!(h[(1, 1)], -1.0 / 3.0, epsilon = 1e-12);
19
20    let s = h * loc.bel.cov * h.transpose() + loc.q;
21    assert_relative_eq!(s[(0, 0)], 0.29, epsilon = 1e-12);
22    assert_relative_eq!(s[(1, 1)], 0.29 / 7.2, epsilon = 1e-12); // = 0.0402777...
23
24    let d2 = loc.correct_known(&Feature { r: 3.2, phi: 0.05, s: None }, 0);
25    assert_relative_eq!(d2, 0.2, epsilon = 1e-12); // exactly 0.2, pleasingly
26
27    assert_relative_eq!(loc.bel.mean.x(), 1.827_586_206_896_55, epsilon = 1e-9);
28    assert_relative_eq!(loc.bel.mean.y(), 2.896_551_724_137_93, epsilon = 1e-9);
29    assert_relative_eq!(loc.bel.cov[(0, 0)], 0.034_482_758_620_69, epsilon = 1e-9);
30    assert_relative_eq!(loc.bel.cov[(1, 2)], -0.020_689_655_172_41, epsilon = 1e-9);
31 }
32
33 /// The poisoning cascade of the derivation above: a decoy 0.6 m behind the
34 /// mapped landmark passes the 95% gate on every one of three sightings, while

```

```

35 // the error grows and the covariance shrinks.
36 #[test]
37 fn one_wrong_match_is_self_reinforcing() {
38     let mut loc = poisoned_setup();
39     let truth_x = 2.0;
40     let expected = [(1.2414, 0.5172, 7.759), (0.0920, 0.5556, 16.667), (0.0338, 0.5696,
41     25.633)];
42
43     for (d2_want, err_want, nees_want) in expected {
44         let d2 = loc.correct_known(&Feature { r: 3.6, phi: 0.0, s: None }, 0);
45         assert!(loc.gate.accepts(d2), "the wrong match is accepted every time");
46         assert_relative_eq!(d2, d2_want, epsilon = 1e-3);
47         assert_relative_eq!(truth_x - loc.bel.mean.x(), err_want, epsilon = 1e-3);
48         assert_relative_eq!(nees(&SE2::new(2.0, 3.0, 0.0), &loc.bel), nees_want, epsilon
49         = 1e-2);
50     }
51     // The point: d2 fell by a factor of thirty-six while the error grew by 10%.
52 }

```

Run the lab end to end and it prints the numbers the chapter has been arguing about:

```

Apartment, 400 steps, 10 landmarks, ML association, chi2_2 gate at 95%
position RMSE      0.094 m
heading  RMSE      2.26 deg
mean NEES          3.04      (consistent: 3.00)
NEES inside 95%    94% of steps (expected 95%)
features matched   413
wrong associations  0
outliers rejected  18
steps with no feature in view  42%

$ cargo run --release --example ekf_loc_lab -- --seed 11 --pathological-pair
Apartment, 400 steps, 10 landmarks (two of them 0.30 m apart), ML association
position RMSE      1.232 m
heading  RMSE      11.54 deg
mean NEES          71.83      <-- the filter is lying
NEES inside 95%    75% of steps
features matched   244
wrong associations  23
outliers rejected  55          <-- it has started refusing the truth
steps with no feature in view  65%
next: see Chapter 12. A Gaussian cannot represent "one of these two".

```

Twenty-three wrong associations out of two hundred and forty-four: a 9% error rate on a discrete decision, which multiplied the RMSE by thirteen. And note the last line of each block. The healthy run is blind 42% of the time; the poisoned one is blind 65% of the time, because it has started rejecting the very landmarks that would have saved it.

11.7 Putting it together

Three instruments, and only one of them is any good.

RMSE is what everyone reports and it answers the wrong question. It is an average, so a filter that behaves for three hundred steps and diverges over the last

hundred still reports a respectable number — and it says nothing at all about whether the covariance beside it is trustworthy.

The covariance is what the filter reports about itself, and the poisoning derivation showed precisely why it cannot be trusted: $\Sigma \leftarrow (I - KH)\Sigma$ never once looks at whether the innovation was correct. A shrinking ellipse is evidence that measurements arrived, not that they were the right ones.

NEES is the instrument that works, and it is the modernization this chapter insists on:

$$\varepsilon_t = (x_t \ominus \mu_t)^\top \Sigma_t^{-1} (x_t \ominus \mu_t) \sim \chi_3^2 \quad \text{if the filter is consistent}$$

It compares the error the filter *makes* to the error it *claims*, so it catches exactly the failure mode that RMSE and covariance both miss. Under a consistent 3-DOF filter each ε_t is marginally χ_3^2 : expectation 3, and a 95% interval of [0.216, 9.348]. The dense run above lands inside that interval on 94% of steps and averages 3.04 — as close to honest as a linearized filter gets. The poisoned run lands inside on 75% of steps and averages 71.8.

Two warnings about reading it. First, consecutive NEES values are strongly correlated — the pose error persists across steps, especially through a landmark-free stretch — so the average of a 25-step window is *not* $\chi_{75}^2/25$, and a band drawn on that assumption will be crossed far more often than 5% of the time. Judge the single-step trace against the single-step interval, and use the running mean only as a trend. Second, NEES needs ground truth, so it is a simulation instrument. Its deployable cousin is **NIS**, $v^\top S^{-1}v$, which needs only the data — and which you have already met, because NIS *is* the gate statistic. Line 9 of `EKF_localization` is a per-measurement consistency test that happens to be used for rejection.

Everything this chapter can do now fails in one specific way, and it fails on purpose. Set the lab to close pair. Watch the wrong-association counter tick, the ellipse tighten, NEES leave the planet, and then watch the rejection counter climb as the filter begins discarding true landmarks because they no longer fit the story it has told itself. There is no parameter setting that fixes this. The gate cannot be tightened (it would reject the truth first) and cannot be loosened (it would admit more phantoms). MHT can hedge for a while, at a price in Gaussians, and then it prunes and the hedge is gone.

The problem is not the gate, and it is not the tuning. It is the representation. A belief with one mean and one covariance cannot say “*I am at one of these two places*” — and that is the sentence the robot needs to be able to say. Chapter 12 gives it the vocabulary, at the cost of everything cheap and closed-form about this chapter.

NOTE

Two forward pointers worth holding onto. Chapter 14 is this chapter with

the map promoted into the state vector — the same H_t , the same gate, the same ML association, now with landmarks whose positions are also uncertain, which makes S^j larger and the gates wider exactly when you least want them to be. And a margin note the design of this chapter promised: to run a UKF localizer instead, swap `Ekf` for `Ukf` behind the `Localizer` trait and delete `jacobians.rs`. It is a one-line change because Chapter 7 did the work.

11.8 Exercises

1 FOUNDATION •• Derive H by hand, all six entries

Derive H_t^j for the range-bearing model from scratch, being careful with the sign of $\partial\delta/\partial x$. Verify each entry against a central-difference Jacobian at $\bar{\mu} = (2, 3, 0)$, $m_j = (5, 3)$.

Then answer the question the second row poses: the bearing entries decay as $1/\sqrt{q}$ while the θ entry is -1 regardless of range. What does that imply about which landmarks constrain heading and which constrain position? Design a five-landmark map for a 12 m corridor that keeps *both* well constrained, and say which of the two your map compromises.

2 FOUNDATION ••• How much clutter is too much?

Suppose false detections arrive uniformly at density λ per square metre of measurement space, and the 95% validation region of a landmark has area A_j (compute it: the ellipse $d_M^2 \leq \gamma$ has area $\pi\gamma\sqrt{\det S^j}$).

(a) Give the probability that at least one clutter detection falls inside a given gate. (b) With $n = 10$ landmarks in view, give the expected number of gates containing at least one false detection. (c) The true detection also falls in the correct gate with probability $1 - \epsilon$. Derive the clutter density at which nearest-neighbour association is wrong more often than right, and evaluate it for the widget's numbers ($\det S \approx 0.0051$, $\gamma = 5.99$).

3 FOUNDATION •• The fixed point of a poisoned filter

In the poisoning derivation, the mapped landmark is at m_j and the object generating the feature is at $m_j + b$ with b along the line of sight. Assume the robot is stationary and observes repeatedly.

Show that μ_x converges to $m_{j,x} - r$ where r is the measured range, and that $\Sigma_{xx} \rightarrow 0$ like $O(1/k)$ after k observations. Then explain, in one sentence each, why (i) NEES diverges, (ii) d_M^2 converges to a value *smaller* than a correct match would typically produce, and (iii) genuine landmarks begin to fail the gate.

4 CONCEPTUAL •• Predict the gate, then check (w11.1)

In the Association Gate, set the confidence to 0.99 and predict — before you drag anything — whether the region where Euclidean and Mahalanobis disagree grows, shrinks, or stays the same. Now predict what happens to the *outlier* verdict in that same region. Check both.

Then answer the harder version for w11.2: during a long landmark-free stretch the ellipse grows, so the gates grow. Does a *tighter* gate help or hurt there? Reconcile your two answers into one rule about when gates should be tight.

5 CONCEPTUAL •• The cheapest hedge that works (w11.4)

In the Hypothesis Forest, find the smallest $H_{\max}$ that still recovers the correct hypothesis, and the largest prune ratio. Then press *look at the beacon* at three different moments and note how the collapse differs.

Finally: what information actually breaks the symmetry, and how much did it cost to get? Frame your answer as an expected-value calculation — the reduction in belief entropy against the detour needed to obtain it. You have just written down the objective of active localization (Chapter 22).

6 PRACTICAL •• Sequential versus summed updates

The EKFLocalizer above re-linearizes after every accepted feature; Thrun's Table 7.2 sums the corrections against a single $\tilde{\Sigma}_t$. Implement both behind a flag and measure, on a seeded 400-step Apartment run with the dense preset: position RMSE, mean NEES, and association precision.

Then find a configuration where the difference is large rather than negligible. (Hint: the two agree when each correction is small. Make one correction large — a very informative landmark seen after a long blind stretch — and process several features in the same step.)

7 PRACTICAL ••• A two-feature JCBB

Implement joint compatibility for pairs. Given two features z^1, z^2 and candidate landmarks j, k , stack the innovations into a 4-vector and build the full 4×4 covariance — including the cross-block $H^j \tilde{\Sigma}_t (H^k)^T$, which per-feature gating discards — then gate against $\chi^2_{4,0.95} = 9.488$.

Show one seeded scenario in the `close pair` preset where both individual gates accept a pairing that the joint gate rejects, and explain geometrically why the cross-covariance knows something the marginals do not. Compare your rejection rate against plain per-feature gating over ten seeds.

8 PRACTICAL ••• Make the filter admit it is lost

Add a `LostDetector` to `EKFLocalizer`: track a running average of the fraction of

features rejected by the gate, and the mean NIS of the accepted ones. Trigger a Lost state when rejections exceed a threshold for several consecutive steps.

Test it by teleporting the true robot two metres mid-run. Then implement the honest response: on Lost, inflate Σ by a factor κ (which widens every gate) rather than tightening anything, and measure how large κ must be to recover, as a function of the teleport distance. At what distance does no finite κ work? That number is the boundary of this chapter.

11.9 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 7 is this chapter's baseline: the taxonomy (§7.2), Markov localization (§7.3–7.4), EKF localization with known and unknown correspondence (Tables 7.2 and 7.3), MHT (§7.7), and the practical-considerations section quoted above. [link](#)

Reid, D. B. (1979). An Algorithm for Tracking Multiple Targets *IEEE Transactions on Automatic Control* 24(6), 843–854.

The original multiple-hypothesis tracker. The branch–weight–prune recursion in w11.4 is Reid's, transplanted from target tracking into localization; the pruning and merging heuristics are still the ones in use. doi:10.1109/TAC.1979.1102177

Bar-Shalom, Y., Li, X.-R., and Kirubarajan, T. (2001). *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software* Wiley-Interscience.

The source for validation gating and for NEES/NIS as consistency tests (§5.4). Where the robotics literature reports RMSE, this book reports whether the filter's covariance is honest — the standard this chapter adopts. doi:10.1002/0471221279

Neira, J. and Tardó s, J. D. (2001). Data Association in Stochastic Mapping Using the Joint Compatibility Test *IEEE Transactions on Robotics and Automation* 17(6), 890–897.

JCBB: the paper that showed individually compatible pairings can be jointly impossible, because measurement prediction errors share a pose and are therefore correlated. Exercise 7 builds its two-feature case. doi:10.1109/70.976019

Solà, J., Deray, J., and Atchuthan, D. (2021). A micro Lie theory for state estimation in robotics *arXiv:1812.01537 (v9)*.

The reference for the $\boxplus/\boxminus$ discipline that fixes the heading-wrap and near- $\pm\pi$ bearing bugs the 2005 presentation silently carries. Its Jacobian conventions are the ones Chapter 7 and this chapter follow. [link](#)

Antonante, P., Tzoumas, V., Yang, H., and Carlone, L. (2022). **Outlier-Robust Estimation: Hardness, Minimally Tuned Algorithms, and Applications** *IEEE Transactions on Robotics* 38(1), 281–301.

Why gating alone can never be enough: outlier-robust estimation is inapproximable in the worst case. The modern answer is a robust cost that can *revise* a bad match, not a threshold that must get it right first time — the route Chapter 15 takes. doi:10.1109/TR0.2021.3094984

Doherty, K. J., Lu, Z., Singh, K., and Leonard, J. J. (2022). **Discrete-Continuous Smoothing and Mapping** *IEEE Robotics and Automation Letters* 7(4), 12395–12402.

The contemporary treatment of the variable this chapter guesses: associations are estimated jointly with the continuous state in a discrete-continuous factor graph, rather than fixed by an argmax and never revisited. doi:10.1109/LRA.2022.3216938

Korotkine, V., Cohen, M., and Forbes, J. R. (2025). **Globally Optimal Data-Association-Free Landmark-Based Localization Using Semidefinite Relaxations** *IEEE Robotics and Automation Letters* (accepted); *arXiv:2504.08547*.

Exactly this chapter’s problem — landmark localization with unknown correspondences — solved to certified global optimality by convex relaxation, with no gate and no initial guess. The clearest statement of how far the field has moved past maximum-likelihood nearest neighbour. [link](#)

Localization II: Global Localization

LOCALIZATION · Intermediate · 55 min



The kidnapped robot problem is more difficult than the global localization problem, in that the robot might believe it knows where it is while it does not.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 7

Chapter 11 ended with a confession. An extended Kalman filter tracks a pose beautifully as long as you tell it where to start, and it has no way whatsoever to say “*I am in one of these four rooms.*” A Gaussian has one mean. Ambiguity is not representable, so the moment the world is ambiguous the filter must lie.

This chapter is the payoff for the whole first half of the book. The Bayes filter of Chapter 5, the nonparametric representations of Chapter 8, the motion samplers of Chapter 9 and the sensor likelihoods of Chapter 10 assemble — with almost no new mathematics — into the two algorithms that actually solve global localization: **grid localization** and **Monte Carlo localization**. MCL in particular is three lines of glue. Everything difficult about it happens *after* the weights are computed, which is where the rest of the chapter lives: recovery from failure, proposals that are not the motion model, and a world that contains people.

🕒 AFTER THIS CHAPTER YOU CAN

- State the three localization problems — tracking, global, kidnapped — and say which representations can express each one
- Derive MCL as the particle filter with the motion model as proposal, and see exactly which factor the importance weight collapses to
- Explain why no resampling scheme can recover from kidnapping, and

why injection is therefore not a hack but the only repair

- Derive the w_{fast} / w_{slow} divergence detector and predict its behaviour on a kidnap and on a single glitched reading
- Cost a pose grid honestly — cells, bytes, and beam evaluations — and choose between grids and particles on the numbers
- Filter measurements caused by people without filtering away the surprise that tells a lost robot it is lost

 ASSUMED

- The particle filter, low-variance resampling and effective sample size (Chapter 8)
- `sample_motion_model_odometry` and the $\alpha_1 \dots \alpha_4$ noise parameters (Chapter 9)
- The beam model and the likelihood field (Chapter 10)
- EKF localization and why a Gaussian belief is a commitment (Chapter 11)

12.1 Waking up somewhere

Thrun, Burgard and Fox split localization into three problems, in increasing order of difficulty, and the split is worth memorizing because it decides which representation you are allowed to use.

position tracking	The initial pose is known and the error stays local. A unimodal belief is enough — this is Chapter 11.
global localization	The initial pose is unknown. The robot knows that it does not know. No bounded-error assumption is available.
kidnapped robot	The robot is teleported during operation. Strictly harder, because now it believes it knows where it is while it does not.

The kidnapped-robot problem sounds like a party trick, and the book is explicit that it is not: almost no localization system can be guaranteed never to fail, so the ability to *recover* is what separates a demo from something you can leave running in a building. Kidnapping is simply the cleanest way to measure that ability.

Here is all three at once. Rusty wakes up with no idea where it is, in the Apartment from Chapter 4.

 INTERACTIVE · w12.1**MCL Theater**

MCL is not a new filter — it is the particle filter with a motion sampler and a sensor likelihood, and everything hard about it lives in what happens after the weights.

Run this simulation in the web edition: `/chapters/ch12-localization-global #w12.1`

Watch one full cycle before reading on. Three things are happening, and each is a section of this chapter.

The belief is a bag of poses, and it starts as fog. Eight thousand particles or two hundred, the object on screen is a sampled representation of $\text{bel}(x_t)$ over the whole free space. Nothing about it is Gaussian, nothing about it is unimodal, and no covariance ellipse could be drawn around it without lying.

Ambiguity survives about as long as the evidence is ambiguous — and not one step longer. Press *Twin rooms*: rooms A and C of the Apartment are exact mirror images about $x = 6$ and near-exact translates of one another (their doorways differ by 10 cm), and the widget seeds exactly mirrored pairs of particles so the initial belief is genuinely symmetric. Two clusters then coexist for a dozen steps before one wins. Pull the tempering slider back to $\kappa = 1$ and repeat: the same evidence now decides in about three steps. How long a filter is *entitled* to stay ambiguous is a property of the map; how long it actually does is a property of your sensor sharpness and your M .

Recovery is not automatic. Press *Kidnap* with plain MCL selected and wait as long as you like. The filter almost never comes back — measured at the end of this chapter, once in twelve seeded attempts. Switch to Augmented MCL, kidnap again, and watch blue particles rain into the free space for a few steps until the cloud finds the robot. That difference is one extra statistic and four extra lines of code, and it is the difference between a localizer and a liability.

 THE ONE SENTENCE VERSION

Global localization needs a belief that can be multi-modal; MCL gets one for free by representing $\text{bel}(x_t)$ with samples, and pays for it with a failure mode — sampled beliefs cannot represent what they have no particles for, so a lost filter must be given new hypotheses from outside the recursion.

12.2 The mathematics

12.2.1 Notation this chapter adds

$\{\mathbf{x}_k\}, \{p_{k,t}\}$	Pose-space grid cells and their probabilities. A cell is a box in (x, y, θ) , not in (x, y) .
$\mathcal{X}_t = \{x_t^{[i]}, w_t^{[i]}\}_{i=1}^M$	The weighted particle set — Chapter 8's object, now over SE(2).
w_{avg}	Mean unnormalized importance weight at time t ; the empirical estimate of the evidence $p(z_t z_{1:t-1}, u_{1:t}, m)$.
w_{fast}, w_{slow}	Exponentially smoothed w_{avg} at two timescales, with gains $\alpha_{fast} \gg \alpha_{slow}$.
ϕ	Mixing rate: the fraction of particles proposed from the measurement rather than the motion model.
χ_{rej}	Rejection threshold on $p(\text{short} z_t^k)$, the posterior probability that a beam was caused by an unmodelled object.
κ	Tempering exponent: the likelihood is used as $p(z x)^{1/\kappa}$. $\kappa > 1$ softens an overconfident sensor model.

12.2.2 Grid localization: brute force, and why it still works

The most direct way to represent "anywhere" is to chop the pose space into cells and keep a number for each one. That is the discrete Bayes filter of Chapter 8, applied to SE(2) instead of a corridor, and it is Thrun's **Table 8.1**:

$$\bar{p}_{k,t} = \sum_j p(\mathbf{x}_k | u_t, \mathbf{x}_j) p_{j,t-1} \quad p_{k,t} = \eta p(z_t | \mathbf{x}_k, m) \bar{p}_{k,t}$$

Both models are evaluated at the cell's centre of mass, $\text{mean}(\mathbf{x}_k)$. The typical metric resolution in the literature is 15 cm in x and y and 5° in θ , and it is worth doing that arithmetic once, because it explains the entire rest of this chapter.

NOTE

The Apartment is 12 m $\times$ 9 m. At 15 cm that is $80 \times 60 = 4,800$ planar cells; at 5° the heading axis adds 72 bins, so the grid holds **345,600** pose cells — 1.4 MB at f32. A 5 000-particle MCL belief is $5,000 \times 32$ bytes = **156 kB**, about nine times smaller. The gap in *time* is worse: correcting the grid with an 8-beam subsampled scan costs 2.8 million likelihood evaluations, against 40 000 for MCL. Same recursion. Seventy times the bill.

Written literally, line 3 is worse still: it is a sum over all cells *for* all cells, which at 345 600 cells is 1.2×10^{11} kernel evaluations per step. Nobody implements it that way. The motion kernel has effectively compact support, so you scatter each cell's mass into the neighbourhood of its predicted pose and skip cells whose probability is negligible — Thrun's *selective updating*, which he notes can save "many orders of magnitude". That is exactly what `lib/localize/grid-localization.ts` does, and what the Rust `GridLocalizer::predict` below does.

ALGORITHM `Grid_localization({p_{k,t-1}}, u_t, z_t, m)` COST $O(|G| \cdot g_u)$
 predict with a bounded kernel of support g_u ; $O(|G| \cdot K)$ correct for K beams -- Table 8.1

in the previous cell probabilities, the control, the measurement, the map
out $\{p_{k,t}\}$

```

1: for all  $k$  do
2:    $\bar{p}_{k,t} = \sum_i p_{i,t-1} \text{motion\_model}(\text{mean}(\mathbf{x}_k), u_t, \text{mean}(\mathbf{x}_i))$ 
3:    $p_{k,t} = \eta \text{measurement\_model}(z_t, \text{mean}(\mathbf{x}_k), m) \bar{p}_{k,t}$ 
4: endfor
5: return  $\{p_{k,t}\}$ 

```

The coarsening correction

There is a trap in "evaluate the model at the cell centre", and it bites in both directions.

On the sensor side, a laser's likelihood varies enormously across a 40 cm cell: the centre of the cell containing the true pose may be 20 cm off, and with $\sigma_{hit} = 18$ cm that costs the true cell a factor of $e^{-0.6} \approx 0.55$ per beam — compounded over twenty beams, the correct cell is crushed. On the motion side the failure is funnier: with 40 cm cells and 10 cm of motion per update, predicting from centre to centre never leaves the cell, so the belief simply refuses to move while the robot drives away.

DERIVATION Why coarse cells need inflated noise, and by how much

$$\sigma_e f^2 = \sigma^2 + (d/2)^2$$

Step 1 — what the exact update wants. The probability of a cell is an integral, not a point evaluation:

$$p_{k,t} = \eta \int_{\mathbf{x}_k} p(z_t | x, m) \bar{p}(x) dx$$

Table 8.1 approximates this with $|\mathbf{x}_k| \cdot p(z_t | \text{mean}(\mathbf{x}_k), m)$, i.e. one sample of the integrand at the centre.

Step 2 — the approximation is a convolution in disguise. Assume $\bar{p}$ is roughly flat inside the cell (true when cells are small relative to the belief). Then the integral is the measurement model *averaged* over the cell, which is exactly the model convolved with the cell's own indicator function, evaluated at the centre:

$$\frac{1}{|\mathbf{x}_k|} \int_{\mathbf{x}_k} p(z | x, m) dx = (p(z | \cdot, m) * U_{\mathbf{x}_k})(\text{mean}(\mathbf{x}_k))$$

Step 3 — convolving a Gaussian with a box. The uniform density on a cell of diameter d has variance $d^2/12$ per axis; a box is not a Gaussian, but matching second moments is the standard cheap approximation, and it gives

$$\sigma_{\text{eff}}^2 = \sigma^2 + \frac{d^2}{12} \quad (\text{box-matched}), \quad \sigma_{\text{eff}}^2 = \sigma^2 + \left(\frac{d}{2}\right)^2 \quad (\text{the conservative form we use})$$

Thrun's phrasing — "the variance of a range finder model's main Gaussian cone may be enlarged by half the diameter of the grid cell" — is the second one. Both have the property that actually matters: the correction vanishes as $d \rightarrow 0$, so a fine grid pays nothing for it.

What it costs. The book is blunt about the price: the smoothed model "reduces the information intake, thereby reducing the localization accuracy". You are trading the *right* answer for a *stable* one. That trade is visible in the widget below as the floor under the coarse grid's error. ■

INTERACTIVE · w12.2

Grid vs. Cloud

Grids and particles run the same recursion, but a grid pays for the whole map at every step while a particle set pays only for the hypotheses it is still entertaining.

Run this simulation in the web edition: [#w12.2](/chapters/ch12-localization-global)

The measured numbers in that widget are worth staring at, because they are the whole argument for particles. Running the same logged route in this browser, the grid's steady-state error is set by its cell size rather than by its sensor — about 0.8 m with 0.8 m cells, about 0.16 m with 0.3 m cells, roughly half a cell once the cells are small enough to resolve the corridor — while its cost per update rises from roughly 3 ms at 1 440 cells to roughly 7 ms at 28 800 cells, paid identically whether the robot is completely lost or perfectly tracked. MCL's error is not quantized at all, and its cost is whatever you chose M to be.

Exhaustive search over a pose grid has not disappeared from robotics: matching a scan against a map at successively finer resolutions is still a standard way to *bootstrap* a global pose, and Chapter 16 builds one. What has disappeared is the fine metric pose grid as a *running belief*, updated every tick whether or not anything is uncertain — and the arithmetic above is why.

12.2.3 Monte Carlo localization is the particle filter, instantiated

Here is the entire algorithm, and it should feel anticlimactic.

ALGORITHM `MCL(X_{t-1}, u_t, z_t, m)` COST $O(M \cdot K)$ for M particles and K beams --

Table 8.2

in the previous particle set, the control, the measurement, the map

out X_t

```

1:  $\bar{X}_t = X_t = \emptyset$ 
2: for  $i = 1$  to  $M$  do
3:    $x_t^{[i]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[i]})$ 
4:    $w_t^{[i]} = \text{measurement\_model}(z_t, x_t^{[i]}, m)$ 
5:    $\bar{X}_t = \bar{X}_t + \langle x_t^{[i]}, w_t^{[i]} \rangle$ 
6: endfor
7: for  $i = 1$  to  $M$  do
8:   draw  $j$  with probability  $\propto w_t^{[j]}$ 
9:    $X_t = X_t + x_t^{[j]}$ 
10: endfor
11: return  $X_t$ 

```

Line 3 is Chapter 9's `sample_motion_model_odometry`. Line 4 is Chapter 10's beam model or likelihood field. Lines 7–10 are Chapter 8's low-variance sampler. There is no line that is new. MCL is the particle filter with two arguments filled in — which is why global localization, the thing an EKF cannot even express, arrives essentially for free.

The one step that deserves care is why the weight is *just* the likelihood.

 **DERIVATION** MCL from importance sampling: why $w = p(z \mid x, m)$

$$w_t^{[i]} \propto p(z_t \mid x_t^{[i]}, m)$$

Importance sampling says: to approximate a target f using samples from a proposal g , weight each sample by $f(x)/g(x)$. Everything below is bookkeeping about which is which.

Step 1 — the proposal. Line 3 draws $x_t^{[i]} \sim p(x_t \mid u_t, x_{t-1}^{[i]})$ from a particle $x_{t-1}^{[i]}$ that is already distributed as $\text{bel}(x_{t-1})$. The marginal distribution of the result is therefore

$$g(x_t) = \int p(x_t | u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1} = \overline{\text{bel}}(x_t)$$

which is precisely line 2 of the Bayes filter. The prediction step *is* the proposal.

Step 2 — the target. We want the posterior, which is line 3 of the Bayes filter:

$$f(x_t) = \text{bel}(x_t) = \eta p(z_t | x_t, m) \overline{\text{bel}}(x_t)$$

Step 3 — divide. The predicted belief appears in both, and cancels:

$$w_t^{[i]} = \frac{f(x_t^{[i]})}{g(x_t^{[i]})} = \frac{\eta p(z_t | x_t^{[i]}, m) \overline{\text{bel}}(x_t^{[i]})}{\overline{\text{bel}}(x_t^{[i]})} = \eta p(z_t | x_t^{[i]}, m)$$

The normalizer η is common to all particles and dies in the normalization, so the weight is the raw measurement likelihood. This single cancellation is the reason line 4 is one function call.

Step 4 — resampling. Drawing M times with probability proportional to w converts the weighted sample set into an unweighted one distributed as $\text{bel}(x_t)$, so the next iteration's Step 1 assumption holds again and the induction closes. As $M \rightarrow \infty$ the approximation is exact.

The condition that will break. Importance sampling requires $g > 0$ wherever $f > 0$: the proposal must be able to *reach* every state the posterior likes. Two failures follow immediately, and they are the next two sections. If the sensor is very sharp, f is concentrated where g has almost no samples — a *good* sensor starves the filter. If the robot is teleported, g has no mass at the true pose at all, and no weighting can create some. ■

The too-good-sensor failure, and the cheap fix

Thrun states the pathology in its most provocative form: if you acquired a *perfect* sensor that always reported the true pose without noise, MCL would fail. The posterior would be a delta function, no particle drawn from the motion model would land on it, and every weight would be zero. "Under certain circumstances, a less accurate sensor would be preferable to a more accurate sensor when using MCL."

The cheap fix is to lie about the sensor in a controlled way — inflate its noise, or equivalently raise the likelihood to a power:

$$\tilde{p}(z_t | x_t, m) = p(z_t | x_t, m)^{1/\kappa}, \quad \kappa \geq 1$$

This is the **tempering** slider in the Theater, and κ is doing something specific: it is not modelling the sensor, it is modelling *the particle filter's own approximation error*. Turn κ up with a small M and watch the effective sample size stop collapsing. Turn it up too far and the filter stops learning from the scan at all. Chapter 10 derives the same knob from the other direction, as the cure for beams that are not independent.

The principled fix is to change the proposal, which we come back to after recovery.

12.2.4 Why plain MCL cannot recover from kidnapping

DERIVATION Resampling cannot manufacture hypotheses

$P(\text{recovery from reweighting alone}) \approx 0$

Step 1 — after convergence, the support is tiny. A converged MCL cloud occupies a region roughly the size of the posterior's standard deviation: for the Theater's likelihood field, a few centimetres. Call the set of poses with at least one particle S_t . Every operation the filter performs maps S_t into a neighbourhood of itself: resampling *selects* from S_t , and the motion model *diffuses* it by the per-step noise, which is centimetres.

Step 2 — kidnapping moves the truth outside S_t . After a teleport of 6 m, the true pose is far outside S_t . Every particle now has a likelihood that is small, but *equally* small — the weights carry no information about which direction to move, because none of the particles is anywhere near right. Worse, normalization erases the absolute scale, so from the *normalized* weights the filter cannot even tell that anything went wrong — which is exactly why the next section reaches for the unnormalized average instead.

Step 3 — diffusion is hopeless. For a particle to reach the true pose by motion noise alone it must random-walk 6 m with per-step steps of order $\sigma \approx 5$ cm. A *single* step of that size has probability $\propto \exp(-\frac{1}{2}(6/0.05)^2) = e^{-7200}$, and the diffusive route needs $(6/0.05)^2 \approx 14,400$ steps of unopposed random walk — during which resampling, which kills any particle that fails to explain the scan, would have eliminated every wanderer many times over. Resampling actively works *against* recovery: it is a contraction on the support.

Conclusion. Recovery cannot come from reweighting or resampling. It must come from *injecting* poses drawn from something other than the previous belief. Everything that follows is about deciding when to inject and where from. ■

The related failure worth knowing. The same argument in miniature explains premature convergence. With M particles split between two indistinguishable modes, the mode counts perform a random walk with absorbing barriers under resampling, and the expected time to absorption grows only like M — so a small filter *will* eventually commit to one room on no evidence at all. In the twin-rooms scenario the effect is far more violent than that argument suggests, because a sharp likelihood does not wait for the random walk: the mode that happens to contain a slightly better-placed particle takes almost the entire weight on the *first* update. Tempering (raising κ) is the direct antidote, and the Theater lets you watch the ambiguity's lifetime stretch as you turn it up.

12.2.5 Augmented MCL: a filter that notices its own surprise

The detector needs a statistic that says "things are going worse than usual" without a map-specific threshold. Thrun's answer uses a quantity the filter already computes and normally throws away: the mean unnormalized weight,

$$w_{avg} = \frac{1}{M} \sum_{i=1}^M w_t^{[i]} \approx p(z_t \mid z_{1:t-1}, u_{1:t}, m)$$

which is the Monte Carlo estimate of the **evidence** — the same η^{-1} that Chapter 5 flagged as a diagnostic and Chapter 11 used for outlier rejection. Smooth it at two rates and compare:

$$w_{fast} \leftarrow w_{fast} + \alpha_{fast} (w_{avg} - w_{fast}), \quad w_{slow} \leftarrow w_{slow} + \alpha_{slow} (w_{avg} - w_{slow}), \quad 0 \leq \alpha_{slow} \ll \alpha_{fast}$$

$$p_{inject} = \max \left\{ 0, 1 - \frac{w_{fast}}{w_{slow}} \right\}$$

DERIVATION Why a ratio of two timescales is a calibration-free divergence test

$$p_{inject} = \max(0, 1 - w_{fast}/w_{slow})$$

Step 1 — what w_{avg} measures. It is the average probability the current belief assigned to the reading that actually arrived. A well-localized filter in a static world produces a steady stream of similar values; the absolute level depends on the map, the beam count and the sensor model, and is meaningless on its own.

Step 2 — absolute levels are useless, ratios are not. Multiply the likelihood by any constant c (change the beam count, change σ_{hit} , change map units)

and both averages scale by c , so w_{fast}/w_{slow} is unchanged. The statistic is *self-calibrating*: this is why the same α 's work in a corridor and a warehouse, and why no per-map threshold appears anywhere.

Step 3 — two timescales turn "abruptly" into a number. w_{slow} is the filter's memory of how well it usually does; w_{fast} is how it is doing now. A kidnap drops w_{avg} in one step, so w_{fast} follows immediately while w_{slow} barely moves, and the ratio dives. A single noisy reading also drops w_{avg} — but for one step only, after which w_{fast} climbs back, and the integrated injection is small. Sensitivity is set by the *separation* of the gains, which is why the algorithm requires $\alpha_{slow} \ll \alpha_{fast}$ and why setting them equal gives a detector that is identically blind ($w_{fast} \equiv w_{slow} \Rightarrow p_{inject} \equiv 0$).

Step 4 — the injection is self-extinguishing. Suppose the likelihood level drops permanently by a factor $(1 - \delta)$ at $t = 0$. Then

$$w_{fast}(n) = 1 - \delta(1 - (1 - \alpha_{fast})^n), \quad w_{slow}(n) = 1 - \delta(1 - (1 - \alpha_{slow})^n)$$

so $p_{inject}(n) = \delta [(1 - \alpha_{slow})^n - (1 - \alpha_{fast})^n] / w_{slow}(n)$, which rises to a peak and then decays to **zero** as both averages converge on the new level. The detector fires on *change*, not on *level* — exactly the right semantics, and the reason it needs no off switch. ■

ALGORITHM Augmented_MCL(X_{t-1} , u_t , z_t , m)

COST $O(M \cdot K)$ plus $O(M)$

bookkeeping -- Table 8.3

in the previous particle set, control, measurement, map; static w_{slow} , w_{fast}

out X_t

```

1: static  $w_{slow}$ ,  $w_{fast}$ 
2:  $\bar{X}_t = X_t = \emptyset$ 
3: for  $i = 1$  to  $M$  do
4:    $x_t^{[i]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[i]})$ 
5:    $w_t^{[i]} = \text{measurement\_model}(z_t, x_t^{[i]}, m)$ 
6:    $\bar{X}_t = \bar{X}_t + \langle x_t^{[i]}, w_t^{[i]} \rangle$ 
7:    $w_{avg} = w_{avg} + \frac{1}{M} w_t^{[i]}$ 
8: endfor
9:  $w_{slow} = w_{slow} + \alpha_{slow} (w_{avg} - w_{slow})$ 
10:  $w_{fast} = w_{fast} + \alpha_{fast} (w_{avg} - w_{fast})$ 
11: for  $i = 1$  to  $M$  do
12:   with probability  $\max\{0, 1 - w_{fast}/w_{slow}\}$  do
```

```

13:     add a random pose to  $\mathcal{X}_t$ 
14:   else
15:     draw  $j$  with probability  $\propto w_t^{[j]}$  and add  $x_t^{[j]}$  to  $\mathcal{X}_t$ 
16:   endwhile
17: endfor
18: return  $\mathcal{X}_t$ 

```

INTERACTIVE · w12.3

Recovery Ward

The kidnapping detector needs no map-specific threshold: it is a ratio of the same statistic measured at two speeds, and it switches itself off once the filter recovers.

Run this simulation in the web edition: `/chapters/ch12-localization-global #w12.3`

Two engineering notes that the pseudocode hides and every implementation hits.

w_{avg} **must be computed from unnormalized weights**. Normalized weights always average to $1/M$, so a filter that normalizes before smoothing has built a detector that can never fire. In our TypeScript port `ParticleFilter.correct` normalizes, so the widgets recompute the raw mean explicitly; the Rust below keeps the raw sum from the weighting loop.

The raw product is unusable as a level. The likelihood of a scan is a product over beams, so a 200-beam sweep with per-beam likelihoods of order 10^{-2} produces 10^{-400} — not a double at all — and even at 20 beams the magnitude swings over dozens of orders of magnitude as you change the beam count, the map or σ_{hit} . The *ratio* survives all of that; the level does not. Both implementations use the per-beam geometric mean $\exp(\log q/K)$, a monotone rescaling that puts w_{avg} on a per-beam scale of order 0.1–1 and makes the two gains portable across maps and beam counts. Production AMCL reaches the same place by another route: its likelihood-field score is a *sum* of per-beam terms rather than a product.

A worked example you can check by hand

Take $\alpha_{fast} = 0.5$, $\alpha_{slow} = 0.05$, and a filter that has been tracking happily long enough that $w_{fast} = w_{slow} = 1.0$. At $t = 1$ the robot is kidnapped and the average likelihood collapses to $w_{avg} = 0.2$, where it stays.

The first step is two lines of arithmetic:

$$w_{fast} = 1 + 0.5(0.2 - 1) = 0.6, \quad w_{slow} = 1 + 0.05(0.2 - 1) = 0.96, \quad p_{inject} = 1 - \frac{0.6}{0.96} = 0.375$$

Continue, and put the single-glitch case — $w_{avg} = 0.2$ for one step only, then back to 1.0 — in the next columns:

t	w_{fast}	w_{slow}	p_{inject}	injected at $M = 1000$	glitch: p_{inject}
1	0.6	0.96	0.3750	375	0.3750
2	0.4	0.922	0.5662	566	0.1684
3	0.3	0.8859	0.6614	661	0.0663
4	0.25	0.85161	0.7064	706	0.0163
5	0.225	0.81903	0.7253	725	0.0000

Read the two right-hand columns against each other, because that comparison is the entire design. A genuine kidnap has the filter replacing two thirds of its particles per step and still climbing; a single bad reading costs a total of 0.63 M particles spread over four steps and then stops by itself, with no threshold, no timer and no state machine. The difference between "the world changed" and "a beam bounced off a chrome table leg" falls out of two exponential filters with different gains.

12.2.6 Mixture proposals: when the sensor should propose

Recovery by *uniform* injection is blunt. Most injected particles land somewhere the current scan flatly rules out and die at the next weighting, so you pay $M \cdot p_{inject}$ likelihood evaluations to keep a handful. The principled alternative is to let a fraction ϕ of the particles be proposed by the measurement itself and reverse the roles in the importance ratio:

$$x_t^{[i]} \sim \eta p(z_t | x_t, m), \quad w_t^{[i]} = \int p(x_t^{[i]} | u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$$

DERIVATION Mixture-MCL weights: swap the roles, swap the weight

$w \propto \overline{\text{bel}}(x)$, the motion side

Step 1 — the identity is symmetric. Importance sampling only requires weight = target/proposal. With proposal $g(x) = \eta p(z_t | x, m)$ and the same target $f(x) = \eta' p(z_t | x, m) \overline{\text{bel}}(x)$, the likelihood cancels instead of the prediction:

$$w = \frac{f}{g} \propto \overline{\text{bel}}(x) = \int p(x | u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$$

So a measurement-proposed particle is weighted by *how plausible the motion history makes it*, which is the mirror image of ordinary MCL.

Step 2 — both halves are hard, honestly. Sampling from $p(z | x, m)$ means inverting the sensor: easy for landmarks (Chapter 10's `sample_pose` inverts a range-bearing reading in closed form), and genuinely hard for a laser

scan — "imagine sampling from the space of all poses that fit a given laser range scan". Evaluating the weight needs bel as a *density*, but we only have samples of it, so it must be estimated — typically by convolving the previous particle set with a narrow Gaussian kernel and evaluating that, at $O(M \log M)$ with a k-d tree.

Step 3 — mix, do not replace. Proposing everything from the measurement throws away the motion history entirely. Thrun's recipe is a mixture: draw a fraction ϕ (5% is a typical value) from the measurement stream and $1 - \phi$ from the ordinary one. The result "yields superior results, but it can be challenging to implement", and it is a *sound* solution to kidnapping rather than a heuristic one, because it constantly seeds particles wherever the current scan says the robot could be, independent of history. ■

The practical descendant of this idea is what every deployed system does: when the sensor *can* be inverted, inject from the measurement instead of from the uniform. Scan-matching a global map to get candidate poses, or a learned place-recognition front end, is mixture-MCL's proposal step wearing 2026 clothes — Chapter 16 builds the scan matcher, and Chapter 25 builds the learned version.

12.2.7 The world moves: localization among people

Every model so far assumed a static world. Real buildings contain people, and a person standing between the laser and a wall produces a reading the map cannot explain. Nothing in MCL treats this as anything other than evidence that the robot is somewhere else, and with twenty beams the effect compounds fast.

The four-way beam mixture of Chapter 10 already *has* a component for this — z_{short} , the exponential ramp in front of the expected range — but having a component that explains people is not the same as being immune to them: the mixture still assigns the reading to the pose that best explains a corridor full of unexpected walls. So we compute, per beam, the posterior probability that the reading was caused by an unmodelled object, and drop the beams where it is large:

$$p(\bar{c}_t^k = \text{short} \mid z_t^k, z_{1:t-1}, u_{1:t}, m) \approx \frac{\sum_i z_{\text{short}} p_{\text{short}}(z_t^k \mid x_t^{[i]}, m)}{\sum_i p(z_t^k \mid x_t^{[i]}, m)} > \chi_{\text{rej}} \Rightarrow \text{reject}$$

The sum runs over particles because the integral over $\text{bel}(x_t)$ has no closed form; Thrun's $\bar{X}_t$ is a *representative* sample, and forty-eight poses estimate a ratio of two sums perfectly well.

ALGORITHM `test_range_measurement(z_t^k,  $\bar{X}_t$ , m)` COST $O(|\bar{X}|)$ ray casts per beam -- Table 8.4

in one beam, a representative sample of the predicted belief, the map

out accept or reject

```

1:  $p = q = 0$ 
2: for  $i = 1$  to  $|\bar{X}_t|$  do
3:    $p = p + z_{short} \cdot p_{short}(z_t^k \mid x_t^{[i]}, m)$ 
4:    $q = q + z_{hit}p_{hit} + z_{short}p_{short} + z_{max}p_{max} + z_{rand}p_{rand}$ 
5: endfor
6: if  $p/q > \chi_{rej}$  then return reject else return accept
```

⚠ WHERE ROBOTS BREAK THIS

An honest note about Table 8.4. The printed pseudocode accumulates p as the *hit* mass and then returns **accept** when $p/q \leq \chi$ — which rejects precisely the beams the map explains best, the opposite of the surrounding text ("the measurement is then rejected if its probability of being caused by an unexpected obstacle exceeds a user-selected threshold χ ") and of the figures. We implement the prose, as written above, and say so rather than quietly picking one.

The asymmetry is the design, not an accident. A surprisingly **short** reading has p_{short} mass and can be rejected; a surprisingly **long** one has none, so it always survives. That matters enormously: long readings are how a delocalized filter discovers that it is lost, and a symmetric outlier rejector would throw away exactly the evidence that drives recovery. People-filtering and kidnap-recovery are designed to coexist.

🖥 INTERACTIVE · w12.5

Crowd Mode

A good sensor model does not handle people: the four-way mixture explains a short reading, it does not stop that reading from moving the belief. Rejecting the short-dominated beams does.

Run this simulation in the web edition: `/chapters/ch12-localization-global #w12.5`

12.3 Implementation in Rust

Three types, all of them implementing Chapter 11’s Localizer trait, so the benchmark harness and the widget’s algorithm switch can iterate over them.

RUST `crates/localize/src/mcl/mod.rs`

```

1  use nalgebra::Matrix3;
2  use rand::rngs::SmallRng;
3  use rand::SeedableRng;
4
5  use motion::{OdomDelta, OdometryModel}; // Ch. 9
6  use particles::{low_variance_resample, ParticleSet}; // Ch. 8
7  use pr_core::geom::SE2; // Ch. 3
8  use sensor::SensorModel; // Ch. 10
9  use sim::{Scan, World}; // Ch. 4
10
11 // Monte Carlo localization -- Thrun et al., **Table 8.2**.
12 ///
13 /// Generic over the sensor model because that is the only genuine choice here:
14 /// swapping `BeamModel` for `LikelihoodField` changes the cost per particle by
15 /// an order of magnitude and nothing else in this file.
16 pub struct Mcl<S: SensorModel> {
17     pub particles: ParticleSet<SE2>,
18     pub motion: OdometryModel,
19     pub sensor: S,
20     /// Seeded, never `thread_rng`: a replayable run is what makes the widget's
21     /// scrubber and the benchmark table possible at all.
22     pub rng: SmallRng,
23 }
24
25 impl<S: SensorModel> Mcl<S> {
26     /// Global localization:  $x_0^i \sim \text{Uniform}(\text{free}(m))$ .
27     pub fn global_init(
28         map: &World,
29         m: usize,
30         motion: OdometryModel,
31         sensor: S,
32         seed: u64,
33     ) -> Self {
34         let mut rng = SmallRng::seed_from_u64(seed);
35         let particles = ParticleSet::from_fn(m, |_| map.sample_free_pose(&mut rng, 0.2));
36         Self { particles, motion, sensor, rng }
37     }
38
39     /// Lines 3-6: propagate every particle and weight it. Returns `w_avg`, the
40     /// mean **unnormalized** weight -- the empirical evidence
41     ///  $p(z_{1:t} | z_{1:t-1}, u_{1:t}, m)$ , which `AugmentedMcl` needs and which
42     /// normalizing would destroy.
43     ///
44     /// Split out from `step` so the augmented variant can insert its own
45     /// resampling rule without duplicating a line of this.
46     pub fn propagate_and_weight(&mut self, u: &OdomDelta, z: &Scan, map: &World) -> f64 {
47         // Destructure to split the borrow: the sampler needs `&mut rng` while we
48         // are iterating the particle states mutably, and `self.motion.sample(...,
49         // &mut self.rng)` inside `self.particles.iter_mut()` does not compile.
50         let Self { particles, motion, sensor, rng } = self;
51
52         let n_beams = z.used_beams();
53         let mut w_avg = 0.0;

```

```

54     for (x, w) in particles.iter_mut() {
55         *x = motion.sample(u, x, rng);
56         // Log space, always: a product over 60 beams leaves f64 behind.
57         let log_q = sensor.log_likelihood(z, x, map);
58         // Per-beam geometric mean keeps w_avg 0(0.1..1) regardless of the
59         // beam count, so alpha_fast/alpha_slow are portable across maps.
60         *w = (log_q / n_beams as f64).exp();
61         w_avg += *w;
62     }
63     w_avg / particles.len() as f64
64 }
65
66 /// The full Table 8.2 recursion.
67 pub fn step(&mut self, u: &OdometryDelta, z: &Scan, map: &World) -> f64 {
68     let w_avg = self.propagate_and_weight(u, z, map);
69     // Lines 7-10 -- Chapter 8's comb, not M independent draws.
70     low_variance_resample(&mut self.particles, &mut self.rng);
71     w_avg
72 }
73
74 /// The pose to publish, and the mass behind it.
75 ///
76 /// Deliberately *not* the mean of the whole set: the mean of a bimodal
77 /// belief sits in the wall between two rooms, a pose the filter assigns
78 /// essentially zero probability. Returning the cluster mass alongside lets
79 /// the caller say "62% of my belief is here" instead of pretending.
80 pub fn estimate(&self) -> (SE2, Matrix3<f64>, f64) {
81     self.particles.dominant_cluster(0.7)
82 }
83 }

```

The augmented variant wraps it. Note that `Injector` is an enum rather than a `bool`: uniform injection is the textbook default, and measurement-driven injection is what production systems actually do — the difference is median recovery time, and it is measurable.

RUST `crates/localize/src/mcl/augmented.rs`

```

1 use rand::Rng as _;
2
3 use super::Mcl;
4
5 /// Where a recovery particle comes from.
6 pub enum Injector {
7     /// Uniform over free space -- Table 8.3 as written.
8     UniformFree,
9     /// Draw from the measurement model (Ch. 10 `sample_pose`, or the likelihood
10    /// field's normalized field). Strictly better when the sensor is invertible:
11    /// injected particles land somewhere the current scan actually allows.
12    FromSensor,
13 }
14
15 /// Augmented MCL -- Thrun et al., **Table 8.3**.
16 pub struct AugmentedMcl<S: SensorModel> {
17     pub mcl: Mcl<S>,
18     pub w_fast: f64,
19     pub w_slow: f64,

```

```

26     /// Decades apart, per the algorithm's requirement  $0 \leq \alpha_{\text{slow}} \leq \alpha_{\text{fast}}$ .
27     pub a_fast: f64,
28     pub a_slow: f64,
29     pub injector: Injector,
30     seeded: bool,
31 }
32
33 impl<S: SensorModel> AugmentedMcl<S> {
34     /// Table 8.3. Returns how many particles were injected this step -- the
35     /// number the Theater draws as blue rain.
36     pub fn step(&mut self, u: &OdomDelta, z: &Scan, map: &World) -> usize {
37         // Lines 3-8: identical to MCL, which is why it is a call and not a copy.
38         let w_avg = self.mcl.propagate_and_weight(u, z, map);
39
40         // Seed both averages on the first call. Starting from zero would report
41         // p_inject = 0 on step one and a spurious spike on step two -- an
42         // artefact of initialization, not a property of the data.
43         if !self.seeded {
44             self.w_fast = w_avg;
45             self.w_slow = w_avg;
46             self.seeded = true;
47         } else {
48             self.w_fast += self.a_fast * (w_avg - self.w_fast);
49             self.w_slow += self.a_slow * (w_avg - self.w_slow);
50         }
51
52         let p_inject = if self.w_slow > 0.0 {
53             (1.0 - self.w_fast / self.w_slow).max(0.0)
54         } else {
55             0.0
56         };
57
58         let m = self.mcl.particles.len();
59         let injected = (0..m).filter(|_| self.mcl.rng.random:<f64>() < p_inject).count();
60
61         // Table 8.3 draws each survivor independently; we hand the survivors to
62         // the low-variance comb in one batch. Same target distribution, lower
63         // variance, and it is what every deployed implementation does.
64         let mut next = low_variance_resample_n(&self.mcl.particles, m - injected, &mut
self.mcl.rng);
65         next.extend((0..injected).map(|_| match self.injector {
66             Injector::UniformFree => map.sample_free_pose(&mut self.mcl.rng, 0.2),
67             Injector::FromSensor => self.mcl.sensor.sample_pose(z, map, &mut self.mcl.rng
)),
68             ));
69         self.mcl.particles.replace(next);
70         injected
71     }
72 }

```

And the dynamic-environment filter, which runs *before* the weighting step:

RUST crates/localize/src/mcl/dynamic.rs

```

1  /// `test_range_measurement` -- Thrun et al., **Table 8.4**, in the direction the
2  /// surrounding text specifies: reject when the reading is probably a person.
3  ///
4  /// `sample` is a representative subset of the predicted particles, not the whole

```



```

5  /// set: this costs one ray cast per pose per beam, and 48 poses estimate a ratio
6  /// of two sums as well as 5 000 do.
7  pub fn test_range_measurement(
8      z_k: f64,
9      bearing: f64,
10     sample: &[SE2],
11     map: &World,
12     params: &BeamParams,
13     chi_rej: f64,
14 ) -> Verdict {
15     let (mut short_mass, mut total) = (0.0, 0.0);
16
17     for x in sample {
18         let z_star = map.raycast(x, bearing, params.max_range);
19         // The mixture is linear in its weights, so each component is just the
20         // full model with the other three weights zeroed. No second copy of
21         // Table 6.1 exists in this crate.
22         short_mass += params.with_only_short().likelihood(z_k, z_star);
23         total += params.likelihood(z_k, z_star);
24     }
25
26     if total <= 0.0 {
27         return Verdict::Accept;
28     }
29     let p_short = short_mass / total;
30     // Asymmetric on purpose: a surprisingly *long* reading has no p_short mass,
31     // so it is never rejected -- and long readings are exactly how a lost filter
32     // finds out that it is lost.
33     if p_short > chi_rej { Verdict::Reject { p_short } } else { Verdict::Accept }
34 }

```

The grid localizer’s predict step is where the coarsening correction lives. It is short, and the two sqrts are the entire content of the derivation above:

RUST crates/localize/src/grid/mod.rs

```

1  impl GridLocalizer {
2      /// Line 3 of Table 8.1, as a scatter with a bounded kernel.
3      ///
4      /// The literal double loop is  $O(|G|^2)$  --  $1.2 * 10^{11}$  kernel evaluations for a
5      ///  $15\text{ cm} * 5?$  grid of this apartment, per step. Scattering each occupied
6      /// cell into the  $3*3*3$  neighbourhood of its predicted pose is  $O(|G| * 27)$ .
7      pub fn predict(&mut self, u: &odomDelta, noise: GridMotionNoise) {
8          let mut next = vec![0.0_f32; self.bel.len()];
9          let half = self.res.xy / 2.0;
10
11          // Coarsening correction: the cell's own extent, in quadrature. Without
12          // it, a 40 cm grid with 10 cm steps never changes cell and the belief
13          // freezes in place while the robot drives away.
14          let sigma_xy = (noise.trans * noise.trans + half * half).sqrt();
15          let sigma_th = (noise.rot * noise.rot
16              + (self.res.theta / 2.0) * (self.res.theta / 2.0))
17              .sqrt();
18
19          for (k, &mass) in self.bel.iter().enumerate() {
20              if mass < 1e-12 {
21                  continue; // selective updating: skip cells with nothing in them
22              }

```

```

23         let predicted = self.cell_center(k).apply_odom(u);
24         self.splat(&mut next, predicted, mass, sigma_xy, sigma_th);
25     }
26
27     self.bel = next;
28     self.normalize();
29 }
30 }

```

12.3.1 The test that pins the worked example

RUST crates/localize/src/mcl/augmented.rs (tests)

```

1  /// The chapter's hand-checked kidnap schedule. Both this test and the
2  /// TypeScript port's `SurpriseDetector` must reproduce these numbers, because
3  /// the prose quotes them.
4  #[test]
5  fn kidnap_injection_schedule() {
6      let mut d = SurpriseDetector::new(0.5, 0.05); // alpha_fast, alpha_slow
7      d.seed(1.0);                                // long-run steady state
8
9      let p1 = d.update(0.2);
10     assert_relative_eq!(d.w_fast, 0.60, epsilon = 1e-12);
11     assert_relative_eq!(d.w_slow, 0.96, epsilon = 1e-12);
12     assert_relative_eq!(p1, 0.375, epsilon = 1e-12);
13
14     let p2 = d.update(0.2);
15     assert_relative_eq!(d.w_fast, 0.40, epsilon = 1e-12);
16     assert_relative_eq!(d.w_slow, 0.922, epsilon = 1e-12);
17     assert_relative_eq!(p2, 0.566_161, epsilon = 1e-6);
18
19     let p3 = d.update(0.2);
20     assert_relative_eq!(p3, 0.661_361, epsilon = 1e-6);
21 }
22
23 /// A single bad reading must *not* trigger sustained injection: that is the
24 /// whole reason there are two timescales.
25 #[test]
26 fn one_glitch_extinguishes() {
27     let mut d = SurpriseDetector::new(0.5, 0.05);
28     d.seed(1.0);
29
30     let spike = d.update(0.2);
31     assert!(spike > 0.37 && spike < 0.38);
32
33     let tail: Vec<f64> = (0..4).map(|_| d.update(1.0)).collect();
34     assert_relative_eq!(tail[0], 0.168_399, epsilon = 1e-6);
35     assert!(tail[3] == 0.0, "injection must switch itself off");
36     // Total particles wasted on a glitch, as a fraction of M:
37     assert!(spike + tail.iter().sum():<f64>() < 0.63);
38 }

```

12.4 Putting it together: choosing a localizer

Thrun’s Table 8.5 compares the localizers along the axes that matter in a deployment. Reproduced below, with our own measurements filled in where this chapter’s port can actually measure something — the analytic columns are arithmetic you can redo, and the timing columns are what the widgets on this page report in your browser, not numbers from a paper.

	EKF (Ch. 11 (Chapter 11))	MHT	Grid, coarse	Grid, fine	MCL	Augmented MCL
Measurements	landmarks	landmarks	raw scans	raw scans	raw scans	raw scans
Posterior	one Gaussian	mixture of Gaussians	histogram over poses	histogram over poses	particles	particles
Memory (this apartment)	~100 B	~1 kB	11 kB @ 0.8 m/45°, f64	225 kB @ 0.3 m/15°, f64; 1.4 MB @ 0.15 m/5°, f32	156 kB @ M = 5 000	same, plus two floats
Update cost	fastest	fast	3 ms measured	7 ms measured	~2 ms @ M = 2 048 measured	+ O(M)
Resolution / accuracy	exact within its mode	exact within modes	half a cell (~0.8 m)	half a cell (~0.16 m)	tunable with M	tunable with M
Global localization	no	yes	yes	yes	yes	yes
Kidnap recovery	no	yes	yes, if cells are reactivated	yes	no	yes
Dynamic environments	via outlier gating	via hypotheses	via beam rejection	via beam rejection	via beam rejection	via beam rejection
Implementation effort	moderate	high	low	low, tuning high	low	low

The two bold cells are the whole story of Part IV. An EKF cannot do global localization at any price, and plain MCL cannot recover at any price; Augmented MCL fixes the second for the cost of two floating-point numbers and a coin flip.

How much does that buy? Running this chapter’s port on twelve seeded kidnaps with $M = 2,048$ and uniform injection, plain MCL re-localized once in twelve; Augmented MCL nine times, with a median of about five filter steps between the teleport and a converged, correct estimate. The three failures are worth taking seriously rather than tuning away. Uniform injection is a lottery: an injected particle has to land within a few tens of centimetres and a few degrees of the truth to out-weight the survivors and seed a cluster, and if the robot’s route after the kidnap happens to be uninformative the lottery can run for a long time. That is exactly the problem the mixture proposal above was invented to solve, and why `Injector::FromSensor` is an exercise rather than a footnote.

12.4.1 Where this algorithm actually lives in 2026

This is not a historical chapter. AMCL — the algorithm on this page, with KLD-adaptive sample sizes from Chapter 8 bolted on — ships as the `nav2_amcl` package of Nav2, the ROS 2 navigation stack, and remains the most widely used map-based localizer in that ecosystem. The ROS maintainers' own 2023 survey describes it as a fully parameterized grid particle filter reaching "localization accuracies of 5 cm or better in many practical environments", and then quietly uses its output as *ground truth* when benchmarking other estimators — which tells you how much a decades-old algorithm is still trusted. `beluga_amcl`, a ground-up C++17 rewrite presented in 2024, exists precisely because so much depends on this algorithm that people wanted better-engineered software running the same mathematics, not different mathematics.

Three things have changed since the textbook, and all three are additive:

- **The proposal.** Mixture-MCL's measurement proposal, impractical for laser scans in 2001, is now routine: scan-match against the map (Chapter 16) or query a learned place-recognition front end, and inject from *that*. Recovery time drops accordingly.
- **The observation model.** IR-MCL (2023) replaces the likelihood field with a neural occupancy field and renders synthetic scans at arbitrary poses to score particles. The filter is unchanged — it is still Table 8.2 — and only line 4 was replaced. That is the modularity this chapter's `SensorModel` trait bound is claiming.
- **The alternative.** Pose-graph localization (SLAM Toolbox's localization mode) tracks the robot by matching scans into a stored graph instead of maintaining a sampled belief. It is more accurate when it works and has no story for global ambiguity as clean as a particle cloud, which is why production stacks still ship AMCL alongside it.

The capstone in Chapter 26 uses `AugmentedMcl` unchanged, and Chapter 17 turns it into a SLAM system by the simple expedient of giving every particle its own map.

12.5 Exercises

1 FOUNDATION •• The detector under a permanent change

A room is remodelled, so the average measurement likelihood drops permanently by a factor $(1 - \delta)$ from an old steady state of 1. Using $w_{fast}(n) = 1 - \delta(1 - (1 - \alpha_{fast})^n)$ and the same expression with α_{slow} , derive $p_{inject}(n)$ in closed form. Show that $p_{inject} \rightarrow 0$ as $n \rightarrow \infty$, find the step at which it peaks for $\delta = 0.8$, $\alpha_{fast} = 0.5$, $\alpha_{slow} = 0.05$, and say in one sentence what a *persistently* nonzero injection rate would tell you about your map.

<details> <summary>Check your peak</summary> The maximum is at $n = 6$ with $p_{\text{inject}} \approx 0.73$; by $n = 100$ it is under 3%. A persistent nonzero rate cannot come from a step change — it means the likelihood is still *falling*, i.e. the map is drifting out of date, which is a maintenance alarm rather than a localization one. </details>

2 FOUNDATION • • • When is a symmetry actually a symmetry?

Let σ be a rigid transform with $\sigma(m) = m$. Show that if (i) $\text{bel}(x_0)$ is invariant under σ , (ii) $p(z \mid x, m) = p(z \mid \sigma(x), m)$ for every z , and (iii) $p(x' \mid u, x) = p(\sigma(x') \mid u, \sigma(x))$, then $\text{bel}(x_t)$ is invariant under σ for all t — the ambiguity is permanent and no amount of driving resolves it.

Now check the conditions for the Apartment's mirror symmetry $\sigma(x, y, \theta) = (12 - x, y, \pi - \theta)$ and a 360° LiDAR. One of (ii) and (iii) fails. Which, and why? Use your answer to explain the behaviour you actually observe in [the Theater](#w12.1), and say what kind of symmetry *would* produce a permanently bimodal posterior for this robot.

<details> <summary>The shape of the answer</summary> Reflection is orientation-reversing. Reflecting the scene reverses the *order* of the beams, so the scan a mirrored robot receives is z read backwards, and (ii) holds only for scans that happen to be left–right symmetric. (iii) fails too: the mirror image of a left turn is a right turn, so a ghost hypothesis fed the robot's own odometry stops being the mirror image the moment the robot rotates. A pure *translation* symmetry — two identical rooms side by side, no reflection — satisfies both, which is why real buildings full of identical offices are so much harder than a symmetric floorplan. </details>

3 FOUNDATION • • • Premature convergence has a timescale

Model the symmetric case as follows: M particles split between two modes with equal weights, and each resampling step draws M particles multinomially from the current split. Show that the number in mode A is a martingale, and that the expected time until one mode is extinguished grows like $O(M)$. What does that imply for the practice of "just use more particles" — does it fix the problem, or only postpone it?

4 CONCEPTUAL • Predict, then check: how small can M be?

In the [MCL Theater](#w12.1), with the likelihood field and $\kappa = 1$, predict whether $M = 256$ will converge before Rusty reaches the corridor junction. Then find the smallest power of two that converges reliably across five re-rolls of the seed. Now set $\kappa = 3$ and repeat. Which direction did the answer move, and why — explain it using the proposal/target argument from the MCL derivation rather than "the sensor got worse".

5 **CONCEPTUAL** •• **Blind the detector on purpose**

In the `<a href="#w12.3">Recovery Ward</a>`, set $\alpha_{fast} = \alpha_{slow}$. Predict, before pressing anything, what the two lines and the injection probability will do on (a) a kidnap and (b) a single glitched reading. Verify. Then state in one sentence why *two* timescales, rather than one timescale and a threshold, is the mechanism — and what a per-map threshold would have to be re-tuned against.

6 **CONCEPTUAL** •• **The cost of rejecting the unexpected**

In the `<a href="#w12.5">Crowd Mode</a>`, record the RMSE with the novelty test off, then at $\chi_{rej} = 0.6$, then at $\chi_{rej} = 0.1$. Now tick *close room B's door* and watch what the rejected-beam stubs do. Explain why the same mechanism that removes people also removes a real change to the map, and propose one signal — available to the filter, not to you — that could tell the two apart.

7 **PRACTICAL** •• **Injector::FromSensor**

Implement measurement-driven injection for the likelihood field: sample a scan endpoint, sample a free cell near it with probability proportional to the field value, and complete the pose with a heading drawn to align the scan. Measure median kidnap-recovery time against uniform injection over 50 seeded kidnaps, and report the distribution rather than the mean — recovery times are heavy-tailed and the mean will lie to you.

8 **PRACTICAL** ••• **Make the grid competitive**

Take the `GridLocalizer` to $15\text{ cm} \times 5^\circ$ and make one update run in under 10 ms. You will need three things from §8.2.3: pre-cached per-cell likelihoods so correction is a lookup, selective updating so untouched cells cost nothing, and delayed motion updates so the convolution runs once per 20 cm rather than once per tick. Then answer the question the chapter dodged: with all three optimizations, is the fine grid ever the right choice over MCL — and for which robot?

12.6 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 8 is this chapter's spine: Tables 8.1–8.4 are reproduced here as `Grid_localization`, `MCL`, `Augmented_MCL` and `test_range_measurement`, and the comparison table is Table 8.5 with our own measurements added. [link](#)

Burgard, W., Fox, D., Hennig, D., and Schmidt, T. (1996). Estimating the Absolute Position of a Mobile Robot Using Position Probability Grids *Proc. AAAI-96*, pp. 896–901.

Grid localization as originally proposed. Reading it next to the cost arithmetic in this chapter shows exactly which engineering compromises the metric-grid era was built on. [link](#)

Dellaert, F., Fox, D., Burgard, W., and Thrun, S. (1999). Monte Carlo Localization for Mobile Robots *Proc. IEEE ICRA 1999*, pp. 1322–1328.

The paper that replaced grids with samples, and the source of the claim this chapter measures: faster, more accurate, and far smaller than a grid at the same accuracy. doi:10.1109/ROBOT.1999.772544

Thrun, S., Fox, D., Burgard, W., and Dellaert, F. (2001). Robust Monte Carlo Localization for Mobile Robots *Artificial Intelligence* 128(1–2), 99–141.

Where Mixture-MCL and the recovery machinery are developed properly, including the perfect-sensor failure and the k-d-tree density estimate the mixture weights need. doi:10.1016/S0004-3702(01)00069-8

Fox, D. (2003). Adapting the Sample Size in Particle Filters Through KLD-Sampling *International Journal of Robotics Research* 22(12), 985–1003.

The adaptive-M bound built in Chapter 8 and switched on inside AMCL: thousands of particles while lost, dozens once converged, with a stated KL guarantee. doi:10.1177/0278364903022012001

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., and Ferguson, M. (2023). From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2 *Robotics and Autonomous Systems* 168, 104493.

The maintainers’ own account of what actually ships in Nav2, and the citation behind this chapter’s claim that AMCL is still the default localizer. doi:10.1016/j.robot.2023.104493

Puga, G., Espinosa, N., Hidalgo, M., García, O., and Paunovic, I. (2024). Beluga: A Modern Monte Carlo Localization Package for ROS and ROS 2 *European Robotics Forum 2024, Springer Proceedings in Advanced Robotics* 32, 126–130.

A ground-up reimplement of AMCL as a generic C++17 particle-filter library. Evidence that the interesting work on this algorithm is now software architecture, not new mathematics. doi:10.1007/978-3-031-76424-0_23

Kuang, H., Chen, X., Guadagnino, T., Zimmerman, N., Behley, J., and Stachniss, C. (2023). IR-MCL: Implicit Representation-Based Online Global Localization *IEEE Robotics and Automation Letters* 8(3), 1627–1634.

MCL with a neural occupancy field as the observation model: line 4 of Table 8.2 replaced, everything else untouched. The cleanest demonstration that the filter and the sensor model are genuinely separable. doi:10.1109/LRA.2023.3239318

Macenski, S. and Jambrecic, I. (2021). SLAM Toolbox: SLAM for the Dynamic World *Journal of Open Source Software* 6(61), 2783.

The pose-graph alternative to a sampled belief, including its localization-only mode. Chapter 16 builds the scan matcher it runs on. doi:10.21105/joss.02783

Part V

Mapping and SLAM

Occupancy Grid Mapping

MAPPING AND SLAM · Intermediate · 55 min



The basic idea of the occupancy grids is to represent the map as a field of random variables, arranged in an evenly spaced grid.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 9

Chapter 12 ended with Rusty localizing beautifully against a map we handed it. This chapter turns the problem inside out: an oracle now hands us the *poses*, and the map is what we have to find. That sounds like a lesser problem, and in one sense it is — but it is where the field's most-used data structure comes from, and it is the substrate that ROS 2's Nav2 costmaps and SLAM Toolbox submaps are still built on twenty-five years later.

The idea is small enough to state in one line. Chop the plane into cells, attach a binary random variable to each, and run the static binary Bayes filter of Chapter 8 on every one of them independently. In log odds that filter is a single $+=$, so a hundred thousand of them cost less than one Kalman update. The map that comes out has three states, not two: occupied, free, and — the state that makes it useful for planning — *gray*, meaning nobody has looked.

The word "independently" is doing an enormous amount of work in that paragraph, and the second half of this chapter is about the bill. Beams couple cells; the true posterior over maps does not factor; and the factored filter does not merely lose information, it *manufactures* contradictions that were never in the data — closing doorways that are demonstrably open. We will watch that happen, then fix it the honest way and count the cost.

🕒 AFTER THIS CHAPTER YOU CAN

- Write down the map posterior, and name precisely which approximation makes it computable

- Derive the log-odds recursion of occupancy grid mapping from Bayes rule, including the $-\ell_0$ term
- Build the hand-crafted inverse range sensor model, and see it for the design choice it is
- Learn an inverse model from the forward model, and prove that doing so is Bayes-optimal
- Show, with numbers, where per-cell independence breaks, and repair it with MAP inference
- Fuse sensors that disagree about what "occupied" means, without deleting the furniture
- Implement the whole thing in Rust as a flat `Vec<f32>` of log odds updated by integer ray traversal

ASSUMED

- The binary Bayes filter with static state and its log-odds form (Chapter 8)
- The beam range-finder model $p(z \mid x, m)$ (Chapter 10) — it comes back as the forward model here
- Rusty, the Apartment, and ray-cast LiDAR (Chapter 4)

13.1 The problem with drawing what you see

Strip the map out of a Chapter 12 log and you are left with a stack of scans and a list of poses. The naive thing to do is obvious: transform every scan endpoint into world coordinates and mark the cell it lands in as occupied. Draw the dots.

This fails within seconds, and it fails in three separate ways. A person walks through the corridor and is permanently inked into the floor plan. A grazing beam that skipped off a door frame puts an obstacle in the middle of the doorway. And, most importantly, the map you get has no way to distinguish "I looked and there was nothing there" from "I never looked" — which is exactly the distinction a path planner needs, because one of those is a corridor and the other might be a staircase.

The probabilistic answer keeps the dots but stops treating them as facts. Each beam is a *piece of evidence* about the cells it crosses, and a map is what you get by accumulating evidence.

INTERACTIVE · w13.1

Map Weaver

A map is not a drawing. It is one binary Bayes filter per cell, run in parallel — and

gray means ignorance, not emptiness.

Run this simulation in the web edition: [#w13.1](/chapters/ch13-occupancy-grids)

Watch a full lap before reading on. Three things are worth noticing, and each corresponds to a piece of mathematics below.

Gray is a state, not a background color. The two north rooms are never entered, and everything in them that the LiDAR cannot reach through a doorway sits at $p = 0.5$ from the first frame to the last. That is the correct answer: no measurement has ever told the robot anything about those cells. A planner reading this map knows not to route through them without looking first — which is the entire subject of Chapter 24.

Repetition is what makes walls crisp. One pass down the corridor leaves a smear a few cells thick, because each beam's evidence is soft and the sensor is noisy. The wall sharpens on the second and third pass, as cells that keep getting the same answer accumulate log odds and cells that got a stray reading get argued back down. No cell is ever certain, which is why a wrong one can be fixed.

Every cell is running its own filter. The chart under the map is not a summary — it is three of the grid's cells, plotted individually. The wall cell climbs, the corridor cell falls, and the closet cell the robot never sees stays flat on zero. Tile that chart a hundred thousand times and you have the map.

💡 THE ONE SENTENCE VERSION

An occupancy grid map is a field of independent binary Bayes filters, one per cell, each accumulating log-odds evidence from an inverse sensor model. The independence is false, it is what makes the algorithm cheap, and knowing exactly how it is false is what separates a map you can plan on from one you cannot.

13.2 Building intuition: the scan as a paintbrush

Here is the metaphor to carry through the chapter. A map is slow-developing film, and every scan is a brush stroke. The stroke is not a point — it is a shape, wide at the far end, that deposits *negative* evidence everywhere the beam passed through and *positive* evidence in a band at the range it stopped. Beyond the stopping point it deposits nothing at all, because a beam that hit a wall says nothing whatsoever about what is behind it.

Undeveloped film is gray. That is the whole trick: the resting state of a cell is not "empty", it is "unexposed".

Two properties of that brush matter, and both come straight from Chapter 8:

1. **In log odds, evidence adds.** A cell's belief after t scans is the sum of t

signed increments plus its prior. That is why occupancy mapping is one of the cheapest algorithms in this book, and why the widget above can fold a sixty-beam scan into *two* ten-thousand-cell maps twelve times a second, in a browser tab, in about a millisecond a frame.

2. **Saturated cells stop listening.** After forty consistent readings a cell’s log odds are so large that forty contradicting readings are needed to move it. That is a genuine bug in a world with doors and people, and the standard patch — clamping $|\ell|$ at some $\ell_{\max}$ — is the second slider in the widget above. It comes from Nav2-era practice, not from Thrun.

13.2.1 Notation for this chapter

$m = \{m_i\}$	The map as a set of cells; each $m_i \in \{0,1\}$ is the binary occupancy of cell i . "1" is occupied.
$p(m_i)$	Occupancy prior of a cell. 0.5 unless stated otherwise.
$\ell_{t,i}$	Log odds of cell i after t measurements (the book-wide symbol, TOC §2).
ℓ_0	Prior in log odds, $\log p(m_i) / (1 - p(m_i))$. Zero for a uniform prior.
ℓ_{occ}, ℓ_{free}	The two evidence levels of the hand-crafted inverse model.
$p(m_i z_t, x_t)$	Inverse sensor model: what one measurement says about one cell.
α, β	Obstacle thickness and beam opening angle — the two constants you choose.
$H(m)$	Map entropy, $\sum_i H_b(p_i)$ in bits: how much is left to learn.

One deliberate exception to the book’s color code lives in this chapter. Maps are drawn in **grayscale** — white free, black occupied, mid-gray unknown — because that convention is older and stronger than ours, and because the middle of the ramp carrying the meaning "I don’t know" is precisely the point being made. Beams, evidence, and inverse-model overlays stay green; the ℓ_0 reference line stays blue.

13.3 The mathematics

13.3.1 The map posterior, and the one approximation everything rests on

The quantity we want is the posterior over maps given all measurements and all poses:

$$p(m \mid z_{1:t}, x_{1:t})$$

Controls $u_{1:t}$ do not appear. The path is given, so the controls carry no extra information about it and drop out of every expression in this chapter.

This posterior is not merely expensive, it is absurd. A modest 12×9 metre apartment at 10 cm resolution is 10,800 cells, so the map lives in a space of 2^{10800} possibilities. Representing one distribution over that space is not a matter of buying more memory.

So we make a decomposition — and, unlike most of the literature, we are going to give it a name and put it on trial later:

💡 APPROXIMATION #1 — PER-CELL INDEPENDENCE

$$p(m \mid z_{1:t}, x_{1:t}) \approx \prod_i p(m_i \mid z_{1:t}, x_{1:t})$$

The joint posterior over maps is replaced by the product of its marginals. Every occupancy mapper in production makes this approximation. Almost none of them admit it.

Each factor $p(m_i \mid z_{1:t}, x_{1:t})$ is now a binary estimation problem with *static* state — the cell does not move, does not change, and has no motion model. That is exactly the filter of Chapter 8, and we already know its log-odds form.

13.3.2 Log odds, and why the map is stored in them

$$\ell_{t,i} = \log \frac{p(m_i \mid z_{1:t}, x_{1:t})}{1 - p(m_i \mid z_{1:t}, x_{1:t})} \iff p(m_i \mid z_{1:t}, x_{1:t}) = \frac{1}{1 + \exp \ell_{t,i}}$$

Two reasons, one numerical and one structural. Numerically, probabilities near 0 and 1 lose precision exactly where a map spends most of its life, and log odds are well behaved out to ± 700 in f64. Structurally — and this is the reason that matters — Bayes' rule for a static binary variable becomes **addition**, so integrating a scan is a loop of += over a flat array.

13.3.3 The recursion

ALGORITHM `occupancy_grid_mapping({ℓ_{t-1,i}}, x_t, z_t)` COST $O(B \cdot L/\rho)$ per scan with ray traversal -- B beams, range L, resolution ρ . Not $O(|m|)$.

in the previous map in log odds, the (known) pose, the scan

out $\{\ell_{t,i}\}$

- 1: for all cells m_i do
- 2: if m_i in perceptual field of z_t then

```

3:      $\ell_{t,i} = \ell_{t-1,i} + \text{inverse\_sensor\_model}(m_i, x_t, z_t) - \ell_0$ 
4:   else
5:      $\ell_{t,i} = \ell_{t-1,i}$ 
6:   endif
7: endfor
8: return  $\{\ell_{t,i}\}$ 

```

That is Thrun et al., Table 9.1, and it is the whole algorithm. Line 3 is three terms: what you believed, what this measurement says, and a correction that removes the prior so it is not counted once per measurement. Everything interesting is hidden inside `inverse_sensor_model` and inside the phrase "perceptual field".

Note the complexity line. Written literally the table sweeps every cell for every scan, which for the Apartment would be $10,800 \text{ cells} \times 60 \text{ beams}$. In practice you invert the loop: walk each beam through the grid with an integer line algorithm and touch only the cells it crosses, which is a few hundred per beam. The two are equivalent for a pencil-thin LiDAR beam. For a sonar whose cone is wider than the spacing between beams they are *not*, and the difference is a plot point in this chapter, not a detail.

DERIVATION D1 — the static-state log-odds recursion

$$\ell_{t,i} = \ell_{t-1,i} + \log \text{it} p(m_i | z_t, x_t) - \ell_0$$

Write $p^+ = p(m_i | z_{1:t}, x_{1:t})$ and $p^- = p(\neg m_i | z_{1:t}, x_{1:t})$, so $\ell_{t,i} = \log p^+ / p^-$. We derive a recursion for the *ratio*, because every normalizer will cancel and never has to be computed.

Step 1 — split off the newest measurement with Bayes' rule.

$$p(m_i | z_{1:t}, x_{1:t}) = \frac{p(z_t | m_i, z_{1:t-1}, x_{1:t}) p(m_i | z_{1:t-1}, x_{1:t})}{p(z_t | z_{1:t-1}, x_{1:t})}$$

The state is static and complete, so z_t depends only on m_i and the current pose: $p(z_t | m_i, z_{1:t-1}, x_{1:t}) = p(z_t | m_i, x_t)$. The second factor is $p(m_i | z_{1:t-1}, x_{1:t-1})$, the previous posterior — a pose the robot has not yet used tells us nothing about a static map.

Step 2 — swap the forward model for the inverse model. This is the step that gives the algorithm its name. Apply Bayes' rule again, in the other direction:

$$p(z_t | m_i, x_t) = \frac{p(m_i | z_t, x_t) p(z_t | x_t)}{p(m_i)}$$

Substituting,

$$p^+ = \frac{p(m_i | z_t, x_t) p(z_t | x_t) p(m_i | z_{1:t-1}, x_{1:t-1})}{p(m_i) p(z_t | z_{1:t-1}, x_{1:t})}$$

Step 3 — write the same thing for the complement. Replace m_i by $\neg m_i$ throughout:

$$p^- = \frac{(1 - p(m_i | z_t, x_t)) p(z_t | x_t) (1 - p(m_i | z_{1:t-1}, x_{1:t-1}))}{(1 - p(m_i)) p(z_t | z_{1:t-1}, x_{1:t})}$$

Step 4 — take the ratio. Both $p(z_t | x_t)$ and the evidence $p(z_t | z_{1:t-1}, x_{1:t})$ appear identically in numerator and denominator and vanish:

$$\frac{p^+}{p^-} = \underbrace{\frac{p(m_i | z_t, x_t)}{1 - p(m_i | z_t, x_t)}}_{\text{this measurement}} \cdot \underbrace{\frac{p(m_i | z_{1:t-1}, x_{1:t-1})}{1 - p(m_i | z_{1:t-1}, x_{1:t-1})}}_{\text{what you believed}} \cdot \underbrace{\frac{1 - p(m_i)}{p(m_i)}}_{\text{prior, removed}}$$

Step 5 — take logs. The product becomes a sum, and the third factor is exactly $-\ell_0$:

$$\ell_{t,i} = \ell_{t-1,i} + \log \frac{p(m_i | z_t, x_t)}{1 - p(m_i | z_t, x_t)} - \ell_0$$

with boundary condition $\ell_{0,i} = \ell_0$. ■

Why the $-\ell_0$ term is not optional. The inverse model returns a *posterior* — it already contains the prior. Adding it t times without subtracting ℓ_0 each time embeds the prior t times, so with $p(m_i) = 0.2$ (that is, $\ell_0 = -1.386$) a cell that receives nothing but uninformative readings drifts steadily toward "free" at 1.386 nats per scan, purely from bookkeeping. With the common choice $p(m_i) = 0.5$ we have $\ell_0 = 0$ and the bug is invisible, which is exactly why it is a bug worth naming.

13.3.4 The inverse sensor model

`inverse_sensor_model` answers a question that runs backwards from everything in Chapter 10. The *forward* model $p(z_t \mid x_t, m)$ reasons from causes to effects: given a map, what would the sensor read? The *inverse* model $p(m_i \mid z_t, x_t)$ reasons from effects to causes: given this reading, what is that cell?

The inverse direction is the unnatural one — it depends on the prior over maps, it is only defined per cell, and no sensor datasheet contains it. Here is the standard hand-built answer:

```
ALGORITHM inverse_range_sensor_model(i, x_t, z_t)  COST 0(1) per cell, given
the nearest-beam lookup
in cell index, pose  $x_t = (x, y, \theta)$ , scan  $z_t$  with bearings  $\theta_{\{k, \text{sens}\}}$ 
out  $\ell \in \{\ell_0, \ell_{\text{occ}}, \ell_{\text{free}}\}$ 
```

```
1: let  $(x_i, y_i)$  be the center of mass of  $m_i$ 
2:  $r = \sqrt{(x_i - x)^2 + (y_i - y)^2}$ 
3:  $\phi = \text{atan2}(y_i - y, x_i - x) - \theta$ 
4:  $k = \arg \min_j |\phi - \theta_{j, \text{sens}}|$ 
5: if  $r > \min(z_{\text{max}}, z_t^k + \alpha/2)$  or  $|\phi - \theta_{k, \text{sens}}| > \beta/2$  then
6:   return  $\ell_0$ 
7: if  $z_t^k < z_{\text{max}}$  and  $|r - z_t^k| < \alpha/2$  then
8:   return  $\ell_{\text{occ}}$ 
9: if  $r \leq z_t^k$  then
10:  return  $\ell_{\text{free}}$ 
11: endif
```

Three regions, in order: no information beyond the reading or outside the cone; occupied in a band of thickness α centered on the reading; free everywhere nearer than that. α is meant to be "how thick an obstacle is, plus the discretization"; β is the beam's opening angle.

⚠ WHERE ROBOTS BREAK THIS

Line 7 of Table 9.2 in the 1999–2000 draft reads $|r - z_{\text{max}}| < \alpha/2$. That cannot be what is meant: taken literally it paints a shell of obstacles at the sensor's maximum range and never marks the actual return. Every working implementation, including `lib/mapping/occgrid.ts` in this repository, uses $|r - z_t^k|$. If you are typing the table in from a photocopy, this is the line that will cost you an afternoon.

Notice what is *not* derived here. ℓ_{occ} , ℓ_{free} , α , and β are four numbers

somebody chose. That is the subject of the next section.

13.3.5 A worked example you can check by hand

Take a single cell with a uniform prior, so $\ell_0 = 0$, and an inverse model that reports $p = 0.7$ for occupied and $p = 0.3$ for free:

$$\ell_{occ} = \ln \frac{0.7}{0.3} = 0.8473, \quad \ell_{free} = \ln \frac{0.3}{0.7} = -0.8473$$

Now feed the cell three readings — occupied, occupied, free — and apply D1 three times. Since $\ell_0 = 0$, each step is one addition:

step	reading	ℓ	$p = 1 - 1/(1 + e^\ell)$
0	—	0	0.5000
1	occupied	0.8473	0.7000
2	occupied	1.6946	0.8448
3	free	0.8473	0.7000

Two things to take from three lines of arithmetic. The second consistent reading buys much less than the first in probability (+0.145 versus +0.200) but exactly as much in log odds — that is what "evidence adds" means. And the contradicting third reading returns the cell precisely to where one reading had put it, because the filter has no memory beyond the running sum. Order does not matter; only the tally does.

RUST `crates/ch13_occgrid/tests/micro_example.rs`

```
1 use approx::assert_relative_eq;
2 use ch13_occgrid::{log_odds_from_prob, prob_from_log_odds};
3
4 /// The worked-example table, pinned. The book prints these six numbers; if this
5 /// test ever fails, the book is wrong, not the test.
6 #[test]
7 fn worked_example_ch13_three_readings() {
8     let l_occ = log_odds_from_prob(0.7);
9     let l_free = log_odds_from_prob(0.3);
10    assert_relative_eq!(l_occ, 0.847_298, epsilon = 1e-5);
11    assert_relative_eq!(l_free, -l_occ, epsilon = 1e-6);
12
13    /// l_t = l_{t-1} + evidence - l_0, and l_0 = 0 for a uniform prior.
14    let readings = [l_occ, l_occ, l_free];
15    let want = [(0.847_298_f32, 0.700_000_f32), (1.694_596, 0.844_827), (0.847_298, 0.700_000)];
16
17    let mut l = 0.0f32;
18    for (evidence, (want_l, want_p)) in readings.iter().zip(want) {
19        l += evidence;
20        assert_relative_eq!(l, want_l, epsilon = 1e-5);
21        assert_relative_eq!(prob_from_log_odds(l), want_p, epsilon = 1e-5);
22    }
23
24    /// The invariant that makes log odds worth using: only the tally matters.
```

```

25     let shuffled: f32 = [l_free, l_occ, l_occ].iter().sum();
26     assert_relative_eq!(shuffled, l, epsilon = 1e-6);
27 }

```

The TypeScript port carries the same check in `lib/__checks__.ts`, and the widgets on this page run that port — so the numbers in the table, the numbers in the Rust test, and the numbers under the cursor in Map Weaver are the same numbers.

13.4 Where does the inverse model come from?

$\alpha, \beta, \ell_{occ}, \ell_{free}$. Four constants, no derivation, and every one of them changes the map. Before accepting them, look at what they actually paint.

INTERACTIVE · w13.3

Inverse-Model Workbench

The inverse sensor model is a design choice, not a law — and the model you get by fitting the forward one disagrees with the hand-crafted version exactly where you would have guessed wrong.

Run this simulation in the web edition: `/chapters/ch13-occupancy-grids #w13.3`

The hand-crafted model is a step function in two variables: it is equally confident about a cell 5 cm off the beam axis and one $\beta/2 - \epsilon$ off it, then falls off a cliff. It is equally confident about a return at 0.5 m and one at 4.5 m, though the second one's cone has swept an arc ten times as long. And it carves free space with total conviction on a max-range reading, which in real hardware is most often a beam that hit glass, a dark surface, or nothing at all.

There is a principled alternative, and it is one of the most under-appreciated sections of the original book.

DERIVATION D3 — a learned inverse model is Bayes-optimal regression

$$\operatorname{argmin} E[\text{cross-entropy}] = p(m_i | z, x)$$

Statement. Let triplets (m, x_t, z_t) be generated by sampling a map from the map prior, sampling a pose in it, and sampling a measurement from the *forward* model $p(z_t | x_t, m)$; let the label be the true occupancy $\text{occ}(m_i)$ of a target cell. Then among *all* functions $f(z_t, x_t, i) \in [0, 1]$, the minimizer of the expected cross-entropy

$$J(f) = -\mathbb{E}[\text{occ}(m_i) \log f + (1 - \text{occ}(m_i)) \log(1 - f)]$$

is $f^\star(z_t, x_t, i) = p(m_i | z_t, x_t)$: exactly the inverse sensor model.

Step 1 — what we cannot compute. Bayes' rule gives the inverse model in closed form,

$$p(m_i | z, x) = \eta \int_{m: m(i)=m_i} p(z | x, m) p(m) dm$$

an integral over every map whose i -th cell takes the given value. It is not merely hard, it is the same $2^{|m|}$ we ran away from at the start of the chapter.

Step 2 — sample instead. We cannot integrate over maps, but we can *draw* from the same distribution: $m^{[k]} \sim p(m)$, then $x^{[k]}$ uniform inside it, then $z^{[k]} \sim p(z | x^{[k]}, m^{[k]})$ — that last step is just running the Chapter 10 beam model in generative mode, which is what a simulator is. Record $\text{occ}(m_i)^{[k]}$.

Step 3 — condition and minimize pointwise. Write the expectation by conditioning on the input $\xi = (z, x, i)$:

$$J(f) = \mathbb{E}_\xi \left[-q(\xi) \log f(\xi) - (1 - q(\xi)) \log(1 - f(\xi)) \right], \quad q(\xi) := p(m_i = 1 | \xi)$$

Because f is unconstrained, the outer expectation is minimized by minimizing the bracket separately at each ξ .

Step 4 — one derivative. For fixed q , $g(f) = -q \log f - (1 - q) \log(1 - f)$ has $g'(f) = -q/f + (1 - q)/(1 - f)$, which vanishes at $f = q$, and $g'' > 0$ throughout $(0, 1)$. So the unique minimizer is $f^\star(\xi) = q(\xi) = p(m_i | z, x)$.

■

Step 5 — what this buys. Any sufficiently flexible function approximator trained by gradient descent on this loss converges toward the true inverse model, *automatically consistent with the sensor physics and with the prior over maps you sampled from*. No α , no β , no ℓ_{occ} . The invariances are enforced by the choice of inputs, not by the loss: give the model the cell's range and bearing **in the beam frame** and the reading, and it cannot possibly learn anything about absolute coordinates.

ALGORITHM `learn_inverse_sensor_model(p(z | x, m), p(m), simulator)` COST
 offline: $O(\text{samples} \times \text{epochs})$. $O(1)$ per cell at run time.

in the forward model, the map prior, and a way to sample from both

out $\hat{f}(r, \psi, z) \approx p(m_i | z, x)$

1: for $k = 1$ to K do

```

2:   sample a map  $m^{[k]} \sim p(m)$ 
3:   sample a pose  $x^{[k]}$  inside it
4:   sample a measurement  $z^{[k]} \sim p(z \mid x^{[k]}, m^{[k]})$ 
5:   pick a target cell; record its beam-frame coordinates  $(r, \psi)$  and its
      true occupancy
6:   endfor
7:   fit  $\hat{f}$  by minimizing  $J(W) = -\sum_k [m^{[k]} \log \hat{f} + (1 - m^{[k]}) \log(1 - \hat{f})]$ 
8:   return  $\hat{f}$ 

```

Toggle **Learned inverse model** in the workbench above and compare. The version running there is a logistic regression on fifteen bounded features of (r, ψ, z) , trained by SGD on 24,000 triplets sampled exactly as in the table — a model small enough to print, which is the point; Chapter 25 does this properly with `candle`. One structural property is worth calling out: a logistic regressor's score $w^\top \varphi$ is a log odds, so the quantity line 3 of Table 9.1 wants comes straight out of the model with no sigmoid and no inversion.

What it learned, and none of it was specified:

- **Soft edges.** The transition from "free" to "occupied" is a smooth ramp about half of α wide, because the reading is noisy and the surface's position is uncertain by more than a cell.
- **The band sits slightly past the reading.** A cone returns the *nearest* point of a surface that usually continues past it, so the on-axis cell at range exactly z is more often in front of the wall than in it.
- **A narrower waist than β .** Evidence concentrates on the beam axis and decays smoothly, rather than filling the cone uniformly and then stopping dead.
- **Its own ℓ_0 .** Far beyond the reading the learned curve flattens onto about -0.3 , not 0. That number is the prior of the maps it was trained on, and line 3 of Table 9.1 must subtract *that* value. Feed a learned model into a mapper that assumes $\ell_0 = 0$ and every cell in every perceptual field picks up a constant drift.
- **Skepticism at $z_{\max}$.** Press **Max-range reading**: the free-space evidence drops by well over half, because one training reading in twenty was a $z_{\max}$ dropout from a wall that was really there. The hand-crafted model carves at full confidence regardless.

13.5 The independence trap

Now the bill for Approximation #1.

The trouble is easiest to see with two cells. Put A and B side by side at the same range, both inside one sonar cone, both with prior 0.5, and take one reading

of $z = 2.0$ m from a sensor whose forward model is the Chapter 10 mixture ($z_{hit} = 0.8$, $\sigma_{hit} = 0.15$, $z_{short} = 0.1$, z_{max} -weight 0.05, $z_{rand} = 0.05$, range 3 m). The forward model needs only the range to the *first* occupied cell, so three of the four maps predict the same $z^* = 2$:

m_A	m_B	z^*	$p(z = 2 \mid m)$	joint posterior
0	0	3.000	0.0309	0.0047
1	0	2.000	2.1600	0.3318
0	1	2.000	2.1600	0.3318
1	1	2.000	2.1600	0.3318

The marginals come out at $p(m_A) = p(m_B) = 0.6635$. Now ask the factored representation for the probability that *both cells are free* — the query a planner makes when it wants to drive between them:

$$\underbrace{(1 - 0.6635)^2}_{\text{factored}} = 0.1132 \quad \text{versus} \quad \underbrace{0.0047}_{\text{true}}$$

The factored map is off by a factor of **24**. And not in a subtle direction: "both free" is the one configuration the measurement flatly rules out — something in that cone stopped the beam — yet the product of marginals gives it eleven percent. The measurement induced a strong *negative* correlation between A and B ("at least one of you is occupied") and a product of marginals is structurally incapable of representing a correlation of any kind.

Scale that from two cells to a doorway.

INTERACTIVE · w13.2

The Independence Trap

Per-cell independence is not harmless bookkeeping: it manufactures conflicts the data never contained, and closes doors that are open.

Run this simulation in the web edition: </chapters/ch13-occupancy-grids> #w13.2

The left pane is Table 9.1 executed literally, cell by cell, with a 22° sonar cone. Beams that graze the door post report a short range, and the inverse model dutifully paints occupied evidence across the *entire* arc at that range — including the cells in the middle of the opening that a different beam has already carved free. The filter's only conflict-resolution mechanism is addition, so the doorway ends up whatever color the vote count makes it: a one-cell wall, with carved free space behind it. That map is not just wrong, it is *impossible* — there is no physical configuration of the world in which a sealed wall has open floor behind it that a sensor could have seen.

13.5.1 Maximum a posteriori occupancy mapping

The honest alternative gives up on representing uncertainty and asks for the single most probable map instead:

$$m^* = \arg \max_m \log p(z_{1:t} \mid x_{1:t}, m) + \log p(m)$$

Both terms decompose. The likelihood is a sum over measurements of the Chapter 10 forward model — no inverse model appears anywhere — and the prior, for i.i.d. cells, collapses to something with only one map-dependent term:

$$\log p(m) = \sum_i \left[m_i \log p(m_i) + (1 - m_i) \log(1 - p(m_i)) \right] = \underbrace{M \log(1 - p(m_i))}_{\text{constant}} + \ell_0 \sum_i m_i$$

using $\log p - \log(1 - p) = \ell_0$. The constant drops out of the argmax, leaving a per-cell charge of ℓ_0 nats for every cell you declare occupied — which, for a prior below one half, is a penalty. What remains is a discrete optimization over $2^{|m|}$ binary maps, which we attack by hill climbing.

ALGORITHM MAP_occupancy_grid_mapping($x_{\{1:t\}}$, $z_{\{1:t\}}$) COST 0(sweeps · $|m|$ · $\bar{B}_i$), where $\bar{B}_i$ is the number of beams incident on a cell
in the path and the measurements; a forward model $p(z \mid x, m)$
out m^* , a single binary map

```

1: set  $m = \{0\}$ 
2: repeat until convergence
3:   for all cells  $m_i$  do
4:      $m_i = \arg \max_{k=0,1} k \ell_0 + \sum_t \log p(z_t \mid x_t, m \text{ with } m_i = k)$ 
5:   endfor
6: endrepeat
7: return  $m$ 
```

Two implementation notes carry the whole cost of the algorithm. First, flipping one cell changes the likelihood of only the beams whose cone passes through it, so a **beam-cell incidence cache** turns each evaluation from a sum over thousands of measurements into a sum over a handful. Second, incidence can be computed once on the empty map: occupancy decides where a ray *stops*, never where it *goes*, so a beam that misses a cell on the empty map can never be affected by it.

The right pane of the widget above hill-climbs by steepest flip rather than in index order — same objective, same local maximum, but the visit order makes the reasoning legible: the first cells to move are the ones that explain

the most measurement mass, and each one is annotated with the beams whose log-likelihood it raised.

Watch what it produces, because it is not what you expect and the surprise is the lesson. The MAP map is **sparse**. It is a scatter of obstacle cells, not a floor plan — and it should be, because a reading is explained the moment *something* in its cone blocks it at the right range, and the optimizer has no reason to pay ℓ_0 for cells the data does not demand. The cells it places behind the doorway sit on the far wall, seen *through* the opening. There is no conflict anywhere, because the joint posterior never asked for the whole arc.

NOTE

Where this fits in practice. MAP mapping with forward models is taught here for its lesson about dependencies, not as production practice. It is a batch algorithm, it needs every scan in memory, it returns a point estimate with no residual uncertainty, and it is only guaranteed to find a local maximum. Modern systems resolve most apparent "conflicting evidence" a different way — by fixing the poses (Chapters 14–16 (Chapter 16)) and fusing submaps — because in real deployments pose error, not cell coupling, is what dominates the artifacts. The cell coupling is real; it is just not usually the biggest lie in the room.

13.6 When two sensors disagree about what "occupied" means

There is a second way to get a wrong map from correct measurements, and it does not need any approximation at all — only two sensors and a naive urge to add.

INTERACTIVE · w13.4

Fusion Fumble

More sensors plus more Bayes does not automatically mean a better map: adding log odds is only valid when every sensor answers the same question.

Run this simulation in the web edition: [#w13.4](/chapters/ch13-occupancy-grids)

The failure has nothing to do with either sensor being wrong. The sonar rides at table height and correctly reports a table. The LiDAR rides at shin height, threads between four six-centimetre legs, and correctly reports free space. Both are telling the truth about their own slice of the world; it is the *question* that differs, and a single per-cell Bayes filter fed both streams silently assumes they answer the same one. Whichever sensor is polled more often wins, so the table's

occupancy probability tracks the beam-count ratio rather than any physical fact.

Thrun's remedy (eq. 9.9) is to keep one grid per modality and combine pessimistically:

$$p(m_i) = \max_k p(m_i^{[k]})$$

This estimator is deliberately biased toward "occupied". That is the correct bias for navigation: a map that hallucinates an obstacle wastes a path, and a map that deletes one wastes a robot. The same logic is why a Nav2 costmap keeps separate obstacle, inflation, and voxel layers rather than one fused probability field.

13.7 Implementation in Rust

Three design decisions before any code. The grid is a flat `Vec<f32>` of log odds with the index math written out, not an `ndarray` — the arithmetic is two multiplications and the visible index is worth more here than the abstraction. Cells store `f32`, because log odds clamped at ± 12 need six significant digits at most, and halving the map halves the cache misses that dominate the inner loop. And the inverse model is a **trait**, so the hand-crafted and learned versions are interchangeable at the call site — which is the entire argument of §9.3, expressed as a type.

RUST `crates/ch13_occgrid/src/lib.rs`

```

1 use nalgebra::Vector2;
2 use pr_core::geom::SE2; // Chapter 3's hand-rolled SE(2)
3
4 /// A cell index in row-major order. A newtype, so `grid.probability(idx)`
5 /// cannot be handed a beam number by mistake.
6 #[derive(Copy, Clone, Debug, PartialEq, Eq)]
7 pub struct GridIdx(pub usize);
8
9 /// An occupancy grid map, stored in log odds.
10 ///
11 /// `log_odds[j * width + i]` is l for the cell whose lower-left corner is at
12 /// `origin + (i, j) * resolution`. Nothing here is generic over the numeric
13 /// type: the map is the hot loop, and `f32` is a decision, not a default.
14 pub struct OccGrid {
15     width: usize,
16     height: usize,
17     /// Metres per cell. 0.05 for a Nav2 costmap, 0.1 for everything in this book.
18     pub resolution: f32,
19     pub origin: Vector2<f32>,
20     /// l = 0.0 ? p = 0.5 ? "nobody has looked here".
21     log_odds: Vec<f32>,
22     /// l_0, the prior in log odds. Subtracted off every update (Table 9.1, line 3).
23     pub l0: f32,
24     /// Symmetric saturation bound. Without it a cell that has seen a wall five
25     /// hundred times needs five hundred contradictions to change its mind, and
26     /// a person who walked past at t = 3 is in the map forever.
27     pub l_clamp: f32,

```

```

28     /// Scratch set, owned by the grid so `integrate_scan` allocates nothing.
29     visited: std::collections::HashSet<usize>,
30 }
31
32 impl OccGrid {
33     pub fn new(width: usize, height: usize, resolution: f32, origin: Vector2<f32>) ->
34         Self {
35         Self { width, height, resolution, origin, log_odds: vec![0.0; width * height],
36             l0: 0.0, l_clamp: 12.0, visited: Default::default() }
37     }
38
39     #[inline]
40     pub fn index(&self, i: usize, j: usize) -> GridIdx { GridIdx(j * self.width + i) }
41
42     #[inline]
43     pub fn len(&self) -> usize { self.log_odds.len() }
44
45     /// World point -> cell coordinates. Returns `None` outside the map rather
46     /// than clamping: a beam leaving the map is not the same as a beam ending
47     /// at its edge, and conflating them paints a fake wall around the border.
48     pub fn world_to_cell(&self, p: Vector2<f32>) -> Option<(usize, usize)> {
49         let c = (p - self.origin) / self.resolution;
50         (c.x >= 0.0 && c.y >= 0.0 && c.x < self.width as f32 && c.y < self.height as f32)
51         .then(|| (c.x as usize, c.y as usize))
52     }
53
54     pub fn cell_center(&self, i: usize, j: usize) -> Vector2<f32> {
55         self.origin + Vector2::new(i as f32 + 0.5, j as f32 + 0.5) * self.resolution
56     }
57
58     /// p = 1 - 1/(1 + exp l), the recovery of eq. (9.6).
59     #[inline]
60     pub fn probability(&self, c: GridIdx) -> f32 { prob_from_log_odds(self.log_odds[c.0]) }
61
62     /// Sigma? H_b(p?) in bits. Starts at exactly `width . height` for a uniform
63     /// prior and falls as the map resolves -- the currency Chapter 24 spends.
64     pub fn entropy(&self) -> f32 {
65         self.log_odds.iter().map(|&l| binary_entropy(prob_from_log_odds(l))).sum()
66     }
67
68     #[inline]
69     pub fn prob_from_log_odds(l: f32) -> f32 { 1.0 - 1.0 / (1.0 + l.exp()) }
70
71     #[inline]
72     pub fn log_odds_from_prob(p: f32) -> f32 { (p / (1.0 - p)).ln() }
73
74     fn binary_entropy(p: f32) -> f32 {
75         if p <= 0.0 || p >= 1.0 { 0.0 } else { -(p * p.log2() + (1.0 - p) * (1.0 - p).log2()) }
76     }
77 }

```

The inverse model as a trait, with Table 9.2 as its first implementor:

RUST crates/ch13_occgrid/src/inverse.rs

```

1 use nalgebra::Vector2;
2 use pr_core::geom::SE2;

```

```

3
4  /// One (range, bearing) ray of a scan -- the inverse model's unit of evidence.
5  #[derive(Copy, Clone, Debug)]
6  pub struct Beam { pub range: f32, pub bearing: f32 }
7
8  /// What one measurement says about one cell, in log odds.
9  ///
10 /// The contract is deliberately `evidence`, not `probability`: implementors
11 /// return the increment of Table 9.1 line 3, `inverse_sensor_model(...) - l_0`,
12 /// already net of their own prior. A learned model's l_0 is whatever the maps it
13 /// trained on happened to imply, and only the model knows it.
14 pub trait InverseSensorModel {
15     fn evidence(&self, cell_center: Vector2<f32>, pose: &SE2, beam: &Beam) -> f32;
16
17     /// True when this model can say nothing beyond `range`, so the caller can
18     /// stop walking the ray. Table 9.1 does not need this; your frame budget does.
19     fn reach(&self, beam: &Beam) -> f32;
20 }
21
22 /// `inverse_range_sensor_model` -- Thrun et al., Table 9.2.
23 pub struct HandCraftedModel {
24     /// Obstacle thickness, metres. Wider => thicker walls, fewer holes, blurrier
doorways.
25     pub alpha: f32,
26     /// Beam opening angle, radians. A LiDAR is a fraction of a degree; a sonar is 15-30°.
27
28     pub beta: f32,
29     pub max_range: f32,
30     pub l_occ: f32,
31     pub l_free: f32,
32     pub l0: f32,
33 }
34
35 impl InverseSensorModel for HandCraftedModel {
36     fn evidence(&self, cell: Vector2<f32>, pose: &SE2, beam: &Beam) -> f32 {
37         let d = cell - pose.translation();
38         let r = d.norm();
39         let phi = wrap_pi(d.y.atan2(d.x) - pose.angle() - beam.bearing);
40
41         // Line 5: beyond the reading, or outside the cone => this beam is silent.
42         if r > self.max_range.min(beam.range + self.alpha / 2.0) || phi.abs() > self.beta
43         / 2.0 {
44             return 0.0;
45         }
46         // Line 7. Note `beam.range`, not `max_range`: see the warning in the text.
47         if beam.range < self.max_range && (r - beam.range).abs() < self.alpha / 2.0 {
48             return self.l_occ - self.l0;
49         }
50         // Line 9.
51         if r <= beam.range { self.l_free - self.l0 } else { 0.0 }
52     }
53
54     fn reach(&self, beam: &Beam) -> f32 { self.max_range.min(beam.range + self.alpha /
55     2.0) }
56 }
57
58 fn wrap_pi(a: f32) -> f32 { a - std::f32::consts::TAU * ((a + std::f32::consts::PI) / std
59 ::f32::consts::TAU).floor() }

```

And the recursion itself. Table 9.1 loops over all cells; we invert the loop and walk the beams. For a 60-beam LiDAR standing in the middle of the Apartment corridor that is the difference between 10,800 inverse-model evaluations per scan and 1,146 — measured, not estimated, by counting the visited set.

RUST crates/ch13_occgrid/src/lib.rs (continued)

```

1 use crate::bresenham::supercover;
2 use crate::inverse::{Beam, InverseSensorModel};
3 use sim::Scan; // Chapter 4's scan type: ranges, bearings, max_range
4
5 impl OccGrid {
6     /// `occupancy_grid_mapping` -- Thrun et al., Table 9.1, restricted to the
7     /// perceptual field by integer ray traversal.
8     pub fn integrate_scan<M: InverseSensorModel>(&mut self, pose: &SE2, scan: &Scan,
9         model: &M) {
10         let Some((ri, rj)) = self.world_to_cell(pose.translation()) else { return };
11         // Cells near the sensor lie on several beams' rasters. Without this set
12         // they would be updated once per beam, over-carving free space into a
13         // hard zero that no later evidence can argue back.
14         self.visited.clear();
15
16         for (&range, &bearing) in scan.ranges.iter().zip(scan.bearings.iter()) {
17             let beam = Beam { range, bearing };
18             let reach = model.reach(&beam);
19             let dir = pose.angle() + bearing;
20             let end = pose.translation() + reach * Vector2::new(dir.cos(), dir.sin());
21             let Some((ei, ej)) = self.world_to_cell(end) else { continue };
22
23             for (i, j) in supercover(ri as isize, rj as isize, ei as isize, ej as isize) {
24                 if i < 0 || j < 0 || i as usize >= self.width || j as usize >= self.
25                     height {
26                         continue;
27                     }
28                 let idx = self.index(i as usize, j as usize);
29                 if !self.visited.insert(idx.0) { continue; }
30
31                 let l = model.evidence(self.cell_center(i as usize, j as usize), pose, &
32                     beam);
33                 // Line 3, and the clamp that keeps the cell revisable.
34                 self.log_odds[idx.0] =
35                     (self.log_odds[idx.0] + l).clamp(-self.l_clamp, self.l_clamp);
36             }
37         }
38     }
39 }

```

RUST crates/ch13_occgrid/src/bresenham.rs

```

1 /// Bresenham's line algorithm, integer arithmetic only, all eight octants.
2 ///
3 /// The *supercover* variant: when the ideal line passes exactly through a
4 /// lattice corner we emit both adjacent cells rather than one. Plain Bresenham
5 /// slips diagonally between two occupied cells, which lets free-space carving
6 /// leak through a wall and shows up as a single-cell hole that a planner will
7 /// happily route a robot through.

```

```

8 pub fn supercover(x0: i32, y0: i32, x1: i32, y1: i32) -> impl Iterator<Item = (i32, i32)>
9 {
10     let (dx, dy) = ((x1 - x0).abs(), -(y1 - y0).abs());
11     let (sx, sy) = (if x0 < x1 { 1 } else { -1 }, if y0 < y1 { 1 } else { -1 });
12     let (mut x, mut y, mut err) = (x0, y0, dx + dy);
13     std::iter::from_fn(move || {
14         if x == x1 && y == y1 { return None; }
15         let out = (x, y);
16         let e2 = 2 * err;
17         if e2 >= dy { err += dy; x += sx; }
18         if e2 <= dx { err += dx; y += sy; }
19         Some(out)
20     })
21     .chain(std::iter::once((x1, y1)))
22 }

```

The learned inverse model of §9.3 is that same trait with a different body — a fixed feature map and a loop of SGD, small enough to print, which is the whole reason to prefer it here over a net:

RUST crates/ch13_occgrid/src/learn.rs

```

1 use nalgebra::SVector;
2 use rand::{rngs::SmallRng, Rng, SeedableRng}; // seeded; never thread_rng()
3 use sensor::BeamModel; // Chapter 10's forward model
4
5 pub const N_FEATURES: usize = 15;
6 type Phi = SVector<f32, N_FEATURES>;
7
8 /// A logistic regression on phi(r, psi, z). Its score is a log odds, so
9 /// `evidence` needs no sigmoid -- which is the one structural reason to prefer
10 /// a logistic link here over anything fancier.
11 pub struct LearnedModel {
12     w: Phi,
13     /// The model's own l_0, read off as its answer about a cell no beam can see.
14     /// Nobody sets it: it is the prior of the maps that were sampled.
15     pub l0: f32,
16 }
17
18 /// `learn_inverse_sensor_model` -- Thrun et al., sec.9.3.2 + sec.9.3.3.
19 pub fn train(fwd: &BeamModel, prior: &MapPrior, seed: u64, epochs: usize) -> LearnedModel
20 {
21     let mut rng = SmallRng::seed_from_u64(seed);
22     let data: Vec<(Phi, f32)> = (0..24_000)
23         .map(|_| {
24             let m = prior.sample(&mut rng); // 1. a map from the map prior
25             let x = m.sample_pose(&mut rng); // 2. a pose inside it
26             let z = fwd.sample(&x, &m, &mut rng); // 3. a reading from the FORWARD
27             model
28             let (r, psi) = m.sample_cell_in_beam_frame(&x, &mut rng);
29             (features(r, psi, z), m.occupancy_at(&x, r, psi) as u8 as f32) // 4. the
30             label
31         })
32         .collect();
33
34     let mut w = Phi::zeros();
35     let mut t = 0usize;

```

```

33     for _ in 0..epochs {
34         for &(phi, y) in shuffled(&data, &mut rng) {
35             // ?J = (sigma(w.phi) - y) phi. That is the whole of eq. (9.20)'s gradient.
36             let p = 1.0 / (1.0 + (-w.dot(&phi)).exp());
37             let lr = 0.06 / (1.0 + t as f32 / 8000.0);
38             w -= lr * (p - y) * phi;
39             t += 1;
40         }
41     }
42
43     // "No information" probe: far beyond a short reading, on the beam axis.
44     let l0 = w.dot(&features(0.92 * fwd.max_range, 0.0, 0.2 * fwd.max_range));
45     LearnedModel { w, l0 }
46 }
47
48 impl InverseSensorModel for LearnedModel {
49     fn evidence(&self, cell: Vector2<f32>, pose: &SE2, beam: &Beam) -> f32 {
50         let (r, psi) = beam.frame(cell, pose, beam);
51         self.w.dot(&features(r, psi, beam.range)) - self.l0
52     }
53     fn reach(&self, _beam: &Beam) -> f32 { self.max_range }
54 }

```

The MAP mapper of Table 9.3, with the two optimizations that make it tractable:

RUST crates/ch13_occgrid/src/map_map.rs

```

1 use sensor::BeamModel; // Chapter 10's forward model, used forwards this time
2
3 /// Which beams each cell can possibly affect.
4 ///
5 /// Built once, on the empty map: occupancy decides where a ray stops, never
6 /// where it goes, so a beam whose free-space raster misses cell `i` can never be
7 /// changed by flipping `i`. That makes this cache exact, not a heuristic.
8 struct Incidence { by_cell: Vec<Vec<u32>> }
9
10 /// `MAP-occupancy-grid-mapping` -- Thrun et al., Table 9.3.
11 ///
12 /// Returns a binary map. There is no residual uncertainty in it, by
13 /// construction; the sensitivity of the log-likelihood to each flip is the
14 /// closest thing available, and it is overconfident because it only inspects
15 /// the mode locally.
16 pub fn map_occupancy_grid_mapping(
17     poses: &[SE2],
18     scans: &[Scan],
19     fwd: &BeamModel,
20     grid: &OccGrid,
21     max_flips: usize,
22 ) -> Vec<bool> {
23     let beams = ConeBeamSet::compile(grid, poses, scans, fwd.beta, fwd.sub_rays);
24     let inc = Incidence::build(&beams, grid.len());
25     let l0 = grid.l0;
26     let mut m = vec![false; grid.len()]; // line 1: start all-free
27
28     for _ in 0..max_flips {
29         // Steepest ascent rather than Table 9.3's index-order sweep: same
30         // objective, same local maximum, but the order in which cells move is

```

```

31 // the order in which they explain the data, which is worth watching.
32 let best = (0..m.len())
33   .map(|c| (c, flip_gain(&beams, &inc, &mut m, fwd, l0, c)))
34   .max_by(|a, b| a.1.total_cmp(&b.1));
35
36 match best {
37     Some((c, g)) if g > 0.0 => m[c] = !m[c],
38     _ => break, // line 2: converged -- no single flip improves the posterior
39 }
40 }
41 m
42 }
43
44 /// Delta log-posterior from flipping cell `c`. Restores `m` before returning.
45 fn flip_gain(
46     beams: &ConeBeamSet, inc: &Incidence, m: &mut [bool],
47     fwd: &BeamModel, l0: f32, c: usize,
48 ) -> f32 {
49     let touched = &inc.by_cell[c];
50     let before: f32 = touched.iter().map(|&b| beams.log_likelihood(b, m, fwd)).sum();
51     m[c] = !m[c];
52     let after: f32 = touched.iter().map(|&b| beams.log_likelihood(b, m, fwd)).sum();
53     let prior_delta = if m[c] { l0 } else { -l0 };
54     m[c] = !m[c];
55     after - before + prior_delta
56 }

```

Finally the fusion rule, which is four lines and one comment:

RUST `crates/ch13_occgrid/src/fusion.rs`

```

1 /// Thrun et al., eq. (9.9): one grid per sensor modality, combined by the most
2 /// pessimistic component.
3 ///
4 /// Deliberately biased toward "occupied". A map that hallucinates an obstacle
5 /// wastes a path; a map that deletes one wastes a robot.
6 pub fn fuse_max(grids: &[OccGrid]) -> Vec<f32> {
7     let n = grids[0].len();
8     (0..n)
9       .map(|c| grids.iter().map(|g| g.probability(GridIdx(c))).fold(0.0f32, f32::max))
10      .collect()
11 }

```

i NOTE

Note which crates are *absent*. There is no `faer`, no `factrs`, no `petgraph` in this chapter: occupancy grid mapping has no linear system to solve and no graph to optimize. It is an array and a for-loop, and that is why it survived twenty-five years of everything else being replaced. `parry2d` appears only inside `sim`, where Chapter 4 built the ray caster that generates the scans.

13.8 Putting it together: what a map is worth

The entropy readout in Map Weaver is not decoration. Treating cells as independent — the same Approximation #1, now used for something benign — the map’s total entropy is

$$H(m) = \sum_i H_b(p_i), \quad H_b(p) = -p \log_2 p - (1-p) \log_2 (1-p)$$

which starts at exactly one bit per cell (10,800 bits for the Apartment at 10 cm) and falls as evidence arrives. Run the widget and watch it drop steeply while Rusty is in new territory and flatten when it is re-walking the corridor: the derivative of that curve is *information gain per second*, and choosing actions to maximize it is precisely what Chapter 24 does. Frontier exploration — drive to the boundary between free and unknown — is the greedy approximation to it.

Entropy also gives you a monitor for free. Individual scans can raise it — a contradicting reading drags a confident cell back toward 0.5, which is the filter working — but in a static world the *trend* is down. A sustained climb means something is disagreeing with the map: a person walking through, a door that has opened, or — far more often — a pose that is wrong.

Which brings us to the confession.

Every algorithm in this chapter assumed an oracle handing us $x_{1:t}$. Turn off **Pose oracle** in Map Weaver and watch what the identical scans, integrated at Rusty’s own dead-reckoned poses, do to the apartment: walls double, corridors bend, and the map degrades into a long-exposure photograph of a moving camera. The cell coupling we spent half this chapter repairing is a real effect worth a factor of a few. Pose error is worth a factor of everything.

That is not a defect in occupancy grids. It is the reason they are usually applied *after* the fact, to a trajectory some other algorithm has already estimated — which is exactly how Nav2 and SLAM Toolbox use them today. Getting that trajectory is the subject of the next six chapters, and it starts by putting the map into the state vector in Chapter 14.

13.9 Exercises

1 FOUNDATION •• Where ℓ_0 enters, and what happens without it

Re-derive D1 with a non-uniform prior $p(m_i) = 0.2$. Show explicitly which step produces the $-\ell_0$ term, then compute what a cell’s log odds are after ten measurements that each return exactly the prior, both with and without the correction. State the drift per scan in nats and in probability.

<details> <summary>Check</summary> $\ell_0 = \ln(0.2/0.8) = -1.3863$. With the correction the cell stays at -1.3863 forever. Without it, $\ell_{10} = 11\ell_0 = -15.25$, i.e. $p = 2.4 \times 10^{-7}$;

the filter has become certain that an unobserved cell is free, on the strength of no evidence at all. </details>

2 FOUNDATION • • • Two cells, one beam, and a query that breaks

Reproduce the four-row joint posterior table in the independence-trap section from the beam mixture of Chapter 10, for cells A and B side by side at range 2 m inside one cone, reading $z = 2.0$, $z_{\max} = 3$. Then do the *collinear* version — A at 1 m and B at 2 m on the same ray — and compare the two coupling structures. Which one does the factored representation get less wrong, and why?

<details> <summary>Check</summary> Collinear: the joint is $(0.0139, 0.0075, 0.9711, 0.0075)$ for $(m_A m_B) = (00, 10, 01, 11)$, with marginals $p(A) = 0.0150$, $p(B) = 0.9786$. The factored form overstates $p(A = 1 \wedge B = 1)$ by about $2\times$ — bad, but nothing like the $24\times$ error on "both free" in the side-by-side case. Collinearity produces a mild correlation (A being occupied *explains away* B); shared membership of one cone produces a hard logical constraint, and hard constraints are what products of marginals cannot express. </details>

3 FOUNDATION • • Entropy is a lower bound in disguise

Show that $H(m) = \sum_i H_b(p_i)$ computed from the factored posterior is an *upper* bound on the entropy of the true joint posterior, and identify the gap. (Hint: the difference is a mutual information, and it is non-negative.) Then explain in one sentence why an information-gain explorer built on this number is systematically over-optimistic about how much it will learn.

4 CONCEPTUAL • • Predict, then verify: which cell flips first?

In w13.2, before pressing play, write down which cells you expect the hill climb to flip first and in what order. Then step it. Use the per-beam bar chart to explain the order you actually got, and say why the very first flip is always worth far more than the tenth. <details> <summary>Check</summary> The steepest flips are the cells lying in the intersection of the most cones at a range matching those cones' readings — near the door posts and the middle of the wall, not the wall's ends. Gain falls off sharply because each flip removes the beams it explains from every subsequent candidate's account, while the ℓ_0 cost of a flip stays constant. </details>

5 CONCEPTUAL • • Predict, then verify: two artifacts of loud evidence

In w13.1, push **evidence strength** to 3.0 and predict two specific artifacts before you look. Then turn on **Person in the corridor** and find the largest clamp $\ell_{\max}$ for which a ghost decays within roughly five seconds of the person leaving. Report the value and explain, from D1, why the decay time is linear in $\ell_{\max}$ and inversely proportional to $|\ell_{\text{free}}|$.

6 PRACTICAL •• Bresenham versus the honest sweep

Implement both `integrate_scan` (beam traversal) and `occupancy_grid_mapping_all_cells` (the literal Table 9.1 loop) and benchmark them on the Apartment at 0.1 m and 0.05 m resolution with a 60-beam LiDAR. Report the speedup and the number of cells each visits. Then construct a sensor for which the two give *different maps*, not just different run times, and explain precisely which line of Table 9.2 is responsible.
 <details> <summary>Hint</summary> Any sensor whose β exceeds the angular spacing between beams. The cone then covers cells that no beam's raster passes through, and only the full sweep reaches them — which is exactly the sonar of w13.2 and w13.4. </details>

7 PRACTICAL •• Max fusion, with a regression test

Implement `fuse_max` for the sonar + LiDAR scene of w13.4 and add a test asserting that the mean occupancy probability over the table footprint stays above 0.55 under max fusion and falls below 0.10 under log-odds summation, for a fixed seed. Then make the test *fail* by changing only the LiDAR's beam count, and explain in the test's comment why that is the correct thing for it to do.

8 PRACTICAL ••• Swap in the learned inverse model

Train the logistic inverse model of §9.3 against your Chapter 10 beam model, implement it as a second `InverseSensorModel`, and run Map Weaver's Apartment tour with each. Score both maps cell-wise against ground truth with the Brier score $\frac{1}{N} \sum_i (p_i - m_i^*)^2$, and report where the learned model wins. Remember to use the learned model's own ℓ_0 in line 3 — and, as a diagnostic, run it once with $\ell_0 = 0$ and measure the drift you get.

13.10 References

Moravec, H. P. and Elfes, A. (1985). High Resolution Maps from Wide Angle Sonar *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, St. Louis, 116–121.

The origin. Probability profiles projected from a 30° sonar cone onto a raster, with overlapping empty volumes reinforcing each other — this chapter's paintbrush metaphor is theirs, forty years early. [link](#)

Elfes, A. (1989). Using Occupancy Grids for Mobile Robot Perception and Navigation *Computer* 22(6), 46–57.

Where the name and the formalism come from: the occupancy grid as a multi-dimensional random field with Bayesian incremental updating from several sensors and viewpoints. doi:10.1109/2.30720

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 9 is this chapter’s spine: Tables 9.1–9.3, the multi-sensor fusion argument of §9.2.1, and the learned-inverse-model construction of §9.3 that D3 turns into a theorem. [link](#)

Hornung, A., Wurm, K. M., Bennewitz, M., Stachniss, C., and Burgard, W. (2013). OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees *Autonomous Robots* 34(3), 189–206.

The 3-D successor, and the reference for log-odds clamping as standard practice: it explicitly represents occupied, free, and unknown, which is the three-state property this chapter argues is the point. Chapter 19 picks it up. doi:10.1007/s10514-012-9321-0

Macenski, S., Martí n, F., White, R., and Clavero, J. G. (2020). The Marathon 2: A Navigation System *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

Nav2, where occupancy grids actually live today. Its layered costmap — separate obstacle, voxel, and inflation layers rather than one fused field — is the production form of this chapter’s max-fusion argument. [link](#)

Nuss, D., Reuter, S., Thom, M., Yuan, T., Krehl, G., Maile, M., Gern, A., and Dietmayer, K. (2018). A Random Finite Set Approach for Dynamic Occupancy Grid Maps with Real-Time Application *The International Journal of Robotics Research* 37(8), 841–866.

What to do when the static-state assumption behind the binary Bayes filter is simply false. Cells become a random finite set with velocity, and the ghost-decay hack this chapter uses becomes a principled filter. doi:10.1177/0278364918775523

van Kempen, R., Lampe, B., Woopen, T., and Eckstein, L. (2021). A Simulation-based End-to-End Learning Framework for Evidential Occupancy Grid Mapping *IEEE Intelligent Vehicles Symposium (IV), Nagoya*.

D3, done at scale and in this decade: a deep inverse sensor model trained on simulated data, with no hand-labelled ground truth, quantifying both first- and second-order uncertainty. [link](#)

Berlenko, T. and Krinkin, K. (2026). Equivalence and Divergence of Bayesian Log-Odds and Dempster’s Combination Rule for 2D Occupancy Grids *arXiv:2602.18872*.

A careful modern comparison of the log-odds update against evidential fusion on simulation, real lidar, and downstream planning — including the finding that which rule looks better depends on how you match the two parameterizations. [link](#)

The SLAM Problem and EKF SLAM

MAPPING AND SLAM · Advanced · 65 min

“

In SLAM, observing a landmark does not just improve the position estimate of this very landmark, but that of other landmarks as well.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 10

Chapter 12 localized Rusty against a map somebody else drew. Chapter 13 drew a map from poses somebody else supplied. Put Rusty in a building nobody has surveyed and both chapters collapse into each other: you cannot localize without a map, and you cannot map without localizing. That circularity is the SLAM problem, and this chapter solves it the way the field first did — by refusing to choose. Put the map *into* the state vector and run the EKF you already have.

What comes out is not "localization plus mapping bolted together". It is one Gaussian over robot and map together, and the interesting part of that Gaussian is its off-diagonal blocks. The correlations are what let a single sighting of a single beacon improve landmarks the robot cannot even see. They are, quite literally, the map's memory of where the robot was when it drew each piece.

Then we perform an autopsy on our own hero. EKF SLAM costs time and memory quadratic in the number of landmarks, and — worse, because you cannot buy your way out of it — it is *provably overconfident*: a linearization, once applied, is welded into Σ_t and the filter has no mechanism for revising it. Those two flaws are the founding charter of everything in Chapter 15 onwards. You should leave this chapter loving the idea and distrusting the filter.

🕒 AFTER THIS CHAPTER YOU CAN

- State the online and full SLAM posteriors, and say exactly which one the EKF computes
- Derive the blockwise EKF SLAM prediction and show why it costs $O(N)$ while the correction costs $\Theta(N^2)$
- Explain why the Kalman gain is dense when the measurement Jacobian is not — and why that is the whole algorithm
- Prove what a SLAM map converges to: relative structure exactly, absolute position never
- Run maximum-likelihood data association with gating, a provisional list, and landmark existence evidence
- Measure a filter's honesty with NEES and a χ^2 band, and diagnose EKF SLAM's built-in overconfidence
- Implement EKFSlam in Rust with a state vector that grows at runtime

📦 ASSUMED

- The EKF and its Jacobian discipline (Chapter 7)
- The velocity motion model and its Jacobians (Chapter 9)
- The range-bearing landmark model $f = (r, \phi, s)$ (Chapter 10)
- Maximum-likelihood association and Mahalanobis gating (Chapter 11)
- Binary Bayes filters in log odds (Chapter 13) — they return here as landmark existence evidence

14.1 A map with nothing to stand on

Rusty rolls into an unlit apartment with a beacon detector and a wheel encoder. The encoder is a liar with a slow accent: each step it reports is off by a percent or two, and those errors are a random walk, so after twenty metres Rusty's idea of where it is has drifted by a metre and its idea of *which way it is pointing* by several degrees. Every beacon it plots goes onto the map at the wrong place, rotated by whatever the heading error happened to be at the moment of sighting.

Localization would fix this — except localization needs the map. Mapping would fix *that* — except mapping needs the poses. The 1999 draft of *Probabilistic Robotics* calls it, memorably, a "chicken-and-egg problem", and the resolution is the one probability theory always offers when two unknowns are entangled: stop estimating them separately. Estimate the joint distribution.

 INTERACTIVE · w14.1**The Correlation Web**

The off-diagonal blocks of Σ are not bookkeeping — they are the map. Delete them and the filter becomes certain of a map that is wrong.

Run this simulation in the web edition: [/chapters/ch14-ekf-slam #w14.1](#)

Watch a full lap before reading on. Three things happen, and each one is a section of this chapter.

The pose ellipse breathes. In the west cluster the beacons pin Rusty down. Out in the open corridor there is nothing to see, prediction runs unopposed, and the purple ellipse swells. That is Chapter 5's "moving smears" with nothing to sharpen it.

The far cluster inherits the drift. Beacons first seen at the east end are born inside an inflated pose, so they are born uncertain — and *correlated*, because their error and Rusty's error are largely the same error. That shared error is the orange in the heatmap's off-diagonal blocks.

Then the web snaps. Rusty returns, sees a west beacon it has not measured in a hundred steps, and the pose uncertainty collapses by a factor of about five in a handful of frames. Landmarks it did not measure move too. Nothing sent them a measurement; Σ did.

 THE ONE SENTENCE VERSION

EKF SLAM is one extended Kalman filter over the joint state $y_t = (x_t, m)$. Its map is not the diagonal of Σ_t — it is the off-diagonal blocks, which is why one observation corrects everything and why the whole thing costs $O(N^2)$.

14.2 Building intuition: the map as a web

Hold on to one picture for the whole chapter. The landmarks are knots. The correlations are threads. The robot is the spider that spun them, and every thread was spun out of the same uncertain pose, which is exactly why the threads exist. Pull one knot — re-observe one landmark after a long excursion — and the entire web moves.

The ablation toggle in the widget above cuts every thread after every update. The marginals survive: each landmark keeps its own 2×2 block, each has an ellipse, the filter runs to completion without complaint. What dies is the filter's ability to know what it does not know. After 400 steps the ablated filter claims a map good to about 6 mm while its landmarks sit, on average, 0.6 m from the truth — a hundredfold lie. The honest filter claims 0.15 m and is wrong by 0.36

m , which is the wrong side of honest but the right order of magnitude.

Two mechanisms cause that collapse, and both are worth naming before the algebra:

1. **Repeated sightings stop being repeated.** Once the pose and a landmark are uncorrelated, every new observation of that landmark looks like fresh, independent evidence about it. Fifty observations from a drifting robot get counted as fifty independent measurements, and the variance falls like $1/k$ — all the way to zero, which is a place no map can go.
2. **Nothing floors it.** In the real filter, a landmark's variance cannot fall below the uncertainty in where the robot *started*. That floor is transmitted entirely through the correlations. Cut them and the floor goes with them.

$y_t = (x_t^\top, m_1^\top, \dots, m_{N_t}^\top)^\top$	The joint SLAM state: pose and map in one vector, of dimension $3 + 2N_t$.
$p(x_t, m \mid z_{1:t}, u_{1:t})$	The online SLAM posterior — current pose and map. This is what the EKF computes.
$p(x_{0:t}, m \mid z_{1:t}, u_{1:t})$	The full SLAM posterior — the whole trajectory and the map. The star of Chapter 15.
$\Sigma_{xx}, \Sigma_{xm}, \Sigma_{mm}$	Pose, pose–map, and map–map blocks of Σ_t . The middle one is the map's memory.
$F_x, F_{x,j}$	Sparse projection matrices selecting the pose (resp. pose plus landmark j) subspace.
N_t	Number of landmarks currently in the state. It grows at run time; that fact has consequences for the type system.
π_k	Mahalanobis distance from an observation to landmark k . Unrelated to the policy π of Part VI.
$\gamma_{\text{gate}}, \chi_{\text{new}}^2$	Association gate and new-landmark threshold, both χ^2 quantiles with 2 degrees of freedom.
ϵ_t	NEES: $(x_t - \mu_{\{t,x\}})^\top \Sigma_{xx}^{-1} (x_t - \mu_{\{t,x\}})$. The consistency instrument.
o_j	Existence log odds for landmark j — Chapter 13's binary Bayes filter, one per landmark.

14.3 The mathematics

14.3.1 Two SLAM problems, one of them ours

The **online SLAM problem** asks for the posterior over the *current* pose and the map:

$$p(x_t, m \mid z_{1:t}, u_{1:t})$$

The **full SLAM problem** asks for the posterior over the *entire trajectory* and the map:

$$p(x_{0:t}, m \mid z_{1:t}, u_{1:t})$$

They are not different problems. They are the same object seen through different apertures, and the relationship is a marginalization:

$$p(x_t, m \mid z_{1:t}, u_{1:t}) = \int \int \cdots \int p(x_{0:t}, m \mid z_{1:t}, u_{1:t}) dx_0 dx_1 \cdots dx_{t-1}$$

Say it in words, because the phrasing is the pivot of Part V: **online SLAM is full SLAM with the past integrated out**. A filter marginalizes each pose the instant it stops being current. That is what makes it constant-time per step, and it is also what makes it unable to change its mind — the information a marginalized pose carried is now smeared into Σ_t in a form nobody can unpick. Chapter 15 is, in one sentence, the decision to stop marginalizing.

SLAM also has a discrete half. When the robot detects something it must decide *which* landmark it detected, and that decision is a correspondence variable c_t^i , not a probability. We do the continuous half first with correspondences handed to us, then earn them.

14.3.2 One state to rule both

Stack the pose and every landmark into a single vector:

$$y_t = \begin{pmatrix} x_t \\ m \end{pmatrix} = (x \ y \ \theta \mid m_{1,x} \ m_{1,y} \mid \cdots \mid m_{N,x} \ m_{N,y})^\top$$

with $\dim y_t = 3 + 2N_t$. Thrun's version carries a signature s_i per landmark as a third component; we keep signatures out of the state and treat them, as most modern front ends do, as an attribute used for association only. The belief is Gaussian:

$$\text{bel}(y_t) = \mathcal{N}(\mu_t, \Sigma_t), \quad \Sigma_t = \begin{pmatrix} \Sigma_{xx} & \Sigma_{xm} \\ \Sigma_{mx} & \Sigma_{mm} \end{pmatrix}$$

Initialization follows Thrun: the robot's starting pose *defines* the world frame, so $\mu_0 = 0$ and $\Sigma_{xx,0}$ is small — in this chapter's lab, $\text{diag}(0.05^2, 0.05^2, 0.02^2)$, a robot that knows where it is to five centimetres and one degree. It is tempting to set it to exactly zero. Don't: a prior the world does not honour makes every consistency number downstream a fiction, and that five-centimetre number will reappear later as a floor no amount of sensing can break through.

The map part starts empty. Landmarks are not initialized at all until they are seen — which is why this filter's state vector has a length that is discovered, not declared.

14.3.3 Prediction: motion is a corner of the matrix

Driving changes the pose and leaves every landmark exactly where it was. Writing that with a projection matrix $F_x = (I_3 \ 0_{3 \times 2N})$ keeps the equations the size of the state while the *work* stays the size of the pose:

$$\bar{\mu}_t = \mu_{t-1} + F_x^\top \delta(u_t, \mu_{t-1}), \quad G_t = I + F_x^\top g_t F_x, \quad \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^\top + F_x^\top R_t F_x$$

Here δ is the exact arc of the velocity model from Chapter 9, and g_t is its 3×3 Jacobian, whose only non-trivial entries are the derivatives with respect to heading:

$$g_t = \begin{pmatrix} 0 & 0 & \frac{v}{\omega} (\cos(\theta + \omega \Delta t) - \cos \theta) \\ 0 & 0 & \frac{v}{\omega} (\sin(\theta + \omega \Delta t) - \sin \theta) \\ 0 & 0 & 0 \end{pmatrix}$$

⚠ WHERE ROBOTS BREAK THIS

Thrun's Table 10.1 prints these two entries with the opposite sign. Differentiate the arc yourself, or check against a central-difference Jacobian, and the signs above are what you get. This is exactly the kind of error that makes a filter *slightly* wrong in a way no test catches until a long run diverges, which is why every Jacobian in this book is checked numerically before it is trusted.

📐 DERIVATION D1 — Why prediction costs $O(N)$, not $O(N^2)$

$$\Sigma_x x \leftarrow G \Sigma_x x G^\top + R, \Sigma_x m \leftarrow G \Sigma_x m, \Sigma_m \text{unchanged}$$

Step 1 — the Jacobian is the identity plus a corner. $G_t = I + F_x^\top g_t F_x$ has g_t 's 3×3 block in the top-left and ones down the rest of the diagonal. Call that top-left block $G_x = I_3 + g_t$.

Step 2 — multiply blockwise. Partition

Σ_{t-1} into its four blocks and write

G_t as the block-diagonal

$\text{diag}(G_x, I_{2N})$. Then

$$G_t \Sigma_{t-1} G_t^\top = \begin{pmatrix} G_x \Sigma_{xx} G_x^\top & G_x \Sigma_{xm} \\ \Sigma_{mx} G_x^\top & \Sigma_{mm} \end{pmatrix}$$

Step 3 — count. The top-left block is two 3×3 products: constant work. The strip $G_x \Sigma_{xm}$ is a 3×3 times a $3 \times 2N$: $\Theta(N)$. The map-map block is

not touched at all. Adding $F_x^T R_t F_x$ writes nine more entries. Total: $\Theta(N)$.

Step 4 — read the middle line again. $\Sigma_{xm} \leftarrow G_x \Sigma_{xm}$. Motion does not erase the pose–map correlations; it *rotates* them, by exactly the rotation the motion applied. The map’s memory of the robot survives every step of dead reckoning. If it did not, the loop closure in the widget above could not do anything, because there would be nothing left to pull on. ■

14.3.4 Landmark birth: inverting the measurement

The first time a feature (r, ϕ) is seen, the landmark has no entry in μ or Σ and one has to be manufactured. Invert the measurement function:

$$\mu_j = \begin{pmatrix} \bar{\mu}_{t,x} + r \cos(\phi + \bar{\mu}_{t,\theta}) \\ \bar{\mu}_{t,y} + r \sin(\phi + \bar{\mu}_{t,\theta}) \end{pmatrix}$$

and push the uncertainty through both Jacobians of that inversion — one with respect to the pose, one with respect to the measurement.

DERIVATION D2 — The birth of a landmark, and its correlations

$$\Sigma_{jj} = G \Sigma_x x G^T + G_z Q G_z^T, \quad \Sigma_j x = G \Sigma_x x$$

Step 1 — the inverse measurement function. Write $\alpha = \phi + \bar{\mu}_{t,\theta}$ and $m_j = (\bar{\mu}_{t,x} + r \cos \alpha, \bar{\mu}_{t,y} + r \sin \alpha)$.

Step 2 — differentiate. With respect to the pose (x, y, θ) and to the measurement (r, ϕ) :

$$G_x = \begin{pmatrix} 1 & 0 & -r \sin \alpha \\ 0 & 1 & r \cos \alpha \end{pmatrix}, \quad G_z = \begin{pmatrix} \cos \alpha & -r \sin \alpha \\ \sin \alpha & r \cos \alpha \end{pmatrix}$$

Step 3 — propagate. The new diagonal block and the new cross-blocks are

$$\Sigma_{jj} = G_x \Sigma_{xx} G_x^T + G_z Q_t G_z^T, \quad \Sigma_{j\bullet} = G_x \Sigma_{x\bullet}$$

where $\Sigma_{x\bullet}$ is the pose’s row of Σ against *everything already in the state*. Read the second equation carefully: a newborn landmark is correlated with every existing landmark, and the entire correlation arrives through the pose. Nothing else connects them. This is the formal statement of "the robot is the spider".

Step 4 — versus Thrun’s table. Table 10.1 instead sets the mean as above and gives the new landmark an effectively infinite prior covariance, letting

the first measurement update do the rest. In the limit $\Sigma_{jj}^{\text{prior}} \rightarrow \infty$ the Kalman update reproduces exactly the expressions in Step 3, because an infinitely uncertain state simply adopts whatever the measurement says. Doing it directly is better behaved numerically, and gets the covariance right on the first frame rather than the second — which matters when something is drawing it. ■

A worked example you can check by hand

Freeze the robot at the origin, facing along $+x$, and let it *know* it is there: $\Sigma_{xx} = 0$. A beacon is detected at $r = 2$ m, $\phi = 0$, with $\sigma_r = 0.1$ m and $\sigma_\phi = 0.05$ rad.

The mean is trivially $\mu_j = (2, 0)$. For the covariance, $\alpha = 0$, so

$$G_z = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \text{ and}$$

$$\Sigma_{jj} = G_z \text{diag}(0.1^2, 0.05^2) G_z^T = \text{diag}(0.01, 4 \times 0.0025) = \text{diag}(0.01, 0.01)$$

The range noise contributes $\sigma_r^2 = 0.01$ along the beam; the bearing noise contributes $r^2 \sigma_\phi^2 = 4 \times 0.0025 = 0.01$ across it. At two metres those happen to be equal, which is a coincidence worth noticing: bearing error scales with range and range error does not, so at ten metres the same beacon would be a long thin sliver, $\text{diag}(0.01, 0.25)$.

Now feed it a second, identical observation. The measurement Jacobian with respect to the landmark alone is

$h_m = \begin{pmatrix} 1 & 0 \\ 0 & 1/2 \end{pmatrix}$ (range differentiates to a unit vector along the beam; bearing to $1/r$ across it), so

$$S = h_m \Sigma_{jj} h_m^T + Q = \text{diag}(0.02, 0.005), \quad K = \Sigma_{jj} h_m^T S^{-1} = \text{diag}(0.5, 1)$$

$$\Sigma_{jj}^+ = (I - K h_m) \Sigma_{jj} = \text{diag}(0.5, 0.5) \text{diag}(0.01, 0.01) = \text{diag}(0.005, 0.005)$$

Two identical measurements halve the variance, exactly as two independent measurements of anything should. Every number here is reproduced by a test in the Rust listing below and by the TypeScript port that runs the widgets.

14.3.5 The dense gain: why one sighting moves the whole map

Now the step that makes SLAM SLAM. The measurement of landmark j is the range-bearing model of Chapter 10, and its Jacobian with respect to the *full* state

is $H_t^i = h_t^i F_{x,j}$, where h_t^i is 2×5 and $F_{x,j}$ is the projection selecting the pose and landmark j . With $\delta = m_j - \mu_{t,xy}$ and $q = \delta^\top \delta$:

$$h_t^i = \frac{1}{q} \begin{pmatrix} -\sqrt{q} \delta_x & -\sqrt{q} \delta_y & 0 & \sqrt{q} \delta_x & \sqrt{q} \delta_y \\ \delta_y & -\delta_x & -q & -\delta_y & \delta_x \end{pmatrix}$$

Five non-zero columns out of $3 + 2N$. The measurement is as local as a measurement gets. And yet:

DERIVATION D3 — A sparse Jacobian, a dense gain

$K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}$ is dense; $\Sigma_t \leftarrow \bar{\Sigma}_t - K_t (\bar{\Sigma}_t H_t^\top)^\top$ writes every entry

Step 1 — the standard EKF update, lifted. Nothing changes from Chapter 7 except the dimension:

$$S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t, \quad K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}, \quad \mu_t = \bar{\mu}_t + K_t (z_t - \hat{z}_t)$$

Step 2 — look at what $\bar{\Sigma}_t H_t^\top$ is. $H_t^\top$ has five non-zero *rows*, so $\bar{\Sigma}_t H_t^\top$ is a linear combination of five *columns* of $\bar{\Sigma}_t$ — the three pose columns and the two columns of landmark j . Those columns are not sparse. Column x of $\bar{\Sigma}_t$ has an entry for every landmark correlated with the pose, which after a few minutes of driving is all of them.

Step 3 — hence K_t is dense. Row k of K_t is non-zero whenever landmark k is correlated with the pose or with landmark j . The 2-vector innovation about *one* beacon therefore produces a correction to *every* coordinate in the state. That is not a side effect; it is the algorithm. The correlations act as a distribution network, and the pose is the hub.

Step 4 — and the covariance update is quadratic. Written efficiently,

$$\Sigma_t = \bar{\Sigma}_t - K_t (\bar{\Sigma}_t H_t^\top)^\top$$

which is a rank-2 downdate of an $n \times n$ matrix: every one of the $(3 + 2N)^2$ entries is read and written. There is no honest way around this in moments form. You may exploit symmetry for a factor of two; you may not change the exponent, because the update genuinely *does* change every entry. A loop closure is this same mechanism with a large, highly informative innovation — one observation, one $O(N^2)$ update, and a map that visibly heals. ■

14.3.6 What converges, and what never does

Take the widget's map after two laps and read the numbers off. The west cluster's landmarks settle at about $\sigma = 0.053$ m in each coordinate. The east cluster's *cross-corridor* uncertainty stalls between 0.25 and 0.3 m and stays there no matter how many times Rusty drives past.

Neither number is a failure. Both are theorems.

DERIVATION D4 — Dissanayake's three facts, proved in one dimension

$\rho \rightarrow 1$, $\text{Var}(m) \rightarrow \text{Var}(x_0)$: relative structure converges, absolute never does

Take the smallest possible SLAM problem: a robot at position x and one landmark at position m , both on a line. The robot knows its own position to σ_0^2 — that is the prior that defines the frame. The sensor measures the *relative* displacement $z = m - x + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma_z^2)$. Note that this is the only kind of measurement a robot ever gets: no sensor reports absolute position.

Step 1 — birth. The first measurement initializes $\hat{m} = \hat{x} + z$. Propagating uncertainty through that sum,

$$\Sigma = \begin{pmatrix} \sigma_0^2 & \sigma_0^2 \\ \sigma_0^2 & \sigma_0^2 + \sigma_z^2 \end{pmatrix}$$

The off-diagonal is σ_0^2 and it is there for a reason: whatever error the robot has in x is copied verbatim into m .

Step 2 — update with $H = (-1, 1)$, holding the robot still so that motion cannot muddy the picture. Then $\Sigma H^\top = (\sigma_{xm} - \sigma_{xx}, \sigma_{mm} - \sigma_{xm})^\top = (0, \sigma_z^2)^\top$ — the pose entry is exactly zero, and it stays zero, because the update multiplies it by itself. So $K = (0, \sigma_z^2/S)^\top$, and the entire first row and column of Σ pass through the update **unchanged**. The robot learns nothing about where it is. Of course it doesn't: a relative measurement cannot locate you.

Step 3 — what does shrink. Let $d = \text{Var}(m - x) = \sigma_{mm} - 2\sigma_{xm} + \sigma_{xx}$, the variance of the quantity actually measured. The scalar update gives $d \leftarrow d \sigma_z^2 / (d + \sigma_z^2)$, so after k measurements $d \approx \sigma_z^2 / k \rightarrow 0$. The *relative* map becomes exact.

Step 4 — take the limit. With $\sigma_{xx} = \sigma_{xm} = \sigma_0^2$ fixed and $d \rightarrow 0$,

$$\sigma_{mm} \longrightarrow 2\sigma_{xm} - \sigma_{xx} = \sigma_0^2, \quad \rho = \frac{\sigma_{xm}}{\sqrt{\sigma_{xx}\sigma_{mm}}} \longrightarrow 1$$

So: **(i)** landmark variances decrease monotonically, **(ii)** correlations grow

toward 1, and (iii) the landmark's variance is bounded below by the *initial vehicle variance* σ_0^2 and can never beat it.

Step 5 — why letting the robot drive changes nothing. The bound survives motion, and the information form says why in one line. The unobservable direction is $v = (1, 1)^\top$ — slide robot and landmark together and every measurement is unchanged — and $Hv = 0$, so a measurement update $\Omega \leftarrow \Omega + H^\top H / \sigma_z^2$ leaves $v^\top \Omega v$ exactly as it was, while a motion update only ever *removes* information. At birth $v^\top \Omega v = 1/\sigma_0^2$, and Cauchy–Schwarz with $\Sigma = \Omega^{-1}$ gives $1 = (e_2^\top v)^2 \leq \text{Var}(m) (v^\top \Omega v)$, hence $\text{Var}(m) \geq \sigma_0^2$ forever. Dissanayake et al. proved the general version in 2001; this is the whole idea in one dimension. ■

Now the two numbers make sense. The west cluster's floor of 0.053 m is $\sigma_0 = 0.05$ m, the prior on the starting pose, plus a little measurement noise: the map cannot be located better than the frame it is expressed in. The east cluster's floor of ≈ 0.25 m is the *other* half of the same statement, because the initial pose prior includes a heading uncertainty of $\sigma_{\theta,0} = 0.02$ rad, and a heading error is a lever: at ten metres of lever arm, $0.02 \times 10 = 0.2$ m of unremovable cross-corridor error. Run the widget with the correlation threads on and you can watch this happen: the east block goes strongly correlated with the pose heading and stays there forever.

NOTE

This is why every modern SLAM system reports poses *relative to* something and treats the global frame as a gauge freedom to be fixed by convention. Chapter 15 makes the gauge explicit as a prior factor on the first pose; Chapter 16 exploits it by optimizing a pose graph that is only ever constrained by relative measurements.

14.3.7 Earning the correspondences

Everything so far assumed a helpful oracle whispering *that is beacon 5*. Real front ends supply (r, ϕ) and a shrug. The classical answer, and Thrun's Table 10.2, is incremental maximum likelihood.

DERIVATION D5 — ML association, gating, and why min-Mahalanobis is the right rule

$$j = \arg \min_k \pi_k, \text{ with } \pi_k = (z^-)^\top (S)^{-1} (z^-)$$

Step 1 — the likelihood of an observation under a hypothesis. If the observation came from landmark k , then under the linearized model $z_t \sim \mathcal{N}(\hat{z}_t^k, S_t^k)$, so

$$-2 \log p(z_t \mid c_t = k) = \underbrace{(z_t - \hat{z}_t^k)^\top (S_t^k)^{-1} (z_t - \hat{z}_t^k)}_{\pi_k} + \log \det S_t^k + \text{const}$$

Step 2 — when the log-det can be dropped. Maximizing the likelihood over k means minimizing $\pi_k + \log \det S_t^k$. Implementations almost always drop the second term, which is exactly right when all candidates have comparable innovation covariances and quietly wrong when they do not: a landmark with a huge S has a wide, shallow gate and will win the π_k contest against a well-known landmark that is genuinely closer. Keep the log-det if your map mixes fresh and mature landmarks, which it always does.

Step 3 — gating. π_k is distributed χ^2 with 2 degrees of freedom under the hypothesis that the association is correct, so thresholds are quantiles, not tuning knobs: $\gamma_{\text{gate}} = 9.21$ is the 99% quantile, $\chi_{\text{new}}^2 = 13.82$ the 99.9%. Between them lies a no-man's-land: too far to trust, too close to call new. Throw those observations away. Discarding information is the cheapest insurance a filter can buy, because a wrong association is $O(N^2)$ work spent making the map worse.

Step 4 — provisional landmarks and existence evidence. A detection that gates to nothing does *not* immediately enter the state. It goes on a provisional list — a shadow map that no update ever touches — and is promoted only after n_{prom} consistent sightings. Clutter does not repeat in the same place, so it never gets promoted. Once promoted, each landmark carries an existence log odds o_j , incremented when observed and decremented when it *should* have been observed and was not, exactly the binary Bayes filter of Chapter 13 with the cell index replaced by a landmark index. Landmarks whose evidence sinks below a floor are deleted, rows and columns and all. ■

The decisive property of all of this — and the reason Chapter 17 exists — is that the decision is **hard**. Table 10.2 picks one correspondence, applies the update, and moves on. There is no residual probability over alternatives, no mechanism to revisit the choice when the tenth observation makes it look silly. The filter does not merely make association errors; it *cannot recover from them*.

The classical alternative is multi-hypothesis tracking: carry a mixture of Gaussians, one component per surviving association history, and prune. It is correct and it is unaffordable, because the number of histories grows exponentially

and each component costs $O(N^2)$ to maintain. Chapter 17 gets the same effect for a bearable price by making each *particle* carry its own association history, and Chapter 15 gets it by making the association a variable you can go back and change.

14.4 The algorithms

ALGORITHM EKF_SLAM_known_correspondences($\mu_{-}\{t-1\}$, $\Sigma_{-}\{t-1\}$, u_t , z_t , c_t)
 COST prediction $O(N)$; each observation update $\Theta(N^2)$
in previous belief, control, features with their landmark ids
out μ_t , Σ_t

```

1:  $F_x = (I_3 \ 0_{3 \times 2N})$ 
2:  $\bar{\mu}_t = \mu_{t-1} + F_x^T \delta(u_t, \mu_{t-1}, \theta)$ 
3:  $G_t = I + F_x^T g_t F_x$ 
4:  $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + F_x^T R_t F_x$ 
5: for all observed features  $z_t^i = (r_t^i \ \phi_t^i)^T$  do
6:    $j = c_t^i$ 
7:   if landmark  $j$  never seen before then initialize it by D2 and continue
8:    $\delta = (\bar{\mu}_{j,x} - \bar{\mu}_{t,x}, \bar{\mu}_{j,y} - \bar{\mu}_{t,y})^T$ ,  $q = \delta^T \delta$ 
9:    $\hat{z}_t^i = (\sqrt{q}, \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta})^T$ 
10:   $H_t^i = h_t^i F_{x,j}$ 
11:   $K_t^i = \bar{\Sigma}_t [H_t^i]^T (H_t^i \bar{\Sigma}_t [H_t^i]^T + Q_t)^{-1}$ 
12:   $\bar{\mu}_t = \bar{\mu}_t + K_t^i (z_t^i - \hat{z}_t^i)$ 
13:   $\bar{\Sigma}_t = (I - K_t^i H_t^i) \bar{\Sigma}_t$ 
14: endfor
15: return  $\mu_t = \bar{\mu}_t$ ,  $\Sigma_t = \bar{\Sigma}_t$ 
```

Line 13 is the entire cost of the algorithm and line 12 is the entire point of it. Note that the observations are folded in **sequentially**, each against the covariance the previous one produced; Thrun's table sums the corrections instead, which is equivalent only to first order and slightly worse in practice.

Unknown correspondence replaces line 6 with a search:

ALGORITHM EKF_SLAM($\mu_{-}\{t-1\}$, $\Sigma_{-}\{t-1\}$, $N_{-}\{t-1\}$, u_t , z_t)
 COST + $O(N)$ gate evaluations per observation, each a 2×2 solve
in previous belief, current map size, control, bare features
out μ_t , Σ_t , N_t

```

1: lines 1–4 of EKF_SLAM_known_correspondences
2: for all observed features  $z_t^i$  do
3:   for  $k = 1$  to  $N_t$  do
4:      $\hat{z}_t^k, H_t^k, S_t^k = H_t^k \bar{\Sigma}_t [H_t^k]^\top + Q_t$ 
5:      $\pi_k = (z_t^i - \hat{z}_t^k)^\top (S_t^k)^{-1} (z_t^i - \hat{z}_t^k)$ 
6:   endfor
7:    $j = \arg \min_k \pi_k$ 
8:   if  $\pi_j < \gamma_{\text{gate}}$  then update against landmark  $j$  (lines 8–13 above)
9:   else if  $\pi_j < \chi_{\text{new}}^2$  then discard  $z_t^i$ 
10:  else add  $z_t^i$  to the provisional list; promote after  $n_{\text{prom}}$  sightings
11: endfor
12:  $o_j \ += \Delta^+$  for each landmark observed;  $o_j \ -= \Delta^-$  for each expected and
    missed
13: retire every landmark with  $o_j < o_{\text{min}}$  (delete its rows and columns)
14: return  $\mu_t, \Sigma_t, N_t$ 

```

INTERACTIVE · w14.4

The Poisoned Web

Association errors do not average out. In a filter they are structural: one wrong claim is written into Σ and can never be withdrawn.

Run this simulation in the web edition: [/chapters/ch14-ekf-slam #w14.4](#)

14.5 The autopsy

EKF SLAM was the field’s crown jewel for fifteen years and is, in 2026, taught rather than deployed. The authors’ own draft of *Probabilistic Robotics* says the quiet part: “the value of the EKF SLAM algorithm presented in this chapter is mostly historical.” Two flaws did it, and they are different in kind. One is an engineering constraint. The other is a lie.

14.5.1 Flaw (a): the cost is quadratic, and the constant does not save you

INTERACTIVE · w14.3

The Growth Meter

N^2 is not a footnote. Measured in your own browser, EKF SLAM's update cost curves away from linear before the map has fifty landmarks.

Run this simulation in the web edition: [/chapters/ch14-ekf-slam #w14.3](/chapters/ch14-ekf-slam/#w14.3)

D3 established that each observation update writes every entry of a $(3 + 2N)^2$ matrix. The widget above measures it rather than asserting it, in your browser, on the same EkfSlam implementation the other three widgets run. Fit a quadratic and extrapolate:

Landmarks N	$\dim \Sigma$	Dense Σ , f64	Multiply-adds per observation
10^2	203	330 kB	8×10^4
10^3	2 003	32 MB	8×10^6
10^4	20 003	3.2 GB	8×10^8
10^5	200 003	320 GB	8×10^{10}

A visual SLAM front end produces 10^5 features before it has left the building. At that size the covariance does not fit in a datacentre node, and one observation update means streaming those 320 GB through the arithmetic units — tens of seconds at the memory bandwidth of a good server, for a single beacon sighting, of which the robot produces dozens per second. The same draft caps EKF SLAM at “relatively scarce maps with less than 1,000 features”, and that is generous.

There is a second, crueller squeeze hidden in the table. Sparse maps make data association *harder* — fewer, more ambiguous landmarks — which is precisely the regime where the hard ML decision of D5 is least reliable. The algorithm needs sparse maps to be affordable and dense maps to be reliable. It cannot have both.

The 2000s produced a whole literature of band-aids for this: compressed and local-map EKFs that defer the global update, submap architectures such as ATLAS and hierarchical SLAM that keep each filter small and stitch the pieces later, and decoupled variants that approximate the correlations they cannot afford to store. They work, they are ingenious, and this book skips them, because the next two chapters remove the exponent instead of amortizing it. If you meet one in a legacy codebase, read it as a very clever way of pretending the covariance is block-diagonal — which is exactly the ablation toggle in w14.1, applied on purpose and with care.

14.5.2 Flaw (b): the filter is provably overconfident

This one is not about speed, and buying a bigger computer does not touch it.

 INTERACTIVE · w14.2**The Consistency Autopsy**

A small reported covariance is not evidence of a good estimate. EKF SLAM grows confident and wrong at the same time.

Run this simulation in the web edition: `/chapters/ch14-ekf-slam #w14.2`

The setup is deliberately rigged in the filter's favour. Correspondences are handed over. The process noise the filter assumes is *exactly* the noise the simulator applies — not a tuned approximation of it, the same numbers. The prior is calibrated: the true initial pose is drawn from it. Under those conditions a correct estimator has

$$\mathbb{E}[\epsilon_t] = \mathbb{E}[(x_t - \mu_{t,x})^\top \Sigma_{xx}^{-1} (x_t - \mu_{t,x})] = \dim(x_t) = 3$$

and $\epsilon_t \sim \chi_3^2$. Averaging 40 independent runs shrinks the 95% acceptance band to roughly [2.29, 3.81]. Short loops sit inside it. Long loops leave it and stay out.

 DERIVATION D6 — Where the confidence comes from

linearization at the estimated heading manufactures information about an unobservable direction

Step 1 — the only remaining approximation. Everything else has been made exact by construction, so whatever breaks must be the Taylor expansion.

Step 2 — the heading error is a rotation. Suppose the filter's heading is wrong by $\tilde{\theta}$. The predicted bearing to every landmark is then wrong by approximately $\tilde{\theta}$, and the predicted displacement to each is the true displacement rotated by $\tilde{\theta}$. The Jacobians h_t^i are evaluated at that rotated geometry.

Step 3 — a rotation looks like information. The filter's linear model has no way to represent "my whole map may be rotated". It sees a set of bearing residuals that are consistent with each other and concludes it has measured something. Part of what it credits as fresh metric information is actually the signature of its own heading error, and Σ shrinks accordingly.

Step 4 — the observability argument. The global position and heading of a SLAM solution are unobservable: any rigid transform of (pose, map) explains the same relative measurements equally well. A correct estimator must keep three unobservable directions with undiminished uncertainty forever. Linearizing at *different, wrong* headings at different times makes

the filter’s implicit constraint sets inconsistent with one another, so the intersection is smaller than the truth — the filter gains information along directions where no information exists.

Step 5 — it can never be repaired. This is the structural half. Each update writes its linearization into Σ_t and then the linearization point is thrown away. Ten steps later, when the pose estimate is much better and the old Jacobian is visibly wrong, there is nothing to recompute: the filter has kept only the consequences. Julier and Uhlmann’s 2001 counterexample makes this concrete with a robot that does not move at all — repeated observation of a single landmark from a stationary vehicle *reduces* the heading variance, which is provably impossible, since a stationary robot observing one point learns nothing about its own orientation. ■

The second chart in the autopsy shows Step 5 as it happens: the reported σ_θ drifts below the heading error the robot is actually making, and once it does, every landmark mapped afterwards is placed with false precision and the gates used to recognize old landmarks become too small. That last consequence is visible in the Poisoned Web above as *duplicate* landmarks — the filter failing to recognize its own beacons after the blind crossing, because its confidence has outrun its accuracy. Flaw (b) manufactures work for flaw (a).

Two exits exist, and the book takes both.

Better linearization. Choose *where* to linearize so that the unobservable directions stay unobservable. First-estimates-Jacobian and observability-constrained EKF freeze the linearization point per state so the filter cannot accumulate inconsistent constraint sets; the invariant EKF of Chapter 7 achieves the same thing more elegantly by linearizing in an error coordinate whose dynamics do not depend on the estimate at all. This helps a great deal and does not fully cure the disease, because the filter still marginalizes.

Stop marginalizing. Keep the past poses, keep the raw measurements, and re-linearize everything whenever the estimate improves. That is Chapter 15, and it is the reason the last two decades of SLAM look the way they do.

14.6 Implementation in Rust

The first thing this filter teaches is a lesson from the type system. Chapter 6’s Kalman filter was `Kf<const N: usize, const U: usize, const M: usize>`: dimensions in the type, allocation on the stack, and the compiler checking that a 3×3 never meets a 5×5 . That is impossible here. N_t is *discovered at run time*, so the state must be heap-allocated and dynamically sized. The type system just told us something true about SLAM: a map is not a fixed-size object.

RUST `crates/ch14_ekfslam/src/lib.rs`

```

1 use nalgebra::{DMatrix, DVector, Matrix2, Matrix2x3, Matrix3};
2 use motion::VelocityCmd;    // Ch. 9
3 use sensor::Feature;        // Ch. 10:  $f = (r, \phi[, s])$ 
4
5 /// Where landmark `j` starts in the state vector.
6 #[inline]
7 pub const fn landmark_index(j: usize) -> usize {
8     3 + 2 * j
9 }
10
11 pub struct SlamConfig {
12     /// Per-metre and per-radian noise, in the *body* frame. The filter rotates
13     /// it into the world frame; that rotation is the only place heading enters
14     ///  $R_t$ , and it is the reason a heading error contaminates everything.
15     pub along_track: f64,
16     pub cross_track: f64,
17     pub heading_per_metre: f64,
18     pub heading_per_radian: f64,
19     ///  $Q_t$ , as standard deviations.
20     pub sigma_r: f64,
21     pub sigma_phi: f64,
22     ///  $\chi^2$  quantiles with 2 dof: 9.21 is 99%, 13.82 is 99.9%.
23     pub gate_chi2: f64,
24     pub new_chi2: f64,
25     pub promote_after: u32,
26     pub retire_below: f64,
27 }
28
29 pub struct EkfSlam {
30     ///  $[x \ y \ \theta \mid m_0? \ m_0? \mid m_1? \ m_1? \mid \dots]$  -- length  $3 + 2N$ , and  $N$  grows.
31     mu: DVector<f64>,
32     sigma: DMatrix<f64>,
33     /// Ground-truth labels for bookkeeping only; never read by the filter.
34     labels: Vec<i32>,
35     /// Ch. 13's binary Bayes filter, one instance per landmark.
36     existence: Vec<f64>,
37     provisional: Vec<Candidate>,
38     /// Sigma entries written since construction -- the measured  $\Theta(N2)$ .
39     pub entries_touched: u64,
40     pub cfg: SlamConfig,
41 }
42
43 pub enum Association {
44     Matched { landmark: usize, nis: f64 },
45     Born { landmark: usize },
46     Candidate { index: usize },
47     Rejected { nis: f64 },
48 }
49
50 impl EkfSlam {
51     pub fn n_landmarks(&self) -> usize {
52         (self.mu.len() - 3) / 2
53     }
54 }

```

Prediction is D1 written literally. The projection matrices F_x never appear: forming them would turn an $O(N)$ step into an $O(N^2)$ one, which is the single most common way a textbook implementation of this algorithm becomes accidentally

slow.

RUST crates/ch14_ekfslam/src/predict.rs

```

1  impl EkfSlam {
2      /// Table 10.1, lines 2-5 -- but blockwise, so the cost is O(N).
3      pub fn predict(&mut self, u: &VelocityCmd) {
4          let (v, w, dt) = (u.v, u.omega, u.dt);
5          let th = self.mu[2];
6          let nt = th + w * dt;
7
8          // Exact arc integration (Ch. 9). The *mean* goes through g, never
9          // through the Jacobian -- a mistake that costs accuracy on every turn.
10         let (dx, dy, g02, g12) = if w.abs() < 1e-6 {
11             (v * th.cos() * dt, v * th.sin() * dt, -v * th.sin() * dt, v * th.cos() * dt)
12         } else {
13             let r = v / w;
14             (
15                 -r * th.sin() + r * nt.sin(),
16                 r * th.cos() - r * nt.cos(),
17                 r * (nt.cos() - th.cos()),
18                 r * (nt.sin() - th.sin()),
19             )
20         };
21         self.mu[0] += dx;
22         self.mu[1] += dy;
23         self.mu[2] = wrap_pi(nt);
24
25         let gx = Matrix3::new(1.0, 0.0, g02, 0.0, 1.0, g12, 0.0, 0.0, 1.0);
26         let r_t = self.process_noise(u, th);
27         let n = self.mu.len();
28
29         // M = G? . Sigma[0..3, ..] against the *old* Sigma. Three rows, N columns.
30         let m = &gx * self.sigma.rows(0, 3);
31
32         // Sigma_xx <- (G? Sigma_xx) G?^T + R_t
33         let xx: Matrix3<f64> = Matrix3::from(m.fixed_view::<<3, 3>(0, 0)) * gx.transpose()
34             + r_t;
35
36         // Sigma_xm <- G? Sigma_xm, and its mirror. The map-map block is never read:
37         // motion cannot tell you anything new about where the walls are.
38         for c in 3..n {
39             for i in 0..3 {
40                 self.sigma[(i, c)] = m[(i, c)];
41                 self.sigma[(c, i)] = m[(i, c)];
42             }
43         }
44         self.sigma.fixed_view_mut::<<3, 3>(0, 0).copy_from(&xx);
45     }
46
47     /// R_t: body-frame noise for this command, rotated into the world frame.
48     fn process_noise(&self, u: &VelocityCmd, theta: f64) -> Matrix3<f64> {
49         let d = u.v.abs() * u.dt;
50         let turn = u.omega.abs() * u.dt;
51         let (sa, sc) = (self.cfg.along_track * d, self.cfg.cross_track * d);
52         let st = self.cfg.heading_per_metre * d + self.cfg.heading_per_radian * turn;
53         let (c, s) = (theta.cos(), theta.sin());
54         let (a2, c2) = (sa * sa, sc * sc);
55         Matrix3::new(
56             c * c * a2 + s * s * c2, c * s * (a2 - c2), 0.0,
57             c * s * (a2 - c2), s * s * a2 + c * c * c2, 0.0,

```

```

57         0.0,           0.0,           st * st,
58     )
59 }
60 }

```

The correction is D3 written literally, and the shape of the code is the complexity argument: one $O(N)$ loop to build $\bar{\Sigma}H^T$, one 2×2 solve, and one $O(N^2)$ loop that no amount of cleverness removes.

RUST crates/ch14_ekfslam/src/correct.rs

```

1  impl EkfSlam {
2      /// One landmark observation, folded in. Table 10.1 lines 12-20.
3      pub fn update_landmark(&mut self, j: usize, f: &Feature) -> UpdateInfo {
4          let n = self.mu.len();
5          let idx = [0, 1, 2, landmark_index(j), landmark_index(j) + 1];
6          let (z_hat, h) = self.expected(j); // h is 2*5, not 2*n
7
8          let innov = nalgebra::Vector2::new(f.r - z_hat[0], wrap_pi(f.phi - z_hat[1]));
9
10         // PHT = Sigma H^T, built from five columns of Sigma. O(N) -- and the reason the
11         // gain is dense: those five columns are not.
12         let mut pht = DMatrix::<f64>::zeros(n, 2);
13         for r in 0..n {
14             for (p, &c) in idx.iter().enumerate() {
15                 pht[(r, 0)] += self.sigma[(r, c)] * h[(0, p)];
16                 pht[(r, 1)] += self.sigma[(r, c)] * h[(1, p)];
17             }
18         }
19
20         let mut s = Matrix2::zeros();
21         for a in 0..2 {
22             for b in 0..2 {
23                 s[(a, b)] = (0..5).map(|p| h[(a, p)] * pht[(idx[p], b)]).sum();
24             }
25         }
26         s[(0, 0)] += self.cfg.sigma_r.powi(2);
27         s[(1, 1)] += self.cfg.sigma_phi.powi(2);
28         let s_inv = s.try_inverse().expect("innovation covariance is singular");
29
30         let k = &pht * s_inv; // n * 2, dense
31         let correction = &k * innov;
32         self.mu += correction;
33         self.mu[2] = wrap_pi(self.mu[2]);
34
35         // Sigma <- Sigma - K (Sigma H^T)^T. Every entry. This single statement is why
36         EKF
37         // SLAM does not scale, and there is no version of it that is cheaper:
38         // the update genuinely changes all n2 numbers.
39         self.sigma -= &k * pht.transpose();
40         symmetrize(&mut self.sigma);
41         self.entries_touched += (n * n) as u64;
42
43         UpdateInfo { innovation: innov, s, nis: (innov.transpose() * s_inv * innov)[0] }
44     }
45
46     /// ? and the 2*5 Jacobian of eq. (10.21), against the five state
47     /// dimensions the observation actually touches.

```



```

47     fn expected(&self, j: usize) -> (nalgebra::Vector2<f64>, nalgebra::Matrix2x5<f64>) {
48         let b = landmark_index(j);
49         let (dx, dy) = (self.mu[b] - self.mu[0], self.mu[b + 1] - self.mu[1]);
50         let q = (dx * dx + dy * dy).max(1e-9);
51         let sq = q.sqrt();
52         let z = nalgebra::Vector2::new(sq, wrap_pi(dy.atan2(dx) - self.mu[2]));
53         let h = nalgebra::Matrix2x5::new(
54             -dx / sq, -dy / sq, 0.0, dx / sq, dy / sq,
55             dy / q, -dx / q, -1.0, -dy / q, dx / q,
56         );
57         (z, h)
58     }
59 }

```

Landmark birth is the only place the state changes shape, and nalgebra's dynamic matrices make the surgery explicit — which is good, because growing a covariance is a place to be careful rather than clever.

RUST crates/ch14_ekfslam/src/lib.rs (continued)

```

1  impl EkfSlam {
2      /// D2: invert the measurement, push the uncertainty through both Jacobians,
3      /// and grow mu and Sigma by two. The new landmark is born correlated with
4      /// everything already in the state -- all of it through the pose.
5      pub fn init_landmark(&mut self, f: &Feature, label: i32) -> usize {
6          let n_old = self.mu.len();
7          let a = wrap_pi(f.phi + self.mu[2]);
8          let (ca, sa) = (a.cos(), a.sin());
9
10         let g_x = Matrix2x3::new(1.0, 0.0, -f.r * sa, 0.0, 1.0, f.r * ca);
11         let g_z = Matrix2::new(ca, -f.r * sa, sa, f.r * ca);
12         let q = Matrix2::new(self.cfg.sigma_r.powi(2), 0.0, 0.0, self.cfg.sigma_phi.powi
(2));
13
14         let cross = &g_x * self.sigma.rows(0, 3);           // 2 * n_old
15         let mm = Matrix2::from(cross.fixed_view::<2, 3>(0, 0)) * g_x.transpose()
+ &g_z * q * g_z.transpose();
16
17         self.mu = self.mu.clone().insert_rows(n_old, 2, 0.0);
18         self.mu[n_old] = self.mu[0] + f.r * ca;
19         self.mu[n_old + 1] = self.mu[1] + f.r * sa;
20
21         self.sigma = self.sigma.clone().insert_rows(n_old, 2, 0.0).insert_columns(n_old,
2, 0.0);
22         for c in 0..n_old {
23             for i in 0..2 {
24                 self.sigma[(n_old + i, c)] = cross[(i, c)];
25                 self.sigma[(c, n_old + i)] = cross[(i, c)];
26             }
27         }
28         self.sigma.fixed_view_mut::<2, 2>(n_old, n_old).copy_from(&mm);
29
30         self.labels.push(label);
31         self.existence.push(1.0);
32         self.n_landmarks() - 1
33     }
34 }
35
36 /// Retire a landmark: delete its two rows, its two columns, and its

```

```

37  /// bookkeeping. The one operation a factor graph never needs.
38  pub fn retire(&mut self, j: usize) {
39      let b = landmark_index(j);
40      self.mu = self.mu.clone().remove_rows(b, 2);
41      self.sigma = self.sigma.clone().remove_rows(b, 2).remove_columns(b, 2);
42      self.labels.remove(j);
43      self.existence.remove(j);
44  }
45  }

```

Association uses `statsrs` for the thresholds rather than magic constants, because a gate is a hypothesis test and deserves to look like one:

RUST `crates/ch14_ekfslam/src/correct.rs (association)`

```

1  use statsrs::distribution::{ChiSquared, ContinuousCDF};
2
3  impl SlamConfig {
4      /// gamma_gate and chi2_new as honest quantiles of chi2(2).
5      pub fn with_gates(mut self, gate_p: f64, new_p: f64) -> Self {
6          let chi2 = ChiSquared::new(2.0).expect("2 dof");
7          self.gate_chi2 = chi2.inverse_cdf(gate_p); // 0.99 -> 9.210
8          self.new_chi2 = chi2.inverse_cdf(new_p); // 0.999 -> 13.816
9          self
10     }
11 }
12
13 impl EkfSlam {
14     /// Table 10.2: maximum-likelihood association with gating and a
15     /// provisional list. The decision is hard, and it is permanent.
16     pub fn correct(&mut self, features: &[Feature], t: u32) -> Vec<Association> {
17         features
18             .iter()
19             .map(|f| {
20                 let (best, pi) = (0..self.n_landmarks())
21                     .map(|k| (k, self.mahalanobis(k, f)))
22                     .min_by(|a, b| a.1.total_cmp(&b.1))
23                     .unwrap_or((usize::MAX, f64::INFINITY));
24
25                 if pi < self.cfg.gate_chi2 {
26                     let info = self.update_landmark(best, f);
27                     Association::Matched { landmark: best, nis: info.nis }
28                 } else if pi < self.cfg.new_chi2 {
29                     // Too far to trust, too close to call new. Throwing data
30                     // away is cheaper than an O(N2) update that is wrong.
31                     Association::Rejected { nis: pi }
32                 } else {
33                     self.handle_candidate(f, t)
34                 }
35             })
36             .collect()
37     }
38 }

```

14.6.1 The tests that pin the worked example

RUST crates/ch14_ekfslam/tests/micro.rs

```

1 use approx::assert_relative_eq;
2 use ch14_ekfslam::{EkfSlam, SlamConfig};
3 use nalgebra::{DMatrix, DVector};
4 use sensor::Feature;
5
6 fn cfg() -> SlamConfig {
7     SlamConfig { sigma_r: 0.1, sigma_phi: 0.05, ..SlamConfig::default() }
8 }
9
10 #[test]
11 fn worked_example_ch14_landmark_birth() {
12     // Pose known exactly: Sigma_xx = 0, so the landmark's covariance is pure
13     // measurement noise pushed through G_z.
14     let mut f = EkfSlam::new(DVector::zeros(3), DMatrix::zeros(3, 3), cfg());
15     let j = f.init_landmark(&Feature { r: 2.0, phi: 0.0, s: Some(7) }, 7);
16
17     let m = f.landmark_mean(j);
18     assert_relative_eq!(m[0], 2.0, epsilon = 1e-12);
19     assert_relative_eq!(m[1], 0.0, epsilon = 1e-12);
20
21     // sigma_r2 = 0.01 along the beam; r2sigma_phi2 = 4 * 0.0025 = 0.01 across it.
22     let c = f.landmark_cov(j);
23     assert_relative_eq!(c[(0, 0)], 0.01, epsilon = 1e-12);
24     assert_relative_eq!(c[(1, 1)], 0.01, epsilon = 1e-12);
25
26     // A second identical observation halves it, as two independent
27     // measurements of anything must.
28     f.update_landmark(j, &Feature { r: 2.0, phi: 0.0, s: Some(7) });
29     let c = f.landmark_cov(j);
30     assert_relative_eq!(c[(0, 0)], 0.005, epsilon = 1e-12);
31     assert_relative_eq!(c[(1, 1)], 0.005, epsilon = 1e-12);
32 }
33
34 #[test]
35 fn d4_floor_is_the_initial_vehicle_variance() {
36     // Now give the robot a real prior. D4(iii) says the landmark variance
37     // converges to sigma_02, not to zero, no matter how many measurements arrive.
38     let s0 = 0.04;
39     let mut p = DMatrix::zeros(3, 3);
40     p[(0, 0)] = s0;
41     p[(1, 1)] = s0;
42     p[(2, 2)] = 1e-10; // heading pinned, to isolate translation
43     let mut f = EkfSlam::new(DVector::zeros(3), p, cfg());
44     let j = f.init_landmark(&Feature { r: 2.0, phi: 0.0, s: Some(1) }, 1);
45
46     for _ in 0..400 {
47         f.update_landmark(j, &Feature { r: 2.0, phi: 0.0, s: Some(1) });
48     }
49
50     let c = f.landmark_cov(j);
51     assert_relative_eq!(c[(0, 0)], s0, epsilon = 1e-3); // the floor, not zero
52     assert!(f.correlation(0, 3) > 0.999); // and rho -> 1
53 }
54
55 #[test]
56 fn covariance_update_is_quadratic() {

```

```

57 // Not a timing test -- a counting test. The number of Sigma entries written per
58 // observation must be exactly (3 + 2N)2.
59 for n in [1usize, 4, 16, 64] {
60     let mut f = EkfSlam::new(DVector::zeros(3), DMatrix::identity(3, 3) * 0.01, cfg())
61 );
62     for k in 0..n {
63         let phi = (k as f64 / n as f64) * std::f64::consts::TAU - std::f64::consts::
64         PI;
65         f.init_landmark(&Feature { r: 2.0, phi, s: Some(k as i32) }, k as i32);
66         let before = f.entries_touched;
67         f.update_landmark(0, &Feature { r: 2.0, phi: -std::f64::consts::PI, s: Some(0) })
68         ;
69         assert_eq!(f.entries_touched - before, ((3 + 2 * n) * (3 + 2 * n)) as u64);
70     }
71 }

```

NOTE

The TypeScript in `web/lib/slam/ekf-slam.ts` is a line-for-line port of the Rust above, and it is what the four widgets on this page execute — the same blockwise prediction, the same five-column $\tilde{\Sigma}H^T$, the same rank-2 downdate. The first two tests here are reproduced there against the same constants, which is why the numbers in the worked example and the numbers on your screen agree.

14.7 Putting it together

Run the loop lab end to end and the shape of the chapter is one picture: a filter that is beautiful for exactly as long as you do not measure it.

The beauty is real. Eight beacons, a nineteen-dimensional Gaussian, and one recursion that solves a problem two chapters could not solve separately. Every loop closure is free — no keyframe database, no pose graph, no optimizer, just an innovation arriving through a covariance that has been quietly tracking who is correlated with whom since the first frame. When people say EKF SLAM is elegant, this is what they mean, and they are right.

The measurements are also real. The cost curves away from linear before the map reaches fifty landmarks. The NEES average leaves its band and does not come back. And the mechanism behind that second failure is not a bug to be fixed but a property of the form: a filter marginalizes the past, so it keeps the *consequences* of its linearizations and discards the *evidence*. Ten steps later, with a far better estimate in hand, there is nothing left to re-linearize.

Which is the question Chapter 15 opens with. What if we kept the past? Keep every pose, keep every raw measurement, treat the whole thing as one big nonlinear least-squares problem, and re-linearize all of it whenever the estimate

improves. The information matrix of that problem turns out to be *sparse* where Σ was dense — the same correlations, stored the other way up — and sparse means faer’s sparse Cholesky instead of a 320 GB dense downdate. The two flaws of this chapter are, exactly, the two things that get fixed.

Chapter 17 takes the other road: factor the same posterior differently, sample the trajectory, and note that *conditioned on a trajectory*, the landmarks are independent — so the $O(N^2)$ web that this chapter spent its length defending simply dissolves. Chapter 18 shows EKF SLAM’s most successful descendant, the MSCKF, which keeps the filter and throws away the landmarks. All three are arguments with this chapter, and none of them make sense until you have felt what it is arguing with.

14.8 Exercises

1 FOUNDATION •• Count the prediction

Carry out D1 explicitly for $N = 3$. Write Σ_{t-1} as a 9×9 matrix with named blocks, form $G_t \Sigma_{t-1} G_t^T$, and mark every entry that changes. Then count multiply-adds as a function of N and confirm the $\Theta(N)$ claim. Finally, explain in one sentence what would go wrong if an implementation formed F_x explicitly and multiplied the full $(3 + 2N) \times (3 + 2N)$ matrices.

2 FOUNDATION •• 1-D linear SLAM by hand

Take the two-state problem of D4 with $\sigma_0^2 = 1$ and $\sigma_z^2 = 1$. Run the Kalman filter by hand for three observe-move cycles, where "move" adds process noise $\sigma_u^2 = 0.5$ to the robot only. Tabulate σ_{xx} , σ_{xm} , σ_{mm} and ρ at each step. Then prove the general claim — now *with* motion — that σ_{mm} is bounded below by the initial vehicle variance no matter how many measurements arrive. <details> <summary>Hint</summary> Work in the information form. The unobservable direction is $v = (1, 1)^T$ and $Hv = 0$, so a measurement leaves $v^T \Omega v$ unchanged while motion can only decrease it. Then bound $\text{Var}(m)$ from below with Cauchy-Schwarz, as in D4 step 5. </details>

3 FOUNDATION ••• No cheaper moments form

Prove that the covariance update of D3 must write $\Theta(N^2)$ entries: show that for a generic $\bar{\Sigma}_t$ with non-zero pose-map correlations, every entry of $K_t(\bar{\Sigma}_t H_t^T)^T$ is non-zero, even though H_t has only five non-zero columns. Then explain why the *information* form does not have this problem, and what it pays instead. (You will meet the answer in Chapter 15; try to predict it first.)

4 **CONCEPTUAL** • **Predict, then verify: the ablation**

In `w14.1`, let a full lap complete with the ablation off and note the claimed map σ and the actual map RMSE. Now predict, before you touch it, what turning the diagonal-only ablation on will do to each of those two numbers — same, up, or down — and by roughly how much. Run it. Most readers get the direction of one of them wrong; explain the mechanism behind whichever one surprised you, in terms of what the update does to a landmark that is no longer correlated with the pose.

5 **CONCEPTUAL** •• **Predict, then verify: the loop length**

In `w14.2`, find the shortest loop length at which the 40-run mean NEES leaves its band and stays out. Then switch to single-run mode at that same setting and try to convince yourself from one trace that the filter is inconsistent. Write down why you cannot, using the two acceptance bands the widget prints, and state how many runs you would need for a band of half the current width.

6 **CONCEPTUAL** •• **Gates that are too small**

In `w14.4`, set the clutter rate to zero and the gate to 5.99 (the 95% quantile), and run three laps. Count the duplicate landmarks. Now raise the gate to 20 and repeat, then lower it to 3. Explain the U-shaped failure curve you get, and connect the left-hand branch to the autopsy in the previous section — specifically, which quantity being wrong makes a correctly-derived χ^2 gate the wrong size?

7 **PRACTICAL** •• **Retire a landmark**

Implement `retire_landmark` end to end: the per-landmark existence log odds, the decrement for landmarks that were expected in view and not seen, and the block deletion. Then measure, on the `w14.4` scenario, the number of false landmarks surviving after 500 steps as a function of clutter rate, with and without the provisional list, and plot the two curves. Report the point at which the provisional list stops helping and explain what changes there.

8 **PRACTICAL** ••• **Re-run the autopsy on a manifold**

Re-implement the pose part of `EkfSlam` as an error-state filter on $SE(2)$, using the $\boxplus$ / $\boxminus$ operators of Chapter 7 so the linearization point for the pose is always the identity. Re-run the 40-run Monte Carlo NEES. Report how much of the inconsistency disappears — it should be a lot — and then find a loop length at which it comes back. That residue is the part no filter can fix, and it is what Chapter 15 exists to remove.

14.9 References

Smith, R., Self, M., and Cheeseman, P. (1990). Estimating Uncertain Spatial Relationships in Robotics *Autonomous Robot Vehicles*, Springer, 167–193.

The stochastic map: the paper that first put robot and landmarks in one correlated state. Everything in this chapter is a footnote to its section 4. doi:10.1007/978-1-4613-8997-2_14

Thrun, S., Burgard, W., and Fox, D. (2005). Probabilistic Robotics *MIT Press*.

Chapter 10 is this chapter’s spine: Tables 10.1 and 10.2, the provisional list, and the map-management machinery of §10.3.3. The two sentences quoted in the autopsy — ‘mostly historical’ and the 1,000-feature ceiling — are from the authors’ 1999–2000 draft of that chapter. [link](#)

Dissanayake, M. W. M. G., Newman, P., Clark, S., Durrant-Whyte, H. F., and Csorba, M. (2001). A solution to the simultaneous localization and map building (SLAM) problem *IEEE Transactions on Robotics and Automation* 17(3), 229–241.

The convergence theorems of D4: monotonically decreasing map covariance, correlations tending to one, and the lower bound set by the initial vehicle uncertainty. doi:10.1109/70.938381

Julier, S. J. and Uhlmann, J. K. (2001). A counter example to the theory of simultaneous localization and map building *Proceedings 2001 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 4, 4238–4243.

The stationary-robot counterexample of D6, step 5: with a range-bearing sensor of non-zero angular uncertainty, the full-covariance filter is always inconsistent. doi:10.1109/ROBOT.2001.933280

Bailey, T., Nieto, J., Guivant, J., Stevens, M., and Nebot, E. (2006). Consistency of the EKF-SLAM Algorithm 2006 *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3562–3568.

The empirical study that set the methodology w14.2 follows: Monte Carlo NEES against a χ^2 band, and the finding that heading uncertainty is the trigger. doi:10.1109/IR0S.2006.281644

Huang, S. and Dissanayake, G. (2007). Convergence and Consistency Analysis for Extended Kalman Filter Based SLAM *IEEE Transactions on Robotics* 23(5), 1036–1049.

Ties the inconsistency directly to linearization at the estimated rather than the true state, and to the unobservable directions D6 step 4 is about. doi:10.1109/TR0.2007.903811

Barrau, A. and Bonnabel, S. (2023). The Geometry of Navigation Problems *IEEE Transactions on Automatic Control* 68(2), 689–704.

The modern treatment of the first exit: choose error coordinates whose dynamics do not depend on the estimate, and the manufactured information of D6 largely disappears. Connects Chapter 7’s invariant EKF to this chapter’s failure. doi:10.1109/TAC.2022.3144328

Rosen, D. M., Doherty, K. J., Terá n Espinoza, A., and Leonard, J. J. (2021). Advances in Inference and Representation for Simultaneous Localization and Mapping *Annual Review of Control, Robotics, and Autonomous Systems* 4, 215–242.

Where the field went after this chapter’s two flaws: the survey to read before Chapter 15, and the clearest statement of why smoothing replaced filtering. doi:10.1146/annurev-control-072720-082553

Carlone, L., Kim, A., Barfoot, T., Cremers, D., and Dellaert, F. (eds.) (2026). SLAM Handbook: From Localization and Mapping to Spatial Intelligence *Cambridge University Press*.

The current community reference, released openly. Its framing of SLAM as inference over a factor graph is the destination the next three chapters are heading for. [link](#)

SLAM as Least Squares: Factor Graphs

MAPPING AND SLAM · Advanced · 70 min



We can think of this information as a constraint between x_{t-1} and x_t , or a "spring" in a spring-mass model of the world.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 11

Chapter 14 ended with a filter that had painted itself into a corner. It was overconfident because it had linearized each landmark exactly once, at the moment of worst uncertainty, and it could not repair that mistake later because the poses it would need to repair had already been marginalized away. The filter's state is the *present*; the evidence that the past was wrong arrives in the *future*; and there is no arrangement of the recursion that lets those two meet.

The fix is radical in concept and mundane in mechanics: **keep the past**. Write down the entire posterior over the whole trajectory and the whole map, take its logarithm, and watch the product shatter into a sum of small quadratic penalties — one per control, one per observation. That sum is a **factor graph**, its minimizer is the MAP estimate, and computing it is sparse nonlinear least squares. Every Jacobian gets re-evaluated at the newest estimate on every iteration, so Chapter 14's central sin is not fixed but *structurally impossible*.

This is the chapter where the book's baseline meets its future. The 2005 text's GraphSLAM chapter already had the pieces — construct, reduce, solve, iterate — but not the vocabulary or the sparse solvers. What follows is the same algorithm, said in the language of 2026, and running in your browser.

🕒 AFTER THIS CHAPTER YOU CAN

- Write the full SLAM posterior and read its factorization as a bipartite graph of variables and factors
- Turn $-\log$ of that posterior into a sum of squared Mahalanobis residuals, so MAP inference becomes nonlinear least squares
- Derive Gauss–Newton and Levenberg–Marquardt on the manifold, with $\boxplus$ retraction and analytic SE(2) Jacobians
- Prove that Ω mirrors the graph — and that eliminating a variable is the Schur complement, which is Thrun’s EIF_reduce under a new name
- Explain why ordering, not sparsity, decides how fast a smoother runs, and measure the difference
- Make a back end survive one lying loop closure, and say precisely what that robustness costs
- Build the whole thing in Rust on nalgebra + faer, then rebuild it in forty lines with factrs 0.3

📦 ASSUMED

- SE(2), $\exp/\log$, and the $\boxplus / \boxminus$ retraction (Chapter 3)
- The information form ξ , Ω and why information adds (Chapter 6)
- Jacobians of models evaluated on a manifold (Chapter 7)
- Motion and measurement models as probability distributions (Chapters 9 and 10)
- EKF SLAM, its dense covariance, and its frozen linearizations (Chapter 14)

15.1 The map that could not be fixed

Here is Rusty’s problem, laid out in full. He drives east along the Apartment corridor, ducks into room C through its doorway, once around the room, and back home along the other side of the corridor — 55 poses, 12 corner reflectors, 176 sightings. His wheel odometry has a 3.5% turn bias: not noise, a *bias*, the kind that does not average away. By the time he gets home the dead-reckoned path says he finished 1.66 m from where he started.

He also knows he is home. The doorway he sees now is the doorway he saw fifty poses ago. That single recognition contradicts every one of the 54 odometry readings, slightly, and there is no filter update that can distribute the contradiction backwards across all of them.

So stop filtering. Put all 189 unknown numbers on the table at once and ask what values make the whole data set most probable.

 INTERACTIVE · w15.1**Spring-Graph Optimizer**

MAP inference is not a black box — it is a graph of springs relaxing. And least squares does not average an outlier away: it lets the outlier pull.

Run this simulation in the web edition: `/chapters/ch15-factor-graphs #w15.1`

Three things in that widget deserve your attention before any mathematics.

The correction is not spread evenly through time — it is spread according to evidence. Pose 0 does not move at all, because the prior pins it. The poses in room C move about 0.4 m. The last pose moves 1.65 m, which is the entire accumulated drift, because that is where the contradiction lives. No rule was written down to produce that gradient; it is what minimizing a sum of squares does when the constraints have different stiffnesses.

Almost everything happens in three iterations. J falls from 3780 to 599 to 161 to 117 and then stops. Least squares on a well-initialized SLAM problem is not an iterative slog; it is two or three re-linearizations and a lot of back-substitution.

One bad spring bends the whole map. Press *inject a false loop closure* and the smoother — which was just praised for using all its evidence — dutifully uses the evidence that is wrong. The RMSE goes from 0.197 m to 0.765 m. Robustness is not something least squares has; it is something you must add, and the last third of this chapter is about the price.

 THE ONE SENTENCE VERSION

The full SLAM posterior factorizes into one term per measurement; minus its logarithm is a sum of squared Mahalanobis residuals; minimizing that sum is sparse nonlinear least squares, and the sparse matrix you factor is a picture of the graph you drew.

15.2 Building intuition: springs, and a ledger of who is connected to whom

Thrun, Burgard and Fox already gave us the metaphor, in the sentence at the top of this chapter. A control is a spring between two consecutive poses. An observation is a spring between a pose and a landmark. The stiffness of a spring is the *information* of the measurement that produced it — a laser with 1 cm noise makes a stiffer spring than sonar with 30 cm noise. The robot's estimate is where the whole assembly comes to rest.

That metaphor is worth more than its charm, because it makes three otherwise separate ideas visibly the same thing:

💡 THE TRIPLE EQUIVALENCE

Probability. The posterior $p(x_{0:t}, m \mid z_{1:t}, u_{1:t})$ factorizes into a product of local terms, one per control and one per observation.

Optimization. Its negative log is $J(y) = \frac{1}{2} \sum_k \|r_k(y)\|_{\Sigma_k}^2$, a sum of spring energies; the MAP estimate is the minimum-energy configuration.

Linear algebra. The Hessian of that energy, $\Omega = \sum_k J_k^T \Sigma_k^{-1} J_k$, has a nonzero block exactly where two variables share a factor — so the matrix you factor is the adjacency structure of the graph you drew.

Every section below is one face of this box. If you ever lose the thread, come back here and ask which face you are looking at.

The rest of the chapter needs a little new notation, all of it local to Part V.

$y = (x_{0:t}, m)$	The stacked vector of every unknown — Thrun’s full-SLAM state. For the Apartment loop, $55 \text{ poses} \times 3 + 12 \text{ landmarks} \times 2 = 189$ tangent dimensions.
ϕ_k	Factor k: a function of the few variables it touches, proportional to the likelihood of one measurement.
$r_k(y), \Sigma_k$	The residual of factor k (how badly its measurement is explained) and its noise covariance.
$\ r\ _{\Sigma}^2 = r^T \Sigma^{-1} r$	Squared Mahalanobis norm: error measured in sigmas, not metres.
$J(y)$	The MAP objective, $-\log$ posterior up to a constant.
$A, \bar{r}$	Whitened stacked Jacobian and residual at the current linearization point.
$\Omega = A^T A, b = A^T \bar{r}$	The normal equations $\Omega \Delta = -b$. Ω is Chapter 6’s information matrix, at trajectory scale.
$\Delta, y \boxplus \Delta$	The tangent-space update and the retraction that applies it.
λ	Levenberg–Marquardt damping: the size of the region in which you trust the linear model.
$\rho(\cdot), w = \rho'(e)/e$	Robust kernel and the IRLS weight it induces.
$\text{nnz}(L)$	Nonzeros in the Cholesky factor — the real currency of a sparse solver.

15.3 The mathematics

15.3.1 The posterior is a graph

Nothing new is assumed here. The Markov property from Chapter 5, the motion model of Chapter 9, the measurement model of Chapter 10, and — for now — known correspondences: we are told which landmark each observation belongs to. (Chapter 16 removes that assumption, and the robust machinery at the end of this chapter is what makes removing it survivable.)

$$p(x_{0:t}, m \mid z_{1:t}, u_{1:t}) \propto p(x_0) \prod_{\tau=1}^t p(x_\tau \mid x_{\tau-1}, u_\tau) \prod_{\tau=1}^t \prod_i p(z_\tau^i \mid x_\tau, m_{c_\tau^i})$$

Read the right-hand side as a drawing. Each factor is a small box; each box is wired to the two or three variables it mentions. The prior box touches x_0 . Each motion box touches a consecutive pair $(x_{\tau-1}, x_\tau)$. Each measurement box touches one pose and one landmark. **No box ever touches two landmarks.** That last sentence is the whole reason this chapter scales, and we will cash it in twice.

DERIVATION From the factorized posterior to a sum of squares

$$J(y) = \frac{1}{2} \sum_k \|r_k(y)\|_{\Sigma_k}^2$$

Step 1 — the factorization. Bayes' rule plus the Markov assumption gives the product above; the normalizer η does not depend on y and therefore cannot move the arg-max, so we drop it and work with an unnormalized posterior throughout. (This is the one place where MAP is genuinely cheaper than full Bayesian inference: we never have to compute the evidence.)

Step 2 — substitute Gaussian noise models. Each factor is a Gaussian in its own residual. For the motion factor, the noise lives on the manifold, so the residual is a $\boxplus$, not a subtraction:

$$r_{u_\tau} = (x_{\tau-1}^{-1} x_\tau) \boxplus \delta_\tau = \log(\delta_\tau^{-1} x_{\tau-1}^{-1} x_\tau) \in \mathbb{R}^3, \quad p(x_\tau \mid x_{\tau-1}, u_\tau) \propto \exp\left(-\frac{1}{2} \|r_{u_\tau}\|_{\Sigma_u}^2\right)$$

where $\delta_\tau = \text{odom}(u_\tau)$ is the relative pose the wheels reported. For a range-bearing observation the residual is ordinary subtraction with the bearing wrapped to $(-\pi, \pi]$:

$$r_{z_\tau^i} = h(x_\tau, m_{c_\tau^i}) - z_\tau^i, \quad h(x, m) = \begin{pmatrix} \|{}^b m\| \\ \text{atan2}({}^b m_y, {}^b m_x) \end{pmatrix}, \quad {}^b m = R(\theta)^\top (m - t)$$

Step 3 — take minus the logarithm. Products become sums and Gaussian densities become half squared Mahalanobis norms:

$$-\log p(y \mid z, u) = \underbrace{\frac{1}{2} \|x_0 \boxminus \bar{x}_0\|_{\Sigma_0}^2}_{\text{prior}} + \underbrace{\frac{1}{2} \sum_{\tau} \|r_{u_\tau}\|_{\Sigma_u}^2}_{\text{controls}} + \underbrace{\frac{1}{2} \sum_{\tau, i} \|r_{z_\tau^i}\|_{\Sigma_z}^2}_{\text{observations}} + \text{const}$$

Step 4 — read each summand as an edge. The formula *is* the picture: one term per box in the drawing, each depending on only the variables its box is wired to. Define

$$\hat{y} = \arg \min_y J(y), \quad J(y) = \frac{1}{2} \sum_k \|r_k(y)\|_{\Sigma_k}^2$$

and MAP inference has become nonlinear least squares. ■

A note on the prior. Without $p(x_0)$ the objective is invariant under a global rigid motion of everything — three directions along which Ω is exactly singular. Solvers hide this with a tiny diagonal nudge; the honest fix is to say where the origin is, which is what the prior factor does.

15.3.2 Whitening: why every residual is measured in sigmas

Carrying Σ_k^{-1} around inside every norm is clumsy. Factor it out once, at the point where the factor is built. Let $\Sigma_k^{-1} = S_k^T S_k$ (Cholesky, from Chapter 2) and define the **whitened** residual $\tilde{r}_k = S_k r_k$. Then

$$\|r_k\|_{\Sigma_k}^2 = \|\tilde{r}_k\|_2^2, \quad \text{so} \quad J(y) = \frac{1}{2} \sum_k \|\tilde{r}_k(y)\|^2$$

Every residual is now dimensionless and measured in standard deviations, which means residuals from a laser, a wheel encoder and a loop-closure detector are directly comparable. It also means the number the widget prints beside a bad spring — $e = 35\sigma$ — has an unambiguous meaning: *this measurement is thirty-five standard deviations from what the current estimate predicts.*

NOTE

From here on, r and J always mean the whitened residual and its Jacobian. This is not a convenience; it is the reason one solver can serve every sensor in the book without knowing what any of them measure.

15.3.3 Gauss–Newton on the manifold

J is not quadratic — h has an atan2 in it and poses live on $\text{SE}(2)$ — but it is *locally* quadratic, and that is enough.

DERIVATION Normal equations from the linearized residuals

$$\Omega \Delta = -b, \quad \Omega = \sum_k J_k^T J_k$$

Step 1 — expand in the tangent space. Perturb the current estimate through the retraction rather than by addition, so the perturbed poses are still poses:

$$r_k(y^0 \boxplus \Delta) \approx \bar{r}_k + J_k \Delta, \quad J_k = \left. \frac{\partial r_k(y^0 \boxplus \Delta)}{\partial \Delta} \right|_{\Delta=0}$$

Step 2 — stack. Let A be the block matrix whose k -th block row is J_k (with columns only where factor k touches a variable) and $\bar{r}$ the stacked residuals. Then

$$J(y^0 \boxplus \Delta) \approx \frac{1}{2} \|A\Delta + \bar{r}\|^2 = \frac{1}{2} \Delta^\top A^\top A \Delta + \Delta^\top A^\top \bar{r} + \frac{1}{2} \|\bar{r}\|^2$$

Step 3 — differentiate and set to zero. With $\Omega = A^\top A$ and $b = A^\top \bar{r}$,

$$\Omega \Delta = -b, \quad \Omega = \sum_k J_k^\top J_k, \quad b = \sum_k J_k^\top \bar{r}_k$$

The sums matter more than the matrices: **assembly is a loop over factors**, each writing into only its own blocks. Nothing is ever formed densely unless you ask for it.

Step 4 — retract, and repeat. $y \leftarrow y \boxplus \Delta$, re-evaluate every $\bar{r}_k$ and every J_k at the new y , solve again. Stop when the relative decrease in J falls below tolerance.

Step 5 — the payoff. Step 4 re-evaluates *every* Jacobian, including the one for a pose fifty steps in the past. There is no stored linearization to go stale, because there is no stored linearization at all. Chapter 14's frozen-Jacobian inconsistency cannot be expressed in this algorithm. ■

DERIVATION What Gauss–Newton throws away, and what Ω means at the optimum

$$\nabla^2 J = \Omega + \sum_k r_k^\top \nabla^2 r_k$$

The exact Hessian of J is

$$\nabla^2 J = \sum_k \left(J_k^\top J_k + \sum_i (\bar{r}_k)_i \nabla^2 (r_k)_i \right)$$

Gauss–Newton keeps the first term and drops the second. The dropped term is weighted by the residuals themselves, so near a good solution —

where residuals are a sigma or two — it is small, and GN inherits Newton’s near-quadratic convergence. Far from the solution, or when the residuals stay large because the model is wrong, the dropped term is not small, and GN can take a step that increases the cost. That is exactly the failure Levenberg–Marquardt insures against.

The consolation prize is worth a section of its own. At convergence, Ω is the information matrix of the Laplace approximation to the posterior:

$$p(y \mid z_{1:t}, u_{1:t}) \approx \mathcal{N}(\hat{y}, \Omega^{-1})$$

so covariances are still available — as *selected entries* of Ω^{-1} , computed by back-substitution rather than by inverting anything. This is Chapter 6’s duality, grown up: the smoother stores Ω because Ω is sparse, and produces Σ on demand because Σ is not.

The Jacobians are where the manifold discipline of Chapter 7 earns its keep. For the between factor $r = \log(\delta^{-1}x_i^{-1}x_j)$, perturbing x_j on the right gives $\log(E \exp(\Delta_j)) \approx r + J_r^{-1}(r)\Delta_j$, while perturbing x_i pushes the perturbation through the adjoint:

$$\frac{\partial r}{\partial \Delta_i} = -J_r^{-1}(r) \text{Ad}_{x_j^{-1}x_i}, \quad \frac{\partial r}{\partial \Delta_j} = J_r^{-1}(r)$$

with J_r the right Jacobian of $\text{SE}(2)$. Both are exact, both are cheap, and the library checks them against finite differences of the $\boxplus$ perturbation to eight digits.

ALGORITHM `linearize_graph(G, y)` COST $O(K)$ -- one pass, $O(1)$ work per factor
in factor graph G with K factors, current estimate y
out Ω , b , and the cost $J(y)$

```

1:  $\Omega \leftarrow 0, b \leftarrow 0, J \leftarrow 0$ 
2: for each factor  $k \in G$  do
3:    $(\bar{r}_k, \{J_{k,a}\}) = \text{linearize}_k(y)$  // whitened
4:    $e_k = \|\bar{r}_k\|, w_k = \rho'(e_k)/e_k, J \mathrel{+}= \rho(e_k)$ 
5:   for each pair  $(a, c)$  of variables touched by  $k$  do
6:      $\Omega_{[a][c]} \mathrel{+}= w_k J_{k,a}^\top J_{k,c}$ 
7:    $b_{[a]} \mathrel{+}= w_k J_{k,a}^\top \bar{r}_k$ 
8: return  $(\Omega, b, J)$ 
```


ALGORITHM `gauss_newton(G, y, tol, n_max)` COST per iteration $O(K)$
 assembly + one sparse Cholesky (ordering-dependent: near-linear on a chain, $\sim O(n^{1.5})$
 on a planar graph, $O(n^3)$ worst case)
in graph, initial estimate, relative-decrease tolerance
out and a report: cost per iteration, $\|\Delta\|$, $\text{nnz}(L)$

```

1:  $y \leftarrow y^0$ 
2: for  $i = 1$  to  $n_{max}$  do
3:    $(\Omega, b, J) = \text{linearize\_graph}(G, y)$ 
4:   solve  $\Omega \Delta = -b$  // sparse Cholesky in a good ordering
5:    $y \leftarrow y \boxplus \Delta$ 
6:   if  $|J_{prev} - J|/J_{prev} < tol$  then break
7: return  $y$ 

```

15.3.4 Damping: how far may you trust a linear model?

Gauss–Newton believes its own linearization completely. Where the cost valley curves — which in SLAM means anywhere a large rotation is being corrected — that belief produces a step that lands somewhere absurd. Levenberg–Marquardt adds a knob:

$$(\Omega + \lambda \text{diag } \Omega) \Delta = -b$$

As $\lambda \rightarrow 0$ this is Gauss–Newton. As $\lambda \rightarrow \infty$ it becomes $\Delta \approx -(\lambda \text{diag } \Omega)^{-1}b$, a very short step along the (scaled) negative gradient: slow, but it cannot be wrong about direction. Marquardt’s $\text{diag } \Omega$ scaling — rather than λI — makes the trust region an ellipsoid shaped like the problem, so a well-determined direction is not damped as hard as a poorly-determined one.

INTERACTIVE · w15.3

Descent Duel

Levenberg–Marquardt is not Gauss–Newton with insurance: λ slides continuously between a step that trusts the linear model completely and one that barely trusts it at all.

Run this simulation in the web edition: [/chapters/ch15-factor-graphs #w15.3](/chapters/ch15-factor-graphs/#w15.3)

The schedule is the algorithm’s only cleverness. Compare the reduction the linear model *promised* with the reduction actually delivered:

$$\rho = \frac{J(y) - J(y \boxplus \Delta)}{\frac{1}{2} \Delta^T (\lambda \text{diag } \Omega \Delta - b)}$$

If ρ is healthy, accept the step and shrink λ ; if the cost went up, reject the step outright and grow λ by an order of magnitude. Rejecting is cheap — you already have Ω and only need to re-solve — and it is what makes LM the default in every production back end.

ALGORITHM `levenberg_marquardt(G, y,  $\lambda_0$ )` COST Gauss-Newton per accepted step,
plus $O(n)$ per rejected trial
in graph, initial estimate, initial damping $\lambda_0 \approx 10^{-3}$
out , plus the accepted/rejected trace

```

1:  $y \leftarrow y^0, \lambda \leftarrow \lambda_0, J_0 = \text{cost}(G, y)$ 
2: repeat
3:    $(\Omega, b, \cdot) = \text{linearize\_graph}(G, y)$ 
4:   solve  $(\Omega + \lambda \text{diag } \Omega) \Delta = -b$ 
5:    $J_1 = \text{cost}(G, y \boxplus \Delta)$ ; compute the gain ratio  $\rho$ 
6:   if  $J_1 < J_0$  then  $y \leftarrow y \boxplus \Delta$ ;  $J_0 \leftarrow J_1$ ;  $\lambda \leftarrow \lambda \cdot \max(\frac{1}{3}, 1 - (2\rho - 1)^3)$ 
7:   else  $\lambda \leftarrow 10\lambda$ 
8: until converged or  $\lambda > \lambda_{max}$ 
9: return  $y$ 

```

15.3.5 A worked example you can check by hand

Three scalar positions on a line, four unit-information factors:

$$y_0 = 0, \quad y_1 - y_0 = 1, \quad y_2 - y_1 = 1, \quad y_2 = 1.5$$

The prior and the fix disagree: dead reckoning says $y_2 = 2$, the absolute measurement says 1.5. Start from $y = (0, 0, 0)$, where the residuals are $r = (0, -1, -1, -1.5)$ and $J = \frac{1}{2}(0 + 1 + 1 + 2.25) = 2.125$.

The Jacobians are the rows $(1, 0, 0)$, $(-1, 1, 0)$, $(0, -1, 1)$, $(0, 0, 1)$. Accumulate $\Omega = \sum J_k^T J_k$ and $b = \sum J_k^T r_k$ by hand — each factor writes into a 1×1 or 2×2 patch and nothing else:

$$\Omega = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 0 \\ -2.5 \end{pmatrix}$$

Notice the shape: tridiagonal, because the only variables sharing a factor are neighbors. Solving $\Omega \Delta = -b$ by elimination gives, in one step (the problem is linear, so Gauss-Newton converges exactly once):

$$\hat{y} = (-0.125, 0.75, 1.625)$$

Now read the answer. Both odometry intervals came out 0.875 — *equally* shortened, though only the second one is adjacent to the disagreeing measurement. And all four residuals end at exactly ± 0.125 , with $J = 4 \cdot \frac{1}{2}(0.125)^2 = 1/32$. Least squares shared the half-metre contradiction evenly across four equally stiff springs, because that is what minimizes a sum of squares: equal marginal cost everywhere. If you make the prior ten times stiffer, the tension moves to the other end. That is the entire behavior of a SLAM back end, in three numbers you can check on paper.

15.4 Sparsity: Ω is a picture of the graph

Now cash in the observation from the factorization: no factor mentions two landmarks.

DERIVATION The block sparsity pattern of Ω

$$\Omega_{[j][k]} \neq 0 \iff j, k \text{ share a factor}$$

From the assembly rule, $\Omega = \sum_k J_k^\top J_k$, and J_k has nonzero columns only where factor k touches a variable. So the product $J_k^\top J_k$ writes into the block (a, c) only when factor k touches *both* a and c . Summing over factors:

$$\Omega_{[j][k]} \neq 0 \iff \exists \text{ a factor touching both } j \text{ and } k$$

which is the adjacency matrix of the graph, with blocks instead of bits. For SLAM this gives the bordered-block-diagonal pattern:

- $\Omega_{[x_\tau][x_{\tau+1}]} \neq 0$ — consecutive poses, from controls: a tridiagonal band, plus one off-band block per loop closure, which is precisely why loop closure is the expensive event;
- $\Omega_{[x_\tau][m_i]} \neq 0$ — when pose τ observed landmark j (the border);
- $\Omega_{[m_i][m_j]} = 0$ for $i \neq j$ — **always**, because no measurement ever constrains two landmarks relative to each other. All a robot ever receives are landmark-relative-to-pose readings.

Count: the Apartment graph has 232 factors over 189 dimensions, and Ω 's lower triangle holds 1917 nonzeros out of a possible 17 955 — 11%. The number of nonzeros grows with the number of *edges*, not with n^2 .

The contrast with Chapter 14 is exact. The EKF's covariance is dense — gloriously, correctly dense, since every landmark really is correlated with every other one. But those correlations are what you get after marginalizing $x_{0:t-1}$ away. The smoother keeps the poses and so never induces them; the correlations still exist, implicitly, as paths through the graph. Sparsity is

not an approximation here. It is what not throwing anything away looks like.

INTERACTIVE · w15.2

Sparsity Scope

Elimination order is not an implementation detail: marginalizing a variable does not simplify the problem, it relocates the problem as fill-in.

Run this simulation in the web edition: [/chapters/ch15-factor-graphs #w15.2](/chapters/ch15-factor-graphs/#w15.2)

15.4.1 Elimination is the Schur complement is EIF_reduce

Sparsity in Ω is not automatically speed. You still have to solve $\Omega\Delta = -b$, and how you do that decides everything.

Solving means eliminating variables one at a time, and eliminating a variable is exactly Gaussian marginalization in information form:

DERIVATION Eliminating the map block

$$\tilde{\Omega} = \Omega_{xx} - \Omega_{xm}\Omega_{mm}^{-1}\Omega_{mx}$$

Step 1 — partition. Order the variables poses-then-landmarks and split the system:

$$\begin{pmatrix} \Omega_{xx} & \Omega_{xm} \\ \Omega_{mx} & \Omega_{mm} \end{pmatrix} \begin{pmatrix} \Delta_x \\ \Delta_m \end{pmatrix} = - \begin{pmatrix} b_x \\ b_m \end{pmatrix}$$

Step 2 — solve the bottom row for Δ_m . $\Delta_m = -\Omega_{mm}^{-1}(b_m + \Omega_{mx}\Delta_x)$.

Step 3 — substitute into the top row.

$$\underbrace{(\Omega_{xx} - \Omega_{xm}\Omega_{mm}^{-1}\Omega_{mx})}_{\tilde{\Omega}} \Delta_x = - \underbrace{(b_x - \Omega_{xm}\Omega_{mm}^{-1}b_m)}_{\tilde{b}}$$

$\tilde{\Omega}$ is the **Schur complement** of Ω_{mm} , and by the Gaussian marginalization identity of Appendix B it is the information matrix of the *marginal* posterior over the trajectory. Nothing has been approximated: solving the reduced system and back-substituting for Δ_m gives bit-for-bit the same answer as solving the full one. (The library asserts this to 10^{-7} on the loop scene.)

Step 4 — read it on the graph. Ω_{mm} is block-diagonal (landmarks never touch each other), so $\Omega_{xm}\Omega_{mm}^{-1}\Omega_{mx}$ decomposes into one small update per landmark: a landmark seen from poses i, j, k contributes a nonzero block to every pose pair among $\{i, j, k\}$. Eliminating it *clique-connects its neighbors*. Thrun's own words for the same operation: "We simply have replaced springs connecting m_j to various poses in our spring mass model by a set of springs directly linking these poses."

Step 5 — the general case. Sparse Cholesky is nothing but this, one variable at a time: eliminate y_i , clique-connect its surviving neighbors, write its column of L , repeat. Each new edge in that clique that was not already there is **fill-in**. ■

ALGORITHM `schur_marginalize( $\Omega$ ,  $b$ ,  $M$ )` `COST  $O(\sum_{v \in M} \deg(v)^2)$` -- local,
because Ω_{mm} is block diagonal
in information system (Ω, b) and the set M of variables to eliminate
out the reduced system $(\tilde{\Omega}, \tilde{b})$ over the remaining variables — exactly Thrun, Table 11.3

- 1: partition Ω, b into keep-blocks x and victim-blocks m
- 2: $K = \Omega_{xm} \Omega_{mm}^{-1}$
- 3: $\tilde{\Omega} = \Omega_{xx} - K \Omega_{mx}$
- 4: $\tilde{b} = b_x - K b_m$
- 5: return $(\tilde{\Omega}, \tilde{b})$ // recover Δ_m afterwards by back-substitution — Table 11.4

15.4.2 Ordering is the algorithm

Fill-in is created by elimination, so the *order* of elimination decides how much of it you pay for. Finding the optimal ordering is NP-hard; the heuristics that matter are minimum degree (AMD, COLAMD) and nested dissection (METIS). Here is what they are worth on the Apartment graph, all four solving the identical system to the identical answer:

Ordering	nnz(L)	as % of dense	fill edges	factorization flops
Minimum degree	2 563	14.3%	91	34 k
Chronological	4 821	26.9%	480	130 k
Poses first	4 821	26.9%	480	130 k
Landmarks first (the "Schur trick")	10 935	60.9%	1 002	743 k

Two lessons, and the second one is the interesting one.

Ordering is worth an order of magnitude, here $22\times$ in flops between the best and worst choice, and the gap widens with problem size. This is why every serious solver runs a fill-reducing ordering as a first-class step, and why $\text{nnz}(\mathbf{L})$ — not $\text{nnz}(\mathbf{\Omega})$ — is the number to quote.

The famous trick is the worst choice on this graph. Eliminating landmarks first is the classical Schur trick from bundle adjustment, and it is excellent when each landmark is seen from few poses. In the Apartment, the corner reflectors are visible from long stretches of corridor, so eliminating one welds a dozen poses into a clique. Play with the *One landmark, seen by all* preset in the widget above: eliminating that hub first leaves $\text{nnz}(\mathbf{L}) = 351$ and twenty fill edges; eliminating it last leaves 216 and five. Same graph, same posterior, 63% more storage and a correspondingly denser factorization. Rules of thumb about ordering are statements about *graph shape*, not about algorithms.

⚠ WHERE ROBOTS BREAK THIS

Nothing in this section changed the answer. Every ordering produces the same $\hat{\mathbf{y}}$ to machine precision. If a change of ordering changes your solution, you have a numerical conditioning problem, not an ordering problem — and the usual culprit is a missing prior, leaving $\mathbf{\Omega}$ singular along the gauge directions.

15.5 One bad spring: robustness

Least squares is maximum likelihood under Gaussian noise, and a Gaussian says a 35σ event has probability smaller than 10^{-260} . When one happens anyway — a false loop closure, a mis-associated landmark, a laser beam through a glass door — the estimator does not shrug. It concludes that the world must be bent, and bends it.

📖 DERIVATION Robust kernels and the IRLS weight

$$w_k = \rho'(e_k)/e_k$$

Step 1 — the diagnosis. With $\rho(e) = \frac{1}{2}e^2$, the derivative $\rho'(e) = e$ is the *influence* of a residual: how hard it pulls. It is unbounded. Double the outlier, double its pull; the rest of the graph must move to meet it.

Step 2 — replace the loss. Choose a ρ that grows more slowly, and minimize $J(\mathbf{y}) = \sum_k \rho(e_k)$ with $e_k = \|r_k(\mathbf{y})\|$. Differentiate:

$$\nabla J = \sum_k \rho'(e_k) \nabla e_k = \sum_k \underbrace{\frac{\rho'(e_k)}{e_k}}_{w_k} J_k^\top r_k$$

Step 3 — recognize what that is. The stationarity condition is identical to that of the *weighted* least-squares problem $\frac{1}{2} \sum_k w_k \|r_k\|^2$ with weights held fixed. So: compute weights from the current residuals, solve the weighted normal equations, recompute the weights, repeat. That is **iteratively reweighted least squares**, and it folds into Gauss–Newton as a single scalar multiply per factor — line 6 of `linearize_graph`, which is why that line was already written with w_k in it.

Step 4 — the menu, and the bill.

Kernel	$\rho(e)$	$w(e) = \rho'(e)/e$	Convex?
L2	$\frac{1}{2}e^2$	1	yes
Huber(k)	$\frac{1}{2}e^2$ if $ e \leq k$, else $k(e - \frac{k}{2})$	$\min(1, k/ e)$	yes
Cauchy(c)	$\frac{c^2}{2} \log(1 + e^2/c^2)$	$1/(1 + e^2/c^2)$	no
Geman–McClure(c)	$\frac{c^2 e^2}{2(c^2 + e^2)}$	$c^4/(c^2 + e^2)^2$	no

Huber’s influence *saturates*: an outlier still pulls, forever, just no harder than a $k\sigma$ inlier does. Cauchy and Geman–McClure **redescend**: influence goes back to zero, so a far enough outlier is switched off entirely. And there is the bill — a redescending ρ is non-convex, so the objective grows extra local minima, one of which sits on top of the outlier. ■

INTERACTIVE · w15.4

Kernel Gallery

Robust kernels are not a free lunch: the ones that ignore an outlier completely are exactly the ones that make the objective non-convex.

Run this simulation in the web edition: [/chapters/ch15-factor-graphs #w15.4](/chapters/ch15-factor-graphs/#w15.4)

Run the numbers on the Apartment loop with a 1.6 m lie injected into one loop closure, and the theory is visible in three digits:

Back end	RMSE vs truth	weight on the false factor	its residual at the optimum
Clean graph (no outlier)	0.197 m	—	—
L2	0.765 m	1.000	6.6σ
Huber, $k = 1$	0.534 m	0.052	19.2σ

Back end	RMSE vs truth	weight on the false factor	its residual at the optimum
Cauchy, $c = 1$	0.211 m	0.0009	34.0σ
Geman–McClure, $c = 1$	0.199 m	0.00002	34.7σ

Read the last column first. Under L2 the false factor got most of what it wanted — its residual is *only* 6.6σ at the optimum, because every pose between the two it names moved to meet it. Under Geman–McClure it sits at 34.7σ , screaming, and nobody is listening: the map is back to 0.199 m, statistically indistinguishable from the clean solution. Huber lands in between, exactly as its influence function promises.

So why is Huber still the industry default? Because convexity is a guarantee and robustness is a gamble. Huber’s objective has one minimum, so the answer does not depend on where you started; Geman–McClure’s depends on it completely, and a smoother that starts from bad odometry can converge to a confident, self-consistent, wrong map. The modern escalation ladder is:

1. **Huber**, when outliers are rare and initialization is poor.
2. **Switchable constraints** (Sünderhauf & Protzel) and their closed-form cousin, dynamic covariance scaling: give every suspect factor its own switch variable and let the optimizer turn it off — robustness as a modeling decision rather than a choice of loss function.
3. **Graduated non-convexity** (Yang et al.), which starts with a convex surrogate and anneals it toward the redescending kernel, so you get the redescending behavior *and* initialization independence.
4. **Certifiable solvers** (SE-Sync and its descendants), which solve a convex relaxation and hand you a certificate that the answer is the global optimum — when the relaxation is tight.

Or you can stop guessing the scale altogether: Chebrolu et al. put ρ itself in a one-parameter family and estimate its shape parameter jointly with the state, so the kernel is tuned by the data rather than by you.

15.6 Why filtering lost

It is worth being precise about the history, because the loser was not stupid and the winner did not appear from nowhere.

The EKF SLAM of Chapter 14 is this graph, filtered. Take the factor graph, and after each step eliminate the previous pose immediately, at whatever linearization point was current. Eliminating x_{t-1} clique-connects everything it touched — which is every landmark it saw — and that clique *is* the EKF’s dense covariance. Marginalization is not a different algorithm; it is this algorithm with a policy of never looking back, and the density and the frozen Jacobians are what that policy costs.

The 2005 book’s GraphSLAM already was Gauss–Newton, without the name. Line them up:

Thrun et al., Chapter 11	This chapter
EIF_initialize (Table 11.1) — chain the controls to get $\mu_{0:t}$	The odometric initial guess y^0
EIF_construct (Table 11.2) — add each control and measurement into Ω, ξ at fixed μ	linearize_graph: linearize and assemble at y
EIF_reduce (Table 11.3) — remove the map from Ω, ξ	schur_marginalize: Schur-complement the landmark block
EIF_solve (Table 11.4) — solve the reduced system, recover the map	Sparse Cholesky solve, then back-substitution
The outer loop of EIF_SLAM (Table 11.5) — repeat with the new μ	The Gauss–Newton iteration

One table, twenty-five years. What the modern formulation adds is not the loop but everything around it: the manifold-correct residuals, the ordering theory that makes the solve fast rather than merely correct, the trust region that makes it converge from a bad start, the robust kernels that make it survive a bad front end, and a sparse linear algebra ecosystem that did not exist in 2000.

And SEIF was the heroic wrong turn. Chapter 12 of the baseline noticed that the *online* information matrix is nearly sparse — most links are weak — and forced it to be exactly sparse by deleting the weak ones, buying constant-time updates. The instinct was right and the mechanism was wrong: deleting a link discards information the filter then does not know it has lost, and the resulting estimator is overconfident. Smoothing gets exact sparsity for free, by never marginalizing in the first place. The debt is still live, though — Chapter 18’s sliding-window estimators must marginalize to bound computation, and they meet SEIF’s dilemma again with better tools.

i NOTE

Deferred, deliberately. This chapter’s solver is *batch*: it re-optimizes everything, every time. Incremental smoothing — iSAM2 and the Bayes tree (Kaess et al., 2012) — re-eliminates only the part of the tree that the newest measurement touched, giving filter-like update rates with smoother-quality answers. No Rust crate implements it yet; fact rs is batch-only, and this book says so rather than pretending otherwise.

15.7 Implementation in Rust

The library mirrors the mathematics one-to-one: a Factor knows its keys, its dimension and how to linearize itself; Values knows how to retract; the optimizer knows nothing about SLAM at all.

RUST crates/ch15_graph/src/factor.rs

```

1 use nalgebra::{DMatrix, DVector, Matrix2, Matrix2x3, Matrix3, Vector2, Vector3};
2 use pr_core::geom::{SE2, se2_right_jacobian_inv}; // Ch. 3
3
4 #[derive(Clone, Copy, PartialEq, Eq, Hash, Debug)]
5 pub enum VarKey {
6     Pose(usize),
7     Landmark(usize),
8 }
9
10 impl VarKey {
11     /// Tangent-space dimension. Poses retract through SE(2); points just add.
12     pub const fn dim(self) -> usize {
13         match self {
14             VarKey::Pose(_) => 3,
15             VarKey::Landmark(_) => 2,
16         }
17     }
18 }
19
20 /// Every unknown in the problem. Two containers, one index.
21 #[derive(Clone, Debug)]
22 pub struct Values {
23     pub poses: Vec<SE2>,
24     pub landmarks: Vec<Vector2<f64>>,
25 }
26
27 impl Values {
28     ///  $y \leftarrow y + [\Delta]$  Delta. The whole reason a heading never gets averaged into nonsense.
29     pub fn retract(&mut self, delta: &DVector<f64>, index: &BlockIndex) {
30         for (slot, key) in index.order.iter().enumerate() {
31             let o = index.offset(slot);
32             match *key {
33                 VarKey::Pose(i) => {
34                     let tau = Vector3::new(delta[o], delta[o + 1], delta[o + 2]);
35                     self.poses[i] = self.poses[i].boxplus(&tau);
36                 }
37                 VarKey::Landmark(l) => {
38                     self.landmarks[l] += Vector2::new(delta[o], delta[o + 1]);
39                 }
40             }
41         }
42     }
43 }
44
45 /// A whitened residual and one Jacobian block per key.
46 pub struct Linearization {
47     pub residual: DVector<f64>,
48     pub jacobians: Vec<DMatrix<f64>>,
49 }
50
51 pub trait Factor {
52     fn keys(&self) -> &[VarKey];

```

```

53     fn dim(&self) -> usize;
54     /// Returns  $\Sigma^{-1/2} r$  and the matching  $\Sigma^{-1/2} J$ , so the caller never
55     /// has to know which sensor this factor came from.
56     fn linearize(&self, v: &Values) -> Linearization;
57     fn kernel(&self) -> Kernel {
58         Kernel::L2
59     }
60 }
61
62 /// One control, or one loop closure:  $r = (x_{i-1} x_j) [-] \delta$ .
63 pub struct BetweenFactor {
64     pub keys: [VarKey; 2],          // [Pose(i), Pose(j)]
65     pub delta: SE2,
66     pub sqrt_info: Matrix3<f64>,
67     pub kernel: Kernel,
68 }
69
70 impl Factor for BetweenFactor {
71     fn keys(&self) -> &[VarKey] { &self.keys }
72     fn dim(&self) -> usize { 3 }
73
74     fn linearize(&self, v: &Values) -> Linearization {
75         let (VarKey::Pose(i), VarKey::Pose(j)) = (self.keys[0], self.keys[1]) else {
76             unreachable!("a between factor joins two poses")
77         };
78         let (xi, xj) = (v.poses[i], v.poses[j]);
79         let rel = xi.inverse() * xj;                      //  $x_{i-1} x_j$ 
80         let r: Vector3<f64> = (self.delta.inverse() * rel).log(); //  $r = \text{rel} [-] \delta$ 
81
82         // Right perturbation:  $x_j$  moves the residual through  $J_{r-1}$ , while  $x_i$ 
83         // must first be transported into  $j$ 's frame -- hence the adjoint.
84         let jr_inv = se2_right_jacobian_inv(&r);
85         let dj = self.sqrt_info * jr_inv;
86         let di = -&dj * (xj.inverse() * xi).adjoint();
87
88         Linearization {
89             residual: DVector::from(self.sqrt_info * r),
90             jacobians: vec![DMatrix::from(di), DMatrix::from(dj)],
91         }
92     }
93
94     fn kernel(&self) -> Kernel { self.kernel }
95 }
96
97 /// Range-bearing sighting: Chapter 10's model, reused verbatim as a factor.
98 pub struct RangeBearingFactor {
99     pub keys: [VarKey; 2],          // [Pose(t), Landmark(j)]
100     pub z: Vector2<f64>,           // (range, bearing)
101     pub sqrt_info: Matrix2<f64>,
102     pub kernel: Kernel,
103 }
104
105 impl Factor for RangeBearingFactor {
106     fn keys(&self) -> &[VarKey] { &self.keys }
107     fn dim(&self) -> usize { 2 }
108
109     fn linearize(&self, v: &Values) -> Linearization {
110         let (VarKey::Pose(t), VarKey::Landmark(j)) = (self.keys[0], self.keys[1]) else {
111             unreachable!("a sighting joins a pose and a landmark")
112         };
113         let x = v.poses[t];

```

```

114     let d = x.inverse_transform_point(&v.landmarks[j]);           // body frame
115     let q = (d.x * d.x + d.y * d.y).max(1e-9);
116     let range = q.sqrt();
117     let r = Vector2::new(range - self.z[0], wrap_angle(d.y.atan2(d.x) - self.z[1]));
118
119     let dh_dd = Matrix2::new(d.x / range, d.y / range, -d.y / q, d.x / q);
120     // ?d/?Delta for a right perturbation of the pose: translate, then rotate.
121     let dd_dx = Matrix2x3::new(-1.0, 0.0, d.y, 0.0, -1.0, -d.x);
122     let dd_dm = x.rotation().matrix().transpose();
123
124     Linearization {
125         residual: DVector::from(self.sqrt_info * r),
126         jacobians: vec![
127             DMatrix::from(self.sqrt_info * dh_dd * dd_dx),
128             DMatrix::from(self.sqrt_info * dh_dd * dd_dm),
129         ],
130     }
131 }
132 }

```

Assembly and the solve. The blocks go into a nalgebra-sparse COO triplet list — sparsity taught through the API before performance enters — and then into faer for the numeric factorization, which is the same engine factrs uses underneath, so graduating later changes the API and not the answer.

RUST crates/ch15_graph/src/optimize.rs

```

1 use faer::linalg::solvers::Llt;
2 use faer::sparse::SparseColMat;
3 use nalgebra_sparse::CooMatrix;
4
5 pub struct System {
6     pub omega: SparseColMat<usize, f64>,
7     pub b: DVector<f64>,
8     pub cost: f64,
9 }
10
11 /// `linearize_graph` -- Table 15.1. One pass over the factors; O(1) work each.
12 pub fn linearize_graph(g: &FactorGraph, v: &Values, index: &BlockIndex) -> System {
13     let n = index.total();
14     let mut coo = CooMatrix::<f64>::new(n, n);
15     let mut b = DVector::zeros(n);
16     let mut cost = 0.0;
17
18     for f in &g.factors {
19         let lin = f.linearize(v);
20         let e = lin.residual.norm();
21         // IRLS: one scalar per factor is the entire cost of robustness.
22         let w = f.kernel().weight(e);
23         cost += f.kernel().rho(e);
24
25         for (a, key_a) in f.keys().iter().enumerate() {
26             let oa = index.offset_of(key_a);
27             let ja = &lin.jacobians[a];
28             b.rows_mut(oa, ja.ncols()).add_assign(w * ja.transpose() * &lin.residual);
29
30             for (c, key_c) in f.keys().iter().enumerate() {

```

```

31         let oc = index.offset_of(key_c);
32         let block = w * ja.transpose() * &lin.jacobians[c];
33         for p in 0..block.nrows() {
34             for q in 0..block.ncols() {
35                 // Duplicate triplets are summed on conversion to CSC,
36                 // which is exactly the accumulation  $\Omega = \Sigma J^T J$  needs.
37                 coo.push(oa + p, oc + q, block[(p, q)]);
38             }
39         }
40     }
41 }
42 }
43
44 System { omega: SparseColMat::try_new_from_triplets(&coo).unwrap(), b, cost }
45 }
46
47 pub struct LmConfig {
48     pub max_iterations: usize,
49     pub tol: f64,
50     pub lambda0: f64,
51 }
52
53 /// `levenberg_marquardt` -- Table 15.3.
54 pub fn levenberg_marquardt(
55     g: &FactorGraph,
56     init: Values,
57     index: &BlockIndex,
58     cfg: &LmConfig,
59     ordering: Ordering,
60 ) -> (Values, Report) {
61     let mut y = init;
62     let mut lambda = cfg.lambda0;
63     let mut cost = g.cost(&y);
64     let mut report = Report::default();
65
66     for _ in 0..cfg.max_iterations {
67         let sys = linearize_graph(g, &y, index);
68         let damped = sys.damped(lambda); //  $\Omega + \lambda \text{diag } \Omega$ 
69         // The symbolic pattern only changes when the graph does, so the
70         // fill-reducing ordering is computed once and reused every iteration.
71         let llt = Llt::try_new_with_symbolic(ordering.symbolic(&damped), damped.as_ref())
72             .expect("Omega is singular -- is a prior factor missing?");
73         let delta = llt.solve(&sys.b);
74
75         let mut trial = y.clone();
76         trial.retract(&delta, index);
77         let trial_cost = g.cost(&trial);
78
79         // Gain ratio: what the linear model promised vs. what we got.
80         let predicted = 0.5 * delta.dot(&(lambda * sys.diag().component_mul(&delta) - &
81 sys.b));
82         let rho = (cost - trial_cost) / predicted;
83
84         if trial_cost < cost {
85             y = trial;
86             cost = trial_cost;
87             lambda = (lambda * (1.0 - (2.0 * rho - 1.0).powi(3))).max(1.0 / 3.0).max(1e
88 -9);
89             report.push_accepted(cost, delta.norm(), lambda);
90             if report.relative_decrease() < cfg.tol { break; }
91         } else {

```

```

96         lambda *= 10.0;
97         report.push_rejected(lambda);
98     }
99 }
100 (y, report)
101 }

```

And the worked example, pinned by a test, exactly as in every other chapter of this book:

RUST crates/ch15_graph/tests/micro_1d.rs

```

1 use approx::assert_relative_eq;
2 use ch15_graph::{FactorGraph, LinearFactor, Values, gauss_newton, linearize_graph};
3
4 /// The 1-D chain from sec.15.4: prior  $y_0 = 0$ , odometry +1, +1, and a fix  $y_2 = 1.5$ .
5 /// Every factor has unit information, so  $\Omega$  is tridiagonal(-1, 2, -1) and the
6 /// whole thing can be checked on paper.
7 #[test]
8 fn micro_chain_matches_the_hand_calculation() {
9     let g = FactorGraph::from(vec![
10         LinearFactor::new(&[0], &[1.0], 0.0, 1.0), // prior
11         LinearFactor::new(&[0, 1], &[-1.0, 1.0], 1.0, 1.0), // odometry
12         LinearFactor::new(&[1, 2], &[-1.0, 1.0], 1.0, 1.0), // odometry
13         LinearFactor::new(&[2], &[1.0], 1.5, 1.0), // absolute fix
14     ]);
15     let index = g.default_index();
16     let init = Values::scalars(vec![0.0, 0.0, 0.0]);
17
18     let sys = linearize_graph(&g, &init, &index);
19     assert_relative_eq!(sys.cost, 2.125, epsilon = 1e-12);
20     assert_relative_eq!(sys.omega[(0, 0)], 2.0, epsilon = 1e-12);
21     assert_relative_eq!(sys.omega[(0, 1)], -1.0, epsilon = 1e-12);
22     assert_relative_eq!(sys.omega[(0, 2)], 0.0, epsilon = 1e-12); // not neighbors
23     assert_relative_eq!(sys.b[2], -2.5, epsilon = 1e-12);
24
25     let (y, report) = gauss_newton(&g, init, &index, &Default::default());
26     assert_relative_eq!(y.scalars[0], -0.125, epsilon = 1e-9);
27     assert_relative_eq!(y.scalars[1], 0.750, epsilon = 1e-9);
28     assert_relative_eq!(y.scalars[2], 1.625, epsilon = 1e-9);
29
30     // Both intervals shrink by the same amount: equal stiffness, equal share.
31     assert_relative_eq!(y.scalars[1] - y.scalars[0], 0.875, epsilon = 1e-9);
32     assert_relative_eq!(y.scalars[2] - y.scalars[1], 0.875, epsilon = 1e-9);
33     // A linear problem is solved by the *first* Gauss-Newton step; every later
34     // iteration moves nothing, which is how the loop knows it has converged.
35     assert!(report.steps()[0].step_norm > 1.0);
36     assert!(report.steps().last().unwrap().step_norm < 1e-12);
37     assert_relative_eq!(report.final_cost(), 1.0 / 32.0, epsilon = 1e-12);
38 }

```

NOTE

The widgets in this chapter run the TypeScript port of exactly this code, in `web/lib/optim/`. Its self-checks (`lib/optim/__checks_ch15__.ts`) assert the same three numbers, every analytic Jacobian against a finite difference of the $\boxplus$ perturbation, $w = \rho'(e)/e$ for all four kernels, and that the Schur-reduced solve agrees with the full solve to 10^{-7} . Prose, Rust and pixels are pinned to the same arithmetic.

15.7.1 The same graph, in factrs

Once the ideas are yours, stop hand-rolling. [factrs 0.3](#) is a GTSAM-shaped Rust library — typed variables, typed factors, automatic differentiation through dual numbers, GN and LM, robust kernels, *serde*, *rerun* output — and it builds the identical Apartment graph in about thirty lines:

RUST `crates/ch15_graph/examples/factrs_same_graph.rs`

```

1 use factrs::{
2     assign_symbols,
3     core::{BetweenResidual, GaussNewton, Graph, Huber, PriorResidual, Values},
4     dtype, fac,
5     linalg::{Const, ForwardProp, Numeric, VectorX, vectorx},
6     residuals::Residual2,
7     traits::*,
8     variables::{SE2, VectorVar2},
9 };
10
11 // Type-tagged keys: X(3) is a pose and L(3) is a landmark, and the compiler
12 // will not let you hand one to a factor that expects the other.
13 assign_symbols!(X: SE2; L: VectorVar2);
14
15 /// factrs ships prior and between residuals; a range-bearing sighting is ours
16 /// to write. The only method that matters is `residual2`, generic over
17 /// `Numeric` -- factrs differentiates it with dual numbers, so this file
18 /// contains no hand-derived Jacobians at all.
19 #[derive(Clone, Debug)]
20 #[factrs::mark]
21 pub struct RangeBearing {
22     range: dtype,
23     bearing: dtype,
24 }
25
26 impl Residual2 for RangeBearing {
27     type V1 = SE2;
28     type V2 = VectorVar2;
29     type DimIn = Const<5>;
30     type DimOut = Const<2>;
31     type Differ = ForwardProp<Self::DimIn>;
32
33     fn residual2<T: Numeric>(&self, x: SE2<T>, m: VectorVar2<T>) -> VectorX<T> {
34         let d = x.inverse().apply(m.into()); // landmark in the body frame
35         let range = (d[0] * d[0] + d[1] * d[1]).sqrt();
36         vectorx![
37             range - T::from(self.range),

```

```

38         d[1].atan2(d[0]) - T::from(self.bearing)
39     ]
40 }
41 }
42
43 fn main() {
44     let data = apartment_loop(42); // the same dataset the widget optimizes
45     let mut values = Values::new();
46     let mut graph = Graph::new();
47
48     for (i, p) in data.odometry_chain().enumerate() {
49         values.insert(X(i), p);
50     }
51     for (l, m) in data.landmark_guesses().enumerate() {
52         values.insert(L(l), VectorVar2::new(m.x, m.y));
53     }
54
55     graph.add_factor(fac![PriorResidual::new(data.first_pose()), X(0), 0.01 as std]);
56
57     for e in data.edges() {
58         // Loop closures get the robust kernel; odometry does not need one.
59         graph.add_factor(match e.kind {
60             EdgeKind::Odometry => fac![BetweenResidual::new(e.delta), (X(e.i), X(e.j)),
0.04 as std],
61             EdgeKind::Loop => {
62                 fac![BetweenResidual::new(e.delta), (X(e.i), X(e.j)), 0.05 as std, Huber::
default()]
63             }
64         });
65     }
66     for z in data.observations() {
67         graph.add_factor(fac![
68             RangeBearing::new(z.range, z.bearing),
69             (X(z.pose), L(z.lm)),
70             (0.18, 0.085) as std
71         ]);
72     }
73
74     let mut opt: GaussNewton = GaussNewton::new_default(graph);
75     println!("{:?}", opt.optimize(values).unwrap());
76 }

```

It agrees with our solver to 10^{-8} and runs at comparable speed, because it is calling the same sparse Cholesky underneath. What the rewrite buys is type-tagged keys, automatic differentiation instead of hand-derived Jacobians, and *serde* graph I/O. What no Rust crate gives you yet is the Bayes tree; for incremental smoothing, GTSAM is still the reference implementation.

15.8 Putting it together

Run the whole thing on the Apartment loop that opened this chapter — the same corridor, the same biased wheels, the same corner reflectors that the filter of Chapter 14 had to swallow one at a time — and the numbers close Part V's arc:

```
<div className="not-prose my-6 overflow-x-auto">
```


	Trajectory RMSE	Landmark RMSE	Final drift	Loop closure
Dead reckoning	0.581 m	—	1.66 m	contradicted
EKF SLAM (Chapter 14)	overconfident: the map is self-consistent and wrong	—	—	absorbed at a frozen linearization
Batch smoothing (this chapter)	0.197 m	0.208 m	0.05 m	satisfied to 0.31σ

</div>

Seven Levenberg–Marquardt steps, 232 factors, 189 unknowns, J from 3780 to 116.5 — a mean residual of 0.88σ across the whole graph, with the worst single factor at 2.6σ , which is what a correctly weighted least-squares fit is supposed to look like. The numeric factorization is 34 thousand floating-point operations under a minimum-degree ordering: microseconds of arithmetic. The reason this beats the filter is not that it is cleverer. It is that it kept the past, so it could change its mind about it.

Three doors lead out of here.

Chapter 16 removes the known-correspondence assumption. Scan matching produces the between-factors from raw LiDAR, place recognition proposes the loop closures, and the robust kernels of this chapter are what stand between a false proposal and a ruined map — the front end and back end finally meet, as RustSLAM-2D.

Chapter 17 attacks the same full posterior from the opposite side: sample the trajectory, and the map factorizes into independent little Gaussians. Same posterior, completely different computational bet.

Chapter 18 scales the graph to cameras and IMUs, where the number of variables outruns the compute budget and something must be marginalized after all. That is where SEIF’s ghost comes back — and where the fill-in you learned to count in the Sparsity Scope becomes an engineering decision with real consequences.

15.9 Exercises

1 FOUNDATION •• Assemble the micro chain by hand

Using only the Jacobian rows $(1, 0, 0)$, $(-1, 1, 0)$, $(0, -1, 1)$, $(0, 0, 1)$, accumulate Ω and b for the 1-D example and solve $\Omega\Delta = -b$ by elimination. Then answer in one sentence why *both* odometry intervals shrank to 0.875 when only one of them is adjacent to the disagreeing measurement. Finally, make the prior ten times stiffer ($\sigma_0 = 0.1$) and predict, before recomputing, which residual grows.

2 FOUNDATION •• The pattern claim, and the clique it hides

Prove that $\Omega_{[j][k]} \neq 0$ if and only if variables j and k appear together in some factor. Draw the block pattern for a graph of 5 poses and 2 landmarks, where landmark 1 is seen from poses 0, 2 and 4. Then Schur-eliminate landmark 1 and show that the pose blocks $(0, 2), (0, 4), (2, 4)$ become nonzero — citing the Gaussian marginalization identity you used. How many fill edges does eliminating landmark 1 create in general, as a function of how many poses saw it?

3 FOUNDATION ••• Huber's weight, from stationarity

Derive $w = \rho'(e)/e$ from the stationarity condition of $\sum_k \rho(e_k)$, being careful about the chain rule through $e_k = \|r_k(y)\|$. Then show that L2's influence $\rho'(e) = e$ is unbounded while Huber's saturates at k , and compute the asymptotic weight of a 30σ residual under Huber($k=1$), Cauchy($c=1$) and Geman–McClure($c=1$). Which of the three still moves the map, and by how much relative to a 1σ inlier?

4 CONCEPTUAL •• Predict, then verify: which ordering wins?

In [w15.2](#), use the *Loop* graph. Before touching anything, predict whether chronological or landmarks-first gives the smaller $\text{nnz}(L)$, and write down your reasoning. Check. Now switch to the *One landmark, seen by all* preset and predict again — your rule of thumb should fail here. Explain the failure in terms of induced cliques, in one paragraph, and state the graph property that actually decides the winner.

5 CONCEPTUAL •• Predict, then verify: where robustness runs out

In [w15.1](#), set the kernel to Huber with $k = 1$ and raise the false-closure error until the RMSE readout doubles. Record that magnitude. Now switch to Geman–McClure and find the magnitude at which it fails. Then use [w15.4](#) to explain the difference in terms of the two influence functions, and — using the bad-init row of the stat tiles — write the one-paragraph pitch for graduated non-convexity that you would give to a colleague who wants to ship Geman–McClure tomorrow.

6 PRACTICAL •• A bearing-only factor and a Cauchy kernel

Implement `BearingOnlyFactor` (one scalar residual, the wrapped bearing) and add `Kernel::Cauchy` to the kernel enum. Verify your Jacobian against a finite-difference of the $\mathbb{H}$ perturbation to 10^{-6} — this is the single most common source of a "my optimizer diverges" bug, and the test takes ten lines. Then build a graph in which a landmark is observed by bearing only from two poses and show that Ω is singular when the two bearings are parallel. What does that singularity mean physically?

7 PRACTICAL • • • Ordering, measured

Implement minimum-degree ordering in `order.rs` (greedy, simulating fill as you go) and compare `nnz(L)` and wall-clock factorization time against chronological and landmarks-first on a 500-pose synthetic loop. Plot `nnz(L)` versus problem size for all three on log axes and report the empirical exponents. Then port the same graph to `factrs 0.3` and to `tiny-solver 0.18` and report agreement (should be 10^{-8}) and relative timing. Which of the three implementations would you actually ship, and why?

15.10 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 11 is this chapter's ancestor — the construct–reduce–solve pipeline that, iterated, is Gauss–Newton, and the source of the spring-mass framing this chapter runs on. Chapter 12's SEIF is the cautionary tale. [link](#)

Dellaert, F. and Kaess, M. (2006). Square Root SAM: Simultaneous Localization and Mapping via Square Root Information Smoothing *The International Journal of Robotics Research* 25(12), 1181–1203.

The paper that made smoothing beat filtering, by pointing out that the sparse matrix factorization community had already solved the hard part. The ordering section above is its argument, thirty years on. doi:10.1177/0278364906072768

Kümmerle, R., Grisetti, G., Strasdat, H., Konolige, K., and Burgard, W. (2011). g2o: A General Framework for Graph Optimization *IEEE International Conference on Robotics and Automation (ICRA)*, 3607–3613.

The back end a generation of SLAM systems was built on, and the clearest statement of the graph-optimization interface that `factrs` and `Ceres` also implement. doi:10.1109/ICRA.2011.5979949

Sünderhauf, N. and Protzel, P. (2012). Switchable Constraints for Robust Pose Graph SLAM *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1879–1884.

Robustness as a modeling decision rather than a loss function: give every loop closure a switch variable and let the optimizer turn it off. Step 2 of this chapter's escalation ladder. doi:10.1109/IROS.2012.6385590

Dellaert, F. and Kaess, M. (2017). Factor Graphs for Robot Perception *Foundations and Trends in Robotics* 6(1–2), 1–139.

The standard modern treatment, and the source of this chapter's formulation. Read it next; its elimination-game chapter is the rigorous version of the Sparsity Scope. doi:10.1561/23000000043

Yang, H., Antonante, P., Tzoumas, V., and Carlone, L. (2020). Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection *IEEE Robotics and Automation Letters* 5(2), 1127–1134.

How to get a redescending kernel’s outlier immunity without its initialization dependence: anneal the surrogate from convex to non-convex. The answer to the failure w15.4 shows. doi:10.1109/LRA.2020.2965893

Chebrolu, N., Lä be, T., Vysotska, O., Behley, J., and Stachniss, C. (2021). Adaptive Robust Kernels for Non-Linear Least Squares Problems *IEEE Robotics and Automation Letters* 6(2), 2240–2247.

Stop hand-picking ρ and its scale: put the kernel in a one-parameter family and estimate the shape parameter jointly with the state. doi:10.1109/LRA.2021.3061331

Doherty, K., Papalia, A., Huang, Y., Rosen, D., Englot, B., and Leonard, J. (2024). MAC: Graph Sparsification by Maximizing Algebraic Connectivity *arXiv:2403.19879 [cs.RO]*.

The other way to control fill-in: instead of ordering the graph you have, choose which measurements to keep, using the spectral quantity that predicts estimation error. Where Chapters 18 and 24 pick this up. [link](#)

Scan Matching and Pose-Graph SLAM

MAPPING AND SLAM · Advanced · 60 min



In cyclic environments the robot has to establish correspondence to previously gathered data with potentially unbounded odometric error, and has to revise pose estimates backwards in time.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics (1999–2000 draft), Chapter 14

Chapter 15 built a back end: a factor graph and a sparse Gauss–Newton solver that swallows constraints and returns a maximum-a-posteriori trajectory. It never said where the constraints come from. Odometry factors are easy — the wheels report something. Loop-closure factors are the hard ones, and they are the only reason the back end is worth having.

This chapter builds the **front end**: the machinery that turns raw LiDAR sweeps into relative-pose measurements, and then decides which of those measurements are safe to believe. The punchline is architectural rather than algorithmic. SLAM in practice is not one method; it is a fast, greedy, local matcher feeding a slow, global, honest optimizer, with a hypothesis test standing between them. Every piece of it — ICP as maximum likelihood, NDT as a Gaussian mixture, loop verification as a χ^2 gate — is probability theory you have used since Chapter 5, wearing a geometer’s coat.

At the end the two halves are wired together into **RustSLAM-2D**, the book’s first complete SLAM system, and run on a full lap of the Apartment. We will measure exactly what it fixes, and — the part that most treatments skip — exactly what it cannot.

🎯 AFTER THIS CHAPTER YOU CAN

- Derive point registration as maximum likelihood, and the closed-form rigid alignment that solves it once correspondences are fixed
- Explain why ICP converges, why only to a local minimum, and how to recognize the two geometries that break it
- Contrast the point-to-point, point-to-plane, and NDT objectives by the shape of their cost surfaces
- Write the SE(2) pose-graph residual and its Jacobians, and hand them to Chapter 15's optimizer unchanged
- Gate a loop-closure candidate with a χ^2 test, and say precisely what a false positive costs
- Assemble a front end and a back end into RustSLAM-2D and measure what each half actually buys

📖 ASSUMED

- SE(2), $\exp/\log$, $\boxplus/\boxminus$ and the adjoint (Chapter 3)
- The simulated LiDAR and the Apartment world (Chapter 4)
- Odometry as a noisy prior (Chapter 9) and likelihood fields (Chapter 10)
- Sparse Gauss–Newton on a factor graph, and robust kernels (Chapter 15)

16.1 The problem: constraints have to come from somewhere

Rusty drives down the Apartment's corridor and back. His wheels report how far he went. They are wrong, in the specific way wheels are always wrong: slightly, consistently, and without any signal that anything has gone amiss. Over one eighteen-metre lap the raw odometry ends up **2.4 metres** from the truth, and a map assembled from it would not be a map of the Apartment at all — it would be two overlapping maps, drawn metres apart.

💻 INTERACTIVE · w16.2

Loop Snap

A loop closure does not fix where the robot is. It rewrites where the robot was — and the map with it.

Run this simulation in the web edition: [/chapters/ch16-scan-matching](#) #w16.2

Watch a full lap before reading on. There are three separate things happening and it is worth naming them now, because the rest of the chapter is those three things in order.

The scan is a far better odometer than the wheels. The orange path leaves the building; the purple one stays within a metre. Nothing about the robot changed — the same wheels, the same noise. What changed is that consecutive LiDAR sweeps overlap, and two overlapping views of the same wall constrain the motion between them. The next section turns that constraint into a likelihood.

Better is not good. The purple path still drifts, because every registration is slightly wrong and the errors compound exactly as Chapter 9's do. Look at the blue map dots on the return leg: they land beside the outbound ones, not on them. A map that disagrees with itself is the most useful error signal a robot ever gets, because it needs no ground truth to notice.

The correction runs backwards. When a green loop factor lands and the optimizer runs, the pose that moves is not the robot's current one. It is a pose from thirty seconds ago, and the ones either side of it, and the map they wrote. Loop closure is not a localization update. It is a retroactive edit of history.

💡 THE ONE SENTENCE VERSION

A scan matcher turns two overlapping sweeps into a relative-pose measurement with an information matrix; a loop detector proposes which distant pairs of sweeps might overlap; a χ^2 gate decides which proposals to believe; and a pose graph reconciles all of them at once.

16.2 Building intuition: two scans that want to snap together

Take two sweeps of the same room from positions half a metre apart. Overlay them at the wrong offset and they look like a double exposure. Slide one until the walls coincide and the double exposure resolves — and the transform you applied to make that happen *is* the motion between the two viewpoints. That is the entire idea. Everything else is deciding which point corresponds to which, and what to do when you are wrong about it.

🖥️ INTERACTIVE · w16.1

ICP Stepper

ICP does not find the alignment; it finds the nearest one. Start it far enough away and it converges, confidently, onto the wrong wall.

Run this simulation in the web edition: [/chapters/ch16-scan-matching #w16.1](#)

Two observations from that widget drive the mathematics.

Correspondence and alignment are separate problems, and each is easy given the other. If you knew which target point each source point came from, the best rigid transform has a closed form — derived below in four lines of algebra. If you knew the transform, the correspondences are a nearest-neighbour lookup. You know neither, so you alternate — and that alternation is the algorithm, with all its virtues and its one fatal flaw.

ICP does not find *the* alignment. It finds the nearest one. Room A of the Apartment is 4.0 m × 3.8 m. Rotate the initial guess past about 40° and ICP settles, in a dozen iterations, onto the room turned a quarter turn: 0.95 m and 90° from the truth, with a residual of 0.13 m that looks perfectly respectable. Nothing in the algorithm can tell the difference. This is why `verify_loop` exists, and why a scan matcher is never allowed to add a factor to a pose graph unsupervised.

16.3 The mathematics

16.3.1 Notation

$\mathcal{P}, \mathcal{Q}$	Source (new) and target (reference) point sets, each in its own sensor frame.
$T = (\mathbf{R}, \mathbf{t}) \in \text{SE}(2)$	The registration transform, acting on a point by rotating it and then translating it.
$c(k)$	Correspondence: which target point the k-th source point is matched to. Chapter 10's correspondence variable, reborn geometrically.
τ	Correspondence rejection radius. Pairs longer than this are discarded — the truncation that makes ICP robust and its cost piecewise.
$\mathbf{n}_k$	Unit surface normal at the matched target point, estimated by local PCA over its neighbours.
μ_i, Σ_i	Mean and covariance of the Gaussian fitted to NDT cell i.
Z_{ij}, Ω_{ij}	Relative-pose measurement between poses i and j, and its information matrix.
$\mathbf{e}_{ij}$	The pose-graph residual: a 3-vector in the tangent space of SE(2), zero when the measurement and the two poses agree.
$\chi^2_{3, 0.95} = 7.815$	The 95% quantile of the chi-squared distribution with three degrees of freedom — the loop-closure acceptance threshold.

16.3.2 Registration is maximum likelihood

Assume — and it is an assumption, whose bill arrives when we ask a match how confident it is — that each matched target point is a noisy observation of the transformed source point:

$$\mathbf{q}_{c(k)} = T \mathbf{p}_k + \boldsymbol{\epsilon}_k, \quad \boldsymbol{\epsilon}_k \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}).$$

The beams are taken to be conditionally independent given the pose, exactly as in Chapter 10's beam model and with exactly the same caveat. Then the likelihood factorizes, the log turns the product into a sum, and the Gaussian turns the sum into squares:

$$T^\star = \arg \max_T \prod_k p(\mathbf{q}_{c(k)} \mid T, \mathbf{p}_k) = \arg \min_{T \in \text{SE}(2)} \sum_k \|T \mathbf{p}_k - \mathbf{q}_{c(k)}\|^2.$$

Least squares *is* maximum likelihood here. That one line is the bridge to Chapter 15: a scan match is a factor, its cost is a negative log-likelihood, and when the Gaussian assumption fails — which is precisely when a correspondence is wrong — you are licensed to wrap the residual in a robust kernel $\rho(\cdot)$ rather than invent a new algorithm.

16.3.3 Rigid alignment in closed form

DERIVATION The minimizer over SE(2) with correspondences fixed

$$\theta^\star = \text{atan2}(W_{21} - W_{12}, W_{11} + W_{22})$$

Statement. With correspondences fixed, the minimizer of $\sum_k \|\mathbf{R} \mathbf{p}_k + \mathbf{t} - \mathbf{q}_k\|^2$ over SE(d) is

$$\mathbf{R}^\star = \mathbf{U} \text{diag}(1, \dots, 1, \det(\mathbf{U} \mathbf{V}^\top)) \mathbf{V}^\top, \quad \mathbf{t}^\star = \bar{\mathbf{q}} - \mathbf{R}^\star \bar{\mathbf{p}},$$

where $\mathbf{W} = \sum_k (\mathbf{q}_k - \bar{\mathbf{q}})(\mathbf{p}_k - \bar{\mathbf{p}})^\top = \mathbf{U} \mathbf{S} \mathbf{V}^\top$ (Arun, Huang and Blostein, 1987).

Step 1 — eliminate the translation. For fixed $\mathbf{R}$ the cost is quadratic in $\mathbf{t}$; setting the gradient to zero gives $\mathbf{t} = \bar{\mathbf{q}} - \mathbf{R} \bar{\mathbf{p}}$. Substituting it back centers both clouds: write $\mathbf{a}_k = \mathbf{p}_k - \bar{\mathbf{p}}$ and $\mathbf{b}_k = \mathbf{q}_k - \bar{\mathbf{q}}$, and the problem becomes $\min_{\mathbf{R}} \sum_k \|\mathbf{R} \mathbf{a}_k - \mathbf{b}_k\|^2$.

Step 2 — expand. Since $\mathbf{R}$ is orthogonal, $\|\mathbf{R} \mathbf{a}_k\|^2 = \|\mathbf{a}_k\|^2$, so

$$\sum_k \|\mathbf{R} \mathbf{a}_k - \mathbf{b}_k\|^2 = \sum_k \|\mathbf{a}_k\|^2 + \sum_k \|\mathbf{b}_k\|^2 - 2 \sum_k \mathbf{b}_k^\top \mathbf{R} \mathbf{a}_k.$$

Only the last term depends on $\mathbf{R}$, and $\sum_k \mathbf{b}_k^\top \mathbf{R} \mathbf{a}_k = \text{tr}(\mathbf{R}^\top \mathbf{W})$. Minimizing the cost is maximizing that trace.

Step 3 — orthogonal Procrustes. With $\mathbf{W} = \mathbf{USV}^\top$ and $\mathbf{M} = \mathbf{V}^\top \mathbf{R}^\top \mathbf{U}$ orthogonal,

$$\text{tr}(\mathbf{R}^\top \mathbf{W}) = \text{tr}(\mathbf{R}^\top \mathbf{USV}^\top) = \text{tr}(\mathbf{MS}) = \sum_i m_{ii} s_i \leq \sum_i s_i,$$

because every entry of an orthogonal matrix satisfies $|m_{ii}| \leq 1$ and the singular values s_i are non-negative. Equality needs $m_{ii} = 1$ for all i , i.e. $\mathbf{M} = \mathbf{I}$, i.e. $\mathbf{R} = \mathbf{UV}^\top$.

Step 4 — exclude reflections. $\mathbf{UV}^\top$ is orthogonal but need not have determinant $+1$; when the data are degenerate (all points collinear, or noise dominating) it can come back as a reflection, which is not a rigid motion. Replacing the last diagonal entry with $\det(\mathbf{UV}^\top)$ gives the best *rotation*, at the cost of $s_{\min}$ in the trace.

Step 5 — the planar case. In 2-D none of this needs an SVD. Writing

$$\mathbf{R} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix},$$

$$\text{tr}(\mathbf{R}^\top \mathbf{W}) = \cos \theta (W_{11} + W_{22}) + \sin \theta (W_{21} - W_{12}),$$

a single sinusoid in θ , maximized at $\theta^* = \text{atan2}(W_{21} - W_{12}, W_{11} + W_{22})$. Note $W_{11} + W_{22} = \sum_k \mathbf{a}_k \cdot \mathbf{b}_k$ and $W_{21} - W_{12} = \sum_k \mathbf{a}_k \times \mathbf{b}_k$: a dot product and a cross product, and nothing else. Because atan2 returns an angle, the result is a rotation by construction — the determinant correction of Step 4 comes for free. ■

16.3.4 Why the iteration converges, and only locally

DERIVATION ICP as alternating minimization — the EM echo

$$J(T_{i+1}, c_{i+1}) \leq J(T_{i+1}, c_i) \leq J(T_i, c_i)$$

Statement. Alternating (a) $c(k) \leftarrow \arg \min_j \|T \mathbf{p}_k - \mathbf{q}_j\|$ and (b) the closed-form alignment of the previous derivation monotonically decreases the joint objective and converges.

Proof. Define the joint cost over both arguments,

$$J(T, c) = \sum_k \min(\|T \mathbf{p}_k - \mathbf{q}_{c(k)}\|^2, \tau^2).$$

Step (a) minimizes $J(T_i, \cdot)$ exactly: for each k independently it selects the

nearest target, and the truncation at τ^2 is applied pointwise, so no other assignment can do better. Step (b) minimizes $J(\cdot, c_i)$ exactly over the pairs that survived truncation. Hence J is non-increasing along the iteration. It is bounded below by zero. A monotone bounded sequence converges. ■

The EM echo. Step (a) assigns latent variables given parameters; step (b) re-estimates parameters given assignments. That is expectation–maximization with hard assignments — the same structure as Chapter 10’s EM for the beam-model intrinsics, and it inherits EM’s guarantee and EM’s weakness in equal measure.

What convergence does *not* mean. J converges; T_i converges to a stationary point of J ; nothing says that point is the global minimum. Two geometries produce spurious minima reliably:

- **Rotational near-symmetry.** A room whose walls nearly map onto themselves under a quarter turn has a second, deep basin. Room A is 4.0 m $\times$ 3.8 m, and [w16.1](#) falls into that basin from any initial heading error beyond roughly 40°.
- **Picket-fence aliasing.** A row of equally spaced features (railings, radiator fins, chair legs) produces a minimum at every multiple of the spacing, and the correct one is not distinguished by the cost.

Both are failures of the *objective*, not of the solver. No amount of iterating fixes them, which is why the answer is a better initial guess (the motion prior) and a verification step.

16.3.5 Point-to-plane: stop fighting the wall

Point-to-point penalizes the whole displacement between a matched pair. But if both points lie on the same flat wall, sliding one along the wall changes nothing physical — the penalty is measuring an artefact of which point happened to be nearest, not a real disagreement. Point-to-plane penalizes only the component along the surface normal (Chen and Medioni, 1992):

$$T^\star = \arg \min_T \sum_k \left(\mathbf{n}_k^\top (\mathbf{R}\mathbf{p}_k + \mathbf{t} - \mathbf{q}_{c(k)}) \right)^2.$$

 **DERIVATION** The 3 $\times$ 3 normal equations for point-to-plane in SE(2)

$$\mathbf{a}_k^\top = (\tilde{\mathbf{p}}_k \times \mathbf{n}_k, n_x, n_y)$$

There is no closed form: the residual is linear in $\mathbf{t}$ but trigonometric in θ . Linearize the rotation about the current estimate with $\mathbf{R} \approx \mathbf{I} + \theta \mathbf{J}$, $\mathbf{J} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, and every residual becomes affine in $\xi = (\theta, t_x, t_y)$:

$$r_k = \mathbf{n}_k^\top (\mathbf{p}_k - \mathbf{q}_{c(k)}) + \theta \mathbf{n}_k^\top \mathbf{J} \mathbf{p}_k + \mathbf{n}_k^\top \mathbf{t} = \mathbf{a}_k^\top \xi - b_k,$$

with

$$\mathbf{a}_k^\top = (\mathbf{n}_k^\top \tilde{\mathbf{p}}_k, n_{k,x}, n_{k,y}) = (\tilde{p}_{k,x} n_{k,y} - \tilde{p}_{k,y} n_{k,x}, n_{k,x}, n_{k,y}), \quad b_k = -\mathbf{n}_k^\top (\mathbf{p}_k - \mathbf{q}_{c(k)}).$$

The tilde matters. $\tilde{\mathbf{p}}_k = \mathbf{p}_k - \bar{\mathbf{p}}$ measures the source point from the cloud's centroid, so the linearized rotation happens about the centroid rather than the world origin. Rotating about an origin ten metres away couples θ to $\mathbf{t}$ so strongly that the 3×3 normal matrix loses six digits of conditioning for no reason.

Minimizing $\sum_k (\mathbf{a}_k^\top \xi - b_k)^2$ gives the normal equations $(\sum_k \mathbf{a}_k \mathbf{a}_k^\top) \xi = \sum_k b_k \mathbf{a}_k$ — one Gauss–Newton step of Chapter 15, specialized. Re-associate and repeat.

Rank. $\mathbf{H} = \sum_k \mathbf{a}_k \mathbf{a}_k^\top$ is singular exactly when the $\mathbf{a}_k$ span fewer than three dimensions. The canonical case is an infinite corridor: every normal is $\pm(0, 1)$, so every row is $(\tilde{p}_{k,x}, 0, \pm 1)$ and the matrix has rank 2. The null vector is "translate along the corridor" — the motion the sensor genuinely cannot see.

Why this must be damped, not merely nudged. In that null direction the gradient is zero too, so the solve is $0/0$. An absolute ridge of 10^{-9} divides the numerical dust in $\mathbf{g}$ by the numerical dust in $\mathbf{H}$ and can slide the scan a metre down the hallway. A ridge proportional to $\text{tr}(\mathbf{H})$ resolves the unconstrained direction to *zero motion* instead, which is the honest answer: the scan says nothing here, so keep the prediction. The library uses $\lambda = 10^{-4} \text{tr}(\mathbf{H})/3$.

Empirically the payoff is convergence rate, not accuracy. On the w16.1 scene both variants reach the same answer, but point-to-plane needs 5–9 iterations where point-to-point needs 10–30, because the cost is flat *along* surfaces and the solver stops spending iterations crabbing sideways.

16.3.6 NDT: remove correspondences entirely

ICP's cost is piecewise. Nudge the pose and a correspondence flips, and the objective takes a step; between flips it is a smooth quadratic; across them it is

not even differentiable. The Normal Distributions Transform (Biber and Straßer, 2003) removes the discrete variable altogether. Partition the target into cells, fit one Gaussian per occupied cell, and score a candidate transform by how likely the source points are under that mixture:

$$s(T) = \sum_k \sum_{i \in \mathcal{N}(k)} \exp\left(-\frac{1}{2} \mathbf{d}_{ik}^\top \Sigma_i^{-1} \mathbf{d}_{ik}\right), \quad \mathbf{d}_{ik} = T\mathbf{p}_k - \boldsymbol{\mu}_i.$$

Up to mixture weights this is the log-likelihood of the sweep under a Gaussian-mixture map. Read the other way it is Chapter 10's likelihood field with an anisotropic kernel *fitted per cell* rather than one isotropic σ everywhere — which is exactly what lets it represent "this cell is a wall running north-east" instead of merely "this cell is occupied".

DERIVATION Newton's method on the NDT score

$$H_{ij} \approx \sum_k s_k (\partial_i \mathbf{d})^\top \Sigma^{-1} (\partial_j \mathbf{d})$$

Write $\mathbf{C} = \Sigma_i^{-1}$, $s_k = \exp(-\frac{1}{2} \mathbf{d}^\top \mathbf{C} \mathbf{d})$, and $\xi = (t_x, t_y, \theta)$. The point derivatives are immediate:

$$\frac{\partial \mathbf{d}}{\partial t_x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \frac{\partial \mathbf{d}}{\partial t_y} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \frac{\partial \mathbf{d}}{\partial \theta} = \mathbf{J} \mathbf{R} \mathbf{p} = \begin{pmatrix} -r_y \\ r_x \end{pmatrix},$$

with $\mathbf{r} = \mathbf{R} \mathbf{p}$. Differentiating s_k once,

$$\frac{\partial s_k}{\partial \xi_i} = -s_k \mathbf{d}^\top \mathbf{C} \frac{\partial \mathbf{d}}{\partial \xi_i},$$

so minimizing $f = -s$ has gradient $g_i = \sum_k s_k \mathbf{d}^\top \mathbf{C} \partial_i \mathbf{d}$. The exact Hessian carries three terms; keeping only the positive semi-definite one,

$$H_{ij} \approx \sum_k s_k (\partial_i \mathbf{d})^\top \mathbf{C} (\partial_j \mathbf{d}),$$

is the same Gauss–Newton bargain Chapter 15 strikes: give up quadratic convergence, keep a descent direction unconditionally. It matters here because the exact Hessian is indefinite away from the optimum — and, in a corridor, at it.

Two implementation notes that are not optional. First, a cell straddling a flat wall has a covariance whose smaller eigenvalue is the range noise squared, often numerically zero, and Σ^{-1} then claims infinite confidence across the wall; clamp $\lambda_{\min} \geq 0.01 \lambda_{\max}$. Second, snapping each point to its containing cell puts a discontinuity on every cell boundary, undoing

the smoothness the method was chosen for; summing over the 3×3 neighbourhood (Biber and Straßer use four overlapping grids) removes it.

16.3.7 The shape of the cost

INTERACTIVE · w16.3

Score Terrain

The registration cost is not a bowl. ICP's is piecewise with plateaus; NDT's is smooth; and in a corridor both are flat along one whole direction.

Run this simulation in the web edition: [/chapters/ch16-scan-matching #w16.3](/chapters/ch16-scan-matching/#w16.3)

That widget is the argument of the last three sections, drawn. Two things are worth reading off it numerically, because they are the two reasons a scan matcher fails.

The terraces are real. ICP's surface is built of flat plateaus separated by ridges. A plateau is a region where no correspondence changes, so the cost is a fixed quadratic; the ridge is where the assignment flips. A gradient method dropped onto a plateau far from the answer has nothing to descend. This is why ICP is not run as a gradient method: the closed-form alignment jumps across plateaus in one step.

Degeneracy is a property of the room, not the algorithm. In room A, translating the sweep 0.6 m along x costs 0.189 m^2 of mean squared residual, and 0.6 m along y costs 0.160 m^2 — the same, to within the noise. In the corridor the same two numbers are **0.053 m^2 and 0.347 m^2** , a factor of 6.5. NDT's smoother surface does not help: its score at $\Delta x = 0.6 \text{ m}$ is still 74% of the peak, while at $\Delta y = 0.6 \text{ m}$ it has collapsed to zero. Neither objective can recover information the geometry does not contain. What a good implementation can do is *notice* — which is what the information matrix in the next section is for.

16.3.8 The pose-graph residual on SE(2)

A relative-pose measurement Z_{ij} between poses T_i and T_j is a factor whose residual lives in the tangent space of the group:

$$\mathbf{e}_{ij} = \log(Z_{ij}^{-1} T_i^{-1} T_j)^\vee \in \mathbb{R}^3, \quad \text{cost} = \sum_{(i,j)} \rho(\mathbf{e}_{ij}^\top \Omega_{ij} \mathbf{e}_{ij}).$$

Equivalently, in the book's manifold operators, $\mathbf{e}_{ij} = T_j \boxminus (T_i \circ Z_{ij})$: "where j actually is, minus where the measurement says it should be", measured in the tangent space at the predicted pose. A **pose graph** is a factor graph containing

only pose variables — the landmarks of Chapter 14 have been marginalized into the relative measurements and never appear.

DERIVATION Jacobians of the pose-graph residual

$$\partial \mathbf{e} / \partial \delta_i = -\text{Ad}_{Z^{-1}}, \quad \partial \mathbf{e} / \partial \delta_j = \mathbf{I}$$

Perturb on the right, $T \leftarrow T \boxplus \delta = T \exp(\delta)$, as Chapter 3 does everywhere.
With respect to T_i .

$$\mathbf{e}(\delta_i) = \log((T_i \exp(\delta_i) Z)^{-1} T_j) = \log(Z^{-1} \exp(-\delta_i) T_i^{-1} T_j).$$

Slide $\exp(-\delta_i)$ past Z^{-1} using the defining property of the adjoint, $Z^{-1} \exp(\tau) Z = \exp(\text{Ad}_{Z^{-1}} \tau)$:

$$\mathbf{e}(\delta_i) = \log(\exp(-\text{Ad}_{Z^{-1}} \delta_i) \exp(\mathbf{e})) \approx \mathbf{e} - \mathbf{J}_l^{-1}(\mathbf{e}) \text{Ad}_{Z^{-1}} \delta_i.$$

With respect to T_j . The perturbation lands on the right of the error directly:

$$\mathbf{e}(\delta_j) = \log(\exp(\mathbf{e}) \exp(\delta_j)) \approx \mathbf{e} + \mathbf{J}_r^{-1}(\mathbf{e}) \delta_j.$$

The approximation everybody makes. $\mathbf{J}_r^{-1}(\mathbf{e}) = \mathbf{I} + O(\|\mathbf{e}\|)$, and both this implementation and g2o's SE(2) drop it, taking

$$\mathbf{A}_i = -\text{Ad}_{Z_{ij}^{-1}}, \quad \mathbf{A}_j = \mathbf{I}.$$

This is legitimate for a reason worth internalizing: Gauss–Newton needs the Jacobian only to choose a *direction*, while the fixed point is determined by the **residual**, which is computed exactly. A slightly wrong Jacobian costs iterations, not correctness. (Get the residual wrong and you converge, confidently, to the wrong trajectory.) The exact SE(2) right-Jacobian is in Appendix C.

With translation-first tangent ordering $\tau = (v_x, v_y, \omega)$, the adjoint of $T = (\mathbf{R}(\theta), \mathbf{t})$ is

$$\text{Ad}_T = \begin{pmatrix} \cos \theta & -\sin \theta & t_y \\ \sin \theta & \cos \theta & -t_x \\ 0 & 0 & 1 \end{pmatrix},$$

so both Jacobians are three lines of code. Assemble $\mathbf{H} = \sum \mathbf{A}^\top \mathbf{\Omega} \mathbf{A}$ and $\mathbf{b} = \sum \mathbf{A}^\top \mathbf{\Omega} \mathbf{e}$, solve $\mathbf{H} \delta = -\mathbf{b}$, retract with $\boxplus$, repeat. That is Chapter 15's optimizer with no modifications whatever.

⚠ WHERE ROBOTS BREAK THIS

The gauge. $\mathbf{H}$ is singular by construction: rigidly transforming *every* pose leaves every relative measurement unchanged, so the cost has a three-dimensional flat direction. Fix one node — zero its rows and columns and put a 1 on the diagonal — and the flat direction disappears. Adding a huge prior instead works too, and quietly costs you conditioning.

16.3.9 A worked example you can check by hand

Two of them, one per half of the chapter.

Closed-form alignment. Take four source points on the unit circle, $\mathcal{P} = \{(1, 0), (0, 1), (-1, 0), (0, -1)\}$, with centroid at the origin. Apply the rotation with $\cos \theta = 0.8$, $\sin \theta = 0.6$ (that is $\theta = 36.8699^\circ$) and the translation $(2, 1)$:

$$\mathcal{Q} = \{(2.8, 1.6), (1.4, 1.8), (1.2, 0.4), (2.6, 0.2)\}, \quad \bar{\mathbf{q}} = (2, 1).$$

Centering $\mathcal{Q}$ gives $(0.8, 0.6), (-0.6, 0.8), (-0.8, -0.6), (0.6, -0.8)$. Now the two sums from Step 5:

$$\sum_k \mathbf{a}_k \cdot \mathbf{b}_k = 0.8 + 0.8 + 0.8 + 0.8 = 3.2, \quad \sum_k \mathbf{a}_k \times \mathbf{b}_k = 0.6 \times 4 = 2.4.$$

Hence $\theta^* = \text{atan2}(2.4, 3.2) = 36.8699^\circ$ — note $3.2/4 = 0.8$ and $2.4/4 = 0.6$, so the recovered cosine and sine are exact — and $\mathbf{t}^* = \bar{\mathbf{q}} - \mathbf{R}^* \bar{\mathbf{p}} = (2, 1)$. The trace bound of Step 3 is $s_1 + s_2 = 4$, attained.

A loop closure, distributed. Three poses on a line. Node 0 is fixed at the origin. Odometry says each step is exactly 1 m forward: $Z_{01} = Z_{12} = (1, 0, 0)$. A loop factor then claims $Z_{02} = (2.6, 0, 0)$. All three information matrices are $\mathbf{I}$. Initialized from odometry, $T_1 = 1.0$ and $T_2 = 2.0$, only the loop factor has a residual, -0.6 , so $\chi^2 = 0.36$.

The optimum minimizes $(t_1 - 1)^2 + (t_2 - t_1 - 1)^2 + (t_2 - 2.6)^2$. Setting both partials to zero gives $t_1 = 1.2$, $t_2 = 2.4$, at which **all three residuals equal 0.2** and $\chi^2 = 3 \times 0.04 = 0.12$. The 0.6 m of loop error did not land on the last pose; it was spread evenly over the three edges of the cycle, one fifth of a metre each. Every pose in the graph moved except the one that was pinned. That is loop closure in miniature, and it is why the purple trajectory in [w16.2](#) shifts along its whole length rather than at its tip.

16.4 The algorithms

ALGORITHM `icp(P, map, T0,  $\tau$ , variant)` COST $O(I \cdot N)$ with a voxel hash; $O(I \cdot N \log M)$ with a k-d tree

in source point cloud P , voxel-hashed target map, initial guess T_0 , rejection radius τ

out $\hat{T}$, rmse, inlier count, the full pose trace

```

1:  $T \leftarrow T_0$ 
2: for  $i = 1$  to  $I_{\max}$  do
3:    $C \leftarrow \emptyset$ 
4:   for all  $\mathbf{p}_k \in \mathcal{P}$  do
5:      $\mathbf{q} \leftarrow$  nearest map point to  $T\mathbf{p}_k$  within  $\tau$ 
6:     if  $\mathbf{q}$  exists then  $C \leftarrow C \cup \{(T\mathbf{p}_k, \mathbf{q}, \mathbf{n}_q)\}$ 
7:   endfor
8:   if  $|C| < c_{\min}$  then break
9:    $\Delta T \leftarrow \text{svd\_align}(C)$  or point_to_plane_step(C)
10:   $T \leftarrow \Delta T \circ T$     the increment lives in the target frame
11:  if  $\|\log \Delta T\| < \varepsilon$  then break
12: endfor
13: return  $T$ ,  $\text{rmse}(C)$ ,  $|C|$ 
```

Line 5 is the only line whose cost depends on the map size, and the only one worth engineering. A voxel hash answers it in expected constant time by examining the $\lceil \tau / \text{cell} \rceil$ -ring of cells around the query — and unlike a k-d tree it does not need rebuilding when the map grows, which for a map that grows every frame is the trade that matters.

Line 9’s choice of τ is where KISS-ICP (Vizzo et al., 2023) earns its name. Rather than tuning τ per dataset, track how wrong the constant-velocity prediction has been and set $\tau_t = 3\sigma_t$ from the running deviation $\delta = \|\Delta \mathbf{t}\| + 2r_{\max} \sin(\Delta\theta/2)$ — the worst displacement the missed rotation could have caused at the far end of the sweep. The demo’s front end does exactly this.

ALGORITHM `verify_loop(Pt, submapj, T0, gap)` COST one ICP run, $O(I \cdot N)$

in the current keyframe’s cloud, a submap around candidate j , the graph’s current relative pose, and how many edges separate them

out `Some((Ztj,  $\Omega_{tj}$ ))` if the match is believable, else `None`

```

1:  $\hat{T} \leftarrow \text{icp}(\mathcal{P}_t, \text{submap}_j, T_0, \tau_{\text{loop}})$ 
```

- 2: if $\text{rmse}(\hat{T}) > \rho_{\max}$ or $\text{inliers}(\hat{T}) < c_{\min}$ then return None geometric fitness
- 3: $\Omega \leftarrow \text{icp_information}(\hat{T}), \text{ridged and capped}$
- 4: $\nu \leftarrow \log(T_0^{-1}\hat{T})^\vee$ innovation: how far the match moved the graph's belief
- 5: $\Omega_{\text{gate}} \leftarrow \text{compound one edge's covariance over gap edges, invert}$
- 6: if $\nu^\top \Omega_{\text{gate}} \nu > \chi_{3,0.95}^2 = 7.815$ then return None
- 7: return Some($\hat{T}, \Omega$)

Line 2 asks a geometric question — did the match converge to something tight and well supported? Line 6 asks a statistical one — is the answer consistent with what the graph already believed, given how uncertain it had a right to be? Both are needed, and they fail differently. A picket fence passes line 2 with a beautiful residual and fails line 6 by landing a whole spacing away. A genuine loop after a long unobserved drive fails line 2 if τ_{loop} is too small, and sails through line 6 because the gate has grown so wide it accepts anything.

That growing gate is the point of line 5. The exact quantity is the graph's marginal covariance over the relative pose $T_i^{-1}T_j$, which Chapter 15 recovers by sparse back-substitution. Compounding a single edge's covariance along the path is the cheap approximation, and it preserves the property that matters: a candidate forty nodes back must clear a much wider bar than one four nodes back, because forty nodes of odometry could have put the robot anywhere.

ALGORITHM `pose_graph_slam(scan stream)` COST front end 0(I·N) per sweep; back end 0(|E| · fill) per optimization via sparse Cholesky
in a stream of LiDAR sweeps and odometry
out a trajectory and a map, both revised whenever a loop is verified

- 1: $G \leftarrow$ graph with one **fixed** node at the origin
- 2: loop
- 3: $\tilde{T}_t \leftarrow T_{t-1} \circ u_t$ odometry prediction: the initial guess, nothing more
- 4: $T_t \leftarrow \text{icp}(\mathcal{P}_t, \text{local map}, \tilde{T}_t, \tau_t)$
- 5: insert $T_t \mathcal{P}_t$ into the local map; evict the oldest sweep
- 6: if $\|T_{\text{kf}}^{-1}T_t\|$ exceeds the keyframe threshold then
- 7: add node j ; add odometry factor $(T_{\text{kf}}^{-1}T_t, \Omega_{\text{icp}})$
- 8: $\mathcal{K} \leftarrow \text{detect_loop_candidates}(G, j)$
- 9: for the best $k \in \mathcal{K}$: if `verify_loop` returns Some then
- 10: add loop factor; `optimize(G)` Chapter 15, unchanged
- 11: push the correction into the front end and rebuild the map

```

12:   endif
13: endloop

```

Line 11 is the one people forget. The front end lives in the world frame; the back end has just moved the world frame under it. Fail to tell it and the next sweep is registered against a local map that no longer agrees with the graph, and the system tears itself apart over the following second.

i NOTE

The ancestor. Thrun, Burgard and Fox's 1999–2000 draft already contains this chapter in embryo. Its `incremental_ML_mapping` (Table 14.1) hill-climbs $p(z_t | s_t)p(s_t | u_t, \hat{s}_{t-1})$ in pose space — scan-to-map matching with a likelihood-field cost and a motion prior, which is line 4 above with gradient ascent in place of ICP. Its `incremental_ML_mapping_for_cycles` (Table 14.5) detects cycles with a second, posterior estimator over poses and then "corrects poses backwards in time" — which is line 10 without the graph, and without the sparsity that makes it tractable. The draft names the two limitations of the basic method honestly: *"1. It is unable to cope with large odometry error. 2. It is unable to correct poses backwards in time."* Twenty-five years later the answer to both is one sparse linear solve.

16.5 Implementation in Rust

The crate is `ch16_slam2d`, and it reuses rather than reinvents: SE2 comes from Chapter 3, the simulated LiDAR and the Apartment from Chapter 4, the occupancy grid from Chapter 13, and the optimizer from Chapter 15.

16.5.1 The local map

RUST `crates/ch16_slam2d/src/cloud.rs`

```

1 use nalgebra::{Point2, Vector2};
2 use rustc_hash::FxHashMap;
3 use smallvec::SmallVec;
4
5 /// A sweep projected into sensor-frame points.
6 ///
7 /// Max-range returns are *dropped*, not clamped. A beam that reports its
8 /// maximum hit nothing; keeping it plants a phantom point on the horizon, and
9 /// ICP will cheerfully match a phantom to a real wall. This is the single most
10 /// common bug in a first scan matcher.
11 pub struct PointCloud {
12     pub points: Vec<Point2<f64>>,
13     pub stamp: f64,
14 }

```

```

15
16 /// KISS-ICP's local map: a voxel hash with a bounded population per cell.
17 ///
18 /// `max_per_cell` bounds memory *and* filters: a cell that has met its quota
19 /// ignores further evidence, so a passing pedestrian cannot stuff the map with
20 /// a wall that is not there.
21 pub struct VoxelMap {
22     cell: f64,
23     max_per_cell: usize,
24     cells: FxHashMap<(i32, i32), SmallVec<[u32; 4]>>,
25     pts: Vec<Point2<f64>>,
26     normals: Vec<Option<Vector2<f64>>>,
27 }
28
29 impl VoxelMap {
30     #[inline]
31     fn key(&self, p: &Point2<f64>) -> (i32, i32) {
32         ((p.x / self.cell).floor() as i32, (p.y / self.cell).floor() as i32)
33     }
34
35     /// Expected O(1): only the ?tau/cell?-ring of cells can hold the answer, and
36     /// each cell holds at most `max_per_cell` points. A k-d tree answers the
37     /// same query in O(log M) but must be rebuilt as the map grows -- which,
38     /// for a map that grows every frame, is the wrong trade.
39     pub fn nearest(&self, p: &Point2<f64>, tau: f64) -> Option<u32> {
40         let ring = (tau / self.cell).ceil().max(1.0) as i32;
41         let (ci, cj) = self.key(p);
42         let mut best: Option<u32> = None;
43         let mut best_d2 = tau * tau;
44         for di in -ring..=ring {
45             for dj in -ring..=ring {
46                 let Some(bucket) = self.cells.get(&(ci + di, cj + dj)) else { continue };
47                 for &idx in bucket {
48                     let d2 = (self.pts[idx as usize] - p).norm_squared();
49                     if d2 < best_d2 {
50                         best_d2 = d2;
51                         best = Some(idx);
52                     }
53                 }
54             }
55         }
56         best
57     }
58
59     pub fn insert(&mut self, pts: &[Point2<f64>], normals: &[Option<Vector2<f64>>]) {
60         for (k, p) in pts.iter().enumerate() {
61             let bucket = self.cells.entry(self.key(p)).or_default();
62             if bucket.len() >= self.max_per_cell {
63                 continue;
64             }
65             bucket.push(self.pts.len() as u32);
66             self.pts.push(*p);
67             self.normals.push(normals.get(k).copied().flatten());
68         }
69     }
70 }

```

16.5.2 The matcher

RUST crates/ch16_slam2d/src/icp.rs

```

1 use nalgebra::{Matrix3, Point2, Rotation2, Vector2, Vector3};
2 use pr_geom::SE2; // Chapter 3
3
4 #[derive(Clone, Copy)]
5 pub enum IcpVariant {
6     PointToPoint,
7     PointToPlane,
8 }
9
10 pub struct IcpResult {
11     pub pose: SE2,
12     pub rmse: f64,
13     pub inliers: usize,
14     /// Pose after every iteration. The interesting part of ICP is never the
15     /// answer; it is the path it took to get there -- and w16.1 draws it.
16     pub trace: Vec<SE2>,
17 }
18
19 /// `svd_align` -- Arun et al. (1987), specialized to the plane.
20 ///
21 ///  $\text{tr}(R^T W) = \cos \theta (W_{1,1} + W_{2,2}) + \sin \theta (W_{2,1} - W_{1,2})$  is one sinusoid, so
22   the
23   /// SVD collapses to a single `atan2` over a dot product and a cross product.
24   /// Because `atan2` returns an *angle*, the det-correction that keeps the 3-D
25   /// version from returning a reflection is free here.
26 pub fn svd_align(src: &[Point2<f64>], dst: &[Point2<f64>]) -> SE2 {
27     let n = src.len().min(dst.len());
28     assert!(n >= 2, "a rigid alignment needs at least two correspondences");
29     let p_bar = centroid(&src[..n]);
30     let q_bar = centroid(&dst[..n]);
31     let (mut s_dot, mut s_cross) = (0.0, 0.0);
32     for k in 0..n {
33         let (a, b) = (src[k] - p_bar, dst[k] - q_bar);
34         s_dot += a.dot(&b);
35         s_cross += a.x * b.y - a.y * b.x;
36     }
37     let theta = s_cross.atan2(s_dot);
38     SE2::new(q_bar.coords - Rotation2::new(theta) * p_bar.coords, theta)
39 }
40
41 /// One Gauss-Newton step of the point-to-plane cost.
42 ///
43 /// Rows are  $a^T = (p \times n, n?, n_y)$  with  $p$  measured from the cloud centroid, so
44   the linearized rotation happens about the centroid rather than the world
45   origin -- the difference between a well-conditioned  $3 \times 3$  and a hopeless one.
46 pub fn point_to_plane_step(pairs: &[Correspondence]) -> SE2 {
47     let c = centroid_of_sources(pairs);
48     let mut h = Matrix3::zeros();
49     let mut g = Vector3::zeros();
50     for pr in pairs {
51         let Some(n) = pr.normal else { continue };
52         let r = pr.src - c;
53         let a = Vector3::new(r.x * n.y - r.y * n.x, n.x, n.y);
54         let b = -n.dot(&(pr.src - pr.dst));
55         h += a * a.transpose();
56         g += a * b;
57     }
58     let theta = g[0].atan2(g[1]);
59     SE2::new(c.coords - Rotation2::new(theta) * c.coords, theta)
60 }

```

```

56     }
57     // Damping *relative to the trace*, not an absolute 1e-9 ridge. Along a
58     // corridor both H and g vanish in the along-wall direction; an absolute
59     // ridge divides the numerical dust in one by the dust in the other and
60     // slides the scan a metre down the hallway. This resolves the
61     // unconstrained direction to zero motion: "the scan says nothing here,
62     // keep the prediction".
63     let ridge = 1e-4 * h.trace() / 3.0 + 1e-12;
64     h += Matrix3::identity() * ridge;
65
66     let x = h.lu().solve(&g).expect("3x3 normal equations are damped, so never singular")
67 ;
68     let (theta, t) = (x[0], Vector2::new(x[1], x[2]));
69     let rot = Rotation2::new(theta);
70     SE2::new(c.coords - rot * c.coords + t, theta)
71 }
72
73 pub fn icp(src: &PointCloud, map: &VoxelMap, init: SE2, cfg: &IcpConfig) -> IcpResult {
74     let mut pose = init;
75     let mut trace = vec![pose];
76     let mut pairs = associate(src, map, pose, cfg.tau);
77
78     for _ in 0..cfg.max_iters {
79         if pairs.len() < cfg.min_pairs {
80             break;
81         }
82         let delta = match cfg.variant {
83             IcpVariant::PointToPoint => svd_align(&sources(&pairs), &targets(&pairs)),
84             IcpVariant::PointToPlane => point_to_plane_step(&pairs),
85         };
86         // The increment is expressed in the *target* frame, so it composes on
87         // the left. Composing on the right is a bug that still converges, just
88         // to the wrong place, which is the worst kind.
89         pose = delta * pose;
90         pairs = associate(src, map, pose, cfg.tau);
91         trace.push(pose);
92         if delta.log().norm() < cfg.tolerance {
93             break;
94         }
95     }
96     IcpResult { rmse: rmse(&pairs, cfg.variant), inliers: pairs.len(), pose, trace }
97 }

```

16.5.3 The back end

RUST crates/ch16_slam2d/src/graph.rs

```

1 use faer::sparse::{SparseColMat, linalg::solvers::Llt};
2 use nalgebra::{Matrix3, Vector3};
3 use petgraph::graph::{NodeIndex, UnGraph};
4 use pr_geom::SE2;
5
6 pub struct PoseEdge {
7     pub z: SE2,
8     pub omega: Matrix3<f64>,
9     pub kind: EdgeKind,
10 }

```

```

11
12 pub struct PoseGraph {
13     /// petgraph owns the topology; the poses are the node weights. Candidate
14     /// search and connectivity queries then come for free.
15     graph: UnGraph<SE2, PoseEdge>,
16     fixed: NodeIndex,
17 }
18
19 impl PoseGraph {
20     /// e_ij = log(Z-1 T?-1 T?)^? -- computed exactly, always.
21     pub fn residual(&self, i: NodeIndex, j: NodeIndex, e: &PoseEdge) -> Vector3<f64> {
22         let (ti, tj) = (self.graph[i], self.graph[j]);
23         ((ti * e.z).inverse() * tj).log()
24     }
25
26     /// One Gauss-Newton iteration. Returns the largest per-node correction,
27     /// which is the number the dashboard reports as "history moved by".
28     pub fn optimize_once(&mut self, huber: f64) -> f64 {
29         let n = self.graph.node_count();
30         let mut triplets = Vec::with_capacity(36 * self.graph.edge_count());
31         let mut b = vec![0.0; 3 * n];
32
33         for edge in self.graph.edge_references() {
34             let (i, j) = (edge.source(), edge.target());
35             let e = self.residual(i, j, edge.weight());
36             // ?e/?delta? = -Ad_{Z-1}, ?e/?delta? = I. Both to first order in ||e||:
37             // the right-Jacobian factor is I + O(||e||) and Gauss-Newton only
38             // needs a direction. The residual above is exact, and the residual
39             // is what fixes the answer.
40             let a_i = -edge.weight().z.inverse().adjoint();
41             let a_j = Matrix3::identity();
42             let omega = robust_weight(&e, &edge.weight().omega, huber);
43
44             for (node, a) in [(i, a_i), (j, a_j)] {
45                 let at_omega = a.transpose() * omega;
46                 accumulate(&mut b, node, &(at_omega * e));
47                 for (other, a_other) in [(i, a_i), (j, a_j)] {
48                     push_block(&mut triplets, node, other, &(at_omega * a_other));
49                 }
50             }
51         }
52
53         // Gauge: the cost is invariant under a rigid transform of the whole
54         // trajectory, so H is singular by construction. Pin one node.
55         clamp_fixed(&mut triplets, &mut b, self.fixed);
56
57         let h = SparseColMat::try_new_from_triplets(3 * n, 3 * n, &triplets).unwrap();
58         let delta = Llt::new(h.as_ref(), faer::Side::Lower)
59             .expect("H is PSD once the gauge is fixed and Omega are PSD")
60             .solve(&nalgebra_to_faer(&b).neg());
61
62         let mut max_step = 0.0_f64;
63         for (k, node) in self.graph.node_indices().enumerate() {
64             if node == self.fixed {
65                 continue;
66             }
67             let d = Vector3::new(delta[3 * k], delta[3 * k + 1], delta[3 * k + 2]);
68             max_step = max_step.max(d.norm());
69             self.graph[node] = self.graph[node].boxplus(&d); // [+], never +
70         }
71         max_step

```

```

72     }
73 }

```

The one line worth staring at is `a_i = -edge.weight().z.inverse().adjoint()`. That is the whole manifold apparatus of Chapter 3 earning its keep: the derivative of a group-valued residual with respect to a group-valued variable, in three characters of algebra and one function call.

16.5.4 Information, honestly

RUST `crates/ch16_slam2d/src/icp.rs (continued)`

```

1  /// The information matrix ICP *would* report for its own answer:  $\Omega = J^T J / \sigma^2$ .
2  ///
3  /// This is the Gauss-Newton Hessian at the solution -- the curvature of the
4  /// cost, which is exactly what an information matrix is. Take it at face value
5  /// and the pose graph will believe a corridor match far more than it should,
6  /// because the derivation assumes every beam is an independent observation and
7  /// consecutive LiDAR beams are anything but. `inflate` divides the effective
8  /// sample size by a constant; it is the same "inflate and admit it" fudge
9  /// Chapter 10 applies to the beam model, and it is not more principled here.
10 pub fn icp_information(pairs: &[Correspondence], sigma: f64, inflate: f64) -> Matrix3<f64>
11 > {
12     let c = centroid_of_sources(pairs);
13     let w = 1.0 / (sigma * sigma * inflate);
14     let mut omega = Matrix3::zeros();
15     for pr in pairs {
16         let r = pr.src - c;
17         for n in pr.constrained_directions() {
18             let a = Vector3::new(n.x, n.y, r.x * n.y - r.y * n.x);
19             omega += w * a * a.transpose();
20         }
21     }
22     omega
23 }
24
25 /// Keep Omega usable *without* breaking it.
26 ///
27 /// The fix for a rank-deficient Omega is a ridge,  $\Omega + \lambda I$ , not a clamp on the
28 /// diagonal: clamping entries one at a time can leave a matrix that is no
29 /// longer positive semi-definite, and a pose graph fed an indefinite Omega does
30 /// not converge slowly -- it explodes. Ask the demo: before this was a ridge,
31 /// one corridor loop closure sent the trajectory 200 m into the car park.
32 pub fn regularize(mut omega: Matrix3<f64>, ridge: f64, cap: f64) -> Matrix3<f64> {
33     omega += Matrix3::identity() * ridge;
34     let peak = omega.diagonal().max();
35     if peak > cap {
36         omega *= cap / peak;
37     }
38     omega
39 }

```


16.5.5 The tests that pin the worked examples

RUST `crates/ch16_slam2d/tests/worked_examples.rs`

```

1 use approx::assert_relative_eq;
2 use nalgebra::Point2;
3
4 #[test]
5 fn svd_align_recovers_a_known_rigid_motion() {
6     // Four points on the unit circle; cos theta = 0.8, sin theta = 0.6, t = (2, 1).
7     let src = [Point2::new(1.0, 0.0), Point2::new(0.0, 1.0),
8               Point2::new(-1.0, 0.0), Point2::new(0.0, -1.0)];
9     let dst = [Point2::new(2.8, 1.6), Point2::new(1.4, 1.8),
10              Point2::new(1.2, 0.4), Point2::new(2.6, 0.2)];
11
12     let t = svd_align(&src, &dst);
13
14     assert_relative_eq!(t.theta().cos(), 0.8, epsilon = 1e-12);
15     assert_relative_eq!(t.theta().sin(), 0.6, epsilon = 1e-12);
16     assert_relative_eq!(t.translation().x, 2.0, epsilon = 1e-12);
17     assert_relative_eq!(t.translation().y, 1.0, epsilon = 1e-12);
18 }
19
20 #[test]
21 fn a_loop_error_spreads_evenly_around_the_cycle() {
22     // Three collinear poses, unit information everywhere. Odometry says 1 m
23     // per step; the loop factor says the two-step displacement is 2.6 m.
24     let mut g = PoseGraph::new();
25     let n0 = g.add_fixed_node(SE2::identity());
26     let n1 = g.add_node(SE2::translation(1.0, 0.0));
27     let n2 = g.add_node(SE2::translation(2.0, 0.0));
28     g.add_edge(n0, n1, SE2::translation(1.0, 0.0), Matrix3::identity(), EdgeKind::
Odometry);
29     g.add_edge(n1, n2, SE2::translation(1.0, 0.0), Matrix3::identity(), EdgeKind::
Odometry);
30     g.add_edge(n0, n2, SE2::translation(2.6, 0.0), Matrix3::identity(), EdgeKind::Loop);
31
32     assert_relative_eq!(g.chi2(), 0.36, epsilon = 1e-12); // (-0.6)^2
33
34     g.optimize(20, 1e-12);
35
36     // 0.6 m of loop error, three edges, 0.2 m each. The *fixed* node did not
37     // move; everything else did.
38     assert_relative_eq!(g.pose(n1).translation().x, 1.2, epsilon = 1e-9);
39     assert_relative_eq!(g.pose(n2).translation().x, 2.4, epsilon = 1e-9);
40     assert_relative_eq!(g.chi2(), 0.12, epsilon = 1e-9);
41 }

```

i NOTE

Both tests pass in the TypeScript port too — `lib/slam/icp.ts` and `lib/slam/posegraph.ts` are line-for-line translations of the Rust above, and they are what every widget on this page runs. The alignment example returns (2.0000, 1.0000, 36.8699°) and the graph example returns

$\chi^2 : 0.36 \rightarrow 0.12$ with poses 0, 1.2, 2.4 in both implementations.

16.6 Putting it together: RustSLAM-2D

Front end, back end, gate, map. One lap of the Apartment’s corridor, east and back.

 INTERACTIVE · w16.4

RustSLAM-2D Control Room

SLAM is not one algorithm. It is a fast greedy front end and a slow honest back end taking turns, and you can watch the hand-off.

Run this simulation in the web edition: [/chapters/ch16-scan-matching #w16.4](#)

16.6.1 What the system actually buys

Here are the numbers for the canonical run — seed 0xC0FFEE, 129 ticks, 41 keyframes, a 120-beam sweep at 6 m with 5 cm range noise:

Quantity	Odometry only	Front end only	Front end + 3 loop factors
Position error at the end of the lap	2.41 m	0.73 m	0.24 m
RMS position error over the lap	1.18 m	0.64 m	0.59 m
Return-leg keyframe error	—	0.54 – 0.75 m	0.17 – 0.34 m
Map self-disagreement	—	0.238 m	0.082 m

(The odometry row is averaged over every tick; the graph rows over keyframes, which is what the graph actually holds. The difference is immaterial at this sample size.)

The first column is the reason scan matching exists: two and a half metres of error over eighteen metres of driving, from wheels that were not even badly calibrated. The second column is the front end doing its job — a threefold reduction, bought with one nearest-neighbour query per beam and no new sensors.

The third column is where it gets interesting, and where most treatments of loop closure stop being honest.

16.6.2 What loop closure fixed, and what it could not

Three candidates were proposed and all three passed the gate, with χ^2 of 4.24, 0.42 and 1.13 against a threshold of 7.815. The largest single correction moved a

past pose by 0.24 m. And the RMS error over the whole lap barely moved: 0.64 m to 0.59 m, an 8% cut.

Look at the other rows and the picture resolves. The **return-leg error fell by a factor of three**, and the **map's disagreement with itself fell by a factor of three**. The doubled walls collapsed. What did *not* improve is the error at the far end of the corridor, around the turnaround: 0.85 m before the closures and 0.89 m after.

That is not a bug, and it is not tuning. The Apartment's free space is a **tree**: every room is a dead end off one corridor, so there is no cycle to close. What Rusty gets is a *revisit* — the return leg passes places the outbound leg saw. A revisit ties the two legs to each other and, via the fixed node, to the origin. It says nothing whatsoever about the turnaround, which was seen once, from one direction, and never re-measured. Error at a place you visit exactly once is **unobservable**, and no optimizer invents information.

💡 WHAT A LOOP CLOSURE IS, PRECISELY

A loop closure is a measurement of the *relative* pose between two moments in time. It makes the trajectory self-consistent and the map single-layered. It reduces absolute error only where the constraint graph actually reaches — which is why exploration planners (Chapter 24) deliberately route the robot back through unvisited junctions, and why every SLAM benchmark drives real cycles.

This is also why the map-disagreement number is the one to watch on a real robot. ATE needs ground truth, which a deployed system never has. Map self-disagreement needs nothing but the map, and it is the quantity that actually determines whether the navigation stack downstream can plan through a doorway.

16.6.3 The production recipe, stated honestly

RustSLAM-2D is the architecture of every deployed 2D system, with the corners rounded off:

- **Cartographer** (Hess et al., 2016) replaces the sliding local map with explicit *submaps* and the radius search with a branch-and-bound multi-resolution correlative matcher that is *exact* — it returns the global optimum of the score within a search window, which removes the basin problem of `<a href="#w16.1">w16.1</a>` at the cost of a much larger constant. We teach the covariance-gated candidate search instead because it fits on a page; if you deploy, read their §IV.
- **SLAM Toolbox** (Macenski and Jambrecic, 2021) is the ROS 2 default and adds the operational parts this chapter ignores entirely: serialization, lifelong map updates, and localization against a map built in an earlier session.

- **Place recognition at scale** replaces the radius search with a descriptor — Scan Context, DBoW — because a radius search over past poses depends on a pose estimate that, by the time you need a loop closure, is exactly the thing that is wrong. Exercise 6 builds the smallest possible version of this.
- **3-D and inertial.** LOAM's feature-based lineage (LIO-SAM, FAST-LIO2) dominates 3-D LiDAR odometry; FAST-LIO2's iterated error-state Kalman filter is filtering's revenge, and connects straight back to Chapter 7. The 2024 LiDAR odometry survey is the map of that territory. We follow KISS-ICP instead — small, nearly parameter-free, and implementable in a weekend — precisely because it shows that the classic objective, done carefully, is still competitive.

16.7 Exercises

1 FOUNDATION •• Finish the Procrustes bound

Complete Step 3 of the alignment derivation: prove $\text{tr}(\mathbf{R}^T \mathbf{W}) \leq \sum_i s_i$ for every rotation $\mathbf{R}$, with equality at $\mathbf{R} = \mathbf{U}\mathbf{V}^T$. (Hint: show every diagonal entry of an orthogonal matrix has absolute value at most 1, using the fact that its rows are unit vectors.) Then construct a two-point planar correspondence set for which $\mathbf{U}\mathbf{V}^T$ has determinant -1 , and say what the reflection it returns would do to a map.

2 FOUNDATION •• The corridor's null vector

Derive the point-to-plane rows $\mathbf{a}_k$ and scalars b_k from the derivation, then take an infinite corridor: all target points lie on $y = 0$ or $y = w$, so every normal is $\pm(0, 1)$. Show that $\mathbf{H} = \sum_k \mathbf{a}_k \mathbf{a}_k^T$ has rank 2 and compute its null vector explicitly. Interpret it physically. Finally, explain why adding the odometry prior as one extra row $\mathbf{a}_0 = (0, 1, 0)$ with weight $1/\sigma_v^2$ restores rank 3, and what that says about the relationship between this section and Chapter 9.

3 FOUNDATION ••• Why the residual must be exact

The pose-graph implementation drops the right-Jacobian factor $\mathbf{J}_r^{-1}(\mathbf{e})$ but computes $\mathbf{e}$ exactly. Show that at any fixed point of the iteration, $\sum_{(i,j)} \mathbf{A}^T \mathbf{\Omega} \mathbf{e}_{ij} = \mathbf{0}$ holds regardless of which invertible approximation of $\mathbf{J}_r^{-1}$ was used, so the *converged* solution is unaffected. Then construct a residual formula that is wrong by a first-order term and show that its fixed point is a different trajectory. This is the formal version of "a wrong Jacobian costs iterations; a wrong residual costs correctness".

4 CONCEPTUAL •• Find the basin edge

In [w16.1](#), predict the largest pure-rotation initial offset from which point-to-point ICP still converges to the true alignment on the room scene. Then measure it by sweeping the heading slider. Repeat for point-to-plane. The two thresholds differ; explain the difference using the terraces in [w16.3](#), and predict what happens to both thresholds when you double τ .

5 CONCEPTUAL •• Gate the loop before you watch it

In [w16.2](#), set the odometry-noise multiplier to its maximum and predict, before pressing play, (a) whether the front end still tracks, (b) whether the χ^2 gate will accept the first loop candidate, and (c) which of ATE and map disagreement will improve more. Run it, then explain your errors in terms of the gate's covariance: path uncertainty widens with the number of edges between the two nodes, so a noisier run has a *more* permissive gate at the same node separation. Is that the right behaviour? Argue both sides.

6 PRACTICAL •• Trimmed ICP

Add `trim: f64` to `IcpConfig`: after association, sort correspondences by residual and keep only the best `1 - trim` fraction before solving. Then contaminate the source cloud with 20% of points drawn from a moving obstacle (a 0.4 m box translating between the two sweeps) and compare vanilla ICP, trimmed ICP at 25%, and a Huber-kernelled variant. Report the bias in the recovered translation for each. Which of the three degrades most gracefully as the contamination fraction rises past 40%, and why?

7 PRACTICAL ••• A descriptor-based loop detector

Replace `detect_loop_candidates`' radius search with a descriptor: for each keyframe compute a 64-bin histogram of ranges, normalized to sum to one, and match by χ^2 distance with brute-force search over all past keyframes. Measure precision and recall against the covariance-gated search on the Apartment lap, over twenty seeds. You will find the descriptor fires in places the radius search never looks — and also in places it should not, because a corridor's range histogram is nearly position-invariant. Explain how Scan Context's rotation-invariant polar encoding addresses exactly this, and what it would cost you here.

16.8 References

Besl, P. J. and McKay, N. D. (1992). A Method for Registration of 3-D Shapes *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14(2), 239–256.

The paper that named ICP and proved the monotone-convergence property re-derived in this chapter’s second derivation. Its honesty about convergence being local only is still the correct warning. doi:10.1109/34.121791

Arun, K. S., Huang, T. S. and Blostein, S. D. (1987). Least-Squares Fitting of Two 3-D Point Sets *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-9(5), 698–700.

The closed-form SVD alignment of Derivation 1, including the determinant correction that excludes reflections. Two pages, and it has held up for forty years. doi:10.1109/TPAMI.1987.4767965

Chen, Y. and Medioni, G. (1992). Object Modelling by Registration of Multiple Range Images *Image and Vision Computing* 10(3), 145–155.

The point-to-plane objective, and the observation that penalizing motion along a surface is penalizing an artefact of the correspondence rather than a disagreement. Derivation 3 is this cost, linearized on SE(2). doi:10.1016/0262-8856(92)90066-C

Lu, F. and Milios, E. (1997). Globally Consistent Range Scan Alignment for Environment Mapping *Autonomous Robots* 4(4), 333–349.

The origin of pose-graph SLAM: relative scan-alignment constraints, a maximum-likelihood objective over all poses at once, and the observation that this is a sparse linear system. Everything in Section 3.6 is here first. doi:10.1023/A:1008854305733

Biber, P. and Straßer, W. (2003). The Normal Distributions Transform: A New Approach to Laser Scan Matching *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2743–2748.

NDT as this chapter presents it, including the covariance regularization and the overlapping-grid fix for cell-boundary discontinuities. doi:10.1109/IROS.2003.1249285

Hess, W., Kohler, D., Rapp, H. and Andor, D. (2016). Real-Time Loop Closure in 2D LIDAR SLAM *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 1271–1278.

Cartographer. The submap formulation and the branch-and-bound correlative matcher that finds the global optimum inside a search window — the principled cure for the wrong-minimum failure of w16.1. doi:10.1109/ICRA.2016.7487258

Macenski, S. and Jambrecic, I. (2021). SLAM Toolbox: SLAM for the Dynamic World *Journal of Open Source Software* 6(61), 2783.

The 2D SLAM system most robots actually run. Worth reading as engineering rather than theory: serialization, lifelong mapping, and localization against a previously built graph. doi:10.21105/joss.02783

Vizzo, I., Guadagnino, T., Mersch, B., Wiesmann, L., Behley, J. and Stachniss, C. (2023). KISS-ICP: In Defense of Point-to-Point ICP — Simple, Accurate, and Robust Registration If Done the Right Way *IEEE Robotics and Automation Letters* 8(2), 1029–1036.

The source of this chapter’s front-end design: constant-velocity prediction, voxel downsampling, a bounded voxel-hash local map, and the adaptive threshold $\tau = 3\sigma$. Its argument — that the classic objective done carefully beats elaborate feature pipelines — is why we teach it. doi:10.1109/LRA.2023.3236571

Lee, D., Jung, M., Yang, W. and Kim, A. (2024). LiDAR Odometry Survey: Recent Advancements and Remaining Challenges *arXiv:2312.17487*.

The map of everything this chapter deliberately skipped: LOAM’s feature lineage, LIO-SAM, FAST-LIO2, multi-LiDAR and LiDAR-inertial fusion, and the datasets they are measured on. [link](#)

FastSLAM and Rao-Blackwellization

MAPPING AND SLAM · Advanced · 55 min



This algorithm is based on an exact factorization of the posterior into a product of conditional landmark distributions and a distribution over robot paths.

Michael Montemerlo, Sebastian Thrun, Daphne Koller, and Ben Wegbreit

FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem (AAAI-02)

Chapter 14 attacked the joint pose-map posterior with one big Gaussian. Chapter 15 and Chapter 16 attacked it with one big least-squares problem. This chapter takes the third road, which is to refuse the joint entirely.

The observation is almost too simple to be worth a theorem: **if the robot's path were known, mapping would be easy** — that is literally what Chapter 13 assumed. And if the path were known, landmarks would not just be easy, they would be *independent of each other*, because the only thing that ever correlated them was uncertainty about where the robot was standing. So sample the path with particles, and let every particle solve its own map exactly.

What makes this more than a hack is that the variance never goes up. Rao-Blackwellization is a theorem, and it says that integrating a variable out analytically instead of sampling it can only help. The result turns an intractable $(3 + 2N)$ -dimensional particle filter into M cheap trajectory hypotheses, each towing a bank of tiny EKF's — or an occupancy grid.

🕒 AFTER THIS CHAPTER YOU CAN

- State and prove the SLAM factorization theorem, and say exactly which assumption each step consumes
- Explain Rao-Blackwellization as a variance theorem rather than an

implementation trick

- Derive FastSLAM 1.0's importance weight as a marginal measurement likelihood
- Derive FastSLAM 2.0's measurement-aware proposal, and measure what it buys in particles
- Build a grid-based Rao-Blackwellized particle filter — the gmapping recipe — with scan-matched proposals and selective resampling
- Say honestly what died with gmapping and what is permanent

ASSUMED

- Particle filters, importance weights, and the low-variance resampler (Chapter 8)
- The odometry motion model and its sampler (Chapter 9)
- Landmark and likelihood-field measurement models (Chapter 10)
- Occupancy grids in log odds (Chapter 13)
- Why EKF SLAM is $O(N^2)$ and unimodal (Chapter 14)

17.1 The map that never existed

Here is the naive thing, stated precisely enough to see it fail. The SLAM state is the pose plus the map, (x_t, m) . A particle filter represents a belief by samples. So: sample the whole state. Each particle is a pose *and a complete map*, weighted by how well it explains the scan.

Count the dimensions. Rusty's pose is 3. Twenty-four landmarks in the plane are 48 more. That is a 51-dimensional space, and the number of samples needed to cover a space to fixed resolution grows exponentially in its dimension — the curse of dimensionality from Chapter 8, arriving on schedule. Ten samples per axis would require 10^{51} particles. At a billion particles per second, drawing that population once takes about 10^{42} seconds, which is some 10^{24} times the current age of the universe. You get to do it at 10 Hz.

Worse, it is wasteful in a way that has nothing to do with dimension. Conditioned on a trajectory, the map has a *closed-form* posterior — it is a product of Gaussians, or a grid of log-odds counters. Sampling something you could have computed is throwing away accuracy for nothing, and there is a theorem later in this chapter that says exactly how much.

So here is the trade. Sample only the part that is genuinely hard — the path — and compute everything else. What the reader sees below is M separate universes, each of which took a different guess about where Rusty went, and each of which built the map that guess implies.

 INTERACTIVE · w17.1**Parallel Universes**

The filter does not have a map. It has M competing maps, and loop closure is natural selection among them.

Run this simulation in the web edition: [/chapters/ch17-fastslam #w17.1](/chapters/ch17-fastslam#w17.1)

Watch it for a lap before reading on. Three things are worth your attention, and each one is a section of this chapter.

There is no map. There are sixteen maps. They disagree, and the disagreement is the belief. Any "the map" you extract — the highest-weight tile, the weighted average, whatever — is a summary you chose, not a thing the filter holds. The reader who has been waiting since Chapter 13 for mapping and localization to be solved *together* should notice that this is what together actually looks like.

Weights are the referee, and the map is the referee's rulebook. Each universe scores the new scan against *its own* walls. A universe that drifted left builds walls that are shifted left, and its future scans keep agreeing with it — until the robot comes back to somewhere it has been, at which point the accumulated inconsistency has nowhere to hide, and the green border fades.

Loop closure is an extinction event, not an optimization step. In Chapter 16 closing a loop meant adding a constraint and running Gauss–Newton until the graph relaxed. Here it means some universes stop being paid and get deleted. No optimizer runs. Nothing is refined. The filter simply stops simulating the futures that turned out to be wrong.

17.2 If you knew the path

The intuition that unlocks all of this was already said out loud, twice, in earlier chapters.

Chapter 13 assumed known poses and built a map by counting. Chapter 11 assumed a known map and tracked a pose with an EKF. Each was easy; SLAM is hard because neither assumption holds. The Rao–Blackwellized move is to *make one of them hold* — not by knowing the path, but by conditioning on a guess about it, and then keeping many guesses around so that the conditioning is honest.

Formally, conditioning on the path does something stronger than making mapping convenient. It makes the landmarks **independent of one another**.

 INTERACTIVE · f17.1**The factorization**

Landmarks are correlated only through the robot's path. Freeze the path and the map falls apart into independent pieces.

Run this simulation in the web edition: `/chapters/ch17-fastslam #f17.1`

Look at the graphical model with the path marginalized out. Landmark m_1 was seen from poses x_0, x_1, x_2 ; landmark m_3 was seen from x_3, x_4, x_5 . They were never observed together, and yet integrating over the trajectory makes them correlated: any evidence that shifts your belief about x_2 shifts x_3 through the motion model, which shifts m_3 . Those induced couplings are precisely the off-diagonal blocks that made Chapter 14's covariance dense and its update $O(N^2)$.

Now condition on the path — click the toggle. The pose nodes become observed, the induced arcs vanish, and what is left is three separate little estimation problems that cannot talk to each other. Each is a 2×2 Gaussian updated by its own sightings.

💡 THE ONE SENTENCE VERSION

Landmarks are correlated only through the robot's trajectory. A particle that carries a whole trajectory therefore carries a map that factors — so sample paths, and solve each landmark exactly.

Two consequences follow immediately, and both are worth stating before the algebra.

The first is a cost argument. Chapter 14's update touched every entry of a $(3 + 2N) \times (3 + 2N)$ covariance. Here, observing one landmark updates one 2×2 matrix inside one particle, and touches nothing else — $O(M)$ per observation regardless of how large the map has grown.

The second is a *representational* argument, and it is the one that survived into 2026. Because each particle owns its map, each particle may also own its **data association**. Universe 3 can believe the doorway it just saw is landmark 7 while universe 9 believes it is a new landmark, and neither has to commit for everyone. The correspondence variable c_t , which Chapter 11 had to guess once and live with forever, becomes a sampled variable like any other.

17.3 The mathematics

17.3.1 Notation

$x_{0:t}^{[i]}$

The path hypothesis of particle i . A particle is a trajectory, not a pose — the whole chapter turns on this.

$m^{[i]}$

Particle i 's private map: a set of landmark Gaussians, or an occupancy grid.

$\mu_{j,t}^{[i]}, \Sigma_{j,t}^{[i]}$	Mean and 2x2 covariance of landmark j 's EKF inside particle i .
c_t	Correspondence: which landmark measurement z_t came from. Given for the factorization theorem; sampled per particle once associations are unknown.
$\pi(x_t \cdot)$	The proposal distribution the new pose is drawn from. Choosing it well is what separates FastSLAM 1.0 from 2.0.
$\hat{x}_t^{[i]}$	The scan-match optimum: the mode of the observation likelihood against particle i 's own map.
N_{eff}	Effective sample size, $1/\Sigma(\tilde{w})^2$. Chapter 8 called it M_{eff} ; the gmapping literature writes N_{eff} , and so do we here.
$\eta^{[i]}$	The weight increment of the improved proposal — the marginal likelihood of the scan under particle i .

17.3.2 The SLAM factorization theorem

Everything rests on one identity. With known correspondences $c_{1:t}$,

$$p(x_{0:t}, m | z_{1:t}, u_{1:t}, c_{1:t}) = p(x_{0:t} | z_{1:t}, u_{1:t}, c_{1:t}) \prod_{j=1}^N p(m_j | x_{0:t}, z_{1:t}, c_{1:t})$$

Read it right to left. The product on the right is N independent two-dimensional Gaussians, each of which a Kalman filter can maintain in closed form. The factor on the left is a distribution over *trajectories*, which is exactly the kind of thing a particle filter is good at. The theorem says this split is **exact** — no approximation has been made yet.

DERIVATION Proving the factorization by induction on t

$$p(m | x_{0:t}, d_t) = \prod_j p(m_j | x_{0:t}, d_t)$$

Write $d_t = (z_{1:t}, u_{1:t}, c_{1:t})$ for the data. The claim is that the map factors over landmarks once the path is fixed. Since $p(x_{0:t}, m | d_t) = p(x_{0:t} | d_t) p(m | x_{0:t}, d_t)$ is just the chain rule, the whole content of the theorem is:

$$p(m | x_{0:t}, d_t) = \prod_{j=1}^N p(m_j | x_{0:t}, z_{1:t}, c_{1:t})$$

Base case. At $t = 0$ no measurement has arrived, so the map posterior is the prior, which we take to be independent across landmarks — an honest assumption, since before seeing anything there is no reason to believe two unobserved landmarks are related.

Inductive step. Assume the factorization holds at $t - 1$. Apply Bayes rule

to the newest measurement, conditioning throughout on the full path:

$$p(m \mid x_{0:t}, d_t) = \eta p(z_t \mid m, x_t, c_t) p(m \mid x_{0:t-1}, d_{t-1})$$

The second factor lost its dependence on x_t and u_t : a control tells you nothing about where landmarks are, and the current pose adds nothing beyond the path that led to it.

Now the key step. With $c_t = j$ given, the measurement z_t depends on exactly one landmark:

$$p(z_t \mid m, x_t, c_t = j) = p(z_t \mid m_j, x_t)$$

This is where the theorem is really won or lost. It is a statement about the *sensor*: one reading touches one landmark. Substituting and using the inductive hypothesis,

$$p(m \mid x_{0:t}, d_t) = \eta p(z_t \mid m_j, x_t) \prod_{k=1}^N p(m_k \mid x_{0:t-1}, d_{t-1})$$

Every factor with $k \neq j$ is untouched. The factor with $k = j$ is multiplied by a likelihood that involves only m_j , and the normalizer η splits across the product because it depends on neither. So the posterior at t is again a product over landmarks, with only the observed one having changed:

$$p(m_k \mid x_{0:t}, d_t) = \begin{cases} \eta_j p(z_t \mid m_j, x_t) p(m_j \mid x_{0:t-1}, d_{t-1}) & k = j \\ p(m_k \mid x_{0:t-1}, d_{t-1}) & k \neq j \end{cases}$$

which closes the induction. ■

Where the proof breaks. Suppose one measurement observed two landmarks at once — a stereo pair, a scan segment spanning a corner, any front end that emits a joint constraint. Then $p(z_t \mid m, x_t, c_t)$ does not collapse to a single factor, the two landmarks acquire a shared likelihood term, and the product structure is gone for that pair. This is not hypothetical: it is exactly what a laser *scan* is. When the grid-based filter later in this chapter integrates a whole scan into one map it is quietly assuming the cells are conditionally independent given the path — the same assumption Chapter 13 made, with the same known-false status.

Where the online version breaks. Note the conditioning: $p(m_j \mid x_{0:t}, \cdot)$, on the *whole path*. The corresponding statement for the marginal $p(x_t, m \mid \cdot)$ — the thing an ordinary filter tracks — is **false**. Landmarks are not conditionally independent given the current pose alone; a robot that returns to an old room correlates what it sees now with what it saw then through everywhere it went in between. That is why a particle here must

carry its whole trajectory, and why FastSLAM's memory grows with time even when its map does not.

17.3.3 Rao-Blackwellization is a theorem, not a trick

Why should sampling paths and computing maps beat sampling both? Because of a variance inequality that predates robotics by fifty years.

Let $\varphi(X, Y)$ be any estimator of a quantity of interest, where X is what we sample and Y is what we could integrate out. The **Rao-Blackwell** estimator is the conditional expectation $\mathbb{E}[\varphi \mid X]$, and it satisfies

$$\text{Var} [\mathbb{E}[\varphi \mid X]] \leq \text{Var} [\varphi(X, Y)]$$

Identify X with the path and Y with the map, and the statement becomes: a filter that samples trajectories and computes map posteriors is *never worse*, in variance, than one that samples both — and is strictly better whenever the map genuinely varies given the path.

DERIVATION The variance inequality, and what it means for SMC

$$\text{Var}[\varphi] = \mathbb{E}[\text{Var}[\varphi \mid X]] + \text{Var}[\mathbb{E}[\varphi \mid X]]$$

Step 1 — the law of total variance. For any two random variables,

$$\text{Var}[\varphi] = \underbrace{\mathbb{E}[\text{Var}[\varphi \mid X]]}_{\geq 0} + \text{Var}[\mathbb{E}[\varphi \mid X]]$$

Both terms on the right are non-negative, so dropping the first gives the inequality directly. The dropped term is the variance *contributed by sampling* Y — which is precisely what Rao-Blackwellization deletes by computing Y 's posterior instead of drawing from it.

Step 2 — unbiasedness is preserved. By the tower rule $\mathbb{E}[\mathbb{E}[\varphi \mid X]] = \mathbb{E}[\varphi]$, so nothing is traded away: the conditional estimator estimates the same quantity, with no more variance.

Step 3 — the SMC version. In sequential Monte Carlo the estimator of interest is the importance weight itself. Doucet et al. (2000) show that when the state factors as (X, Y) with Y analytically tractable given $X_{0:t}$, the correct weight for a sample of $X_{0:t}$ alone is the *marginalized* weight

$$w_t \propto \frac{p(x_{0:t} \mid z_{1:t})}{\pi(x_{0:t} \mid z_{1:t})} \quad \text{with} \quad p(z_t \mid x_{0:t}, z_{1:t-1}) = \int p(z_t \mid x_t, y) p(y \mid x_{0:t-1}, z_{1:t-1}) dy$$

That integral is the whole game: it is what the FastSLAM 1.0 weight

evaluates in closed form for a Gaussian landmark, and what the gmapping recipe approximates by summing over K probe poses when the map is a grid.

Step 4 — the practical reading. Fewer particles for the same accuracy. How many fewer depends on how much of the state’s variance lived in Y ; for SLAM, where Y is a $2N$ -dimensional map and X is a 3-dimensional pose per step, the answer is *nearly all of it*. That is why a 51-dimensional problem can be attacked with sixteen particles.

NOTE

Rao-Blackwellization is not specific to SLAM. Anywhere your state splits into a hard nonlinear part and a conditionally-linear-Gaussian part — a maneuvering target with linear dynamics and an unknown mode, a robot with an unknown sensor bias — the same theorem applies, and the same architecture follows. Chapter 22 meets it again as the reason POMCP’s particles carry world states rather than full belief distributions.

17.3.4 The FastSLAM 1.0 weight

Now make it concrete. Particle i holds a pose $x_{t-1}^{[i]}$ and, for each landmark, a Gaussian $\mathcal{N}(\mu_j^{[i]}, \Sigma_j^{[i]})$. It draws a new pose from the motion model,

$$x_t^{[i]} \sim p(x_t \mid x_{t-1}^{[i]}, u_t)$$

using `sample_motion_model_odometry` from Chapter 9 — the proposal is the motion prior, which is the same choice Monte Carlo localization made in Chapter 12, for the same reason: everything cancels except the measurement likelihood. Here that likelihood must be *marginal over the landmark*, and the result is

$$w_t^{[i]} = |2\pi\mathbf{S}_t^{[i]}|^{-1/2} \exp\left(-\frac{1}{2} \boldsymbol{\nu}^\top (\mathbf{S}_t^{[i]})^{-1} \boldsymbol{\nu}\right), \quad \boldsymbol{\nu} = \mathbf{z}_t - \hat{\mathbf{z}}_t^{[i]}, \quad \mathbf{S}_t^{[i]} = \mathbf{H}_m \Sigma_{c_t}^{[i]} \mathbf{H}_m^\top + \mathbf{Q}_t$$

That $\mathbf{S}_t$ should look familiar: it is the innovation covariance from Chapter 11, the same matrix that defines the Mahalanobis gate. Here it does double duty — it weights the particle *and* gates its data association.

DERIVATION Where the weight comes from, and why the landmark integrates out

$$w_t^{[i]} = \mathcal{N}(\mathbf{z}_t; \hat{\mathbf{z}}_t^{[i]}, \mathbf{H}_m \Sigma \mathbf{H}_m^\top + \mathbf{Q}_t)$$

Step 1 — weight = target / proposal. From Chapter 8, importance sampling

assigns $w \propto \text{target/proposal}$. The target at time t is the path posterior; the proposal is the path posterior at $t - 1$ extended by the motion model. Telescoping the recursion, everything cancels except the newest measurement's *predictive* likelihood:

$$w_t^{[i]} \propto p(z_t | x_{0:t}^{[i]}, z_{1:t-1}, c_{1:t})$$

This is the step where Rao-Blackwellization enters: the weight is a likelihood with the map marginalized out, not evaluated at some sampled map.

Step 2 — marginalize the landmark. Insert the landmark and integrate it back out:

$$p(z_t | x_{0:t}^{[i]}, z_{1:t-1}, c_t = j) = \int p(z_t | x_t^{[i]}, m_j) p(m_j | x_{0:t-1}^{[i]}, z_{1:t-1}) dm_j$$

Step 3 — linearize and use the Gaussian identity. The measurement model $h(x_t, m_j)$ is nonlinear (a range and a bearing), so expand it about the landmark mean:

$$h(x_t^{[i]}, m_j) \approx \hat{z}_t^{[i]} + \mathbf{H}_m (m_j - \mu_j^{[i]}), \quad \mathbf{H}_m = \left. \frac{\partial h}{\partial m} \right|_{x_t^{[i]}, \mu_j^{[i]}}$$

Now both factors are Gaussian in m_j , and Appendix B's marginal identity applies: if $m_j \sim \mathcal{N}(\mu, \Sigma)$ and $z | m_j \sim \mathcal{N}(\hat{z} + \mathbf{H}_m(m_j - \mu), \mathbf{Q})$ then

$$z \sim \mathcal{N}(\hat{z}, \mathbf{H}_m \Sigma \mathbf{H}_m^\top + \mathbf{Q})$$

which is the stated weight. Note what the two terms mean: $\mathbf{Q}$ is what the *sensor* does not know, $\mathbf{H}_m \Sigma \mathbf{H}_m^\top$ is what the *map* does not know, and a landmark seen for the hundredth time contributes almost nothing to the second.

Step 4 — then update the landmark. Having weighted the particle, correct the observed landmark's EKF at the sampled pose, with $\mathbf{K} = \Sigma \mathbf{H}_m^\top \mathbf{S}^{-1}$ and the usual Kalman update. Landmarks that were not observed are not touched at all — no loop over the map, no covariance propagation. That is the $O(1)$ -per-observation claim, made good.

Step 5 — initialization. A landmark seen for the first time has no prior to marginalize. Invert the measurement instead: put $\mu_j = g(x_t, z_t)$ (project range and bearing out from the pose) and $\Sigma_j = \mathbf{H}_m^{-1} \mathbf{Q}_t \mathbf{H}_m^{-\top}$. Notice this covariance contains *no* pose uncertainty — inside this particle, the pose is known exactly. That is Rao-Blackwellization paying out in the most visible possible way.

17.3.5 Why 1.0 wastes particles, and what 2.0 does about it

FastSLAM 1.0 proposes from the motion model and then judges the result with the measurement. When the wheels are vague and the sensor is precise — which describes every robot built since about 1995 — this is a bad plan. The proposal spreads samples over a region tens of centimetres wide; the likelihood concentrates them in a region a few centimetres wide; and almost every particle lands somewhere the measurement considers absurd.

INTERACTIVE · w17.3

Proposal Quality

A better proposal beats more particles: sampling where the measurement already says the robot is turns wasted particles into useful ones.

Run this simulation in the web edition: `/chapters/ch17-fastslam #w17.3`

The counters underneath make the cost quantitative. At the default $\sigma_r = 5$ cm the motion-prior proposal puts about 43% of its samples inside the posterior's 90% region and lands an effective sample size of $N_{\text{eff}}/M \approx 0.18$; the measurement-aware proposal puts exactly 90% of them there — as it must, since it *is* the posterior — with $N_{\text{eff}}/M = 1.00$. In round numbers, one FastSLAM 2.0 particle is doing the work of five of FastSLAM 1.0's, and the ratio widens as the sensor sharpens: at $\sigma_r = 2$ cm it is six, at $\sigma_r = 40$ cm it is under two.

The fix is to fold z_t into the proposal itself:

$$\pi(x_t) = p(x_t \mid x_{1:t-1}^{[i]}, u_{1:t}, z_{1:t}, c_{1:t}) \propto p(z_t \mid x_t, m^{[i]}) p(x_t \mid x_{t-1}^{[i]}, u_t)$$

This is the *optimal* proposal in Doucet's sense — among proposals that use the information available at time t , it minimizes the variance of the importance weights. Linearizing the measurement in the pose makes it a Gaussian you can compute with one EKF-shaped step.

DERIVATION FastSLAM 2.0's proposal, by completing the square

$$\Sigma_x = [\mathbf{H}_x^\top \mathbf{Q}_j^{-1} \mathbf{H}_x + \mathbf{R}_t^{-1}]^{-1}$$

Step 1 — name the two factors. The motion prior, linearized about the odometry-predicted pose $\hat{x}_t = g(u_t, x_{t-1}^{[i]})$, is $\mathcal{N}(\hat{x}_t, \mathbf{R}_t)$, where $\mathbf{R}_t$ is the α -model covariance of Chapter 9 pushed into pose coordinates by the Jacobian of the rotate–translate–rotate composition — the same $\mathbf{VMV}^\top$ construction Chapter 11 used.

The measurement factor, with the landmark already marginalized as in

Derivation 3, is $\mathcal{N}(z_t; h(x_t, \mu_j), \mathbf{Q}_j)$ with $\mathbf{Q}_j = \mathbf{Q}_t + \mathbf{H}_m \Sigma_j \mathbf{H}_m^\top$.

Step 2 — linearize in the pose. Expand h about $\hat{x}_t$ this time, not about the landmark:

$$h(x_t, \mu_j) \approx \hat{z}_t + \mathbf{H}_x(x_t - \hat{x}_t), \quad \mathbf{H}_x = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_t, \mu_j}$$

For a range-bearing sensor $\mathbf{H}_x$ is the 2×3 matrix $\begin{bmatrix} -\delta_x/r, -\delta_y/r, 0; \delta_y/q, -\delta_x/q, -1 \end{bmatrix}$ with $\delta = \mu_j - x_t$, $q = \|\delta\|^2$, $r = \sqrt{q}$.

Step 3 — multiply two Gaussians. The product of two Gaussian densities in x_t is an unnormalized Gaussian whose **information matrices add**:

$$\Sigma_x^{-1} = \mathbf{H}_x^\top \mathbf{Q}_j^{-1} \mathbf{H}_x + \mathbf{R}_t^{-1}, \quad \mu_x = \hat{x}_t + \Sigma_x \mathbf{H}_x^\top \mathbf{Q}_j^{-1} (z_t - \hat{z}_t)$$

This is the information form of Chapter 6, run once per particle. Sample $x_t^{[i]} \sim \mathcal{N}(\mu_x, \Sigma_x)$ and the proposal step is done.

Step 4 — the weight, and why its variance falls. Because the sample came from a proposal that already used z_t , the importance weight is no longer the measurement likelihood. Carrying the ratio through gives the *predictive* likelihood

$$w_t^{[i]} \propto \int p(z_t | x_t) p(x_t | x_{t-1}^{[i]}, u_t) dx_t = \mathcal{N}(z_t; \hat{z}_t, \mathbf{H}_x \mathbf{R}_t \mathbf{H}_x^\top + \mathbf{Q}_j)$$

Look at what this expression does *not* contain: $x_t^{[i]}$. The weight depends on where the particle came from, not on where the proposal happened to put it — so two particles with the same history get the same weight no matter how their draws differed. That is the variance reduction, visible in the widget as $N_{\text{eff}}/M = 1.00$.

Step 5 — when it fails. Everything above assumed the measurement likelihood is unimodal enough to linearize. If it is not — a corridor where a scan matches equally well in two places, a landmark with an ambiguous signature — the Gaussian proposal will happily concentrate all M particles at one of the modes and the filter will lose the other. FastSLAM 1.0's dumb proposal, by contrast, covers both. There is no free lunch: a sharper proposal is a stronger commitment.

17.3.6 Grids: the gmapping recipe

Landmarks are a luxury. In a bare corridor there is nothing to call a landmark and the map is an occupancy grid, so the closed forms of the last section evaporate: there is no $\mathbf{H}_m$, no Σ_j , no integral you can do by hand.

Grisetti, Stachniss and Burgard's answer — the algorithm the world knew as *gmapping* for fifteen years — is to keep the *shape* of FastSLAM 2.0 and replace algebra with local sampling.

1. **Find the mode.** Scan-match z_t against the particle's own map $m_{t-1}^{[i]}$, starting from the odometry-predicted pose, giving $\hat{x}_t^{[i]}$. Any matcher from Chapter 16 does; ours is a greedy hill climb on the map-match score, which is what gmapping itself used.
2. **Probe around it.** Draw K poses $\{x_k\}$ in a small neighbourhood of $\hat{x}_t^{[i]}$.
3. **Weight the probes** by observation $\times$ motion, $\omega_k = p(z_t \mid m^{[i]}, x_k) p(x_k \mid x_{t-1}^{[i]}, u_t)$.
4. **Fit and sample.** Compute the weighted mean and covariance of the probes, and draw the new pose from that Gaussian. This is Step 3 of the previous derivation, done numerically.
5. **Pay the normalizer.** Multiply the particle's weight by $\eta^{[i]} = \sum_k \omega_k$ — the Monte Carlo estimate of the same predictive likelihood integral that FastSLAM 2.0 evaluated in closed form.
6. **Map, then select.** Integrate the scan into $m^{[i]}$ at the sampled pose with Chapter 13's log-odds update, and resample only if $N_{\text{eff}} < M/2$.

Two details in that list are load-bearing, and both are visible in the widgets.

The scan matcher must be allowed to fail. On a fresh map the score surface is flat — every pose explains an unexplored grid equally well — and the "optimum" it returns is numerical noise. The implementation compares the matched score against the seed score and falls back to plain motion sampling when the improvement is not real. Without that guard, the filter confidently follows garbage for the first ten steps and never recovers.

Resampling is a cost, not a service. Chapter 8 said this about localization; here it decides whether there are still competing universes left when the loop finally closes.

INTERACTIVE · w17.2

Depletion Meter

Resampling is not free: it fixes weight degeneracy by spending diversity, and the meter that shows the first is blind to the second.

Run this simulation in the web edition: [/chapters/ch17-fastslam #w17.2](#)

The three policies expose two different failure modes, and the instructive part is that the meter everyone watches only sees one of them.

Never resample and the weights degenerate: N_{eff}/M slides to about 0.1, meaning that of sixteen universes being simulated and paid for, one holds essentially all the probability. The filter has quietly become a single-hypothesis estimator with a fifteen-fold overhead, and the error curve shows it — depending on the seed, the lap error grows by anywhere from 20% to a factor of three.

Resample every step and N_{eff} looks fine forever, because the weights are wiped clean each step and the statistic only ever reports one step of evidence. What dies instead is ancestry: the lineage gauge falls to 1, meaning every surviving universe is a copy of one great-grandparent and the population can no longer disagree about anything. This is Chapter 8's particle deprivation wearing a different hat, and N_{eff} is structurally blind to it.

Resample when $N_{\text{eff}} < M/2$ buys the middle — and be honest about how little it buys here. Even the selective policy fires on roughly half the steps of this lap, and the lineage gauge still collapses inside about twenty steps; on error alone the always-resample policy is indistinguishable from it on this benign out-and-back. The difference shows up in the cases this lap does not contain: an ambiguous junction, a symmetric wing, a loop closed after ten minutes of drift, where the hypothesis that turns out to be right must still exist when the evidence arrives. Selective resampling slows the loss of diversity; nothing on that panel restores it, which is why the recovery machinery of Chapter 12 is not optional in a deployed system.

17.4 The algorithms

ALGORITHM FastSLAM_1_0($_t$, u_t , z_t , c_t) COST $O(M)$ per observation; $O(MN)$ memory, or $O(M \log N)$ with shared trees
in the particle set, the control, one measurement, its correspondence
out $_t$

```

1:  $\bar{\mathcal{X}}_t = \emptyset$ 
2: for  $i = 1$  to  $M$  do
3:    $x_t^{[i]} \sim p(x_t | x_{t-1}^{[i]}, u_t)$  // sample_motion_model_odometry
4:    $j = c_t$ 
5:   if landmark  $j$  never seen then
6:      $\mu_j^{[i]} = g^{-1}(x_t^{[i]}, z_t)$ ,  $\Sigma_j^{[i]} = \mathbf{H}_m^{-1} \mathbf{Q}_t \mathbf{H}_m^{-\top}$ ,  $w^{[i]} = p_0$ 
7:   else
8:      $\hat{z} = h(x_t^{[i]}, \mu_j^{[i]})$ ,  $\mathbf{S} = \mathbf{H}_m \Sigma_j^{[i]} \mathbf{H}_m^{\top} + \mathbf{Q}_t$ 
```

```

9:       $w^{[i]} = |2\pi\mathbf{S}|^{-1/2} \exp(-\frac{1}{2}(z_t - \hat{z})^T \mathbf{S}^{-1}(z_t - \hat{z}))$ 
10:      $\mathbf{K} = \Sigma_j^{[i]} \mathbf{H}_m^T \mathbf{S}^{-1}; \mu_j^{[i]} += \mathbf{K}(z_t - \hat{z}); \Sigma_j^{[i]} = (\mathbf{I} - \mathbf{K}\mathbf{H}_m) \Sigma_j^{[i]}$ 
11:   endif
12:   all  $k \neq j$ : copy  $\mu_k^{[i]}, \Sigma_k^{[i]}$  unchanged
13:   add  $\langle x_t^{[i]}, \{\mu, \Sigma\}, w^{[i]} \rangle$  to  $\bar{\mathcal{X}}_t$ 
14: endfor
15:  $\mathcal{X}_t = \text{Low\_variance\_sampler}(\bar{\mathcal{X}}_t)$  // Table 4.4
16: return  $\mathcal{X}_t$ 

```

FastSLAM 2.0 changes exactly two lines. Line 3 samples from the measurement-aware proposal, and line 9 uses the predictive covariance $\mathbf{H}_x \mathbf{R}_t \mathbf{H}_x^T + \mathbf{Q}_j$ instead of $\mathbf{S}$. Everything else — the per-landmark EKF, the untouched landmarks, the resampler — is identical.

ALGORITHM FastSLAM_unknown_correspondence($\{t-1\}$, u_t , z_t) COST $O(M \cdot N)$ per observation; the gate usually makes it $O(M \cdot k)$ for k candidates in view
in the particle set, the control, one measurement with no correspondence
out $_t$ with per-particle data association

```

1: for  $i = 1$  to  $M$  do
2:   sample  $x_t^{[i]}$  from the chosen proposal
3:   for each landmark  $k$  in particle  $i$ 's map do
4:      $\mathbf{v}_k = z_t - h(x_t^{[i]}, \mu_k^{[i]}), \mathbf{S}_k = \mathbf{H}_m \Sigma_k^{[i]} \mathbf{H}_m^T + \mathbf{Q}_t$ 
5:     if  $\mathbf{v}_k^T \mathbf{S}_k^{-1} \mathbf{v}_k > \chi_{2,0.99}^2$  then skip  $k$  // the gate
6:      $\ell_k = \mathcal{N}(\mathbf{v}_k; 0, \mathbf{S}_k)$ 
7:   endfor
8:    $\hat{c}^{[i]} = \arg \max_k \ell_k$ ; if  $\max_k \ell_k < p_0$  then  $\hat{c}^{[i]} = \text{new landmark}$ 
9:   update particle  $i$  with correspondence  $\hat{c}^{[i]}$  as in FastSLAM_1_0
10: endfor
11: resample

```

Line 8 is the line that matters, and it is worth pausing on. The maximum-likelihood association is computed *per particle*, against that particle's own map and its own pose. Different particles reach different conclusions, and the ambiguity is resolved not by a decision but by survival: whichever association was right leads to consistent future scans and its universe multiplies. Chapter 11's EKF had to pick one and could never take it back.

```

ALGORITHM gmapping_step(_{t-1}, u_t, z_t) COST  $O(M \cdot (\text{scan\_match} + K \text{ likelihood evaluations} + \text{grid update}))$ ; memory  $O(M \cdot \text{cells})$ , shared copy-on-write
in grid particles, odometry, a laser scan
out _t

```

```

1: for  $i = 1$  to  $M$  do
2:    $x'_t = g(u_t, x_{t-1}^{[i]})$  // odometry prediction
3:    $\hat{x}_t^{[i]} = \text{scan\_match}(m_{t-1}^{[i]}, z_t, x'_t)$ 
4:   if the match did not beat  $x'_t$  then // flat score: unexplored map
5:      $x_t^{[i]} \sim p(x_t | x_{t-1}^{[i]}, u_t)$ ;  $w^{[i]} *= p(z_t | m^{[i]}, x_t^{[i]})$ 
6:   else
7:     sample  $\{x_k\}_{k=1}^K$  around  $\hat{x}_t^{[i]}$ ;  $\omega_k = p(z_t | m^{[i]}, x_k) p(x_k | x_{t-1}^{[i]}, u_t)$ 
8:      $\mu = \frac{1}{\sum \omega} \sum_k \omega_k x_k$ ;  $\Sigma = \frac{1}{\sum \omega} \sum_k \omega_k (x_k - \mu)(x_k - \mu)^\top$ 
9:      $x_t^{[i]} \sim \mathcal{N}(\mu, \Sigma)$ ;  $w^{[i]} *= \eta^{[i]} = \sum_k \omega_k$ 
10:  endif
11:   $m^{[i]} = \text{occupancy\_grid\_mapping}(m^{[i]}, x_t^{[i]}, z_t)$  // Table 9.1
12: endfor
13: normalize  $\{w^{[i]}\}$ ;  $N_{\text{eff}} = 1 / \sum_i (\tilde{w}^{[i]})^2$ 
14: if  $N_{\text{eff}} < M/2$  then  $\mathcal{X}_t = \text{Low\_variance\_sampler}(\tilde{\mathcal{X}}_t)$ , reset weights
15: return  $\mathcal{X}_t$ 

```

17.5 Implementation in Rust

Before the code, one observation about the dependency list. This chapter uses `nalgebra` for 2×2 and 3×3 blocks, `rand + rand_distr` with a seeded `Pcg64`, `statrs` in the tests to cross-check the Gaussian densities, and `rayon` behind a native-only feature for the per-particle loop. What it does **not** use is `faer` or `factrs` — and that absence is the architectural point. Chapter 15 needed a sparse linear solver because it was inverting one enormous coupled system. Here there is no system to invert: the factorization theorem has already broken the problem into $M \times N$ pieces of size 2. The largest matrix anyone factors in this entire chapter is 3×3 .

Three types carry the rest: a landmark's EKF, a particle that owns a path and a bank of them, and a copy-on-write grid for the map-based variant.

```

RUST crates/ch17_fastslam/src/landmark_ekf.rs

```

```

1 use nalgebra::{Matrix2, Matrix2x3, Vector2};
2 use pr_core::geom::SE2;
3
4 // One landmark's posterior inside one particle: a 2-D Gaussian, and nothing

```

```

5  // else. There is no pose block and no cross-covariance with any other
6  // landmark, because conditioned on this particle's path there is nothing for
7  // them to correlate through. That absence *is* the factorization theorem.
8  #[derive(Clone, Debug)]
9  pub struct LandmarkEKF {
10     pub mu: Vector2<f64>,
11     pub sigma: Matrix2<f64>,
12     // Sightings so far -- FastSLAM's cheap stand-in for existence evidence.
13     pub hits: u32,
14 }
15
16 // A range-bearing feature, Thrun's  $f = (r, \phi, s)$  without the signature.
17 #[derive(Clone, Copy, Debug)]
18 pub struct Feature {
19     pub r: f64,
20     pub phi: f64,
21 }
22
23 impl LandmarkEKF {
24     // Initialize from a first sighting: invert the measurement, then push the
25     // sensor covariance through the inverse Jacobian.
26     ///
27     // Sigma carries *no* pose uncertainty. Inside this particle the pose is known
28     // exactly -- that is the whole point of sampling the path.
29     pub fn from_first_sighting(x: &SE2, z: Feature, q: &Matrix2<f64>) -> Self {
30         let bearing = x.theta() + z.phi;
31         let mu = Vector2::new(x.x() + z.r * bearing.cos(), x.y() + z.r * bearing.sin());
32         let h = landmark_jacobian(x, &mu);
33         let h_inv = h.try_inverse().expect("range-bearing Jacobian is singular only at r
34 = 0");
35         Self { mu, sigma: h_inv * q * h_inv.transpose(), hits: 1 }
36     }
37
38     // Innovation nu and its covariance S = H Sigma H^T + Q -- Chapter 11's gate,
39     // unchanged, and the one matrix this chapter uses for three jobs at once:
40     // weighting, gating, and the Kalman gain.
41     pub fn innovation(&self, x: &SE2, z: Feature, q: &Matrix2<f64>) -> (Vector2<f64>,
42 Matrix2<f64>) {
43         let (r_hat, phi_hat) = predict(x, &self.mu);
44         let h = landmark_jacobian(x, &self.mu);
45         let nu = Vector2::new(z.r - r_hat, wrap_angle(z.phi - phi_hat));
46         (nu, h * self.sigma * h.transpose() + q)
47     }
48
49     // FastSLAM 1.0's importance weight: the measurement's *marginal*
50     // likelihood, with the landmark integrated out (Derivation 3).
51     pub fn weight_1_0(&self, x: &SE2, z: Feature, q: &Matrix2<f64>) -> f64 {
52         let (nu, s) = self.innovation(x, z, q);
53         let s_inv = s.try_inverse().expect("S is positive definite whenever Q is");
54         let det = (std::f64::consts::TAU * s).determinant();
55         det.powf(-0.5) * (-0.5 * (nu.transpose() * s_inv * nu)[0]).exp()
56     }
57
58     // Standard EKF correction of this landmark at the sampled pose.
59     pub fn update(&mut self, x: &SE2, z: Feature, q: &Matrix2<f64>) {
60         let (nu, s) = self.innovation(x, z, q);
61         let h = landmark_jacobian(x, &self.mu);
62         let k = self.sigma * h.transpose() * s.try_inverse().unwrap();
63         self.mu += k * nu;
64         self.sigma = (Matrix2::identity() - k * h) * self.sigma;
65         self.hits += 1;

```



```

64     }
65 }
66
67 /// ?h/?m for the range-bearing model.
68 fn landmark_jacobian(x: &SE2, mu: &Vector2<f64>) -> Matrix2<f64> {
69     let d = Vector2::new(mu.x - x.x(), mu.y - x.y());
70     let q = d.norm_squared().max(1e-12);
71     let r = q.sqrt();
72     Matrix2::new(d.x / r, d.y / r, -d.y / q, d.x / q)
73 }
74
75 /// ?h/?x. Only FastSLAM 2.0 needs this one -- 1.0 never differentiates the
76 /// measurement with respect to the pose, which is precisely its weakness.
77 fn pose_jacobian(x: &SE2, mu: &Vector2<f64>) -> Matrix2x3<f64> {
78     let d = Vector2::new(mu.x - x.x(), mu.y - x.y());
79     let q = d.norm_squared().max(1e-12);
80     let r = q.sqrt();
81     Matrix2x3::new(-d.x / r, -d.y / r, 0.0, d.y / q, -d.x / q, -1.0)
82 }

```

The particle is where the theorem becomes a data structure. Note the path field: a particle is a trajectory, and the reason it must be is in the collapsible proof above.

RUST crates/ch17_fastslam/src/fastslam.rs

```

1 use nalgebra::{Matrix2, Matrix3, Vector2, Vector3};
2 use rand::SeedableRng;
3 use rand_distr::{Distribution, StandardNormal};
4 use rand_pcg::Pcg64;
5 use pr_core::geom::SE2;
6 use pr_core::models::OdomDelta;
7
8 pub struct FsParticle {
9     pub pose: SE2,
10     /// The particle IS a path. Dropping this field would break the
11     /// factorization theorem, not merely lose a visualization.
12     pub path: Vec<SE2>,
13     pub landmarks: Vec<LandmarkEkf>,
14     pub log_w: f64,
15 }
16
17 /// FastSLAM 1.0 vs 2.0, as a choice of proposal rather than a fork of the code.
18 #[derive(Clone, Copy, PartialEq, Eq, Debug)]
19 pub enum Proposal {
20     MotionPrior,
21     MeasurementAware,
22 }
23
24 pub struct FastSlam {
25     particles: Vec<FsParticle>,
26     proposal: Proposal,
27     /// Measurement noise Q = diag(sigma_r2, sigma_phi2).
28     q: Matrix2<f64>,
29     /// Chapter 9's odometry alpha's, used for both the sampler and R.t.
30     alphas: [f64; 4],
31     /// p_0 from `FastSLAM_unknown_correspondence`: the likelihood credited to a
32     /// brand-new landmark, and therefore the bar an old one has to clear.

```

```

33     p0: f64,
34     rng: Pcg64,
35 }
36
37 impl FastSlam {
38     pub fn step(&mut self, u: &OdomDelta, z: &[Feature], c: Option<&[usize]>) -> FsReport
39     {
40         for p in &mut self.particles {
41             let r = odometry_pose_covariance(&p.pose, u, &self.alphas);
42             // 2.0 folds one already-mapped feature into the proposal. An
43             // unmapped one carries no pose information, so there is nothing to
44             // fold and we fall back to 1.0's motion prior for this step.
45             let anchor = (self.proposal == Proposal::MeasurementAware)
46                 .then(|_| z.first().and_then(|f| self.associate(p, u, f, c)))
47                 .flatten();
48
49             let pose = match anchor {
50                 Some(j) => {
51                     let (x, log_w) = self.propose_2_0(p, u, &r, z[0], j);
52                     p.log_w += log_w;
53                     x
54                 }
55                 None => sample_motion_model_odometry(u, &p.pose, &self.alphas, &mut self.
56                     rng),
57             };
58
59             for (k, f) in z.iter().enumerate() {
60                 match self.associate_at(p, &pose, f, c.map(|cs| cs[k])) {
61                     Some(j) => {
62                         // 1.0 pays here; 2.0 already paid for the anchor above.
63                         if !(k == 0 && anchor.is_some()) {
64                             p.log_w += p.landmarks[j].weight_1_0(&pose, *f, &self.q).ln()
65                         };
66                     }
67                     None => {
68                         p.landmarks[j].update(&pose, *f, &self.q);
69                     }
70                 }
71             }
72
73             p.pose = pose;
74             p.path.push(pose);
75         }
76         self.select()
77     }
78
79     /// FastSLAM 2.0's proposal (Derivation 4): add precisions, let the
80     /// measurement pull the mean, and return the predictive log-likelihood as
81     /// the weight increment.
82     fn propose_2_0(
83         &mut self,
84         p: &FsParticle,
85         u: &OdomDelta,
86         r: &Matrix3<f64>,
87         z: Feature,
88         j: usize,
89     ) -> (SE2, f64) {

```

```

90     let lm = &p.landmarks[j];
91     let x_hat = p.pose.apply_odom(u);
92     let (r_hat, phi_hat) = predict(&x_hat, &lm.mu);
93     let hm = landmark_jacobian(&x_hat, &lm.mu);
94     let hx = pose_jacobian(&x_hat, &lm.mu);
95
96     let qj = hm * lm.sigma * hm.transpose() + self.q;
97     let qj_inv = qj.try_inverse().unwrap();
98     let nu = Vector2::new(z.r - r_hat, wrap_angle(z.phi - phi_hat));
99
100    // Sigma_x = [Hx^T Qj-1 Hx + R-1]-1 -- information matrices add.
101    let sigma_x = (hx.transpose() * qj_inv * hx + r.try_inverse().unwrap())
102        .try_inverse()
103        .expect("R is positive definite, so the sum is too");
104    let mu_x = Vector3::from(x_hat) + sigma_x * hx.transpose() * qj_inv * nu;
105
106    let l = sigma_x.cholesky().expect("Sigma_x ? 0").l();
107    let noise: Vector3<f64> =
108        Vector3::from_fn(|_, _| StandardNormal.sample(&mut self.rng));
109    let sample = SE2::from_vector(mu_x + l * noise);
110
111    // The weight uses the *predictive* covariance and is evaluated at the
112    // motion mean -- so it does not depend on where the sample landed.
113    let s = hx * r * hx.transpose() + qj;
114    (sample, log_mvn_pdf(&nu, &s))
115 }
116 }

```

The grid variant needs one more idea. Resampling clones particles, most clones die before they ever write a cell, and copying a 40×30 grid per clone per step is pure waste. FastSLAM’s classic answer was a shared balanced tree over landmarks; the idiomatic Rust answer is Arc plus make_mut, which gives the same asymptotic win with none of the pointer bookkeeping.

RUST crates/ch17-fastslam/src/grid_rbpf.rs

```

1 use std::sync::Arc;
2 use pr_core::mapping::OccGrid;
3
4 pub struct GridParticle {
5     pub pose: SE2,
6     pub path: Vec<SE2>,
7     /// Shared until written. `Arc::make_mut` performs the deep copy on the
8     /// first mutation after a clone -- and only if someone else is still
9     /// reading, which after a resample is usually nobody.
10    pub map: Arc<OccGrid>,
11    pub log_w: f64,
12 }
13
14 impl Clone for GridParticle {
15     fn clone(&self) -> Self {
16         // 0(1). The cells are not touched.
17         Self { pose: self.pose, path: self.path.clone(), map: Arc::clone(&self.map),
18             log_w: 0.0 }
19     }
20 }

```

```

21 pub struct GridRbpf {
22     particles: Vec<GridParticle>,
23     /// K in the improved proposal: how many poses to probe around the mode.
24     k_samples: usize,
25     /// Resample below this fraction of M. Grisetti et al. use 1/2.
26     neff_threshold: f64,
27     alphas: [f64; 4],
28     angles: Vec<f64>,
29 }
30
31 impl GridRbpf {
32     /// One `mapping_step`. On native builds the per-particle loop is a rayon
33     /// `par_iter_mut`; on WASM it is the same code, single-threaded, because
34     /// every particle is independent until the weights are normalized.
35     pub fn step(&mut self, u: &OdomDelta, scan: &[f64], rng: &mut Pcg64) -> RbpfReport {
36         for p in &mut self.particles {
37             let predicted = p.pose.apply_odom(u);
38             let matched = scan_match(&p.map, &predicted, scan, &self.angles);
39
40             let (pose, increment) = if matched.score - matched.seed_score >
SCAN_MATCH_GAIN {
41                 self.improved_proposal(p, u, matched.pose, scan, rng)
42             } else {
43                 // Flat score surface: the map has nothing to say here yet, and
44                 // trusting the "optimum" would be trusting numerical noise.
45                 let x = sample_motion_model_odometry(u, &p.pose, &self.alphas, rng);
46                 let w = log_map_match_score(&p.map, &x, scan, &self.angles);
47                 (x, w)
48             };
49
50             p.pose = pose;
51             p.path.push(pose);
52             // Tempered: beams are not independent (Chapter 10), so the raw
53             // product over-counts the evidence badly enough to collapse the
54             // population on the first resample.
55             p.log_w += WEIGHT_TEMPER * increment;
56
57             // The copy happens here, or not at all.
58             Arc::make_mut(&mut p.map).integrate_scan(&pose, scan, &self.angles);
59         }
60
61         let weights = normalize_log_weights(&self.particles);
62         let neff = effective_sample_size(&weights);
63         let resampled = neff < self.neff_threshold * self.particles.len() as f64;
64         if resampled {
65             // Clones are O(1) here; the deep copies happen lazily, above.
66             self.particles = low_variance_sampler(&self.particles, &weights, rng);
67         }
68         RbpfReport { neff, resampled }
69     }
70 }
71
72 /// N_eff = 1 / Sigma w^2. M when the weights are uniform, 1 when a single particle
73 /// carries everything.
74 pub fn effective_sample_size(w: &[f64]) -> f64 {
75     let s: f64 = w.iter().map(|x| x * x).sum();
76     if s > 0.0 { 1.0 / s } else { 0.0 }
77 }

```

NOTE

The widgets on this page run the TypeScript port of exactly these algorithms — `lib/filters/fastslam.ts` and `lib/filters/rbpf.ts`. The scan matcher in `w17.1` is the same greedy hill climb; the proposal in `w17.3` is the same information-form product; the copy-on-write grid is the same recount trick with Arc spelled out by hand.

17.5.1 A worked example you can check by hand

Two particles, one landmark, one sighting. Everything below is arithmetic you can do on paper, and the numbers are pinned by the test that follows.

Particle i sits at the origin facing along $+x$. Its EKF for landmark j says $\mu_j = (4, 0)$ with $\Sigma_j = \text{diag}(0.21, 0.12)$. The sensor has $\mathbf{Q} = \text{diag}(0.2^2, 0.05^2) = \text{diag}(0.04, 0.0025)$.

Step 1 — the Jacobian. With $\delta = (4, 0)$, $r = 4$ and $q = 16$:

$$\mathbf{H}_m = \begin{bmatrix} \delta_x/r & \delta_y/r \\ -\delta_y/q & \delta_x/q \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0.25 \end{bmatrix}$$

Step 2 — the innovation covariance. $\mathbf{H}_m \Sigma_j \mathbf{H}_m^\top = \text{diag}(0.21, 0.0625 \cdot 0.12) = \text{diag}(0.21, 0.0075)$, so

$$\mathbf{S} = \mathbf{H}_m \Sigma_j \mathbf{H}_m^\top + \mathbf{Q} = \text{diag}(0.25, 0.01)$$

Bearing uncertainty is dominated by the sensor, range uncertainty by the map — read straight off the two terms.

Step 3 — the normalizer. $|2\pi\mathbf{S}|^{1/2} = 2\pi\sqrt{0.25 \cdot 0.01} = 2\pi(0.05) = 0.31416$, so the leading factor is $1/0.31416 = 3.1831$.

Step 4 — the weights. Particle 1 gets $z = (4.1, 0.05)$, hence $\mathbf{v}_1 = (0.1, 0.05)$:

$$\mathbf{v}_1^\top \mathbf{S}^{-1} \mathbf{v}_1 = \frac{0.01}{0.25} + \frac{0.0025}{0.01} = 0.04 + 0.25 = 0.29 \quad \implies \quad w_1 = 3.1831 e^{-0.145} = 2.7534$$

Particle 2 gets $z = (4.3, 0.15)$, hence $\mathbf{v}_2 = (0.3, 0.15)$ — exactly three times as wrong:

$$\mathbf{v}_2^\top \mathbf{S}^{-1} \mathbf{v}_2 = 0.36 + 2.25 = 2.61 \quad \implies \quad w_2 = 3.1831 e^{-1.305} = 0.8632$$

Step 5 — normalize, and check the resampling rule. $\tilde{w} = (0.7613, 0.2387)$, so

$$N_{\text{eff}} = \frac{1}{0.7613^2 + 0.2387^2} = \frac{1}{0.6367} = 1.571$$

Three lessons in one number. Tripling the innovation cost particle 2 a factor of 3.19 in weight, not a factor of 3 — the exponential is unforgiving, and it is unforgiving in the *bearing* channel (which contributed 2.25 of the 2.61) because that is where $\mathbf{S}$ is small. And with $N_{\text{eff}} = 1.571 > M/2 = 1$, gmapping’s rule says **do not resample**: a 3:1 weight ratio between two particles is not yet degeneracy.

RUST crates/ch17_fastslam/src/landmark_ekf.rs (tests)

```

1  #[cfg(test)]
2  mod tests {
3      use super::*;
4      use approx::assert_relative_eq;
5
6      #[test]
7      fn worked_example_ch17_two_universes() {
8          let q = Matrix2::new(0.04, 0.0, 0.0, 0.0025);
9          let lm = LandmarkEkf {
10              mu: Vector2::new(4.0, 0.0),
11              sigma: Matrix2::new(0.21, 0.0, 0.0, 0.12),
12              hits: 1,
13          };
14          let x = SE2::new(0.0, 0.0, 0.0);
15
16          let (_, s) = lm.innovation(&x, Feature { r: 4.1, phi: 0.05 }, &q);
17          assert_relative_eq!(s[(0, 0)], 0.25, epsilon = 1e-12);
18          assert_relative_eq!(s[(1, 1)], 0.01, epsilon = 1e-12);
19          assert_relative_eq!(s[(0, 1)], 0.0, epsilon = 1e-12);
20
21          let w1 = lm.weight_1_0(&x, Feature { r: 4.1, phi: 0.05 }, &q);
22          let w2 = lm.weight_1_0(&x, Feature { r: 4.3, phi: 0.15 }, &q);
23          assert_relative_eq!(w1, 2.753_451_476_665, epsilon = 1e-9);
24          assert_relative_eq!(w2, 0.863_168_987_665, epsilon = 1e-9);
25
26          // Two particles, a 3.19:1 weight ratio -- and still above the M/2 line.
27          let total = w1 + w2;
28          let normalized = [w1 / total, w2 / total];
29          assert_relative_eq!(normalized[0], 0.761_332_714_843, epsilon = 1e-9);
30          assert_relative_eq!(effective_sample_size(&normalized), 1.570_870_837_6, epsilon
31      = 1e-9);
32          assert!(effective_sample_size(&normalized) > normalized.len() as f64 / 2.0);
33      }

```

The TypeScript port reproduces the same five digits, from the same `landmarkInnovation` the widgets call. If the prose and the code ever disagree, the test settles it.

17.6 Putting it together

17.6.1 What the numbers say

Here is the Apartment lap from w17.1 — out along the corridor, turn, and back over ground already mapped — run to completion under both proposals, at five particle counts, on seeds 3, 42, 11 and 7. Numbers are the RMS position error of

the highest-weight particle over the whole lap, in metres. Raw odometry on the same log has an RMS error of **3.37 m**.

M	measurement-aware (median / worst)	motion prior (median / worst)
4	0.36 / 1.06	1.22 / 2.27
8	0.20 / 0.73	0.79 / 1.34
16	0.18 / 0.26	0.25 / 0.30
24	0.16 / 0.25	0.33 / 0.56
40	0.17 / 3.89	0.26 / 0.44

Three readings, in order of how much they matter.

The proposal is worth particles. On the median, eight measurement-aware particles match sixteen motion-prior ones, and four of them beat eight. This is Grisetti et al.’s headline claim reproduced in a browser: the reason gmapping could map a building with thirty particles when FastSLAM 1.0 needed hundreds is not a better resampler, it is that the scan matcher tells each particle roughly where it is *before* the particle has to guess. Note the median-versus-worst columns, though — at small M a good proposal lowers the typical error without doing much for the tail, because the tail is dominated by runs where every particle was wrong together.

More particles is not monotone. At $M = 40$ one of the four seeds diverged to 3.9 m. Nothing about a large M prevents every particle from being descended from one universe that mapped a doorway in the wrong place; once the map is wrong and self-consistent, the scan matcher agrees with it enthusiastically. Particle count buys coverage of the *proposal’s* uncertainty, not insurance against a confidently wrong map.

The floor is the grid. Every configuration plateaus near 0.16 m, which is about half a cell. That is not a coincidence and not a bug: the map-match score cannot distinguish poses that put the scan endpoints in the same cells. If you want centimetres, you need Chapter 16’s continuous correspondences, not more universes.

17.6.2 The road not taken: EM mapping

⚠ WHERE ROBOTS BREAK THIS

A historical aside. The 1999–2000 draft that this book modernizes does not contain FastSLAM — it had not been invented. Its Chapter 13 solved the same problem a different way: **EM mapping**. The E-step ran forward–backward smoothing over the robot’s poses given a map; the M-step chose the maximum-likelihood map given those smoothed poses; iterate to convergence.

It was the first system to treat correspondence as a genuine latent variable rather than a decision, and that idea outlived it. But it was offline, it needed several passes over the whole log, and it converged to local optima with no way to tell you it had. Online Rao-Blackwellization beat it in the one dimension robots care about — you can run it while the robot is driving. Its E-step, though, came back: smoothing over a whole trajectory is exactly what Chapter 15 does, only with sparse least squares instead of forward-backward.

17.6.3 The verdict, in 2026

It would be dishonest to end this chapter without saying what happened next.

gmapping lost. In production 2-D SLAM the default is a pose graph — Chapter 16’s architecture, shipped as `slam_toolbox` and its relatives. The reasons are not subtle. A pose graph stores one map and $O(T)$ poses; an RBPF stores M maps, and while copy-on-write hides much of that, a large building at 5 cm resolution times thirty particles is real memory. A pose graph can revisit and *correct* an old decision when a loop closes; an RBPF can only fail to have deleted the universe that was right. And a pose graph gives you lifelong mapping — load yesterday’s graph, add today’s — where a particle filter’s ancestry is gone the moment it is resampled away.

Rao-Blackwellization won. The theorem is permanent, and it is everywhere: sample the part that is genuinely nonlinear, integrate the rest. Chapter 18 marginalizes landmarks out of a visual-inertial window rather than sampling them. Chapter 22 builds POMCP on particles that carry world states because the value function given a state is computable. Chapter 23 samples control sequences and integrates the dynamics analytically along each one. Every one of those is the same inequality from Derivation 2.

Per-particle data association survived too, and it is still the cleanest answer to genuine ambiguity. A corridor of identical doors, a symmetric building, a warehouse of identical racks: a single-hypothesis front end must guess, and a pose graph with a wrong loop closure produces a map that is confidently, catastrophically folded. Modern systems fight this with robust kernels and switchable constraints, which are ways of *softening* a commitment. FastSLAM never had to commit. That is why multi-hypothesis loop closing keeps being reinvented, and why the ideas in this chapter are worth knowing even though you will probably ship the pose graph.

17.7 Exercises

1 FOUNDATION •• Prove the factorization, and break it

Reproduce the induction of Derivation 1 with every normalizer tracked explicitly (the sketch quietly absorbs one). Then construct a measurement model that observes *two* landmarks in a single reading — say a stereo pair whose baseline is known — and show exactly which line of the proof fails and what the map posterior factors into instead. Finally: a laser scan observes hundreds of grid cells at once. What is this chapter's grid-based filter implicitly assuming, and which chapter already told you that assumption is false?

2 FOUNDATION •• The variance you did not have to pay

Starting from the law of total variance, show that the Rao-Blackwellized estimator's variance deficit is exactly $\mathbb{E}[\text{Var}[\varphi \mid X]]$. Then estimate that quantity for landmark SLAM: if each of N landmarks has posterior covariance Σ_j and you sampled them instead of integrating them, roughly how does the weight variance scale with N ? Use your answer to explain why the naive $(3 + 2N)$ -dimensional particle filter of the opening section fails at $N = 24$ rather than at $N = 2$.

3 FOUNDATION ••• Derive the 2.0 proposal for range-bearing

Fill in every step of Derivation 4 for the range-bearing model: write $\mathbf{H}_x$ explicitly, complete the square in x_t , and obtain (μ_x, Σ_x) . Then answer two questions the algebra makes easy. (a) What happens to Σ_x as $\mathbf{Q} \rightarrow 0$ with a single landmark observed — is the pose fully determined, and if not, which direction is left free? (b) What happens when two landmarks are observed at once, and why does FastSLAM 2.0 as published only use one of them in the proposal?

4 CONCEPTUAL • Predict the depletion, then measure it

In w17.3, before touching anything, predict what happens to the "samples in 90%" counter for FastSLAM 1.0 when you halve σ_r — will it halve, stay flat, or something else? Now do it, and read the counter. Then find the value of σ_r at which 1.0's N_{eff}/M stops falling, and explain what is limiting it there. (Hint: what else is in $\mathbf{Q}_j$ besides $\mathbf{Q}$?)

5 CONCEPTUAL •• Find the smallest sufficient universe count

In w17.1, turn the measurement-aware proposal off. Find the smallest M that completes the lap with the tiles still recognizable as the Apartment, on at least four of five re-rolled seeds. Now turn the proposal back on and repeat. Report both numbers and the ratio. Then explain, using w17.2, why raising M further stops helping — and identify, from

the lineage gauge, the moment at which extra particles stopped being independent hypotheses.

6 PRACTICAL •• Per-particle data association in the corridor of identical doors

Implement `FastSLAM_unknown_correspondence` (the second algorithm box) against the landmark filter: Mahalanobis gating at $\chi^2_{2,0.99}$, maximum-likelihood association, and new-landmark creation below p_0 . Then build the experiment the chapter keeps promising: a corridor with six identical doors spaced identically, entered at an unknown offset. Show that the particle population holds multiple association hypotheses simultaneously, and that the Chapter 11 EKF on the same log commits to one and never recovers. Report the fraction of seeds each approach gets right.

7 PRACTICAL ••• Copy-on-write versus the classic tree

Replace `Arc::make_mut` with FastSLAM's original data structure: a balanced binary tree over landmarks, shared between particles, where an update creates $O(\log N)$ new nodes and shares the rest. Benchmark both against $M \in \{10, 100, 1000\}$ and $N \in \{50, 500, 5000\}$, on native and on WASM. Report where the constant factors cross, and how much of the difference is allocation rather than copying. Then argue whether the tree is worth it for the *grid* variant, where the shared object is 1200 cells rather than N small structs.

17.8 References

Montemerlo, M., Thrun, S., Koller, D., and Wegbreit, B. (2002). FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem *Proceedings of AAAI-02, Edmonton*, 593–598.

The paper this chapter's epigraph and its 1.0 weight derivation come from. Read it for the factorization argument and the shared-tree data structure that copy-on-write replaces here. [link](#)

Montemerlo, M., Thrun, S., Koller, D., and Wegbreit, B. (2003). FastSLAM 2.0: An Improved Particle Filtering Algorithm for Simultaneous Localization and Mapping that Provably Converges *Proceedings of IJCAI-03, Acapulco*, 1151–1156.

The measurement-aware proposal of Derivation 4, plus the convergence proof for the linear-Gaussian case that the title advertises. [link](#)

Doucet, A., de Freitas, N., Murphy, K., and Russell, S. (2000). Rao-Blackwellised Particle Filtering for Dynamic Bayesian Networks *Proceedings of the 16th Conference on Uncertainty in Artificial Intelligence (UAI)*, 176–183.

Where the architecture of this chapter comes from, two years before anyone applied it to SLAM. Section 3 has the marginalized-weight result quoted in Derivation 2. [link](#)

Doucet, A., Godsill, S., and Andrieu, C. (2000). On Sequential Monte Carlo Sampling Methods for Bayesian Filtering *Statistics and Computing* 10(3), 197–208.

The optimal-proposal theorem that makes FastSLAM 2.0 more than a heuristic, and the source of the N_{eff} resampling criterion used throughout. doi:10.1023/A:1008935410038

Grisetti, G., Stachniss, C., and Burgard, W. (2007). Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters *IEEE Transactions on Robotics* 23(1), 34–46.

gmapping. The six-step recipe in this chapter is this paper: scan-matched proposals, the K-sample Gaussian fit, and selective resampling on N_{eff} . Their parameter table is a good place to start when tuning your own. doi:10.1109/TR0.2006.889486

Kok, M., Solin, A., and Schö n, T. B. (2024). Rao-Blackwellized Particle Smoothing for Simultaneous Localization and Mapping *Data-Centric Engineering* 5, e15.

The modern continuation: the same factorization run as a smoother rather than a filter, for magnetic-field and radio maps. Shows the theorem is alive well outside laser SLAM. doi:10.1017/dce.2024.12

Macenski, S. and Jambrecic, I. (2021). SLAM Toolbox: SLAM for the Dynamic World *Journal of Open Source Software* 6(61), 2783.

What replaced gmapping in production 2-D SLAM, and the concrete reference for the verdict at the end of this chapter: pose graphs, lifelong mapping, serialized maps you can reload. doi:10.21105/joss.02783

Carlone, L., Kim, A., Barfoot, T. D., Cremers, D., and Dellaert, F. (eds.) (2026). SLAM Handbook: From Localization and Mapping to Spatial Intelligence *Cambridge University Press* (public pre-release, 2025).

The current state-of-the-field survey. Useful here for situating filtering-era SLAM against what the field actually builds now, and for the pointers into robust and multi-hypothesis loop closing. [link](#)

Visual and Visual-Inertial SLAM

MAPPING AND SLAM · Advanced · 60 min



Our first contribution is a preintegration theory that properly addresses the manifold structure of the rotation group.

Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza

On-Manifold Preintegration for Real-Time Visual-Inertial Odometry (2017)

Every sensor in this book so far has measured a *distance*. A wheel encoder counts a distance travelled; a LiDAR beam returns a distance to a wall. A camera measures something stranger and much less helpful: the *direction* to a point, with the distance divided away.

That single fact — depth is annihilated by the projection — organizes the entire chapter. It is why a camera needs two views before it can say anything metric, why monocular systems are blind to scale, why an IMU is such a perfect partner, and why the estimation problem, once you write the measurement model down honestly, turns out to be a factor graph you already know how to solve.

There is no new theory here. There is one new measurement model (the pinhole camera), one new motion sensor (the strapdown IMU), and one genuinely beautiful trick for making the second one affordable. Everything else is Chapter 3, Chapter 7, and Chapter 15 meeting a camera.

🕒 AFTER THIS CHAPTER YOU CAN

- Write the camera down as a generative measurement model, exactly as Chapter 10 wrote the LiDAR
- Derive the reprojection factor and both of its Jacobian blocks under the $\boxtimes$ retraction

- Recognize bundle adjustment as Chapter 15's sparse least squares with the points eliminated first
- Explain why a single camera cannot know scale, and what the IMU contributes that fixes it
- Derive IMU preintegration on $SO(3)$ and its $O(1)$ bias correction — the chapter's centerpiece
- Compare MSCKF-style filtering with sliding-window smoothing, and price the marginalization each one performs

ASSUMED

- $SO(3)$ and $SE(3)$: $\exp$, $\log$, the right Jacobian, and the $\boxplus$ / $\boxminus$ retraction (Chapter 3)
- Error-state filtering and why linearization points matter (Chapter 7)
- Sensor models as probability distributions, written before any algorithm (Chapter 10)
- Sparse nonlinear least squares, Schur elimination, and robust kernels (Chapter 15)
- EKF SLAM's marginalization and the overconfidence it produced (Chapter 14)

18.1 The photograph that cannot locate you

Point a calibrated camera at the Apartment and take one picture. A corner of the doorway lands at pixel (336, 232). Where is the camera?

The honest answer is that the photograph has told you almost nothing about position. It has told you a *direction*: the corner lies somewhere on the ray through the optical centre and that pixel. Slide the camera forward along that ray, and slide the corner with it, and the picture is byte-for-byte identical. A pixel is a bearing.

INTERACTIVE · w18.1

Reprojection Playground

A pixel is a bearing, not a position: depth is only ever inferred from parallax, and the baseline sets the exchange rate.

Run this simulation in the web edition: `/chapters/ch18-visual-slam #w18.1`

Watch the left-hand image pane while the purple estimate slides along the green ray. Camera 0's residual is not *small*; it is exactly zero, at every depth,

forever. The measurement is perfectly explained by an entire one-parameter family of 3-D points, and no amount of care in the front end will change that, because the information was destroyed by the division by Z before the pixel was ever written.

Camera 1 is the only witness that objects, and how loudly it objects is set by the **baseline**. Drag the baseline slider to 0.1 m and watch the same metre of depth error shrink from tens of pixels to a handful, while the reported σ_Z climbs. That number is not decoration: it is $\sigma_Z = Z^2 \sigma_{px} / (fb)$, and it is the whole economics of stereo in one expression. Depth error grows with the *square* of range and inversely with the baseline you can afford to carry.

💡 THE ONE SENTENCE VERSION

A camera measures bearings. Depth exists only in the disagreement between two bearings, so all of visual SLAM is the business of accumulating enough disagreement — and an IMU is what supplies the metre that a lone camera never had.

18.2 Building intuition: three things a camera cannot do

Before any algebra, three failure modes worth carrying through the chapter. Each is visible in the widget above, and each is a theorem in the next section.

It cannot see depth from one view. Already established. The consequence for estimation is that a fresh landmark has *no* prior at all along its ray; systems either delay initialization until parallax accumulates, or parameterize by inverse depth so that "infinitely far" is a finite number.

It cannot see scale, ever, if it is alone. Take the true reconstruction and multiply every camera centre and every point by two. Every ray is unchanged, every pixel is unchanged, every residual is unchanged. Monocular SLAM does not have a scale error; it has no scale at *all*, and reporting one would be a lie. This is a gauge freedom, not a noise problem, and no amount of data removes it.

It cannot triangulate under pure rotation. Spin the camera on the spot and the scene sweeps across the image dramatically — an enormous amount of pixel motion carrying exactly zero information about depth, because the two rays to any point are the same ray. Parallax, not motion, is the currency.

An IMU repairs the second one and helps with the first. It measures angular rate and specific force — that is, metres per second squared — so integrating it twice produces *metres*. Bolting a gyro and an accelerometer to the camera makes scale observable, makes roll and pitch observable (gravity is a permanent plumb line), and leaves exactly four directions unobservable forever: global position and heading. That count is worth remembering, because a system that reports a covariance for global yaw is a system that has fooled itself.

18.3 The mathematics

18.3.1 The camera as a probabilistic sensor

Chapter 10 established the discipline: write $p(z \mid x, m)$ first, and only then ask what algorithm can invert it. For a pinhole camera the model is one line.

$\mathbf{K}, (f_x, f_y, c_x, c_y)$	Camera intrinsics: focal lengths and principal point in pixels. Lens distortion exists and is calibrated away before any model here applies.
$\pi : \mathbb{R}^3 \rightarrow \mathbb{R}^2$	Perspective projection: divide by depth, scale by the focal lengths, add the principal point. Defined only in front of the camera.
$T_{cw} \in \text{SE}(3)$	World-to-camera pose. A world point becomes a camera point under it; the translation of its inverse is the camera centre.
$\mathbf{q}$	The calibrated bearing of a pixel — the inverse intrinsics applied to the homogeneous pixel, normalized to unit length.
$\mathbf{e}_{kj}$	Reprojection residual of point j in keyframe k , measured in pixels.
$\mathbf{E} = [\mathbf{t}] \times \mathbf{R}$	Essential matrix: the epipolar constraint between the two calibrated bearings of the same point.
$\mathbf{b} = (\mathbf{b}_g, \mathbf{b}_a)$	Gyroscope and accelerometer biases; slowly varying, modeled as random walks, and always part of the state.
$\Delta \mathbf{R}_{ij}, \Delta \mathbf{v}_{ij}, \Delta \mathbf{p}_{ij}$	Preintegrated rotation, velocity, and position deltas between keyframes i and j .
$\mathbf{g}$	Gravity in the world frame. It never appears inside the deltas — that is precisely the point of them.

The measurement equation, and everything else in this chapter is a consequence of it:

$$\mathbf{z}_{kj} = \pi(T_{cw,k} {}^w \mathbf{P}_j) + \delta_{kj}, \quad \delta_{kj} \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{px}}^2 \mathbf{I}_2)$$

Two remarks on the noise. First, σ_{px} of one to two pixels is a statement about the *detector*, not the optics, and it is the same number for a point at 1 m and a point at 50 m — which is exactly why depth uncertainty explodes with range while pixel uncertainty does not. Second, this Gaussian is a lie in the tails: a mismatched feature is not a large draw from $\mathcal{N}(0, \sigma^2)$, it is a draw from a different distribution entirely. Chapter 15’s robust kernels exist for that, and every deployed visual system uses one.

18.3.2 Derivation 1: the reprojection factor and its Jacobians

The MAP estimate over keyframe poses $\{T_{cw,k}\}$ and points $\{{}^w \mathbf{P}_j\}$ minimizes the negative log-likelihood of every observation. With Gaussian pixel noise that is a sum of squares:

$$\{T^*, \mathbf{P}^*\} = \arg \min \sum_{(k,j) \in \mathcal{O}} \frac{1}{2} \underbrace{\| \mathbf{z}_{kj} - \pi(T_{cw,k} {}^w \mathbf{P}_j) \|_{\mathbf{Q}_{px}^{-1}}^2}_{\mathbf{e}_{kj}}$$

That is bundle adjustment, and it is a factor graph: one variable per keyframe, one per point, one binary factor per observation. Chapter 15's Gauss–Newton needs exactly two Jacobian blocks per factor, and both come from the chain rule.

DERIVATION The 2×3 and 2×6 blocks of a reprojection factor

$$\partial \pi / \partial \xi = \mathbf{J}_\pi \mathbf{R}_{cw} (\mathbf{I} \mid -[{}^w \mathbf{P}]_\times)$$

Step 1 — differentiate the projection itself. With ${}^c \mathbf{P} = (X, Y, Z)^\top$,

$$\mathbf{J}_\pi = \frac{\partial \pi}{\partial {}^c \mathbf{P}} = \begin{pmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \end{pmatrix}$$

Everything painful about cameras lives in this matrix. Its overall scale is f/Z , so a distant point barely moves; its third column carries the $1/Z^2$ that blows up as a point approaches the image plane, which is why every real system gates the near frustum before the optimizer can divide by something close to zero.

Step 2 — perturb the pose on the manifold, not in coordinates. The book's retraction is the right one, $T \boxplus \xi = T \exp(\xi^\wedge)$ with $\xi = (\rho, \phi)$, translation first. To first order, $\exp(\xi^\wedge) \mathbf{P} \approx \mathbf{P} + \rho + \phi \times \mathbf{P} = \mathbf{P} + \rho - [\mathbf{P}]_\times \phi$, so

$${}^c \mathbf{P}(\xi) = T_{cw} \exp(\xi^\wedge) {}^w \mathbf{P} \approx {}^c \mathbf{P} + \mathbf{R}_{cw} \rho - \mathbf{R}_{cw} [{}^w \mathbf{P}]_\times \phi$$

Step 3 — compose. Multiplying by $\mathbf{J}_\pi$ gives the 2×6 pose block and, since ${}^c \mathbf{P} = \mathbf{R}_{cw} {}^w \mathbf{P} + \mathbf{t}$, the 2×3 point block:

$$\frac{\partial \pi}{\partial \xi} = \mathbf{J}_\pi \mathbf{R}_{cw} \begin{pmatrix} \mathbf{I}_3 & -[{}^w \mathbf{P}]_\times \end{pmatrix}, \quad \frac{\partial \pi}{\partial {}^w \mathbf{P}} = \mathbf{J}_\pi \mathbf{R}_{cw}$$

The residual is $\mathbf{e} = \mathbf{z} - \pi$, so its Jacobians are these with the sign flipped — which cancels in $\mathbf{J}^\top \mathbf{J}$ and survives in $\mathbf{J}^\top \mathbf{e}$.

Step 4 — read the numbers. Take $f_x = f_y = 400$, $(c_x, c_y) = (320, 240)$, a camera at the origin looking down $+Z$, and the point ${}^w\mathbf{P} = (0.2, -0.1, 5)$. Then $f/Z = 80$ and

$$\mathbf{J}_\pi = \begin{pmatrix} 80 & 0 & -3.2 \\ 0 & 80 & 1.6 \end{pmatrix}, \quad \frac{\partial \pi}{\partial \xi} = \begin{pmatrix} 80 & 0 & -3.2 & 0.32 & 400.64 & 8.00 \\ 0 & 80 & 1.6 & -400.16 & -0.32 & 16.00 \end{pmatrix}$$

Every entry is checkable by hand from Step 3, and the comparison to make is between the third column and the fifth. Rotating the camera by one milliradian about its y axis moves this pixel by 0.40 px; translating it one *centimetre along its own optical axis* moves the same pixel by 0.03 px. A camera is a superb angle sensor and a hopeless rangefinder, and that ratio — 125:1 at 5 m — grows linearly with depth.

Conventions. Some texts perturb on the left, $\exp(\xi^\wedge)T$, which replaces $[{}^w\mathbf{P}]_\times$ with $[{}^c\mathbf{P}]_\times$ and changes nothing about the optimum. Chapter 3 fixes the book's convention; mixing the two is the single most common source of a bundle adjuster that converges to the wrong thing at a quarter of the expected rate.

18.3.3 Derivation 2: bundle adjustment eliminates the points first

Stack the unknowns as poses ξ then points $\mathbf{p}$. Because no factor ever touches two points, the normal equations have an arrowhead:

$$\begin{pmatrix} \mathbf{H}_{pp} & \mathbf{H}_{pl} \\ \mathbf{H}_{pl}^\top & \mathbf{H}_{ll} \end{pmatrix} \begin{pmatrix} \Delta \xi \\ \Delta \mathbf{p} \end{pmatrix} = \begin{pmatrix} \mathbf{g}_p \\ \mathbf{g}_l \end{pmatrix}, \quad \mathbf{H}_{ll} = \text{diag}(\mathbf{H}_1, \dots, \mathbf{H}_L)$$

$\mathbf{H}_{ll}$ is block diagonal with 3×3 blocks, so inverting it costs L tiny inverses. Eliminating the points is Chapter 15's variable elimination with a favourable ordering, and it has a name of its own in the vision literature: the **Schur complement trick**.

DERIVATION Schur elimination, and where the fill-in goes

$$\mathbf{S} = \mathbf{H}_{pp} - \mathbf{H}_{pl}\mathbf{H}_{ll}^{-1}\mathbf{H}_{pl}^\top$$

Step 1 — solve the second block row for the points. From $\mathbf{H}_{pl}^\top \Delta \xi + \mathbf{H}_{ll} \Delta \mathbf{p} = \mathbf{g}_l$,

$$\Delta \mathbf{p} = \mathbf{H}_{ll}^{-1}(\mathbf{g}_l - \mathbf{H}_{pl}^\top \Delta \xi)$$

Step 2 — substitute into the first. This gives the *reduced camera system*

$$\underbrace{(\mathbf{H}_{pp} - \mathbf{H}_{pl}\mathbf{H}_{ll}^{-1}\mathbf{H}_{pl}^T)}_{\mathbf{S}} \Delta\boldsymbol{\xi} = \mathbf{g}_p - \mathbf{H}_{pl}\mathbf{H}_{ll}^{-1}\mathbf{g}_l$$

whose size is $6K$ — six per keyframe — no matter how many million points the map holds.

Step 3 — ask what $\mathbf{S}$ looks like. The correction term is a sum over points, and point j contributes only to the pose pairs (k, k') that both observed it. So the sparsity pattern of the reduced system is exactly the **covisibility graph**: two keyframes are coupled if and only if they saw a common landmark. That is where the map's information went — not deleted, relocated.

Step 4 — count. The chapter's worked scene has 8 cameras on a ring, 40 points, and 320 observations. The full system is 162×162 ; with camera 0 pinned as the gauge, the reduced system is 42×42 — and the cost of forming it is linear in the number of points. That ratio is why bundle adjustment scales to city blocks, and it is literally the same reduce step the information filter of Chapter 14 performed, with 3×3 blocks instead of 2×2 .

NOTE

Notice what has just happened. The "map" in visual SLAM is not privileged: points are *eliminated* because they are cheap to eliminate, and the surviving problem is over poses. The same algebra, applied to old poses instead of points, is the marginalization that Derivation 5 will make you pay for.

18.3.4 Derivation 3: two views in ninety seconds

Everything metric a camera pair can say comes from one geometric fact: the two rays and the baseline joining the two camera centres are coplanar.

 **DERIVATION** Epipolar constraint, triangulation, and the scale you cannot see

$$\mathbf{q}_2^T[t] \times \mathbf{R} \mathbf{q}_1 = 0$$

Step 1 — coplanarity. Let camera 2 be at $(\mathbf{R}, \mathbf{t})$ relative to camera 1, and let $\mathbf{q}_1, \mathbf{q}_2$ be the calibrated bearings of the same point. Expressed in camera 2's frame, the three vectors $\mathbf{q}_2, \mathbf{R}\mathbf{q}_1$ and $\mathbf{t}$ lie in one plane, so their triple product vanishes:

$$\mathbf{q}_2 \cdot (\mathbf{t} \times \mathbf{R}\mathbf{q}_1) = 0 \quad \Longleftrightarrow \quad \underbrace{\mathbf{q}_2^T [\mathbf{t}]_{\times} \mathbf{R}}_{\mathbf{E}} \mathbf{q}_1 = 0$$

Step 2 — count degrees of freedom. $\mathbf{E}$ has 5: three for $\mathbf{R}$, two for the *direction* of $\mathbf{t}$. Its magnitude is absent from the constraint, because scaling $\mathbf{t}$ scales the whole equation. Five constraints from five point correspondences determine it — the five-point algorithm, which we name and do not derive.

Step 3 — the gauge. Take any reconstruction and map $\mathbf{C}_k \mapsto s\mathbf{C}_k$, ${}^w\mathbf{P}_j \mapsto s\,{}^w\mathbf{P}_j$, leaving rotations alone. Then ${}^c\mathbf{P} = \mathbf{R}({}^w\mathbf{P} - \mathbf{C}) \mapsto s\,{}^c\mathbf{P}$, and since $\pi(s\,{}^c\mathbf{P}) = \pi({}^c\mathbf{P})$ — the s cancels in X/Z — *every residual is exactly invariant*. Scale is a null direction of the Hessian, not a poorly determined one. Fixing it needs information from outside the images: a known baseline (stereo), a known object, or an accelerometer.

Step 4 — triangulate, and propagate the noise. Two rays over-determine a point; the linear (DLT) solution stacks the two cross-product constraints per view and solves a 4×3 least-squares system. For the classic fronto-parallel pair with baseline b , disparity is $d = fb/Z$, so $|dZ/dd| = Z^2/(fb)$ and

$$\sigma_Z = \frac{Z^2 \sigma_{\text{px}}}{f b}$$

Step 5 — degeneracy. If the motion is a pure rotation then $\mathbf{t} = \mathbf{0}$, $\mathbf{E}$ vanishes identically, the two camera centres coincide, and the parallax angle at every point is zero. No depth, at any noise level, from any number of frames. Modern monocular systems test the parallax angle before initializing a landmark for exactly this reason.

The worked example, by hand. Camera intrinsics $f_x = f_y = 400$, $(c_x, c_y) = (320, 240)$; camera 0 at the origin, camera 1's centre at $(0.5, 0, 0)$, both looking down $+Z$; a point at ${}^w\mathbf{P} = (0.2, -0.1, 5)$.

- Camera 0 sees it at $(400 \cdot 0.2/5 + 320, 400 \cdot (-0.1)/5 + 240) = (336, 232)$.
- Camera 1 sees the point at ${}^c\mathbf{P} = (-0.3, -0.1, 5)$, so at $(296, 232)$. Disparity: 40 px.
- Stretch the point 10% along camera 0's ray, to $(0.22, -0.11, 5.5)$. Camera 0's pixel does not move *at all* — the coordinates scaled together and the ratio

is unchanged. Camera 1's moves to $400 \cdot (-0.28)/5.5 + 320 = 299.636$, that is $40/11 = 3.636$ px.

- One pixel of matching noise therefore buys $\sigma_Z = 5^2 \cdot 1/(400 \cdot 0.5) = 0.125$ m of depth uncertainty. Move the point to 10 m and it is 0.5 m; shrink the baseline to 10 cm and it is 0.625 m.

Those five numbers are the reason this chapter exists, and each of them is pinned by a test in `lib/vision/__checks_ch18__.ts` and reproduced by the Rust below.

18.3.5 Derivation 4: IMU preintegration on the manifold

Now the second sensor. A strapdown IMU reports, in the body frame,

$$\tilde{\boldsymbol{\omega}}_k = \boldsymbol{\omega}_k + \mathbf{b}_g + \boldsymbol{\eta}_g, \quad \tilde{\mathbf{a}}_k = \mathbf{R}_k^\top (\mathbf{a}_k - \mathbf{g}) + \mathbf{b}_a + \boldsymbol{\eta}_a$$

Read that second equation carefully, because it contains the chapter's other big misconception. An accelerometer does not measure acceleration; it measures **specific force**, which is acceleration *minus gravity*, rotated into the body frame. An IMU sitting still on a table reads 9.81 m/s^2 upward. Nothing about position appears anywhere, and nothing about attitude appears in the gyro: both are things you get only by integrating, and integrating is where the trouble starts.

The trouble is not accuracy, it is *cost*. Put the raw samples into a factor graph naively and each of them becomes a factor between consecutive states — thousands of variables per second. Put them into a single factor between keyframes and the integral starts from $\mathbf{R}_i, \mathbf{v}_i, \mathbf{p}_i$, which the optimizer is currently changing, so every iteration re-runs the whole integration. Lupton and Sukkarieh's insight, made rigorous on $\text{SO}(3)$ by Forster et al., is that the dependence on the initial state can be factored out completely.

DERIVATION Preintegrated deltas, their covariance, and the O(1) bias update

$$\Delta \mathbf{R}_{ij} = \prod \exp((\tilde{\boldsymbol{\omega}}_k - \mathbf{b}_g)^\wedge \Delta t)$$

Step 1 — write the strapdown integration, one sample at a time. With step Δt :

$$\begin{aligned} \mathbf{R}_{k+1} &= \mathbf{R}_k \exp((\tilde{\boldsymbol{\omega}}_k - \mathbf{b}_g - \boldsymbol{\eta}_g)^\wedge \Delta t) \\ \mathbf{v}_{k+1} &= \mathbf{v}_k + \mathbf{g} \Delta t + \mathbf{R}_k(\tilde{\mathbf{a}}_k - \mathbf{b}_a - \boldsymbol{\eta}_a) \Delta t \\ \mathbf{p}_{k+1} &= \mathbf{p}_k + \mathbf{v}_k \Delta t + \frac{1}{2} \mathbf{g} \Delta t^2 + \frac{1}{2} \mathbf{R}_k(\tilde{\mathbf{a}}_k - \mathbf{b}_a - \boldsymbol{\eta}_a) \Delta t^2 \end{aligned}$$

Unrolling from i to j gives $\mathbf{R}_j$ as a product of exponentials and $\mathbf{v}_j, \mathbf{p}_j$ as sums in which *every* term carries an $\mathbf{R}_k$ — and every $\mathbf{R}_k$ carries $\mathbf{R}_i$.

Step 2 — move the initial state to the left. Define the three deltas by *solving* those equations for the measurement-dependent part:

$$\begin{aligned}\Delta \mathbf{R}_{ij} &\equiv \mathbf{R}_i^\top \mathbf{R}_j &= \prod \exp((\tilde{\boldsymbol{\omega}}_k - \mathbf{b}_g)^\wedge \Delta t) \\ \Delta \mathbf{v}_{ij} &\equiv \mathbf{R}_i^\top (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g} \Delta t_{ij}) &= \sum \Delta \mathbf{R}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_a) \Delta t \\ \Delta \mathbf{p}_{ij} &\equiv \mathbf{R}_i^\top (\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} \mathbf{g} \Delta t_{ij}^2) &= \sum [\Delta \mathbf{v}_{ik} \Delta t + \frac{1}{2} \Delta \mathbf{R}_{ik} (\tilde{\mathbf{a}}_k - \mathbf{b}_a) \Delta t^2]\end{aligned}$$

The right-hand sides contain measurements and biases and *nothing else*. Gravity has moved to the left. That is the whole trick: compute the right-hand sides once, and the left-hand sides become a residual the optimizer can evaluate in constant time, for any states it currently believes in.

Step 3 — the residual. With $\mathbf{r} = (\mathbf{r}_{\Delta R}, \mathbf{r}_{\Delta v}, \mathbf{r}_{\Delta p})$,

$$\begin{aligned}\mathbf{r}_{\Delta R} &= \text{Log}(\Delta \tilde{\mathbf{R}}_{ij}^\top \mathbf{R}_i^\top \mathbf{R}_j) \\ \mathbf{r}_{\Delta v} &= \mathbf{R}_i^\top (\mathbf{v}_j - \mathbf{v}_i - \mathbf{g} \Delta t_{ij}) - \Delta \tilde{\mathbf{v}}_{ij} \\ \mathbf{r}_{\Delta p} &= \mathbf{R}_i^\top (\mathbf{p}_j - \mathbf{p}_i - \mathbf{v}_i \Delta t_{ij} - \frac{1}{2} \mathbf{g} \Delta t_{ij}^2) - \Delta \tilde{\mathbf{p}}_{ij}\end{aligned}$$

One nine-dimensional factor tying two navigation states. Purple is what the estimator believes; green is what the IMU reported.

Step 4 — propagate the covariance alongside. Linearizing the error $(\delta \phi, \delta \mathbf{v}, \delta \mathbf{p})$ through one step gives $\boldsymbol{\Sigma} \leftarrow \mathbf{A} \boldsymbol{\Sigma} \mathbf{A}^\top + \mathbf{B} \boldsymbol{\Sigma}_\eta \mathbf{B}^\top$ with

$$\mathbf{A} = \begin{pmatrix} \Delta \mathbf{R}_{k,k+1}^\top & \mathbf{0} & \mathbf{0} \\ -\Delta \mathbf{R}_{ik} [\tilde{\mathbf{a}}_k - \mathbf{b}_a]_\times \Delta t & \mathbf{I} & \mathbf{0} \\ -\frac{1}{2} \Delta \mathbf{R}_{ik} [\tilde{\mathbf{a}}_k - \mathbf{b}_a]_\times \Delta t^2 & \mathbf{I} \Delta t & \mathbf{I} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{J}_r^k \Delta t & \mathbf{0} \\ \mathbf{0} & \Delta \mathbf{R}_{ik} \Delta t \\ \mathbf{0} & \frac{1}{2} \Delta \mathbf{R}_{ik} \Delta t^2 \end{pmatrix}$$

$\mathbf{J}_r$ is the right Jacobian of SO(3) from Chapter 3, and it is not decoration: adding a small rotation increment to a rotation vector is not the same as composing rotations, and $\mathbf{J}_r$ is precisely the correction. The lower-left blocks of $\mathbf{A}$ are the leak: an attitude error tilts the integrated acceleration, so gyro noise pours into velocity and then into position. That is why σ_p grows like $\Delta t^{3/2}$ from accelerometer noise alone but heads toward $\Delta t^{5/2}$ once the gyro contributes.

Step 5 — absorb bias changes to first order. The optimizer will change its bias estimate, and re-integrating 200 samples every time it does would undo the whole saving. So accumulate $\partial \Delta \mathbf{R} / \partial \mathbf{b}_g$, $\partial \Delta \mathbf{v} / \partial \mathbf{b}_{g,a}$ and $\partial \Delta \mathbf{p} / \partial \mathbf{b}_{g,a}$ by the same recursion, and update analytically:

$$\Delta \mathbf{R}_{ij}(\mathbf{b}_g + \delta \mathbf{b}_g) \approx \Delta \mathbf{R}_{ij}(\mathbf{b}_g) \exp\left((\mathbf{J}_{\Delta R}^{b_g} \delta \mathbf{b}_g)^\wedge\right), \quad \Delta \mathbf{v} += \mathbf{J}_{\Delta v}^{b_g} \delta \mathbf{b}_g + \mathbf{J}_{\Delta v}^{b_a} \delta \mathbf{b}_a$$

Five small matrix products instead of an $O(n)$ integration. The approximation is first order in $\delta\mathbf{b}$, so systems re-linearize — that is, actually re-integrate — once the bias estimate has moved far enough.

A preintegration example you can check by hand. Feed the integrator a constant yaw rate: $\tilde{\omega}_z = 0.51$ rad/s for 2 s, at 200 Hz, integrated at $\mathbf{b}_g = \mathbf{0}$. Since the rotation axis never changes, the product telescopes and $\Delta\mathbf{R} = \mathbf{R}_z(0.51 \cdot 2) = \mathbf{R}_z(1.02 \text{ rad})$. The bias Jacobian for a fixed axis is $\mathbf{J}_{\Delta\mathbf{R}}^{b_g} = -\Delta t_{ij}\mathbf{I}$, so correcting to $b_{g,z} = 0.01$ rad/s gives

$$\Delta\mathbf{R} \exp((-0.01 \cdot 2) \mathbf{e}_z^\wedge) = \mathbf{R}_z(1.02 - 0.02) = \mathbf{R}_z(1.00 \text{ rad})$$

which is exactly what a full re-integration at $b_{g,z} = 0.01$ produces — to 10^{-9} rad in the implementation. On a general trajectory the agreement is not exact, only second order in $\delta\mathbf{b}$: over a one-second turn, a bias error of 0.1 rad/s (about $6^\circ/\text{s}$, an appallingly bad gyro) leaves the corrected $\Delta\mathbf{p}$ 0.2 mm away from the re-integrated one, and cutting the bias error by ten cuts that by a hundred.

INTERACTIVE · w18.2

Preintegration Timeline

Hundreds of IMU samples compress into one factor between two keyframes — and a change of bias re-shapes that factor in $O(1)$, without touching a single raw sample.

Run this simulation in the web edition: </chapters/ch18-visual-slam#w18.2>

Two things in that widget deserve a second look. First, the covariance panel: after one second at 200 Hz the factor's own uncertainty is 3.5 mrad in attitude, 2.6 cm/s in velocity, and 1.3 cm in position — and the off-diagonal blocks are large, because those three errors are not independent. An IMU factor is a strongly correlated nine-dimensional Gaussian, and treating its blocks separately throws away most of what it knows.

Second, the residual bars. The IMU has a real gyro bias of +0.05 rad/s that the factor knows nothing about, and the rotation residual sits at fourteen standard deviations until you dial the bias in. That is the honest picture of why bias must be *in the state*: it is not noise, it does not average away, and a filter that omits it is exactly the overconfident-and-wrong filter of Chapter 5's Markov discussion.

18.3.6 Derivation 5: two architectures, one posterior

You now have two measurement models and a factor graph. There remain two ways to keep the problem bounded as the robot keeps driving, and the difference between them is the difference between the two families of deployed VIO systems.

MSCKF: keep poses, never keep landmarks. Mourikis and Roumeliotis's

multi-state constraint Kalman filter maintains a sliding window of camera poses inside an error-state EKF (Chapter 7) and refuses to put landmarks in the state at all. When a feature track ends, stack its M residuals:

$$\mathbf{r} \approx \mathbf{H}_x \delta \mathbf{x} + \mathbf{H}_f \delta \mathbf{f} + \mathbf{n}, \quad \mathbf{H}_f \in \mathbb{R}^{2M \times 3}$$

Let $\mathbf{N}$ be a basis for the left null space of $\mathbf{H}_f$, so $\mathbf{N}^\top \mathbf{H}_f = \mathbf{0}$ (it has $2M - 3$ columns). Multiplying through,

$$\mathbf{N}^\top \mathbf{r} \approx \mathbf{N}^\top \mathbf{H}_x \delta \mathbf{x} + \mathbf{N}^\top \mathbf{n}$$

The landmark is gone — not estimated, not marginalized, *projected out* — and what remains is a constraint among the poses that saw it. The state stays a fixed size, the update is a standard EKF update, and the cost is linear in the track length. This is the same "eliminate the structure" move as the Schur complement of Derivation 2, performed with a QR instead of an inverse.

Sliding-window smoothing: keep everything recent, marginalize the rest. VINS-style systems keep the last few keyframes, their velocities and biases, and the live features, and solve the full nonlinear problem every frame. When a keyframe falls off the back, they marginalize it in information form:

$$\mathbf{\Omega}' = \mathbf{\Omega}_{\beta\beta} - \mathbf{\Omega}_{\beta\alpha} \mathbf{\Omega}_{\alpha\alpha}^{-1} \mathbf{\Omega}_{\alpha\beta}$$

which is, once again, the Schur complement — the same operation, now applied to states rather than points. It is exact. It is also not free, and the bill has two lines.

INTERACTIVE · w18.3

Marginalization Fill-In

Marginalization does not delete an old state — it redistributes that state's information onto everything it touched, and the prior densifies.

Run this simulation in the web edition: [/chapters/ch18-visual-slam #w18.3](#)

Line one: fill-in. Every pair of survivors that touched the departing variable becomes directly coupled. The prior densifies, monotonically, and there is no ordering that prevents it — the information genuinely *is* shared between them now. Watch the block count in the widget: exact marginalization only ever adds.

Line two: frozen linearization. The marginal prior is a quadratic built at the linearization point that was current when the variable left. Nothing can ever re-linearize it, because the variable it was linearized about is gone. This is Chapter 14's original sin, resurrected inside a smoother, and the standard defence is the

first-estimates Jacobian rule: evaluate the Jacobians of anything touching the prior at the estimates that were current when the prior was built, so the linearized system keeps the same unobservable subspace as the real one. Skip it and the estimator will manufacture information along global yaw (Huang, Mourikis & Roumeliotis 2010).

⚠ WHERE ROBOTS BREAK THIS

SEIF's ghost. The 1999–2000 draft of *Probabilistic Robotics* devoted a chapter to sparse extended information filters, whose central move was to *delete* weak links to keep the information matrix sparse. Turn on the widget's toggle and you are running that idea: sparsity comes back, and the oldest surviving keyframe's reported σ shrinks even though no measurement arrived to justify it. Modern practice answers the same question differently — keep the dense prior, but keep it small and freeze its linearization point — which is why this book teaches SEIF as a lesson rather than as an algorithm.

Which architecture wins? Neither, and the reason is instructive: they are approximations of the *same posterior* under different budgets. MSCKF is cheaper and degrades gracefully on weak hardware; sliding-window smoothing re-linearizes more and is usually more accurate when it can afford the iterations. The modern square-root formulations narrow the gap further by storing the marginal prior as a matrix square root, which survives single-precision arithmetic where the Hessian form does not (Demmel et al. 2021).

18.4 The algorithms

ALGORITHM `project_jacobians(K, T_cw, P_w)`

COST 0(1)

in intrinsics, world-to-camera pose, world point

out pixel z , pose block (2×6), point block (2×3)

- 1: ${}^c\mathbf{P} = T_{cw} {}^w\mathbf{P}$
- 2: if ${}^cP_z \leq z_{\min}$ then return $\emptyset$ (*the near-frustum gate*)
- 3: $\mathbf{z} = (f_x {}^cP_x / {}^cP_z + c_x, f_y {}^cP_y / {}^cP_z + c_y)^\top$
- 4: $\mathbf{J}_\pi = \begin{pmatrix} f_x/Z & 0 & -f_x X/Z^2 \\ 0 & f_y/Z & -f_y Y/Z^2 \end{pmatrix}$
- 5: $\mathbf{J}_{\text{point}} = \mathbf{J}_\pi \mathbf{R}_{cw}$
- 6: $\mathbf{J}_{\text{pose}} = (\mathbf{J}_{\text{point}} \mid -\mathbf{J}_{\text{point}} [{}^w\mathbf{P}]_\times)$
- 7: return $(\mathbf{z}, \mathbf{J}_{\text{pose}}, \mathbf{J}_{\text{point}})$

ALGORITHM `ba_solve(poses, points, observations)` COST per iteration: $O(F)$ to build, $O(L)$ to eliminate points, $O((6K)^3)$ to solve the reduced system
in initial poses and points, pixel observations, gauge
out MAP poses and points

```

1: repeat
2:    $\mathbf{H} \leftarrow \mathbf{0}$ ,  $\mathbf{g} \leftarrow \mathbf{0}$ 
3:   for each observation  $(k, j)$  do (accumulate;  $O(1)$  each)
4:      $\mathbf{e} = \mathbf{z}_{kj} - \pi(T_k \mathbf{P}_j)$ , weight  $w = \rho'(\|\mathbf{e}\|)$  (Huber, Ch. 15)
5:     add  $w \mathbf{J}^T \mathbf{J}$  into  $\mathbf{H}_{pp}$ ,  $\mathbf{H}_{pl}$ ,  $\mathbf{H}_{ll}$  and  $w \mathbf{J}^T \mathbf{e}$  into  $\mathbf{g}$ 
6:     damp:  $\mathbf{H}_{ii} += \lambda(1 + \mathbf{H}_{ii})$  (Levenberg–Marquardt)
7:      $\mathbf{S} = \mathbf{H}_{pp} - \mathbf{H}_{pl} \mathbf{H}_{ll}^{-1} \mathbf{H}_{pl}^T$  (Schur;  $L$  small inverses)
8:     solve  $\mathbf{S} \Delta \xi = \mathbf{g}_p - \mathbf{H}_{pl} \mathbf{H}_{ll}^{-1} \mathbf{g}_l$ 
9:      $\Delta \mathbf{p} = \mathbf{H}_{ll}^{-1} (\mathbf{g}_l - \mathbf{H}_{pl}^T \Delta \xi)$  (back-substitute)
10:     $T_k \leftarrow T_k \boxplus \Delta \xi_k$ ,  $\mathbf{P}_j \leftarrow \mathbf{P}_j + \Delta \mathbf{p}_j$ 
11: until  $\|\Delta\| < \epsilon$ 

```

ALGORITHM `preintegrate(samples[i..j], b)` COST $O(n)$ once per interval -- never again inside the optimizer
in IMU samples over one keyframe interval, the bias to linearize at
out $\Delta \mathbf{R}$, $\Delta \mathbf{v}$, $\Delta \mathbf{p}$, Σ (9×9), bias Jacobians

```

1:  $\Delta \mathbf{R} \leftarrow \mathbf{I}$ ,  $\Delta \mathbf{v} \leftarrow \mathbf{0}$ ,  $\Delta \mathbf{p} \leftarrow \mathbf{0}$ ,  $\Sigma \leftarrow \mathbf{0}$ ,  $\mathbf{J}_\bullet \leftarrow \mathbf{0}$ 
2: for each sample  $(\tilde{\omega}, \tilde{\mathbf{a}})$  do
3:    $\hat{\omega} = \tilde{\omega} - \mathbf{b}_g$ ,  $\hat{\mathbf{a}} = \tilde{\mathbf{a}} - \mathbf{b}_a$ 
4:    $\Sigma \leftarrow \mathbf{A} \Sigma \mathbf{A}^T + \mathbf{B} \Sigma_\eta \mathbf{B}^T$  (before the means:  $\mathbf{A}$  uses the old  $\Delta \mathbf{R}$ )
5:   update the bias Jacobians by the same recursion
6:    $\Delta \mathbf{p} += \Delta \mathbf{v} \Delta t + \frac{1}{2} \Delta \mathbf{R} \hat{\mathbf{a}} \Delta t^2$ 
7:    $\Delta \mathbf{v} += \Delta \mathbf{R} \hat{\mathbf{a}} \Delta t$ 
8:    $\Delta \mathbf{R} \leftarrow \Delta \mathbf{R} \exp(\hat{\omega}^\wedge \Delta t)$  (re-orthonormalize)
9: return the factor

```

ALGORITHM `marginalize( $\Omega$ ,  $\alpha$ )` COST $O(|\alpha|^3 + |\alpha||\beta|^2)$ -- and the fill-in is permanent
in window information matrix, the variables leaving
out a dense prior over the survivors β

- 1: partition $\Omega = \begin{pmatrix} \Omega_{\alpha\alpha} & \Omega_{\alpha\beta} \\ \Omega_{\beta\alpha} & \Omega_{\beta\beta} \end{pmatrix}$, $\xi = (\xi_\alpha, \xi_\beta)$
- 2: $\Omega' = \Omega_{\beta\beta} - \Omega_{\beta\alpha} \Omega_{\alpha\alpha}^{-1} \Omega_{\alpha\beta}$
- 3: $\xi' = \xi_\beta - \Omega_{\beta\alpha} \Omega_{\alpha\alpha}^{-1} \xi_\alpha$
- 4: freeze the linearization point of every variable in β that touches Ω' (FEJ)
- 5: return (Ω', ξ')

18.5 Implementation in Rust

This is deliberately the smallest Practical section in Part V. The book's robot is a 2-D LiDAR platform; the job here is to transfer the *probabilistic structure* of visual-inertial estimation, not to build a fourth SLAM system. Two artifacts: a pinhole camera with exact Jacobians, and a preintegrator. The optimizer is Chapter 15's, unchanged.

RUST crates/ch18_vio/src/pinhole.rs

```

1 use nalgebra::{Matrix2x3, Matrix2x6, Matrix3, Vector2, Vector3};
2 use sophus::lie::Isometry3F64; // SE(3): the Ch. 3 type, in 3-D
3
4 // A calibrated pinhole camera. Distortion is assumed already removed --
5 // every model in this chapter operates on rectified pixels.
6 #[derive(Clone, Copy, Debug)]
7 pub struct Pinhole {
8     pub fx: f64,
9     pub fy: f64,
10    pub cx: f64,
11    pub cy: f64,
12 }
13
14 // Points nearer than this are not projected. Not numerical fussiness: the
15 // Jacobian's 1/Z2 entries diverge, and one point drifting behind the camera
16 // during a step would otherwise produce an infinite gradient.
17 pub const Z_MIN: f64 = 0.05;
18
19 impl Pinhole {
20     /// pi(P) for a point already in the camera frame.
21     pub fn project_cam(&self, p: &Vector3<f64>) -> Option<Vector2<f64>> {
22         if p.z <= Z_MIN {
23             return None;
24         }
25         Some(Vector2::new(
26             self.fx * p.x / p.z + self.cx,
27             self.fy * p.y / p.z + self.cy,
28         ))
29     }
30
31     /// pi(T_cw . P_w).
32     pub fn project(&self, t_cw: &Isometry3F64, p_w: &Vector3<f64>) -> Option<Vector2<f64>> {

```

```

33     self.project_cam(&(t_cw * p_w))
34 }
35
36 /// ?pi/?P_c -- the 2*3 block with the famous 1/Z and 1/Z2 entries.
37 fn d_pi(&self, p_c: &Vector3<f64>) -> Matrix2x3<f64> {
38     let iz = 1.0 / p_c.z;
39     Matrix2x3::new(
40         self.fx * iz, 0.0, -self.fx * p_c.x * iz * iz,
41         0.0, self.fy * iz, -self.fy * p_c.y * iz * iz,
42     )
43 }
44
45 /// Both Jacobian blocks of the projection, for the right retraction
46 /// T [+] xi = T . exp(xi), xi = (rho, phi) with translation first (Derivation 1).
47 ///
48 /// The residual is z - pi, so a factor negates both of these; the sign
49 /// cancels in J^TJ and survives in J^Te.
50 pub fn jacobians(
51     &self,
52     t_cw: &Isometry3F64,
53     p_w: &Vector3<f64>,
54 ) -> Option<(Matrix2x6<f64>, Matrix2x3<f64>)> {
55     let p_c = t_cw * p_w;
56     if p_c.z <= Z_MIN {
57         return None;
58     }
59     let r_cw: Matrix3<f64> = t_cw.rotation.to_rotation_matrix().into_inner();
60     let j_point = self.d_pi(&p_c) * r_cw; /// ?pi/?P_w
61     let skew = skew3(p_w); /// [P_w]*
62     let mut j_pose = Matrix2x6::zeros();
63     j_pose.fixed_columns_mut::<3>(0).copy_from(&j_point);
64     j_pose.fixed_columns_mut::<3>(3).copy_from(&(-j_point * skew));
65     Some((j_pose, j_point))
66 }
67 }
68
69 fn skew3(v: &Vector3<f64>) -> Matrix3<f64> {
70     Matrix3::new(0.0, -v.z, v.y, v.z, 0.0, -v.x, -v.y, v.x, 0.0)
71 }
72
73 /// The 1sigma depth uncertainty of a fronto-parallel stereo pair, sigma_Z = Z2sigma/(f b
74 /// Quadratic in range, inverse in baseline -- the economics of stereo.
75 pub fn depth_sigma(cam: &Pinhole, baseline: f64, z: f64, sigma_px: f64) -> f64 {
76     z * z * sigma_px / (cam.fx * baseline)
77 }
78
79 /// A single reprojection factor: keyframe `kf` saw point `pt` at pixel `z`.
80 pub struct ReprojFactor {
81     pub kf: usize,
82     pub pt: usize,
83     pub z: Vector2<f64>,
84     /// The square-root information Sigma^{-1/2}. Pixel noise is isotropic, so one
85     /// scalar 1/sigma stands in for a 2*2 matrix -- and whitening here is what makes
86     /// the cost comparable across factors of different kinds.
87     pub inv_sigma: f64,
88 }

```

The bundle adjuster is thirty lines of bookkeeping around Chapter 15's solver.

The only part worth printing is the elimination, because that is where the structure of the problem shows up as code.

RUST crates/ch18_vio/src/tiny_ba.rs

```

1 use std::ops::SubAssign;
2
3 use nalgebra::{DMatrix, DVector, Matrix3, Matrix6, Matrix6x3, Vector3, Vector6};
4 use sophus::lie::Isometry3F64;
5
6 pub struct TinyBa {
7     pub poses: Vec<Isometry3F64>,
8     pub points: Vec<Vector3<f64>>,
9     pub factors: Vec<ReprojFactor>,
10    /// Reprojection error is blind to a rigid motion of the whole scene and,
11    /// monocular, to its scale. Something must be pinned: this is the gauge.
12    pub fixed: Vec<bool>,
13    pub sigma_px: f64,
14    pub huber: Option<f64>,
15 }
16
17 pub struct BaReport {
18     pub rmse_px: f64,
19     pub reduced_dim: usize,
20     pub full_dim: usize,
21 }
22
23 impl TinyBa {
24     /// One damped Gauss-Newton step with the points Schur-eliminated.
25     pub fn step(&mut self, lambda: f64) -> BaReport {
26         let (mut h_pp, h_pl, mut h_ll, g_p, g_l) = self.accumulate();
27
28         // Damping first, so the elimination sees the damped blocks. lambda also
29         // quietly supplies the missing gauge: the scale direction has exactly
30         // zero curvature, and with damping the solver declines to move along it
31         // instead of dividing by zero.
32         for i in 0..g_p.len() { h_pp[(i, i)] += lambda * (1.0 + h_pp[(i, i)]); }
33         for b in h_ll.iter_mut() { for a in 0..3 { b[(a, a)] += lambda * (1.0 + b[(a, a)
34         )); } }
35
36         // S = H_pp - Sigma_j W_j H_pl[:,j]^T, with W_j = H_pl[:,j] H_ll[j]-1.
37         // Each landmark costs one 3*3 inverse and touches only the cameras that
38         // saw it -- which is why the *pattern* of S is the covisibility graph.
39         let mut s = h_pp.clone();
40         let mut rhs = g_p.clone();
41         let w: Vec<Vec<Matrix6x3<f64>>> = h_ll
42             .iter()
43             .enumerate()
44             .map(|(j, hjj)| {
45                 let inv = hjj.try_inverse().expect("a landmark seen twice is invertible");
46
47                 h_pl.iter().map(|strip| strip[j] * inv).collect()
48             })
49             .collect();
50         for (j, wj) in w.iter().enumerate() {
51             for (a, wa) in wj.iter().enumerate() {
52                 rhs.fixed_rows_mut:::<6>(6 * a).sub_assign(&(wa * g_l[j]));
53                 for (b, _) in wj.iter().enumerate() {
54                     let block: Matrix6<f64> = wa * h_pl[b][j].transpose();
55                     s.fixed_view_mut:::<6, 6>(6 * a, 6 * b).sub_assign(&block);
56                 }
57             }
58         }
59     }
60 }

```

```

55     }
56 }
57
58 let d_xi = s.cholesky().expect("S is SPD once damped").solve(&rhs);
59
60 // Back-substitution, then retract. Poses move on the manifold with [+];
61 // points are in R3 and simply add.
62 for (a, k) in self.free_indices().enumerate() {
63     let xi: Vector6<f64> = d_xi.fixed_rows::<<6>(6 * a).into();
64     self.poses[k] = self.poses[k] * Isometry3F64::exp(&xi);
65 }
66 for (j, h_jj) in h_ll.iter().enumerate() {
67     let mut r = g_l[j];
68     for (a, strip) in h_pl.iter().enumerate() {
69         r -= strip[j].transpose() * d_xi.fixed_rows::<<6>(6 * a);
70     }
71     self.points[j] += h_jj.try_inverse().unwrap() * r;
72 }
73
74 self.report()
75 }
76 }

```

Preintegration is the artifact worth writing by hand, because the ordering of the statements *is* the derivation: the covariance and the bias Jacobians both read the previous $\Delta \mathbf{R}$, so they must be updated before the deltas advance.

RUST crates/ch18_vio/src/preint.rs

```

1 use nalgebra::{Matrix3, SMatrix, Vector3};
2 use sophus::lie::Rotation3F64; // SO(3) with exp, log, and right Jacobian
3
4 /// The 9*9 covariance of (deltaphi, deltav, deltap) and the 9*6 bias Jacobian. nalgebra'
5 s
6 /// named aliases stop at 6, and const generics are clearer here anyway.
7 pub type Matrix9 = SMatrix<f64, 9, 9>;
8 pub type Matrix9x6 = SMatrix<f64, 9, 6>;
9
10 #[derive(Clone, Copy, Debug)]
11 pub struct ImuSample { pub gyro: Vector3<f64>, pub acc: Vector3<f64> }
12
13 #[derive(Clone, Copy, Debug, Default)]
14 pub struct ImuBias { pub gyro: Vector3<f64>, pub acc: Vector3<f64> }
15
16 /// One preintegrated interval: a single Gaussian factor standing in for
17 /// hundreds of raw samples.
18 pub struct Preintegrated {
19     pub dt_ij: f64,
20     pub n: usize,
21     pub d_rot: Rotation3F64,
22     pub d_vel: Vector3<f64>,
23     pub d_pos: Vector3<f64>,
24     /// Covariance of (deltaphi, deltav, deltap), in that block order.
25     pub cov: Matrix9,
26     /// ?(DeltaR, Deltav, Deltap)?(b_g, b_a): the O(1) bias update of Derivation 4, Step
27     5.
28     pub j_bias: Matrix9x6,
29     /// The bias these deltas were integrated at. Corrections are relative to it.

```

```

28     pub bias: ImuBias,
29 }
30
31 pub fn preintegrate(
32     samples: &[ImuSample],
33     dt: f64,
34     bias: ImuBias,
35     noise: ImuNoise,
36 ) -> Preintegrated {
37     let (mut d_rot, mut d_vel, mut d_pos) = (Rotation3F64::identity(), Vector3::zeros(),
38     Vector3::zeros());
39     let mut cov = Matrix9::zeros();
40     let mut j_bias = Matrix9x6::zeros();
41
42     for s in samples {
43         let w = s.gyro - bias.gyro;
44         let a = s.acc - bias.acc;
45         let d_rk = Rotation3F64::exp(&(w * dt));
46         let jr = Rotation3F64::right_jacobian(&(w * dt));
47         let dr_a = d_rot.matrix() * skew3(&a); // DeltaR_ik [ ? - b_a]*
48
49         // Sigma <- A Sigma A^T + B Sigma_eta B^T. The lower-left blocks of A are the
50         leak: an
51         // attitude error tilts the integrated acceleration, which is why gyro
52         // noise ends up dominating position drift over long intervals.
53         let (a_mat, b_mat) = transition_blocks(&d_rot, &d_rk, &dr_a, &jr, dt);
54         cov = a_mat * cov * a_mat.transpose() + b_mat * noise.discrete(dt) * b_mat.
55         transpose();
56
57         // Bias Jacobians, by the same recursion and with the same ordering.
58         update_bias_jacobians(&mut j_bias, &d_rot, &d_rk, &dr_a, &jr, dt);
59
60         // Only now may the deltas advance.
61         let dra = d_rot * a;
62         d_pos += d_vel * dt + 0.5 * dra * dt * dt;
63         d_vel += dra * dt;
64         d_rot = (d_rot * d_rk).orthonormalized();
65     }
66
67     Preintegrated { dt_ij: samples.len() as f64 * dt, n: samples.len(), d_rot, d_vel,
68     d_pos, cov, j_bias, bias }
69 }
70
71 /// The payoff: a new bias estimate costs five small products, not a re-run.
72 pub fn correct_for_bias(pre: &Preintegrated, b: ImuBias) -> (Rotation3F64, Vector3<f64>,
73     Vector3<f64>) {
74     let db_g = b.gyro - pre.bias.gyro;
75     let db_a = b.acc - pre.bias.acc;
76     let j = &pre.j_bias;
77     (
78         pre.d_rot * Rotation3F64::exp(&(j.fixed_view::<3, 3>(0, 0) * db_g)),
79         pre.d_vel + j.fixed_view::<3, 3>(3, 0) * db_g + j.fixed_view::<3, 3>(3, 3) * db_a,
80
81         pre.d_pos + j.fixed_view::<3, 3>(6, 0) * db_g + j.fixed_view::<3, 3>(6, 3) * db_a,
82     )
83 }
84
85 /// `imu_residual` -- the 9-vector tying navigation states i and j.
86 /// Gravity appears *here*, not inside the deltas. That is exactly what makes
87 /// the deltas state-independent, and also why the accelerometer makes roll and

```

```

82 // pitch observable while leaving global position and yaw free.
83 pub fn imu_residual(pre: &Preintegrated, si: &NavState, sj: &NavState, g: &Vector3<f64>,
84   b: ImuBias)
85   -> SMatrix<f64, 9, 1>
86 {
87   let (d_rot, d_vel, d_pos) = correct_for_bias(pre, b);
88   let dt = pre.dt_ij;
89   let r_i = si.rot.matrix();
90   let r_rot = (d_rot.inverse() * si.rot.inverse() * sj.rot).log();
91   let r_vel = r_i.transpose() * (sj.vel - si.vel - g * dt) - d_vel;
92   let r_pos = r_i.transpose() * (sj.pos - si.pos - si.vel * dt - 0.5 * g * dt * dt) -
93   d_pos;
94   SMatrix::<f64, 9, 1>::from_iterator(
95     r_rot.iter().chain(r_vel.iter()).chain(r_pos.iter()).copied(),
96   )
97 }

```

18.5.1 The worked example, and the test that pins it

Everything the chapter claimed numerically, in one test file. The projection numbers are checkable with a calculator; the preintegration one is checkable with a pen, because a constant-axis rotation makes the bias correction exact.

RUST crates/ch18_vio/tests/worked_example.rs

```

1 use approx::assert_relative_eq;
2 use ch18_vio::{correct_for_bias, depth_sigma, preintegrate, ImuBias, ImuNoise, ImuSample,
3   Pinhole};
4 use nalgebra::{Vector2, Vector3};
5 use sophus::lie::Isometry3F64;
6
7 const CAM: Pinhole = Pinhole { fx: 400.0, fy: 400.0, cx: 320.0, cy: 240.0 };
8
9 #[test]
10 fn worked_example_ch18_two_views_of_one_point() {
11   let t1 = Isometry3F64::identity(); // camera 0 at the origin
12   let t2 = Isometry3F64::from_translation(&Vector3::new(-0.5, 0.0, 0.0)); // centre at
13   +0.5 x
14   let p = Vector3::new(0.2, -0.1, 5.0);
15
16   let z1 = CAM.project(&t1, &p).unwrap();
17   let z2 = CAM.project(&t2, &p).unwrap();
18   assert_relative_eq!(z1, Vector2::new(336.0, 232.0), epsilon = 1e-12);
19   assert_relative_eq!(z2, Vector2::new(296.0, 232.0), epsilon = 1e-12);
20   assert_relative_eq!(z1.x - z2.x, 40.0, epsilon = 1e-12); // disparity
21
22   // Stretch the point 10% along camera 0's ray. Camera 0 cannot tell;
23   // camera 1 moves by 40/11 px. This is depth-blindness, quantified.
24   let stretched = p * 1.1;
25   assert_relative_eq!(CAM.project(&t1, &stretched).unwrap(), z1, epsilon = 1e-12);
26   let moved = (CAM.project(&t2, &stretched).unwrap() - z2).norm();
27   assert_relative_eq!(moved, 40.0 / 11.0, epsilon = 1e-9);
28
29   // One pixel of matching noise costs 12.5 cm of depth at 5 m.
30   assert_relative_eq!(depth_sigma(&CAM, 0.5, 5.0, 1.0), 0.125, epsilon = 1e-12);
31 }

```



```

30
31 #[test]
32 fn worked_example_ch18_bias_correction_is_exact_for_a_fixed_axis() {
33     // 2 s of constant yaw rate at 200 Hz, integrated at b = 0.
34     let samples = vec![ImuSample { gyro: Vector3::new(0.0, 0.0, 0.51), acc: Vector3::new
(0.0, 0.0, 9.81) }; 400];
35     let pre = preintegrate(&samples, 2.0 / 400.0, ImuBias::default(), ImuNoise::mems());
36
37     let raw = pre.d_rot.log().z;
38     assert_relative_eq!(raw, 1.02, epsilon = 1e-9);
39
40     let b = ImuBias { gyro: Vector3::new(0.0, 0.0, 0.01), acc: Vector3::zeros() };
41     let (corrected, _, _) = correct_for_bias(&pre, b);
42     assert_relative_eq!(corrected.log().z, 1.00, epsilon = 1e-9);
43
44     // ...and the O(1) correction agrees with the O(n) re-integration exactly,
45     // because the rotation axis never changed. On a general trajectory the
46     // agreement is second order in deltab, not exact.
47     let exact = preintegrate(&samples, 2.0 / 400.0, b, ImuNoise::mems());
48     assert_relative_eq!(corrected.log().z, exact.d_rot.log().z, epsilon = 1e-9);
49 }

```

The TypeScript port that runs the widgets on this page is checked against the same numbers: `lib/vision/__checks_ch18__.ts` holds sixteen invariants, including analytic-versus-finite-difference agreement for both Jacobian blocks, exactness of DLT triangulation on noiseless pixels, the $\sqrt{\Delta t}$ and $\Delta t^{3/2}$ growth laws for the preintegrated covariance, and the fact that the Schur complement leaves the survivors' marginals numerically unchanged.

18.5.2 The same graph, in factrs

Hand-rolling twice is pedagogy; hand-rolling in production is a bug farm. `factrs` 0.3 ships typed SE(3) variables, an `ImuBias` variable, and an IMU preintegrator, so the visual-inertial graph of this chapter is a page of code:

RUST `crates/ch18_vio/examples/factrs_vi.rs`

```

1 use factrs::{
2     assign_symbols,
3     core::{GaussNewton, Graph, Huber, Values},
4     dtype, fac,
5     linalg::{Const, ForwardProp, Numeric, VectorX, vectorx},
6     residuals::{
7         Residual2,
8         imu_preint::{Accel, Gravity, Gyro, ImuCovariance, ImuPreintegrator},
9     },
10    traits::*,
11    variables::{ImuBias, SE3, VectorVar3},
12 };
13
14 // Type-tagged keys. The compiler will not let a velocity into a pose slot.
15 assign_symbols!(X: SE3; V: VectorVar3; B: ImuBias; L: VectorVar3);
16
17 /// The reprojection residual of Derivation 1 -- written once, differentiated by
18 /// factrs' dual numbers, so this file contains no hand-derived Jacobians.

```

```

19 #[derive(Clone, Debug)]
20 #[factrs::mark]
21 pub struct Reprojection {
22     px: dtype,
23     py: dtype,
24     fx: dtype,
25     fy: dtype,
26     cx: dtype,
27     cy: dtype,
28 }
29
30 impl Residual2 for Reprojection {
31     type V1 = SE3;           // T_wc: the camera in the world
32     type V2 = VectorVar3;    // the landmark
33     type DimIn = Const<9>;
34     type DimOut = Const<2>;
35     type Differ = ForwardProp<Self::DimIn>;
36
37     fn residual2<T: Numeric>(&self, t_wc: SE3<T>, p_w: VectorVar3<T>) -> VectorX<T> {
38         let p_c = t_wc.inverse().apply(p_w.into());
39         vectorx![
40             T::from(self.fx) * p_c[0] / p_c[2] + T::from(self.cx) - T::from(self.px),
41             T::from(self.fy) * p_c[1] / p_c[2] + T::from(self.cy) - T::from(self.py)
42         ]
43     }
44 }
45
46 fn main() {
47     let data = two_keyframe_scene(0x5EE3D); // 200 IMU samples, 6 shared landmarks
48     let mut values = Values::new();
49     let mut graph = Graph::new();
50
51     // Two keyframes, each with a pose, a velocity, and a bias.
52     for (k, kf) in data.keyframes().enumerate() {
53         values.insert(X(k), kf.pose);
54         values.insert(V(k), VectorVar3::from(kf.velocity));
55         values.insert(B(k), ImuBias::identity());
56     }
57     for (j, p) in data.landmark_guesses().enumerate() {
58         values.insert(L(j), VectorVar3::from(p));
59     }
60
61     // One preintegrated factor stands in for all 200 raw samples.
62     let mut preint = ImuPreintegrator::new(ImuCovariance::default(), ImuBias::identity(),
63     Gravity::up());
64     for s in data.imu_samples() {
65         preint.integrate(&Gyro::new(s.gyro), &Accel::new(s.acc), s.dt);
66     }
67     graph.add_factor(preint.build(X(0), V(0), B(0), X(1), V(1), B(1)));
68
69     // Pixel observations get a Huber kernel: a mismatched feature is not a
70     // large Gaussian draw, it is a draw from a different distribution.
71     for o in data.observations() {
72         let r = Reprojection { px: o.u, py: o.v, fx: 400.0, fy: 400.0, cx: 320.0, cy:
240.0 };
73         graph.add_factor(fac![r, (X(o.kf), L(o.pt)), 1.0 as std, Huber::default()));
74     }
75
76     let result = GaussNewton::new(graph).optimize(values).unwrap();
77     report(&result); // velocity to ~2 cm/s, gyro bias to ~5e-4 rad/s
78 }

```

⚠ WHERE ROBOTS BREAK THIS

fact rs runs native-side in this book. Its WebAssembly build is not exercised by the book's CI, so the widgets on this page run the TypeScript port instead — the same equations, checked against the same numbers. Where a chapter's crate does not build for the browser, the book says so rather than pretending.

18.6 Putting it together: two real systems

Everything above is machinery. A working visual-inertial SLAM system is machinery plus a great deal of engineering judgement about when to trust it, and the two most-cited open systems make almost opposite engineering choices while agreeing completely about the mathematics.

🖥 INTERACTIVE · f18.1**ORB-SLAM3 and VINS architectures**

Two systems, no new ideas: both are compositions of blocks this book has already built — the engineering is in the plumbing, not in a hidden theory.

Run this simulation in the web edition: [/chapters/ch18-visual-slam #f18.1](#)

ORB-SLAM3 (Campos et al., 2021) is feature-based and map-centric: ORB descriptors, three parallel threads, and an *Atlas* of sub-maps so that losing tracking starts a new map instead of corrupting the old one. Its visual-inertial contribution is to make the IMU initialization itself a maximum-a-posteriori problem rather than a heuristic, and its reported accuracy on the standard drone benchmarks — an average of 3.6 cm on EuRoC for the stereo-inertial configuration — is what made "2 to 5 times more accurate than previous approaches" a defensible claim rather than marketing.

VINS-Mono / VINS-Fusion (Qin, Li & Shen, 2018) is optical-flow-based and window-centric: KLT tracks, a loosely-coupled bootstrap that aligns a vision-only reconstruction with the preintegrated IMU to recover scale and gravity, then a tightly-coupled sliding window whose past is a marginalization prior, and finally a global pose graph that optimizes only four degrees of freedom because gravity has already fixed roll and pitch.

Read the two columns of that figure again. There is not one block in either system that this book has not built: a measurement model, a manifold retraction, a sparse least-squares solve, an elimination, a loop closure. The systems are hard because integration is hard — thread safety, initialization corner cases, feature-track bookkeeping — not because the estimation theory is deeper than

what you have just derived.

NOTE

What we deliberately skipped. Feature detection, description and matching (Hartley & Zisserman and Szeliski are the real textbooks); the five-point algorithm's derivation; rolling shutter; event cameras; and dense or learned visual SLAM. That last one is moving fastest: systems like [MASt3R-SLAM](#) replace the whole geometric front end with a learned two-view reconstruction prior and still hand the result to a second-order optimizer. Chapter 25 is where that thread is picked up — and the question it asks is the one this chapter has been asking all along: what is the *likelihood* of the thing the network just told you?

18.7 Exercises

1 FOUNDATION • • • The other convention

Redo Derivation 1 under a *left* perturbation, $T \mapsto \exp(\xi^\wedge) T$ (not the book's retraction — that is the point of the exercise), and show that the 2×6 pose Jacobian becomes $J_\pi(\mathbf{I} \mid -[{}^c\mathbf{P}]_\times)$ — the skew is now of the point in the *camera* frame. Then show that for a point on the optical axis ($X = Y = 0$) the third column of J_π vanishes, and say in one sentence what that means for the observability of forward motion. Check both Jacobians numerically against central differences of `Pinhole::project`.

2 FOUNDATION • • Scale is a null space, not a large variance

Prove that the map $\mathbf{C}_k \mapsto s\mathbf{C}_k$, ${}^w\mathbf{P}_j \mapsto s {}^w\mathbf{P}_j$ leaves every reprojection residual exactly invariant, and conclude that the bundle-adjustment Hessian is singular in that direction. Then show that adding the term $\mathbf{g} \Delta t_{ij}$ in $\mathbf{r}_{\Delta v}$ destroys the invariance — that is, write the residual after the scaling and identify precisely which term fails to scale with s .

3 FOUNDATION • • • Where the uncertainty comes from

Derive the first two block rows of the preintegration covariance recursion (Derivation 4, Step 4) from the error dynamics of $(\delta\phi, \delta\mathbf{v})$. Use them to show that with accelerometer noise alone $\sigma_v \propto \sqrt{\Delta t}$ and $\sigma_p \propto \Delta t^{3/2}$, and predict the exponent when gyro noise dominates. Verify against w18.2 by reading σ_v at keyframe spacings of 0.5 s and 2 s.

4 CONCEPTUAL • Sideways or along the ray?

In w18.1, pause the sweep and drag the tracked point 10 cm *along* camera 0's green ray,

then 10 cm *across* it. Before you look: using Derivation 1, Step 4 — at $Z = 5$ m the pixel moves 80 px per metre of lateral displacement and 3.2 px per metre of depth — predict the residual change each drag produces in each camera, and their ratio. Check both. Then explain why camera 0 reads exactly 0.00 px for the along-ray drag while camera 1 does not, and connect that to Derivation 3's degenerate motions.

5 CONCEPTUAL •• Count the fill-in before you see it

In w18.3, pause the simulation at slide 0 and count: the window has eight keyframes and twelve landmarks, each landmark seen by four keyframes. Predict how many *new* block couplings the first slide creates when the oldest keyframe and the landmarks it hosted are eliminated — remember that eliminating a landmark clique-connects everything that saw it, and that some of those pairs were already coupled. Run one slide and check against the fill-in tile. Then turn on link-dropping and explain which entries it removes first, and why those are exactly the ones whose deletion is most defensible — and still not free.

6 PRACTICAL ••• A stereo rig needs no gauge

Extend TinyBa with a second set of ReprojFactors from a right camera at a *known* fixed baseline. Show experimentally that the reduced system is now full rank without pinning scale: compare the smallest eigenvalue of $\mathbf{S}$ with and without the stereo factors. Then corrupt 10% of the matches with 30 px outliers, watch the solve fail, and rescue it with the Huber kernel from Chapter 15. Report RMSE before and after.

7 PRACTICAL ••• Project the landmark out

Implement the MSCKF null-space trick. Given a feature tracked over M poses, build $\mathbf{H}_f \in \mathbb{R}^{2M \times 3}$, compute a basis $\mathbf{N}$ of its left null space by Givens rotations (do not form $\mathbf{H}_f^\dagger$), and verify numerically that $\|\mathbf{N}^\top \mathbf{H}_f\| < 10^{-12}$ and that the projected residual has dimension $2M - 3$. Then compare, on the same synthetic track, the pose correction it produces against a full bundle adjustment that keeps the landmark — and explain the difference you see.

18.8 References

Forster, C., Carlone, L., Dellaert, F., and Scaramuzza, D. (2017). On-Manifold Preintegration for Real-Time Visual-Inertial Odometry *IEEE Transactions on Robotics* 33(1), 1–21.

The chapter’s centerpiece. Derivation 4 follows its notation, including the bias Jacobians and the covariance recursion; this is also the paper that made the structureless (MSCKF-style) visual factor respectable inside a smoother. doi:10.1109/TR0.2016.2597321

Lupton, T. and Sukkarieh, S. (2012). Visual-Inertial-Aided Navigation for High-Dynamic Motion in Built Environments Without Initial Conditions *IEEE Transactions on Robotics* 28(1), 61–76.

Where preintegration was invented: summarize many high-rate measurements into one constraint that does not depend on the initial conditions. Forster et al. later fixed the rotation to live on SO(3) rather than in Euler angles. doi:10.1109/TR0.2011.2170332

Mourikis, A. I. and Roumeliotis, S. I. (2007). A Multi-State Constraint Kalman Filter for Vision-aided Inertial Navigation *IEEE International Conference on Robotics and Automation (ICRA)*, 3565–3572.

The filtering half of Derivation 5. The null-space projection that keeps landmarks out of the state is still the reason MSCKF-family estimators run on hardware that could never afford a smoother. doi:10.1109/ROBOT.2007.364024

Triggs, B., McLauchlan, P. F., Hartley, R. I., and Fitzgibbon, A. W. (2000). Bundle Adjustment — A Modern Synthesis *Vision Algorithms: Theory and Practice (IWVA 1999)*, LNCS 1883, 298–372.

The canonical treatment of Derivation 2, including gauge freedom, the Schur complement, and robust cost functions. Read §6 if you ever have to argue with someone about what a covariance means when the problem has a null space. doi:10.1007/3-540-44480-7_21

Campos, C., Elvira, R., Gómez Rodríguez, J. J., Montiel, J. M. M., and Tardós, J. D. (2021). ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM *IEEE Transactions on Robotics* 37(6), 1874–1890.

The left column of f18.1, and the source of this chapter’s accuracy claims. Its real contribution is treating IMU initialization as MAP estimation rather than a bootstrap heuristic. doi:10.1109/TR0.2021.3075644

Qin, T., Li, P., and Shen, S. (2018). VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator *IEEE Transactions on Robotics* 34(4), 1004–1020.

The right column of f18.1: sliding-window smoothing with a marginalization prior, and the clearest published account of why the final pose graph optimizes four degrees of freedom and not six. doi:10.1109/TR0.2018.2853729

Demmel, N., Schubert, D., Sommer, C., Cremers, D., and Usenko, V. (2021). **Square Root Marginalization for Sliding-Window Bundle Adjustment** *IEEE/CVF International Conference on Computer Vision (ICCV)*, 13240–13248.

The modern answer to Derivation 5’s numerical problem: store the marginalization prior as a square root rather than a Hessian, and single-precision arithmetic stops destroying it. Also the cleanest description of nullspace-based landmark elimination in a window. doi:10.1109/ICCV48922.2021.01301

Murai, R., Dexheimer, E., and Davison, A. J. (2025). **MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors** *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, *highlight*.

Where this chapter’s assumptions are being renegotiated: a learned two-view reconstruction prior replaces the geometric front end, without a parametric camera model — and the back end is still second-order optimization over poses. Chapter 25 picks up the thread. [Link](#)

Modern Map Representations

MAPPING AND SLAM · Intermediate · 55 min

“

We show that under certain assumptions, this isosurface is optimal in the least squares sense.

Brian Curless and Marc Levoy

A Volumetric Method for Building Complex Models from Range Images, SIGGRAPH 1996

Chapter 13 built a map that answers exactly one question — *is this cell occupied?* — at exactly one resolution, in memory proportional to the floor area whether or not anything interesting is there. That map got us through localization, SLAM, and loop closure. It is about to fail us.

The planner in Chapter 20 does not want to know whether a cell is occupied. It wants to know *how far away the nearest obstacle is, and which direction is away from it*, at a hundred thousand query points per second. The controller in Chapter 23 wants the same thing, differentiated. A renderer wants a surface. An operator wants a picture. Each of these is a different question, and a representation is nothing more than a decision about which question gets to be $O(1)$.

This chapter walks the modern menagerie — octrees, truncated signed distance fields, Euclidean distance fields, meshes, radiance fields — and lands one punchline: **every serious map representation is a recursive estimator wearing a costume.** The octree runs Chapter 13’s log-odds filter per node. TSDF fusion’s “weighted running average” is a scalar information filter for a static state, with a Kalman gain you can read off the code. Even NeRF-style mapping is $\arg \max_{\theta} p(z \mid \theta, x)$ with a differentiable renderer standing in for the measurement model. Learn the costumes; the actor underneath has not changed since Chapter 5.

🕒 AFTER THIS CHAPTER YOU CAN

- Read every modern map representation as a recursive estimator with a

different state, not as a different idea

- Derive TSDF fusion as a per-voxel information filter, and identify its Kalman gain, its accumulated precision, and its forgetting factor
- Extract a surface from a distance field with marching squares, and a planning field with an $O(n)$ distance transform
- Predict which representation wins a given query workload — and measure it, on one log, in the browser
- Explain radiance-field mapping as MAP estimation under a differentiable measurement model, without hand-waving and without a system zoo

ASSUMED

- Occupancy grid mapping and the log-odds recursion (Chapter 13)
- The binary Bayes filter for a static state (Chapter 8)
- The Kalman gain as precision-weighted trust, and the information form (Chapter 6)
- Ray casting and beam geometry (Chapters 4 and 10); optimized poses from a pose graph (Chapter 16)

19.1 The map that cannot answer the question

Here is the concrete failure. Rusty has a 5 cm occupancy grid of the Apartment — 43,200 cells, 43 kB, every one of them a well-calibrated posterior over occupancy. A planner asks: *what is the clearance at (7.3, 4.1)?*

The grid cannot answer. It can tell you about the cell containing that point, and about any other cell you name, but "distance to the nearest occupied cell" is not stored anywhere — it has to be *searched* for, ring by expanding ring, until an occupied cell turns up. Measured on this chapter's Apartment log, that search costs about **16 s** per query against **0.3 s** for an occupancy lookup: a factor of fifty, paid on every one of the millions of collision checks a sampling planner performs.

So store the distance instead of the bit. That single decision is what the rest of the chapter falls out of.

INTERACTIVE · w19.1

TSDF Sculptor

TSDF fusion is not a graphics trick: every cell runs a one-dimensional Kalman filter, and $W_{\max}$ is its memory.

Run this simulation in the web edition: `/chapters/ch19-map-representations #w19.1`

Watch one lap before reading on. Three things are happening, and each is a section of this chapter.

The field is signed, and truncated. Blue is positive — free space in front of a surface — and orange is negative, behind it. Neither extends far: past $\tau = 15$ cm from the nearest observed surface the field simply stops being maintained, because a projective range measurement stops being a good estimate of true distance out there. Truncation is not a memory optimization. It is an honesty policy.

The surface is not stored; it is extracted. The purple contour is the zero level set $\{\mathbf{x} : D(\mathbf{x}) = 0\}$, pulled out of the numbers by marching squares every few frames at sub-cell accuracy. Nothing in the field is "a wall". Walls are where the estimate changes sign.

Every cell is running a filter. Click any cell and the inspector shows three numbers: the prior D_{t-1} , the projective sample d_t this scan delivered, and the fused posterior D_t — with the gain K_t that produced it. That gain is Chapter 6's Kalman gain, and the weight W beside it is Chapter 6's information Ω . This is the chapter's thesis, and it is visible before we prove it.

Now press **move the chair** and watch the ghost. At $W_{\max} = 64$ the abandoned chair loses half its error only every 45 re-observations, so it is still there most of a lap later; drag the slider to 4 and the half-life falls to three, the ghost evaporates — and the rest of the map turns visibly noisy. That trade has a closed form, derived in §3.3, and it is the same trade Chapter 6 called the noise ratio.

19.2 Building intuition: four maps, one log

Before any mathematics, the comparison the whole chapter exists to make. The same 200-scan lap, with the same poses (optimized by Chapter 16's pose graph, so this is a *mapping* chapter and not a SLAM chapter), fused into four representations at once, with meters underneath measuring what each one actually costs.

INTERACTIVE · w19.2

Representation Gallery

There is no best map. Change the query and the ranking inverts — representation choice is workload choice.

Run this simulation in the web edition: `/chapters/ch19-map-representations #w19.2`

The ranking is not stable. That is the point.

- **Occupancy probes** — "is this point occupied?" The flat grid wins: one multiply, one array index. The quadtree pays for eight pointer hops down its levels and comes second by a hair. Both are so cheap that the difference rarely decides an architecture.

- **Distance probes** — "how far to the nearest obstacle?" The ESDF wins by *sixty times* over the grid's ring search and *a hundred and seventy times* over brute-force distance to the extracted segments. This is the query workload that reorganizes modern mapping stacks.
- **Surface extraction** — "give me the boundary as a polyline." The mesh wins trivially: it is already the answer. Everything else has to run marching squares first, which costs 15 ms on this grid.

And memory tells a third story that contradicts the first two. The quadtree, the representation whose entire pitch is compression, uses **five times more memory than the flat grid it was supposed to beat**. That is not a bug in the implementation, and §3.1 derives exactly when it is true.

💡 THE ONE SENTENCE VERSION

A map representation is a choice of which query is $O(1)$. Choose it from the workload — occupancy, distance, surface, appearance — because no representation is best at all four, and the estimator running underneath is the same one in every case.

19.2.1 Notation for this chapter

$\phi(\mathbf{x})$	Signed distance to the nearest surface. Negative inside obstacles, positive in free space, zero on the surface.
τ	Truncation distance. The field is only maintained where $ \phi \leq \tau$.
$\Phi(\mathbf{x})$	The truncated signed distance, $\text{clamp}(\phi(\mathbf{x}), -\tau, +\tau)$ — the quantity a TSDF actually estimates.
$D_t(v), W_t(v)$	Fused distance estimate and accumulated weight of voxel v after t scans. The filter state.
$d_t(v), w_t(v)$	One scan's projective distance observation of voxel v , and the weight (inverse variance) it carries.
$W_{\max}$	Weight clamp. The fading-memory knob: bounds how much evidence a voxel may bank.
$K_t = w_t / (W_{t-1} + w_t)$	The fusion gain. Identical in form and in content to Chapter 6's Kalman gain.
$\text{esdf}(\mathbf{x})$	True Euclidean distance from $\mathbf{x}$ to the extracted surface — defined everywhere, not just near it.
$\ell_{\min}, \ell_{\max}$	OctoMap's clamping bounds on log odds. Without them nothing in a tree ever merges.
$\sigma(\mathbf{x}), T(s), \alpha_i$	Volume density, transmittance along a ray, and the alpha of one sample — the radiance-field vocabulary.

One deliberate exception to the book's color code lives in this chapter, as it did in Chapter 13. The signed distance field is drawn with a **diverging** ramp

— orange for negative (behind a surface), through transparent at zero, to blue for positive (free) — because the sign *is* the meaning, and a sequential ramp would hide the only feature the reader is meant to look for. Every other element keeps its role: measurements `<span style={{ color: 'var(-pr-measurement)'}>green</span>`, extracted posterior surfaces `<span style={{ color: 'var(-pr-posterior)'}>purple</span>`, ground truth `<span style={{ color: 'var(-pr-truth)'}>gray dashed</span>`.

19.3 The mathematics

19.3.1 Hierarchy: the octree is Chapter 13's filter plus a compression theorem

Start with the representation that changes the least. An octree (in 2-D, a quadtree) subdivides space recursively; each leaf stores exactly what a grid cell stored, the log odds ℓ_i of occupancy, and each leaf runs exactly the recursion of Chapter 13, with one addition:

$$\ell_{t,i} = \text{clamp}\left(\ell_{t-1,i} + \text{inverse_sensor_model}(m_i, x_t, z_t) - \ell_0, \ell_{\min}, \ell_{\max}\right)$$

The clamp came into Chapter 13 as a patch for the "stubborn map" problem: an unclamped cell that has seen forty consistent readings needs forty contradicting ones to change its mind. Hornung et al. (2013) noticed that the same patch is what makes the *tree* work, and that observation is the whole of OctoMap.

DERIVATION D1 — clamping makes pruning lossless, and pruning makes the tree worth having

prune when four siblings agree; memory ratio = $b \cdot c_d \cdot S \cdot r / V$

Statement. With clamped log odds, four sibling leaves holding identical values may be replaced by their parent with no change to any query the tree can answer. The resulting node count is proportional to the *surface measure* of the environment rather than its volume, and the memory ratio against a dense array of b -byte cells is

$$\frac{\text{octree bytes}}{\text{dense bytes}} = \frac{b_{\text{node}} c_d S r}{V}, \quad c_d = \frac{2^{d-1}}{2^{d-1} - 1},$$

for surface measure S (length in 2-D, area in 3-D), resolution r , bounded volume V , and b_{node} bytes per node.

Step 1 — the merge is information-preserving. A query descends to a leaf and returns its value. If all four children of a node hold the same ℓ , then every point in the parent's region returns the same number whether the children exist or not, and the parent stores that number. Nothing that can be asked can distinguish the two trees. If a later scan touches one child, the node splits again and *copies the parent's value into all four children* — which restores exactly the state the merge discarded, because that state was four copies of one number.

Step 2 — clamping is what makes agreement common. Without a clamp, two neighboring free cells observed a different number of times hold different log odds forever, and four siblings essentially never agree. With the clamp, free space saturates at $\ell_{\min}$ after a handful of misses — OctoMap's published defaults are $p_{\text{miss}} = 0.4$ with $p \in [0.12, 0.97]$, so $\lceil \ln(0.12/0.88)/\ln(0.4/0.6) \rceil = 5$ misses saturate a cell — and from then on entire subtrees hold one identical number. Saturation is not a rounding artifact here; it is the compression mechanism.

Step 3 — count the nodes. At leaf resolution, the cells the tree must keep refined are those straddling a surface: roughly S/r^{d-1} of them. One level up, the surface still needs refining but each node is 2^d times larger and covers 2^{d-1} times more surface, so the count falls by 2^{d-1} . Summing the geometric series gives $c_d S/r^{d-1}$ total nodes with $c_d = 2^{d-1}/(2^{d-1} - 1)$: $c_2 = 2$ in the plane, $c_3 = 4/3$ in space. The dense baseline is V/r^d cells. Divide, multiply by bytes, and Step 1's ratio follows. ■

Where the crossover is. Set the ratio to 1 and solve for the resolution:

$$r^* = \frac{V}{b_{\text{node}} c_d S}$$

Rusty's Apartment is $V = 108 \text{ m}^2$ of floor with $S = 180.6 \text{ m}$ of (two-sided, 30 cm thick) wall surface, and this book's quadtree node is 20 bytes — one f32 value plus four u32 child indices. So $r^* = 108/(20 \cdot 2 \cdot 180.6) = 1.5 \text{ cm}$. **Above 1.5 cm resolution the quadtree loses to a byte-per-cell array**, which is why w19.2's memory meter reads 239 kB against the grid's 43 kB at 5 cm. (The tree is built on a 16 m power-of-two root, so its leaves are actually 6.25 cm — a detail that makes this comparison *kinder* to the tree, not harsher, and one the formula handles: $r = 0.0625$ predicts a ratio of 4.2.) The measured 5.5 is worse still, for a reason worth knowing: the tree also refines along the boundary between free and *unknown* space — the frontier that Chapter 24 will hunt for — and that boundary is a surface too. The formula also says exactly when octrees do win: r^* grows with V/S , and in three dimensions volume grows as the cube of scale while surface grows only as the square. Take a drone mapping a $200 \times 200 \times 60 \text{ m}$ block of airspace at 10 cm — $V = 2.4 \times 10^6 \text{ m}^3$ against maybe $S = 6 \times 10^4 \text{ m}^2$ of

ground and façades, with a 36-byte node (a value plus eight child indices). Then $r^* = 83$ cm, the working resolution is eight times finer than that, and the octree wins by roughly 8×: 288 MB against a dense array's 2.4 GB, which is not a saving so much as the difference between running and not running.

There is a second effect the formula does not capture, and in practice it is the larger one. A dense array must allocate its whole bounding box before it knows anything; a tree stores never-observed space as a *single node* and grows only where scans arrive. A robot inside a building observes a shell, and the box around that shell is mostly nothing. The Apartment at 5 cm in the plane — small, fully traversed, coarse relative to r^* — is precisely the regime where neither effect pays. This book shows you a quadtree that loses because a chapter that only showed you the wins would not be teaching you how to choose.

ALGORITHM `octree_insert_scan(tree, x_t, z_t)` COST $O(B \cdot L/r \cdot \log(L/r))$ for B beams of range L ; the prune pass is amortized $O(\text{nodes})$

in the tree, the (known) pose, the scan

out the updated, pruned tree

```

1: for each beam  $k$  in  $z_t$  do
2:    $C \leftarrow$  cells traversed from  $x_t$  to the return, by integer ray traversal
3:   for each cell  $i \in C$  not yet touched by this scan do
4:      $\delta \leftarrow \ell_{occ}$  if  $|r_i - z_t^k| < \alpha/2$ , else  $\ell_{free}$  if  $r_i < z_t^k$ , else skip
5:     descend to depth  $d_{max}$ , splitting nodes on the way, and set
        $\ell \leftarrow \text{clamp}(\ell + \delta - \ell_0)$ 
6:   endfor
7: endfor
8: prune(tree): bottom-up, merge any four leaves holding identical  $\ell$ 
9: return tree

```

Line 8 is the only line Chapter 13 did not already have. Line 3's "not yet touched by this scan" is not an optimization either: cells near the sensor lie on many beams, and counting them once per beam would fabricate evidence the scan does not contain — the same per-beam independence lie the Chapter 10 beam model told, arriving here in a new outfit.

19.3.2 Distance, not occupancy

Now change the state. Instead of a bit per cell, store a number: the signed distance to the nearest surface,

$$\phi(\mathbf{x}) = \begin{cases} +\min_{\mathbf{s} \in \mathcal{S}} \|\mathbf{x} - \mathbf{s}\| & \mathbf{x} \text{ outside} \\ -\min_{\mathbf{s} \in \mathcal{S}} \|\mathbf{x} - \mathbf{s}\| & \mathbf{x} \text{ inside} \end{cases}$$

with the surface recovered as the zero level set $\mathcal{S} = \{\mathbf{x} : \phi(\mathbf{x}) = 0\}$. This buys sub-cell surface accuracy for free — a cell holding $\phi = 0.017$ m says the wall is 1.7 cm away, information a binary cell cannot express at any resolution — and it makes surface extraction a root-finding problem instead of a segmentation problem.

It also creates a problem. A range sensor does not measure ϕ . Standing at x_t and firing a beam that returns at range z , the natural estimate for a point at arc length s along that beam is the **projective distance** $d = z - s$, which equals the true signed distance only where the beam meets the surface head-on. If the surface normal makes an angle θ with the beam, then $\phi = (z - s) \cos \theta$, so the projective estimate is too large by $1/\cos \theta$ and diverges at grazing incidence.

i NOTE

The bias vanishes exactly where we use the field. At $s = z$ the projective observation is zero whatever θ is, so the *root* of the linear model is right even when its *slope* is wrong. That is why TSDF reconstructions are sharper than the projective approximation deserves — and why everything else about the field needs fixing before a planner can use it (§3.4).

The standard fix for the divergence is to define the estimand as the *truncated* signed distance $\Phi(\mathbf{x}) = \text{clamp}(\phi(\mathbf{x}), -\tau, +\tau)$ and to fit it only where $|d| \leq \tau$: near the surface, where the approximation holds. Everything else is declared out of scope. Choose τ at two to four cells and never smaller than the thinnest structure you must reconstruct — a wall thinner than 2τ gets observed from both sides, and the two negative claims cancel into nothing.

19.3.3 The centerpiece: TSDF fusion is a per-voxel information filter

Curless and Levoy's fusion rule, written the way graphics writes it, is a running weighted average:

$$D_t = \frac{W_{t-1}D_{t-1} + w_t d_t}{W_{t-1} + w_t}, \quad W_t = \min(W_{t-1} + w_t, W_{\max})$$

Two lines of code, no probability anywhere in sight. It is a Bayes filter, and here is the proof.

 **DERIVATION D2 — the weighted average is the scalar information filter for a static state**

$$D_t = D_{t-1} + K_t(d_t - D_{t-1}), K_t = w_t / (W_{t-1} + w_t), \Omega_t = W_t / \sigma^2$$

Model. Voxel v has an unknown, *static* truncated signed distance $\Phi(v)$. Scan t delivers a noisy projective observation of it,

$$d_t(v) = \Phi(v) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}\left(0, \frac{\sigma^2}{w_t(v)}\right)$$

so the weight w_t is an inverse variance: a beam we trust twice as much gets twice the weight. The prior is flat.

Step 1 — Gaussian likelihoods multiply into weighted least squares. With independent observations,

$$-\log p(d_{1:t} | \Phi) = \sum_{k=1}^t \frac{w_k (d_k - \Phi)^2}{2\sigma^2} + \text{const}$$

Differentiate, set to zero, and the MAP estimate is the weighted mean — the same argument Chapter 15 used to turn a factor graph into least squares, here with a one-dimensional state and no Jacobians to compute:

$$\hat{\Phi}_t = \frac{\sum_k w_k d_k}{\sum_k w_k}, \quad \Omega_t = \frac{\sum_k w_k}{\sigma^2} = \frac{W_t}{\sigma^2}$$

The posterior precision is the accumulated weight, up to the constant σ^2 . That is Chapter 6's information matrix Ω , in one dimension, and its additivity — $\Omega_t = \Omega_{t-1} + w_t / \sigma^2$ — is the information filter's update rule verbatim.

Step 2 — make it recursive. Split the newest term out of both sums:

$$\hat{\Phi}_t = \frac{W_{t-1}\hat{\Phi}_{t-1} + w_t d_t}{W_{t-1} + w_t} = \underbrace{\hat{\Phi}_{t-1}}_{K_t} + \frac{w_t}{W_{t-1} + w_t} (d_t - \hat{\Phi}_{t-1})$$

Prior, plus gain times innovation. This is line for line the scalar Kalman correction.

Step 3 — check the gain against Chapter 6. For a static state the prediction step is the identity, so $\Sigma_t^- = \Sigma_{t-1} = \sigma^2/W_{t-1}$, and the measurement variance is $R_t = \sigma^2/w_t$. Then

$$K_t = \frac{\Sigma_t^-}{\Sigma_t^- + R_t} = \frac{\sigma^2/W_{t-1}}{\sigma^2/W_{t-1} + \sigma^2/w_t} = \frac{w_t}{W_{t-1} + w_t} \quad \checkmark$$

Two consequences fall straight out. A voxel with $W_{t-1} = 0$ has infinite variance, so $K = 1$ and the first observation *replaces* the prior entirely — which is why an unobserved voxel needs no special case in the code. And as $W \rightarrow \infty$, $K \rightarrow 0$: the estimate stops listening, exactly like a Kalman filter whose covariance has collapsed.

Step 4 — the clamp is exponential forgetting. Once $W_{t-1} = W_{\max}$ the gain freezes at $K = w/(W_{\max} + w)$ and the recursion becomes an exponentially weighted moving average. Writing $e_t = D_t - \Phi^{\text{new}}$ for the error after the world changes,

$$e_t = (1 - K) e_{t-1} = \lambda e_{t-1}, \quad \lambda = \frac{W_{\max}}{W_{\max} + w}$$

with half-life $n_{1/2} = \ln 2 / \ln(1 + w/W_{\max}) \approx 0.69 W_{\max}/w$. At $W_{\max} = 16$ and unit weights a stale observation is half-forgotten after 11 scans; at $W_{\max} = 128$, after 89. That is the chair's ghost in w19.1, and it is now a number you can predict before you drag the slider.

Step 5 — what the model hides. Three lies, stated rather than buried.

1. *Per-voxel independence.* Each voxel is estimated alone, exactly as in Chapter 13. Neighboring voxels observe the same wall through the same beam and their errors are anything but independent. The consequence is the same as it was there: the field is overconfident, and the extracted surface is smoother than the evidence justifies.
2. *The observation is biased.* $d_t = z - s$ is projective, not Euclidean (§3.2). The truncation τ is what keeps the bias bounded, and Voxblox's weighting — $w \propto 1/z^2$ for range noise, times a $\cos \theta$ term for incidence — is variance modeling in exactly the sense of Chapter 10, not a heuristic.
3. *Gaussian noise.* A beam that hits glass, or a person walking past, is not $\mathcal{N}(0, \sigma^2/w)$ about the wall behind them. Nothing in this recursion is robust; production systems add space-carving and dynamic-object filters on top, and this book adds none of them here.

ALGORITHM `tsdf_integrate(D, W, x_t, z_t)` **COST** $O(B \cdot \tau/r)$ with carving off -- only the truncation band is touched; $O(B \cdot L/r)$ with carving on
in the field (D, W), the pose, the range scan
out the updated field

```

1: for each beam  $k$  with range  $z$  and bearing  $\theta_k$  do
2:   for  $s = s_{\text{start}}$  to  $z + \tau$  step  $r/2$  do
3:      $v \leftarrow$  voxel at  $x_t + s (\cos \theta_k, \sin \theta_k)$ ; skip if already visited this scan
4:      $d \leftarrow z - s$ ; if  $d < -\tau$  then continue
5:      $w \leftarrow w_{\text{beam}}$  in front of the surface, tapering linearly to 0 at  $d = -\tau$ 
6:      $K \leftarrow w / (W(v) + w)$ 
7:      $D(v) \leftarrow D(v) + K (\text{clamp}(d, -\tau, \tau) - D(v))$ 
8:      $W(v) \leftarrow \min(W(v) + w, W_{\text{max}})$ 
9:   endfor
10: endfor

```

Line 2’s s_{start} is the one real decision. Start at $z - \tau$ and the cost is $O(B\tau/r)$ — measured at **0.11 ms per scan** on this chapter’s log — but the field never learns that the corridor is empty, only that the walls are where they are. Start at the sensor and you also *carve* free space, at $O(BL/r)$ and a measured **0.54 ms per scan**, five times dearer for a contour that is identical to within five segments out of 2,236. You pay that factor of five for one thing only: knowing the difference between free and unknown. Chapter 24 needs it; a pure reconstruction pipeline does not.

19.3.4 A worked example you can check by hand

One voxel, $\tau = 0.20$ m, $W_{\text{max}} = 4$, unit-weight scans except where noted. The voxel starts unobserved: $W_0 = 0$.

scan	d_t	w_t	$K_t = w_t / (W_{t-1} + w_t)$	D_t	W_t
1	0.10	1	$1/1 = 1$	0.100	1
2	0.06	1	$1/2 = 0.5$	0.080	2
3	0.00	2	$2/4 = 0.5$	0.040	4
4	0.00	1	$1/5 = 0.2$	0.032	4 (clamped)
5	0.00	1	0.2	0.0256	4
6	0.00	1	0.2	0.02048	4

Four things are checkable with a pencil, and each one is a claim from D2.

Row 1: the gain is exactly one. No prior information means no reason to keep the prior. The $+\tau$ that unobserved cells are initialized to never enters the

answer.

Row 3: the recursion equals the batch estimate. The weighted mean of everything so far is $(1 \cdot 0.10 + 1 \cdot 0.06 + 2 \cdot 0.00)/4 = 0.16/4 = 0.040$, which is what the recursion produced. Up to the clamp, the filter is not approximating the batch solution — it *is* the batch solution.

Row 4: the clamp breaks that equality on purpose. The batch mean would now be $0.16/5 = 0.032\dots$ which is also what the table says, coincidentally, because W saturated on this exact step. Row 5 is where they part: batch says $0.16/6 = 0.0267$, the clamped filter says 0.0256.

Rows 4–6: geometric decay at rate $\lambda = 0.8$. $0.040 \rightarrow 0.032 \rightarrow 0.0256 \rightarrow 0.02048$, each $0.8\times$ the last, and $\lambda = W_{\max}/(W_{\max} + w) = 4/5$ as D2's Step 4 promised. The half-life is $\ln 2 / \ln 1.25 = 3.1$ scans.

RUST `crates/ch19_maps/tests/worked_example.rs`

```

1 use approx::assert_relative_eq;
2 use ch19_maps::tsdf::{Tsdf2, Tsdf2Config};
3
4 /// The chapter's worked table, pinned. If this test fails the book is wrong.
5 #[test]
6 fn worked_example_ch19_voxel_filter() {
7     let mut f = Tsdf2::new(Tsdf2Config {
8         cells: (1, 1),
9         resolution: 1.0,
10        origin: nalgebra::Point2::origin(),
11        truncation: 0.20,
12        w_max: 4.0,
13    });
14
15    // (observation, weight, expected gain, expected D, expected W)
16    let script = [
17        (0.10, 1.0, 1.0, 0.100_00, 1.0),
18        (0.06, 1.0, 0.5, 0.080_00, 2.0),
19        (0.00, 2.0, 0.5, 0.040_00, 4.0),
20        (0.00, 1.0, 0.2, 0.032_00, 4.0), // clamp engages: W stays at W_max
21        (0.00, 1.0, 0.2, 0.025_60, 4.0),
22        (0.00, 1.0, 0.2, 0.020_48, 4.0),
23    ];
24
25    for (d, w, want_k, want_d, want_w) in script {
26        let ev = f.fuse((0, 0), d, w);
27        assert_relative_eq!(ev.gain, want_k, epsilon = 1e-9);
28        assert_relative_eq!(ev.after, want_d, epsilon = 1e-9);
29        assert_relative_eq!(ev.weight_after, want_w, epsilon = 1e-9);
30    }
31
32    // Up to the clamp, recursive == batch. This is D2 Step 1 and Step 2 agreeing.
33    let batch = (1.0 * 0.10 + 1.0 * 0.06 + 2.0 * 0.00) / 4.0;
34    assert_relative_eq!(batch, 0.040, epsilon = 1e-12);
35
36    // After the clamp, the error decays geometrically at lambda = W_max/(W_max + w).
37    let lambda = 4.0 / 5.0;
38    assert_relative_eq!(0.032 * lambda, 0.025_60, epsilon = 1e-9);
39    assert_relative_eq!(0.025_60 * lambda, 0.020_48, epsilon = 1e-9);
40 }

```

The TypeScript port in `lib/mapping/tsdf.ts` runs the identical arithmetic, which is why the voxel inspector in `w19.1` prints these gains when you click a cell: the table, the Rust test, and the number under your cursor are the same computation three times.

19.3.5 From the field to a surface, and from the field to a plan

A TSDF is not directly usable by anything. Two extraction steps make it so, and they answer two different questions.

Marching squares answers *where is the boundary?* Walk the dual grid; at each cell, the signs of the four corner samples select one of 16 cases; on every edge whose endpoints disagree, place a crossing by linear interpolation,

$$t = \frac{D_a}{D_a - D_b}, \quad \mathbf{p} = \mathbf{c}_a + t(\mathbf{c}_b - \mathbf{c}_a)$$

which is *exact* for the piecewise-linear field the samples define — not an approximation of it. Two of the sixteen cases (the diagonal ones, 0101 and 1010) are genuinely ambiguous: two crossings on four edges are consistent with two different topologies, and the table must simply pick one. In three dimensions that same ambiguity is what puts holes in a naive marching-cubes mesh, and the literature since Lorensen and Cline (1987) is largely about disambiguating it.

```
ALGORITHM marching_squares(D, W)    COST 0(cells), embarrassingly parallel -- no
cell needs any other cell's result
in the fused field and its weights
out a set of line segments approximating {D = 0}
```

```
1: for each cell (i, j) of the dual grid do
2:   if any of the four corners has W < Wmin then continue  (never
   observed: no opinion)
3:   code ←  $\sum_{n=0}^3 2^n [D_n < 0]$ 
4:   for each edge pair in TABLE[code] do
5:     emit the segment joining the two interpolated crossings
6:   endfor
7: endfor
```

Line 2 is the honest line. A cell with no evidence has no sign, and extracting a surface from it is inventing geometry — the failure mode that makes unvalidated reconstructions look confident in exactly the places they are guessing.

The distance transform answers *how far to the boundary, from anywhere?* This

is the query the TSDF conspicuously cannot serve: past τ its value is a constant, and a constant has no gradient.

INTERACTIVE · w19.3

Wavefront

A TSDF is not a distance field with a smaller number in it. Past the truncation band it is flat, and flat fields have no gradients to plan with.

Run this simulation in the web edition: </chapters/ch19-map-representations#w19.3>

That widget is the section in one image. Toggle it off and the rings stop at 30 cm, the arrows vanish, and the probe freezes: a truncated field is not a distance field with smaller numbers in it, it is a distance field with a *hole where the planner lives*.

DERIVATION D3 — the exact Euclidean distance transform in two linear passes

$DT(q) = \min_p [(q-p)^2 + f(p)]$ — a lower envelope of parabolas, $\Theta(n)$ per row

Statement. For samples $f : \{0, \dots, n-1\} \rightarrow \mathbb{R} \cup \{\infty\}$, the sampled distance transform

$$DT_f(q) = \min_p [(q-p)^2 + f(p)]$$

can be computed for all q in $\Theta(n)$ time, and the two-dimensional transform in $\Theta(\text{width} \cdot \text{height})$ by running the one-dimensional version once per row and once per column.

Step 1 — the definition is a lower envelope. Each sample p contributes an upward parabola $y = (q-p)^2 + f(p)$, rooted at $q = p$ and shifted up by $f(p)$. All parabolas have the same curvature, so any two intersect exactly once, at

$$s = \frac{(f(p) + p^2) - (f(p') + p'^2)}{2p - 2p'}$$

DT_f is their pointwise minimum: the lower envelope.

Step 2 — build the envelope in one sweep. Scan p left to right maintaining a stack of the parabolas currently on the envelope and the abscissae where consecutive ones cross. Adding a new parabola: compute where it crosses the top of the stack; if that crossing lies left of the previous one, the stack's top parabola is now entirely hidden — pop it and retry. Each index is pushed once and popped at most once, so the whole pass is $\Theta(n)$, not

$O(n \log n)$ and certainly not the $O(n^2)$ the definition suggests. A second left-to-right sweep evaluates the envelope at each q .

Step 3 — separability. In two dimensions the squared Euclidean distance splits:

$$(q_x - p_x)^2 + (q_y - p_y)^2 + f(p) = (q_x - p_x)^2 + \underbrace{\left[(q_y - p_y)^2 + f(p) \right]}_{\text{a 1-D transform down a column}}$$

so transforming every row and then every column of the result is exact, not an approximation. ■

Step 4 — seeding it from a TSDF instead of from a mask. The classical transform starts from a binary mask: $f = 0$ on obstacles, ∞ elsewhere, giving distances quantized to whole cells. A TSDF can do better. A cell holding $D = 0.017$ m is 0.017 m from the surface, so seed it with $f = (D/r)^2$ in squared-cell units and the transform starts from the sub-cell offset. The result is accurate to a fraction of a cell where a mask-seeded transform is accurate to a whole one — the same trick Voxblox uses, and the reason the ESDF's median error against brute-force distance-to-contour on this chapter's map is **0.6 of a cell**.

ALGORITHM `esdf_from_tsdf(D, W)` COST $\Theta(n)$ for n cells -- two sweeps per row, two per column

in the fused field, its weights, seed band and minimum weight

out `esdf`: signed Euclidean distance to the extracted surface, plus $O(1)$ query and gradient

```

1: for each cell  $v$  do
2:    $f(v) \leftarrow (D(v)/r)^2$  if  $W(v) \geq W_{\min}$  and  $|D(v)| \leq \text{band}$ , else  $\infty$ 
3: endfor
4:  $g \leftarrow \text{distance\_transform\_1d}$  over each row of  $f$ 
5:  $g \leftarrow \text{distance\_transform\_1d}$  over each column of  $g$ 
6:  $\text{esdf}(v) \leftarrow \text{sign}(v) \sqrt{g(v)} r$ , with the sign taken from  $D(v)$  where the
   TSDF has an opinion
7: return esdf
```

Two properties make the result worth its 173 kB. Away from the medial axis a true distance field satisfies the **eikonal equation** $\|\nabla \text{esdf}\| = 1$, so its gradient is a pure direction — "this way is away from the nearest obstacle" — with no magnitude to tune. The widget's meter reads a median $\|\nabla d\| = 0.93$ over a grid of

probes, against 0.01 for the raw TSDF: the shortfall from 1 is discretization plus the medial-axis points where the gradient genuinely does vanish. And both the value and the gradient are $O(1)$ bilinear reads. That pair, $(d, \nabla d)$ in constant time, is the contract Chapter 20 and Chapter 23 are written against.

The honesty note this field deserves: the transform measures distance to the *extracted contour*, and marching squares refuses to emit a contour where a cell has unobserved corners. Where those two disagree — the lip of a doorway seen from one side only — the ESDF reports a distance that is too *small*. Measured on the Apartment at 5 cm: median error 3.0 cm, 99th percentile 7.5 cm, worst case 28 cm, and the worst case is conservative. For a collision cost, being wrong in the safe direction is the correct way to be wrong; for a reconstruction metric it would be a bug. The representation is not neutral about what it is for.

19.4 Implementation in Rust

Three types, one trait, and a benchmark harness that is also the chapter's argument.

The trait first, because it is the thesis in code: four representations, four different answers to the same four questions, and a `None` where a representation genuinely cannot answer in constant time. Nothing is faked — a grid returning a searched distance would hide the whole point.

RUST `crates/ch19_maps/src/gallery.rs`

```
1 use nalgebra::Point2;
2 use crate::{Scan, Se2};
3
4 /// The chapter's thesis as an interface: a map is what it can answer cheaply.
5 pub trait MapRepr {
6     /// Fold one scan in, with a known pose (Chapter 16's optimized graph).
7     fn integrate_scan(&mut self, pose: &Se2, scan: &Scan);
8
9     /// P(occupied) at a point. `None` = never observed, which is *not* 0.5.
10    fn occupancy(&self, p: Point2<f64>) -> Option<f64>;
11
12    /// Distance to the nearest surface, in  $O(1)$ . `None` = this representation
13    /// cannot answer without a search, and saying so is the honest API.
14    fn distance(&self, p: Point2<f64>) -> Option<f64>;
15
16    /// Measured, not estimated: what the harness reports in w19.2.
17    fn memory_bytes(&self) -> usize;
18 }
```

Then the field itself. Two `f32` planes — one for the estimate, one for its accumulated precision — and a fuse that is six lines because D2 says it should be six lines.

RUST `crates/ch19_maps/src/tsdf.rs`

```
1 use nalgebra::{Point2, Vector2};
```



```

2 use ndarray::Array2;
3
4 /// A 2-D truncated signed distance field: Curless & Levoy (1996), read as a
5 /// per-voxel information filter (Chapter 19, D2).
6 pub struct Tsdf2 {
7     /// D_t(v): fused signed distance in metres. Unobserved cells hold +tau.
8     d: Array2<f32>,
9     /// W_t(v): accumulated weight = accumulated precision, up to sigma2.
10    w: Array2<f32>,
11    resolution: f64,
12    origin: Point2<f64>,
13    truncation: f64,
14    /// The fading-memory knob. `f32::INFINITY` recovers the batch estimator.
15    w_max: f32,
16 }
17
18 /// One fusion event, returned so a widget (or a test) can inspect the filter.
19 #[derive(Copy, Clone, Debug)]
20 pub struct FusionEvent {
21     pub before: f64,
22     pub weight_before: f64,
23     pub observation: f64,
24     pub after: f64,
25     pub weight_after: f64,
26     ///  $K_t = w_t / (w_{t-1} + w_t)$  -- Chapter 6's Kalman gain, verbatim.
27     pub gain: f64,
28 }
29
30 impl Tsdf2 {
31     #[inline]
32     fn cell_center(&self, ij: (usize, usize)) -> Point2<f64> {
33         Point2::new(
34             self.origin.x + (ij.0 as f64 + 0.5) * self.resolution,
35             self.origin.y + (ij.1 as f64 + 0.5) * self.resolution,
36         )
37     }
38
39     /// The whole chapter in six lines: precision-weighted averaging, clamped.
40     ///
41     ///  $W = 0$  gives  $K = 1$ , so an unobserved voxel adopts its first observation
42     /// outright -- no special case, just infinite prior variance doing its job.
43     pub fn fuse(&mut self, ij: (usize, usize), d_obs: f64, w_obs: f64) -> FusionEvent {
44         let before = self.d[ij] as f64;
45         let weight_before = self.w[ij] as f64;
46         let denom = weight_before + w_obs;
47         let gain = if denom > 0.0 { w_obs / denom } else { 1.0 };
48         let after = before + gain * (d_obs - before);
49
50         self.d[ij] = after as f32;
51         self.w[ij] = denom.min(self.w_max as f64) as f32;
52
53         FusionEvent { before, weight_before, observation: d_obs,
54             after, weight_after: self.w[ij] as f64, gain }
55     }
56
57     /// `tsdf_integrate`: march each beam through the truncation band.
58     ///
59     /// The visited set is per *scan*, not per beam. Cells near the sensor sit on
60     /// many beams' paths, and fusing them once per beam would manufacture
61     /// evidence the scan does not contain.
62     pub fn integrate(&mut self, pose: &Se2, scan: &Scan, carve: bool) {

```

```

63     let tau = self.truncation;
64     let step = self.resolution * 0.5;
65     let mut visited = std::collections::HashSet::new();
66
67     for (&z, &bearing) in scan.ranges.iter().zip(scan.bearings.iter()) {
68         let hit = z < scan.max_range;
69         let dir = Vector2::new((pose.theta + bearing).cos(), (pose.theta + bearing).
sin());
70         let s0 = if carve { step * 0.5 } else { (z - tau).max(step * 0.5) };
71         let s1 = if hit { z + tau } else { scan.max_range };
72
73         let mut s = s0;
74         while s <= s1 {
75             let p = pose.translation() + s * dir;
76             let Some(ij) = self.world_to_cell(p) else { s += step; continue };
77             if !visited.insert(ij) { s += step; continue }
78
79             let sdf = if hit { z - s } else { tau };
80             if sdf >= -tau {
81                 // Weight tapers to zero behind the surface: a beam says
82                 // progressively less about what lies past what it hit.
83                 let w = if sdf >= 0.0 { 1.0 } else { 1.0 + sdf / tau };
84                 if w > 0.0 {
85                     self.fuse(ij, sdf.clamp(-tau, tau), w);
86                 }
87             }
88             s += step;
89         }
90     }
91 }
92 }

```

Then the distance transform. The inner loop is the one piece of this chapter that looks like nothing else in the book — no probability, no geometry, just a stack of parabolas — and it is worth typing out once in your life.

RUST `crates/ch19_maps/src/esdf.rs`

```

1  use ndarray::{Array2, Axis};
2
3  const BIG: f64 = 1e12;
4
5  /// `dt_1d` -- Felzenszwalb & Huttenlocher's lower envelope of parabolas.
6  ///
7  /// Every index is pushed once and popped at most once, so this is Theta(n). The
8  /// arithmetic is exact in f64 for grids far larger than any robot needs.
9  fn distance_transform_1d(f: &[f64], out: &mut [f64]) {
10     let n = f.len();
11     let mut v = vec![0usize; n]; // parabolas currently on the envelope
12     let mut z = vec![0.0f64; n + 1]; // where consecutive ones cross
13     let mut k = 0usize;
14     v[0] = 0;
15     // The sentinel -inf is not decoration: it is what stops `k` underflowing in
16     // the pop loop below, since no finite crossing can be left of it.
17     z[0] = -BIG;
18     z[1] = BIG;
19
20     for q in 1..n {

```

```

21     let mut s = intersect(f, q, v[k]);
22     while s <= z[k] {
23         k -= 1; // the top parabola is now fully hidden
24         s = intersect(f, q, v[k]);
25     }
26     k += 1;
27     v[k] = q;
28     z[k] = s;
29     z[k + 1] = BIG;
30 }
31
32 k = 0;
33 for q in 0..n {
34     while z[k + 1] < q as f64 {
35         k += 1;
36     }
37     let dq = q as f64 - v[k] as f64;
38     out[q] = dq * dq + f[v[k]];
39 }
40 }
41
42 #[inline]
43 fn intersect(f: &[f64], p: usize, q: usize) -> f64 {
44     let (pf, qf) = (p as f64, q as f64);
45     ((f[p] + pf * pf) - (f[q] + qf * qf)) / (2.0 * pf - 2.0 * qf)
46 }
47
48 /// `esdf_from_tsdf` -- seed with the TSDF's own sub-cell offsets, then sweep.
49 ///
50 /// Seeding with (D/r)2 rather than with a binary mask is what buys sub-cell
51 /// accuracy: a cell holding D = 1.7 cm is 1.7 cm from the surface, not 0.
52 pub fn esdf_from_tsdf(tsdf: &Tsdf2, min_weight: f32, band_cells: f64) -> Esdf2 {
53     let r = tsdf.resolution();
54     let band = band_cells * r;
55     let mut f = Array2::from_elem(tsdf.dim(), BIG);
56
57     for (ij, &d) in tsdf.values().indexed_iter() {
58         if tsdf.weight(ij) < min_weight || (d as f64).abs() > band {
59             continue;
60         }
61         let sub = d as f64 / r;
62         f[ij] = sub * sub;
63     }
64
65     sweep(&mut f, Axis(0)); // rows ...
66     sweep(&mut f, Axis(1)); // ... then columns: separability makes this exact
67     Esdf2::from_squared(f, tsdf, r)
68 }
69
70 impl Esdf2 {
71     /// The Chapter 20 / 23 contract: distance and gradient, both O(1).
72     pub fn distance(&self, p: Point2<f64>) -> f64 { self.bilinear(p) }
73
74     /// Away from the medial axis ||?d|| = 1, so this is a pure direction.
75     pub fn gradient(&self, p: Point2<f64>) -> Vector2<f64> {
76         let h = self.resolution();
77         Vector2::new(
78             (self.bilinear(p + Vector2::x() * h) - self.bilinear(p - Vector2::x() * h)) /
79             (2.0 * h),
80             (self.bilinear(p + Vector2::y() * h) - self.bilinear(p - Vector2::y() * h)) /
81             (2.0 * h),

```

```

80     )
81   }
82 }

```

Finally the cross-check, because a distance field that is subtly wrong is worse than no distance field at all. `parry2d` will compute exact point-to-polyline distance with a BVH; the ESDF must agree with it.

RUST `crates/ch19_maps/tests/esdf_vs_parry.rs`

```

1  use parry2d::query::PointQuery;
2  use parry2d::shape::Polyline;
3
4  /// The ESDF approximates distance-to-contour. `parry2d` computes it exactly.
5  /// Where they disagree by more than a cell, the ESDF must be *conservative* --
6  /// too small -- because a planner that overestimates clearance kills robots.
7  #[test]
8  fn esdf_matches_parry_within_a_cell_and_errs_safe() {
9      let (tsdf, contour) = crate::fixtures::apartment_5cm(); // 200 scans, seed 0
10     xC0FFEE
11     let esdf = ch19_maps::esdf::esdf_from_tsdf(&tsdf, 0.5, 1.5);
12     let mesh = Polyline::new(contour.vertices, Some(contour.indices));
13
14     let mut worst = 0.0f32;
15     for p in crate::fixtures::observed_free_probes(4_000) {
16         let ours = esdf.distance(p.cast::(<f64>())) as f32;
17         let truth = mesh.distance_to_local_point(&p, /* solid = */ false);
18         let err = ours - truth;
19         assert!(err < 0.05, "ESDF overestimated clearance by {err} m at {p}");
20         worst = worst.max(err.abs());
21     }
22     // Measured: median 0.030 m, p99 0.075 m, max 0.282 m at one doorway lip.
23     assert!(worst < 0.30);
24 }

```

19.5 The frontier: rendering as a measurement model

Everything so far estimates *geometry* from *ranges*. The last five years of mapping research have been about estimating geometry **and appearance** from *images*, using neural radiance fields (Mildenhall et al., ECCV 2020) and 3D Gaussian splatting (Kerbl et al., SIGGRAPH 2023). The literature reads like a different subject. It is not.

 **DERIVATION D4** — a differentiable renderer is a measurement model, so fitting one is MAP estimation

$$-\log p(I|\theta, T_c w) \propto \|I - (\theta, T_c w)\|^2$$

Step 1 — the forward model. A radiance field is a function $F_\theta(\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$ giving color and volume density at a point. Along a camera ray $\mathbf{r}(s)$ the rendered color is the volume rendering integral

$$\hat{\mathbf{C}}(\mathbf{r}) = \int_0^\infty T(s) \sigma(\mathbf{r}(s)) \mathbf{c}(\mathbf{r}(s), \mathbf{d}) ds, \quad T(s) = \exp\left(-\int_0^s \sigma(\mathbf{r}(u)) du\right)$$

$T(s)$ is transmittance: the probability that a photon reaches s without being absorbed. Read that way, the integral is an expectation over "where the ray terminates" — the same object a LiDAR beam model integrates over in Chapter 10.

Step 2 — discretize. With piecewise-constant density on samples of width δ_i ,

$$\alpha_i = 1 - e^{-\sigma_i \delta_i}, \quad T_i = \prod_{j < i} (1 - \alpha_j), \quad \hat{\mathbf{C}} = \sum_i T_i \alpha_i \mathbf{c}_i$$

which is exactly alpha compositing. The depth version, which the 1-D toy below renders, replaces $\mathbf{c}_i$ by the sample distance t_i and adds a background term for rays that get through: $\hat{z} = \sum_i T_i \alpha_i t_i + T_n t_{\text{far}}$.

Step 3 — put noise on it and it becomes a likelihood. Assume Gaussian photometric noise on each pixel. Then

$$-\log p(I_t \mid \theta, T_{cw,t}) \propto \|I_t - \hat{I}(\theta, T_{cw,t})\|^2$$

and the map is

$$\hat{\theta} = \arg \max_{\theta} \prod_t p(I_t \mid \theta, T_{cw,t}) p(\theta)$$

— MAP estimation with a rendering measurement model. Same $\eta p(z \mid m, x) p(m)$ as Chapter 13. The difference is entirely in *how* it is optimized: no closed-form inverse sensor model exists, so the gradient is taken through the renderer instead. **The inverse sensor model is computed by autodiff.**

Step 4 — the same residual, optimized over poses, is tracking. Freeze θ and minimize the photometric residual over $T_{cw,t}$ and you have scan-to-map matching with images instead of scans — Chapter 16's ICP objective with a different sensor. Every NeRF-SLAM and 3DGS-SLAM system is some interleaving of these two minimizations.

Step 5 — read the gradient and the failure modes fall out. For the 1-D depth renderer the derivative is available in closed form,

$$\frac{\partial \hat{z}}{\partial \sigma_k} = \delta \left[T_{k+1} t_k - \left(\sum_{i>k} T_i \alpha_i t_i + T_n t_{\text{far}} \right) \right]$$

"adding density here pulls the predicted depth toward t_k and away from everything behind it." Two facts follow immediately. Behind an opaque region $T \approx 0$, so **occluded geometry receives no gradient at all** — these optimizers refine geometry they roughly have and cannot invent geometry they do not. And the derivative peaks at the current surface, which is why initialization and coarse-to-fine scheduling matter so much in practice.

⚠ WHERE ROBOTS BREAK THIS

A quick number for how non-opaque "opaque" is. Take one bin at $t = 3$ m with $\sigma\delta = 5$, so $\alpha = 1 - e^{-5} = 0.9933$, and a far plane at 10 m. The rendered depth is $0.9933 \cdot 3 + 0.0067 \cdot 10 = 3.047$ m — 4.7 cm too far, from a wall the model considers solid. Depth from a radiance field is biased by construction, and the bias shrinks only exponentially in $\sigma\delta$. This is why NeRF-SLAM systems that need metric geometry add depth supervision rather than trusting rendered depth.

🖥 INTERACTIVE · w19.4

Photometric Descent

Radiance-field mapping is not magic: it is MAP estimation under a rendering measurement model, optimized by gradient descent.

Run this simulation in the web edition: [/chapters/ch19-map-representations #w19.4](#)

The toy is 80 bins, ten rays, and plain gradient descent — no network, no rasterizer, a couple of hundred lines — and it reproduces the three behaviors that dominate the literature. Density sharpens only where transmittance survives. The fitted residual falls to the noise floor while the *held-out* rays sometimes do not, which is the geometry–appearance ambiguity in its simplest possible form. And on an unlucky initialization the field parks density in the wrong place and stays there, because Step 5's gradient cannot see past an opaque bin it invented.

📌 NOTE

Why this book does not canonize a system. Between 2023 and 2026 the

NeRF/3DGS-SLAM field produced dozens of systems, each state of the art for about a quarter. The *mechanism* — D4 — is stable; the systems are not. Tosi et al.’s survey (arXiv:2402.13255) is the map of that territory and is updated; this chapter teaches the mechanism and points there. What is genuinely open, and worth knowing you are choosing when you adopt one of these maps: there is no closed-form posterior uncertainty, so nothing downstream can weight the map’s own error the way an ESDF’s weight field lets you.

19.6 Putting it together: the artifact Part VI consumes

Everything above, measured on one log — Rusty’s 200-scan lap of the Apartment at 5 cm, seed 0xC0FFEE, poses from Chapter 16’s optimized graph — with the port that runs the widgets on this page.

representation	memory	update / scan	occupancy probe	distance probe
flat occupancy grid, 1 B/cell	43.2 kB	2.03 ms	317 ns	15.9 s (ring search)
log-odds quadtree, 20 B/node, 6.25 cm leaves	239 kB (11,957 nodes)	0.45 ms	429 ns	same search, tree-shaped
TSDF, f32 D and W	346 kB	0.52 ms (0.11 without carving)	422 ns (sign of D)	— truncated at τ
ESDF, f32	+173 kB	+13.4 ms (full rebuild)	—	254 ns
extracted contour, 2,236 segments	35.8 kB	15.5 ms (marching squares)	$O(F)$ crossing count	45 s brute force

Read the bold entries: **four different winners in four columns**. The cheapest map on the page is the extracted contour, which cannot answer either probe cheaply. The fastest to update is the quadtree, which costs five times the memory of the flat grid it was supposed to compress. The fastest occupancy lookup is the flat grid, which cannot answer a distance query without searching. And the query that actually runs a planner belongs to the distance field, which costs twelve times the grid’s memory to buy a sixtyfold speedup on it. There is no row that dominates, and there was never going to be.

The handoff. What Part VI receives from this chapter is one file — `esdf.bin`, 173 kB, a f32 plane plus an origin and a resolution — and one guarantee: $(d, \nabla d)$ in $O(1)$, eikonal to within about 7% away from the medial axis, and conservative where it is wrong. Chapter 20 uses it as a collision oracle, Chapter 23 differentiates it as an obstacle cost, and Chapter 24 uses the *weight* plane W — the thing that makes this a filter rather than a picture — to find frontiers.

💡 THE COSTUMES, AND THE ACTOR UNDERNEATH

Ocree: Chapter 13's clamped log-odds filter, plus a lossless merge rule. **TSDF:** a scalar information filter per voxel, gain $K = w/(W + w)$, forgetting factor $\lambda = W_{\max}/(W_{\max} + w)$. **ESDF:** not an estimator at all — a deterministic $\Theta(n)$ transform of one. **Mesh:** a root find on the estimate. **Radiance field:** MAP estimation with a differentiable measurement model. Four state spaces, one recursion.

The honesty note this chapter owes you. Per-voxel independence is Chapter 13's structural lie, and every representation here inherits it: OctoMap estimates each node alone, TSDF fusion estimates each voxel alone, and neither has a way to say "these two cells are both wrong in the same direction because the pose was off." The principled alternative is a *correlated* field — Gaussian process implicit surfaces, and in particular the log-GP formulation of Wu et al. (2021), which recovers a genuine Euclidean distance field with calibrated uncertainty by solving the eikonal equation in the transformed space. It costs a dense covariance and is currently practical at room scale rather than building scale. When someone makes it cheap, this chapter changes.

19.7 Exercises

1 FOUNDATION •• Finish D2: the steady-state variance of a clamped voxel

D2 stops after showing that the clamp turns fusion into an exponentially weighted average with forgetting factor $\lambda = W_{\max}/(W_{\max} + w)$. Complete it. (a) Show that the fused estimate under repeated unit-weight observations of a static truth has steady-state variance $\text{Var}[D_{\infty}] = \sigma^2/(2W_{\max} + w)$. (b) Compare with the unclamped filter's σ^2/W_t and state, in one sentence, what the clamp costs you and what it buys. (c) At which $W_{\max}$ does a $W_{\max}$ -clamped voxel have the same variance as an unclamped voxel after 50 unit-weight scans? <details> <summary>Hint</summary> Write the recursion as $D_t = (1 - K)D_{t-1} + Kd_t$ with K constant, take variances of both sides assuming independent observations, and solve the resulting fixed point $V = (1 - K)^2V + K^2\sigma^2/w$. </details>

2 FOUNDATION •• Where hierarchy pays

Using D1's ratio $b_{\text{node}}c_dSr/V$: (a) reproduce the Apartment's crossover $r^* = 1.5$ cm from $S = 180.6$ m, $V = 108$ m², $b_{\text{node}} = 20$ B; (b) redo it in three dimensions for a 3 m ceiling, where a node needs eight child indices; (c) find the environment shape (as a ratio V/S) at which a 5 cm octree first beats a dense byte array, and name a real robot workspace that qualifies.

3 FOUNDATION • • • Why the zero crossing survives a biased observation

§3.2 claims the projective observation $d = z - s$ is biased by $1/\cos \theta$ away from the surface but unbiased *at* it. (a) Prove the claim for a single planar surface and a single beam. (b) Now fuse two beams meeting the same plane at $\theta_1 = 0$ and $\theta_2 = 60^\circ$ with equal weights, and find the zero crossing of the fused linear field along a line perpendicular to the surface. Is it still exact? (c) What does your answer predict about reconstruction error on walls seen only at a grazing angle — and how does Voxblox's $\cos \theta$ weight term address it?

4 CONCEPTUAL • Predict the ghost, then watch it

Using $\lambda = W_{\max}/(W_{\max} + w)$ from D2, predict how many re-observations it takes for the teleported chair's ghost to lose half its distance error at $W_{\max} \in \{4, 24, 64\}$. Now open w19.1, click the chair's cell to inspect it, press `<em>move the chair</em>`, and count. Where does your prediction fail, and why? (The weight a carving update contributes is not 1.)

5 CONCEPTUAL • • Make each representation win

In w19.2, find a query workload under which each of the four panes is the winner at some point, and write a one-sentence description of a real robot task with that workload. One pane never takes the green frame under any of the three workloads: name it, and say — using D1, and the cost of descending a tree instead of indexing an array — exactly what it is paying for and what it would take to make that payment worthwhile.

6 PRACTICAL • • A fifth representation

Implement MapRepr for a k-d tree over raw scan endpoints and add it to the gallery harness. It should win the distance query at low point counts and lose it as the cloud grows. Report the crossover against the ESDF, and then answer the real question: why do production systems keep point clouds as the *log* and never as the primary map?

7 PRACTICAL • • • Two channels beat one

Extend the 1-D renderer from depth-only to depth plus reflectance. Show that on the seed where the depth-only fit parks density in the wrong place (w19.4, re-roll until the held-out error stays high while the training residual collapses), the two-channel version escapes — and explain which term of D4 Step 5's gradient the extra channel repaired. Then find a seed where two channels are still not enough, and say what a third would have to measure.

19.8 References

Curless, B. and Levoy, M. (1996). A Volumetric Method for Building Complex Models from Range Images *SIGGRAPH '96*, 303–312.

The origin of TSDF fusion, and the source of this chapter’s epigraph. Appendix A sketches the least-squares optimality that D2 turns into a recursive filter. doi:10.1145/237170.237269

Lorensen, W. E. and Cline, H. E. (1987). Marching Cubes: A High Resolution 3D Surface Construction Algorithm *SIGGRAPH '87, Computer Graphics* 21(4), 163–169.

The extraction step of §3.4, in three dimensions. The ambiguous cases this chapter dodges in 2-D are the reason the follow-up literature exists. doi:10.1145/37401.37422

Felzenszwalb, P. F. and Huttenlocher, D. P. (2012). Distance Transforms of Sampled Functions *Theory of Computing* 8(19), 415–428.

D3’s lower-envelope algorithm, with the amortized $\Theta(n)$ argument. The clearest exposition of the trick that makes ESDFs affordable. [link](#)

Hornung, A., Wurm, K. M., Bennewitz, M., Stachniss, C., and Burgard, W. (2013). OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees *Autonomous Robots* 34(3), 189–206.

The hierarchical representation of §3.1, including the clamping-and-pruning argument D1 formalizes and the node layout the memory bound counts. doi:10.1007/s10514-012-9321-0

Oleynikova, H., Taylor, Z., Fehr, M., Siegwart, R., and Nieto, J. (2017). Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning *IEEE/RSJ IROS 2017*, 1366–1373.

The system that made ESDF-from-TSDF standard practice in robotics, and the source of the weighting choices (inverse-square range, incidence angle) that D2 Step 5 reads as variance modeling. doi:10.1109/IROS.2017.8202315

Reijgwart, V., Cadena, C., Siegwart, R., and Ott, L. (2023). Efficient Volumetric Mapping of Multi-Scale Environments Using Wavelet-Based Compression *Robotics: Science and Systems XIX*.

Where §3.1 goes next: wavelets instead of a plain octree, keeping multi-resolution queries while fixing the memory constant that makes this chapter’s quadtree lose. doi:10.15607/RSS.2023.XIX.065

Millane, A., Oleynikova, H., Wirbel, E., Steiner, R., Ramasamy, V., Tingdahl, D., and Siegwart, R. (2024). nvblox: GPU-Accelerated Incremental Signed Distance Field Mapping *IEEE ICRA 2024*.

The production path for everything in §3.3 and §3.4: the same recursion, moved to the GPU, with measured speedups of two orders of magnitude on ESDF construction. [link](#)

Tosi, F., Zhang, Y., Gong, Z., Sandström, E., Mattoccia, S., Oswald, M. R., and Poggi, M. (2024). How NeRFs and 3D Gaussian Splatting are Reshaping SLAM: a Survey *arXiv:2402.13255*.

The frontier map for this chapter's last section. Read it instead of any single system paper: it is maintained, and it states each system's assumptions in the terms D4 uses. [link](#)

Part VI

Planning and Acting under Uncertainty

Motion Planning: From Geometry to Probability

PLANNING AND ACTING UNDER UNCERTAINTY · Intermediate · 60 min



Motion planning, however, does not usually occur in the workspace. Instead, it occurs in the configuration space Q (also called C -space), the set of all robot configurations.

Howie Choset, Kevin Lynch, Seth Hutchinson, George Kantor, Wolfram Burgard, Lydia Kavraki, and Sebastian Thrun

Principles of Robot Motion, Chapter 1

Parts II through V taught Rusty to *know* things: where it is, what the room looks like, how sure it should be. This chapter is the first one that makes it *go somewhere*. It is also, deliberately, the least probabilistic chapter in Part VI. Uncertainty steps back for one chapter so you can master the deterministic skeleton of acting — configuration space, graph search, potential fields, sampling-based planners — before Chapters 21 through 24 wrap all of it back up in probability.

The word "probability" earns its place in the title twice, and the chapter is careful about the difference. First, randomness appears as an *algorithmic device*: PRM and RRT sample configurations the way the particle filter sampled states, and they trade an exact guarantee for one that holds with probability approaching one. Second, everything here is the deterministic core that the uncertainty-aware layers drive — the MDP of Chapter 21 generalizes the wave-front planner you will build below, and the MPPI controller of Chapter 23 tracks paths that come from this chapter.

There is one idea underneath all of it. A planner searches a space the robot never physically visits. Once you see obstacles as *C-obstacles* — subsets of the space of configurations rather than the space the robot moves through — every planner in the zoo collapses into the same two-line recipe: build a graph in the free part of that space, then search it.

🎯 AFTER THIS CHAPTER YOU CAN

- Reduce a robot with a shape to a point, by building the configuration space and its C-obstacles
- Prove *A* returns an optimal path under admissibility, and bound weighted *A* by ϵ
- Explain why an additive potential field must have traps, and why the wave-front cannot
- State probabilistic completeness and asymptotic optimality precisely — including their constants
- Implement RRT and RRT* in Rust so that the second is the first plus two operations
- Plan for a car: Dubins words, the U-turn worked by hand, and where hybrid A* fits

📖 ASSUMED

- SE(2) poses and the $\boxplus/\boxminus$ operators (Chapter 3)
- The Apartment world and parry2d collision queries (Chapter 4)
- Sampling as a representation of a set you cannot enumerate (Chapter 8)
- Occupancy grids as a planning substrate (Chapter 13) and distance fields (Chapter 19)

20.1 A goal is not a plan

Put Rusty at one end of a room, put a goal at the other, and write the first controller anyone writes: drive toward the goal, and push away from whatever you are about to hit. It is two lines of code, it needs no map beyond a range sensor, and it is exactly right — until it is catastrophically wrong.

🖥️ INTERACTIVE · w20.3

The Potential Well

“Just follow the gradient” is a planner that reports success by standing still.

Run this simulation in the web edition: [/chapters/ch20-motion-planning #w20.3](#)

Watch the bead until it stops. Then read the readout: the status is not *failed*, it is *stuck*, and the gradient norm is essentially zero. Nothing has gone wrong with the code. The controller has arrived at a point where the pull of the goal and the push of the wall cancel exactly, and by its own definition of success — descend

the potential — it has *succeeded*. It is standing in the middle of a cup, reporting that there is nowhere better to go, and it is telling the truth.

This is the failure mode that motivates every other algorithm in the chapter. A reactive controller knows the world only through its local gradient. Escaping the cup requires going *up* the potential — moving away from the goal for a while — and a rule that only ever descends can never decide to do that. To decide to do it, something has to have looked at the whole room first.

Now flip on **wave-front mode**. The shading changes, the identical greedy descent runs again, and the bead walks out of the cup and around the obstacle. Nothing about the controller changed; the field it descends did. That single toggle is the arc of this chapter compressed into one control, and by the end of the Foundation section you will know exactly why the second field cannot have a trap and the first one must.

💡 THE ONE SENTENCE VERSION

A planner is a search over the *configuration space* — the space of everything the robot could be, not the space it moves through — and the only real questions are how you build a graph in the free part of it and what guarantee you get back when you search that graph.

20.2 Four planners, one query

Before any formalism, watch the four algorithms this chapter derives race on a single query: from room A in the south-west corner of the Apartment to the bedroom in the north-east, out through one doorway, down the corridor, and in through another. Every planner sees the same map, the same start, the same goal, and the same sample budget.

🖥️ INTERACTIVE · w20.1

The Planner Arena

RRT does not find good paths — it finds feasible ones fast. Optimality is a separate contract, and RRT* is its price.

Run this simulation in the web edition: [/chapters/ch20-motion-planning #w20.1](#)

Four things are worth noticing, and each becomes a theorem later.

A* is exhaustive and it shows.** The blue wash is every lattice cell ***A has opened or closed — about eight hundred of them at the default resolution. It returns the best path the lattice contains, and it pays for that with memory proportional to the area it searched.

RRT answers first and answers badly. The tree lunges into open space, finds

a path, and stops improving. Its answer is jagged, it usually goes through a doorway a person would not have chosen, and depending on the seed it costs 14–32% more than A*'s. *That is not a bug; it is the contract. RRT promises feasibility*, fast, and nothing else.*

RRT* starts identical to RRT and then bends. Same samples, same seed, same first solution at the same iteration — and then the cost curve walks steadily downward as the rewiring ball shrinks. Optimality here is not a property of the answer at any moment. It is a limit.

PRM builds something none of the others build: a reusable structure. On this one query the roadmap is mostly wasted work. Ask a thousand queries in the same apartment and it is the only planner here that is not starting from scratch each time.

One number in the scoreboard should look wrong. RRT* ends up cheaper than A*'s "optimum" — around 14.0 m against 15.06 m. Both are correct. A* is optimal on the lattice, and the lattice is not the plane: an 8-connected grid can only travel in eight directions, so it pays an octile detour that a sampler placing nodes anywhere never pays. Keep that discrepancy in mind. It is the honest face of resolution completeness*, and we make it precise below.

20.3 The mathematics

$\mathcal{W} \subset \mathbb{R}^2$	Workspace — the physical space the robot moves through.
$\mathcal{W}O_i$	The i-th workspace obstacle. The free workspace is what is left after removing them all.
$\mathcal{Q}, q$	Configuration space and a configuration. For Rusty, $q = (x, y, \text{theta})$ in SE(2).
$R(q) \subset \mathcal{W}$	Footprint: the set of workspace points the robot occupies at configuration q .
$\mathcal{Q}O_i, \mathcal{Q}_{\text{free}}$	C-obstacle (configurations whose footprint hits obstacle i) and everything else.
$\tau : [0, 1] \rightarrow \mathcal{Q}_{\text{free}}$	A path: a continuous map into free space. Its cost is written $c(\tau)$.
$D(q)$	Clearance: distance from q to the nearest obstacle. This is Chapter 19's distance field.
$g(v), h(v), f(v)$	Cost-to-come, heuristic estimate of cost-to-go, and their sum — the A* priority.
$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q)$	Artificial potential: an attractive well at the goal plus a repulsive barrier at obstacles.
$r_n = \gamma (\log n / n)^{1/d}$	RRT* connection radius after n samples, with d the dimension of the configuration space.
ρ	Minimum turning radius of a car-like robot (Dubins and Reeds–Shepp).

δ

Clearance of a path — the margin used in every probabilistic-completeness statement.

One notational warning, made once and precisely. Throughout this book x_t is the *state the filter believes in* — a random variable with a distribution over it. In this chapter q is a *configuration the planner imagines* — a deterministic point it is considering, which the robot may never occupy. For Rusty the two are numerically the same triple (x, y, θ) , and it is tempting to conflate them. Do not. Chapter 22 plans over distributions rather than points, and the whole difficulty of that chapter is the difference between these two symbols.

20.3.1 The space the robot never visits

The robot has a shape. That single fact is what makes planning hard, because "does this path collide?" is a question about a swept *area*, not a point. The configuration-space construction makes the shape disappear.

💡 DEFINITION: CONFIGURATION SPACE

A **configuration** q is a complete specification of the robot's position: enough numbers to place every point of it. The **configuration space** Q is the set of all of them. For a disc robot in the plane $Q = \mathbb{R}^2$; for Rusty, whose heading matters, $Q = \text{SE}(2)$.

Write $R(q) \subset \mathcal{W}$ for the set of workspace points the robot occupies at configuration q . Then the obstacle in the workspace induces an obstacle in the configuration space:

$$QO_i = \{q \in Q : R(q) \cap \mathcal{WO}_i \neq \emptyset\}, \quad Q_{\text{free}} = Q \setminus \bigcup_i QO_i$$

A **path** is a continuous map $\tau : [0, 1] \rightarrow Q_{\text{free}}$ with $\tau(0) = q_{\text{start}}$ and $\tau(1) = q_{\text{goal}}$. A **trajectory** is a path parameterized by *time*, which is a strictly stronger object: it has velocities, and therefore dynamics. This chapter plans paths. Chapter 23 turns them into trajectories.

The payoff is immediate for the case Rusty is closest to.

📐 DERIVATION The C-obstacle of a disc robot is the Minkowski inflation

$$QO = \mathcal{WO} \oplus B_r$$

Statement. For a disc robot of radius r whose configuration is the position of its center,

$$QO = \mathcal{WO} \oplus B_r = \{w + b : w \in \mathcal{WO}, \|b\| \leq r\}$$

so planning for a disc among obstacles is *exactly* planning for a point among obstacles grown by r .

Step 1 — unfold the definition. At configuration q the footprint is the closed ball $R(q) = B_r(q)$. So $q \in QO$ if and only if there exists $w \in \mathcal{WO}$ with $\|w - q\| \leq r$.

Step 2 — rewrite as a distance. "There exists a point of the obstacle within r " is the statement $\text{dist}(q, \mathcal{WO}) \leq r$. Writing $D(q)$ for the distance from q to the nearest obstacle,

$$q \in Q_{\text{free}} \iff D(q) > r$$

Step 3 — recognize the Minkowski sum. $\{q : \text{dist}(q, \mathcal{WO}) \leq r\}$ is precisely the set of points expressible as an obstacle point plus a vector of length at most r , which is the definition of $\mathcal{WO} \oplus B_r$. ■

Why this matters computationally. Step 2 is the whole implementation. The free-space test is a single lookup into a scalar field D , and *you already built that field*: Chapter 19 computed the Euclidean signed distance transform of the map. Collision checking for a disc robot costs one array read.

What breaks for a robot that is not a disc. If the footprint rotates, $R(q)$ depends on θ and the C-obstacle lives in $\text{SE}(2)$, not $\mathbb{R}^2$. Its θ -slice is $\mathcal{WO} \oplus (-R(0, 0, \theta))$ — the obstacle grown by the *reflected* footprint at that heading — and that inflation genuinely changes as the robot turns. This is why a long rectangular robot can pass through a doorway diagonally that it cannot pass through square-on, and why the θ axis of C-space is not decoration. The practical compromise, used by nearly every 2-D stack including the one in this book, is to inflate by the *circumscribed* radius (conservative: never lets you plan a colliding path, sometimes refuses a feasible one) and fall back to an exact parry2d footprint test only in the tight places.

Everything downstream inherits this reduction. When a widget in this chapter says "free", it is asking whether $D(q) > r$; when it draws the C-obstacle, it is drawing the inflated map. Toggle **show the C-obstacle** in the Planner Arena and watch the doorways narrow: the doorway that matters to the planner is not the one in the wall, it is the one in the inflated map.

20.3.2 What it means to have solved the problem

Planners come with wildly different promises, and the differences are not marketing. State them crisply, because the rest of the chapter is a tour of who satisfies which.

💡 THE FOUR GUARANTEES

A planner is **complete** if it returns a solution in finite time whenever one exists, and reports failure otherwise. It is **resolution complete** if it is complete with respect to a discretization parameter: any solution that survives the discretization is found, and refining the discretization eventually captures any solution with positive clearance. It is **probabilistically complete** if the probability of failing to find an existing solution goes to zero as the number of samples grows. It is **asymptotically optimal** if the cost of its best solution converges to the optimal cost with probability one.

Note what each one does *not* say. Resolution completeness says nothing about a passage narrower than a cell — that passage does not exist as far as the planner is concerned, and refining the grid is the only cure. Probabilistic completeness says nothing about *when*: the failure probability decays, but as we will compute below, its constants can be so bad that the guarantee is worthless at any budget you would actually run. And asymptotic optimality is a statement about a limit, not about the answer on your screen.

Planner	Guarantee	Cost of that guarantee
Visibility graph, exact cell decomposition	Complete, and optimal for polygonal worlds	Exponential in dimension; needs exact geometry
A^* / Dijkstra on a lattice	Resolution complete; optimal <i>on the lattice</i>	Memory proportional to the searched area
Gradient descent on $U_{att} + U_{rep}$	None	Nearly free per step, and it lies to you
Wave-front / value iteration	Resolution complete; optimal on the lattice	One full sweep of the grid per goal
PRM, RRT	Probabilistically complete	Constants that depend on the narrowest passage
RRT $\backslash$, PRM $\backslash$, BIT $\backslash$ *	Probabilistically complete <i>and</i> asymptotically optimal	A shrinking-ball rewire on every sample

20.3.3 Search: Dijkstra, A^* , and what the heuristic actually buys

Discretize Q_{free} into an 8-connected lattice of cells and you have a graph $G = (V, E)$ with non-negative edge costs — one cell width for an orthogonal move, $\sqrt{2}$ cell widths for a diagonal. Now planning is shortest path, a solved problem since 1959.

The only interesting question is how much of the graph you have to touch.

ALGORITHM `a_star(G, s, g, h)` COST $O((V + E) \log V)$ worst case; expansions shrink as h approaches h^*
in graph G with non-negative costs, start s , goal g , heuristic h
out a minimum-cost path from s to g , or failure

```

1:  $g(s) \leftarrow 0$ ; OPEN  $\leftarrow \{s\}$  keyed on  $f(s) = h(s)$ ; CLOSED  $\leftarrow \emptyset$ 
2: while OPEN  $\neq \emptyset$  do
3:    $v \leftarrow \arg \min_{u \in \text{OPEN}} f(u)$ ; move  $v$  to CLOSED
4:   if  $v = g$  then return the path recovered through the parent pointers
5:   for each neighbor  $w$  of  $v$  with  $w \notin \text{CLOSED}$  do
6:      $t \leftarrow g(v) + c(v, w)$ 
7:     if  $t < g(w)$  then  $g(w) \leftarrow t$ ; parent( $w$ )  $\leftarrow v$ ; insert  $w$  into OPEN
       with  $f(w) = g(w) + \varepsilon h(w)$ 
8: return failure

```

Setting $\varepsilon = 0$ deletes the heuristic and line 3 pops in order of g alone: that is Dijkstra. Setting $\varepsilon = 1$ is A^* . Setting $\varepsilon > 1$ is weighted A^* , which is faster and no longer optimal — by a factor you can bound exactly.

The heuristic must satisfy one property to keep the answer honest.

💡 DEFINITION: ADMISSIBLE AND CONSISTENT

Let $h^*(v)$ be the true optimal cost-to-go from v . A heuristic is **admissible** if $h(v) \leq h^*(v)$ for every v — it never overestimates. It is **consistent** (or monotone) if $h(u) \leq c(u, v) + h(v)$ for every edge and $h(g) = 0$ — a triangle inequality on the graph. Consistency implies admissibility.

On an 8-connected lattice with unit-square cells the natural choice is the **octile distance**, which is the exact cost-to-go in an *empty* grid:

$$h(v) = \left\lceil \max(\Delta i, \Delta j) + (\sqrt{2} - 1) \min(\Delta i, \Delta j) \right\rceil \cdot \Delta x$$

It is admissible because obstacles can only make the true path longer, and consistent because it is itself a metric that lower-bounds every edge cost.

📐 DERIVATION A^* returns an optimal path when h is admissible

$g(\text{goal}) = C^*$ on first expansion

Statement. If h is admissible, then the first time A^* selects the goal at

line 3, its recorded cost $g(g)$ equals the optimal cost C^* . If h is additionally consistent, no node is ever expanded twice.

Step 1 — suppose otherwise. Assume A^* is about to expand g with $f(g) = g(g) > C^*$ (since $h(g) = 0$, $f(g) = g(g)$). We derive a contradiction with line 3's choice.

Step 2 — find a witness on the optimal path. Let $\pi^* = (s = v_0, v_1, \dots, v_k = g)$ be an optimal path. Not every v_i can be in CLOSED, or g itself would already carry the optimal cost. Let v_i be the *first* node of π^* still in OPEN.

Step 3 — the witness carries its optimal cost-to-come. Every node before v_i on π^* is closed, and each was relaxed toward its successor at line 7 when it was expanded. Inductively $g(v_{i-1}) = g^*(v_{i-1})$, and the relaxation of the edge (v_{i-1}, v_i) therefore left $g(v_i) \leq g^*(v_{i-1}) + c(v_{i-1}, v_i) = g^*(v_i)$. Since g values are always the cost of *some* real path, $g(v_i) = g^*(v_i)$.

Step 4 — admissibility caps the witness's priority.

$$f(v_i) = g^*(v_i) + h(v_i) \leq g^*(v_i) + h^*(v_i) = C^*$$

using $h(v_i) \leq h^*(v_i)$ and the fact that v_i lies on an optimal path.

Step 5 — contradiction. So $f(v_i) \leq C^* < f(g)$, and v_i is in OPEN. Line 3 selects the minimum- f node, so it would have selected v_i , not g . ■

The consistency half. Admissibility alone leaves a hole: a node may be closed with a *suboptimal* g and need reopening later, which is why line 5's "skip if closed" is unsafe for a merely-admissible heuristic. Consistency plugs it. Along any edge (u, v) relaxed from u ,

$$f(v) = g(u) + c(u, v) + h(v) \geq g(u) + h(u) = f(u)$$

so f is non-decreasing along the sequence of expansions. A node is therefore expanded only after every node that could improve it, and its g at expansion time is already final. The octile heuristic is consistent, so the closed-set skip in our implementation is safe.

The classic counterexample. Make h optimistic everywhere except at one node where it overestimates by a lot, and A^* will delay expanding that node until after it has committed to a worse path through the goal. Exercise 2 asks you to build a four-node instance of exactly this and watch the crate's own `a_star_grid` return the wrong answer.

Weighted A^* is bounded, not wild. With h admissible and priority $f = g + \varepsilon h$ for $\varepsilon \geq 1$, repeat Steps 2–5. When the goal is popped, $g(g) \leq f_\varepsilon(v_i) = g^*(v_i) + \varepsilon h(v_i) \leq \varepsilon(g^*(v_i) + h^*(v_i)) = \varepsilon C^*$. So the returned path is never worse than ε times optimal.

That bound is loose in practice, which is exactly why weighted A^* is everywhere in real stacks. Here is the Apartment query, measured on the same

lattice the arena uses (0.2 m cells, 0.25 m robot radius, start (1, 1), goal (11, 8)):

ε	Path cost	Expansions	Bound says
0 (Dijkstra)	15.057 m	1883	—
1 (A^*)	15.057 m	792	≤ 15.057 m
1.5	15.057 m	261	≤ 22.6 m
2	15.057 m	373	≤ 30.1 m
3	15.222 m	374	≤ 45.2 m
5	15.554 m	392	≤ 75.3 m

Read three lessons off it. The heuristic buys expansions and nothing else: A^* and Dijkstra return byte-identical paths, and A^* touches 42% as many cells. Weighting further is startlingly cheap — at $\varepsilon = 1.5$ the search expands a *third* as many cells as A^* and still happens to return the optimal path. And the theoretical bound is nowhere near tight: at $\varepsilon = 5$ the guarantee permits a five-fold blowup and the measured penalty is 3.3%.

The non-monotonicity between $\varepsilon = 1.5$ and $\varepsilon = 2$ is real and worth a moment. Weighted A^* is not a smooth dial. Raising ε changes which ties break which way, and a greedier search can wander into a dead-end room and pay to come back out. The only honest way to pick ε is to measure it on your map.

20.3.4 Potential fields, and why they must die

The controller from the hook, written down properly. Khatib’s 1986 formulation puts a quadratic well at the goal — conic far away, so a distant goal does not produce an absurd pull:

$$U_{\text{att}}(q) = \begin{cases} \frac{1}{2}\zeta d^2(q, q_{\text{goal}}) & d \leq d_{\text{goal}}^* \\ d_{\text{goal}}^* \zeta d(q, q_{\text{goal}}) - \frac{1}{2}\zeta \left(d_{\text{goal}}^*\right)^2 & d > d_{\text{goal}}^* \end{cases}$$

and a barrier at every obstacle that switches off entirely beyond a radius of influence Q^* :

$$U_{\text{rep}}(q) = \begin{cases} \frac{1}{2}\eta \left(\frac{1}{D(q)} - \frac{1}{Q^*} \right)^2 & D(q) \leq Q^* \\ 0 & D(q) > Q^* \end{cases}$$

Note $D(q)$: the repulsive term is a function of the *distance field*, so its gradient is just the direction away from the nearest obstacle, scaled. Chapter 19’s ESDF is doing all the geometric work again.

ALGORITHM `gradient_descent_potential(q-s, U,  $\alpha$ )` COST $O(1)$ per step for a distance-field potential

in a start configuration, the potential U , a step size

out a path to the goal, or the announcement that it has stopped

```

1:  $q \leftarrow q_s; \tau \leftarrow (q)$ 
2: while  $\|\nabla U(q)\| > \epsilon$  do
3:    $q \leftarrow q - \alpha \frac{\nabla U(q)}{\|\nabla U(q)\|}$ 
4:   append  $q$  to  $\tau$ 
5: return  $\tau$ 

```

Line 2 is the whole tragedy. The loop terminates when the gradient vanishes — and the gradient vanishes at the goal *and at every other critical point of U* . The algorithm cannot tell the two apart, because locally they are the same event.

DERIVATION An additive potential must have critical points that are not the goal

$$\nabla U = 0 \text{ at some } q \neq q_{\text{goal}}$$

Statement. $U = U_{\text{att}} + U_{\text{rep}}$ generally admits critical points in Q_{free} other than the goal, and for a non-convex obstacle some of them are local minima. No choice of ζ, η, Q^* removes them.

Step 1 — construct the trap. Take the widget's cup: a back wall at $x = 4.6$ spanning $y \in [1.4, 3.6]$, with arms along $y = 1.4$ and $y = 3.6$ reaching back toward the robot. Put the goal to the right of it, on the line $y = 2.5$, and start the robot to the left on the same line.

Step 2 — kill the transverse component by symmetry. The obstacle set is symmetric about $y = 2.5$ and so is the goal, hence $U(x, 2.5+s) = U(x, 2.5-s)$. A smooth even function has zero derivative at its center, so $\partial U / \partial y = 0$ everywhere on the axis. The motion is one-dimensional.

Step 3 — find the zero of the remaining component. Along the axis, $\partial U_{\text{att}} / \partial x < 0$ (the goal pulls right, with magnitude bounded below by ζd_{goal}^* far from the goal), while $\partial U_{\text{rep}} / \partial x > 0$ near the back wall and grows without bound as $D \rightarrow 0$. It is exactly 0 outside the radius of influence. A continuous function that is negative at $x = Q^*$ -away-from-the-wall and $+\infty$ at the wall has a zero in between.

Step 4 — confirm it is a minimum, not a saddle. At that zero, $\partial^2 U / \partial x^2 > 0$ because the repulsive term's second derivative dominates ($\propto D^{-4}$). Transversally, the two arms of the cup push *inward* from both sides, so $\partial^2 U / \partial y^2 > 0$ as well. The Hessian is positive definite: a genuine local minimum inside free space, from which every direction is uphill. ■

Why no tuning saves you. Raising η moves the trap, it does not delete it — Step 3's argument only used a sign change. Try it: the widget's η slider spans a factor of 40, and the bead dies for every value.

The topological version. This is not bad luck with a formula, it is Morse theory. On a compact free space with holes, any smooth function has a number of critical points constrained by the topology; you cannot have exactly one. The best achievable object is a **navigation function** — a potential whose only critical points besides the goal are non-degenerate *saddles*, whose basins of attraction have measure zero, so almost every start still reaches the goal. Koditschek and Rimon (1990) constructed these for "sphere worlds" and extended them by diffeomorphism to star-shaped worlds. The construction is beautiful and it does not generalize to an arbitrary floorplan, which is why almost nobody ships it.

20.3.5 The cure you have already built

There is a navigation function for an arbitrary floorplan, it is trivial to compute, and you have met it twice already. Run Dijkstra *backwards from the goal* over the whole lattice, with no early exit, and label every free cell with its cost-to-go.

ALGORITHM `wavefront(grid, q_goal)` COST $O(|C| \log |C|)$ for $|C|$ cells -- one sweep,
then every query is free
in an occupancy lattice and a goal cell
out $V(c)$ = cost-to-go for every free cell

- 1: $V(c) \leftarrow \infty$ for all cells c ; $V(q_{\text{goal}}) \leftarrow 0$
- 2: push q_{goal} onto a min-priority queue keyed on V
- 3: while the queue is non-empty do
- 4: pop the cheapest cell c
- 5: for each free neighbor c' of c do
- 6: if $V(c) + w(c, c') < V(c')$ then $V(c') \leftarrow V(c) + w(c, c')$ and push c'
- 7: return V

Every free cell except the goal now has a neighbor with strictly smaller V — that is not a heuristic claim, it is the construction: $V(c)$ was assigned by relaxing an edge from the predecessor on its shortest path, and that predecessor has V smaller by the edge weight. So greedy descent on V *cannot* get stuck, and the wave-front is a discrete navigation function. The self-check wavefront: every free cell has a strictly cheaper neighbour verifies this over all 1953 reachable cells of the Apartment.

Go back to the Potential Well and toggle wave-front mode with that in mind.

The bead does not get smarter. The field it descends stopped being an invention and started being a *computed answer*.

💡 YOU HAVE JUST DONE VALUE ITERATION

V is a value function: the cost-to-go from every state under the optimal policy, and greedy descent on it is the optimal policy. Chapter 21 replaces "distance" with "expected discounted reward", replaces the deterministic neighbor with a transition distribution, and solves the same recursion — and gets the same guarantee for the same reason.

One more identity, and it closes a loop from the previous chapter. Run the identical Dijkstra with *every obstacle cell* as a source instead of the goal, and you get the distance from each free cell to the nearest obstacle. Choset calls that the **brushfire** algorithm and pictures it as a wave washing out from the obstacles. It is the Euclidean distance transform of Chapter 19, computed by a different route — approximately, because the 8-connected lattice can only measure length in octile steps, and the octile metric overestimates Euclidean distance by up to $\cos 22.5^\circ + (\sqrt{2} - 1) \sin 22.5^\circ = 1.0824$, i.e. 8.24%, attained at exactly 22.5° . The self-check pins both error sources: on a 0.1 m lattice, brushfire never *under*-estimates the true distance and never exceeds $1.0824 D + 2\sqrt{2} \Delta x$.

Brushfire is the same algorithm as the wave-front is the same algorithm as Dijkstra. Only the source set changes. That is the kind of collapse worth remembering.

20.3.6 Sampling: build the graph out of luck

Grids die of dimension. A 3-D pose lattice at 5 cm and 5° is already 20 million cells; a 6-DOF arm is hopeless. The move that rescues planning is the same one Chapter 8 made for beliefs: stop trying to represent Q_{free} and *sample* it.

ALGORITHM `build_prm(n, k, sample, connect)` COST $O(n \log n)$ nearest-neighbor work plus $n \cdot k$ local plans
in a sample budget n , a neighbor count k , a sampler, a local planner
out a roadmap graph G whose vertices are collision-free milestones

```

1:  $V \leftarrow \emptyset; E \leftarrow \emptyset$ 
2: for  $i = 1$  to  $n$  do
3:    $q \leftarrow \text{sample until } q \in Q_{\text{free}}; V \leftarrow V \cup \{q\}$ 
4:   for each  $q'$  among the  $k$  nearest neighbors of  $q$  in  $V$  do
5:     if the local planner connects  $q$  to  $q'$  without collision then
```

- $E \leftarrow E \cup \{(q, q')\}$
- 6: **query** (q_s, q_g) : connect both to their nearest milestones, then run Dijkstra on G
- 7: return the roadmap, which answers every later query for free

The learning phase and the query phase are separate on purpose. That separation is PRM's reason to exist — and, on the single query in the arena, its handicap.

Now the guarantee. It is worth stating with its constants visible, because the constants are the part that bites.

DERIVATION Probabilistic completeness of PRM, with constants

$$P(\text{fail}) \leq a e^{-bn}$$

Statement. Suppose the query admits a path τ of length L with **clearance** $\delta > 0$ — every point of τ is at least δ from the boundary of free space. Then a uniform-sampling PRM with n milestones and a straight-line local planner of reach at least $\delta/2$ fails to answer the query with probability at most

$$P(\text{fail}) \leq \underbrace{\lceil 4L/\delta \rceil}_a \exp \left(- \underbrace{\frac{\pi \delta^2}{16 \mu(Q_{\text{free}})}}_b n \right)$$

in the plane, where μ is area. The same exponential form holds for RRT.

Step 1 — cover the path with balls. Place centers $c_0, \dots, c_m$ along τ at arc-length spacing $\delta/4$, so $m = \lceil 4L/\delta \rceil$. Around each, take the ball B_k of radius $\delta/4$. Each B_k lies entirely in Q_{free} , because its center is on τ and $\delta/4 < \delta$.

Step 2 — samples in consecutive balls are connectable. Take $p \in B_k$ and $p' \in B_{k+1}$. Then $\|p' - c_k\| \leq \|p' - c_{k+1}\| + \|c_{k+1} - c_k\| \leq \delta/4 + \delta/4 = \delta/2$, and likewise $\|p - c_k\| \leq \delta/4$. The segment $\overline{pp'}$ lies in the convex hull of two points both within $\delta/2$ of c_k , so every point of it is within $\delta/2 < \delta$ of $c_k \in \tau$ — hence free. The local planner succeeds.

Step 3 — a chain of hits is a solution. If every ball receives at least one milestone, the milestones $p_0 \in B_0, \dots, p_m \in B_m$ form a connected chain in the roadmap from a neighborhood of q_s to a neighborhood of q_g . The roadmap answers the query.

Step 4 — bound one ball's failure. A uniform sample lands in B_k with probability $\rho = \mu(B_k)/\mu(Q_{\text{free}}) = \pi(\delta/4)^2/\mu(Q_{\text{free}})$. After n independent samples, $P(B_k \text{ empty}) = (1 - \rho)^n \leq e^{-\rho n}$.

Step 5 — union bound. $P(\text{some ball empty}) \leq m e^{-\rho n}$, which is the claim with $a = m$ and $b = \rho$. ■

The clearance is load-bearing. If $\delta = 0$ — the only path scrapes a wall — then $b = 0$ and the bound says nothing, forever. No amount of sampling finds a passage of zero volume, because uniform sampling hits a set with probability equal to its measure and that measure is zero. This is not a weakness of the proof; it is a true statement about the algorithm.

The expansiveness refinement. Choset's treatment replaces "clearance" with $(\epsilon, \alpha, \beta)$ -**expansiveness**, which measures how much of the free space each point can see. It gives the same exponential decay with constants that degrade as visibility shrinks, and it explains why the narrow-passage problem is really a visibility problem: a corridor is hard not because it is small but because almost nothing outside it can see into it.

INTERACTIVE · w20.2

The Narrow Passage

Uniform random sampling does not see everything: it sees a passage with probability proportional to the passage's volume.

Run this simulation in the web edition: [/chapters/ch20-motion-planning #w20.2](#)

Now do the arithmetic the theorem invites, on the widget's own geometry. At a corridor width of 0.65 m the free area is $\mu = 51.2 \text{ m}^2$, Rusty's radius shrinks the clear width to 0.35 m so the best path has clearance $\delta = 0.175 \text{ m}$, and the route is about 8 m long. That gives $a = 183$ and $b = 1.18 \times 10^{-4}$. At the widget's budget of $n = 180$ milestones, the bound evaluates to 179 — a probability bound larger than one, which is to say: *the theorem tells you nothing at all*. To force the bound below 0.1 you would need about **64,000** samples. At a corridor width of 0.35 m, you would need about **3.9 million**.

Meanwhile, measure it. At $n = 180$ the roadmap connects the two rooms in roughly one run in six — the widget's 30-trial sweep reports 13% — and at $n = 800$ it succeeds 90% of the time. So the experiment reaches 90% success at **800** samples and the theorem promises 90% at **64,000**: the guarantee is right, and it is eighty times too slow to be useful. Shrink the corridor to 0.35 m and the

experiment reports 0% at any budget you will wait for, which is the same fact seen from the other side.

⚠ WHERE ROBOTS BREAK THIS

This gap is the single most important practical fact about sampling-based planning. Probabilistic completeness is an asymptotic statement whose constants depend on $\delta^d / \mu(Q_{\text{free}})$ — a ratio that collapses as passages narrow and as dimension grows. "It is probabilistically complete" is never an answer to "will it work in my apartment at 20 Hz."

The escape is not more samples, it is *better-placed* ones. Toggle bridge-test sampling and watch the success curve stop collapsing. The bridge test of Hsu et al. (2003) draws a point, draws a second point a Gaussian step away, and keeps the **midpoint** only when both endpoints are in collision and the midpoint is free. That configuration — free point straddled by two blocked ones — is a geometric *signature* of a narrow passage rather than a bet on its volume, which is why the green curve in the widget holds at 83% where the blue one has fallen to 13%. It is also why pure bridge sampling is useless on its own: it finds passages and nothing else, so the library mixes it 60/40 with uniform samples, exactly as Hsu et al. prescribe.

20.3.7 RRT: growing into the space instead of covering it

PRM builds a graph for all queries. RRT builds a tree for one, and it grows it by repeatedly throwing a dart and reaching toward wherever it lands.

ALGORITHM `rrt_extend(T, q_rand)` COST $O(\log n)$ nearest-neighbor query per iteration with a k-d tree
 in a tree T rooted at the start, a random configuration
 out reached / advanced / trapped

- 1: $q_{\text{near}} \leftarrow$ the vertex of T nearest to q_{rand}
- 2: $q_{\text{new}} \leftarrow$ steer from q_{near} toward q_{rand} , at most ϵ
- 3: if the edge $(q_{\text{near}}, q_{\text{new}})$ is not collision-free then return **trapped**
- 4: add q_{new} to T with parent q_{near}
- 5: if $q_{\text{new}} = q_{\text{rand}}$ then return **reached** else return **advanced**

Line 1 is why it works. Nearest-neighbor selection makes the probability of extending from a given vertex proportional to the volume of that vertex's Voronoi cell, so the tree is *biased toward unexplored territory* by construction — the "rapidly exploring" in the name is a Voronoi bias, not a metaphor. Add a small probability

(5% in the arena) of setting $q_{\text{rand}} = q_{\text{goal}}$ and the tree also remembers what it is for.

Line 4 is why it fails. A vertex's parent is chosen once, at insertion, from whatever was nearest at the time — and it is never reconsidered. The tree commits to whichever edges were cheap to reach early, and those commitments become the skeleton of every path it will ever return.

💡 RRT DOES NOT CONVERGE TO THE OPTIMUM. EVER.

Karaman and Frazzoli proved something stronger than "RRT is suboptimal in practice": $P(\lim_{n \rightarrow \infty} c_n^{\text{RRT}} = c^*) = 0$. The limit exists — the cost stops improving almost immediately — and it is almost surely *not* optimal. Watch the arena: RRT's line in the cost chart is flat after its first solution, because the only way it can improve is to stumble on a shorter route to the goal region through the tree it already froze.

20.3.8 RRT*: what optimality costs

The repair is two operations, inserted between lines 3 and 4, both confined to a ball of radius r_n around the new node.

ALGORITHM `rrt_star(q_s, q_g, n)`

COST $O(n \log n)$ expected total

in start, goal, sample budget

out a tree whose best solution converges to the optimum

- 1: as `rrt_extend`, lines 1–3, producing q_{new}
- 2: $r_n \leftarrow \min(\gamma(\log n/n)^{1/d}, r_{\text{max}})$; $Q_{\text{near}} \leftarrow$ vertices within r_n of q_{new}
- 3: **choose parent**: attach q_{new} to the $q \in Q_{\text{near}}$ minimizing $\text{cost}(q) + \|q - q_{\text{new}}\|$ over collision-free edges
- 4: **rewire**: for each $q \in Q_{\text{near}}$, if $\text{cost}(q_{\text{new}}) + \|q_{\text{new}} - q\| < \text{cost}(q)$ and the edge is free, re-parent q to q_{new}
- 5: propagate the new cost through q 's subtree
- 6: update the incumbent solution if q_{new} reaches the goal region more cheaply

Line 5 is the step everyone forgets. Re-parenting a node changes the cost-to-come of everything below it, and a tree whose costs are stale still *looks* rewired while silently optimizing the wrong objective. Our implementation pushes the update down the subtree, and the self-check `rrt*`: cost-to-come is consistent through every edge after rewiring asserts $\text{cost}(v) = \text{cost}(\text{parent}(v)) + \|v - \text{parent}(v)\|$ at *every* node of the tree, to machine precision, after a full run. It is the invariant

most worth testing in this chapter, because it is the one whose violation is invisible.

DERIVATION Asymptotic optimality of RRT*, and what γ has to be

$P(\lim_n c_n = c^*) = 1$, for γ above the bound

Statement (Karaman and Frazzoli 2011). Let $d = \dim Q$, let ζ_d be the volume of the unit d -ball, and let the connection radius be $r_n = \gamma (\log n/n)^{1/d}$ with

$$\gamma > 2 \left(1 + \frac{1}{d}\right)^{1/d} \left(\frac{\mu(Q_{\text{free}})}{\zeta_d}\right)^{1/d}$$

Then RRT* is asymptotically optimal: $P(\lim_{n \rightarrow \infty} c_n^{\text{RRT}^*} = c^*) = 1$, provided c^* is **robustly** optimal — there is a sequence of strongly δ -clear paths whose costs converge to c^* .

Step 1 — why RRT cannot and RRT* can. RRT's tree is a subgraph chosen greedily and never revised, so its path costs are determined by an early, arbitrary commitment. RRT* instead maintains the shortest-path tree of the r_n -disc graph on the sampled points. That is a different object: it depends only on which points were sampled, not on the order.

Step 2 — the disc graph must stay connected. The expected number of samples inside a ball of radius r_n is

$$n \cdot \frac{\zeta_d r_n^d}{\mu(Q_{\text{free}})} = \frac{\zeta_d \gamma^d}{\mu(Q_{\text{free}})} \log n$$

which grows like $\log n$. This is the classical connectivity threshold for random geometric graphs: if the constant multiplying $\log n$ exceeds a critical value, the graph is connected with probability tending to one, and Borel–Cantelli upgrades that to *almost surely, eventually*. The lower bound on γ is precisely the statement that the constant clears the threshold.

Step 3 — the radius shrinks slowly enough to stay useful and fast enough to stay cheap. $r_n \rightarrow 0$, so the disc graph's edges converge to straight-line segments in Q_{free} and its shortest path converges to a true shortest path. Meanwhile $|Q_{\text{near}}| = O(\log n)$ per iteration, so the total work is $O(n \log n)$ rather than $O(n^2)$: optimality costs a logarithm, not a polynomial.

Step 4 — robustness is not a technicality. If the optimal path must thread a gap of exactly zero clearance, no sequence of δ -clear paths approaches it

and the theorem does not apply — nor should it, since the completeness proof already showed that sampling cannot find a path of zero clearance at all.

What the widget actually runs. For the Apartment with a 0.25 m disc, the free area is $\mu \approx 78.2 \text{ m}^2$, $d = 2$, $\zeta_2 = \pi$, so the bound is

$$\gamma > 2 \left(\frac{3}{2}\right)^{1/2} \left(\frac{78.2}{\pi}\right)^{1/2} = 12.22$$

The arena runs $\gamma = 12.3$ — above the bound, which most implementations quietly are not. It also caps r_n at 2 m so the first few hundred iterations do not try to rewire the entire tree; the cap is inactive by $n \approx 300$ and therefore does not touch the limit.

The 2020 correction. Solovey, Janson, Schmerling, Frazzoli and Pavone found a logical gap in the original proof — it treats as independent a family of events that the sequential construction correlates — and gave a rigorous replacement. Their corrected rate is $r_n = \gamma'(\log n/n)^{1/(d+1)}$: the extra dimension accounts for the *ordering* of the samples, and since $\log n/n < 1$ this radius is strictly *larger* than Karaman and Frazzoli's. The practical reading is reassuring — RRT* is asymptotically optimal, and if you want the theorem rather than the folklore you should connect a little more generously than the 2011 formula says.

20.4 Rusty is not a point

Every planner so far returned a polyline. Rusty cannot drive a polyline. A differential-drive robot can spin in place, so it can technically follow one — badly, stopping at every vertex — but a car-like robot cannot follow one at all, and even Rusty's velocity controller (Chapter 9) turns each corner into a slow, error-accumulating pivot.

The constraint is that a wheel cannot slide sideways. In the plane that is one linear equation on the velocity:

$$\dot{x} \sin \theta - \dot{y} \cos \theta = 0$$

This is a **Pfaffian** constraint: linear in the velocities, and *non-integrable* — there is no function $F(x, y, \theta)$ whose level sets it defines. That non-integrability is what makes it **nonholonomic**, and it has a precise and initially surprising consequence: the constraint removes no configurations at all. A car can reach any pose in the plane; parallel parking is the constructive proof. What the constraint restricts is the *set of paths*, not the set of destinations.

So the local planner has to change. Instead of a straight line between two configurations, we need the shortest *drivable* curve — and for a car with a minimum turning radius, that curve has a closed form.

```

ALGORITHM dubins_shortest_path( $q_0, q_1, \rho$ )      COST 0(1) -- six closed-form
evaluations, no search
in two poses in SE(2) and a minimum turning radius
out the shortest forward-only path, as one of six words

```

- 1: normalize: rotate so $q_1 - q_0$ lies along $+x$, scale by ρ , giving (α, β, d)
- 2: for each word $w \in \{LSL, RSR, LSR, RSL, RLR, LRL\}$ do
- 3: evaluate w 's closed form; skip it if the discriminant is negative (that word does not exist for this query)
- 4: record its total length $(t + p + q) \rho$
- 5: return the shortest word found

Dubins proved in 1957 that the optimum is always one of exactly six words built from three primitives — L (left at full lock), R (right at full lock), S (straight). The reason is Pontryagin's maximum principle: the steering that minimizes time is bang-bang, so the wheel is always hard over or dead ahead, and enumerating the possible switch structures leaves only CSC and CCC patterns.

INTERACTIVE · w20.4

The Dubins Dial

A car's shortest path is not turn-straight-turn: at close range the three-arc words LRL and RLR win.

Run this simulation in the web edition: </chapters/ch20-motion-planning#w20.4>

20.4.1 A worked example you can check by hand

Take the hardest query a car ever faces: turn around on the spot. Start at the origin facing east, finish at the origin facing west, with $\rho = 1$:

$$q_0 = (0, 0, 0), \quad q_1 = (0, 0, \pi), \quad \rho = 1$$

The "obvious" answer is turn, drive, turn. Evaluate LSL : the left-turn circle at the start is centered at $(0, 1)$ and the one at the goal at $(0, -1)$, so the straight run is the $2\rho = 2$ between their centers, and each turn is three-quarters of a circle. Segments $(3\pi/2, 2, 3\pi/2)$, total

$$c_{LSL} = 3\pi + 2 \approx 11.4248$$

RSR ties it by reflection. LSR and RSL do not exist at all: an internal tangent needs the two circles at least 2ρ apart, and here the start's left circle and the goal's right circle are the *same* circle. Their discriminants come out negative, which is exactly how the closed form reports "no such word".

Now the three-arc answer. Three unit circles are involved: the right-turn circle at the start, at $C_1 = (0, -1)$; the right-turn circle at the goal, at $C_3 = (0, +1)$; and a middle circle C_2 that must be tangent to both, so $\|C_2 - C_1\| = \|C_2 - C_3\| = 2\rho = 2$. With C_1 and C_3 two apart, the solution is $C_2 = (\pm\sqrt{3}, 0)$ — **the three centers form an equilateral triangle of side 2ρ** . Take $C_2 = (\sqrt{3}, 0)$.

The tangent points are the midpoints of the triangle's sides: $(\frac{\sqrt{3}}{2}, -\frac{1}{2})$ and $(\frac{\sqrt{3}}{2}, +\frac{1}{2})$. Measured at C_1 , the start $(0, 0)$ sits at 90° and the first tangent point at 30° , so the first right arc turns $60^\circ = \pi/3$. By symmetry the last arc is $\pi/3$. Measured at C_2 , the two tangent points sit at 210° and 150° , and the middle arc is a *left* turn, so it goes the long way round: $360^\circ - 60^\circ = 300^\circ = 5\pi/3$. Total:

$$c_{RLR} = \frac{\pi}{3} + \frac{5\pi}{3} + \frac{\pi}{3} = \frac{7\pi}{3} \approx 7.3304$$

The three-arc word wins by 36%. *LRL* ties it by reflection. Check the heading bookkeeping: the net turn is $-60^\circ + 300^\circ - 60^\circ = 180^\circ$, as required. This is the fact the Dubins Dial exists to make visceral — bring the two poses together and the answer stops being turn–straight–turn.

RUST crates/ch20_planning/src/steer.rs (tests)

```
1  #[test]
2  fn worked_example_ch20_dubins_u_turn() {
3      use std::f64::consts::PI;
4      let q0 = Isometry2::new(Vector2::zeros(), 0.0);
5      let q1 = Isometry2::new(Vector2::zeros(), PI);
6
7      let best = dubins_shortest_path(&q0, &q1, 1.0).expect("a U-turn is always feasible");
8
9      // The CCC word wins, and its arcs are pi/3, 5pi/3, pi/3 -- the equilateral
10     // triangle of turning-circle centers, worked by hand in the text.
11     assert!(matches!(best.word, Word::RLR | Word::LRL));
12     assert_relative_eq!(best.segments[0], PI / 3.0, epsilon = 1e-12);
13     assert_relative_eq!(best.segments[1], 5.0 * PI / 3.0, epsilon = 1e-12);
14     assert_relative_eq!(best.length, 7.0 * PI / 3.0, epsilon = 1e-12);
15
16     // ...and the CSC alternative really is 56% longer, not merely "longer".
17     let lsl = dubins_word(Word::LSL, &q0, &q1, 1.0).unwrap();
18     assert_relative_eq!(lsl.length, 3.0 * PI + 2.0, epsilon = 1e-12);
19     assert!(dubins_word(Word::LSR, &q0, &q1, 1.0).is_none()); // circles coincide
20 }
```

Allowing reverse gives the **Reeds–Shepp** family: 48 word classes, with up to two cusps where the car changes direction. It is never worse than Dubins and often much better at close range, which is what the widget's reverse toggle shows. Our library implements only the pure-reverse subset — plan forwards with both headings flipped, then drive the curve backwards — and says so, because a genuine Reeds–Shepp planner also allows switching direction *mid-path*, which the subset cannot express.

20.4.2 Hybrid A*, at recipe level

Dubins gives us a drivable *edge*. To get a drivable *plan* in a real map, the DARPA Urban Challenge produced a construction that is now the default in every parking-lot planner. It is a recipe rather than a theorem, and this chapter presents it as one.

Search an (x, y, θ) lattice with A*, but keep a **continuous** state in each cell instead of snapping to the cell center: expand by integrating a small set of motion primitives (full left, full right, straight — forwards and backwards) from the actual* pose stored there. Keep only the cheapest continuous state per cell, which is what makes the search finite. Near the goal, attempt an **analytic expansion**: a single Dubins or Reeds–Shepp shot to the goal pose, accepted if it is collision-free. And drive it with a dual heuristic,

$$h = \max \left(h_{\text{nonholonomic, no obstacles}}, h_{\text{holonomic, with obstacles}} \right)$$

The first term is a precomputed Reeds–Shepp lookup table that knows about the turning radius but not the walls; the second is exactly the wave-front from earlier in this chapter, which knows about the walls but not the turning radius. Neither dominates, both are admissible, and their maximum is admissible too — so the dual heuristic is *free* accuracy. Dolgov, Thrun, Montemerlo and Diebel report roughly an order-of-magnitude reduction in expansions from that maximum alone.

The result is not optimal — the continuous-state-per-cell pruning discards states that could have been part of the optimum — but it is drivable, fast, and it is what is under the hood when a car parks itself.

i NOTE

Where D* went. Choset devotes real space to D* and D* Lite, which repair an existing search when the costmap changes rather than replanning from scratch. Modern stacks mostly do not: costmaps change everywhere at sensor rate, incremental repair loses its advantage when the change is not local, and a receding-horizon controller (Chapter 23) absorbs the reactive role anyway. The contemporary answer, and the one the ROS 2 navigation maintainers document, is “re-run A* or hybrid A* at costmap rate and let a sampling-based controller handle the 20 Hz.” Know that D* exists, and reach for it only when your map updates are genuinely sparse.

20.5 Implementation in Rust

Three types carry the chapter: the configuration space, the local planner, and the tree. Everything else is a function of them.

RUST crates/ch20_planning/src/cspace.rs

```

1 use nalgebra::{Isometry2, Point2, Vector2};
2 use parry2d_f64::query::intersection_test;
3 use parry2d_f64::shape::{ConvexPolygon, Polyline};
4
5 use ch19_maps::Esdf; // Chapter 19's Euclidean distance transform
6 use sim::World;      // Chapter 4's Apartment
7
8 /// Configuration space of a planar robot over a known map.
9 ///
10 /// The whole chapter rests on one reduction (derivation F.1): a disc of radius
11 /// `r` is free at `q` exactly when the distance to the nearest obstacle exceeds
12 /// `r`. So `is_free` never touches geometry -- it reads the distance field that
13 /// Chapter 19 already built, at 0(1).
14 pub struct CSpace2 {
15     /// D(q). Also the clearance the potential field differentiates, and the
16     /// cost term the integration lab weights paths by.
17     esdf: Esdf,
18     /// Circumscribed radius: the conservative inflation.
19     radius: f64,
20     /// Inscribed radius: the *optimistic* one. Between the two we must ask parry.
21     inradius: f64,
22     /// `None` means the robot really is a disc, and the ESDF test is exact.
23     footprint: Option<ConvexPolygon>,
24     walls: Polyline,
25 }
26
27 impl CSpace2 {
28     #[inline]
29     pub fn clearance(&self, p: &Point2<f64>) -> f64 {
30         self.esdf.at(p.x, p.y)
31     }
32
33     /// Three-tier test. The cheap answers are conclusive far from walls in both
34     /// directions; only the annulus between the inscribed and circumscribed
35     /// radii costs a real collision query, and in a floorplan that is a few
36     /// percent of configurations.
37     pub fn is_free(&self, q: &Isometry2<f64>) -> bool {
38         let p = Point2::from(q.translation.vector);
39         let d = self.clearance(&p);
40         match &self.footprint {
41             None => d > self.radius,
42             Some(poly) => {
43                 if d > self.radius {
44                     return true; // circumscribed disc fits: certainly free
45                 }
46                 if d <= self.inradius {
47                     return false; // inscribed disc does not: certainly blocked
48                 }
49                 !intersection_test(q, poly, &Isometry2::identity(), &self.walls)
50                     .expect("convex polygon vs polyline is supported")
51             }
52         }
53     }
54
55     /// Swept check along a segment, by **sphere marching**.
56     ///
57     /// Standing at `p` with clearance `c`, every point within `c - r` of `p` is
58     /// free, so we may advance by exactly that much and re-test. The distance
59     /// field is 1-Lipschitz, which is what makes the skip sound; `min_step`

```

```

66     /// keeps the loop terminating when the path grazes a wall. A 6 m edge
67     /// across the corridor costs about a dozen lookups instead of 300 samples.
68 pub fn edge_free(&self, a: &Point2<f64>, b: &Point2<f64>, min_step: f64) -> bool {
69     let delta = b - a;
70     let len = delta.norm();
71     if len < 1e-9 {
72         return self.clearance(a) > self.radius;
73     }
74     let u = delta / len;
75     let mut s = 0.0;
76     while s < len {
77         let c = self.clearance(&(a + u * s)) - self.radius;
78         if c <= 0.0 {
79             return false;
80         }
81         s += c.max(min_step);
82     }
83     self.clearance(b) > self.radius
84 }
85 }
86
87 /// The local planner, as a trait. Straight lines for a holonomic point; Dubins
88 /// or Reeds-Shepp for Rusty. Every sampling planner below is generic over it,
89 /// which is the entire difference between "plans a path" and "plans a path the
90 /// robot can drive".
91 pub trait Steer {
92     /// A curve from `a` toward `b`, truncated at `max_len`, with its cost.
93     fn steer(
94         &self,
95         a: &Isometry2<f64>,
96         b: &Isometry2<f64>,
97         max_len: f64,
98     ) -> Option<(Vec<Isometry2<f64>>, f64)>;
99 }
100
101 pub struct StraightLine;
102 pub struct Dubins {
103     pub rho: f64,
104 }
105 pub struct ReedsShepp {
106     pub rho: f64,
107 }

```

The lattice search is deliberately *not* a petgraph graph. Materializing 2700 nodes and 20,000 edges to represent a grid whose neighbors are computable by adding ± 1 to an index is a pessimization, and knowing when a graph library helps is part of the skill.

RUST crates/ch20_planning/src/search.rs

```

1 use std::cmp::Ordering;
2 use std::collections::BinaryHeap;
3
4 /// An open-list entry. `Ord` is deliberately reversed -- `BinaryHeap` is a *max*
5 /// heap -- and ties break on the cell index so a run is reproducible: two cells
6 /// with equal f must be expanded in a defined order, or the animated frontier
7 /// flickers between frames.

```

```

8  #[derive(PartialEq)]
9  struct Open {
10     f: f64,
11     cell: u32,
12 }
13
14 impl Eq for Open {}
15 impl Ord for Open {
16     fn cmp(&self, other: &Self) -> Ordering {
17         other
18             .f
19             .partial_cmp(&self.f)
20             .unwrap_or(Ordering::Equal)
21             .then_with(|| other.cell.cmp(&self.cell))
22     }
23 }
24 impl PartialOrd for Open {
25     fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
26         Some(self.cmp(other))
27     }
28 }
29
30 /// Octile distance in metres: exact cost-to-go on an *empty* 8-connected grid,
31 /// hence admissible on any grid, hence A* stays optimal (derivation F.2).
32 #[inline]
33 fn octile(di: i32, dj: i32, cell: f64) -> f64 {
34     let (a, b) = ((di.abs() as f64), (dj.abs() as f64));
35     (a.max(b) + (std::f64::consts::SQRT_2 - 1.0) * a.min(b)) * cell
36 }
37
38 /// `a_star(G, s, g, h)` over an implicit 8-connected lattice.
39 ///
40 /// `epsilon` is the heuristic weight: 0 is Dijkstra, 1 is A*, and epsilon > 1 is
41 /// weighted A*, whose path is guaranteed within a factor epsilon of optimal.
42 pub fn a_star_grid(
43     grid: &Lattice,
44     start: u32,
45     goal: u32,
46     epsilon: f64,
47 ) -> Option<(Vec<u32>, f64)> {
48     let n = (grid.nx * grid.ny) as usize;
49     let mut g = vec![f64::INFINITY; n];
50     let mut parent = vec![u32::MAX; n];
51     let mut closed = vec![false; n];
52     let mut heap = BinaryHeap::new();
53
54     let (gi, gj) = grid.coords(goal);
55     let h = |c: u32| {
56         let (i, j) = grid.coords(c);
57         octile(i - gi, j - gj, grid.cell)
58     };
59
60     g[start as usize] = 0.0;
61     heap.push(Open { f: epsilon * h(start), cell: start });
62
63     while let Some(Open { cell, .. }) = heap.pop() {
64         if closed[cell as usize] {
65             continue; // a stale entry: we already expanded this cell more cheaply
66         }
67         closed[cell as usize] = true;
68         if cell == goal {

```

```

69         return Some((trace(&parent, start, goal), g[goal as usize]));
70     }
71
72     let (i, j) = grid.coords(cell);
73     for &(di, dj, w) in Lattice::NEIGHBORS {
74         let (ni, nj) = (i + di, j + dj);
75         let Some(nb) = grid.index(ni, nj).filter(|&k| grid.free(k)) else { continue };
76     };
77
78     // No corner cutting: a diagonal move needs both orthogonal cells
79     // free, or the disc clips a corner the lattice claims is fine.
80     if di != 0 && dj != 0 && !(grid.free_at(ni, j) && grid.free_at(i, nj)) {
81         continue;
82     }
83     let tentative = g[cell as usize] + w * grid.cell;
84     if tentative < g[nb as usize] {
85         g[nb as usize] = tentative;
86         parent[nb as usize] = cell;
87         heap.push(Open { f: tentative + epsilon * h(nb), cell: nb });
88     }
89 }
90 None
91 }

```

The roadmap, on the other hand, is a graph — an explicit one that outlives every query — and that is exactly the case petgraph is for.

RUST crates/ch20_planning/src/prm.rs

```

1 use nalgebra::Point2;
2 use petgraph::algo::astar;
3 use petgraph::graph::{NodeIndex, UnGraph};
4 use rand::rngs::SmallRng;
5
6 use crate::cspace::CSpace2;
7
8 /// A probabilistic roadmap: milestones as nodes, verified local plans as edges.
9 ///
10 /// `petgraph` owns the topology and the traversal; this type owns the geometry
11 /// and the collision checking. The split matters because the roadmap is built
12 /// once and queried forever -- which is PRM's whole reason to exist, and the
13 /// reason it loses on the single query in w20.1.
14 pub struct Prm {
15     graph: UnGraph<Point2<f64>, f64>,
16     k: usize,
17     max_edge: f64,
18     /// Straight-line checks attempted. The honest measure of PRM's real cost:
19     /// it is dominated by collision checking, not by graph work.
20     pub local_plans: usize,
21 }
22
23 impl Prm {
24     /// Draw one milestone and wire it to its k nearest neighbours.
25     pub fn step(&mut self, cs: &CSpace2, rng: &mut SmallRng) -> Option<NodeIndex> {
26         let q = cs.sample_free(rng)?;
27         let id = self.graph.add_node(q);
28
29         // A linear neighbour scan is O(n), so building the roadmap is O(n^2).

```



```

30 // At the scale this chapter runs it that is not what dominates the
31 // profile -- the ~14,000 swept collision checks are. Reach for a k-d
32 // tree when a profiler says to, not when the asymptotics do.
33 let mut cand: Vec<_> = self
34     .graph
35     .node_indices()
36     .filter(|&j| j != id)
37     .map(|j| (j, (self.graph[j] - q).norm()))
38     .filter(|&(_, d)| d <= self.max_edge)
39     .collect();
40 cand.sort_by(|a, b| a.1.total_cmp(&b.1));
41
42 for &(j, d) in cand.iter().take(self.k) {
43     self.local_plans += 1;
44     if cs.edge_free(&q, &self.graph[j], 0.02) {
45         self.graph.add_edge(id, j, d);
46     }
47 }
48 Some(id)
49 }
50
51 /// Attach start and goal, then let petgraph's A* answer the query. The
52 /// attachment is temporary: the roadmap itself is never modified, so the
53 /// next query pays only for its own two attachments.
54 pub fn query(&self, cs: &CSpace2, s: Point2<f64>, g: Point2<f64>) -> Option<(Vec<
Point2<f64>>, f64)> {
55     let mut view = self.graph.clone();
56     let (si, gi) = (self.attach(&mut view, cs, s), self.attach(&mut view, cs, g));
57     let goal_pt = view[gi];
58     let (cost, path) = astar(
59         &view,
60         si,
61         |n| n == gi,
62         |e| *e.weight(),
63         // Euclidean distance to the goal: admissible, because no roadmap
64         // edge is shorter than the straight line it was checked along.
65         |n| (view[n] - goal_pt).norm(),
66     )?;
67     Some((path.into_iter().map(|n| view[n]).collect(), cost))
68 }
69 }

```

RRT and RRT* are one type, because they differ by exactly the two operations of the algorithm box. Writing them separately would hide the chapter's point.

RUST crates/ch20_planning/src/rrt.rs

```

1 use nalgebra::{Isometry2, Point2};
2 use rand::rngs::SmallRng;
3 use rand::{Rng, SeedableRng};
4
5 use crate::cspace::{CSpace2, Steer};
6
7 pub struct Node {
8     pub q: Isometry2<f64>,
9     pub parent: Option<u32>,
10    /// Cost-to-come along tree edges. The quantity rewiring exists to lower.
11    pub cost: f64,

```

```

12 }
13
14 pub struct RrtStar<S: Steer> {
15     steer: S,
16     nodes: Vec<Node>,
17     children: Vec<Vec<u32>>,
18     nn: KdTree2,
19     /// gamma in r_n = gamma (log n / n)^{1/d}. Must exceed the Karaman-Frazzoli bound
20     /// or asymptotic optimality is forfeit -- so we check, loudly, in debug.
21     gamma: f64,
22     /// `false` gives plain RRT: no choose-parent, no rewire, no guarantee.
23     rewire: bool,
24     goal: Isometry2<f64>,
25     best: Option<(u32, f64)>,
26 }
27
28 impl<S: Steer> RrtStar<S> {
29     pub fn new(cs: &Cspace2, steer: S, start: Isometry2<f64>, goal: Isometry2<f64>, gamma:
30         f64) -> Self {
31         /// 2 (1 + 1/d)^{1/d} (mu(Q_free)/zeta_d)^{1/d}, with d = 2 and zeta_2 = pi.
32         debug_assert!(
33             gamma > 2.0 * 1.5_f64.sqrt() * (cs.free_measure() / std::f64::consts::PI).
34             sqrt(),
35             "gamma below the Karaman-Frazzoli bound: this tree is not asymptotically
36             optimal",
37         );
38         /* ... */
39     }
40
41     /// r_n = min(gamma (log n / n)^{1/d}, r_max), with d = 2. The cap keeps the
42     /// first few hundred iterations from rewiring the whole tree; with gamma = 12.3
43     /// and r_max = 2 m it is inactive by n ~ 200, so it cannot touch the limit
44     /// the theorem is about.
45     fn connection_radius(&self) -> f64 {
46         let n = self.nodes.len().max(2) as f64;
47         (self.gamma * (n.ln() / n).sqrt()).min(self.max_radius)
48     }
49
50     pub fn step(&mut self, cs: &Cspace2, rng: &mut SmallRng) -> ExtendStatus {
51         let q_rand = if rng.random_bool(self.goal_bias) {
52             self.goal
53         } else {
54             cs.sample_uniform(rng)
55         };
56
57         let near_idx = self.nn.nearest(&q_rand);
58         let Some((path, len)) = self.steer.steer(&self.nodes[near_idx].q, &q_rand, self.
59             step_size)
60         else {
61             return ExtendStatus::Trapped;
62         };
63
64         let q_new = *path.last().unwrap();
65         if !cs.path_free(&path) {
66             return ExtendStatus::Trapped;
67         }
68
69         let (mut parent, mut cost) = (near_idx, self.nodes[near_idx].cost + len);
70
71         if self.rewire {
72             let r = self.connection_radius();
73             let near = self.nn.within(&q_new, r);

```

```

69
70         // choose parent: cheapest collision-free connection in the ball,
71         // which need not be the nearest one.
72         for &i in &near {
73             if let Some((p, l)) = self.steer.steer(&self.nodes[i].q, &q_new, f64::
INFINITY) {
74                 let c = self.nodes[i].cost + l;
75                 if c < cost && cs.path_free(&p) {
76                     parent = i;
77                     cost = c;
78                 }
79             }
80         }
81         let id = self.push(q_new, Some(parent), cost);
82
83         // rewire: does anyone in the ball reach home more cheaply through
84         // the newcomer?
85         for &i in &near {
86             if i == parent {
87                 continue;
88             }
89             if let Some((p, l)) = self.steer.steer(&q_new, &self.nodes[i].q, f64::
INFINITY) {
90                 if cost + l < self.nodes[i].cost && cs.path_free(&p) {
91                     self.reparent(i, id, cost + l);
92                 }
93             }
94         }
95         self.check_goal(id);
96         return ExtendStatus::Advanced;
97     }
98
99     let id = self.push(q_new, Some(parent), cost);
100     self.check_goal(id);
101     ExtendStatus::Advanced
102 }
103
104 // Hang `i` under `new_parent`, then push the saving down its subtree.
105 //
106 // Skipping the propagation is the classic RRT* bug: the tree *looks*
107 // rewired, but the costs it minimises are stale, so it quietly stops being
108 // asymptotically optimal while every visual check still passes.
109 fn reparent(&mut self, i: u32, new_parent: u32, cost: f64) {
110     if let Some(old) = self.nodes[i as usize].parent {
111         self.children[old as usize].retain(|&c| c != i);
112     }
113     self.nodes[i as usize].parent = Some(new_parent);
114     self.nodes[i as usize].cost = cost;
115     self.children[new_parent as usize].push(i);
116
117     let mut stack = vec![i];
118     while let Some(p) = stack.pop() {
119         for k in 0..self.children[p as usize].len() {
120             let c = self.children[p as usize][k];
121             let edge = self.edge_cost(p, c);
122             self.nodes[c as usize].cost = self.nodes[p as usize].cost + edge;
123             stack.push(c);
124         }
125     }
126 }
127 }

```

20.5.1 The scoreboard, and the test that pins it

RUST crates/ch20_planning/examples/planner_race.rs

```

1 fn main() -> anyhow::Result<()> {
2     let world = sim::World::apartment();
3     let cs = CSpace2::disc(&world, 0.25, 0.05);
4     let lattice = Lattice::from_cspace(&cs, 0.2);
5     let (start, goal) = (Point2::new(1.0, 1.0), Point2::new(11.0, 8.0));
6
7     // Every run is seeded, and each planner gets its *own* stream: sharing one
8     // would make RRT's samples depend on how many PRM rejected, and the
9     // RRT/RRT* comparison would stop being controlled. Seed 20 is the figure
10    // in the text and in w20.1.
11    let (mut r_prm, mut r_rrt, mut r_star) = (
12        SmallRng::seed_from_u64(20),
13        SmallRng::seed_from_u64(21),
14        SmallRng::seed_from_u64(21), // identical to RRT's: same darts, different tree
15    );
16
17    let (cells, a_cost) = a_star_grid(&lattice, lattice.nearest_free(&start),
18                                     lattice.nearest_free(&goal), 1.0).unwrap();
19    let mut prm = Prm::new(&cs, 8, 2.5);
20    let mut rrt = RrtStar::new(&cs, StraightLine, start.into(), goal.into(), 12.3).plain
21    ();
22    let mut star = RrtStar::new(&cs, StraightLine, start.into(), goal.into(), 12.3);
23    for _ in 0..2500 {
24        prm.step(&cs, &mut r_prm);
25        rrt.step(&cs, &mut r_rrt);
26        star.step(&cs, &mut r_star);
27    }
28
29    println!("A*          {:6.2} m   {:5} expansions   {} cells on path",
30            a_cost, a_star_expansions(), cells.len());
31    println!("PRM          {:6.2} m   {:5} milestones", prm.query(start, goal)?.cost, prm.
32    len());
33    println!("RRT          {:6.2} m   first solution at sample {}", rrt.best_cost(), rrt.
34    first_at());
35    println!("RRT*         {:6.2} m   {} improvements", star.best_cost(), star.improvements
36    ());
37    Ok(())
38 }

```

A*	15.06 m	792 expansions	68 cells on path
PRM	14.29 m	1768 milestones	
RRT	17.09 m	first solution at sample 974	
RRT*	13.97 m	15 improvements	

RUST crates/ch20_planning/src/lib.rs (tests)

```

1 /// The scoreboard is not decoration: it is the chapter's claim, and a claim
2 /// that is not tested is a claim that will rot. These are *invariants*, not
3 /// frozen numbers -- the ordering has to hold for every seed, which is a much

```

```

4  // stronger statement than "seed 20 produced 13.97".
5  #[test]
6  fn apartment_scoreboard_invariants() {
7      let (cs, lattice, start, goal) = apartment_query();
8      let a_cost = a_star_grid(&lattice, lattice.nearest_free(&start),
9                               lattice.nearest_free(&goal), 1.0).unwrap().1;
10     let dijkstra = a_star_grid(&lattice, lattice.nearest_free(&start),
11                               lattice.nearest_free(&goal), 0.0).unwrap().1;
12
13     // The heuristic buys expansions, never optimality.
14     assert_relative_eq!(a_cost, dijkstra, epsilon = 1e-9);
15
16     for seed in 20..28u64 {
17         let mut r1 = SmallRng::seed_from_u64(seed);
18         let mut r2 = SmallRng::seed_from_u64(seed);
19         let rrt = run(&cs, start, goal, 2500, false, &mut r1);
20         let star = run(&cs, start, goal, 2500, true, &mut r2);
21
22         // At an equal budget, rewiring is strictly better. Every time.
23         assert!(star < rrt, "seed {seed}: RRT* {star} did not beat RRT {rrt}");
24         // And it beats the lattice, because the lattice is not the plane:
25         // eight headings cost an octile detour a sampler never pays.
26         assert!(star < a_cost);
27     }
28 }

```

20.6 Putting it together: the Apartment lab, and where it breaks

Run the race yourself in the arena above, then read the table. It says something more interesting than "RRT* wins".

Planner	Cost	Guarantee it actually earned
A* (0.2 m lattice)	15.06 m	Optimal on the lattice ; resolution complete
PRM ($n = 2500$, $k = 8$)	14.29 m	Probabilistically complete; reusable for every later query
RRT ($n = 2500$)	17.09 m	Probabilistically complete; feasible, and permanently 22% worse than RRT* at the same budget
RRT* ($n = 2500$, $\gamma = 12.3$)	13.97 m	Asymptotically optimal, and it shows

The number to sit with is A*'s. *It is the only one in the table backed by a proof of optimality, and it is 8% worse than RRT**. Both statements are true because they are about different problems: A* solves the lattice exactly, and the lattice is a lossy model of the plane. So refine it, and watch what happens:

Cell size	A* cost	Cells stored	Expansions
0.4 m	15.89 m	660	232
0.2 m	15.06 m	2,700	792
0.1 m	14.68 m	10,800	2,621
0.05 m	14.62 m	43,200	10,847

Sixty-five times the memory buys 1.27 m, and then the curve flattens well short of RRT's 13.97 m. *That plateau is not a resolution problem and refining further will not fix it: the 8-connected lattice measures length in the octile* metric, which overestimates Euclidean distance by up to 8.24%, and that bias is a property of the connectivity, not the cell size.* To remove it you have to change the graph — 16-connected neighborhoods, a state lattice of motion primitives — or stop trusting the graph at the end. Run 300 rounds of random pairwise shortcutting over the 0.2 m path and it drops to **14.11 m**: cheaper than the 0.05 m lattice, at one sixty-fifth of the memory. Every production stack does this, and now you know what it is repairing.

This is the real content of "resolution complete", and it is why a costmap planner's resolution *and* its connectivity are design parameters rather than implementation details.

20.6.1 The last step, which does not work yet

Take RRT's 13.97 m path, run 300 rounds of random pairwise shortcutting on it (splice in the straight line between two random points on the path whenever that line is free), and it drops to 13.93 m across 10 waypoints. Beautiful. Now hand it to Rusty.

Rusty cannot drive a corner. Chain a Dubins curve between each consecutive pair of waypoints, heading each waypoint along the local path direction, and measure what it costs:

Turning radius ρ	Drivable length	Inflation	Sampled poses in collision
0.2 m	15.19 m	+9.1%	1 / 327
0.4 m	16.47 m	+18.3%	23 / 352
0.8 m	23.98 m	+72.2%	128 / 505

Look at the last column. The *geometrically optimal* path, rendered drivable, **hits walls**. Not because the planner was wrong — every waypoint is free and every straight segment between waypoints is free — but because the planner reasoned about a robot that can turn in place, and the arcs needed to connect its corners bulge outside the corridor it planned through. At $\rho = 0.8$ m the plan is not merely expensive; it is unexecutable.

There are exactly three honest repairs, and each of them is a later chapter.

1. **Plan in the right space.** Put steering inside the planner: give RRT* a Dubins local planner instead of `StraightLine`, so every tree edge is drivable by construction and the collision check runs on the arc, not the chord. That is a one-line change with the `Steer` trait above, and it is what the exercises ask you to do.
2. **Plan a policy, not a path.** A path says "go here, then here". A *policy* says "from wherever you actually are, do this" — which is what you want when the wheels do not deliver what you commanded. Chapter 21 makes that switch, and the wave-front you built above is already its simplest instance.
3. **Re-plan continuously and let a controller close the gap.** Keep the geometric path as a *reference*, and run an optimizer at 20 Hz that finds the drivable, collision-free trajectory nearest to it. That is Chapter 23, and the reference it tracks comes from this chapter.

And the deeper omission, the one Part VI exists to fix: everything above assumed the robot knows where it is. Rusty does not. It has a belief — a covariance ellipse that grows down the featureless corridor and shrinks in the doorway, exactly as Chapter 11 showed. Planning through the middle of a room is shortest; planning along a wall keeps the localization sharp. Chapter 22 is about that trade, and it is why robots hug walls.

20.7 Exercises

1 FOUNDATION •• The C-obstacle of a polygon

Derive the C-obstacle of a disc robot of radius r against a convex polygonal obstacle with vertices $v_1, \dots, v_k$. Show that its boundary consists of the k edges offset outward by r , joined by circular arcs of radius r centered on the vertices, and that the total turning of the boundary is 2π regardless of k .

Then, in two sentences: why does a *rectangular* Rusty make QO depend on θ , and what does the θ -slice look like at $\theta = 0$ versus $\theta = \pi/4$?

2 FOUNDATION •• Break A* with a bad heuristic

Construct a four-node graph, a start, a goal, and a heuristic h that is *inadmissible* at exactly one node, such that A* with the closed-set optimization returns a strictly suboptimal path. State the ratio between what it returns and the optimum, and identify which step of the optimality derivation your example breaks.

Then verify it in code. `a_star_grid` hard-codes the octile heuristic precisely so that it *cannot* be broken this way, so build your counterexample on an explicit `petgraph::graph::UnGraph` with a pluggable h , and assert that the cost it returns strictly exceeds the Dijkstra cost on the same graph. Finally, show that reopening closed nodes repairs optimality but not the running time.

3 FOUNDATION ••• How many samples does the theorem want?

A corridor of clear width w joins two rooms of combined free area μ , and the shortest route through it has length L . Using the bound derived above, write $n(w)$ — the number of milestones needed to push the failure bound below 0.1 — as a function of w , and show that n grows like $w^{-2} \log(1/w)$ in the plane and like $w^{-d} \log(1/w)$ in dimension d .

Now evaluate it for the widget's geometry ($\mu = 51 \text{ m}^2$, $L = 8 \text{ m}$) at $w = 0.35 \text{ m}$ and at $w = 0.05 \text{ m}$, and say in one sentence what the resulting numbers mean for a planner that has 50 ms per query.

4 CONCEPTUAL • Predict the collapse

In the Narrow Passage widget, the corridor sits at 0.65 m with a measured success probability near 13% at $n = 180$ uniform milestones. Before touching the slider, predict the success probability at 0.35 m — and predict whether it falls by the ratio of the corridor *widths* (roughly 2×) or by something faster.

Now move the slider. Reconcile what you see with the exponent $b \propto \delta^2$ in the bound: which of the two constants, a or b , explains the collapse?

5 CONCEPTUAL •• Which tree answers first?

Before touching the Planner Arena, predict: at the same sample budget and the same seed, does RRT or RRT* *return its first** solution after fewer samples? Argue for your answer, then read the "first answer at" column across three re-rolled seeds.

The result is not a coincidence, and explaining it is the exercise. (Hint: rewiring changes which node is a node's *parent*. What does it change about the set of node *positions*?) Then say in one sentence what does differ between the two — and why the arena's chart, not its scoreboard, is where you see it.

Second part, no widget: note roughly how many samples RRT needs for its first answer against A's 792 *expansions*, then argue what a map would have to look like for the lattice to win decisively* on time-to-first-solution, and what it would have to look like for RRT to win by 10×. Compare the number of lattice cells against the ratio of free volume to the volume of the region every solution must pass through; one of those two numbers is what each planner pays.

6 PRACTICAL •• Steer with Dubins

Give RrtStar a Dubins { $\rho: 0.4$ } local planner instead of StraightLine, so tree edges are arcs and the collision check runs on the arc rather than the chord. You will need to (a) make the tree store SE(2) nodes, (b) use a distance for nearest-neighbor lookup that respects heading, and (c) notice that Dubins distance is *asymmetric* — steering from a to b is not the same cost as b to a — so the "rewire" step must re-evaluate, not reuse.

Reproduce the table in "The last step": how much longer is the drivable plan, and

how many sampled poses are in collision now? The second number should be zero. Explain why (c) is the reason RRT* on a nonholonomic system is genuinely harder than the version in this chapter.

7 PRACTICAL • • • Bidirectional RRT-Connect, and a fair benchmark

Implement RRT-Connect: grow two trees, one from the start and one from the goal, and after each extension have the *other* tree greedily extend toward the new node until it reaches it or is trapped. Add it as a fifth lane to the arena.

Then benchmark honestly. On 50 seeded runs of the narrow-passage map at 0.5 m corridor width, report median samples-to-first-solution and success rate at $n = 180$ for RRT, RRT-Connect, and RRT-Connect with bridge sampling. State whether bidirectional search changes the *exponent* in the completeness bound or only the constant — and justify your answer with the covering argument, not with the measurement.

20.8 References

Choset, H., Lynch, K. M., Hutchinson, S., Kantor, G. A., Burgard, W., Kavraki, L. E., and Thrun, S. (2005). *Principles of Robot Motion: Theory, Algorithms, and Implementations* MIT Press.

The spine of this chapter and the source of its epigraph. Ch. 3 is configuration space, Ch. 4 potential functions and navigation functions, Ch. 7 the sampling-based planners, App. H the search algorithms. [Link](#)

Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths *IEEE Transactions on Systems Science and Cybernetics* 4(2), 100–107.

A* itself, including the admissibility theorem this chapter derives. Worth reading for how carefully the original states what the heuristic is allowed to know. doi:10.1109/TSSC.1968.300136

Khatib, O. (1986). Real-Time Obstacle Avoidance for Manipulators and Mobile Robots *The International Journal of Robotics Research* 5(1), 90–98.

The artificial potential field, in the formulation the Potential Well widget implements. Khatib is explicit that it is a real-time control law, not a planner — a distinction the field then spent a decade forgetting. doi:10.1177/027836498600500106

Koditschek, D. E. and Rimon, E. (1990). Robot Navigation Functions on Manifolds with Boundary *Advances in Applied Mathematics* 11(4), 412–442.

Why you cannot wish local minima away, and what the best possible potential looks like: a navigation function on a sphere world, extended by diffeomorphism to star worlds. doi:10.1016/0196-8858(90)90017-S

Kavraki, L. E., vestka, P., Latombe, J.-C., and Overmars, M. H. (1996). Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces *IEEE Transactions on Robotics and Automation* 12(4), 566–580.

PRM, and the paper that made ‘probabilistically complete’ a guarantee worth stating. The learning/query split is its central design decision. doi:10.1109/70.508439

Hsu, D., Jiang, T., Reif, J., and Sun, Z. (2003). The Bridge Test for Sampling Narrow Passages with Probabilistic Roadmap Planners *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 4420–4426.

The narrow-passage fix in widget w20.2, including the 60/40 hybrid with uniform sampling — pure bridge sampling populates passages and nothing else. doi:10.1109/ROBOT.2003.1242285

Dubins, L. E. (1957). On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents *American Journal of Mathematics* 79(3), 497–516.

The six words. Predates robotics entirely; the U-turn worked by hand in this chapter is a direct consequence of its case analysis. doi:10.2307/2372560

Karaman, S. and Frazzoli, E. (2011). Sampling-based Algorithms for Optimal Motion Planning *The International Journal of Robotics Research* 30(7), 846–894.

RRT, PRM, the proof that RRT converges to a suboptimal solution with probability one, and the γ bound the Planner Arena is configured to respect. doi:10.1177/0278364911406761

Dolgov, D., Thrun, S., Montemerlo, M., and Diebel, J. (2010). Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments *The International Journal of Robotics Research* 29(5), 485–501.

Hybrid A*, with the continuous-state-per-cell trick, the analytic Reeds–Shepp expansion near the goal, and the dual heuristic this chapter sketches. doi:10.1177/0278364909359210

Solovey, K., Janson, L., Schmerling, E., Frazzoli, E., and Pavone, M. (2020). Revisiting the Asymptotic Optimality of RRT* *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2189–2195.

Identifies a logical gap in the 2011 optimality proof and repairs it, with a corrected connection radius that shrinks as $(\log n / n)^{1/(d+1)}$ — larger than the original, to account for the ordering of the samples. doi:10.1109/ICRA40945.2020.9196553

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., and Ferguson, M. (2023). From the Desks of ROS Maintainers: A Survey of Modern and Capable Mobile Robotics Algorithms in the Robot Operating System 2 *Robotics and Autonomous Systems* 168, 104493.

What actually ships. The source for this chapter’s claim that modern stacks replan with A/hybrid A at costmap rate rather than repairing with D*, and a good map of where each planner sits in a real navigation pipeline. doi:10.1016/j.robot.2023.104493

Orthey, A., Chamzas, C., and Kavraki, L. E. (2024). Sampling-Based Motion Planning: A Comparative Review *Annual Review of Control, Robotics, and Autonomous Systems* 7, 285–310.

The current map of the planner zoo, benchmarked on 24 problems. Read it after this chapter to see where BIT, AIT and the informed-sampling family fit, and which planner actually wins on which structure. doi:10.1146/annurev-control-061623-094742

Decision Making I: MDPs and Value Iteration

PLANNING AND ACTING UNDER UNCERTAINTY · Intermediate · 55 min



One way to cope with the resulting uncertainty is to generate a policy for action selection defined for all states that the robot might encounter.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 15

Chapter 20 ended with a beautiful path and a robot sliding off it. That is not a bug in the planner. A path is a function of *time* — do this, then this, then this — and the moment the wheels deliver something other than what was commanded, the robot is at a state the path has nothing to say about. Every chapter since Chapter 9 has been about how reliably the wheels fail to deliver.

This chapter makes the move the rest of Part VI pivots on: stop computing a path and start computing a **policy**, a function of *state* that answers "what should I do?" everywhere, so that drifting off is not a failure mode but a Tuesday. The framework is the Markov decision process, the algorithm is value iteration, and the object it produces — the value function — is Chapter 20's potential field done properly: a scalar field that *cannot* have spurious local minima, because it is defined as expected cost-to-go rather than sculpted out of geometry.

The chapter closes on the question Chapter 22 exists to answer. A policy is a function of the state. What if the robot does not know the state?

◎ AFTER THIS CHAPTER YOU CAN

- Define a Markov decision process and say which of its five pieces each earlier chapter already built
- Derive the Bellman optimality equation from the finite-horizon recursion,

and take the limit honestly

- Prove that the Bellman backup is a γ -contraction, and turn the proof into a stopping rule you can code
- Bound the loss of acting greedily on a value function that has not converged
- Derive policy iteration, and recognize policy evaluation as one sparse linear solve
- Formulate navigation as a stochastic shortest path, and see Chapter 20's wave-front planner as its deterministic special case
- Compile an occupancy grid plus the Chapter 9 velocity model into a finite MDP whose slip parameter has a pedigree

ASSUMED

- Expectation and conditional probability (Chapter 2)
- The Markov assumption and the motion model $p(x_t \mid x_{t-1}, u_t)$ (Chapter 5)
- The velocity motion model and its noise parameters $\alpha_1 \dots \alpha$ (Chapter 9)
- Occupancy grids and configuration-space inflation (Chapters 13 and 20)
- Graph search and the wave-front planner (Chapter 20)

21.1 The plan that slides off the floor

Here is the experiment, run twice on the same world, with the same solver and the same random numbers. On the left, Rusty commits to the optimal *action sequence* and replays it open-loop. On the right, Rusty consults the same solver's answer at every step.

INTERACTIVE · f21.4

Plan versus policy, same world, same dice

A plan is a line through the state space and has nothing to say about the states you actually reach. A policy is defined everywhere, so drifting off is not a failure mode.

Run this simulation in the web edition: [/chapters/ch21-mdps #f21.4](#)

At zero slip the two pictures are identical, and that identity is the whole reason deterministic planning ever worked: when execution is perfect, the sequence of actions along the optimal path *is* the optimal policy restricted to that path. Turn the slip up and the pictures diverge immediately, and they diverge in a specific, diagnostic way.

The plan does not fail gradually. It fails at the first divergence and then accumulates. Once the robot is one cell off the planned trajectory, the remaining actions in the sequence were computed for somebody else's position. Step seventeen of a plan is a claim about where you will be at step seventeen, and after a few unlucky draws it is a claim about a place you have never been.

The policy is unbothered. It never notices the "error", because the notion of error requires a reference trajectory and there isn't one. It reads the current cell and answers. The realized path is longer than the nominal one — noise costs something, and the cost is real — but the arrival rate barely moves. Run that world four thousand times with matched seeds and the split is brutal:

slip s	0.00	0.05	0.10	0.15	0.30
open-loop plan reaches the dock	100%	21.1%	3.7%	0.9%	0.0%
policy reaches the dock	100%	100%	100%	100%	99.8%
policy's mean steps (nominal 16)	16.0	17.9	20.3	23.5	43.5

A five percent chance of veering — which [the next section](#) prices at about 13.5° of heading uncertainty, an ordinary afternoon for a localizer — already sinks the plan four times out of five. The policy pays for the same noise in steps rather than in failures, and steps are a currency you can budget.

The two objects are different types. This is worth being pedantic about, because the type is the lesson:

$$\text{plan} : \{0, 1, \dots, T\} \rightarrow \mathcal{U} \qquad \text{policy} : \mathcal{X} \rightarrow \mathcal{U}$$

A plan is indexed by time. A policy is indexed by state. Everything else in this chapter follows from taking that difference seriously.

💡 THE ONE SENTENCE VERSION

A plan is a line through the state space; a policy is a vector field over it. Under stochastic execution the line is a fiction after the first step, and the field is not.

The obvious objection is that replanning fixes this: run the planner again from wherever you actually are. It does, and it is what most deployed systems do. But notice what replanning converges to as the replanning rate goes up — an answer for every state you might visit, computed on demand. A policy is what you get when you precompute all of them, and it is worth building once so you can see what the on-demand version is approximating. (When the state space is too big to precompute, you go back to on-demand: that is Chapter 23.)

21.2 Building intuition

21.2.1 Paint a reward, get a behavior

The following widget is the chapter in one object. A gridworld is laid over the Apartment floorplan; you paint payoff onto cells; value iteration re-converges live and the arrow field is read off whatever the value function currently is. Nothing is precomputed, and Rusty walks the current policy — including while it is still wrong.

INTERACTIVE · w21.1

Policy Painter

A policy is not a stored path. It is an answer for every cell — and the answer bends away from danger as the floor gets slipperier.

Run this simulation in the web edition: [/chapters/ch21-mdps #w21.1](/chapters/ch21-mdps/#w21.1)

Three things repay a minute of watching, and each one is a theorem later in the chapter.

Value propagates backward from the payoff, roughly a ring per sweep. Immediately after you paint a goal, only its neighbours know about it. That is why value iteration feels like a wave, and why the sweep count scales with the *diameter* of the map rather than its area. Look closely and the wave is lopsided — it runs further in one direction per sweep than the other — because the backups are done in place, so a cell late in the sweep already sees the cells updated before it. Flip the **synchronous sweeps** toggle and the lopsidedness disappears, along with some of the speed.

The arrows straighten out long before the numbers do. Watch the residual readout: the arrow field stops changing while $\|\Delta V\|_\infty$ is still large. That is not a rendering artifact, and it is not luck either — [the loss bound](#) explains why a value function nowhere near converged still yields a usable policy, and the measurement below shows this scene's policy going *exactly* optimal after 45 of its 72 sweeps. $\arg\max$ only cares about the ordering of the Q -values, and orderings settle first.

Slip bends the field away from the hazard. At $s = 0$ the arrows run right past the spill cell — why not, you never veer. Push s toward 0.3 and a visible berth opens up. Nobody programmed a safety margin; the margin is what the expectation computes when some of the futures in it involve veering into the thing you are hugging. Measured on the default scene, the optimal policy's mean journey grows from 37.0 steps at $s = 0$ to 45.5 at $s = 0.2$ to 56.9 at $s = 0.4$. The detour margin *is* the noise, priced.

NOTE

Everything in that widget is the real algorithm. The sweeps are `sweepInPlace` from `lib/decision/mdp.ts`, the arrows are `greedyPolicy`, and the robot’s steps are drawn from the same sparse transition rows the solver plans against — the TypeScript twin of the Rust in [Implementation in Rust](#). If the two ever disagreed, the numerical self-checks would say so.

21.3 The mathematics

21.3.1 Notation

$\mathcal{X}, \mathcal{U}$	Finite state and action sets. Here: free grid cells, and the eight compass moves.
$p(x' \mid x, u)$	Transition model — the same object the Bayes filter predicts with, now indexed by a decision.
$r(x, u)$	Expected immediate payoff of taking u in x . Negative for cost.
$\gamma \in (0, 1]$	Discount factor. $\gamma < 1$ for discounted problems; $\gamma = 1$ only for stochastic shortest paths.
$\pi : \mathcal{X} \rightarrow \mathcal{U}$	A deterministic stationary policy — an action for every state, not a sequence.
R_T	Expected cumulative discounted payoff over horizon T .
$V^\pi(x), V^*(x)$	Value of following π from x ; value of acting optimally from x .
$Q^*(x, u)$	Value of doing u once, then acting optimally. Chapter 22 reuses these directly.
$\mathcal{T}$	The Bellman backup operator, $(\mathcal{T}V)(x) = \max_u [r + \gamma \sum p V]$.
s	Slip: probability of veering into each of the two neighbouring directions. Intended direction: $1 - 2s$.

Thrun et al. write the payoff as $c(x')$, collected on *entering* a state, and put the discount outside the integral. We fold the expected entry payoff into $r(x, u)$, which is the textbook form and changes not a single number:

$$r(x, u) = -\text{cost}(u) + \sum_{x'} p(x' \mid x, u) c(x')$$

The book’s grid compiler does exactly this, which is why goal payoff shows up in the *neighbours* of the goal rather than in the goal cell itself.

21.3.2 The five pieces, and where each one came from

A Markov decision process is the tuple

$$(\mathcal{X}, \mathcal{U}, p(x' \mid x, u), r(x, u), \gamma)$$

with three standing assumptions, each of which is load-bearing and each of which is false somewhere.

1. **Markov.** $p(x' | x, u)$ depends on the past only through x . This is Chapter 5's assumption, and it is what licenses a policy to be a function of the current state alone. Drop it and the policy needs history.
2. **Full observability.** The robot knows x exactly at decision time. This is the assumption this chapter buys its tractability with, and it is the one Chapter 22 pays back.
3. **Stationarity.** p and r do not depend on t . Together with an infinite horizon, this is what makes the optimal policy itself stationary — one lookup table, not one per timestep.

Only the first two pieces are new here. $\mathcal{X}$ is Chapter 13's occupancy grid with the walls inflated by the robot radius, exactly the configuration-space trick of Chapter 20. $p(x' | x, u)$ is Chapter 9's velocity motion model, discretized. The reward is the only genuinely new modelling decision, and it is where all the honesty lives: an MDP does not tell you what to want, it tells you what to do given what you want.

21.3.3 Where the slip parameter comes from

Gridworld papers announce a slip probability. We are going to earn ours, because the book's promise is that every probability has a pedigree.

The robot commands "move to the neighbouring cell in direction d ". It drives for $\Delta t = \ell/v$ seconds, where ℓ is the cell size. The cell it lands in is decided by the *bearing* of its net displacement: inside the $\pm\pi/8$ sector around d and it landed on the intended neighbour; outside, and it landed on one of the two flanking cells. So

$$2s = \Pr \left[\left| \text{atan2}(\Delta y, \Delta x) \right| > \frac{\pi}{8} \right]$$

where $(\Delta x, \Delta y)$ is drawn by the Chapter 9 sampler: perturb the commanded (v, ω) by $\hat{v} = v + \varepsilon_{\alpha_1 v^2}$ and $\hat{\omega} = \omega + \varepsilon_{\alpha_3 v^2}$, then integrate the arc for Δt , starting from a heading that is itself uncertain by σ_θ — because the robot aims using the heading it *believes* it has.

DERIVATION Integrating the velocity model over one cell transit

$s = \Phi(-(\pi/8) / \text{sqrt}(\sigma_\theta^2 + \alpha_3 \ell^2/4))$ — and it does not depend on speed

Start the transit at pose $(0, 0, \theta_0)$ with $\theta_0 \sim \mathcal{N}(0, \sigma_\theta^2)$, the heading error the localizer has left on the table. Thrun et al., Table 5.3 gives the endpoint of

an arc with perturbed controls $(\hat{v}, \hat{\omega})$ after Δt :

$$\Delta x = -\frac{\hat{v}}{\hat{\omega}} \sin \theta_0 + \frac{\hat{v}}{\hat{\omega}} \sin(\theta_0 + \hat{\omega} \Delta t), \quad \Delta y = \frac{\hat{v}}{\hat{\omega}} \cos \theta_0 - \frac{\hat{v}}{\hat{\omega}} \cos(\theta_0 + \hat{\omega} \Delta t)$$

Step 1 — why bearing and not lateral offset. A tempting alternative is to ask whether the lateral displacement exceeds half a cell. It essentially never does: with $\ell = 0.3$ m you would need $|\theta_0| > 30^\circ$. But the grid does not ask "did you drift half a cell sideways", it asks "which cell are you in", and over a single short transit that question is answered by direction. Binning on bearing is the discretization that matches the state space.

Step 2 — the bearing has a closed form, and it is exact. Apply the sum-to-product identities to both components:

$$\Delta x = 2 \frac{\hat{v}}{\hat{\omega}} \sin\left(\frac{\hat{\omega} \Delta t}{2}\right) \cos\left(\theta_0 + \frac{\hat{\omega} \Delta t}{2}\right), \quad \Delta y = 2 \frac{\hat{v}}{\hat{\omega}} \sin\left(\frac{\hat{\omega} \Delta t}{2}\right) \sin\left(\theta_0 + \frac{\hat{\omega} \Delta t}{2}\right)$$

The two share a common scalar factor, and that factor is positive whenever $\hat{v} > 0$ — the sign of $\hat{\omega}$ cancels between $1/\hat{\omega}$ and $\sin(\hat{\omega} \Delta t/2)$. So

$$\text{atan2}(\Delta y, \Delta x) = \theta_0 + \frac{1}{2} \hat{\omega} \Delta t$$

exactly, not to leading order: the chord of a circular arc bisects the turn. And $\hat{v}$ has dropped out entirely — a robot that drives too fast overshoots but does not veer.

Step 3 — collect the variance. With $\theta_0 \sim \mathcal{N}(0, \sigma_\theta^2)$ and $\hat{\omega} \sim \mathcal{N}(0, \alpha_3 v^2)$ independent, the bearing is Gaussian, and substituting $\Delta t = \ell/v$ makes the second term's dependence on speed vanish too:

$$\text{Var}\left[\frac{1}{2} \hat{\omega} \Delta t\right] = \frac{1}{4} \alpha_3 v^2 \cdot \frac{\ell^2}{v^2} = \frac{\alpha_3 \ell^2}{4} \quad \Rightarrow \quad s = \Phi\left(-\frac{\pi/8}{\sqrt{\sigma_\theta^2 + \alpha_3 \ell^2/4}}\right)$$

With $\alpha_3 = 0.05$ and $\ell = 0.3$ m the turn-rate term contributes 0.034 rad — a shade under two degrees — no matter how fast Rusty drives. The library's `slipFromVelocityModel` samples the model rather than assuming this, and self-check 11 pins the sampler to the closed form.

Step 4 — read the number. Sampled with 40,000 seeded draws and full noise, against the formula:

σ_θ	5°	10°	15°	20°
closed form	1.3×10^{-5}	0.0136	0.0684	0.1314
sampled	0.0005	0.0139	0.0684	0.1315



The one place the two rows disagree is instructive. At $\sigma_\theta = 5^\circ$ the sampler reports 0.0005 where the formula says 1.3×10^{-5} — a factor of forty. The excess is not noise; it is $\Pr[\hat{v} < 0] = \Phi(-1/\sqrt{\alpha_1}) = 7.8 \times 10^{-4}$, the fraction of draws in which the *Gaussian* velocity model of Chapter 9 hands back a negative speed and Rusty reverses into the cell behind him. Below about six degrees of heading uncertainty, the dominant source of "slip" in this model is an artifact of describing a non-negative quantity with a Gaussian. Every noise model has a regime where its tails stop being a harmless approximation, and this is where the velocity model's is.

That table is the most important thing in the section, and it says something the gridworld literature usually hides. **A well-localized robot barely slips.** At five degrees of heading uncertainty, honest discretized slip is 1.3×10^{-5} . The slip parameter is not a property of the floor; it is overwhelmingly a property of the *localizer*. Discretized slip is a state-estimation error wearing a motion-model costume — which is a broad hint that the honest version of this chapter's problem is Chapter 22's.

The widgets let you set s up to 0.4 anyway. Read those settings as "a robot with a bad localizer, a wet floor, or 0.3 m cells that are too coarse for its dynamics" — all three are real, and the middle one is why the parameter is worth a slider.

21.3.4 Return, and why γ buys convergence

Fix a policy π and run it forever. The **return** is the discounted sum of payoffs,

$$R_T = \mathbb{E} \left[\sum_{\tau=0}^T \gamma^\tau r_\tau \right], \quad V^\pi(x) = \mathbb{E} \left[\sum_{\tau=0}^{\infty} \gamma^\tau r(x_\tau, \pi(x_\tau)) \mid x_0 = x \right]$$

Discounting does two jobs, and it is worth separating them because only one is mathematical.

The mathematical job: if $|r| \leq r_{\max}$ then $|V^\pi| \leq r_{\max}/(1 - \gamma)$, so an infinite sum of infinitely many terms is a finite number and the whole theory has something to be about. Without it, "reach the dock eventually" and "reach the dock in nine steps" both score $-\infty$ and arg max has nothing to chew on.

The modelling job: γ is a statement about how far ahead the robot cares. The **effective horizon** is $1/(1 - \gamma)$ steps, in the sense that payoff beyond it is discounted below $1/e$:

γ	0.8	0.9	0.95	0.98	0.99
effective horizon $1/(1-\gamma)$	5	10	20	50	100

For a 0.3 m grid, $\gamma = 0.98$ means the robot can see about 15 metres of consequence. Set $\gamma = 0.9$ and the charging dock becomes invisible from the far bedroom — the value function goes flat out there, the arrows go arbitrary, and the robot wanders. That failure is not a bug in the solver; it is the solver correctly reporting that under your stated preferences, the dock is not worth walking to. You can watch it happen with the Policy Painter’s γ slider.

21.3.5 The Bellman optimality equation

Define the optimal value function $V^*(x) = \max_{\pi} V^{\pi}(x)$ and the **state–action value**

$$Q^*(x, u) = r(x, u) + \gamma \sum_{x'} p(x' | x, u) V^*(x')$$

— the value of committing to u *once* and behaving optimally forever after. Then the whole of dynamic programming is the observation that V^* is the pointwise max of Q^* :

$$V^*(x) = \max_{u \in \mathcal{U}} \left[r(x, u) + \gamma \sum_{x'} p(x' | x, u) V^*(x') \right]$$

Read it as a sentence: *the best you can do from here is the best over first moves of (what that move pays now) plus (discounted, averaged over where it might land you, the best you can do from there)*. The colors match the widget below: green is the immediate payoff, blue is the discounted future, purple is the answer.

INTERACTIVE · w21.2

Bellman Stepper

A backup is bookkeeping, not linear algebra: add the payoff, add the discounted neighbours weighted by how likely you are to reach them, keep the biggest.

Run this simulation in the web edition: [#w21.2](/chapters/ch21-mdps)

That widget exists because the equation above looks like linear algebra and is not. A backup is clerical work: read a handful of neighbours, weight them by how likely you are to reach them, scale by γ , add the payoff, keep the biggest. Step it a few times and watch the value wave crawl backwards from the goal and

bend through the doorway — the same wave Chapter 20’s wave-front planner produced, which is not a coincidence and is proved below.

DERIVATION Deriving the Bellman equation from the finite-horizon recursion

$$V^*(x) = \max_u [r(x, u) + \gamma \sum p(x' | x, u) V^*(x')]$$

Thrun et al. build this by induction on the horizon, and so will we, because the induction is where the optimal-substructure argument actually lives.

Step 1 — horizon one. With exactly one action left, there is no future to trade against:

$$V_1(x) = \max_u r(x, u)$$

Step 2 — horizon two, by conditioning on the first move. Take an action u , collect $r(x, u)$, land in x' with probability $p(x' | x, u)$, and then you have a one-step problem left. The best you can do from x' with one step left is $V_1(x')$ by definition, so

$$V_2(x) = \max_u \left[r(x, u) + \gamma \sum_{x'} p(x' | x, u) V_1(x') \right]$$

Step 3 — why the tail is allowed to be optimal. This is the step that deserves suspicion. The claim is that the tail of an optimal 2-step plan is an optimal 1-step plan. It is true here for a specific reason: the payoff decomposes additively across time, the discount factors out, and the future dynamics depend on the past only through x' (Markov). So the total is $r(x, u) + \gamma \mathbb{E}[\text{tail}]$, and the tail term is maximized independently of the choice of u — for each possible x' separately. Break any of those three properties and the induction fails; risk-sensitive objectives, for instance, are not additively decomposable and genuinely do not admit this argument.

Step 4 — induct.

$$V_T(x) = \max_u \left[r(x, u) + \gamma \sum_{x'} p(x' | x, u) V_{T-1}(x') \right]$$

Step 5 — take the limit. With $|r| \leq r_{\max}$, the horizon- T and horizon- $(T+1)$ values differ by at most the tail of a geometric series, $\|V_{T+1} - V_T\|_{\infty} \leq \gamma^T r_{\max}$, which vanishes for $\gamma < 1$. So V_T is Cauchy in the sup norm and converges to some V_{∞} ; passing to the limit inside the finite max and finite sum gives $V_{\infty} = \mathcal{T}V_{\infty}$. The next derivation shows that fixed point is unique, so $V_{\infty} = V^*$. ■

21.3.6 Value iteration converges, and here is the stopping rule

Write the Bellman equation as an operator on value functions:

$$(\mathcal{T}V)(x) = \max_u \left[r(x, u) + \gamma \sum_{x'} p(x' | x, u) V(x') \right]$$

Value iteration is $V_{k+1} = \mathcal{T}V_k$, starting from $V_0 = 0$. The theorem that makes it an algorithm rather than a hope is that $\mathcal{T}$ is a contraction.

 **DERIVATION** $\mathcal{T}$ is a γ -contraction in the sup norm, and the stopping bound that follows

$$\|TU - TV\|_\infty \leq \gamma \|U - V\|_\infty \text{ and } \|V_k - V^*\|_\infty \leq \gamma / (1 - \gamma) \cdot \|V_k - V_{k-1}\|_\infty$$

Step 1 — the max inequality. For any two families of reals $\{a_u\}, \{b_u\}$ over a finite index set,

$$\left| \max_u a_u - \max_u b_u \right| \leq \max_u |a_u - b_u|$$

Proof: let $u^* = \arg \max_u a_u$. Then $\max_u a_u - \max_u b_u \leq a_{u^*} - b_{u^*} \leq \max_u |a_u - b_u|$. Swap the roles of a and b for the other direction. (Exercise 1 asks you to exhibit vectors where it is tight.)

Step 2 — apply it to the backup. Fix x and set $a_u = r(x, u) + \gamma \sum_{x'} p U(x')$, $b_u = r(x, u) + \gamma \sum_{x'} p V(x')$. The rewards cancel:

$$|(\mathcal{T}U)(x) - (\mathcal{T}V)(x)| \leq \gamma \max_u \left| \sum_{x'} p(x' | x, u) (U(x') - V(x')) \right|$$

Step 3 — push the expectation through. The transition row is a probability distribution, so it is an averaging operator and cannot amplify:

$$\left| \sum_{x'} p(x' | x, u) (U(x') - V(x')) \right| \leq \sum_{x'} p(x' | x, u) \|U - V\|_\infty = \|U - V\|_\infty$$

Step 4 — take the sup over x . $\|\mathcal{T}U - \mathcal{T}V\|_\infty \leq \gamma \|U - V\|_\infty$. Since $\gamma < 1$ and the space of bounded value functions with the sup norm is complete, Banach's fixed-point theorem gives a unique fixed point V^* and geometric convergence $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$ from any start.

Step 5 — the stopping rule. You cannot measure $\|V_k - V^*\|_\infty$; you can measure the change one sweep made. Chain them with the triangle inequality:

$$\|V_k - V^*\|_\infty \leq \sum_{j \geq k} \|V_{j+1} - V_j\|_\infty \leq \sum_{j \geq k} \gamma^{j-k+1} \|V_k - V_{k-1}\|_\infty = \frac{\gamma}{1-\gamma} \|V_k - V_{k-1}\|_\infty$$

So to guarantee $\|V_k - V^*\|_\infty \leq \epsilon$, stop when a sweep moves the value function by less than $\epsilon(1-\gamma)/\gamma$. That is the entire content of `stoppingThreshold(gamma, eps)` in the library, and it is why a request for $\epsilon = 10^{-6}$ at $\gamma = 0.98$ actually iterates until sweeps move things by 2×10^{-8} : at γ near one, the residual you can see is a wild underestimate of the error you have. ■

⚠ WHERE ROBOTS BREAK THIS

The factor $\gamma/(1-\gamma)$ is the single most common way to be wrong about a value function. At $\gamma = 0.99$ it is 99: a sweep that changes nothing by more than 0.01 may still leave you a full unit of reward away from the truth. Reporting "converged" because the residual looked small is how you get a policy that is confidently suboptimal in the one region you cared about.

21.3.7 A half-converged value function still gives a good policy

Greedy extraction is the reason value iteration is useful at all:

$$\pi_V(x) = \arg \max_u \left[r(x, u) + \gamma \sum_{x'} p(x' | x, u) V(x') \right]$$

📖 DERIVATION Greedy on V^* is optimal; greedy on a nearby V is nearly optimal

$$V^{\pi_V} \geq V^* - 2\gamma \|V - V^*\|_\infty / (1-\gamma)$$

Part A — exactness at the fixed point. Let $\pi^* = \pi_{V^*}$ and let $\mathcal{T}_\pi$ be the *linear* backup that follows π without a max: $(\mathcal{T}_\pi V)(x) = r(x, \pi(x)) + \gamma \sum_{x'} p(x' | x, \pi(x)) V(x')$. By the definition of $\arg \max$, $\mathcal{T}_{\pi^*} V^* = \mathcal{T} V^* = V^*$, so V^* is a fixed point of $\mathcal{T}_{\pi^*}$. But $\mathcal{T}_{\pi^*}$ is also a γ -contraction (Steps 3–4 above never used the max), so its fixed point is unique, and its fixed point is by definition V^{π^*} . Hence $V^{\pi^*} = V^*$: greedy is not merely good, it is exact.

Part B — the loss bound. Let $\epsilon = \|V - V^*\|_\infty$ and $\pi = \pi_V$. Three inequalities chained:

$$\|V^\pi - V^*\|_\infty \leq \underbrace{\|V^\pi - \mathcal{T}_\pi V\|_\infty}_{(i)} + \underbrace{\|\mathcal{T}_\pi V - V^*\|_\infty}_{(ii)}$$

For (ii): $\mathcal{T}_\pi V = \mathcal{T}V$ because π is greedy for V ; and $\|\mathcal{T}V - V^*\|_\infty = \|\mathcal{T}V - \mathcal{T}V^*\|_\infty \leq \gamma\epsilon$. For (i): $V^\pi = \mathcal{T}_\pi V^\pi$, so $\|V^\pi - \mathcal{T}_\pi V\|_\infty \leq \gamma\|V^\pi - V\|_\infty \leq \gamma(\|V^\pi - V^*\|_\infty + \epsilon)$. Substituting and solving for $\|V^\pi - V^*\|_\infty$:

$$\|V^\pi - V^*\|_\infty \leq \frac{2\gamma\epsilon}{1-\gamma}$$

■

Two readings of that bound, and they point in opposite directions.

Pessimistic: the amplification factor $2\gamma/(1-\gamma)$ is 98 at $\gamma = 0.98$. A value function good to 0.1 certifies only a policy within 9.8 of optimal — nearly vacuous.

Optimistic, and what actually happens: the bound is worst-case over adversarial MDPs. In a gridworld, $\arg \max$ depends only on the *order* of the Q -values at each state, and orderings stabilize long before magnitudes do. Measured on the Apartment scene, the greedy policy becomes **exactly** optimal — identical to π^* in all 925 free cells — after 45 of the 72 sweeps value iteration eventually takes. The residual at that moment is 2.07, so the theorem certifies only $\|V^\pi - V^*\|_\infty \leq 101$, on a problem whose optimal value at the start is 7.9. The bound is not wrong; it is simply not the thing that happens.

That gap is what real-time dynamic programming (Barto, Bradtke and Singh, 1995) is built to exploit: you may act on a value function you have no right to trust, as long as you keep backing it up along the states you actually visit. It is also why the Policy Painter can run the robot while the wave is still crossing the map.

21.3.8 Policy iteration: solve, improve, repeat

Value iteration nudges every state a little on every sweep. Policy iteration takes the other extreme: pick a policy, evaluate it *exactly*, then improve it everywhere at once.

The evaluation step is where the linear algebra finally shows up. Fix π ; there is no $\max$ any more, so the Bellman equation for V^π is a linear system:

$$V^\pi = r_\pi + \gamma P_\pi V^\pi \quad \Longleftrightarrow \quad (I - \gamma P_\pi) V^\pi = r_\pi$$

with P_π the $|\mathcal{X}| \times |\mathcal{X}|$ row-stochastic matrix $[P_\pi]_{xx'} = p(x' | x, \pi(x))$ — sparse, with at most three nonzeros per row in our gridworld. That is the *same shape of*

solve as the sparse Cholesky in Chapter 15, and the Rust implementation hands it to the same crate.

DERIVATION Policy iteration improves monotonically and terminates

$\pi_{k+1} \geq \pi_k$, with equality only at π^* ; at most $|U| \wedge |X|$ iterations, in practice a handful

Step 1 — the improvement is not worse. Let $\pi' = \pi_{V^\pi}$ be greedy with respect to V^π . By construction $\mathcal{T}_{\pi'} V^\pi = \mathcal{T} V^\pi \geq \mathcal{T}_{\pi} V^\pi = V^\pi$, pointwise.

Step 2 — monotone operators propagate that. $\mathcal{T}_{\pi'}$ is monotone: if $U \geq W$ pointwise then $\mathcal{T}_{\pi'} U \geq \mathcal{T}_{\pi'} W$, because it only adds a fixed vector and averages with nonnegative weights. Applying it repeatedly to $\mathcal{T}_{\pi'} V^\pi \geq V^\pi$ gives $\mathcal{T}_{\pi'}^k V^\pi \geq V^\pi$ for every k , and the left side converges to $V^{\pi'}$. Hence $V^{\pi'} \geq V^\pi$ pointwise. (This is the policy improvement theorem, and notice it is the monotonicity, not the contraction, doing the work.)

Step 3 — strictness and termination. If π' and π have the same value function, then $V^\pi = \mathcal{T} V^\pi$, so $V^\pi = V^*$ and π is optimal. Otherwise the improvement is strict at some state. There are finitely many deterministic stationary policies — $|U|^{|X|}$ of them — and the sequence of values is strictly increasing, so no policy repeats and the loop terminates.

Step 4 — what happens in practice. The bound is astronomical and irrelevant. On the four-cell hallway below, policy iteration finishes in **one** improvement. On the 927-state Apartment with $\gamma = 0.98$ it takes **23**, the first of which fixes 178 states and the last of which fixes 3. The rule of thumb — iterations grow like log of the state count, not like the state count — is observed everywhere and proved nowhere useful.

Step 5 — the interpolation. You do not have to solve the linear system exactly. Doing m linear backups instead is *modified policy iteration*: $m = 1$ is value iteration, $m = \infty$ is Howard's policy iteration, and the sweet spot for large sparse problems is usually $m \approx 20$. ■

21.3.9 Stochastic shortest paths: what navigation actually is

Discounting is a strange thing to want from a delivery robot. Nobody prefers a package delivered now over the same package delivered in a minute by a factor of 0.98^{60} ; we just want it delivered, quickly, and the discount was a mathematical convenience. The formulation that says what we mean is the **stochastic shortest path (SSP)**:

- $\gamma = 1$ — no discounting;
- one or more absorbing goal states with zero payoff;
- $r(x, u) = -1$ per step (or $-\text{cost}(u)$), so every action strictly hurts.

Then $-V^*(x)$ is literally the minimum expected number of steps to the goal,

which is a quantity you can hold in your head. But the contraction argument is gone: $\gamma = 1$ makes the modulus 1, and there is no Banach theorem to invoke. What replaces it is a condition on the *problem* rather than on the discount.

💡 THE SSP CONDITION

Value iteration converges for an undiscounted goal-absorbing MDP provided (i) at least one **proper** policy exists — one that reaches the goal with probability 1 from every state — and (ii) every improper policy has infinite expected cost from some state. Bertsekas and Tsitsiklis (1991) prove this and characterize exactly when it fails.

Both conditions are checkable, and both fail in ways you will meet. Condition (i) fails if the inflated map has a walled-off room: the goal is unreachable, $V^* = -\infty$ there, and a solver that does not detect it will happily return -10^{12} and an arrow field of noise. Condition (ii) fails if you give the robot a free action — a zero-cost stay — because then "stand still forever" is an improper policy with finite cost 0, and it is *optimal*. This is not a hypothetical; it is the single most common bug in hand-rolled SSP solvers, and it is why the library's stay action still charges `stepCost`.

21.3.10 The wave-front planner, unmasked

Now set the slip to zero. Each action has exactly one successor, $x' = f(x, u)$, so the sum over x' collapses to a single term and the SSP Bellman equation becomes

$$-V^*(x) = \min_u \left[\text{cost}(u) + (-V^*(f(x, u))) \right]$$

That is the Bellman–Ford relaxation. Sweep it synchronously and you have Bellman–Ford. Sweep it in-place in order of increasing $-V^*$ — always expanding the unfinished state closest to the goal — and you have Dijkstra. Fill it outward from the goal one ring at a time and you have Chapter 20's wave-front planner, exactly.

So the wave-front planner is not analogous to value iteration; it *is* value iteration, on a deterministic MDP with unit costs, with a particularly clever sweep order. Everything this chapter adds is what happens when the arrow out of a cell is no longer a promise. And the "potential field with no local minima" that Chapter 20 wanted is now definitional: V^* has no spurious local optimum because it is not a field somebody sculpted, it is the expected cost-to-go, and a state whose neighbours are all worse than it would be violating its own Bellman equation.

21.4 How much detour is noise worth?

We have said twice now that noise makes the optimal route longer. Here is the quantitative version, in the smallest world that can carry it: two corridors from start to goal, one short and flanked by a drop, one long and flanked by walls.

Before touching the widget, commit to a guess. The ledge is 6 cells; the detour is 16 — nearly three times as long. A veer on the ledge costs a penalty of 6 *and* dumps the robot back at the start. At what slip s does the optimal policy abandon the ledge?

INTERACTIVE · w21.3

Cliff Run

Optimal means best in expectation, not best case. The route flips at a critical noise level — and it is far lower than it feels.

Run this simulation in the web edition: </chapters/ch21-mdps#w21.3>

Write your number down, then read the derivation.

DERIVATION Closed-form route values and the critical slip

$$V_{ledge} = -(1 + 2sC)(1 - q^L)/(2sq^L), V_{detour} = -M/q, q = 1 - 2s$$

Let $q = 1 - 2s$ be the probability that a step goes as commanded, L the ledge length, M the detour length, and C the cliff penalty.

The detour. A veer bumps a wall, costing one step and no progress. The number of attempts needed for one cell of progress is geometric with success probability q , so its mean is $1/q$, and by linearity

$$V_{detour} = -\frac{M}{q}$$

Note this is *already* worse than $-M$: even the safe route pays for noise.

The ledge. Let V_k be the value with k cells still to go, so $V_0 = 0$, and let $W = V_L$ be the value at the start. Each step costs 1, and with probability $2s$ the robot goes over, pays C , and restarts at the start:

$$V_k = \underbrace{-1 - 2sC}_A + q V_{k-1} + 2s W$$

Write $B = A + 2sW$, a constant with respect to k . Then $V_k = B + qV_{k-1}$ with $V_0 = 0$ unrolls to a geometric sum:

$$V_k = B \frac{1 - q^k}{1 - q} = B \frac{1 - q^k}{2s}$$

Now impose self-consistency at $k = L$, where $V_L = W$. Writing $\rho = (1 - q^L)/(2s)$:

$$W = (A + 2sW)\rho \implies W(1 - 2s\rho) = A\rho \implies W = \frac{A\rho}{q^L}$$

using $1 - 2s\rho = 1 - (1 - q^L) = q^L$. Substituting A and ρ :

$$V_{\text{ledge}} = - \frac{(1 + 2sC)(1 - q^L)}{2s q^L}$$

Sanity checks. As $s \rightarrow 0$, $(1 - q^L)/(2s) \rightarrow L$ and $q^L \rightarrow 1$, so $V_{\text{ledge}} \rightarrow -L$: the deterministic answer. And V_{ledge} blows up like q^{-L} , exponentially in the length of the exposure, while V_{detour} degrades only like $1/q$. That asymmetry is the whole story.

The critical slip. s^* is the root of $V_{\text{ledge}}(s) = V_{\text{detour}}(s)$. It has no closed form, so bisect — on the two formulas, not on a simulation. For $L = 6$, $M = 16$, $C = 6$:

$$s^* = 0.0671$$

■

Six and a bit percent. Most readers guess somewhere between 0.15 and 0.3, because the detour is *so* much longer. The values say otherwise:

slip s	0.00	0.02	0.05	0.08	0.15	0.25
V^π ledge	-6.0	-8.6	-14.1	-22.6	-70.0	-504.0
V^π detour	-16.0	-16.7	-17.8	-19.0	-22.9	-32.0

The ledge is exponentially fragile and the detour is merely linearly annoying, so they cross early. And notice what $s^* = 0.067$ corresponds to in the units of the previous section: **about 15° of heading uncertainty**. The decision of whether to take the short route through the narrow gap is being made, in effect, by the localizer.

⚠ WHERE ROBOTS BREAK THIS

Two honest caveats the widget makes visible. First, long after the policy has switched, *individual* runs on the ledge still sometimes beat the detour — expectation is a claim about the average and nothing else, and if you

need a guarantee about the worst case you want a risk-sensitive objective (Akella et al., 2025), which is a different and harder problem. Second, the mean realized return converges to V^* slowly and from either side; twenty finished runs tell you almost nothing.

21.5 The algorithms

ALGORITHM `MDP_value_iteration(p, r,  $\gamma$ ,  $\epsilon$ )` COST $O(|X| \cdot |U| \cdot b)$ per sweep for branching factor b ; $O(\log \epsilon / \log \gamma)$ sweeps

in transition model, reward, discount, target accuracy

out V within ϵ of V^* , and the greedy policy π

```

1: for all  $x$  do  $\hat{V}(x) = 0$ 
2: repeat
3:    $\delta = 0$ 
4:   for all  $x$  do
5:      $v = \hat{V}(x)$ 
6:      $\hat{V}(x) = \max_u [r(x, u) + \gamma \sum_{x'} p(x' | x, u) \hat{V}(x')]$ 
7:      $\delta = \max(\delta, |\hat{V}(x) - v|)$ 
8:   endfor
9: until  $\delta < \epsilon(1 - \gamma)/\gamma$ 
10:  $\pi(x) = \arg \max_u [r(x, u) + \gamma \sum_{x'} p(x' | x, u) \hat{V}(x')]$ 
11: return  $\hat{V}, \pi$ 
```

This is Thrun et al.’s Table 15.1 with two additions: the stopping rule from the contraction proof (line 9), and the explicit policy extraction (line 10). One subtlety hides in line 6. If $\hat{V}$ is a single array that you write into as you go, later states in the sweep see the *updated* values of earlier ones — that is the Gauss–Seidel or **asynchronous** variant, and it is what the library does by default. If you write into a fresh array, it is the synchronous Jacobi variant. Thrun’s draft is explicit that this does not affect *whether* value iteration converges, only how fast. On the Apartment, Gauss–Seidel needs 72 sweeps where Jacobi needs 84 — and if you order the sweep by breadth-first distance from the goal, so that every backup reads neighbours that were updated moments ago, it drops to 33. Same algorithm, same fixed point, less than half the work, purely from the order of a for loop.

ALGORITHM `policy_iteration(p, r,  $\gamma$ )` COST a sparse $|X| \times |X|$ solve per iteration;
a handful of iterations
in transition model, reward, discount
out V, π — exactly, in finitely many steps

```

1: initialize  $\pi$  arbitrarily
2: repeat
3:   evaluate: solve  $(I - \gamma P_\pi)V^\pi = r_\pi$ 
4:   improve:  $\pi'(x) = \arg \max_u [r(x, u) + \gamma \sum_{x'} p(x' | x, u) V^\pi(x')]$ 
5:   if  $\pi' = \pi$  then return  $V^\pi, \pi$ 
6:    $\pi \leftarrow \pi'$ 
7: forever

```

ALGORITHM `prioritized_sweeping(p, r,  $\gamma$ ,  $\varepsilon$ ,  $V_0$ )` COST $O(\log|X|)$ per backup for
the heap; total backups data-dependent
in the MDP, a target accuracy, and a warm start
out V , backed up only where it mattered

```

1:  $H \leftarrow$  empty max-heap; for all  $x$  with residual  $\rho(x) > \theta$ : push  $x$  with  

   key  $\rho(x)$ 
2: while  $H$  nonempty do
3:    $x \leftarrow$  pop-max; if its key is stale, continue
4:    $v = V(x)$ ;  $V(x) = (\mathcal{T}V)(x)$ ;  $\Delta = |V(x) - v|$ 
5:   for each predecessor  $\tilde{x}$  of  $x$  do
6:      $\kappa = \gamma \cdot \max_u p(x | \tilde{x}, u) \cdot \Delta$ 
7:     if  $\kappa > \theta$  then push  $\tilde{x}$  with key  $\kappa$ 
8: return  $V$ 

```

The priority key on line 6 is an upper bound on how much this backup could possibly move that predecessor — so the heap orders states by *potential* change without paying for a trial backup. Moore and Atkeson (1993) introduced this for learned models; it is just as useful for known ones.

i NOTE

Prioritized sweeping is not universally faster, and it is worth knowing when it loses. Solving the Apartment MDP **cold** takes 226,050 prioritized backups against 86,400 for 72 Gauss–Seidel sweeps — $2.6\times$ *worse*, because when

every state needs updating, the heap is pure overhead. Its win is **repair**: after painting one new hazard cell, prioritized sweeping restores optimality in 4,487 backups where warm-started Gauss–Seidel needs 18 sweeps, or 21,600. That is the regime the Policy Painter lives in, and the regime a robot with a changing map lives in.

21.6 A worked example you can check by hand

Four states in a line: A , B , C , and the goal G , which absorbs. Rusty has two actions.

- **roll** — nudge forward. Advances one cell with probability 0.8; with probability 0.2 the wheels spin and nothing happens. Costs 1.
- **lunge** — dump enough current into the motors to guarantee the cell change. Costs 2.

Undiscounted, $\gamma = 1$: a stochastic shortest path. Every number below is reproduced by a Rust unit test and by self-check 1 in `lib/decision/__checks_ch21__.ts`.

21.6.1 The fixed point, by hand

Work backwards from the goal. At C , `roll` gives $-1 + 0.8 \cdot V(G) + 0.2 \cdot V(C) = -1 + 0.2 V(C)$, and if `roll` is optimal there, that equals $V(C)$:

$$0.8 V(C) = -1 \implies V(C) = -1.25$$

which is just $1/0.8$: the expected number of attempts to make one cell of progress. Each cell costs the same, so

$$V(B) = -1 + 0.8(-1.25) + 0.2 V(B) \implies V(B) = -2.5, \quad V(A) = -3.75$$

Is `roll` really optimal? Check the alternative at each state: `lunge` from C scores -2 against -1.25 ; from B , $-2 + (-1.25) = -3.25$ against -2.5 ; from A , -4.5 against -3.75 . Buying determinism at a price of 2 is a bad deal when the stochastic option costs 1.25 in expectation. The max gate has something to do at every state, and it always makes the same choice.

$$V^* = (-3.75, -2.5, -1.25, 0)$$

21.6.2 The first three sweeps

Now run the algorithm synchronously from $V_0 = 0$ and watch it get there. Every entry is one line of arithmetic:

sweep	$V(A)$	$V(B)$	$V(C)$	$V(G)$
0	0	0	0	0
1	-1	-1	-1	0
2	-2	-2	-1.2	0
3	-3	-2.36	-1.24	0
4	-3.488	-2.464	-1.248	0
∞	-3.75	-2.5	-1.25	0

Check sweep 3 at B yourself: $-1 + 0.8 \cdot V_2(C) + 0.2 \cdot V_2(B) = -1 + 0.8(-1.2) + 0.2(-2) = -2.36$. And notice the shape of the convergence. After three sweeps C is 99% of the way to its answer and A is only 80%; after four, C is done to three decimals and A still is not. Information flows backward from the goal at one cell per synchronous sweep, which is the whole reason sweep order matters.

Value iteration needs 24 sweeps to reach 10^{-12} here. Policy iteration needs **one** improvement: evaluate the initial all-roll policy exactly, discover it is already greedy, stop.

21.7 Implementation in Rust

The module is `crates/ch21_mdp`. The design constraint that shapes everything: a gridworld MDP has tens of thousands of states and at most three successors per state-action, so the transition model must be sparse and the inner loop must not allocate.

21.7.1 The model

RUST `crates/ch21_mdp/src/mdp.rs`

```
1 use nalgebra::DVector;
2
3 // One successor and its probability. `p` is f32 because a transition row is
4 // read far more often than it is written, and halving the row halves the cache
5 // misses in the inner loop -- the only place this crate spends time.
6 #[derive(Clone, Copy, Debug)]
7 pub struct Transition {
8     pub s: u32,
9     pub p: f32,
10 }
11
12 // A sparse row of  $p(\cdot | x, u)$ . Probabilities sum to 1 up to f32 epsilon.
13 pub type SparseDist = Vec<Transition>;
14
15 // A finite MDP with a compile-time action count.
16 ///
17 // `A` is a const generic because the action set of a gridworld is fixed by its
18 // connectivity (4, 8, or 9 with `stay`), and pinning it lets the per-state
19 // arrays live inline instead of behind a second indirection.
20 pub struct Mdp<const A: usize> {
21     pub n_states: usize,
22     // Indexed [state][action].
```

```

23 pub trans: Vec<SparseDist; A>,
24 ///  $r(x, u)$ : the expected immediate payoff, with entry payoff folded in.
25 pub reward: Vec<f64; A>,
26 ///  $\gamma$  in  $(0, 1]$ .  $\gamma = 1$  is legal only when `absorbing` is non-empty (SSP).
27 pub gamma: f64,
28 ///  $V(x) \geq 0$  here, and the episode stops.
29 pub absorbing: Vec<bool>,
30 pub action_labels: [&'static str; A],
31 }
32
33 impl<const A: usize> Mdp<A> {
34     ///  $Q(x, u) = r(x, u) + \gamma \sum_{x'} p(x' | x, u) V(x')$ .
35     #[inline]
36     pub fn q(&self, v: &[f64], x: usize, u: usize) -> f64 {
37         if self.absorbing[x] {
38             return 0.0;
39         }
40         let mut acc = 0.0;
41         for t in &self.trans[x][u] {
42             acc += f64::from(t.p) * v[t.s as usize];
43         }
44         self.reward[x][u] + self.gamma * acc
45     }
46
47     /// The max gate:  $(TV)(x)$  and the action that attains it.
48     #[inline]
49     pub fn backup(&self, v: &[f64], x: usize) -> (f64, u8) {
50         if self.absorbing[x] {
51             return (0.0, 0);
52         }
53         let mut best = f64::NEG_INFINITY;
54         let mut arg = 0u8;
55         for u in 0..A {
56             let q = self.q(v, x, u);
57             if q > best {
58                 best = q;
59                 arg = u as u8;
60             }
61         }
62         (best, arg)
63     }
64
65     ///  $\|TV - V\|_{\infty}$  over non-absorbing states. Zero exactly at the fixed point,
66     /// which makes it the only honest convergence certificate.
67     pub fn max_residual(&self, v: &DVector<f64>) -> f64 {
68         (0..self.n_states)
69             .filter(|&x| !self.absorbing[x])
70             .map(|x| (self.backup(v.as_slice(), x).0 - v[x]).abs())
71             .fold(0.0, f64::max)
72     }
73 }

```

21.7.2 Value iteration, with the stopping rule that the proof earned

RUST crates/ch21_mdp/src/vi.rs

```

1 use nalgebra::DVector;
2 use crate::mdp::Mdp;
3
4 pub struct ViResult {
5     pub v: DVector<f64>,
6     pub policy: Vec<u8>,
7     pub sweeps: usize,
8     /// ||V_k - V_{k-1}||_inf after each sweep -- the widgets' convergence wave.
9     pub residuals: Vec<f64>,
10    pub converged: bool,
11 }
12
13 /// Stop when one sweep moves V by less than this, and ||V - V*||_inf <= epsilon is
14 /// guaranteed. From ||V_k - V*|| <= gamma/(1-gamma).||V_k - V_{k-1}||.
15 ///
16 /// gamma = 1 (a stochastic shortest path) has no such bound: the contraction
17 /// modulus is 1 in the sup norm, so we fall back to the raw residual and the
18 /// caller is on the hook for a proper policy existing.
19 pub fn stopping_threshold(gamma: f64, eps: f64) -> f64 {
20     if gamma >= 1.0 { eps } else { eps * (1.0 - gamma) / gamma }
21 }
22
23 /// One in-place Gauss-Seidel sweep. Returns ||DeltaV||_inf.
24 ///
25 /// In place on purpose: a backup late in the sweep sees the states updated
26 /// earlier in it, so with a good ordering information crosses the whole map in
27 /// a single pass instead of one cell per sweep.
28 pub fn sweep_in_place<const A: usize>(mdp: &Mdp<A>, v: &mut DVector<f64>) -> f64 {
29     let mut residual: f64 = 0.0;
30     for x in 0..mdp.n_states {
31         if mdp.absorbing[x] {
32             v[x] = 0.0;
33             continue;
34         }
35         let next = mdp.backup(v.as_slice(), x).0;
36         residual = residual.max((next - v[x]).abs());
37         v[x] = next;
38     }
39     residual
40 }
41
42 /// Thrun et al., Table 15.1 -- `MDP_value_iteration`.
43 pub fn value_iteration<const A: usize>(mdp: &Mdp<A>, eps: f64, max_sweeps: usize) ->
44     ViResult {
45     let mut v = DVector::zeros(mdp.n_states);
46     let threshold = stopping_threshold(mdp.gamma, eps);
47     let mut residuals = Vec::new();
48
49     let converged = loop {
50         if residuals.len() >= max_sweeps {
51             break false;
52         }
53         let r = sweep_in_place(mdp, &mut v);
54         residuals.push(r);
55         if r < threshold {
56             break true;
57         }
58     };
59
60     ViResult { policy: greedy_policy(mdp, &v), sweeps: residuals.len(), v, residuals,
61         converged }

```

```

60 }
61
62 /// pi(x) = argmax_u Q(x, u). Optimal once V = V*; within 2gammaepsilon/(1-gamma) before
that.
63 pub fn greedy_policy<const A: usize>(mdp: &Mdp<A>, v: &DVector<f64>) -> Vec<u8> {
64     (0..mdp.n_states).map(|x| mdp.backup(v.as_slice(), x).1).collect()
65 }

```

21.7.3 Policy evaluation as a sparse solve

The exact evaluation step is where faer earns its place. The matrix $I - \gamma P_\pi$ is sparse, nonsymmetric, and — for $\gamma < 1$ — strictly diagonally dominant by rows, which means it is nonsingular and an LU with partial pivoting is stable without any reordering heroics.

RUST crates/ch21_mdp/src/pi.rs

```

1 use faer::sparse::{SparseColMat, Triplet};
2 use faer::prelude::*;
3 use nalgebra::DVector;
4 use crate::mdp::Mdp;
5 use crate::vi::greedy_policy;
6
7 /// Solve (I - gamma P_pi) V^pi = r_pi exactly.
8 ///
9 /// Same shape of problem as the normal equations in Chapter 15, and the same
10 /// crate solves it -- the difference is that here the matrix is nonsymmetric, so
11 /// it is LU rather than Cholesky.
12 pub fn policy_evaluation<const A: usize>(mdp: &Mdp<A>, pi: &[u8]) -> DVector<f64> {
13     let n = mdp.n_states;
14     let mut triplets: Vec<Triplet<usize, usize, f64>> = Vec::with_capacity(4 * n);
15     let mut rhs = Mat::<f64>::zeros(n, 1);
16
17     for x in 0..n {
18         if mdp.absorbing[x] {
19             // V(x) ? 0: a 1*1 identity row keeps the system square and the
20             // absorbing convention explicit rather than implied.
21             triplets.push(Triplet::new(x, x, 1.0));
22             continue;
23         }
24         let u = pi[x] as usize;
25         triplets.push(Triplet::new(x, x, 1.0));
26         for t in &mdp.trans[x][u] {
27             // Accumulating duplicates is exactly right here: a self-loop lands
28             // on the diagonal and must be subtracted from the identity.
29             triplets.push(Triplet::new(x, t.s as usize, -mdp.gamma * f64::from(t.p)));
30         }
31         rhs[(x, 0)] = mdp.reward[x][u];
32     }
33
34     let a = SparseColMat::try_new_from_triplets(n, n, &triplets)
35         .expect("policy transition matrix is well formed by construction");
36     let v = a.sp_lu().expect("I - gammaP_pi is nonsingular for gamma < 1").solve(&rhs);
37     DVector::from_iterator(n, (0..n).map(|i| v[(i, 0)]))
38 }
39

```

```

40 /// Howard's policy iteration: evaluate exactly, improve greedily, repeat.
41 pub fn policy_iteration<const A: usize>(mdp: &Mdp<A>, max_iter: usize) -> (DVector<f64>,
42   Vec<u8>, usize) {
43     let mut pi = vec![0u8; mdp.n_states];
44     let mut v = DVector::zeros(mdp.n_states);
45
46     for it in 0..max_iter {
47       v = policy_evaluation(mdp, &pi);
48       let next = greedy_policy(mdp, &v);
49       // Switch only on a strict improvement. Ties between equally good
50       // actions would otherwise make the loop oscillate forever.
51       let mut changed = 0usize;
52       let mut merged = pi.clone();
53       for x in 0..mdp.n_states {
54         let (a, b) = (next[x] as usize, pi[x] as usize);
55         if a != b && mdp.q(v.as_slice(), x, a) > mdp.q(v.as_slice(), x, b) + 1e-12 {
56           merged[x] = next[x];
57           changed += 1;
58         }
59       }
60       pi = merged;
61       if changed == 0 {
62         return (v, pi, it + 1);
63       }
64     }
65     (v, pi, max_iter)
66 }

```

21.7.4 Compiling a world into an MDP

This is the function that keeps the book's promise. It takes Chapter 13's occupancy grid and Chapter 9's velocity model and produces a finite MDP with no invented constants.

RUST crates/ch21_mdp/src/gridworld.rs

```

1 use rand::rngs::SmallRng;
2 use rand::SeedableRng;
3 use rand_distr::{Distribution, Normal};
4 use crate::mdp::{Mdp, SparseDist, Transition};
5
6 /// The eight compass moves, counter-clockwise from north.
7 pub const MOVES8: [(i32, i32); 8] =
8   [(0, 1), (-1, 1), (-1, 0), (-1, -1), (0, -1), (1, -1), (1, 0), (1, 1)];
9
10 pub struct GridSpec {
11   pub width: usize,
12   pub height: usize,
13   pub blocked: Vec<bool>,
14   /// Payoff collected on *entering* a cell. Goals positive, hazards negative.
15   pub payoff: Vec<f64>,
16   pub terminal: Vec<bool>,
17   /// Probability of veering into *each* lateral neighbour. Intended: 1 - 2s.
18   pub slip: f64,
19   pub gamma: f64,
20   pub step_cost: f64,

```

```

21     pub no_corner_cutting: bool,
22 }
23
24 /// The slip parameter, derived rather than decreed: drive the Chapter 9
25 /// velocity model one cell and bin the displacement by which neighbour its
26 /// bearing points at.
27 ///
28 /// `sigma_theta` is the heading error the localizer has left on the table, and
29 /// it dominates. The closed form derived in the text is
30 ///
31 ///  $s = \Phi(-(\pi/8) / \sqrt{\sigma_{\theta}^2 + \alpha_3 l^2/4})$ 
32 ///
33 /// so with  $\alpha_3 = 0.05$  and  $l = 0.3$  m the wheels contribute a fixed 0.034 rad
34 /// regardless of speed, while fifteen degrees of heading error puts  $s$  at 0.068.
35 /// We sample rather than evaluate the formula so that changing the motion model
36 /// changes the answer, which is the whole point of deriving it.
37 pub fn slip_from_velocity_model(
38     v: f64, cell: f64, alpha1: f64, alpha3: f64, sigma_theta: f64, seed: u64,
39 ) -> f64 {
40     let mut rng = SmallRng::seed_from_u64(seed); // never thread_rng: demos are
41     reproducible
42     let dt = cell / v;
43     let half_sector = std::f64::consts::FRAC_PI_8;
44     let heading = Normal::new(0.0, sigma_theta.max(1e-9)).unwrap();
45     let dv = Normal::new(0.0, (alpha1 * v * v).sqrt().max(1e-12)).unwrap();
46     let dw = Normal::new(0.0, (alpha3 * v * v).sqrt().max(1e-12)).unwrap();
47
48     const N: usize = 40_000;
49     let lateral = (0..N)
50         .filter(|_| {
51             let th0 = heading.sample(&mut rng);
52             let vh = v + dv.sample(&mut rng);
53             let wh = dw.sample(&mut rng);
54             // Thrun et al., Table 5.3: the arc traced by perturbed (v, omega).
55             let (dx, dy) = if wh.abs() < 1e-9 {
56                 (vh * th0.cos() * dt, vh * th0.sin() * dt)
57             } else {
58                 let r = vh / wh;
59                 (
60                     -r * th0.sin() + r * (th0 + wh * dt).sin(),
61                     r * th0.cos() - r * (th0 + wh * dt).cos(),
62                 )
63             };
64             dy.atan2(dx).abs() > half_sector
65         })
66         .count();
67     // Both sides together are 2s.
68     (lateral as f64 / N as f64 / 2.0).min(0.49)
69 }
70
71 /// Compile a spec into a finite MDP over the eight compass moves.
72 ///
73 /// The rule, in one sentence: the commanded direction happens with probability
74 /// 1 - 2s, each 45° neighbour of it with probability s, and any outcome that
75 /// would leave the map or enter a wall leaves the robot where it was -- while
76 /// still charging for the attempt, because the wheels turned.
77 pub fn grid_world_mdp(spec: &GridSpec) -> Mdp<8> {
78     let n = spec.width * spec.height;
79     let s = spec.slip.clamp(0.0, 0.49);
80     let idx = |i: i32, j: i32| (j as usize) * spec.width + (i as usize);
81     let free = |i: i32, j: i32| {

```

```

81     i >= 0 && j >= 0 && (i as usize) < spec.width && (j as usize) < spec.height
82     && !spec.blocked[idx(i, j)]
83 };
84
85 let mut trans = Vec::with_capacity(n);
86 let mut reward = Vec::with_capacity(n);
87 let mut absorbing = vec![false; n];
88
89 for j in 0..spec.height as i32 {
90     for i in 0..spec.width as i32 {
91         let x = idx(i, j);
92         absorbing[x] = spec.blocked[x] || spec.terminal[x];
93
94         let mut rows: [SparseDist; 8] = Default::default();
95         let mut rs = [0.0f64; 8];
96         for a in 0..8usize {
97             let land = |dir: usize| -> u32 {
98                 let (di, dj) = MOVES8[dir % 8];
99                 if !free(i + di, j + dj) { return x as u32; }
100                 if spec.no_corner_cutting && di != 0 && dj != 0
101                     && (!free(i + di, j) || !free(i, j + dj)) {
102                     return x as u32;
103                 }
104                 idx(i + di, j + dj) as u32
105             };
106             let row = condense(&[
107                 Transition { s: land(a),          p: (1.0 - 2.0 * s) as f32 },
108                 Transition { s: land(a + 7),      p: s as f32 },
109                 Transition { s: land(a + 1),      p: s as f32 },
110             ]);
111             // r(x,u) = -cost(u) + Sigma p(x'|x,u).payoff(x'), so goal payoff is
112             // collected by the *neighbours* of the goal.
113             let (di, dj) = MOVES8[a];
114             let len = ((di * di + dj * dj) as f64).sqrt();
115             rs[a] = -spec.step_cost * len
116                 + row.iter().map(|t| f64::from(t.p) * spec.payoff[t.s as usize]).sum
117                 ::<f64>());
118             rows[a] = row;
119         }
120         trans.push(rows);
121         reward.push(rs);
122     }
123 }
124
125 Mdp {
126     n_states: n, trans, reward, gamma: spec.gamma, absorbing,
127     action_labels: ["N", "NW", "W", "SW", "S", "SE", "E", "NE"],
128 }
129
130 /// Merge duplicate successors and renormalize. Sparse rows must sum to 1, and
131 /// after wall-clamping they very often do not without this.
132 fn condense(pairs: &[Transition]) -> SparseDist {
133     let mut out: SparseDist = Vec::with_capacity(pairs.len());
134     for t in pairs.iter().filter(|t| t.p > 0.0) {
135         match out.iter_mut().find(|o| o.s == t.s) {
136             Some(o) => o.p += t.p,
137             None => out.push(*t),
138         }
139     }
140     let total: f32 = out.iter().map(|t| t.p).sum();

```

```

141     for t in out.iter_mut() { t.p /= total; }
142     out.sort_unstable_by_key(|t| t.s);
143     out
144 }

```

21.7.5 The worked example, as a test

RUST crates/ch21_mdps/examples/hallway_ssp.rs

```

1  use ch21_mdps::mdp::{Mdp, Transition};
2  use ch21_mdps::vi::value_iteration;
3
4  /// A, B, C, G in a line. `roll` advances w.p. p and costs 1; `lunge` is
5  /// deterministic and costs 2. gamma = 1, G absorbing: a stochastic shortest path.
6  pub fn hallway_ssp(p: f64, lunge_cost: f64) -> Mdp<2> {
7      let mut trans = Vec::new();
8      let mut reward = Vec::new();
9      for x in 0..4usize {
10         if x == 3 {
11             trans.push([vec![Transition { s: 3, p: 1.0 }], vec![Transition { s: 3, p: 1.0
12             }]]);
13             reward.push([0.0, 0.0]);
14             continue;
15         }
16         trans.push([
17             vec![
18                 Transition { s: x as u32, p: (1.0 - p) as f32 },
19                 Transition { s: x as u32 + 1, p: p as f32 },
20             ],
21             vec![Transition { s: x as u32 + 1, p: 1.0 }],
22         ]);
23         reward.push([-1.0, -lunge_cost]);
24     }
25     Mdp {
26         n_states: 4, trans, reward, gamma: 1.0,
27         absorbing: vec![false, false, false, true],
28         action_labels: ["roll", "lunge"],
29     }
30 }
31
32 fn main() {
33     let mdp = hallway_ssp(0.8, 2.0);
34     let out = value_iteration(&mdp, 1e-12, 10_000);
35     println!("V* = {:?}", out.v.as_slice()); // [-3.75, -2.5, -1.25, 0.0]
36     println!("pi* = {:?}", out.policy);      // [0, 0, 0, 0] (all `roll`)
37     println!("sweeps = {}", out.sweeps);    // 24
38 }
39
40 #[cfg(test)]
41 mod tests {
42     use super::*;
43     use approx::assert_relative_eq;
44     use nalgebra::DVector;
45
46     /// The chapter's hand-computed fixed point, to the digit.
47     #[test]
48     fn hallway_ssp_worked_example() {

```



```

48     let mdp = hallway_ssp(0.8, 2.0);
49     let out = value_iteration(&mdp, 1e-12, 10_000);
50     for (got, want) in out.v.iter().zip([-3.75, -2.5, -1.25, 0.0]) {
51         assert_relative_eq!(got, &want, epsilon = 1e-9);
52     }
53     // Determinism at 2 per cell is a bad deal against 1/0.8 = 1.25.
54     assert_eq!(out.policy, vec![0, 0, 0, 0]);
55 }
56
57 /// And the first three sweeps, which the chapter prints as a table.
58 #[test]
59 fn hallway_ssp_first_three_sweeps() {
60     let mdp = hallway_ssp(0.8, 2.0);
61     let mut v = DVector::zeros(4);
62     let want = [
63         [-1.0, -1.0, -1.0, 0.0],
64         [-2.0, -2.0, -1.2, 0.0],
65         [-3.0, -2.36, -1.24, 0.0],
66     ];
67     // Synchronous, to match the table: Gauss-Seidel would already be ahead.
68     for row in want {
69         let mut next = v.clone();
70         for x in 0..4 { next[x] = mdp.backup(v.as_slice(), x).0; }
71         v = next;
72         for (got, w) in v.iter().zip(row) {
73             assert_relative_eq!(got, &w, epsilon = 1e-12);
74         }
75     }
76 }
77 }

```

21.8 Putting it together

Run the whole thing on the Apartment: 0.3 m cells, walls inflated by 0.16 m for Rusty's radius, $40 \times 30 = 1200$ cells of which 927 are free states, eight compass moves, no corner cutting, $s = 0.2$, $\gamma = 0.98$, a charging dock worth +100 and a spill worth -60.

method	work to $\epsilon = 10^{-6}$	notes
value iteration, synchronous (Jacobi)	84 sweeps	100,800 backups
value iteration, in place (Gauss-Seidel)	72 sweeps	86,400 backups — 14% fewer, for free
the same, swept in breadth-first order from the goal	33 sweeps	39,600 backups — the order is the algorithm
policy iteration	23 improvements	23 sparse solves; agrees with VI to 7×10^{-9}
prioritized sweeping, cold	226,050 backups	2.6× worse — the heap is pure overhead

method	work to $\epsilon = 10^{-6}$	notes
prioritized sweeping, repairing one painted hazard	4,487 backups	vs 21,600 for warm Gauss–Seidel

Then the check that matters: does the value function mean anything? V^* at the start cell is 7.898. Twenty thousand seeded rollouts of the greedy policy, sampling transitions from the same sparse rows, average 8.05 — within 2%, with a mean journey of 45.5 steps. Monte Carlo meeting dynamic programming on the page, which is the only way to be sure that the number the solver printed is the number the robot will actually experience.

And the sweep across noise, which is the chapter's thesis in five rows:

s	0.0	0.1	0.2	0.3	0.4
$V^*(x_0)$	20.01	13.48	7.90	1.45	−3.15
mean steps to dock	37.0	41.3	45.5	51.7	56.9
VI sweeps to 10^{-8}	41	64	77	114	105

Noise costs value, costs steps, and costs sweeps. Nothing in the policy was "made safe"; the expectation did all of it.

21.8.1 What this chapter is not

Not reinforcement learning. Q-learning, SARSA and actor–critic solve *these* Bellman equations with p and r unknown, learning from samples instead of from a model. That is a genuinely different problem — exploration becomes an issue, and convergence proofs get much harder — and it belongs to Chapter 25. What is worth carrying forward is that the equation being solved is the one above, unchanged.

Not continuous state. Discretizing a 3-DOF pose at any useful resolution is already millions of states, and adding velocities makes it hopeless. The answer in practice is not a finer grid but a different algorithm: sample trajectories from the current state, evaluate them, act, throw them away, repeat. That is Chapter 23, where the value function of this chapter reappears as the terminal cost that makes a short horizon behave like a long one.

Not observable. Here is the bridge, and it is worth being blunt about. Take the Policy Painter's converged policy — provably optimal, arrows correct in every cell — and kidnap Rusty. The policy is still optimal. It is also useless, because $\pi(x)$ requires x , and after a kidnapping the robot has a *belief*, not a state. Nothing in this chapter's machinery has an input port for a distribution.

Chapter 22 fixes this in the only way the mathematics allows: promote the belief to the state. The belief-MDP is a genuine MDP over a continuous, high-dimensional space, the Bellman equation is unchanged, and every algorithm

here still applies in principle. Whether they apply in practice is what makes that chapter hard. One of them — QMDP — is nothing more than this chapter's Q^* averaged against the belief, $\sum_x b(x)Q^*(x, u)$, so keep the Q -values; you will need them in about twenty pages.

21.9 Exercises

1 FOUNDATION • The max inequality, and when it is tight

Prove $|\max_u a_u - \max_u b_u| \leq \max_u |a_u - b_u|$ for finite index sets, and exhibit vectors $a, b \in \mathbb{R}^3$ where it holds with equality. Then find vectors where the left side is 0 and the right side is as large as you like, and say in one sentence what that means for the *tightness* of the contraction bound on a real gridworld.

2 FOUNDATION •• The hallway in closed form

For the four-cell hallway with success probability p (and lunge disabled), derive $V(A), V(B), V(C)$ in closed form and verify the $p = 0.8$ numbers. Then: at what lunge cost does the optimal policy start preferring it, and is the answer the same at every state? Finally, predict how many *synchronous* sweeps value iteration needs to reach $\|V_k - V^*\|_\infty < 0.01$ using the geometric bound with the effective contraction modulus $1 - p$, and compare with the measured count.

3 FOUNDATION ••• Why undiscounted is harder

Build two four-state undiscounted goal-absorbing MDPs, both starting value iteration from $V_0 = 0$. On the first, value iteration **diverges**: some state's value runs to $-\infty$. On the second it **converges to a finite fixed point whose optimal policy never reaches the goal**. Say which of the two conditions in the SSP box each one violates. (Hint for the second: one free action is enough.) Then repair each in two ways — a strictly positive cost on every action, and a discount $\gamma < 1$ — and say what each repair changes about the *optimal policy*, not merely about the solver's ability to find it.

4 CONCEPTUAL •• Predict the cliff

In the Cliff Run, before committing your prediction: estimate s^* for the default geometry using only the two closed forms in the derivation and a calculator. Then use the widget's guess slider to commit, reveal, and bisect with the slip slider to find where the arrow at the start actually flips. Finally — and this is the interesting part — explain why the slip at which *realized* runs start preferring the detour is noticeably higher than s^* , and what that gap is made of.

5 CONCEPTUAL •• Make the discount change the topology

Using the Policy Painter, construct a painting in which lowering γ does not merely make the arrows lazier but routes the robot through a *different doorway*. Record both arrow fields and both values at the start cell, and explain the mechanism in terms of the effective horizon $1/(1 - \gamma)$ and the two routes' lengths. Then predict, before checking, whether raising the slip makes the effect appear at a higher or lower γ .

6 PRACTICAL •• Prioritized sweeping, and when it loses

Implement `prioritized_sweeping` in Rust with a `BinaryHeap` keyed on the Bellman residual and the priority rule from the algorithm box. Reproduce both halves of this chapter's measurement on the Apartment: that it *loses* to Gauss–Seidel on a cold solve (226k backups vs 86k), and that it wins by roughly 5× when repairing a single changed reward cell. Plot backups against $\|TV - V\|_\infty$ for both, and explain the crossover in terms of what fraction of states have a nonzero residual.

7 PRACTICAL ••• The wave-front planner, generated

Add an SSP mode ($\gamma = 1$) to the crate with proper-policy detection: build the predecessor graph with `petgraph`, run a reverse traversal from the goal set, and flag every state that cannot reach a goal instead of letting its value run to $-\infty$. Then set the slip to zero and assert that $-V^*$ equals, cell for cell, the distance field produced by Chapter 20's wave-front planner on the same map. If the assert fails, the most likely culprit is diagonal cost: check that you charged $\sqrt{2}$.

21.10 References

Bellman, R. (1957). A Markovian Decision Process *Indiana University Mathematics Journal* 6(4), 679–684.

The five-page paper that named the object and wrote down the optimality equation this whole chapter revolves around. doi:10.1512/iumj.1957.6.56038

Howard, R. A. (1960). Dynamic Programming and Markov Processes *MIT Press*.

Policy iteration's origin, still the clearest account of why evaluate-then-improve terminates. The algorithm in this chapter's second box is Howard's, essentially unmodified. [Link](#)

Bertsekas, D. P. and Tsitsiklis, J. N. (1991). An Analysis of Stochastic Shortest Path Problems *Mathematics of Operations Research* 16(3), 580–595.

The proper-policy conditions in the SSP box, proved. This is the reference for why undiscounted navigation MDPs are well posed — and exactly when they are not. doi:10.1287/moor.16.3.580

Moore, A. W. and Atkeson, C. G. (1993). Prioritized sweeping: Reinforcement learning with less data and less time *Machine Learning* 13(1), 103–130.

The priority rule implemented in this chapter’s third algorithm box, including the argument for keying on how much of a change can reach each predecessor. doi:10.1007/BF00993104

Barto, A. G., Bradtke, S. J., and Singh, S. P. (1995). Learning to act using real-time dynamic programming *Artificial Intelligence* 72(1), 81–138.

Asynchronous DP made into a control architecture: back up only the states you actually visit, and act on the half-converged value function. The formal justification for what the Policy Painter does every frame. doi:10.1016/0004-3702(94)00011-0

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 15 is this chapter’s baseline: the MDP framing, the payoff notation, and the MDP_value_iteration algorithm of Table 15.1, which the first algorithm box reproduces with a stopping rule added. [link](#)

Kurniawati, H. (2022). Partially Observable Markov Decision Processes and Robotics *Annual Review of Control, Robotics, and Autonomous Systems* 5, 253–277.

The modern survey of what happens when you drop this chapter’s full-observability assumption. Read the introduction now for the bridge to Chapter 22; read the rest after it. doi:10.1146/annurev-control-042920-092451

Akella, P., Dixit, A., Ahmadi, M., Lindemann, L., Chapman, M. P., Pappas, G. J., Ames, A. D., and Burdick, J. W. (2025). Risk-Aware Robotics: Tail Risk Measures in Planning, Control, and Verification *IEEE Control Systems* 45(4), 46–78.

What to do when maximizing the expectation is not what you meant — CVaR and other tail measures, and what they cost you in tractability. The honest answer to the Cliff Run’s caveat about individual runs. doi:10.1109/MCS.2025.3577050

Decision Making II: POMDPs and Belief-Space Planning

PLANNING AND ACTING UNDER UNCERTAINTY · Advanced · 70 min

“

Thus, even though the value function is defined over a continuum, it can be represented on a digital computer—up to the accuracy of floating point numbers.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics (1999–2000 draft), Chapter 16

Chapter 21 ended with a policy that is optimal and useless. It answers "what should I do in state x ?" for every state in the map, and Rusty has never once been in a position to look up the answer, because Rusty does not know x . It knows $\text{bel}(x)$.

This chapter closes the book's central circle. The belief that Parts II through IV taught you to *maintain* becomes the thing you *plan over*. That single move — from planning in state space to planning in belief space — is what makes a robot pay for a sensor reading, hug a wall it could have driven past, or take a detour whose only payoff is finding out where it is. None of those behaviors is hand-coded here. They are what the Bellman equation returns once the state of the decision problem is a distribution.

The price is severe and worth naming up front: exact POMDP solving is combinatorial, not merely expensive. Most of this chapter is therefore about the four honest ways out — collapse the uncertainty (QMDP), sample the belief space (PBVI), sample the future (POMCP, DESPOT), or compress the belief to a statistic you can plan over (AMDP). Each one is exactly one paragraph of theory and one page of Rust, and each one fails somewhere you can see.

🎯 AFTER THIS CHAPTER YOU CAN

- State the POMDP tuple and recognize its belief transition $\tau(\mathbf{b}, \mathbf{u}, \mathbf{z})$ as the Chapter 5 Bayes filter, unchanged
- Convert a POMDP into an MDP over the belief simplex, and say precisely what that conversion costs
- Prove that a finite-horizon POMDP value function is piecewise-linear and convex, and read the policy off the envelope's breakpoints
- Count the exact backup's candidates, and place POMDP solving on the complexity map — approximation is structural, not laziness
- Derive QMDP and prove the exact class of decisions it can never make
- Implement point-based (PBVI) and online sampled (POMCP) solvers on top of the particle machinery Chapter 8 already built
- Compress a pose belief to (MAP, entropy) and watch coastal navigation fall out of value iteration rather than being scripted

📖 ASSUMED

- The Bayes filter recursion and the evidence term η^{-1} (Chapter 5)
- Entropy of a discrete distribution, and of a Gaussian (Chapter 2)
- Particle sets and low-variance resampling (Chapter 8)
- Monte Carlo localization and what a bimodal belief looks like (Chapter 12)
- MDPs, value iteration, the γ -contraction, and $Q^*(\mathbf{x}, \mathbf{u})$ (Chapter 21)

22.1 Two doors, and a microphone that costs a dollar

Rusty is in the Hallway in front of two identical doors. Behind one is the charging dock. Behind the other is an open stairwell. The doors look the same, the floor looks the same, and the map is no help: this corridor is symmetric.

There is a microphone. Pressing it costs one unit of battery and returns a direction — *left* or *right* — that is correct 85% of the time. Opening the right door earns +10; opening the wrong one costs −100 and a trip to the repair bench.

That is the whole problem, and it is the oldest toy in the field: Kaelbling, Littman and Cassandra's **tiger problem** (1998), which we will use for every number in this chapter because every number in it can be checked by hand. Play it before reading further. The widget opens on the optimal pilot; take control with **You play** and try to beat it.

 INTERACTIVE · w22.1**The Tiger Door Console**

Sensing is an action with a price, and the price is worth paying exactly when the belief is near the middle. Acting on the most likely state throws that away.

Run this simulation in the web edition: [/chapters/ch22-pomdps #w22.1](#)

Three things are worth noticing, and each is a section of this chapter.

Sensing is an action, and it has a price. Listening does not move the robot and does not open anything. Its entire value is that it moves the *belief*. In Chapter 21's MDP there was no such action — every action was worth something because of where it took you. Here an action is worth something because of what it tells you, and the Bellman equation has to be able to say so.

The value function lives over the belief, and it is made of straight lines. The lower panel is the actual object this chapter is about: V plotted against b (tiger-left). Each faint gray line is one α -vector; the bold curve is their upper envelope. The policy is not stored anywhere — it is *read off* the envelope, one action per linear piece.

Certainty is worth money, and the shape of V says how much. V is convex. It is lowest in the middle, where the robot knows nothing, and highest at the ends, where it knows everything. The vertical gap between the chord and the curve is not a metaphor — it is the expected value of perfect information, in the same units as the reward. At $b = \frac{1}{2}$ the tiger's converged numbers give $28.4028 - 19.3714 = 9.03$, so a free oracle would be worth just over nine points, and a microphone at one point per press is a bargain. That is why the optimal policy burns battery on presses that accomplish nothing physical.

 THE ONE SENTENCE VERSION

A POMDP is an MDP whose state is a belief. Because the belief reward and the belief transition are both built out of *linear* operations on b , the optimal finite-horizon value function is a maximum of finitely many linear functions — which is the only reason a problem defined over a continuum of distributions can be solved on a computer at all.

22.2 Building intuition

22.2.1 Averaging over worlds is not planning under uncertainty

The tempting move — and the one almost every first implementation makes — is this: I do not know which world I am in, so let me solve each candidate world, then average the answers. Compute $Q^*(x, u)$ for the tiger-left world and

the tiger-right world, weight by b , act greedily.

That is a real algorithm, it is called QMDP, and we will implement it. It is also structurally incapable of the behavior in the widget above, for a reason the draft of *Probabilistic Robotics* states flatly: “the planning problem in partially observable environment cannot be solved by considering all possible environments and averaging the solution.”

The reason is that averaging happens *before* the max. In each individual world the robot knows where the tiger is, so listening is a waste of a battery unit; the microphone earns its keep only in the counterfactual where you might have been wrong. Average first and that counterfactual is gone. Switch the widget’s pilot to **QMDP** and watch the envelope collapse from nine linear pieces to three — and look at the middle one. It is *horizontal*: QMDP assigns listening the same value 189 at every belief, because under its assumption the fog lifts anyway. A flat piece is an action whose worth does not depend on what you know, which is precisely an action with no informational value.

22.2.2 Why V is convex, in one paragraph and no algebra

Suppose someone offers you a choice. Option A: you are told, honestly, which of two belief states b_1 or b_2 you are in — a coin decides, with probability λ and $1 - \lambda$. Option B: you are told nothing, and you must act on the mixture $\lambda b_1 + (1 - \lambda)b_2$.

Option A is worth $\lambda V(b_1) + (1 - \lambda)V(b_2)$, because you get to run the best policy for whichever one came up. Option B is worth $V(\lambda b_1 + (1 - \lambda)b_2)$, and whatever policy is best there was *available* under option A too. So

$$V(\lambda b_1 + (1 - \lambda)b_2) \leq \lambda V(b_1) + (1 - \lambda)V(b_2).$$

That is convexity, and it is the formal statement of *information has non-negative value*. The gap between the two sides at any belief is what the robot should be willing to pay to have its uncertainty resolved. Read the widget’s envelope again with that in mind: the deeper the sag in the middle, the more the microphone is worth.

22.3 The mathematics

22.3.1 Notation

$X, \mathcal{U}, \mathcal{Z}$	Finite state, action and observation sets. Sizes $ X $, $ U $, $ Z $.
$b, b(x)$	The belief — exactly $\text{bel}(x)$ from Chapter 5, now serving as the state of a planning problem.
$\Delta^{ X -1}$	The belief simplex. A segment for two states, a triangle for three.
$\tau(b, u, z)$	The belief transition: one Bayes-filter predict-and-correct, written as a function.

$p(z \mid b, u)$	Evidence under a belief — the branching probability of the belief MDP. Chapter 5's η^{-1} .
$\rho(b, u) = \sum_x b(x)r(x, u)$	Belief reward. Linear in b , which is where the whole theory comes from.
$\alpha^{(k)}$	An α -vector: one linear piece of V , tagged with the action it commits to.
$\Gamma_T = \{\alpha^{(k)}\}$	The α -vector set at horizon T . $V_T(b) = \max_k \langle \alpha^{(k)}, b \rangle$.
$g_k^{u,z}$	Backprojection of $\alpha^{(k)}$ through (u, z) : what that piece is worth seen from x , before observing.
$\bar{b} = (\arg \max_x b(x), H[b])$	The AMDP compression: most likely state plus one number of uncertainty.

⚠ WHERE ROBOTS BREAK THIS

Notation collision. α -vectors are standard POMDP vocabulary and have nothing whatever to do with the motion-noise parameters $\alpha_1 \dots \alpha_6$ of Chapter 9. We write value vectors with a superscript, $\alpha^{(k)}$, noise parameters with a subscript, α_1 , and never both in one equation.

22.3.2 The POMDP tuple

A **partially observable Markov decision process** is Chapter 21's MDP with one thing added and one thing taken away:

$(\mathcal{X}, \mathcal{U}, \mathcal{Z}, p(x' \mid x, u), p(z \mid x', u), r(x, u), \gamma, b_0)$

Added: the observation set $\mathcal{Z}$ and the measurement model $p(z \mid x', u)$ — which is Chapter 10's object, now allowed to depend on which action you took, because "take a photograph" and "drive forward" do not return the same kind of reading. Taken away: the assumption that the agent is ever told x . It sees only $u_{1:t}$ and $z_{1:t}$, and starts from a prior b_0 .

Everything else is inherited. In particular $r(x, u)$ is still Chapter 21's payoff, collected on taking u in x , and $\gamma \in (0, 1)$ still buys convergence.

The tiger, completely specified, is the instance every number below refers to:

$\mathcal{X}$	$\{x_L, x_R\}$ — the tiger is behind the left or the right door
$\mathcal{U}$	$\{\text{listen}, \text{open}_L, \text{open}_R\}$
$\mathcal{Z}$	$\{z_L, z_R\}$ — the growl came from the left or the right
$p(z_L \mid x_L, \text{listen})$	0.85 (the hearing-accuracy slider)
$r(\cdot, \text{listen})$	−1
correct door / tiger door	+10 / −100

$\mathcal{X}$	$\{x_L, x_R\}$ — the tiger is behind the left or the right door
after opening	the tiger is re-placed uniformly and the next growl is pure noise, so the belief resets to $(\frac{1}{2}, \frac{1}{2})$
γ	0.95

22.3.3 The belief transition is the Bayes filter, wearing a new hat

Nothing new happens here at all, and that is the point worth making loudly. Given a belief, an action and an observation, the next belief is

$$b'(x') = \tau(b, u, z)(x') = \eta \, p(z \mid x', u) \sum_x p(x' \mid x, u) b(x)$$

which is Chapter 5's two lines — predict, then correct — with the integral become a sum. The normalizer is the **evidence**, and here it stops being house-keeping and becomes load-bearing, because it is the *transition probability of the belief MDP*:

$$p(z \mid b, u) = \eta^{-1} = \sum_{x'} p(z \mid x', u) \sum_x p(x' \mid x, u) b(x)$$

Two consequences that shape everything downstream. First, the belief dynamics are *deterministic* given (u, z) : all the stochasticity of a POMDP sits in which observation arrives. Second, the belief MDP's branching factor is $|Z|$, not $|\mathcal{X}|$ — a tree over action–observation histories, which is exactly the object POMCP builds later in this chapter.

Run the tiger numbers by hand. From $b(x_L) = \frac{1}{2}$, hearing z_L :

$$b'(x_L) = \frac{0.85 \times 0.5}{0.85 \times 0.5 + 0.15 \times 0.5} = 0.85, \quad p(z_L \mid b, \text{listen}) = 0.5$$

A second, consistent z_L :

$$b'(x_L) = \frac{0.85 \times 0.85}{0.85 \times 0.85 + 0.15 \times 0.15} = \frac{0.7225}{0.745} = 0.969799, \quad p(z_L \mid b, \text{listen}) = 0.745$$

and one contradicting z_R from there takes it straight back down. From 0.85, a single z_R returns the belief to exactly $\frac{1}{2}$ — the two growls cancel, because the likelihood ratio is the reciprocal. **That is the ladder the marker climbs in the widget above**, and the whole optimal policy is a statement about where on that ladder you may stop.

22.3.4 The belief MDP

 **DERIVATION** A POMDP is an MDP over the belief simplex

$$V^*(b) = \max_u \left[\rho(b, u) + \gamma \sum_z p(z | b, u) V^*(\tau(b, u, z)) \right]$$

Statement. Define the belief MDP $(\Delta^{|\mathcal{X}|-1}, \mathcal{U}, \tilde{p}, \rho, \gamma)$ with

$$\tilde{p}(b' | b, u) = \sum_z p(z | b, u) \mathbf{1}[b' = \tau(b, u, z)], \quad \rho(b, u) = \sum_x b(x) r(x, u).$$

Then an optimal policy of this MDP, composed with the Bayes filter, is an optimal policy of the POMDP.

Step 1 — the belief is a sufficient statistic of the history. This is Chapter 5's completeness argument, not a new one: under the Markov assumption, $\text{bel}(x_t) = p(x_t | z_{1:t}, u_{1:t})$ is all of $z_{1:t}, u_{1:t}$ that matters for predicting x_{t+1} and z_{t+1} . The filter is a *lossless* compressor of history with respect to the future.

Step 2 — write the objective in terms of histories. A policy for a POMDP is by definition a map from histories $h_t = (u_{1:t}, z_{1:t})$ to actions, and its expected return is $\mathbb{E}[\sum_t \gamma^t r(x_t, u_t)]$ with the expectation over states *and* observations. Condition the inner expectation on h_t : since r depends on x_t only through its expectation under $p(x_t | h_t)$, the t -th term is $\gamma^t \rho(\text{bel}_t, u_t)$.

Step 3 — collapse histories to beliefs. By Step 1, two histories with the same belief have the same distribution over every future quantity, so no optimal policy can benefit from distinguishing them. The policy may therefore be taken to be a function of bel_t alone.

Step 4 — read off the induced process. Given b and u , the next belief is determined by which z arrives, and z arrives with probability $p(z | b, u)$. That is a bona fide MDP over the simplex, with reward ρ . Chapter 21's Bellman equation applies verbatim, giving the result above. ■

What it costs. The state space is now a continuum — for $|\mathcal{X}|$ states, a $(|\mathcal{X}|-1)$ -dimensional simplex — so "for all x " in Chapter 21's value-iteration sweep is now "for all points of a continuum". Every technique in this chapter is a way of coping with that one sentence.

22.3.5 The value function is piecewise-linear and convex

This is the theorem that makes the chapter possible — Sondik's, published with Smallwood in 1973. Every exact POMDP algorithm since is a way of maintaining the object it describes.

DERIVATION V_T is a maximum of finitely many linear functions

$$V_T(b) = \max_{\alpha \in \Gamma_T} \langle \alpha, b \rangle, \quad |\Gamma_T| \leq |\mathcal{U}| |\Gamma_{T-1}|^{|\mathcal{Z}|}$$

Statement. For every finite horizon T there is a finite set of vectors $\Gamma_T \subset \mathbb{R}^{|\mathcal{X}|}$ with $V_T(b) = \max_{\alpha \in \Gamma_T} \langle \alpha, b \rangle$. Consequently V_T is convex and piecewise linear.

Step 1 — the base case is the immediate reward. With one step to go there is nothing to anticipate: $V_1(b) = \max_u \sum_x b(x) r(x, u) = \max_u \langle r(\cdot, u), b \rangle$. So $\Gamma_1 = \{r(\cdot, u) : u \in \mathcal{U}\}$, of size $|\mathcal{U}|$. For the tiger these are three vectors you can write down without a computer:

$$\alpha^{\text{listen}} = (-1, -1), \quad \alpha^{\text{open}_L} = (-100, 10), \quad \alpha^{\text{open}_R} = (10, -100).$$

Step 2 — substitute the inductive hypothesis into the Bellman backup.

$$V_T(b) = \max_u \left[\langle r(\cdot, u), b \rangle + \gamma \sum_z p(z | b, u) \max_k \langle \alpha^{(k)}, \tau(b, u, z) \rangle \right]$$

Step 3 — the normalizer cancels. This is the trick. Write the inner product out and notice that τ 's normalizer is precisely the factor sitting in front of it:

$$p(z | b, u) \cdot \langle \alpha^{(k)}, \tau(b, u, z) \rangle = \sum_{x'} \alpha^{(k)}(x') p(z | x', u) \sum_x p(x' | x, u) b(x)$$

A quantity that looked like *(probability) × (value of a normalized posterior)* is in fact **linear in b** . Name that linear functional's coefficient vector the **backprojection**:

$$g_k^{u,z}(x) = \sum_{x'} p(z | x', u) p(x' | x, u) \alpha^{(k)}(x')$$

so that $p(z | b, u) \langle \alpha^{(k)}, \tau(b, u, z) \rangle = \langle g_k^{u,z}, b \rangle$ exactly. Read $g_k^{u,z}$ as: *what $\alpha^{(k)}$ is worth, seen from state x , through the pair (u, z) — the value discounted by how likely that observation was.*

Step 4 — a sum of maxima is a max over tuples. Now

$$V_T(b) = \max_u \left[\langle r(\cdot, u), b \rangle + \gamma \sum_z \max_k \langle g_k^{u,z}, b \rangle \right]$$

and since the z -terms are independent, choosing the best k for each z separately is the same as choosing the best *assignment* $k(\cdot) : \mathcal{Z} \rightarrow \{1, \dots, |\Gamma_{T-1}|\}$ jointly:

$$V_T(b) = \max_u \max_{k(\cdot)} \left\langle r(\cdot, u) + \gamma \sum_z g_{k(z)}^{u,z}, b \right\rangle$$

Step 5 — read off Γ_T and count it. Each pair (action, assignment) contributes one vector, so

$$\Gamma_T = \left\{ r(\cdot, u) + \gamma \sum_z g_{k(z)}^{u,z} : u \in \mathcal{U}, k : \mathcal{Z} \rightarrow \{1, \dots, |\Gamma_{T-1}|\} \right\}, \quad |\Gamma_T| \leq |\mathcal{U}| \cdot |\Gamma_{T-1}|^{|\mathcal{Z}|}.$$

Step 6 — convexity is free. V_T is a pointwise maximum of linear functions, and a pointwise maximum of convex functions is convex. ■

The interpretive payoff. An α -vector is not "a value". It is a whole *conditional plan*: take action u now, and if you then see z , continue with the plan that $\alpha^{(k(z))}$ represents. $\langle \alpha, b \rangle$ is that plan's expected return under b , and $V(b)$ picks the best plan for the belief you are actually in. That is why the envelope's breakpoints are the policy's decision thresholds: they are where one conditional plan overtakes another.

22.3.6 The explosion, watched live

The count in Step 5 is the whole story. For the tiger, $|\Gamma_1| = 3$, and then

$$3 \longrightarrow 3 \cdot 3^2 = 27 \longrightarrow 3 \cdot 27^2 = 2187 \longrightarrow 3 \cdot 2187^2 = 14\,348\,907 \longrightarrow 6.2 \times 10^{14}$$

This is doubly exponential in the horizon, on a problem with **two states**. It is not a scaling problem to be solved with a bigger machine; it is a combinatorial one, and every practical method in this chapter exists because of this line.

Pruning rescues it, and by an enormous margin — but only because most of those vectors are never the maximum anywhere on the simplex. Turn the toggle off in the widget below and watch the counter run away while the picture stops being drawable.

INTERACTIVE · w22.3

The Alpha Forge

Exact POMDP value iteration does not scale badly — it explodes combinatorially. Approximation is structural, not laziness.

Run this simulation in the web edition: [#w22.3](/chapters/ch22-pomdps)

For the record, here is what our solver actually produces on the tiger at $\gamma = 0.95$, accuracy 0.85 — candidates generated versus vectors that survive:

horizon T	1	2	3	4	5	6	8	10	12
cross-sum candidates	3	27	75	243	147	507	1 083	2 187	4 107
kept after pruning	3	5	9	7	13	15	25	27	35

Two honest observations. The candidate counts stay small only because each row is computed from the *pruned* previous row — that is the feedback loop that keeps exact value iteration alive at all. And the kept counts are not monotone: at $T = 4$ the minimal set is *smaller* than at $T = 3$. Nothing is wrong. The minimal representation of a value function has no obligation to grow, and a solver that assumes it does will over-prune or under-prune.

NOTE

Where this sits on the complexity map. Deciding whether a finite-horizon POMDP has a policy achieving a given value is **PSPACE-complete** (Papadimitriou and Tsitsiklis, 1987) — harder than NP-complete under the usual assumptions, and — unless $P = PSPACE$ — strictly harder than the fully observable case of Chapter 21, which the same paper places in P. Approximation in this chapter is therefore *structural*, not laziness: there is no machine and no clever data structure that makes the exact problem go away.

22.3.7 Pruning, and what our pruning misses

Given a candidate set Γ , the minimal set is $\{\alpha \in \Gamma : \exists b \in \Delta, \langle \alpha, b \rangle > \langle \alpha', b \rangle \forall \alpha' \neq \alpha\}$ — the vectors that own at least one witness point. Three tests, in increasing cost:

1. **Duplicate merge.** As value iteration converges the backup keeps re-deriving the *same* linear piece through different observation branches. Without a merge, $|\Gamma|$ grows without bound while V stands still. Sort lexicographically and sweep: $O(n \log n)$.
2. **Pointwise dominance.** If $\alpha'(x) \geq \alpha(x)$ for every x , then α can be deleted. Free, exact, and it never removes a vector it should keep — but it misses vectors that are dominated only by *combinations* of others.
3. **Witness test.** α survives iff the linear program $\max_{b, \delta} \delta$ subject to $\langle \alpha - \alpha', b \rangle \geq \delta$ for all $\alpha' \neq \alpha, b \in \Delta$, has $\delta > 0$. This is exact, and it is one LP per candidate.

We implement 1 and 2 always. For **two states** we then do something better than an LP: with $b = (t, 1-t)$ every α -vector is the *line* $\langle \alpha, b \rangle = \alpha(x_R) + (\alpha(x_L) - \alpha(x_R))t$, and the minimal set is exactly the upper envelope of a set of lines — a convex hull,

computable exactly in $O(n \log n)$ with no numerical tolerance games. That is what draws the bold curve in both widgets, and it is why the thresholds quoted in this chapter are exact rather than sampled. For three or more states we fall back to a witness *search*: test the simplex corners, then a few thousand Dirichlet-sampled interior points.

⚠ WHERE ROBOTS BREAK THIS

Be clear about what the witness search misses. It can only ever keep too *few* vectors: a piece whose ownership region is smaller than the sampling density is silently dropped, and the value function loses a sliver it should have had. It never keeps too many. In exchange we avoid a dependency on an LP solver, which for the problem sizes in this book is the right trade — but a production solver should use the LP, and Thrun et al. §16.2.4 writes it out.

22.3.8 QMDP, and exactly what it cannot see

The cheapest approximation is to keep only the first backup and stop. Assume that after one action the fog lifts entirely and the state becomes observable. Then the value of doing u in belief b is just the average, over states, of Chapter 21's Q^* :

$$Q_{\text{MDP}}(b, u) = \sum_x b(x) Q^*(x, u), \quad \Gamma_{\text{QMDP}} = \{ Q^*(\cdot, u) : u \in \mathcal{U} \}$$

This is an α -vector set with exactly $|\mathcal{U}|$ members, for any horizon, and it costs one Chapter 21 solve. It is also an **upper bound**: $V_{\text{QMDP}}(b) \geq V^*(b)$, because it prices a relaxed problem in which someone tells you the state after one step, and more information cannot hurt. On the tiger the bound is spectacularly loose — $V_{\text{QMDP}}(\frac{1}{2}) = 189$ against $V^*(\frac{1}{2}) = 19.37$, a factor of ten — but it is *sound everywhere and free*, which is why it is the standard upper bound in DESPOT's and SARSOP's branch-and-bound. A bad estimator can still be a useful certificate.

Now the sharp statement of what QMDP gives up.

📦 DERIVATION QMDP is blind to the sensor

$$p(\cdot \mid x, u) = p(\cdot \mid x, u') \text{ and } r(\cdot, u) = r(\cdot, u') \implies Q_{\text{MDP}}(b, u) = Q_{\text{MDP}}(b, u') \quad \forall b$$

Statement. Let u and u' be two actions with identical transition models and identical rewards, differing *only* in their observation models. Then $Q_{\text{MDP}}(b, u) = Q_{\text{MDP}}(b, u')$ for every belief b — QMDP is exactly indifferent between them, no matter how much better one sensor is than the other.

Step 1. V^* here is the value function of the *underlying, fully observable* MDP.

Its defining equation, $V^*(x) = \max_u [r(x, u) + \gamma \sum_{x'} p(x' | x, u) V^*(x')]$, contains no term involving $p(z | x', u)$. The observation model is not an input.

Step 2. Hence $Q^*(x, u) = r(x, u) + \gamma \sum_{x'} p(x' | x, u) V^*(x')$ depends on u only through $r(\cdot, u)$ and $p(\cdot | \cdot, u)$, which by hypothesis agree for u and u' . So $Q^*(\cdot, u) = Q^*(\cdot, u')$ componentwise.

Step 3. $Q_{\text{MDP}}(b, u) = \langle b, Q^*(\cdot, u) \rangle = \langle b, Q^*(\cdot, u') \rangle = Q_{\text{MDP}}(b, u')$. ■

Corollary — the failure, stated exactly. Give the tiger agent two microphones at the same price: a sharp one (0.95 accurate) and a dull one (0.55). QMDP's α -vectors for the two actions are *bit-identical* — both (189, 189) — and it picks between them by tie-break. The exact solver picks the sharp microphone at every belief. This is the precise sense in which QMDP "never values information": it cannot distinguish actions that differ only in what they reveal. The Rust test suite below pins it as an equality assertion, which is the most honest possible way to document a limitation.

There is a subtler failure that the widget above is built to expose, and it matters more in practice. QMDP is *not* incapable of choosing the microphone in the tiger problem — listening also happens to be a cheap way of not dying, so it gets chosen for the wrong reason. What QMDP gets wrong is *when to stop*: it opens a door as soon as $b(x_L) \geq 0.9$, where the optimal policy waits for 0.9603.

Work out where 0.9 comes from, because it is a two-line calculation. Under full observability the optimal policy is "open the correct door", worth $V^* = 10 + \gamma V^*$, so $V^* = 200$ at $\gamma = 0.95$. Then $Q^*(x, \text{listen}) = -1 + 0.95(200) = 189$ for both states, while $Q^*(\cdot, \text{open}_R) = (10 + 190, -100 + 190) = (200, 90)$. Opening beats listening when $90 + 110t \geq 189$, i.e. $t \geq 0.9$ — and notice that **the number 0.9 contains no reference to the hearing accuracy at all**. Drag the accuracy slider in the widget from 0.55 to 0.99 and QMDP's threshold does not move by one digit. That is Step 1 of the derivation, made visible.

i NOTE

A cautionary note about benchmarks. At the default accuracy 0.85 the two pilots score identically, because the reachable belief ladder from $\frac{1}{2}$ is $0.85 \rightarrow 0.969799$ and steps clean over the gap between 0.9 and 0.9603: both thresholds say "listen once more, then open". A tournament run at 0.85 would certify QMDP as optimal.

Move the slider in either direction and the agreement dissolves. Over 20 000 paired rounds — same tiger placement, same growl stream, every pilot:

hearing accuracy	listens per round, optimal / QMDP	mauled, optimal / QMDP	score per round
0.70	9.2 / 6.4	3.2% / 7.2%	-2.88 / -4.32
0.80	4.8 / 2.9	1.6% / 5.5%	3.47 / 1.04
0.85	2.7 / 2.7	2.9% / 2.9%	4.19 / 4.19
0.95	2.2 / 1.0	0.3% / 4.8%	7.52 / 3.77

The last row is the one to remember. With a 95%-accurate microphone a single growl already puts the belief at 0.95, which clears QMDP's fixed 0.9 — so QMDP opens after one listen and is eaten roughly eighteen times as often as the optimal pilot, which pays one more unit for a second opinion. **A better sensor made the approximation worse.** An approximation that looks perfect on your benchmark may be one parameter away from being dangerous, in either direction.

22.3.9 PBVI: keep the value function only where the robot can get to

The exact backup maintains V everywhere on the simplex. Point-based value iteration (Pineau, Gordon and Thrun, 2003) keeps it only at a finite set B of belief points, and — this is the whole idea — keeps *exactly one* α -vector per point:

$$\alpha_b = \arg \max_{\alpha \in \text{backup}(\Gamma)} \langle \alpha, b \rangle, \quad \Gamma' = \{\alpha_b : b \in B\}$$

Crucially you never *build* the cross-sum. For a fixed b , the best assignment $k(\cdot)$ is found one observation at a time — $k(z) = \arg \max_k \langle g_k^{u,z}, b \rangle$ — because the z -terms are independent. The backup therefore costs $O(|B| |\mathcal{U}| |\mathcal{Z}| |\Gamma| |X|)$ and returns at most $|B|$ vectors. Doubly exponential becomes polynomial, and the price is that Γ' is a *lower* bound on V^* : every α_b is the value of a real conditional plan, just not necessarily the best one somewhere else on the simplex.

The error bound is $\|V^B - V^*\|_\infty \leq \frac{(r_{\max} - r_{\min}) \epsilon_B}{(1 - \gamma)^2}$, with $\epsilon_B = \max_{b' \in \Delta} \min_{b \in B} \|b - b'\|_1$ the density of B in the simplex. Be honest about it: on the tiger with $r_{\max} - r_{\min} = 110$ and $\gamma = 0.95$ the prefactor is 44 000, so with our five belief points ($\epsilon_B \approx 0.35$) the bound reads "the error is at most 15 400", while the actual error at $b = \frac{1}{2}$ is 2.62. The bound proves convergence as B densifies. It certifies nothing at any $|B|$ you would actually run.

What *does* matter is which points you pick, and that is PBVI's real contribution: expand B along **reachable** beliefs, $B \leftarrow B \cup \{\tau(b, u, z)\}$, rather than gridding the simplex. Switch the Alpha Forge to PBVI mode and take the belief count down to three. With $B = \{(\frac{1}{2}, \frac{1}{2}), (1, 0), (0, 1)\}$ the envelope collapses onto QMDP's answer — threshold exactly 0.900 — because the corners of the simplex cannot see that

listening is worth anything at 0.9. Add the reachable point 0.85 and the threshold jumps to 0.960; add its mirror 0.15 and it settles at 0.958, within 0.003 of the exact 0.9603. Five well-chosen points buy most of what the exact solver spends millions of vectors on. SARSOP (Kurniawati, Hsu and Lee, 2008) industrialized precisely this observation, by targeting beliefs reachable *under near-optimal policies*.

22.3.10 Online planning: POMCP searches histories, not states

Everything so far computes a policy for *all* beliefs, offline. The modern robotics answer is to give that up. At each timestep, plan only from the belief you are actually in, spend a fixed compute budget, act, and re-plan.

POMCP (Silver and Veness, 2010) is Monte-Carlo tree search over action–observation histories, and it fits this book unreasonably well because it needs two things you already built:

- A **generative model** G : a black box that, given (x, u) , samples (x', z, r) . That is the Chapter 4 simulator, verbatim. POMCP never sees a transition matrix, which is why it scales to state spaces you could not write one for.
- A **particle set** for the root belief. That is Chapter 8’s object, and Chapter 12’s MCL produces one every frame.

The mechanism: each simulation samples a state from the root particles, then walks down the tree picking actions by UCB1 — $\arg \max_u Q(h, u) + c\sqrt{\log N(h)/N(h, u)}$ — stepping the generative model at each node, and appending the sampled state to whichever child node the generated observation leads to. When it falls off the tree it runs a rollout policy to the horizon and backs the return up along the path.

The beautiful part is the belief representation. Nobody runs a Bayes filter inside the tree. The particles that *happen to reach* node h are, by construction, distributed according to $\tau(\dots \tau(b_0, u, z) \dots)$: the observation branching does the conditioning, and the surviving particles are the posterior. Sampling *is* the filter.

INTERACTIVE · w22.4

POMCP Tree Peek

Online planning is not exhaustive lookahead. It is sampled, bandit-guided lookahead over particle beliefs — and it needs no transition matrix at all.

Run this simulation in the web edition: `/chapters/ch22-pomdps #w22.4`

Two caveats, both inherited from Chapter 8 and both real. **Particle deprivation gets worse with depth**: a node three observations down may hold four particles, and its value estimate is noise. **Particle deprivation gets worse on re-rooting**: when you execute u and observe z , the new root is node huz , whose particles are all you have — which is why practical implementations reinvigorate with

noise, exactly as Augmented MCL does. And the UCB1 constant is not a tuning afterthought: set it too small in the widget above and both opening arms get exactly one pull, the unlucky one draws the tiger, and neither is ever tried again. The planner then listens forever on the strength of a single rollout.

22.3.11 DESPOT, in one page

POMCP’s weakness is that the tree branches on every observation that happens to be sampled, so with many observations it becomes a bush of singleton nodes. **DESPOT** (Ye et al., 2017) fixes this with one idea: fix K **scenarios** in advance.

A scenario $\phi = (x_0, \phi_1, \phi_2, \dots)$ is a start state plus a stream of random numbers. Determinize the generative model so that $(x', z, r) = G(x, u, \phi_t)$ is a *function*. Now a policy’s outcome under scenario ϕ is deterministic, and the sparse tree contains only the $\leq K$ observation branches that actually occur under the K scenarios — the branching factor becomes K instead of $|\mathcal{Z}|$, and the whole tree evaluates all K scenarios at once.

Two consequences worth remembering. First, the value estimated on K scenarios *overfits*, exactly as empirical risk does: a policy can be tailored to the sampled futures. DESPOT’s regularized objective, $\max_{\pi} [\hat{V}_{\phi}(\pi) - \lambda|\pi|]$, penalizes policy size and comes with a regret bound in terms of the *representation size* of the optimal policy — the analogue of a generalization bound, and the reason the regularization is not an optional extra. Second, it is anytime and bounded: the search is guided by an upper bound (QMDP serves) and a lower bound (any default policy), and the gap between them at the root is a certificate you can stop on. It is a genuinely small amount of code once POMCP exists — which is why it is this chapter’s stretch exercise rather than its main listing.

22.3.12 AMDP: compress the belief, then use Chapter 21 unchanged

For Rusty in the Apartment, none of the above applies directly: $\text{bel}(x)$ over $\text{SE}(2)$ is an infinite-dimensional object and $\mathcal{Z}$ is a LiDAR scan. The oldest robotics answer is also still one of the best: throw almost all of the belief away and keep two numbers.

$$\bar{b} = \left(\arg \max_x b(x), H[b] \right), \quad H[b] = - \sum_x b(x) \log_2 b(x)$$

Where I probably am, and how sure I am. Plan in that augmented space with Chapter 21’s value iteration — nothing else changes — and you get the **augmented MDP** (Roy and Thrun, 1999; Thrun et al. §16.5). For a 2-D isotropic Gaussian belief the entropy coordinate collapses to a single position standard deviation: the *differential* entropy — the continuous analogue of the sum above, which may be negative, and only its differences matter here — is

$$H[b] = \frac{1}{2} \log_2 ((2\pi e)^2 \det \Sigma) = \log_2 (2\pi e \sigma^2) \text{ bits}, \quad \Sigma = \sigma^2 I_2,$$

so "minimize entropy" and "minimize σ " are the same instruction, and σ has the advantage of being drawable as the width of a ribbon.

The transitions in the augmented space need $\sigma' = f(\sigma, x, u)$, and here Chapter 6 hands us the answer in one line of information form:

$$\frac{1}{\sigma'^2} = \frac{1}{\sigma^2 + \sigma_m^2} + v(x')$$

Prediction adds variance; correction adds information. The only genuinely new modelling object is $v(x)$, the **localizability** of a place: how much information one scan taken there delivers. We model it as $v(x) = v_{\max} \exp(-(d(x)/R)^2)$, where $d(x)$ is the distance to the nearest wall the robot can actually localize against and R is the LiDAR range. A scan is informative when there is geometry inside it — and note that *occupancy* and *information* are two different maps, because a long featureless wall constrains distance-to-wall and nothing else, which is exactly the degeneracy Chapter 16's scan matcher suffers from.

⚠ WHERE ROBOTS BREAK THIS

The compression lies, and here is where. $\bar{b}$ is only a legitimate state if it is a sufficient statistic for value — $V(b) = V(\bar{b})$ for every b the robot may encounter. The draft says it plainly: *"In practice, this assumption will rarely hold true."* The clean counterexample is the one Chapter 12 already showed you. A belief split evenly between two symmetric corridors and a belief smeared uniformly over one large room can have the *same* entropy and the *same* MAP cell, and they demand opposite actions: the first needs a disambiguating detour, the second needs patience. H cannot tell them apart. Chapter 24 builds a widget whose entire job is to exploit this.

22.4 The algorithms

ALGORITHM `finite_world_POMDP(T)` COST $|U| \cdot |\Gamma|^Z$ candidates per backup, each $O(|X|^2)$ to form; then pruning
in a finite POMDP and a horizon T
out $\Gamma_1 \dots \Gamma_T$, with pre- and post-prune counts

- 1: $\Gamma_1 = \{r(\cdot, u) : u \in \mathcal{U}\}$
- 2: for $t = 2$ to T do
- 3: for all u, z, k do $g_k^{u,z}(x) = \sum_{x'} p(z | x', u) p(x' | x, u) \alpha^{(k)}(x')$
- 4: $\Gamma_t = \emptyset$
- 5: for all $u \in \mathcal{U}$ do

```

6:   for all assignments  $k : \mathcal{Z} \rightarrow \{1, \dots, |\Gamma_{t-1}|\}$  do
7:      $\Gamma_t \leftarrow \Gamma_t \cup \{ \langle u, r(\cdot, u) + \gamma \sum_z g_{k(z)}^{u,z} \rangle \}$ 
8:    $\Gamma_t \leftarrow \text{prune}(\Gamma_t)$ 
9: return  $\Gamma_1, \dots, \Gamma_T$ 

```

This is Thrun et al.'s Table 16.1 with the pruning step promoted from a remark to line 8, and with the action tag kept on each vector so that a policy can be read off without a second pass.

ALGORITHM `prune( $\Gamma$ )` COST $O(n \log n)$ for $|\mathcal{X}| = 2$; $O(n^2|\mathcal{X}|) + \text{witness search}$ otherwise
in a candidate α -vector set
out the minimal subset with the same upper envelope

```

1:  $\Gamma \leftarrow \text{merge vectors agreeing componentwise to tolerance}$ 
2: if  $|\mathcal{X}| = 2$  then return the upper envelope of the lines  $\alpha(x_R) + (\alpha(x_L) - \alpha(x_R))t$ 
3:  $\Gamma \leftarrow \{ \alpha \in \Gamma : \nexists \alpha' \text{ with } \alpha'(x) \geq \alpha(x) \forall x \}$ 
4:  $W \leftarrow \text{the } |\mathcal{X}| \text{ simplex corners } \cup n \text{ Dirichlet samples}$ 
5: return  $\{ \arg \max_{\alpha \in \Gamma} \langle \alpha, b \rangle : b \in W \}$ 

```

ALGORITHM `QMDP( $\text{pomdp}$ )` COST one Chapter 21 value iteration
in a POMDP — its observation model is not read
out Γ_{QMDP} , an upper bound on V^*

```

1:  $V^*, Q^* \leftarrow \text{MDP\_value\_iteration on } (\mathcal{X}, \mathcal{U}, p(x' | x, u), r, \gamma)$ 
2: return  $\{ \langle u, Q^*(\cdot, u) \rangle : u \in \mathcal{U} \}$ 

```

ALGORITHM `PBVI( $\text{pomdp}, B, n$ )` COST $O(|B| \cdot |\mathcal{U}| \cdot |\mathcal{Z}| \cdot |\Gamma| \cdot |\mathcal{X}|)$ per iteration
in a POMDP, a belief set B , an iteration count
out Γ with $|\Gamma| \leq |B|$, a lower bound on V^*

```

1:  $\Gamma \leftarrow \{ r(\cdot, u) \}_u$ 
2: repeat  $n$  times
3:   compute all  $g_k^{u,z}$  from  $\Gamma$ 
4:   for all  $b \in B$  do

```

```

5:   for all  $u$  do  $\alpha_{b,u} = r(\cdot, u) + \gamma \sum_z g_{k^*(z)}^{u,z}$  with  $k^*(z) = \arg \max_k \langle g_k^{u,z}, b \rangle$ 
6:    $\alpha_b = \arg \max_u \langle \alpha_{b,u}, b \rangle$ 
7:    $\Gamma \leftarrow \{\alpha_b : b \in B\}$ , deduplicated
8: return  $\Gamma$ 

```

ALGORITHM POMCP_search(b , n_sims , c) COST $O(n_sims \cdot d_max)$ generative-model calls

in root particle set, simulation budget, exploration constant
out an action; anytime — stop whenever the budget runs out

```

1: repeat  $n\_sims$  times: sample  $x \sim b$ ; simulate( $x, h_{root}, 0$ )
2: return  $\arg \max_u N(h_{root}, u)$  (most-visited, not highest-valued)
3: simulate( $x, h, d$ ):
4:   if  $d \geq d_{max}$  then return 0
5:   if  $h$  is a leaf then expand  $h$ ;  $N(h) += 1$ ; return rollout( $x, d$ )
6:    $u = \arg \max_u [Q(h, u) + c \sqrt{\log N(h)/N(h, u)}]$ 
7:    $(x', z, r) \sim G(x, u)$ ; append  $x'$  to the particle set of  $huz$ 
8:    $R = r + \gamma \text{simulate}(x', huz, d + 1)$ 
9:    $N(h) += 1$ ;  $N(h, u) += 1$ ;  $Q(h, u) += \frac{R - Q(h, u)}{N(h, u)}$ 
10:  return  $R$ 

```

ALGORITHM Augmented_MDP_value_iteration(sim , $grid$) COST Chapter 21 value iteration over $|cells| \cdot |bins|$ states

in a simulator, a (cell, σ -bin) grid
out V and π over the augmented space

```

1: for all  $(x, \sigma)$  and  $u$ : estimate  $p(x' | x, u)$  from the motion model
2:    $\sigma' = [(\sigma^2 + \sigma_m^2)^{-1} + v(x')]^{-1/2}$ , binned
3:    $r((x, \sigma), u) = -\text{step cost} - w \Pr[\text{collision} | \sigma'] + \text{goal terms}$ 
4: run MDP_value_iteration on the resulting finite MDP
5: return  $V, \pi$ 

```

Line 3 is the only place uncertainty enters the reward, and it is worth pausing on: the collision term is $\Pr[\|e\| > d]$ for a 2-D isotropic Gaussian error, which is a Rayleigh tail, $\exp(-d^2/2\sigma^2)$, with d the clearance to the nearest wall. A robot that is uncertain is charged for being uncertain *in proportion to how tight the space is* — which is why the emergent behavior is not "always hug walls" but "hug walls

where it pays".

22.5 A worked example you can check by hand

Everything in this section is doable on paper in about fifteen minutes, and every number is pinned by a test in the next section.

22.5.1 Γ_1 and Γ_2 for the tiger

Γ_1 is the immediate-reward set, written above: $(-1, -1)$, $(-100, 10)$, $(10, -100)$.

For Γ_2 we need the backprojections. Under **listen** the transition is the identity, so $g_k^{\text{listen},z}(x) = p(z | x) \alpha^{(k)}(x)$ — no sum survives. Take the assignment "if I hear left, follow α^{open_R} ; if I hear right, follow α^{listen} ".

$$\alpha'(x_L) = -1 + 0.95[0.85 \times 10 + 0.15 \times (-1)] = -1 + 0.95(8.35) = 6.9325$$

$$\alpha'(x_R) = -1 + 0.95[0.15 \times (-100) + 0.85 \times (-1)] = -1 + 0.95(-15.85) = -16.0575$$

Under **open_R** the transition resets to $(\frac{1}{2}, \frac{1}{2})$ and the growl carries nothing, so every backprojection is $\frac{1}{4}(\alpha^{(k)}(x_L) + \alpha^{(k)}(x_R))$ regardless of z . With both branches following $\alpha^{\text{listen}} = (-1, -1)$, each backprojection is $\frac{1}{4}(-2) = -\frac{1}{2}$, and there are two observations to sum over:

$$\alpha'(x_L) = 10 + 0.95[-\frac{1}{2} - \frac{1}{2}] = 9.05, \quad \alpha'(x_R) = -100 + 0.95[-\frac{1}{2} - \frac{1}{2}] = -100.95$$

Note what just happened: because opening resets the world, *every* assignment $k(\cdot)$ with the same total $\sum_z (\alpha^{(k(z))}(x_L) + \alpha^{(k(z))}(x_R))$ gives the same vector. Nine assignments collapse to three distinct candidates before pruning even starts.

Grind through all $3 \cdot 3^2 = 27$ assignments and exactly **five** vectors survive pruning:

α	action	the plan it encodes
$(-100.95, 9.05)$	open_L	open the left door now
$(-16.0575, 6.9325)$	listen	listen; open left on z_R , listen again on z_L
$(-1.95, -1.95)$	listen	listen twice, commit to nothing
$(6.9325, -16.0575)$	listen	listen; open right on z_L , listen again on z_R
$(9.05, -100.95)$	open_R	open the right door now

Their upper envelope has four breakpoints, and each is a two-line calculation. Where does "open right now" overtake "listen, then open right"?

$$110t - 100.95 = 22.99t - 16.0575 \implies t = \frac{84.8925}{87.01} = 0.975664$$

and where does the do-nothing plan give way to it?

$$-1.95 = 22.99t - 16.0575 \implies t = \frac{8.8825}{22.99} = 0.613636$$

So the two-step policy is: open right above 0.975664, listen below it, and by symmetry open left below 0.024336.

22.5.2 The converged answer, and a chain that closes

Run the backup to convergence and the tiger's minimal set has $|\Gamma^*| = 9$ vectors and exactly three policy regions:

$$\pi^*(b) = \begin{cases} \text{open}_L & b(x_L) \leq 0.039654 \\ \text{listen} & 0.039654 < b(x_L) < 0.960346 \\ \text{open}_R & b(x_L) \geq 0.960346 \end{cases}$$

with values $V^*(\frac{1}{2}) = 19.3714$, $V^*(0.85) = 21.4435$, $V^*(0.969799) = 25.0807$ and $V^*(1) = 28.4028$. Those four numbers are not independent, and checking that they close is the best possible test that the solver is right:

$$\begin{aligned} V^*(1) &= 10 + \gamma V^*(\tfrac{1}{2}) = 10 + 0.95(19.3714) = 28.4028 \quad \checkmark \\ V^*(\tfrac{1}{2}) &= -1 + \gamma V^*(0.85) = -1 + 0.95(21.4435) = 19.3713 \quad \checkmark \\ V^*(0.85) &= -1 + \gamma [0.745 V^*(0.969799) + 0.255 V^*(\tfrac{1}{2})] \\ &= -1 + 0.95 [0.745(25.0807) + 0.255(19.3714)] = 21.4436 \quad \checkmark \end{aligned}$$

(The residual digit in the last two lines is rounding, not error: run the solver at full precision and the three identities close to six decimals. The book's test asserts them to 10^{-3} . If you read 0.9611 rather than 0.9603 in the widget above, that is not a discrepancy either — it solves to horizon 20 so that the accuracy slider stays interactive, and the last 0.0008 of the threshold is carried by the discounted tail beyond step twenty.)

Read the second and third lines as sentences. *From complete ignorance, the optimal thing to do is pay one unit, and you will find yourself at 0.85. From 0.85, pay one more unit; with probability 0.745 the growl agrees and you land at 0.969799, which is past the threshold, so you open; with probability 0.255 it disagrees and you are back where you started. Listen, listen, open — and the number 0.960346 is the entire reason it is two listens and not one or three.*

22.6 Implementation in Rust

The crate is `crates/ch22_pomdp`. It depends on `ch21_mdp` (for QMDP's inner solve and for the augmented-MDP lab), on `ch08_particles` (for POMCP's root beliefs), and on nothing else that is not in the book's stack.

22.6.1 The model: dimensions as types

RUST `crates/ch22_pomdp/src/model.rs`

```
1 use nalgebra::{SMatrix, SVector};
2
3 /// A finite POMDP with `S` states, `A` actions and `Z` observations.
4 ///
5 /// The dimensions are const generics because they are genuinely fixed by the
6 /// problem, and because it makes the compiler check the one thing that is easy
7 /// to get wrong: whether a matrix is indexed [x][x'] or [x'][x].
8 pub struct FinitePomdp<const S: usize, const A: usize, const Z: usize> {
9     /// t[u] is row-stochastic: row x, column x' holds  $p(x' \mid x, u)$ .
10    pub t: [SMatrix<f64, S, S>; A],
11    /// o[u] is row-stochastic: row x, column z holds  $p(z \mid x', u)$ .
12    /// Note the *next* state: the observation follows the transition.
13    pub o: [SMatrix<f64, S, Z>; A],
14    ///  $r(x, u)$ , with Chapter 21's convention -- collected on taking  $u$  in  $x$ .
15    pub r: SMatrix<f64, S, A>,
16    pub gamma: f64,
17 }
18
19 pub type Belief<const S: usize> = SVector<f64, S>;
20
21 impl<const S: usize, const A: usize, const Z: usize> FinitePomdp<S, A, Z> {
22     ///  $b(x') = \text{Sigma}_x p(x' \mid x, u) b(x)$ . Prediction: the step that loses information.
23     ///
24     /// The transpose is the whole content of "rows are x", columns are x'".
25     pub fn predict(&self, b: &Belief<S>, u: usize) -> Belief<S> {
26         self.t[u].transpose() * b
27     }
28
29     ///  $\tau(b, u, z)$ , together with the evidence  $p(z \mid b, u)$ .
30     ///
31     /// The evidence is returned rather than discarded because in a POMDP it is
32     /// not a diagnostic -- it is the branching probability of the belief MDP,
33     /// and the PWLC derivation turns on it cancelling against this normalizer.
34     pub fn update(&self, b: &Belief<S>, u: usize, z: usize) -> (Belief<S>, f64) {
35         let bar = self.predict(b, u);
36         let mut un = bar.component_mul(&self.o[u].column(z).into_owned());
37         let evidence = un.sum();
38         if evidence > 0.0 {
39             un /= evidence;
40         }
41         (un, evidence)
42     }
43
44     ///  $\rho(b, u) = \text{Sigma}_x b(x) r(x, u)$ . Linear in  $b$  -- the load-bearing fact.
45     pub fn reward(&self, b: &Belief<S>, u: usize) -> f64 {
46         self.r.column(u).dot(b)
47     }
48 }
```

```

49     /// The underlying MDP, with the observation model **dropped on the floor**.
50     ///
51     /// This method is the QMDP indifference theorem, expressed as code: nothing
52     /// downstream of here can possibly depend on how good the sensor is.
53     pub fn underlying_mdp(&self) -> ch21_mdp::Mdp<A> { /* ... */ }
54 }
55
56 pub const LISTEN: usize = 0;
57 pub const OPEN_LEFT: usize = 1;
58 pub const OPEN_RIGHT: usize = 2;
59
60 /// Kaelbling, Littman & Cassandra (1998), with this book's discount.
61 pub fn tiger(accuracy: f64, gamma: f64) -> FinitePomdp<2, 3, 2> {
62     let a = accuracy;
63     let stay = SMatrix::<f64, 2, 2>::identity();
64     // Opening a door ends the round: the tiger is re-placed uniformly and the
65     // growl that follows carries no information at all.
66     let reset = SMatrix::<f64, 2, 2>::repeat(0.5);
67     FinitePomdp {
68         t: [stay, reset, reset],
69         o: [SMatrix::<f64, 2, 2>::new(a, 1.0 - a, 1.0 - a, a), reset, reset],
70         r: SMatrix::<f64, 2, 3>::new(
71             -1.0, -100.0, 10.0, // tiger-left: listen, open-left, open-right
72             -1.0, 10.0, -100.0, // tiger-right: ...
73         ),
74         gamma,
75     }
76 }

```

22.6.2 The exact backup, and the pruning that saves it

RUST crates/ch22_pomdp/src/exact.rs

```

1  use nalgebra::SVector;
2  use crate::model::{Belief, FinitePomdp};
3
4  /// One linear piece of V, tagged with the action it commits to at step one.
5  ///
6  /// The tag is not decoration. Without it you would have to re-derive the
7  /// policy from the value function, which for a PWLC function means recomputing
8  /// the argmax of the backup -- the expensive half of value iteration.
9  #[derive(Clone, Debug)]
10 pub struct AlphaVec<const S: usize> {
11     pub v: SVector<f64, S>,
12     pub action: usize,
13 }
14
15 impl<const S: usize> AlphaVec<S> {
16     #[inline]
17     pub fn value_at(&self, b: &Belief<S>) -> f64 {
18         self.v.dot(b)
19     }
20 }
21
22 ///  $g^{u,z}_k(x) = \text{Sigma}_{x'} p(z \mid x', u) p(x' \mid x, u) \alpha^k(x')$ , indexed  $[u][z][k]$ .
23 ///
24 /// In matrix form this is one scaled matrix-vector product: componentwise-scale

```

```

25 // alpha by the observation column, then push it back through the transition. That
26 // is the whole "backprojection" -- value seen from x, through (u, z).
27 pub(crate) fn backprojections<const S: usize, const A: usize, const Z: usize>(
28     m: &FinitePomdp<S, A, Z>,
29     set: &[AlphaVec<S>],
30 ) -> Vec<Vec<Vec<SVector<f64, S>>>> {
31     (0..A)
32         .map(|u| {
33             (0..Z)
34                 .map(|z| {
35                     let col = m.o[u].column(z).into_owned();
36                     set.iter().map(|a| m.t[u] * a.v.component_mul(&col)).collect()
37                 })
38                 .collect()
39             })
40         .collect()
41 }
42
43 // The exact backup: one alpha-vector per (action, assignment of one surviving
44 // vector to each observation). Returns |U|.Gamma^|Z| candidates -- the
45 // combinatorial explosion, un-hidden.
46 pub fn backup<const S: usize, const A: usize, const Z: usize>(
47     m: &FinitePomdp<S, A, Z>,
48     set: &[AlphaVec<S>],
49 ) -> Vec<AlphaVec<S>> {
50     let g = backprojections(m, set);
51     let k = set.len();
52     let mut out = Vec::with_capacity(A * k.pow(Z as u32));
53
54     for u in 0..A {
55         let mut pick = [0usize; Z];
56         loop {
57             let mut v = m.r.column(u).into_owned();
58             for z in 0..Z {
59                 v += m.gamma * g[u][z][pick[z]];
60             }
61             out.push(AlphaVec { v, action: u });
62
63             // Odometer over Gamma^{|Z|}: one alpha index per observation.
64             let mut z = Z;
65             let carried = loop {
66                 if z == 0 {
67                     break true;
68                 }
69                 z -= 1;
70                 pick[z] += 1;
71                 if pick[z] < k {
72                     break false;
73                 }
74                 pick[z] = 0;
75             };
76             if carried {
77                 break;
78             }
79         }
80     }
81     out
82 }
83
84 // Merge vectors that agree componentwise to `tol`.
85 //

```

```

86  /// Not cosmetic. As value iteration converges the backup keeps re-deriving the
87  /// *same* linear piece through different observation branches, and without this
88  /// merge |Gamma| grows without bound while V stands still.
89  pub(crate) fn dedupe<const S: usize>(set: &[AlphaVec<S>], tol: f64) -> Vec<AlphaVec<S>> {
90      /* ... */
91
92  /// Prune to the minimal set.
93  ///
94  /// For two states this is exact and cheap: every alpha is a *line* over
95  /// b = (t, 1-t), so the minimal set is the upper envelope of a set of lines,
96  /// which a monotone-chain hull gives in O(n log n) with no tolerance games --
97  /// and the breakpoints it returns are the policy's decision thresholds.
98  pub fn prune<const S: usize>(set: &[AlphaVec<S>]) -> Vec<AlphaVec<S>> {
99      let merged = dedupe(set, 1e-6);
100     if S == 2 {
101         return upper_envelope(&merged).into_iter().map(|(i, _)| merged[i].clone()).
102         collect();
103     }
104     // Otherwise: free pointwise dominance, then a witness *search*. This can
105     // only ever keep too few vectors, never too many -- see the chapter text.
106     let survivors = drop_dominated(&merged, 1e-9);
107     witness_survivors(&survivors, 4096, 0xC0FFEE)
108 }
109
110 /// V(b) = max_k ?alpha^(k), b?, together with the action the winning piece commands.
111 pub fn value_at<const S: usize>(set: &[AlphaVec<S>], b: &Belief<S>) -> (f64, usize) { /*
112     ... */ }
113
114 /// The upper envelope's decision thresholds. Only meaningful for |X| = 2, where
115 /// the envelope is a chain of line segments and its breakpoints *are* the policy.
116 pub struct Breakpoints {
117     pub open_left: f64,
118     pub open_right: f64,
119 }
120
121 pub fn policy_breakpoints<const S: usize>(set: &[AlphaVec<S>]) -> Breakpoints { /* ... */
122     }
123
124 /// Back up until a dense sweep of the simplex stops moving. The Bellman backup
125 /// is a gamma-contraction (Chapter 21's proof, unchanged), so this terminates.
126 pub fn solve_to_convergence<const S: usize, const A: usize, const Z: usize>(
127     m: &FinitePomdp<S, A, Z>,
128     tol: f64,
129 ) -> Vec<AlphaVec<S>> { /* ... */ }
130
131 pub struct Stage<const S: usize> {
132     pub horizon: usize,
133     /// |U|. |Gamma_{t-1}|^{ |Z|} -- the analytic count, reported even when we refuse
134     /// to materialize it. The widget's runaway counter is this field.
135     pub raw: u128,
136     pub gamma: Vec<AlphaVec<S>>,
137 }
138
139 /// Thrun et al., Table 16.1 -- `finite_world_POMDP`.
140 pub fn finite_world_pomdp<const S: usize, const A: usize, const Z: usize>(
141     m: &FinitePomdp<S, A, Z>,
142     horizon: usize,
143 ) -> Vec<Stage<S>> {
144     let mut set: Vec<AlphaVec<S>> =
145         (0..A).map(|u| AlphaVec { v: m.r.column(u).into_owned(), action: u }).collect();
146     set = prune(&set);

```

```

143
144     let mut stages = vec![Stage { horizon: 1, raw: A as u128, gamma: set.clone() }];
145     for t in 2..=horizon {
146         let raw = (A as u128) * (set.len() as u128).pow(Z as u32);
147         set = prune(&backup(m, &set));
148         stages.push(Stage { horizon: t, raw, gamma: set.clone() });
149     }
150     stages
151 }

```

22.6.3 QMDP and PBVI: the upper bound and the lower one

RUST crates/ch22_pomdp/src/qmdp.rs

```

1 use nalgebra::SVector;
2 use crate::exact::AlphaVec;
3 use crate::model::FinitePomdp;
4
5 /// Gamma_QMDP = { Q*(., u) }_u -- one vector per action, for every horizon.
6 ///
7 /// Note what is not passed to `value_iteration`: `m.o`. That omission is the
8 /// theorem. An action's QMDP value cannot depend on what it reveals, because
9 /// the function that computes it never receives the observation model.
10 pub fn qmdp<const S: usize, const A: usize, const Z: usize>(
11     m: &FinitePomdp<S, A, Z>,
12 ) -> Vec<AlphaVec<S>> {
13     let mdp = m.underlying_mdp();
14     let sol = ch21_mdp::value_iteration(&mdp, 1e-10, 100_000);
15     (0..A)
16         .map(|u| AlphaVec {
17             v: SVector::from_fn(|x, _| mdp.q(sol.v.as_slice(), x, u)),
18             action: u,
19         })
20         .collect()
21 }

```

RUST crates/ch22_pomdp/src/pbvi.rs

```

1 use crate::exact::{backprojections, dedupe, AlphaVec};
2 use crate::model::{Belief, FinitePomdp};
3
4 /// Pineau, Gordon & Thrun (2003): back up at `B` only, one vector per point.
5 ///
6 /// The cross-sum is never built. For a fixed b the best assignment factorizes
7 /// over observations, so the inner argmax is |Z| independent scans instead of
8 /// one search over |Gamma|^|Z| tuples -- which is the entire complexity win.
9 pub fn point_backup<const S: usize, const A: usize, const Z: usize>(
10     m: &FinitePomdp<S, A, Z>,
11     set: &[AlphaVec<S>],
12     beliefs: &[Belief<S>],
13 ) -> Vec<AlphaVec<S>> {
14     let g = backprojections(m, set);
15     let mut out = Vec::with_capacity(beliefs.len());

```

```

16     for b in beliefs {
17         let mut best: Option<AlphaVec<S>> = None;
18         for u in 0..A {
19             let mut v = m.r.column(u).into_owned();
20             for z in 0..Z {
21                 let k = (0..set.len())
22                     .max_by(|&i, &j| g[u][z][i].dot(b).total_cmp(&g[u][z][j].dot(b)))
23                     .unwrap_or(0);
24                 v += m.gamma * g[u][z][k];
25             }
26             if best.as_ref().is_none_or(|c| v.dot(b) > c.v.dot(b)) {
27                 best = Some(AlphaVec { v, action: u });
28             }
29         }
30     }
31     out.extend(best);
32 }
33 dedupe(&out, 1e-9)
34 }
35
36 /// PBVI's belief-set expansion: B <- B ? { tau(b,u,z) }.
37 ///
38 /// Reachability is the point. Gridding the simplex wastes points on beliefs no
39 /// filter will ever produce; this walks the same tree the robot will walk.
40 pub fn expand<const S: usize, const A: usize, const Z: usize>(
41     m: &FinitePomdp<S, A, Z>,
42     beliefs: &[Belief<S>],
43     tol: f64,
44 ) -> Vec<Belief<S>> {
45     let mut out = beliefs.to_vec();
46     for b in beliefs {
47         for u in 0..A {
48             for z in 0..Z {
49                 let (bp, evidence) = m.update(b, u, z);
50                 if evidence > 1e-9 && !out.iter().any(|q| (q - bp).amax() < tol) {
51                     out.push(bp);
52                 }
53             }
54         }
55     }
56     out
57 }

```

22.6.4 POMCP: planning with the simulator you already have

RUST crates/ch22_pomdp/src/pomcp.rs

```

1 use rand::rngs::SmallRng;
2 use rand::Rng;
3
4 /// The black box POMCP plans with. For the labs in this chapter it is
5 /// literally the Chapter 4 simulator: no transition matrix exists anywhere.
6 pub trait Generative {
7     type State: Clone;
8     const N_ACTIONS: usize;
9
10     /// (x, u) ? (x', z, r). Observations are interned to `u32` so that a laser

```



```

11 // scan can be hashed into a discrete branch without changing this trait.
12 fn step(&self, x: &Self::State, u: usize, rng: &mut SmallRng) -> (Self::State, u32,
13 f64);
14
15 fn terminal(&self, _x: &Self::State) -> bool {
16     false
17 }
18
19 // The default policy a leaf is evaluated with, and the thing to tune first
20 // when the budget is small. On the tiger, "listen twice then guess" is
21 // worth 23 points of root value over uniform-random at 16 simulations, 2
22 // points at 256, and nothing at all by 1024: a good rollout buys you the
23 // early part of the anytime curve, which is the part a control loop gets.
24 fn rollout(&self, _x: &Self::State, _depth: usize, rng: &mut SmallRng) -> usize {
25     rng.random_range(0..Self::N_ACTIONS)
26 }
27
28 // A history node. `particles` is the belief -- Chapter 8's object, arriving by
29 // natural selection rather than by a filter update.
30 struct BeliefNode<X> {
31     n: u32,
32     particles: Vec<X>,
33     arms: Vec<u32>,
34     obs: Option<u32>,
35 }
36
37 // A bandit arm:  $N(h, u)$  and the running mean  $Q(h, u)$ .
38 struct ActionNode {
39     action: usize,
40     n: u32,
41     q: f64,
42     children: Vec<(u32, u32)>, // (observation, belief-node id)
43 }
44
45 pub struct Pomcp<G: Generative> {
46     model: G,
47     beliefs: Vec<BeliefNode<G::State>>,
48     actions: Vec<ActionNode>,
49     root: usize,
50     // UCB1 exploration weight. Scale it to the *spread of returns*, not to 1:
51     // on the tiger the returns span 110, and  $c = 1$  explores nothing.
52     pub c: f64,
53     pub max_depth: usize,
54     pub gamma: f64,
55     pub simulations: u64,
56 }
57
58 impl<G: Generative> Pomcp<G> {
59     // Run `n` more simulations. Anytime: stop whenever the control loop says so.
60     pub fn search(&mut self, n: u64, rng: &mut SmallRng) {
61         for _ in 0..n {
62             let i = rng.random_range(0..self.beliefs[self.root].particles.len());
63             let x = self.beliefs[self.root].particles[i].clone();
64             self.simulate(x, self.root, 0, rng);
65             self.simulations += 1;
66         }
67     }
68
69     fn simulate(&mut self, x: G::State, node: usize, depth: usize, rng: &mut SmallRng) ->
70     f64 {

```

```

76     if depth >= self.max_depth {
77         return 0.0;
78     }
79     if self.beliefs[node].arms.is_empty() {
80         self.expand(node);
81         self.beliefs[node].n += 1;
82         // A leaf's value is a rollout, not zero. Returning zero here is the
83         // single most common way to make MCTS pathologically pessimistic.
84         return self.rollout_value(x, depth, rng);
85     }
86
87     let arm = self.select_ucbl(node);
88     let u = self.actions[arm].action;
89     let (xp, z, r) = self.model.step(&x, u, rng);
90     let child = self.child_for(arm, z);
91     // The child's belief is *built* from the particles that reach it: an
92     // unweighted particle filter, running inside the search tree.
93     self.beliefs[child].particles.push(xp.clone());
94
95     let future = if self.model.terminal(&xp) {
96         0.0
97     } else {
98         self.simulate(xp, child, depth + 1, rng)
99     };
100    let ret = r + self.gamma * future;
101
102    self.beliefs[node].n += 1;
103    let a = &mut self.actions[arm];
104    a.n += 1;
105    a.q += (ret - a.q) / f64::from(a.n);
106    ret
107 }
108
109 /// UCB1: exploit Q, but keep an eye on arms you have barely tried.
110 fn select_ucbl(&self, node: usize) -> usize {
111     let n = f64::from(self.beliefs[node].n + 1);
112     let mut best = (f64::NEG_INFINITY, self.beliefs[node].arms[0] as usize);
113     for &arm in &self.beliefs[node].arms {
114         let a = &self.actions[arm as usize];
115         if a.n == 0 {
116             return arm as usize; // every arm gets one free pull
117         }
118         let score = a.q + self.c * (n.ln() / f64::from(a.n)).sqrt();
119         if score > best.0 {
120             best = (score, arm as usize);
121         }
122     }
123     best.1
124 }
125
126 /// Pure bookkeeping: create one action node per arm, find-or-create the
127 /// belief node for an observation, and roll a default policy to the horizon.
128 fn expand(&mut self, node: usize) { /* ... */ }
129 fn child_for(&mut self, arm: usize, z: u32) -> usize { /* ... */ }
130 fn rollout_value(&self, x: G::State, depth: usize, rng: &mut SmallRng) -> f64 { /* ... */ }
131
132 /// The recommendation: the *most visited* arm, not the highest-valued one.
133 /// A high Q on two pulls is a rumour; a high visit count is a decision.
134 pub fn best_action(&self) -> usize { /* ... */ }

```

```

130     /// Execute u, observe z: re-root at `huz` and keep its particles as the new
131     /// belief. Reinvalidate them -- depth-three nodes are exactly where Chapter
132     /// 8's particle deprivation reappears.
133     pub fn advance(&mut self, u: usize, z: u32, rng: &mut SmallRng) { /* ... */ }
134 }

```

22.6.5 The worked example, as tests

RUST crates/ch22_pomdp/tests/tiger.rs

```

1 use approx::assert_relative_eq;
2 use ch22_pomdp::{exact::*, model::*, qmdp::qmdp};
3 use nalgebra::SVector;
4
5 /// A tiger with two microphones at the same price, listed sharp-first. The
6 /// dynamics and the rewards of actions 0 and 1 are identical by construction;
7 /// only `o[0]` and `o[1]` differ.
8 const SHARP: usize = 0;
9 const DULL: usize = 1;
10
11 /// Complete ignorance: the belief every round of the tiger starts from.
12 fn half() -> Belief<2> {
13     SVector::new(0.5, 0.5)
14 }
15
16 #[test]
17 fn belief_ladder_matches_the_text() {
18     let m = tiger(0.85, 0.95);
19     let (b1, ev1) = m.update(&half(), LISTEN, 0);
20     assert_relative_eq!(b1[0], 0.85, epsilon = 1e-12);
21     assert_relative_eq!(ev1, 0.5, epsilon = 1e-12); // a first growl is a coin flip
22
23     let (b2, ev2) = m.update(&b1, LISTEN, 0);
24     assert_relative_eq!(b2[0], 0.7225 / 0.745, epsilon = 1e-12); // 0.969799...
25     assert_relative_eq!(ev2, 0.745, epsilon = 1e-12);
26
27     // A contradicting growl undoes exactly one consistent one.
28     let (b3, _) = m.update(&b1, LISTEN, 1);
29     assert_relative_eq!(b3[0], 0.5, epsilon = 1e-12);
30 }
31
32 #[test]
33 fn gamma_two_has_five_survivors_and_the_right_threshold() {
34     let m = tiger(0.85, 0.95);
35     let stages = finite_world_pomdp(&m, 2);
36     assert_eq!(stages[1].raw, 27); // |U| . |Gamma_1|^|Z| = 3 . 32
37     assert_eq!(stages[1].gamma.len(), 5);
38
39     let listen_then_open = stages[1]
40         .gamma
41         .iter()
42         .find(|a| a.action == LISTEN && a.v[0] > 0.0)
43         .expect("the plan that listens, then opens right on z=L");
44     assert_relative_eq!(listen_then_open.v[0], 6.9325, epsilon = 1e-9);
45     assert_relative_eq!(listen_then_open.v[1], -16.0575, epsilon = 1e-9);
46
47     let thresholds = policy_breakpoints(&stages[1].gamma);

```

```

48     assert_relative_eq!(thresholds.open_right, 84.8925 / 87.01, epsilon = 1e-9);
49 }
50
51 #[test]
52 fn converged_policy_and_the_value_chain() {
53     let m = tiger(0.85, 0.95);
54     let g = solve_to_convergence(&m, 1e-9);
55     assert_eq!(g.len(), 9);
56
57     let t = policy_breakpoints(&g);
58     assert_relative_eq!(t.open_right, 0.960346, epsilon = 1e-4);
59     assert_relative_eq!(t.open_left, 1.0 - t.open_right, epsilon = 1e-9); // symmetry
60
61     let v = |p: f64| value_at(&g, &SVector::new(p, 1.0 - p)).0;
62     // The three identities from the text -- the chain has to close.
63     assert_relative_eq!(v(1.0), 10.0 + 0.95 * v(0.5), epsilon = 1e-3);
64     assert_relative_eq!(v(0.5), -1.0 + 0.95 * v(0.85), epsilon = 1e-3);
65     assert_relative_eq!(
66         v(0.85),
67         -1.0 + 0.95 * (0.745 * v(0.969799) + 0.255 * v(0.5)),
68         epsilon = 1e-3
69     );
70 }
71
72 #[test]
73 fn qmdp_cannot_tell_a_good_microphone_from_a_bad_one() {
74     // Two listening actions: identical cost, identical (identity) dynamics,
75     // wildly different sensors. The exact solver always picks the sharp one.
76     let m = tiger_with_two_microphones(0.95, 0.55);
77     let g = qmdp(&m);
78     assert_relative_eq!(g[SHARP].v, g[DULL].v, epsilon = 1e-12); // the theorem
79
80     let exact = solve_to_convergence(&m, 1e-9);
81     assert_eq!(value_at(&exact, &half()).1, SHARP);
82 }
83
84 #[test]
85 fn qmdp_threshold_ignores_the_sensor_entirely() {
86     for accuracy in [0.55, 0.7, 0.85, 0.99] {
87         let t = policy_breakpoints(&qmdp(&tiger(accuracy, 0.95)));
88         assert_relative_eq!(t.open_right, 0.9, epsilon = 1e-9);
89     }
90 }

```

NOTE

The widgets in this chapter run the TypeScript twin of every listing above — `lib/pomdp/finite.ts`, `lib/pomdp/pomcp.ts`, `lib/pomdp/amdp.ts`. The thresholds printed on screen, the vector counts, and the tournament returns all come out of those functions, so if you redo the algebra by hand you get the number on the screen.

22.7 Putting it together: why robots hug walls

The tiger is a two-state cartoon. Rusty's belief lives over $SE(2)$, and its observation is a LiDAR scan — a POMDP with a continuum of states and effectively a continuum of observations. Exact solving is not merely intractable here; the objects do not exist.

So we do the compression. The state becomes (cell, σ -bin): 610 free cells $\times$ 12 uncertainty bins plus one absorbing terminal, 7321 states, solved by Chapter 21's value iteration in about forty Gauss–Seidel sweeps. Two policies come out of the same code. One is told to pretend σ is always at its floor — the **certainty-equivalent** planner, which is what "plan a path on the MAP estimate, then track it" amounts to. The other is given the honest σ dynamics.

INTERACTIVE · w22.2

The Coastal Navigator

The optimal route under uncertainty is not the shortest collision-free one. Wall-hugging is not a heuristic — it is what the Bellman equation returns when σ is part of the state.

Run this simulation in the web edition: [/chapters/ch22-pomdps #w22.2](/chapters/ch22-pomdps/#w22.2)

Nothing in the map says "prefer walls". The reward function contains a step cost, a collision hazard, and a doorway bonus; the word "wall" appears only inside $v(x)$, as the distance to something a scan can lock onto. Yet the purple pilot leaves the straight line, spends about 1.6 extra steps in the neighbourhood of the textured south wall, pinches its uncertainty ribbon back down, and only then turns for the door — arriving through the gap 65% of the time against the straight pilot's 27% over 400 seeded runs.

Then push the LiDAR range slider past 5 m. The information field floods the room, both pilots reach 96%, and the coastal behavior evaporates. This is not a disappointment; it is the honest boundary of the phenomenon, and it reproduces Figure 16.6 of the Thrun draft, where entropy at the goal is plotted against sensor range and the two curves meet. **Coastal navigation is a statement about when sensing is informative, not a law of robotics.** A robot with GPS should drive straight.

WHAT ACTUALLY GENERALIZES

Three things transfer from this chapter to every belief-space problem you will meet, and none of them requires an exact solver.

1. **Put the uncertainty in the state.** The moment σ (or H , or a covariance trace) is a coordinate the planner optimizes over, information-gathering

behavior stops being a heuristic you bolt on and starts being a consequence.

2. **Price the failure, not the uncertainty.** The AMDP does not penalize σ directly. It penalizes $\Pr[\text{collision} \mid \sigma]$ and $\Pr[\text{miss the door} \mid \sigma]$, which is why the detour appears in a tight corridor and disappears in an open field.
3. **Budget, then re-plan.** Online solvers beat offline ones in robotics not because the trees are better but because a fixed compute budget spent on the belief you are actually in is worth more than an optimal policy for beliefs you will never hold.

22.7.1 What this chapter is not

Continuous observations. POMCP branches on observations, so a continuous observation space gives every node exactly one child and the tree degenerates into a set of independent rollouts. The fixes — progressive widening, observation clustering, adaptive discretization — are a live research area; Hoerger et al. (2024) is a good entry point, and Lauri et al. (2023) surveys the landscape.

Belief-space trajectory optimization. For unimodal Gaussian beliefs you can skip trees entirely and run a continuous optimizer over a trajectory of (μ, Σ) pairs, with the covariance propagated by an EKF inside the cost. It is fast and it is what most manipulation systems do; it also cannot represent the tiger, because a Gaussian cannot be bimodal. Chapter 23 takes the continuous-control ground with sampling instead.

Learning the policy. Everything here assumes a model. Deep RL over histories learns the same object without one, and pays in sample complexity and in the loss of every guarantee in this chapter. Chapter 25 makes the case for keeping learning *inside* the Bayesian frame rather than replacing it.

Where this goes next: Chapter 23 is a receding-horizon POMDP solver in disguise — sampled futures, reweighted, re-planned every frame. Chapter 24 formulates active SLAM as a POMDP and then approximates it exactly the way this chapter licenses, with the AMDP’s failure mode as the thing to watch. And Chapter 26 wires the decision layer to everything else.

22.8 Exercises

1 FOUNDATION •• Compute Γ_2 by hand, all 27 of it

Enumerate the $|\mathcal{U}| \cdot |\Gamma_1|^{|Z|} = 27$ candidate vectors for the tiger at $\gamma = 0.95$, accuracy 0.85. The text asserts that the nine open_L assignments collapse to three distinct vectors while the nine listen ones do not. Verify it, and identify the **two** separate reasons —

one is a property of the transition and observation matrices after opening, the other is an accident of the particular numbers in Γ_1 . Then prune to the five survivors in the table above, and verify the breakpoint 0.975664 from the two lines that cross there.

2 FOUNDATION ••• Convexity without α -vectors

Prove directly from the definition of V^* as a supremum over policies that V^* is convex in b , without using the piecewise-linear representation. (Hint: a policy's expected return is linear in the initial belief; V^* is a pointwise supremum of such functions.) Then explain in one paragraph why convexity is the formal statement of "certainty is worth money", and say what the sag below the chord at $b = \frac{1}{2}$ equals in dollars.

3 CONCEPTUAL •• Predict the divergence, then verify

In the Tiger Door Console, the optimal threshold at accuracy 0.85 is 0.9611 and QMDP's is 0.9000, yet the two pilots make identical decisions. The belief after n consistent growls from $\frac{1}{2}$ is $a^n / (a^n + (1-a)^n)$, and the two policies differ exactly when a rung of that ladder lands in the gap between the thresholds. Using only that formula and a calculator, predict the set of accuracies at which they diverge — it is a union of *bands*, not an interval, and 0.85 happens to fall in a gap between two of them. Verify with the slider, then explain why raising the sensor accuracy from 0.85 to 0.95 makes QMDP *worse*. Finally: which of the two thresholds moves when you change γ , and which cannot?

4 CONCEPTUAL •• Find where the coast stops paying

In the Coastal Navigator, find the LiDAR range at which the two pilots' arrival rates come within one standard error of each other, using enough seeded runs that the answer means something. Then relate what you see to $v(x) = v_{\max} \exp(-(d/R)^2)$: at that range, what fraction of the room is informative? Finally, describe a concrete belief in the Apartment for which the AMDP compression ($\arg \max_x b(x), H[b]$) would give the planner *actively misleading* advice, and say which action it would wrongly recommend.

5 PRACTICAL •• Implement QMDP on Chapter 21's solver

`qmdp.rs` ships as a stub. Implement it on top of `ch21_mdp::value_iteration` — the only work is building `underlying_mdp()`, and the only thing to get right is that the observation model must not appear anywhere in the conversion. Then reproduce the chapter's paired tournament: 20 000 rounds at accuracies 0.70, 0.85 and 0.95, with the optimal, QMDP and impatient (open the more likely door immediately) pilots sharing one seeded growl stream per round. Your maul rates at 0.70 should land near 3.2% and 7.2%.

6 **PRACTICAL** • • • **POMCP over an MCL belief**

Wire Pomcp to the Chapter 12 MCL filter and the Chapter 4 simulator, on a T-junction with two mirror-image corridors and a bimodal particle belief. The generative model is the simulator; the root particle set is MCL's output, resampled to 300. Report how often the planner drives *past* the disambiguating doorway before committing to a corridor, as a function of the simulation budget — and find the budget below which it commits blind. Then break it deliberately: shrink the UCB1 constant until a single unlucky rollout retires the correct arm.

7 **PRACTICAL** • • • **DESPOT's determinized scenarios (stretch)**

Add a `search_despot` to `pomcp.rs`: draw K scenarios (a start state plus a seeded random stream each), share them across the whole tree so that every node evaluates all K at once, and branch only on the observations those scenarios actually produce. Use QMDP as the upper bound and "listen twice then guess" as the lower one. Compare regret against simulation count with POMCP on the tiger and on your T-junction world, and report where the regularization term $\lambda|\pi|$ starts to matter.

22.9 References

Smallwood, R. D. and Sondik, E. J. (1973). The Optimal Control of Partially Observable Markov Processes over a Finite Horizon *Operations Research* 21(5), 1071–1088.

Where piecewise-linear convexity comes from. This chapter's centerpiece derivation is their theorem, restated in modern α -vector notation; the paper is also the origin of the exact backup our `exact.rs` implements. doi:10.1287/opre.21.5.1071

Papadimitriou, C. H. and Tsitsiklis, J. N. (1987). The Complexity of Markov Decision Processes *Mathematics of Operations Research* 12(3), 441–450.

The PSPACE-completeness result quoted in the complexity box. The same paper shows the fully observable case is only P-complete, which is exactly the gap Chapter 21 was living in. doi:10.1287/moor.12.3.441

Kaelbling, L. P., Littman, M. L., and Cassandra, A. R. (1998). Planning and Acting in Partially Observable Stochastic Domains *Artificial Intelligence* 101(1–2), 99–134.

The paper that made POMDPs legible to a generation, and the source of the tiger problem this chapter runs every number on. Read §3–5 alongside the mathematics section here. doi:10.1016/S0004-3702(98)00023-X

Roy, N. and Thrun, S. (1999). Coastal Navigation with Mobile Robots *Advances in Neural Information Processing Systems 12 (NIPS 1999)*.

The augmented-MDP compression and the wall-hugging behavior the Coastal Navigator reproduces. Their Figure comparing entropy at the goal against sensor range is the experiment the widget’s slider runs live. [link](#)

Pineau, J., Gordon, G., and Thrun, S. (2003). Point-based value iteration: An anytime algorithm for POMDPs *IJCAI 2003*, 1025–1032.

PBVI, its reachable-belief expansion, and the error bound quoted (and criticized) above. The algorithm is thirty lines; the insight — approximate the belief set, not the value function — is the one that stuck. [link](#)

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics MIT Press*.

Chapter 16 is this chapter’s baseline: the finite-world exact solver of Table 16.1, the LP pruning we replace with an envelope, and §16.5’s augmented MDP. The epigraph is from §16.2 of the 1999–2000 draft, which differs from the published edition. [link](#)

Kurniawati, H., Hsu, D., and Lee, W. S. (2008). SARSOP: Efficient Point-Based POMDP Planning by Approximating Optimally Reachable Belief Spaces *Robotics: Science and Systems IV*.

PBVI industrialized: sample beliefs reachable under near-optimal policies rather than under any policy. Still the offline solver to reach for when your state space is small enough to have one. [link](#)

Silver, D. and Veness, J. (2010). Monte-Carlo Planning in Large POMDPs *Advances in Neural Information Processing Systems 23 (NIPS 2010)*.

POMCP: the algorithm in this chapter’s fifth box, and the reason a robot with a simulator and a particle filter already owns most of a POMDP solver. [link](#)

Ye, N., Somani, A., Hsu, D., and Lee, W. S. (2017). DESPOT: Online POMDP Planning with Regularization *Journal of Artificial Intelligence Research 58*, 231–266.

The journal version of the 2013 NIPS paper: determinized scenarios, the regret bound in terms of policy representation size, and the regularization that stops a K-scenario tree from overfitting its own futures. doi:10.1613/jair.5328

Lauri, M., Hsu, D., and Pajarinen, J. (2023). Partially Observable Markov Decision Processes in Robotics: A Survey *IEEE Transactions on Robotics 39(1)*, 21–40.

The current map of the field, written for roboticists. The best single answer to ‘which solver should I actually use for my problem’, and the place to go for continuous-space and multi-robot variants this chapter skips. doi:10.1109/TR0.2022.3200138

Hoerger, M., Kurniawati, H., Kroese, D., and Ye, N. (2024). Adaptive Discretization using Voronoi Trees for Continuous POMDPs *The International Journal of Robotics Research* 43(9), 1283–1298.

A recent answer to the continuous-action problem POMCP degenerates on: refine the action space where the value function says it matters. Read it after building the tree widget’s model and asking what happens when u is a real number. doi : 10.1177/02783649231188984

Stochastic MPC: MPPI and Friends

PLANNING AND ACTING UNDER UNCERTAINTY · Advanced · 55 min



Control inputs drawn from the optimal distribution achieve a lower cost, in expectation, than any other control distribution.

Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou

Information Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving (2017), §III-A

Chapter 20 produced a path. Chapter 21 and Chapter 22 produced policies on grids small enough to enumerate. None of them produce the thing a real robot actually needs, which is a pair of numbers — a linear and an angular velocity — every fifty milliseconds, in a room where somebody has moved a chair since the path was planned.

The classical answer is a feedback controller that tracks the path. It works beautifully until the world stops matching the map, and then it drives into the chair with excellent tracking error. The modern answer is to stop separating planning from execution: re-solve a short-horizon optimal control problem every cycle, execute only its first command, and throw the rest away. That is *model predictive control*, and its dominant sampling-based form, MPPI, turns out to be something the reader has already proved correct. It is importance sampling over control sequences: the particle filter of Chapter 8, aimed forward in time.

◎ AFTER THIS CHAPTER YOU CAN

- State the receding-horizon control problem and say exactly what the robot commits to each cycle
- Derive the Dynamic Window Approach from acceleration limits and

braking distance, and name the one thing its hypothesis class cannot express

- Prove the free-energy bound and identify the optimal control distribution that attains it
- Derive the MPPI weight from the likelihood ratio — including the control-cost cross term almost every tutorial drops
- Read the temperature λ as the sharpness of an importance-sampling tilt, and the effective sample size as the gauge that tells you when it is wrong
- Implement MPPI in Rust on the book's own dynamics and cost traits, with rayon rollouts and a WASM build that stays deterministic
- Say honestly where sampling beats gradients, and where MPPI is not a planner and must not be asked to be one

ASSUMED

- Importance sampling, self-normalized weights, and effective sample size (Chapter 8)
- The velocity motion model and its noise-free integration (Chapter 9)
- Euclidean signed distance fields over a map (Chapter 19)
- A reference path to track, and why sampling-based planners produce one (Chapter 20)
- Cost-to-go and terminal cost as decision-theoretic objects (Chapter 21)

23.1 The problem with a plan

A path is a claim about a world that no longer exists. It was computed from a map, and the map was recorded before the chair moved, before somebody left a box in the corridor, before the person walking toward Rusty decided which side to pass on.

A tracking controller cannot fix this, because it is not being asked to. Give it a path and it will minimize the distance to that path, which is precisely the wrong objective when the path goes through a chair. Re-running the global planner instead is honest but expensive: at twenty hertz, with a planner that takes a hundred milliseconds and returns a structurally different path each time, Rusty would spend his life twitching between homotopy classes.

The receding-horizon answer is narrower and cheaper. Do not re-plan the route. Re-solve, every cycle, a *short* optimal control problem — two seconds, not two minutes — that already knows about the chair, and execute one step of its answer.

 INTERACTIVE · w23.1**Rollout Storm**

MPPI is importance sampling over control sequences — and the command it executes is a weighted blend of the rollouts, never one of them.

Run this simulation in the web edition: `/chapters/ch23-mppi #w23.1`

Watch a few cycles before reading on. Three things are worth naming now; each one becomes a theorem later in the chapter.

Nothing here is optimized in the usual sense. There is no gradient, no line search, no convexity. Two hundred perturbed plans are simulated forward, each is assigned a number, and the next plan is a weighted average. That is the entire algorithm, and it is the reason the obstacle cost is allowed to be a hard if statement over an occupancy grid.

The purple plan is not one of the orange rollouts. It is their weighted mean, and it is visibly smoother than any of them — the signature of an *expectation* rather than a selection. When the temperature is pushed toward zero this stops being true, and the plan snaps onto whichever single rollout got lucky. Watch the effective sample size collapse toward 1 when you do it.

The storm re-centres itself. Each cycle inherits the previous plan, shifted forward one step. That warm start is why two hundred samples suffice where a cold start would need thousands, and it is the exact analogue of the particle filter’s prediction step.

 THE ONE SENTENCE VERSION

There is a distribution over control sequences whose samples are optimal in expectation; you cannot sample from it, so you sample Gaussian perturbations of your current plan, weight them by $\exp(-\text{cost}/\lambda)$, and take the weighted mean. That is MPPI, and it is self-normalized importance sampling.

23.2 Building intuition: search velocities, not paths

Before the path-integral machinery, the classical baseline — and it is a good one, still shipping in production stacks thirty years later. Fox, Burgard and Thrun’s **Dynamic Window Approach** starts from a physical observation: a differential-drive robot cannot execute an arbitrary curve. Over one short control period it executes an arc, and which arcs are available is decided by its acceleration limits. So do not search over paths. Search over the *velocities* reachable in the next period.

That search space is two-dimensional and tiny, which is why DWA ran on 1997 hardware at speeds up to 95 cm/s. Its structure is worth internalizing because

MPPI keeps one half of it and replaces the other:

- **Keep:** commands are the decision variable, and the reachable set is a box set by acceleration limits.
- **Replace:** each candidate is scored along *one constant-curvature arc*, evaluated at its endpoint. A hypothesis class of single arcs cannot express "slow, drift left, then turn hard right" — not badly, but *at all*.

INTERACTIVE · w23.2

Dynamic Window Bench

DWA searches one arc at a time inside the reachable velocity box; MPPI searches whole sequences. Neither one is a planner.

Run this simulation in the web edition: `/chapters/ch23-mppi #w23.2`

The velocity plane on the left is where both controllers actually live, and putting them in the same axes is the fastest way to see what changed. DWA lays a grid over the reachable box and asks one question per cell: *if I hold this command, where do I end up and what do I hit?* MPPI scatters two hundred samples over the same plane, but each dot is the **head of a whole sequence**, and its weight depends on what the other twenty-four steps of that sequence cost.

Two honest observations from the bench, both of which the numbers later in this chapter pin down.

First, DWA threads the chairs down the centreline with **five centimetres** to spare, and no setting of its clearance weight changes that. The reason is worth the paragraph: Fox et al.'s clearance term is $\text{dist}(v, \omega)$, the distance travelled before the arc *touches* something. A gap you fit through at all returns the same value as a gap with a metre to spare, so the term is saturated and its weight is inert. Swap in the modern repair — score the *minimum margin* anywhere along the arc, which the widget lets you toggle — and the weight bites, but what it buys is paralysis: above $w_c \approx 0.1$ Rusty stops 0.7 m short of the chair and never moves again. There is no setting in between, because the maneuver that keeps margin *and* makes progress is an S-curve, and an S-curve is not one arc.

Second, in the counter pocket **both** controllers stall. A local controller is local. Getting out of that pocket is Chapter 20's job, and no temperature setting substitutes for it.

23.3 The mathematics

23.3.1 Setting

$H, \Delta t$	Horizon in steps, and the control period. The plan spans $H \cdot \Delta t$ seconds; the robot commits to Δt of it.
$U = (u_0, \dots, u_{H-1})$	The control sequence being optimized. For Rusty, $u_k = (v_k, \omega_k)^T$.
$V = (v_0, \dots, v_{H-1})$	The sequence actually applied: the commanded mean plus injected noise, $v_k = u_k + \varepsilon_k$.
$x_{k+1} = f(x_k, u_k)$	Rollout dynamics — the noise-free Chapter 9 velocity model, integrated exactly.
$S(V) = \phi(x_H) + \sum_k c(x_k, u_k)$	Trajectory cost of one rollout: stage costs plus a terminal cost.
Σ_u	Covariance of the control perturbation. Its inverse appears in the weight, which is where the control cost comes from.
λ	Temperature. The chapter's one headline parameter: how sharply $\exp(-S/\lambda)$ prefers cheap rollouts.
$\alpha, \gamma = \lambda(1 - \alpha)$	Which base distribution defines the control cost, and the resulting coefficient of the cross term.
$w^{(i)}, \eta$	Rollout weight and its normalizer — the same η the Bayes filter has used since Chapter 5.
$\mathcal{F}(S)$	Free energy of the control system, $-\lambda \log E_p[\exp(-S/\lambda)]$.

Receding-horizon control is the following loop, and its most consequential line is the last one.

ALGORITHM `receding_horizon(bel(x_t), U)` COST one horizon optimization per control period
in the current belief, the plan carried over from last cycle
out one command, and the plan for next cycle

- 1: $x_0 \leftarrow \operatorname{argmax}_x \operatorname{bel}(x_t)$ (or the mean; see the honesty note below)
- 2: $U^* \leftarrow \operatorname{argmin}_U \mathbb{E}[\phi(x_H) + \sum_k c(x_k, u_k)]$ subject to $x_{k+1} = f(x_k, u_k)$
- 3: execute u_0^* for Δt
- 4: $U \leftarrow \operatorname{shift}(U^*)$ (drop u_0^* , slide, repeat the tail)
- 5: discard everything else and go to 1

Line 5 is the trick. The controller solves a problem it does not intend to finish, which buys two things: the optimization only has to be *locally* right, and every cycle gets a fresh measurement. Feedback enters not through a gain matrix but through re-solving.

⚠ WHERE ROBOTS BREAK THIS

Line 1 collapses a belief into a point, and the whole of Parts II–IV was about why that is a lie. Doing it properly means optimizing over belief trajectories — belief-space MPC, of which Chapter 22 is the exact-solution end. The cheap partial repair used in practice, and in this chapter’s Rust, is to inflate the robot’s footprint by the localization covariance: replace the body radius r with $r + \kappa \sqrt{\lambda_{\max}(\Sigma_t)}$, where Σ_t is the MCL position covariance from Chapter 12 and $\kappa \approx 2$. A robot that does not know where it is drives as though it were bigger.

23.3.2 DWA, formally

The dynamic window is the intersection of three sets. The actuator envelope $V_s = [v_{\min}, v_{\max}] \times [-\omega_{\max}, \omega_{\max}]$; the *reachable* set

$$V_d = [v - a_{\max}\Delta t, v + a_{\max}\Delta t] \times [\omega - \dot{\omega}_{\max}\Delta t, \omega + \dot{\omega}_{\max}\Delta t]$$

around the current command; and the *admissible* set V_a , the commands from which the robot can still stop before whatever its arc runs into.

📐 DERIVATION Where the admissibility inequality comes from

$$|v| \leq \sqrt{2 \operatorname{dist}(v, \omega) a_{\max}}$$

Let $\operatorname{dist}(v, \omega)$ be the arclength travelled along the arc of curvature ω/v before the robot’s body first touches an obstacle. Braking at $a_{\max}$ from speed v takes $t_b = |v|/a_{\max}$ and covers

$$d_b = |v|t_b - \frac{1}{2}a_{\max}t_b^2 = \frac{v^2}{2a_{\max}}.$$

Requiring $d_b \leq \operatorname{dist}(v, \omega)$ gives the inequality above, which is Fox et al.’s admissibility condition. The same argument on ω with the angular limit $\dot{\omega}_{\max}$ bounds the rotation.

Two things are worth noticing. The first is that dist must be measured along the arc, not as a straight line — a fast arc that curves away from a wall is safe, and a slow one that curves into it is not. The second is that this condition is *exact only if the robot keeps the command*. It is a one-step guarantee, re-derived every cycle, which is why DWA at speed depends on a fast control loop rather than on a long horizon.

ALGORITHM `dwa_plan(x, x_goal, u_prev, esdf)` COST $0(n_v \cdot n_\omega \cdot H)$ -- 651 arcs $\times$ 40 integration steps in this chapter's lab
in pose, goal, previous command, an obstacle distance field
out the command (v, ω) maximizing the objective over the admissible window

```

1:  $V_d \leftarrow$  reachable box around  $u_{prev}$  from the acceleration limits
2: for all  $(v, \omega)$  on a grid over  $V_s \cap V_d$  do
3:   integrate the arc;  $\text{dist} \leftarrow$  arclength to first contact
4:   if  $|v| > \sqrt{2} \text{dist } a_{\max}$  then mark inadmissible; continue
5:    $G(v, \omega) \leftarrow \sigma(w_h \cdot \text{heading} + w_c \cdot \text{clearance} + w_v \cdot \text{velocity})$ 
6: endfor
7: return  $\text{argmax } G$  over the admissible candidates

```

σ is normalization across the window: each term is scaled to $[0, 1]$ before the weighted sum, so that metres and radians cannot decide the outcome by their units. (Fox et al. call the three weights α, β, γ ; this chapter needs those letters shortly for the temperature family, so they are w_h, w_c, w_v here.) Everything else is a design choice, and every one of those choices is a place where a real deployment spends a week.

23.3.3 The optimal distribution

Now the modern half. Fix a **base distribution** $p(V)$ over control sequences — think of it as a prior over what the robot might do — and define the **free energy** of the control system,

$$\mathcal{F}(S) = -\lambda \log \mathbb{E}_p \left[\exp\left(-\frac{1}{\lambda} S(V)\right) \right].$$

This object is the bridge between "minimize a cost" and "sample from a distribution", and the bridge is a single inequality.

 **DERIVATION** The free-energy bound and the distribution that attains it

$$q^\star(V) = \frac{1}{\eta} p(V) e^{-S(V)/\lambda}$$

Statement. For every distribution q absolutely continuous with respect to p ,

$$\mathcal{F}(S) \leq \mathbb{E}_q[S(V)] + \lambda \text{KL}(q \| p),$$

with equality if and only if $q = q^\star$, where $q^\star(V) = \frac{1}{\eta} p(V) e^{-S(V)/\lambda}$ and

$$\eta = \mathbb{E}_p[e^{-S/\lambda}].$$

Step 1 — insert the proposal. Multiply and divide inside the expectation by q , which is the importance-sampling identity and nothing more:

$$\mathcal{F}(S) = -\lambda \log \mathbb{E}_q \left[\frac{p(V)}{q(V)} e^{-S(V)/\lambda} \right].$$

Step 2 — Jensen. The logarithm is concave, so $\log \mathbb{E}_q[Z] \geq \mathbb{E}_q[\log Z]$. Multiplying by $-\lambda < 0$ flips it:

$$\mathcal{F}(S) \leq -\lambda \mathbb{E}_q \left[\log \frac{p(V)}{q(V)} - \frac{S(V)}{\lambda} \right] = \mathbb{E}_q[S(V)] + \lambda \mathbb{E}_q \left[\log \frac{q(V)}{p(V)} \right].$$

Step 3 — name the second term. That expectation is exactly $\text{KL}(q\|p)$, giving the bound. Read it as an optimal control objective: *expected cost, plus a temperature times the price of deviating from the base behaviour*. The KL term is the control cost, and it arrived without being postulated.

Step 4 — equality. Jensen is tight exactly when its argument is q -almost-surely constant: $\frac{p(V)}{q(V)} e^{-S(V)/\lambda} = c$. Solving for q gives $q(V) \propto p(V) e^{-S(V)/\lambda}$, which normalizes to q^* . Substituting q^* back into the right-hand side recovers $-\lambda \log \eta = \mathcal{F}(S)$, so the bound is attained. ■

Two readings of the same formula. Statistically, q^* is a Boltzmann distribution: a prior tilted by an exponentiated cost. Bayesianly, it is a posterior — with the prior $p(V)$ and the "likelihood" $e^{-S(V)/\lambda}$, so that **a cost is a negative log-likelihood and λ is the noise level you are willing to assume about your own objective**. Every posterior in this book has had that shape; this one just happens to be over the future.

Convention note. Williams et al. define $F = \log \mathbb{E}_p[e^{-S/\lambda}]$ and call λ the *inverse* temperature, so their bound reads $-\lambda F \leq \mathbb{E}_Q[S] + \lambda \text{KL}(Q\|P)$. The statement is identical; this book absorbs the $-\lambda$ into $\mathcal{F}$ and calls λ the temperature, because larger λ flattens the tilt exactly as temperature flattens a Boltzmann distribution.

23.3.4 The twin theorem: MPPI is importance sampling

We now have a target, q^* , and no way to sample from it — the familiar predicament of Chapter 8, one dimension per horizon step deeper. The familiar remedy applies.

 **DERIVATION** From q^* to the MPPI update, cross term and all

$$w^{(i)} \propto \exp \left(-\frac{1}{\lambda} (\tilde{S}^{(i)} - \rho) \right)$$

Step 1 — what we actually want. Since $q^\star$ is a distribution and the robot must emit a single sequence, project: choose the Gaussian $q(\cdot \mid U, \Sigma_u)$ closest to $q^\star$ in KL. Expanding $\text{KL}(q^\star \parallel q(\cdot \mid U, \Sigma_u))$ and dropping the terms that do not involve U leaves a quadratic minimization whose solution is moment matching,

$$u_k^\star = \mathbb{E}_{q^\star}[v_k], \quad k = 0, \dots, H-1.$$

The optimal open-loop plan is the **mean of the optimal control distribution**. Not its mode, not its best sample.

Step 2 — a proposal we can sample. Let $\hat{U}$ be the plan we are carrying (the shifted result of last cycle) and draw $V^{(i)} = \hat{U} + \epsilon^{(i)}$, $\epsilon_k^{(i)} \sim \mathcal{N}(0, \Sigma_u)$. Self-normalized importance sampling gives

$$\mathbb{E}_{q^\star}[v_k] = \mathbb{E}_{q(\cdot \mid \hat{U})}[w(V) v_k], \quad w(V) = \frac{q^\star(V)}{q(V \mid \hat{U}, \Sigma_u)}.$$

Step 3 — expand the ratio. Split it through the base distribution:

$$w(V) = \frac{1}{\eta} e^{-S(V)/\lambda} \frac{p(V)}{q(V \mid \hat{U}, \Sigma_u)}.$$

Take the base to be Gaussian about some nominal $\tilde{U}$, i.e. $p = q(\cdot \mid \tilde{U}, \Sigma_u)$. Then the log of that last ratio is a difference of two quadratics, and the quadratic terms in v cancel:

$$\log \frac{p(V)}{q(V \mid \hat{U})} = -\frac{1}{2} \sum_k \left[(v_k - \tilde{u}_k)^\top \Sigma_u^{-1} (v_k - \tilde{u}_k) - (v_k - \hat{u}_k)^\top \Sigma_u^{-1} (v_k - \hat{u}_k) \right] = \text{const} - \sum_k (\hat{u}_k - \tilde{u}_k)^\top \Sigma_u^{-1} v_k.$$

Step 4 — the cross term. Substitute $v_k = \hat{u}_k + \epsilon_k$ and drop everything that does not depend on the sample (self-normalization kills it). Writing the standard family $\tilde{U} = \alpha \hat{U}$ with $\alpha \in [0, 1]$, so that $\hat{u}_k - \tilde{u}_k = (1 - \alpha)\hat{u}_k$, and setting $\gamma = \lambda(1 - \alpha)$:

$$\tilde{S}^{(i)} = \underbrace{S(V^{(i)})}_{\text{state cost}} + \gamma \sum_{k=0}^{H-1} \hat{u}_k^\top \Sigma_u^{-1} \epsilon_k^{(i)}.$$

This second term is the one every quick tutorial drops. It is not decoration: it *is* the quadratic control cost, arriving through the likelihood ratio rather than by decree. Choosing α chooses what "control cost" means.

- $\alpha = 0$: the base is the **uncontrolled** system, $\tilde{U} \equiv 0$, so $\gamma = \lambda$ and the objective charges $\frac{\lambda}{2} \sum_k u_k^\top \Sigma_u^{-1} u_k$ — genuine control effort. A robot under this base is reluctant to move at all unless the state cost pays for it.
- $\alpha = 1$: the base is **the current plan**, so $\gamma = 0$ and the cross term vanishes. Effort is free; what the KL term now charges is $\frac{1}{2} \sum_k (u_k - \hat{u}_k)^\top \Sigma_u^{-1} (u_k - \hat{u}_k)$, the *deviation from last cycle's plan*. This is a trust region, it is why $\alpha = 1$ implementations produce smooth motion, and it is the setting this chapter's lab uses.

Step 5 — weights and the update. Subtract $\rho = \min_i \tilde{S}^{(i)}$ before exponentiating. Self-normalized weights are invariant to any constant shift of the cost, so this changes nothing mathematically and everything numerically: it guarantees the largest exponent is exactly $e^0 = 1$.

$$w^{(i)} = \frac{1}{\eta} \exp\left(-\frac{1}{\lambda}(\tilde{S}^{(i)} - \rho)\right), \quad \eta = \sum_j \exp\left(-\frac{1}{\lambda}(\tilde{S}^{(j)} - \rho)\right),$$

$$u_k \leftarrow \hat{u}_k + \sum_{i=1}^K w^{(i)} \epsilon_k^{(i)}.$$

That is the whole algorithm, and it is the self-normalized importance-sampling estimator of $\mathbb{E}_{q^*}[v_k]$ — biased at finite K , consistent as $K \rightarrow \infty$, exactly like every particle filter estimate in Part II. ■

```

ALGORITHM MPPI(x_0, U, K, λ, Σ_u) COST O(K · H) dynamics and cost evaluations --
embarrassingly parallel over K

in current state, the plan carried over, sample count, temperature, perturbation covariance
out the command to execute, and the improved plan

```

```

1: for  $i = 1$  to  $K$  do (in parallel)
2:    $x \leftarrow x_0$ ;  $\tilde{S}^{(i)} \leftarrow 0$ 
3:   for  $k = 0$  to  $H - 1$  do
4:      $\epsilon_k^{(i)} \sim \mathcal{N}(0, \Sigma_u)$ ;  $v_k \leftarrow g(u_k + \epsilon_k^{(i)})$ 
5:      $x \leftarrow f(x, v_k)$ 
6:      $\tilde{S}^{(i)} \leftarrow \tilde{S}^{(i)} + c(x, v_k) + \gamma u_k^T \Sigma_u^{-1} \epsilon_k^{(i)}$ 
7:   endfor
8:    $\tilde{S}^{(i)} \leftarrow \tilde{S}^{(i)} + \phi(x)$ 
9: endfor
10:  $w \leftarrow \text{ComputeWeights}(\tilde{S}^{(1:K)}, \lambda)$ 
11: for  $k = 0$  to  $H - 1$  do  $u_k \leftarrow u_k + \sum_i w^{(i)} \epsilon_k^{(i)}$ 
12:  $U \leftarrow \text{SavitzkyGolay}(U)$ 
13: execute  $u_0$ ; shift  $U$  (and report  $\text{ESS} = 1/\sum_i (w^{(i)})^2$ )

```

ALGORITHM $\text{ComputeWeights}(\tilde{S}_{1:K}, \lambda)$

COST $\mathcal{O}(K)$

in the K tilted rollout costs and the temperature

out self-normalized weights

```

1:  $\rho \leftarrow \min_i \tilde{S}^{(i)}$ 
2:  $\eta \leftarrow \sum_i \exp(-(\tilde{S}^{(i)} - \rho)/\lambda)$ 
3: for  $i = 1$  to  $K$  do  $w^{(i)} \leftarrow \frac{1}{\eta} \exp(-(\tilde{S}^{(i)} - \rho)/\lambda)$ 
4: return  $w$ 

```

Set the two algorithms side by side and there is nothing left to argue about.

sub-step	particle filter (Ch. 8)	MPPI
hypothesis	a state $x^{(i)}$	a control sequence $U^{(i)}$
proposal	motion model from the previous belief	Gaussian around the previous plan
weight	$w^{(i)} \propto p(z_t x^{(i)})$	$w^{(i)} \propto e^{-\tilde{S}^{(i)}/\lambda}$
estimate	$\hat{x} = \sum_i w^{(i)} x^{(i)}$	$U \leftarrow \hat{U} + \sum_i w^{(i)} \epsilon^{(i)}$
renewal	resample, then predict forward	execute u_0 , shift, re-centre
failure	particle deprivation: no sample near the truth	no rollout near the good maneuver
gauge	$\text{ESS} = 1/\sum_i (w^{(i)})^2$	the same formula, unchanged

 INTERACTIVE · w23.3**Twin Vision**

MPPI is not a new algorithm. It is the particle filter’s importance-sampling step, aimed at the future.

Run this simulation in the web edition: `/chapters/ch23-mppi #w23.3`

The correspondence is not an analogy, and the widget is not a metaphor: both panels run library code that the rest of this book already uses. The left one calls `lowVarianceResample` from `lib/filters/pf.ts`; the right one calls the same `Mppi` class that drives the corridor at the top of this chapter, restricted to one dimension.

One asymmetry is real and worth stating. The particle filter’s weights are handed to it — the likelihood is physics, and there is no knob. MPPI’s weights come from a cost *you invented*, so λ is a free parameter: it says how much you believe your own objective. That freedom is the subject of the next section.

23.3.5 Temperature, and the two limits **DERIVATION The $\lambda \rightarrow 0$ and $\lambda \rightarrow \infty$ limits**

best-of-K, and the unweighted mean

Order the sampled costs so that $\tilde{S}^{(1)} < \tilde{S}^{(2)} \leq \dots$, and assume the minimum is unique. Then

$$w^{(i)} = \frac{e^{-(\tilde{S}^{(i)} - \tilde{S}^{(1)})/\lambda}}{\sum_j e^{-(\tilde{S}^{(j)} - \tilde{S}^{(1)})/\lambda}}.$$

As $\lambda \rightarrow 0^+$ every exponent with $\tilde{S}^{(i)} > \tilde{S}^{(1)}$ has a numerator going to zero, so $w^{(1)} \rightarrow 1$ and the update becomes $u_k \leftarrow \hat{u}_k + \epsilon_k^{(1)}$: the single cheapest rollout wins outright, and $\text{ESS} \rightarrow 1$. MPPI degenerates into random search with K candidates, which is why the executed command starts jittering — a different sample wins every cycle, and the difference between the winner and the runner-up is Monte-Carlo noise.

As $\lambda \rightarrow \infty$ every exponent goes to 0 and $w^{(i)} \rightarrow 1/K$, so $\text{ESS} \rightarrow K$ and the update is $u_k \leftarrow \hat{u}_k + \frac{1}{K} \sum_i \epsilon_k^{(i)}$. The perturbations are zero-mean, so the correction is $O(\sigma/\sqrt{K})$ — noise, not signal. The plan stops responding to the cost at all.

Neither limit is a failure of the derivation: they are the correct behaviour of an exponential tilt at the two ends of its range. The engineering question is where in between to sit, and the ESS is the instrument that answers it.

Here is that trade-off measured, on the corridor run of w23.1 (seed 23, $K = 200$, $H = 25$, $\Delta t = 0.1$ s, $\alpha = 1$). "Jitter" is the mean $|\Delta u|$ between consecutive executed commands — the number that shows up on screen as twitching.

λ	ESS (of 200)	time to goal	min clearance	jitter
0.5	1.5	17.9 s	0.16 m	0.583
2	6.1	18.8 s	0.21 m	0.416
8	26.0	15.2 s	0.23 m	0.156
30	108.6	18.9 s	0.21 m	0.103
120	187.8	34.1 s	0.11 m	0.080

Read the extremes. At $\lambda = 0.5$ the controller is fast to react and horrible to ride, and its clearance suffers because a single lucky rollout is allowed to steer. At $\lambda = 120$ it is serene, twice as slow, and cuts its margin in half — the weights are nearly uniform, so the barrier cost barely moves the plan. The useful band is the one where the ESS sits somewhere around a tenth of K : enough samples voting to average out the noise, few enough that the cheap ones dominate.

i NOTE

The same gauge, the same reading, the same fix as Chapter 8: if the ESS is pinned near 1, your proposal does not cover the target. In a filter you fix it with a better proposal; here you fix it by raising λ , by widening Σ_u , or — most often — by smoothing the cost so that the rollouts are not all equally catastrophic.

23.3.6 Constraints, and how sampling gets away with them

Input limits. Do not reject samples and do not solve a QP. Push the clamp into the dynamics: $x_{k+1} = f(x_k, g(v_k))$ with g the saturation. The sampler stays a plain Gaussian, and the non-smooth g costs nothing because the update law never differentiates f . Clamp *before* integrating, so that the trajectory you score is one the actuators would actually produce.

Obstacles. Two options, and the difference is measured in ESS. An indicator

cost $c_{obs} = M \cdot \mathbb{I}[\text{in collision}]$ is legal — no gradient is required — but in a tight corridor it makes most rollouts equally infinite, the weights collapse onto whichever sample squeaked through, and the ESS falls to 1. The smooth barrier over the signed distance field of Chapter 19,

$$c_{obs}(x) = w_o \exp\left(-\frac{d_{\text{ESDF}}(x) - r}{\sigma_o}\right),$$

grades the rollouts instead of censoring them, which keeps the importance weights informative. This chapter’s lab uses the barrier *plus* a flat 900 for actual collision: graded where grading helps, brutal where it must be.

Chance constraints. Inflating the footprint by the belief covariance, as in the callout above, is the cheap version. The honest version optimizes over belief trajectories and is a research field.

23.4 Implementation in Rust

The design is two traits and one struct. Making Dynamics and CostFn traits rather than closures is what leaves the socket open for the gradient methods at the end of this chapter: iLQR needs the same simulator, plus derivatives.

RUST crates/ch23_mppi/src/dynamics.rs

```
1 use nalgebra::SVector;
2
3 /// A simulator. Note what is not required: no Jacobians, no differentiability,
4 /// no continuity. This trait is the entire interface MPPI needs from a model.
5 pub trait Dynamics<const NX: usize, const NU: usize>: Sync {
6     fn step(&self, x: &SVector<f64, NX>, u: &SVector<f64, NU>, dt: f64) -> SVector<f64,
7         NX>;
8
9     /// Input constraints, pushed into the dynamics (Williams et al. sec.III-D3).
10    /// Clamp before integrating: a plan scored on commands the motors would
11    /// refuse is a plan scored on a lie.
12    fn clamp(&self, u: SVector<f64, NU>) -> SVector<f64, NU>;
13 }
14
15 /// Rusty: unicycle kinematics, the Chapter 9 velocity model with the noise off.
16 pub struct DiffDrive {
17     pub v_lim: (f64, f64),
18     pub w_max: f64,
19     pub a_max: f64,
20 }
21
22 impl Dynamics<3, 2> for DiffDrive {
23     fn step(&self, x: &SVector<f64, 3>, u: &SVector<f64, 2>, dt: f64) -> SVector<f64, 3>
24     {
25         let (v, w) = (u[0], u[1]);
26         let th = x[2];
27         // The exact arc solution (Thrun et al., eq. 5.9). As omega -> 0 the radius
28         // blows up while the arc flattens; rather than trust that cancellation
29         // numerically we switch to the straight-line limit, exactly as the
30         // TypeScript port in `lib/sim/world.ts` does.
31         if w.abs() < 1e-9 {
```



```

30         SVector::<f64, 3>::new(x[0] + v * th.cos() * dt, x[1] + v * th.sin() * dt, th
31     )
32     } else {
33         let r = v / w;
34         let nt = th + w * dt;
35         SVector::<f64, 3>::new(
36             x[0] - r * th.sin() + r * nt.sin(),
37             x[1] + r * th.cos() - r * nt.cos(),
38             wrap_pi(nt),
39         )
40     }
41 }
42
43 fn clamp(&self, u: SVector<f64, 2>) -> SVector<f64, 2> {
44     SVector::<f64, 2>::new(
45         u[0].clamp(self.v_lim.0, self.v_lim.1),
46         u[1].clamp(-self.w_max, self.w_max),
47     )
48 }
49
50 pub trait CostFn<const NX: usize, const NU: usize>: Sync {
51     fn stage(&self, x: &SVector<f64, NX>, u: &SVector<f64, NU>, k: usize) -> f64;
52     fn terminal(&self, x: &SVector<f64, NX>) -> f64;
53 }
54
55 /// Track Chapter 20's path, keep clear of Chapter 19's distance field.
56 pub struct TrackAndClear<'a> {
57     pub path: &'a [SVector<f64, 2>],
58     pub esdf: &'a ch19_maps::Esdf2,
59     pub obstacles: &'a [Circle],
60     pub robot_radius: f64,
61     pub w_track: f64,
62     pub w_progress: f64,
63     pub w_obs: f64,
64     pub sigma_o: f64,
65     pub collision: f64,
66 }
67
68 impl CostFn<3, 2> for TrackAndClear<'_> {
69     fn stage(&self, x: &SVector<f64, 3>, u: &SVector<f64, 2>, _k: usize) -> f64 {
70         let (cross, s) = self.project(x[0], x[1]);
71         let clear = self.clearance(x[0], x[1]);
72         let obs = if clear <= 0.0 {
73             // A discontinuity. Legal here, fatal for a gradient method.
74             self.collision
75         } else {
76             self.w_obs * (-clear / self.sigma_o).exp()
77         };
78         self.w_track * cross * cross + obs - self.w_progress * s + 0.4 * u[1] * u[1]
79     }
80
81     fn terminal(&self, x: &SVector<f64, 3>) -> f64 {
82         let (_, s) = self.project(x[0], x[1]);
83         8.0 * (self.path_length() - s)
84     }
85 }

```

The controller itself. The only subtle line is where the randomness happens:

perturbations are drawn **serially** from a seeded generator and only then handed to rayon, because a parallel iterator that draws its own noise would make the run non-reproducible — and every simulation in this book is reproducible on purpose.

RUST crates/ch23_mppi/src/mppi.rs

```

1 use nalgebra::{SMatrix, SVector};
2 use rand::{rngs::SmallRng, SeedableRng};
3 use rand_distr::{Distribution, Normal};
4
5 pub struct Mppi<D, C, const NX: usize, const NU: usize, const H: usize>
6 where
7     D: Dynamics<NX, NU>,
8     C: CostFn<NX, NU>,
9 {
10     pub lambda: f64,
11     pub sigma_u: SMatrix<f64, NU, NU>,
12     pub k_samples: usize,
13     // gamma = lambda(1 - alpha). Zero means alpha = 1: the base distribution is the
14     current
15     // plan, so the KL term charges deviation from it rather than effort.
16     pub gamma: f64,
17     nominal: [SVector<f64, NU>; H],
18     dynamics: D,
19     cost: C,
20     rng: SmallRng,
21 }
22
23 pub struct MppiDiag<const NX: usize> {
24     pub weights: Vec<f64>,
25     pub ess: f64,
26     pub s_min: f64,
27     // Every rollout, for the widget's storm. Native builds drop this.
28     pub rollouts: Vec<Vec<SVector<f64, NX>>>,
29 }
30
31 impl<D, C, const NX: usize, const NU: usize, const H: usize> Mppi<D, C, NX, NU, H>
32 where
33     D: Dynamics<NX, NU>,
34     C: CostFn<NX, NU>,
35 {
36     // One control cycle. Returns the command to execute and the diagnostics
37     // the widget renders; the improved plan is left in `self.nominal`.
38     pub fn plan(&mut self, x0: &SVector<f64, NX>, dt: f64) -> (SVector<f64, NU>, MppiDiag
39     <NX>) {
40         let sigma_inv = self.sigma_u.try_inverse().expect("Sigma_u must be positive
41         definite");
42         let normals: Vec<Normal<f64>> = (0..NU)
43             .map(|i| Normal::new(0.0, self.sigma_u[(i, i)].sqrt()).unwrap())
44             .collect();
45
46         // Draw every perturbation up front, from the seeded generator. This is
47         // the price of determinism under rayon, and it is worth paying: the
48         // same seed must produce the same storm on 1 core and on 32.
49         let (k_samples, rng) = (self.k_samples, &mut self.rng);
50         let eps: Vec<SVector<f64, NU>; H> = (0..k_samples)
51             .map(|_| {
52                 std::array::from_fn(|_| SVector::<f64, NU>::from_fn(|i, _| normals[i].
53                 sample(rng)))
54             })

```

```

50         })
51         .collect();
52
53         // Embarrassingly parallel: K independent simulations of H steps. On
54         // wasm32 there are no threads, so the same closure runs serially -- and
55         // at K = 200, H = 25 that is 5 000 rollout steps a cycle, which this
56         // book's TypeScript port measures at about 1.6 ms on a laptop core.
57         #[cfg(not(target_arch = "wasm32"))]
58         let costs: Vec<f64> = { use rayon::prelude::*; eps.par_iter().map(|e| self.
rollout(x0, e, &sigma_inv, dt)).collect() };
59         #[cfg(target_arch = "wasm32")]
60         let costs: Vec<f64> = eps.iter().map(|e| self.rollout(x0, e, &sigma_inv, dt)).
collect();
61
62         let (weights, ess, s_min) = information_theoretic_weights(&costs, self.lambda);
63
64         // u_k <- u_k + Sigma_i w_i epsilon_k^(i): the self-normalized IS estimate.
65         for k in 0..H {
66             let mut delta = SVector::<f64, NU>::zeros();
67             for (i, e) in eps.iter().enumerate() {
68                 delta += weights[i] * e[k];
69             }
70             self.nominal[k] = self.dynamics.clamp(self.nominal[k] + delta);
71         }
72         savitzky_golay_inplace(&mut self.nominal); // sec.III-D4: kill the Monte-Carlo
chatter
73
74         let u0 = self.nominal[0];
75         (u0, MppiDiag { weights, ess, s_min, rollouts: vec![] })
76     }
77
78     /// S = S(V) + gamma Sigma_k u_k^T Sigma_u-1 epsilon_k -- state cost plus the cross
term.
79     fn rollout(
80         &self,
81         x0: &SVector<f64, NX>,
82         eps: &[SVector<f64, NU>; H],
83         sigma_inv: &SMatrix<f64, NU, NU>,
84         dt: f64,
85     ) -> f64 {
86         let mut x = *x0;
87         let mut s = 0.0;
88         for k in 0..H {
89             let u = self.dynamics.clamp(self.nominal[k] + eps[k]);
90             x = self.dynamics.step(&x, &u, dt);
91             s += self.cost.stage(&x, &u, k);
92             s += self.gamma * self.nominal[k].dot(&(sigma_inv * eps[k]));
93         }
94         s + self.cost.terminal(&x)
95     }
96
97     /// The receding step: drop the executed command, slide, repeat the tail.
98     /// The twin of the particle filter's prediction -- the same belief, moved
99     /// one step into the future and re-centred there.
100     pub fn shift(&mut self) {
101         for k in 0..H - 1 {
102             self.nominal[k] = self.nominal[k + 1];
103         }
104     }
105 }
106

```

```

107 // `ComputeWeights` -- Williams et al., Algorithm 2.
108 pub fn information_theoretic_weights(costs: &[f64], lambda: f64) -> (Vec<f64>, f64, f64)
109 {
110     let rho = costs.iter().cloned().fold(f64::INFINITY, f64::min);
111     let mut w: Vec<f64> = costs.iter().map(|s| (-(s - rho) / lambda).exp()).collect();
112     let eta: f64 = w.iter().sum();
113     w.iter_mut().for_each(|x| *x /= eta);
114     let ess = 1.0 / w.iter().map(|x| x * x).sum::<f64>();
115     (w, ess, rho)
116 }

```

23.4.1 A worked example you can check by hand

Take the smallest MPPI that is still MPPI: horizon $H = 1$, $K = 3$ rollouts, temperature $\lambda = 2$, perturbation $\sigma_v = 0.4$ m/s, and a nominal command $\hat{u} = 0.5$ m/s. Suppose the three perturbations come out $\epsilon = (+0.4, 0, -0.4)$ and the simulator returns state costs

$$S = (4, 2, 6).$$

With $\alpha = 1$ (the base is the current plan, $\gamma = 0$, no cross term) the shift is $\rho = 2$, so the unnormalized weights are e^{-1} , e^0 , $e^{-2} = 0.36788$, 1 , 0.13534 , summing to $\eta = 1.50321$. Normalizing:

$$w = (0.24473, 0.66524, 0.09003), \quad \text{ESS} = \frac{1}{\sum_i w_i^2} = \frac{1}{0.51054} = 1.959.$$

Two of three rollouts are effectively voting. The update is the weighted mean of the perturbations,

$$\Delta u = 0.4(0.24473) + 0 - 0.4(0.09003) = +0.06188, \quad u \leftarrow 0.5619 \text{ m/s}.$$

Now switch to $\alpha = 0$, the uncontrolled base, so $\gamma = \lambda = 2$ and $\Sigma_u^{-1} = 1/0.16 = 6.25$. Each rollout picks up $\gamma \hat{u} \Sigma_u^{-1} \epsilon = 2(0.5)(6.25)\epsilon = 6.25 \epsilon$:

$$\tilde{S} = (6.5, 2, 3.5) \implies w = (0.06680, 0.63381, 0.29939), \quad \Delta u = -0.09303, \quad u \leftarrow 0.4070 \text{ m/s}.$$

The same three rollouts, the same state costs, and the update **changes sign**. Judged on state cost alone, going faster looked good; charged for the effort of going faster against a base distribution that would sit still, it no longer does. That is what the cross term is for, and why dropping it silently changes the problem you are solving.

RUST `crates/ch23_mppi/src/mppi.rs (tests)`

```

1  #[test]
2  fn worked_example_ch23_three_rollouts() {
3      // alpha = 1: no cross term. Weights are  $e^{-1}$ ,  $e^{\{0\}}$ ,  $e^{-2}$ , normalized.
4      let (w, ess, rho) = information_theoretic_weights(&[4.0, 2.0, 6.0], 2.0);
5      assert_relative_eq!(rho, 2.0, epsilon = 1e-12);
6      assert_relative_eq!(w[0], 0.2447284, epsilon = 1e-6);
7      assert_relative_eq!(w[1], 0.6652406, epsilon = 1e-6);
8      assert_relative_eq!(w[2], 0.0900310, epsilon = 1e-6);
9      assert_relative_eq!(ess, 1.958699, epsilon = 1e-5);
10
11     let eps = [0.4, 0.0, -0.4];
12     let du: f64 = w.iter().zip(eps).map(|(wi, e)| wi * e).sum();
13     assert_relative_eq!(du, 0.0618787, epsilon = 1e-6);
14
15     // alpha = 0: gamma = lambda = 2, Sigma_u-1 = 1/0.42 = 6.25, nominal u = 0.5.
16     let tilted: Vec<f64> = [4.0, 2.0, 6.0]
17         .iter()
18         .zip(eps)
19         .map(|(s, e)| s + 2.0 * 0.5 * 6.25 * e)
20         .collect();
21     assert_relative_eq!(tilted[0], 6.5, epsilon = 1e-12);
22
23     let (w2, _, _) = information_theoretic_weights(&tilted, 2.0);
24     let du2: f64 = w2.iter().zip(eps).map(|(wi, e)| wi * e).sum();
25     assert_relative_eq!(du2, -0.0930347, epsilon = 1e-6);
26     assert!(du2 < 0.0, "the control cost must reverse the update's sign");
27 }
28
29 /// Self-normalized weights cannot see a constant shift of the cost -- which is
30 /// exactly why subtracting rho is safe.
31 #[test]
32 fn shift_invariance() {
33     let (a, _, _) = information_theoretic_weights(&[3.0, 7.0, 11.0, 2.0], 1.5);
34     let (b, _, _) = information_theoretic_weights(&[1003.0, 1007.0, 1011.0, 1002.0], 1.5);
35     ;
36     for (x, y) in a.iter().zip(b) {
37         assert_relative_eq!(*x, y, epsilon = 1e-12);
38     }
39 }

```

The TypeScript port runs the same assertions in `lib/control/__checks_ch23__.ts`, and the widgets on this page call the same functions the tests do. If the prose and the code ever disagree, the tests settle it.

23.5 Putting it together

Here is the corridor run of w23.1, executed headless over twelve seeds: Rusty tracks a path planned before two chairs moved, with a third obstacle being pushed along the corridor while he drives. $K = 200$, $H = 25$ (2.5 s), $\Delta t = 0.1$ s, $\lambda = 8$, $\Sigma_u = \text{diag}(0.22^2, 0.55^2)$, $\alpha = 1$.

metric	MPPI	DWA ($w_c = 0.2$)
runs that reached the goal	12 of 12 seeds	1 of 1 (deterministic)
collisions	0	0
worst clearance over all runs	0.194 m	0.050 m
mean cross-track error	0.051 m	0.000 m
time to goal (mean)	16.8 s	15.9 s
command jitter, mean $ \Delta u $	0.169	0.008
cost per cycle	5 000 dynamics + cost evaluations	651 arcs $\times$ 40 steps

Both controllers get there, and reading this table as "MPPI wins" would be the wrong lesson. DWA is *smoother* and marginally faster, because it always picks a single arc and the arc it picks is usually "straight ahead, fast". What it will not do is deviate laterally to buy margin — and, as the bench shows, cannot be made to by tuning. Its clearance term saturates the moment an arc is collision-free, and replacing it with a true minimum margin turns the controller timid rather than graceful: measured on this run, $w_c \leq 0.08$ still shaves the chairs at 0.050 m, and $w_c \geq 0.12$ freezes 0.7 m short. MPPI's exponential barrier grades the last few centimetres continuously and its hypothesis is a whole 2.5-second maneuver, so it swings out and back and keeps four times the margin at the same speed.

The differences that *are* structural are these. MPPI evaluates a whole 2.5-second maneuver as one hypothesis, so it can commit to a plan whose first half looks worse. And MPPI does not care what the cost is made of — the barrier, the flat 900 for collision, and a raw occupancy lookup are all the same to it. Neither property is available to a controller that scores one arc by its endpoint.

23.5.1 When gradients beat samples

NOTE

Rule of thumb. Sampling for contact-rich, discontinuous, or multi-modal problems. Gradients for smooth, high-dimensional, or certified ones.

	MPPI (sampling)	iLQR / DDP / SQP (gradients)
needs	a simulator of f , pointwise costs	$\nabla f, \nabla c$, and a smooth world
cost functions	anything, including indicators and grids	must be differentiable; ESDF, not occupancy
parallelism	embarrassing; K independent rollouts	sequential backward pass

	MPPI (sampling)	iLQR / DDP / SQP (gradients)
dimension	sample complexity grows badly	handles tens of states comfortably
constraints	soft, via cost; jitters near boundaries	hard, via KKT; tight and reliable
local minima	escapes shallow ones by sampling both sides	commits to one homotopy class
what it returns	a mean, re-estimated every cycle	a locally optimal trajectory <i>and a feedback gain</i>

That last row is the one people forget. iLQR returns $u_k = \bar{u}_k + K_k(x_k - \bar{x}_k)$ — a time-varying feedback law, valid between control cycles. MPPI returns a single open-loop command and relies on re-solving fast enough that open loop never has time to matter. At 20 Hz on a diff-drive that is a fine bet; on a machine whose joints need commands hundreds of times faster it is not, which is why sampling controllers at those rates are wrapped in a fast ancillary feedback law that tracks the sampled plan between updates.

23.5.2 What MPPI is not

The counter pocket in w23.2 is in this chapter because it is the failure most likely to bite a reader who has just watched the storm and concluded that sampling solves planning. Rusty sits behind the kitchen counter; the goal is two metres north, through it. Escaping costs 1.9 m of travel in the wrong direction before a single metre is repaid.

MPPI does not escape it. Not at $H = 10$, not at $H = 60$: the perturbations are white noise around the current plan, so the probability that some rollout traces a coherent four-metre detour is negligible, and every rollout that starts the detour looks worse than standing still. DWA does not escape it either, and neither of them is broken. **A local controller optimizes; it does not search.** Give the same MPPI a reference path around the counter and it follows it out in 13.6 seconds. That is the layered architecture the whole of Part VI has been building toward, and the reason Chapter 20 exists.

23.5.3 The small end of a real continuum

It would be easy to read this chapter as a toy. It is not. The algorithm above is what Williams et al. ran on a fifth-scale rally car sliding around a dirt track: about 1 200 rollouts of 2.5 seconds at 40 Hz, roughly 4.8 million queries to the full nonlinear vehicle dynamics every second, on a GPU, over more than 100 km of autonomous driving. The same update law, with a stack of about a dozen pluggable cost critics, ships as a stock controller in ROS 2's Nav2 for exactly the kind of differential-drive robot Rusty is. The reader's rayon loop is the small end of that continuum, not a different thing.

And there is a thread left deliberately loose. Every cost in this chapter has

been about where the robot should *be*. Nothing stops a term in $c(x_k, u_k)$ from being about what the robot would *learn* — the expected entropy reduction of the map, say. Put that term in the cost and the same sampler that dodges chairs starts choosing where to look. Chapter 24 makes goals out of information.

23.6 Exercises

1 FOUNDATION •• Derive the cross term, then delete it

Starting from $w(V) \propto e^{-S(V)/\lambda} p(V)/q(V \mid \hat{U}, \Sigma_u)$ with $p = q(\cdot \mid \alpha \hat{U}, \Sigma_u)$, carry out the two-quadratic expansion in Step 3 of the twin derivation and confirm that the sample-dependent part of the log-ratio is exactly $-\sum_k (1 - \alpha) \hat{u}_k^\top \Sigma_u^{-1} \epsilon_k$. Then explain, using the free-energy bound rather than intuition, what objective you are optimizing when you set $\gamma = 0$: what replaces the effort penalty, and why that makes the resulting motion smoother rather than faster.

2 FOUNDATION •• Both limits, and the gauge between them

Prove the two limits of the previous derivation carefully: that $\lambda \rightarrow 0^+$ gives best-of- K (state the tie-breaking assumption you need) and that $\lambda \rightarrow \infty$ gives an update of size $O(\sigma/\sqrt{K})$. Then show that $\text{ESS} = 1$ and $\text{ESS} = K$ are attained exactly in those limits, and use that to explain why the measured table above has its *shortest* time-to-goal in the middle rather than at either end.

3 FOUNDATION ••• Why the mean, not the mode

Step 1 of the twin derivation projects q^* onto a Gaussian by minimizing $\text{KL}(q^* \| q(\cdot \mid U, \Sigma_u))$ and lands on moment matching. Do the algebra. Then argue what would change if you minimized the *reverse* KL instead, $\text{KL}(q(\cdot \mid U, \Sigma_u) \| q^*)$, and relate your answer to what happens in w23.1 as $\lambda \rightarrow 0$. Which of the two divergences describes a controller that commits to one option in a bimodal cost landscape, and which describes one that steers between them into the obstacle?

4 CONCEPTUAL • Predict, then check: starve the proposal

In w23.1, set the exploration σ_v to roughly a quarter of its default (0.06 m/s) and leave everything else alone. Before you press play, predict: does Rusty still find the gap beside the first chair, and does the ESS go *up* or *down*? Run it. Explain what you see in the proposal-coverage language of Chapter 8 — and note which of the two possible ESS answers is the *bad* one here, which is the opposite of the filtering case.

5 CONCEPTUAL •• Try to make DWA safe

In w23.2, run the corridor with DWA and note the minimum clearance. Predict what raising the clearance weight w_c from 0.02 to 0.5 will do, then do it — and explain the result from the definition of $\text{dist}(v, \omega)$ rather than from the plot. Now switch on minimum-margin clearance and find the largest w_c that still reaches the goal, and the smallest that freezes the robot. Report both, and say why there is no useful value in between: what shape of trajectory would be needed, and why is it not in DWA's hypothesis class? Finally, name the property of $\exp(-d/\sigma_o)$ that a normalized linear term lacks.

6 PRACTICAL •• Colored noise

White perturbations spend most of their probability mass on plans that reverse direction every step — plans no robot would consider. Implement temporally correlated sampling in `mppi.rs`: draw $\epsilon_k = \beta \epsilon_{k-1} + \sqrt{1 - \beta^2} \xi_k$ with $\xi_k \sim \mathcal{N}(0, \Sigma_u)$, so the marginal variance is unchanged. Measure, on the corridor run over twelve seeds, the mean $|\Delta u|$, the minimum clearance, and the time to goal as a function of $\beta \in \{0, 0.3, 0.6, 0.9\}$. At which β does the controller stop being able to react to the moving obstacle, and why?

7 PRACTICAL ••• A gradient rival on the same traits

Implement one iLQR iteration against the same `Dynamics` and `CostFn` traits, adding only the two Jacobian methods it needs. Race it against MPPI on (a) the smooth-ESDF corridor and (b) a version whose obstacle cost is a raw occupancy-grid lookup with no smoothing. Report time to goal, minimum clearance, and iterations to convergence for both, and reproduce the top two rows of this chapter's scorecard with your own numbers. If iLQR fails on (b), say precisely at which line of your implementation it fails.

23.7 References

Fox, D., Burgard, W., and Thrun, S. (1997). The Dynamic Window Approach to Collision Avoidance *IEEE Robotics & Automation Magazine* 4(1), 23–33.

The classical baseline of this chapter, from the same authors as the book's spine. Section III is the source of the admissibility inequality and the three-term objective; the RHINO experiments ran at up to 95 cm/s in populated corridors. doi:10.1109/100.580977

Theodorou, E., Buchli, J., and Schaal, S. (2010). A Generalized Path Integral Control Approach to Reinforcement Learning *Journal of Machine Learning Research* 11, 3137–3181. Where the exponentiated-cost weighting entered robotics, as PI². Read it for the continuous-time path-integral derivation this chapter deliberately replaces with the discrete-time information-theoretic one. [link](#)

Williams, G., Aldrich, A., and Theodorou, E. A. (2017). Model Predictive Path Integral Control: From Theory to Parallel Computation *Journal of Guidance, Control, and Dynamics* 40(2), 344–357.

The paper that named MPPI and made the case for GPU rollouts. Its parallel-computation analysis is the reason the Rust here draws noise serially and parallelizes only the simulations. doi:10.2514/1.G001921

Williams, G., Drews, P., Goldfain, B., Rehg, J. M., and Theodorou, E. A. (2018). Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving *IEEE Transactions on Robotics* 34(6), 1603–1622.

This chapter’s Foundation section follows its §III exactly: the free-energy bound, the optimal distribution, the importance-sampling weight with its cross term, and Algorithms 1–2. The preprint is arXiv:1707.02342, and the epigraph is from its §III-A. doi:10.1109/TR0.2018.2865891

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., and Ferguson, M. (2023). From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2 *Robotics and Autonomous Systems* 168, 104493.

Deployment evidence for the claim that MPPI is now a default rather than a curiosity: written by the Nav2 maintainers, it surveys the controller stack that ships MPPI alongside DWB, the direct descendant of the 1997 paper above. doi:10.1016/j.robot.2023.104493

Kazim, M., Hong, J., Kim, M.-G., and Kim, K.-K. K. (2024). Recent Advances in Path Integral Control for Trajectory Optimization: An Overview in Theoretical and Algorithmic Perspectives *Annual Reviews in Control* 57, 100931.

The map of the field since 2018 — cross-entropy variants, covariance adaptation, smoothing schemes — and the best single source for what to read next after this chapter. doi:10.1016/j.arcontrol.2023.100931

Trevisan, E. and Alonso-Mora, J. (2024). Biased-MPPI: Informing Sampling-Based Model Predictive Control by Fusing Ancillary Controllers *IEEE Robotics and Automation Letters* 9(6), 5871–5878.

The modern answer to this chapter’s counter-pocket failure: keep the proposal, but seed it with samples from controllers that already know the way out. The importance weights stay valid, which is exactly the point of deriving them properly. doi:10.1109/LRA.2024.3397083

Homburger, H., Messerer, F., Diehl, M., and Reuter, J. (2025). Optimality and Suboptimality of MPPI Control in Stochastic and Deterministic Settings *IEEE Control Systems Letters*.

An honest accounting of what MPPI actually returns: the suboptimality of the sampled solution grows second-order in the noise scaling for smooth unconstrained problems. Read it before promising anyone that MPPI is optimal. doi:10.1109/LCSYS.2025.3574151

Exploration and Active SLAM

PLANNING AND ACTING UNDER UNCERTAINTY · Advanced · 60 min

“

The central question in exploration is: Given what you know about the world, where should you move to gain as much new information as possible?

Brian Yamauchi

A Frontier-Based Approach for Autonomous Exploration (1997), §2

Every chapter until now handed Rusty a goal. Localize here. Map this. Drive there. This chapter asks the question autonomy actually begins with: *where should the robot go in order to learn?*

That question turns the machinery of Parts II through V inside out. The belief has been the **output** of everything we have built; from here it is the **objective**. The score a robot maximizes is a property of its own uncertainty, and the actions available to it are moves that change what it will see next. This is the same structure as Chapter 22’s POMDP — because it *is* a POMDP, and an intractable one. What the field does instead is a three-stage pipeline that Placed et al. named in their 2023 survey: **identify** the candidate actions, **select** one by a utility, **execute** it while the estimator keeps running. This chapter builds all three, twice: once over the occupancy grid, where the objective is map entropy, and once over the pose graph, where the objective is the determinant of the information matrix you have been assembling since Chapter 15.

The reward for getting there is the book’s pre-capstone: a robot dropped into a floorplan it has never seen, which maps the whole thing by itself and stops when stopping is the right call.

◎ AFTER THIS CHAPTER YOU CAN

- Write down the entropy of a log-odds occupancy map, and read it as a score the robot is trying to drive down
- Derive expected information gain as a mutual information, and prove it

is never negative — while a single measurement still can be

- Compute the expected gain of a sensing pose by ray-casting through the map you already have, with the reach probability that makes occlusion honest
- Define frontiers, and prove that chasing them until none remain maps everything reachable
- Choose motions that sharpen the pose belief, not just the map — active localization as a depth-1 POMDP backup
- Score candidate trajectories by what they do to the pose graph: log det Ω , D-optimality, and why odometry alone changes neither
- Build the identify–select–execute loop in Rust and run a robot that maps an apartment it has never seen

ASSUMED

- Entropy and mutual information (Chapter 2)
- The log-odds binary Bayes filter and occupancy grids (Chapters 8 and 13)
- Monte Carlo localization and multi-modal beliefs (Chapter 12)
- The pose graph and its information matrix Ω (Chapters 15 and 16)
- Grid planning and trajectory tracking (Chapters 20 and 23), used here as services
- The POMDP frame and why exact belief-space planning is intractable (Chapter 22)

24.1 The lawnmower fallacy

Here is the obvious way to map a room you have never seen: drive a boustrophedon sweep. Lanes up, lanes down, like mowing a lawn. Coverage path planning is a solved problem with a beautiful theory behind it (Choset’s cellular decompositions), and if you already have the floorplan it is exactly what you should do.

Rusty does not have the floorplan. That is the entire point. So the lanes run into walls the script did not know about, they cross rooms that were finished three lanes ago, and every metre of re-scanned corridor is a metre of odometry drift bought for nothing. The widget below will run it for you: 93 metres of lawn-mowing leaves 4% of the apartment unmapped, and it does not know that, because a script has no notion of what it has already learned.

 INTERACTIVE · w24.1**Frontier Chaser**

Exploration is not “go to the nearest unknown”. Nearest is one setting of a utility, and usually the wrong one.

Run this simulation in the web edition: </chapters/ch24-exploration #w24.1>

Switch it to **Utility** and watch what changes. The robot now has an opinion. Every few seconds it stops, looks at its own map, finds the places where mapped free space touches the unknown, asks how many bits a scan from each of them would be worth and how many metres each would cost, and drives to the argmax. Over ten seeded runs it clears 4000 of the map’s 4800 bits in a median of **33.6 metres**, against 46.0 for the lawnmower.

Three things in that sentence are doing real work, and the rest of the chapter is about them.

"How many bits." Not "how much unknown area" — bits, of the entropy of the same log-odds grid Chapter 13 built. The objective was already there; nobody had asked the map what it was worth.

"Would be worth." The robot has to score a measurement it has not taken. That is an expectation over a measurement it does not have, evaluated through the map it does have — and the map is wrong. Every honest word about exploration lives in that sentence.

"And how many metres each would cost." Information alone is not a policy. A robot that maximizes bits without pricing motion will happily cross the apartment for one extra doorway. The weights that trade the two are the difference between a good explorer and a busy one.

 THE ONE SENTENCE VERSION

Exploration is a Bayes filter run forwards: instead of asking what the measurement did to the belief, ask what a measurement *would* do, average over the measurements you might get, and drive to wherever that number is largest per unit of effort.

24.2 Building intuition

Before any algebra, three observations you can make from the widget above.

Nearest is not a different algorithm; it is one setting of a knob. Push w_C — the cost weight — above about 0.9 and the utility policy stops crossing the apartment and starts creeping to whatever boundary is closest. It has not switched strategies; the information term has simply been priced out of the argmax. Yamauchi’s original frontier explorer is the $w_I \rightarrow 0$ corner of the same

family, and so is every "go to the nearest unexplored cell" implementation shipped since.

Greed is efficient per metre right up to the point where it stalls. Select **Nearest frontier** and watch the distance readout. It stops moving. The robot is not frozen: with $w_I = 0$ the utility is $-C(a)$, and the cheapest frontier to reach is the one directly under the wheels, which costs nothing. So Rusty stands there re-scanning until the map pushes the boundary far enough away to be worth walking to. Across ten seeds that policy is the *most* efficient of the three per metre travelled — 83 bits/m against the utility policy's 77 — and it reaches the 4000-bit mark in two runs out of ten. Efficiency per metre is the wrong statistic when the metres stop happening.

The score falls in steps, not smoothly. The bits-per-metre sparkline spikes every time Rusty crosses a threshold and a whole room resolves at once, then decays as the room fills in. Exploration is not a gradient descent on entropy; it is a sequence of decisions to go somewhere and be surprised. That shape is what makes stopping hard, and it is the last section of this chapter.

24.3 The mathematics

24.3.1 Entropy, from the map you already have

Chapter 13 stored the map as an array of log odds $\ell_i = \log \frac{p(m_i=1)}{p(m_i=0)}$, one per cell, and recovered probabilities with the logistic

$$p_i = 1 - \frac{1}{1 + e^{\ell_i}}.$$

Each cell is a Bernoulli variable, so its uncertainty is the binary entropy

$$H_b(p) = -p \log_2 p - (1 - p) \log_2 (1 - p) \quad \text{bits},$$

which is 1 at $p = \frac{1}{2}$ (a cell you know nothing about) and 0 at $p \in \{0, 1\}$. Under Chapter 13's standing approximation that cells are independent given the poses, the entropy of the whole map is a sum:

$$H(m) = \sum_i H_b(p_i).$$

That is the objective. It has units, it is computable in one pass over the array, and it starts at exactly the number of cells: the apartment at 15 cm resolution is $80 \times 60 = 4800$ cells, so a virgin map holds **4800 bits of ignorance**. Everything Rusty does for the rest of this chapter is an attempt to spend metres to remove bits.

⚠ WHERE ROBOTS BREAK THIS

Per-cell independence is a lie, and it is Chapter 13's lie, inherited here without repair. Real walls are correlated: knowing one cell of a wall is occupied tells you a great deal about the next one along. The sum above therefore *over-counts* the ignorance in structured environments, and the expected gains computed from it are systematically optimistic. We keep the approximation because the whole pipeline is built on it and because the ranking of candidate actions survives it better than the absolute numbers do — but a gain of 40 bits is not a promise of 40 bits.

$H(b)$	=	Entropy of a belief, in bits. For a pose belief this is over MCL particles or grid cells; for the map it is the per-cell sum above.
$-\sum_x b(x) \log_2 b(x)$		
$a \in \mathcal{A}$		A candidate action: a sensing pose, or a whole trajectory. $\mathcal{A}$ is the candidate set produced by the identify stage.
$I(a) = H(b) - \mathbb{E}_{z_a}[H(b z_a)]$		Expected information gain of a — the mutual information between what we do not know and what a would tell us.
$C(a)$		Cost of a : path length through known-free space, or time, or energy. Never Euclidean distance.
$U(a) = w_I I(a) + w_G \Delta_a - w_C C(a)$		Utility. The weights carry units: bits, nats, and metres are not interchangeable until you say what they are worth.
Ω		The pose graph's information matrix from Chapter 15 — from here on, a decision variable.
$\text{Dopt}(\Omega)$	=	D-optimality, with $n = \dim \Omega$: the geometric mean of the eigenvalues of Ω , and the criterion that makes graphs of different sizes comparable.
$\exp(\frac{1}{n} \log \det \Omega)$		

NOTE

The weights are w_I , w_G , w_C — never α or β . Those two are spoken for: $\alpha_{1\dots 6}$ are the motion-noise parameters of Chapter 9, and α -vectors are the value-function representation of Chapter 22.

24.3.2 Expected information gain

Fix a candidate action a — say, "drive to that doorway and take one scan." Before acting, the robot's uncertainty about the map is $H(m)$. Afterwards it will be $H(m | z_a = z)$ for whichever reading z arrives. The robot does not have z . What it has is a distribution over it, so the honest thing to score is the average:

$$I(a) = H(m) - \mathbb{E}_{z \sim p(z_a)}[H(m | z_a = z)] = I(m; z_a).$$

The right-hand equality is the definition of mutual information, and it is the whole reason this quantity behaves. Two consequences follow immediately, and they are the two facts every practitioner needs.

DERIVATION Sensing never hurts in expectation — but a measurement can

$$I(a) = \text{KL}(p(m, z_a) \| p(m)p(z_a)) \geq 0$$

Step 1 — rewrite the gain as a mutual information. The *conditional entropy* $H(m | z_a)$ is by definition the average of $H(m | z_a = z)$ over z , so the second term is already exactly it:

$$I(a) = H(m) - \mathbb{E}_{z \sim p(z_a)} [H(m | z_a = z)] = H(m) - H(m | z_a) = I(m; z_a).$$

Confusing $H(m | z_a = z)$ — one realized reading — with $H(m | z_a)$ — the average over all of them — is where the sign confusion in Step 4 comes from, so it is worth writing both out once.

Step 2 — recognize the KL divergence. Writing out the definitions,

$$I(m; z_a) = \sum_{m,z} p(m, z) \log_2 \frac{p(m, z)}{p(m)p(z)} = \text{KL}(p(m, z) \| p(m)p(z)).$$

Step 3 — apply Gibbs' inequality. A KL divergence is non-negative, with equality if and only if its two arguments are equal — that is, if and only if the measurement is independent of the map. Therefore

$$I(a) \geq 0, \quad \text{with } I(a) = 0 \iff z_a \text{ tells you nothing about } m.$$

Pointing a range finder at a wall you have already mapped scores exactly zero, and this is the formal reason why.

Step 4 — the counterexample that matters. *Expected* entropy cannot rise. *Realized* entropy absolutely can, and you can compute the case by hand. Take one cell you believe is occupied, $p = 0.9$, so $H_b(0.9) = 0.4690$ bits. The sensor is right with probability $q = 0.9$. If it reports **free**, Bayes' rule gives

$$p' = \frac{p(1-q)}{p(1-q) + (1-p)q} = \frac{0.09}{0.09 + 0.09} = 0.5, \quad H_b(0.5) = 1 \text{ bit.}$$

The entropy of that cell went *up* by 0.531 bits. It happens with probability $p(1-q) + (1-p)q = 0.18$. The other 82% of the time the reading is **occupied**, the posterior is $0.81/0.82 = 0.9878$, and the entropy falls to 0.0950 bits. Averaging:

$$\mathbb{E}_z[H_b] = 0.18 \cdot 1 + 0.82 \cdot 0.0950 = 0.2579, \quad I = 0.4690 - 0.2579 = \mathbf{0.2111 \text{ bits}} > 0.$$

So: the theorem holds, and your log will still show the map entropy jumping upward on individual scans. That is a *surprising measurement*, not a bug — the same event that makes the evidence term η^{-1} small in Chapter 5's recursion. Do not add a clamp.

24.3.3 The gain of a sensing pose, beam by beam

The mutual information above is defined; computing it for a real LiDAR pose is the work. Two factorizations make it cheap enough to evaluate for a dozen candidates several times a second.

Over cells. Chapter 13 already assumes the cells are independent, so the map entropy is a sum and the gain is a sum of per-cell gains. For one cell with prior p , observed through a binary symmetric channel that reports correctly with probability q , the gain is closed form:

$$I(m_i; z_i) = H_b(p) - \left[P(\hat{z}=1) H_b(p \mid \hat{z}=1) + P(\hat{z}=0) H_b(p \mid \hat{z}=0) \right].$$

Two limits are worth carrying in your head. At $p = \frac{1}{2}$ this collapses to the channel capacity $1 - H_b(q)$ — for $q = 0.9$ that is 0.5310 bits, and no scan of an unknown cell can ever be worth more. And as $p \rightarrow 0$ or $p \rightarrow 1$ it goes to zero: a cell you already know says nothing back.

Along a beam. Cells on a ray are not reached independently. A beam arrives at the k -th cell only if every cell before it was empty, which gives the **reach probability**

$$R_k = \prod_{j < k} (1 - p_j),$$

a running product computed as the ray is traced. The expected gain of one beam is then each cell's gain, discounted by the chance the beam gets there:

$$I(\text{beam}) = \sum_k R_k \cdot I(m_k; z_k).$$

DERIVATION Beam-wise expected gain over an occupancy grid

$$I(a) \approx \sum_{\text{beams}} \sum_{k \in \text{ray}} R_k I(m_k; z_k)$$

Step 1 — factor the map entropy over cells. By Chapter 13's independence approximation, $H(m) = \sum_i H_b(p_i)$ and, conditioned on a scan, $H(m \mid z) = \sum_i H_b(p_i \mid z)$. So the gain of a scan is the sum of the per-cell gains of the

cells the scan touches; cells outside the perceptual field contribute exactly zero, which is what makes this a local computation instead of a sweep over the whole map.

Step 2 — condition each cell on the beam getting there. A range finder is an *occluded* sensor. Cell k along a ray is measured only if the beam was not stopped earlier. Treating the cells' occupancies as independent (again), the probability of a clear path is the product $R_k = \prod_{j < k} (1 - p_j)$. If the beam is stopped before k , the scan says nothing about cell k and its contribution is zero. Hence the contribution of cell k is $R_k I(m_k; z_k)$.

Step 3 — model the per-cell measurement. We do not carry Chapter 10's four-component beam mixture into the planner. The planner needs one number from the sensor: how often the inverse model gets a cell right. That is q , and the per-cell gain is the binary-symmetric-channel expression above. This is deliberately cruder than the forward model — a planner that is more expensive than the estimator it feeds is a planner nobody runs.

Step 4 — sum over beams, and admit the double count. Summing the per-beam gains treats adjacent beams as independent, which they are not: two beams through the same doorway largely determine each other. The crudest part of the error is fixed by a visited set — score each cell at most once per scan, exactly as `integrateScan` does — but what remains is an over-estimate. Note also that the recursion runs through the *current* map, so unknown cells are assumed to behave like their prior; where the truth is a wall, the estimator predicts the beam will fly straight on and over-values everything behind it.

Step 5 — truncate. Once R_k falls below about 10^{-3} the remaining terms cannot matter, and the loop stops. A beam already stopped by three half-occupied cells will not tell you about the fourth.

ALGORITHM `expected_info_gain(m, x, sensor)` COST 0(#beams · $z_{\max}$ / resolution)
per candidate

in the current log-odds map m , a candidate sensing pose x , the planner's sensor model (n beams, FOV, $z_{\max}$, q)

out expected map-entropy reduction in bits

-
- 1: $I \leftarrow 0$; $\mathcal{V} \leftarrow \emptyset$ (*the visited set*)
 - 2: for each beam bearing ϕ do
 - 3: $R \leftarrow 1$
 - 4: for each cell i on the Bresenham ray from x along ϕ out to $z_{\max}$ do
 - 5: if $R < R_{\min}$ then break

```

6:       $p \leftarrow 1 - 1/(1 + e^{\ell_i})$ 
7:      if  $i \notin \mathcal{V}$  then  $I \leftarrow I + R \cdot I(m_i; z_i)$ ;  $\mathcal{V} \leftarrow \mathcal{V} \cup \{i\}$ 
8:       $R \leftarrow R \cdot (1 - p)$  (occlusion is a property of the map, not of the
   bookkeeping)
9:    endfor
10: endfor
11: return  $I$ 

```

24.3.4 A worked example you can check by hand

Take one beam, five cells long, and two situations.

Into the unknown. Every cell sits at the prior, $p_k = 0.5$, and the sensor is right with $q = 0.9$. Then every cell has the same per-cell gain, the channel capacity:

$$I(m_k; z_k) = 1 - H_b(0.9) = 1 - 0.4690 = 0.5310 \text{ bits},$$

and the reach probability halves at every step, $R_k = (1/2)^{k-1}$. The total is a five-term geometric sum:

k	p_k	R_k	$I(m_k; z_k)$	$R_k \cdot I$
1	0.50	1.0000	0.5310	0.5310
2	0.50	0.5000	0.5310	0.2655
3	0.50	0.2500	0.5310	0.1328
4	0.50	0.1250	0.5310	0.0664
5	0.50	0.0625	0.5310	0.0332
total				1.0288

Or in closed form, $0.5310 \times (2 - 2^{-4}) = 0.5310 \times 1.9375 = 1.0288$ bits.

Into space you have already mapped. Two or three sweeps of Chapter 13's inverse model drive a free cell to about $p = 0.01$. Now the per-cell gain collapses to 0.02486 bits, but the beam sails through — $R_k = 0.99^{k-1}$, still 0.96 at the fifth cell — so the sum is

$$0.02486 \times \frac{1 - 0.99^5}{0.01} = 0.02486 \times 4.9010 = 0.1218 \text{ bits}.$$

Eight and a half times less, from the same sensor over the same five cells. That ratio is the entire content of "point the sensor at the unknown," and now it is a number rather than a slogan.

Notice that the mapped ray is not worth *zero*, and it never will be. A log-odds map with a clamp never becomes certain, so re-scanning known space always scores something, and an explorer with a badly tuned w_C will happily spend metres collecting it. The clamp that keeps Chapter 13's map revisable is the same clamp that keeps this number off the floor.

RUST crates/ch24_explore/src/info_gain.rs (tests)

```

1  #[test]
2  fn worked_example_ch24_five_cell_ray() {
3      // A fresh ray into the unknown:  $p = 0.5$ , sensor reliability  $q = 0.9$ .
4      let unknown = ray_info_gain(&[0.5; 5], 0.9, 1e-3);
5      assert_relative_eq!(unknown.cells[0].mutual_info, 1.0 - binary_entropy(0.9), epsilon
6      = 1e-12);
7      assert_relative_eq!(unknown.cells[3].reach, 0.125, epsilon = 1e-12);
8      assert_relative_eq!(unknown.total, 1.0288, epsilon = 1e-4);
9
10     // The same ray through space that two sweeps have already emptied.
11     let mapped = ray_info_gain(&[0.01; 5], 0.9, 1e-3);
12     assert_relative_eq!(mapped.total, 0.12183, epsilon = 1e-4);
13     assert!(unknown.total / mapped.total > 8.0);
14
15     // A cell that is already certain is worth nothing, whatever the sensor.
16     assert_relative_eq!(cell_mutual_information(0.0, 0.99), 0.0, epsilon = 1e-12);
17 }

```

24.3.5 Frontiers, and why chasing them is enough

Information gain tells you what a pose is worth. It does not tell you which poses to bother scoring, and scoring every free cell in an 80×60 grid at every replan is both wasteful and pointless — most of them are worth nothing.

Yamauchi's 1997 answer is still the right one. Define:

- A **frontier cell** is a known-free cell with at least one unknown 4-neighbour.
- A **frontier region** is a connected component of frontier cells, summarized by its centroid and its size in cells.

That is all. Frontier regions are the only places in the map where a sensor can convert ignorance into evidence, they are found in one pass plus a flood fill, and grouping them into regions matters more than it looks: scoring individual cells produces hundreds of near-identical candidates and a robot that dithers between two of them.

ALGORITHM detect_frontiers(m , min_size) COST $O(\#cells)$: one marking pass, one 8-connected BFS

in the log-odds map m , the smallest region worth driving to

out a list of frontier regions, each with cells, centroid, and a representative cell

- 1: for all cells i do
- 2: $\text{mark}[i] \leftarrow [\text{class}(i) = \text{free} \wedge \exists j \in N_4(i) : \text{class}(j) = \text{unknown}]$
- 3: endfor
- 4: $F \leftarrow \emptyset$

```

5: for all marked, unvisited cells  $i$  do
6:    $R \leftarrow$  8-connected BFS flood from  $i$  over marked cells
7:   if  $|R| \geq \text{min\_size}$  then append  $R$  with its centroid and nearest-to-
      centroid member to  $F$ 
8: endfor
9: return  $F$ 

```

The representative cell is not a detail. The centroid of a C-shaped frontier region can land inside a wall, and driving to a point inside a wall is not a plan; taking the member cell nearest the centroid guarantees a real, occupiable target.

DERIVATION Frontier exploration is complete

No reachable frontier the reachable closure is mapped

Claim. With an ideal sensor and a planner that is complete over the currently known-free space, repeatedly navigating to any reachable frontier until none remain maps every cell reachable from the start.

Step 1 — the frontier separates. Let K be the set of known-free cells and U the unknown cells. Any 4-connected path from the robot's cell into U must contain a first cell in U ; its predecessor is in K and is 4-adjacent to an unknown cell, so the predecessor is a frontier cell by definition. Every route from the known into the unknown crosses the frontier. There is no back door.

Step 2 — visiting a frontier strictly grows the known set. Stand on a frontier cell and take a scan. With an ideal sensor of positive range, at least one cell that was unknown — the unknown neighbour that made this a frontier cell — is now classified. So $|K|$ strictly increases, or the cell stops being a frontier cell. Either way the number of unresolved cells strictly decreases.

Step 3 — monotone and bounded means terminating. The grid is finite, so a strictly decreasing count of unresolved cells cannot go on forever. The loop terminates.

Step 4 — termination means done. At termination there are no reachable frontiers. By Step 1, that means there is no path from the robot through known-free space into unknown space. Every cell reachable from the start has therefore been classified. ■

Where this fails on a real robot, and it always does. Every hypothesis in the claim is load-bearing. *Ideal sensor:* glass and dark felt return $z_{\max}$, so a window is permanently a frontier that never resolves and Rusty will visit it forever. *Complete planner:* inflate the obstacle map by one cell too many

and a 0.9 m doorway closes, which makes an entire wing "unreachable" — the map then reports completion at 60% coverage. *Positive-range guarantee*: a frontier cell in a corner may see nothing new at all if the beam that would resolve it grazes a wall. Each of these turns up in the widget above as a run that stops early with frontiers still drawn on the map, and the honest engineering answer is a blacklist of targets that failed to grow the known set, not a better theorem.

24.3.6 Utility: bits are not metres

With a candidate set from `detect_frontiers` and a score from `expected_info_gain`, the selection stage is one line:

$$a^\star = \arg \max_{a \in \mathcal{A}} w_I I(a) + w_G \Delta_a - w_C C(a).$$

Three things about that expression are worth stating plainly, because they are where deployed systems go wrong.

$C(a)$ is not Euclidean distance. A frontier three metres away through a wall is not three metres away. The cost has to come from a planner over the *known-free* cells — in the implementation below, a Dijkstra expansion from the robot's cell, which doubles as the reachability test the completeness argument needs. Using straight-line distance produces a robot that repeatedly chooses a target it cannot reach.

The weights carry units, and pretending otherwise is how you get a magic constant. w_I prices bits, w_C prices metres, w_G prices nats of graph information. There is no principled universal setting; there is a *mission* that says what a bit is worth in metres. Saying so out loud is better than hiding a 0.35 in a config file.

Every classical explorer is a corner of this expression. Set $w_I = w_G = 0$ and you have nearest-frontier. Set $w_C = w_G = 0$ and you have greedy maximum-information, which crosses the building for a big room. Set $w_C = 0$ and keep both others and you have the 2005 textbook's information-gain explorer. The interesting question was never which of these is right; it is what w_G is for, and that is the next section.

24.4 Active localization: motion as a sensing action

Everything so far treated the pose as known. It is not, and the coupling runs both ways: a robot that is unsure *where it is* cannot integrate a scan into the right cells, so pose uncertainty corrupts the map that the exploration objective is defined on.

The narrower problem — choose motions that sharpen the *pose* belief — is **active localization**, and Fox, Burgard and Thrun formalized it in 1998 with a rule

that is one line of decision theory:

$$a^\star = \arg \max_a \left[H(\text{bel}) - \mathbb{E}_{z_a} [H(\text{bel}')] \right] - w_C C(a),$$

where bel' is the belief after pushing bel through the motion model for a and then correcting with a hypothetical z_a . Both halves of the Bayes filter appear, run forwards, on a measurement that has not happened.

INTERACTIVE · w24.2

Disambiguation Detour

Motion is a sensing action. When the belief is ambiguous, the informative route can be the fast route.

Run this simulation in the web edition: [/chapters/ch24-exploration #w24.2](/chapters/ch24-exploration/#w24.2)

The corridor in that widget is built to make one point unavoidable. Its doors repeat every 2.5 m, and every candidate motion except one is a whole number of door spacings — so each of them lands *both* hypotheses at the same offset inside a feature, and each of them scores an expected gain of exactly 0.000 bits. The corridor ahead cannot tell those two futures apart, and the bars say so without being told. Only the alcove, 3.75 m *behind*, breaks the symmetry.

Over two hundred seeded trials at the default noise, the entropy-greedy policy takes the detour and ends in the right place 95% of the time. The goal-greedy policy heads for the goal, arrives confidently, and is in the wrong place almost exactly half the time — 48%, which is what a coin flip looks like when a filter has no evidence and still has to produce a maximum. Nothing in that filter ever complains, because nothing it saw was surprising.

DERIVATION Active localization is a depth-1 POMDP backup

$$a^\star = \arg \max_a [H(\text{bel}) - \mathbb{E}_{z_a} [H(\text{bel}')]] - w_C C(a)$$

Step 1 — push the belief through the motion model. For each candidate a with displacement δ_a , form the predicted belief exactly as Chapter 5's line 2 does:

$$\overline{\text{bel}}(x) = \int p(x \mid x', a) \text{bel}(x') dx'.$$

Prediction is a convolution, so $H(\overline{\text{bel}}) \geq H(\text{bel})$: the motion itself always costs information. This is why "hold still and scan" is a candidate worth including — and why a pure information objective, left alone, produces a robot that never goes anywhere.

Step 2 — enumerate the measurements you might get. For each possible reading z , its probability under the predicted belief is the *evidence* term of the Bayes filter, used as a forecast rather than as a normalizer after the fact:

$$p(z | a) = \int p(z | x) \overline{\text{bel}}(x) dx.$$

With a discrete belief and a finite outcome set — a door detector with three answers, say — this sum is exact and no sampling is needed, which is why the widget’s numbers can be checked by hand. With a LiDAR you sample z from the simulator of Chapter 4 instead, at a few dozen draws per candidate.

Step 3 — average the posterior entropies. For each z , correct the predicted belief and measure what is left:

$$\mathbb{E}_z[H(\text{bel}')] = \sum_z p(z | a) H(\eta p(z | \cdot) \overline{\text{bel}}(\cdot)).$$

By the same KL argument as before, $H(\overline{\text{bel}}) - \mathbb{E}_z[H(\text{bel}')] = I(x; z | a) \geq 0$. Note *which* entropy this is non-negative relative to: the **predicted** one. The action’s net value, $H(\text{bel}) - \mathbb{E}_z[H(\text{bel}')]$, can be and often is negative, because the motion smeared the belief faster than the scan could sharpen it. Those are two different numbers and confusing them is the most common bug in a first implementation.

Step 4 — recognize what has just been computed. This is exactly one step of a POMDP value backup with the value function replaced by negative entropy and the horizon truncated at one. It inherits Chapter 22’s honest caveat: entropy is a property of the belief, not of the state, so an AMDP-style approximation that plans on the belief’s *mean* cannot express this objective at all. And truncating at depth 1 has a real cost — a symmetric corridor whose only distinguishing feature lies two decisions away defeats any one-step lookahead, because the first step toward it scores zero. The last exercise in this chapter replaces the argmax with a POMCP search and measures when the lookahead earns its compute.

ALGORITHM `active_localize(bel, A, world, n_z)` COST $O(|A| \cdot M \cdot n_z)$ for M particles -- trivially parallel over candidates

in a belief over poses (particles or a grid), candidate motions A , a generative sensor model, n_z simulated measurements per candidate

out the chosen action, plus the per-candidate expected gains

```

1:  $H_0 \leftarrow H(\text{bel})$ 
2: for all  $a \in \mathcal{A}$  do
3:    $\overline{\text{bel}} \leftarrow \text{predict}(\text{bel}, a)$ 
4:    $\bar{H} \leftarrow 0$ 
5:   for each candidate measurement  $z$  (enumerated, or sampled from
      $\overline{\text{bel}}$ ) do
6:      $\text{bel}' \leftarrow \eta p(z \mid \cdot) \overline{\text{bel}}(\cdot)$ 
7:      $\bar{H} \leftarrow \bar{H} + p(z \mid a) H(\text{bel}')$ 
8:   endfor
9:    $U(a) \leftarrow w_I (H_0 - \bar{H}) - w_C C(a) - w_T T(a)$  ( $T$  is the task cost still owed)
10: endfor
11: return  $\arg \max_a U(a)$ , and the table of  $U$  for the reader

```

The $w_T T(a)$ term is not decoration. Without a task cost, the argmax of expected entropy reduction is a robot that finds the most informative corner of the building and stays there. In the widget, $T(a)$ is the distance still owed to the goal under the MAP hypothesis, and it is what makes Rusty leave the alcove once the belief has collapsed.

24.5 Active SLAM: the information matrix becomes a decision variable

Map entropy scores what the robot will *see*. It says nothing about whether the robot will know *where it was standing* when it saw it — and the map is only as trustworthy as the trajectory it was painted along. Chapter 16's rubber-band problem is what an un-closed loop looks like: a corridor that arrives back at its own start two metres off, and a map that is locally crisp and globally wrong.

So the second objective is the pose graph itself. Chapter 15 built the information matrix $\Omega = \sum_e A_e^\top \Omega_e A_e$ and used it to solve for the MAP trajectory. Its inverse is (to first order) the covariance of that trajectory, so a *small* Ω^{-1} — equivalently, a *large* Ω — is a well-determined map. "Large" needs a definition, and optimal experiment design has supplied three for seventy years.

💡 A-, D-, AND E-OPTIMALITY

Let $\lambda_1 \leq \dots \leq \lambda_n$ be the eigenvalues of Ω . The covariance ellipsoid has semi-axes $\lambda_k^{-1/2}$.

A-optimality minimizes $\text{tr}(\Omega^{-1}) = \sum_k \lambda_k^{-1}$ — the *sum of the squared semi-axes*, or equivalently the harmonic mean of Ω 's eigenvalues. It needs an inverse, so implementations often substitute $\text{tr } \Omega / n$ as a cheap stand-in, and neither

form is invariant to rescaling the state: metres and radians end up being added together.

D-optimality maximizes $(\prod_k \lambda_k)^{1/n} = \exp(\frac{1}{n} \log \det \Omega)$ — the reciprocal *volume* of the uncertainty ellipsoid, in geometric-mean form. Monotone, invariant under reparameterization, and computable from a factorization you already have. This is where the field converged.

E-optimality maximizes λ_1 — the *worst* direction. The conservative choice, and the right one when a single unconstrained direction is what will kill you.

The $1/n$ in D-optimality is not cosmetic. Raw $\log \det \Omega$ grows with the number of nodes, so it ranks a longer trajectory above a better-constrained one purely for being longer.

Because $\log \det$ falls straight out of a Cholesky factorization — $\log \det \Omega = 2 \sum_k \log L_{kk}$ — evaluating it for a candidate costs one sparse solve, roughly $O(n^{1.5})$ for a planar graph. That is what makes scoring a dozen candidate trajectories per decision affordable at all.

INTERACTIVE · w24.3

Loop or Push On?

A good exploration policy does not maximize coverage. Without deliberate loop closing, the map you cover is a map you cannot trust.

Run this simulation in the web edition: </chapters/ch24-exploration #w24.3>

That widget hides a theorem, and the theorem is the reason active SLAM exists as a separate subject from exploration.

DERIVATION Odometry cannot change D-optimality

$$\det \Omega_{\text{tree}} = \prod_e \det \Omega_e$$

Claim. If the pose graph is a **tree** — one gauge-fixed root, and every other node reached by exactly one relative-pose factor — then

$$\det \Omega = \prod_e \det \Omega_e,$$

independent of the geometry of the trajectory. Consequently $\text{Dopt}(\Omega)$ is *constant* along any odometry-only path whose factors share the same precision, no matter how far the robot drives or where it goes.

Step 1 — write Ω as a Gram matrix. Chapter 15 assembles $\Omega =$

$A^T \bar{\Omega} A$, where $A \in \mathbb{R}^{3m \times 3n}$ stacks the per-factor Jacobians and $\bar{\Omega} = \text{diag}(\Omega_{e_1}, \dots, \Omega_{e_m})$ is block diagonal. For a relative-pose factor between i and j , Chapter 16 derived $\partial e / \partial \delta_i = -\text{Ad}_{Z^{-1}}$ and $\partial e / \partial \delta_j = \mathbf{I}$.

Step 2 — a tree makes A square. With n free nodes and one factor into each, $m = n$, so A is $3n \times 3n$.

Step 3 — order the nodes and read off the determinant. Number the free nodes so that every node's parent precedes it (a BFS order from the gauge root). Then row-block k of A has $\mathbf{I}$ in column-block k and $-\text{Ad}_{Z_k^{-1}}$ in the column-block of its parent, which is *earlier*. So A is block lower triangular with identity blocks on the diagonal, and $\det A = 1$.

Step 4 — conclude.

$$\det \Omega = \det(A^T) \det(\bar{\Omega}) \det(A) = \det \bar{\Omega} = \prod_e \det \Omega_e.$$

Nothing about the trajectory survives. The adjoint Ad_T of a rigid motion has determinant 1, so even the individual blocks cannot smuggle geometry into the answer. ■

And now the consequence. $\text{Dopt} = \exp(\frac{1}{3n} \sum_e \log \det \Omega_e)$; with identical factors this is $(\det \Omega_e)^{1/3}$, a constant. **A robot that only drives learns nothing about its own trajectory that it did not already know.** Every metre adds one variable and one constraint, and the two cancel exactly.

What does change it. Add one factor that closes a cycle. Now $m = n + 1$, A is no longer square, and the matrix determinant lemma gives the increase in closed form:

$$\log \det(\Omega + A_c^T \Omega_c A_c) - \log \det \Omega = \log \det(\mathbf{I} + \Omega_c A_c \Omega^{-1} A_c^T) > 0,$$

which is large exactly when $A_c \Omega^{-1} A_c^T$ — the prior uncertainty of the *relative* pose being measured — is large. In words: a loop closure is worth the most when it constrains two poses whose relative geometry you were least sure of. Closing a loop against the node you visited ten steps ago is nearly worthless; closing it against the node you left twenty metres of corridor ago is the whole game.

Numbers, so you can check it. Take a chain with $\sigma_x = \sigma_y = 0.05$ m and $\sigma_\theta = 0.03$ rad, giving $\log \det \Omega_e = 2 \log(1/0.05^2) + \log(1/0.03^2) = 18.996$ nats per factor:

graph	free nodes	factors	$\log \det \Omega$	$\text{Dopt}(\Omega)$
2-step chain	2	2	37.99	562.29
5-step chain	5	5	94.98	562.29

graph	free nodes	factors	$\log \det \Omega$	$\text{Dopt}(\Omega)$
12-step chain	12	12	227.95	562.29
12-step chain + 1 loop closure	12	13	236.63	715.51

Three chains of wildly different length, one D-optimality. Add a single loop-closure factor with $\sigma_\theta = 0.02$ and $\log \det$ jumps by 8.68 nats. Note also that $\text{Dopt} = (\det \Omega_e)^{1/3} = (400 \cdot 400 \cdot 1111.1)^{1/3} = 562.29$, which you can do on a calculator.

RUST crates/ch24_explore/src/utility.rs (tests)

```

1  #[test]
2  fn worked_example_ch24_tree_logdet_is_geometry_free() {
3      let omega = information_from_sigmas(0.05, 0.05, 0.03);
4      let per_edge = omega.determinant().ln(); // 18.996 nats
5      let mut rng = SmallRng::seed_from_u64(24);
6
7      // Three chains with completely different shapes.
8      for n in [2usize, 5, 12] {
9          let g = random_odometry_chain(n, &omega, &mut rng);
10         assert_relative_eq!(graph_log_det(&g), n as f64 * per_edge, epsilon = 1e-9);
11         // ...and therefore the same D-optimality, whatever the shape.
12         assert_relative_eq!(d_optimality(&g), omega.determinant().cbrt(), epsilon = 1e-9);
13     }
14
15     // One loop closure, and only then does the criterion move.
16     let mut g = random_odometry_chain(12, &omega, &mut rng);
17     let before = graph_log_det(&g);
18     g.add_edge(0, 12, g.relative_pose(0, 12), information_from_sigmas(0.05, 0.05, 0.02),
19     Loop);
20     assert_relative_eq!(graph_log_det(&g) - before, 8.675, epsilon = 1e-3);
21 }

```

Which is exactly what the widget shows: pushing on into room E adds 646 nats of $\log \det$ and moves D-optimality by **zero**, while closing the loop adds 659 nats and moves D-optimality from 1314 to 1423. The raw determinant puts the two branches within 2% of each other because it is mostly counting nodes; the normalized criterion separates them cleanly. And the trajectory error — the number neither criterion is allowed to see — falls from 0.84 m to 0.34 m.

⚠ WHERE ROBOTS BREAK THIS

The tree identity is exact for the *linearized* information matrix with the Jacobians Chapter 16 uses. It is not a claim that dead reckoning is harmless: the trajectory *error* grows without bound along a chain, and the graph's covariance Ω^{-1} of any single node relative to the root grows with it. What is constant is the D-optimality of the whole joint distribution over increments

— which is precisely why a criterion that scores the whole graph needs the $1/n$, and why scoring the marginal covariance of the *latest* pose is a defensible alternative that some systems use instead.

24.6 When to stop

Every run in this chapter ends. None of them ends for a principled reason.

INTERACTIVE · w24.4

Stop Sign

“Explore until the map is finished” is a budget, not a plan: the last few percent of the entropy costs a third of the run.

Run this simulation in the web edition: [#w24.4](/chapters/ch24-exploration)

The candidates are all defensible and all flawed. An **absolute entropy threshold** (“stop below 200 bits”) depends on the size of the map, so it does not transfer between buildings. An **entropy plateau detector** needs a window length that nobody can justify, and exploration’s step-shaped progress means a genuine plateau and the pause before entering a new wing look identical. A **coverage percentage** requires knowing the area to be covered, which is what you were trying to find out.

The least-bad rule with units the mission can argue about is a **gain-per-cost floor**:

$$\text{stop when } \max_{a \in \mathcal{A}} \frac{I(a)}{C(a)} < \epsilon_{\text{task}},$$

because “a bit is worth a metre” is a statement about the job rather than about information theory. The chart above shows what it buys on one real run: at $\epsilon = 15$ b/m Rusty stops after 15 metres with barely half the information; at $\epsilon = 4$ it stops at 42 metres holding 93% of it; and the remaining 57 metres — more than half the entire run — buy the last 7%.

Placed et al. list principled stopping among active SLAM’s open problems, and this chapter has nothing better to offer. What it does offer is the discipline of plotting the curve before choosing the number.

24.7 Implementation in Rust

The crate is `ch24_explore`, and it is Part VI’s integrator: it depends on `ch13_occgrid` for the map, `ch16_slam2d` for the graph, `ch20_planning` for the paths, and `ch23_mppi` for the execution. It adds four things of its own: frontier detection,

the gain estimator, the utility, and the loop that ties them together.

24.7.1 Frontiers and the navigation field

RUST crates/ch24_explore/src/frontier.rs

```

1  use std::collections::HashMap;
2
3  use ch13_occgrid::{log_odds_to_prob, GridIdx, OccGrid};
4  use nalgebra::Point2;
5  use petgraph::unionfind::UnionFind;
6
7  #[derive(Clone, Copy, PartialEq, Eq, Debug)]
8  pub enum CellClass { Free, Occupied, Unknown }
9
10
11  /// Two thresholds, not one. The band in between is what "unknown" means, and
12  /// the width of that band is why an explorer with a timid free-space model
13  /// lags its own sensor by several scans.
14  #[derive(Clone, Copy)]
15  pub struct ClassThresholds { pub free_below: f64, pub occ_above: f64 }
16
17  impl Default for ClassThresholds {
18      fn default() -> Self { Self { free_below: 0.3, occ_above: 0.7 } }
19  }
20
21  pub struct Frontier {
22      pub cells: Vec<GridIdx>,
23      pub centroid: Point2<f64>,
24      /// The member cell nearest the centroid. The centroid of a C-shaped region
25      /// can sit inside a wall; a member cell never does.
26      pub representative: GridIdx,
27      pub size: usize,
28  }
29
30  /// `detect_frontiers` -- Yamauchi (1997), cell for cell.
31  ///
32  /// One marking pass, then union-find over 8-connected marked pairs. Union-find
33  /// rather than BFS because `petgraph` already has it, it is a single pass in
34  /// scan order, and it makes the "which region is this cell in" query that the
35  /// renderer wants  $O(\alpha(n))$ .
36  pub fn detect_frontiers(grid: &OccGrid, th: ClassThresholds, min_size: usize) -> Vec<
37      Frontier> {
38      let n = grid.len();
39      let mut marked = vec![false; n];
40
41      for (idx, &l) in grid.log_odds().iter().enumerate() {
42          if classify(log_odds_to_prob(l), th) != CellClass::Free { continue; }
43          // Free, and touching the unknown: that is the whole definition.
44          marked[idx] = grid.neighbors4(idx).any(|j| {
45              classify(log_odds_to_prob(grid.log_odds()[j]), th) == CellClass::Unknown
46          });
47      }
48
49      let mut uf = UnionFind::<usize>::new(n);
50      for idx in 0..n {
51          if !marked[idx] { continue; }
52          for j in grid.neighbors8(idx) {
53              if marked[j] { uf.union(idx, j); }
54          }
55      }
56  }

```



```

53     }
54
55     let mut regions: HashMap<usize, Vec<GridIdx>> = HashMap::new();
56     for idx in 0..n {
57         if marked[idx] { regions.entry(uf.find(idx)).or_default().push(grid.to_idx2(idx))
58     }; }
59
60     regions
61         .into_values()
62         .filter(|cells| cells.len() >= min_size)
63         .map(|cells| {
64             let centroid = mean_point(grid, &cells);
65             let representative = *cells
66                 .iter()
67                 .min_by(|a, b| {
68                     let da = (grid.center(**a) - centroid).norm_squared();
69                     let db = (grid.center(**b) - centroid).norm_squared();
70                     da.partial_cmp(&db).unwrap()
71                 })
72             .expect("regions are non-empty by construction");
73             Frontier { size: cells.len(), cells, centroid, representative }
74         })
75         .collect()
76 }

```

The navigation field is the other half, and it earns its own type because two different consumers need it: the utility needs a *cost*, and the completeness argument needs a *reachability predicate*. They are the same Dijkstra expansion.

RUST crates/ch24_explore/src/frontier.rs (continued)

```

1 use std::cmp::Reverse;
2 use std::collections::BinaryHeap;
3
4 use ordered_float::OrderedFloat;
5
6 /// Dijkstra over known-free cells, 8-connected, in metres.
7 ///
8 /// `cost[i] = inf` means "not reachable through what we have mapped so far",
9 /// which is exactly the predicate the completeness proof needs -- and exactly
10 /// the predicate an over-inflated obstacle map quietly falsifies.
11 pub struct NavField<'a> {
12     grid: &'a OccGrid,
13     cost: Vec<f64>,
14     parent: Vec<Option<usize>>,
15     start: usize,
16 }
17
18 impl<'a> NavField<'a> {
19     pub fn new(grid: &'a OccGrid, start: GridIdx, th: ClassThresholds, inflate: usize) ->
20         Self {
21         let blocked = inflate_obstacles(grid, th, inflate);
22         let mut cost = vec![f64::INFINITY; grid.len()];
23         let mut parent = vec![None; grid.len()];
24         let start = grid.to_flat(start);
25
26         // The robot is standing where it is standing: inflation may never

```

```

26 // strand it, so the source is seeded regardless of `blocked`.
27 cost[start] = 0.0;
28 let mut heap = BinaryHeap::new();
29 heap.push(Reverse((OrderedFloat(0.0), start)));
30
31 while let Some(Reverse((OrderedFloat(c), i))) = heap.pop() {
32     if c > cost[i] { continue; }
33     for (j, step) in grid.neighbors8_with_step(i) {
34         if blocked[j] { continue; }
35         if classify(log_odds_to_prob(grid.log_odds()[j]), th) != CellClass::Free
{ continue; }
36         let next = c + step;
37         if next < cost[j] - 1e-12 {
38             cost[j] = next;
39             parent[j] = Some(i);
40             heap.push(Reverse((OrderedFloat(next), j)));
41         }
42     }
43 }
44 Self { grid, cost, parent, start }
45 }
46
47 /// The cheapest reachable cell within `radius` of a frontier target.
48 ///
49 /// Frontier cells sit against the unknown, so inflation often makes the
50 /// exact cell unreachable while its neighbour two cells back is fine.
51 /// Aiming at the neighbour is what keeps an explorer from declaring a
52 /// doorway impossible.
53 pub fn nearest_reachable(&self, goal: GridIdx, radius: i32) -> Option<GridIdx, f64>
{ /* ... */ }
54
55 pub fn path_to(&self, goal: GridIdx) -> Vec<Point2<f64>> { /* walk `parent` back */ }
56 }

```

24.7.2 Expected gain

RUST crates/ch24_explore/src/info_gain.rs

```

1 use ch02_prob::binary_entropy;
2 use ch13_occgrid::{bresenham, log_odds_to_prob, OccGrid};
3 use nalgebra::{Isometry2, Vector2};
4 use rustc_hash::FxHashSet;
5
6 /// The sensor as the *planner* models it. Deliberately cruder than Chapter 10's
7 /// beam mixture: all the planner needs from a range finder is how many beams,
8 /// how far, and how much to trust one cell's verdict.
9 #[derive(Clone, Copy)]
10 pub struct SensingParams {
11     pub n_beams: usize,
12     pub fov: f64,
13     pub max_range: f64,
14     /// q: the probability the inverse model calls a cell correctly. The
15     /// crossover of a binary symmetric channel, and the only sensor number
16     /// this estimator ever sees.
17     pub z_hit: f64,
18 }
19

```

```

20 ///  $I(m? ; z?)$  in bits, for one cell with prior `p` through a channel of
21 /// reliability `q`.
22 ///
23 /// At  $p = ?$  this is the channel capacity  $1 - H_b(q)$ ; as  $p \rightarrow 0$  or  $1$  it is zero.
24 /// Both limits are worth remembering, because between them they explain every
25 /// decision an information-gain explorer makes.
26 pub fn cell_mutual_information(p: f64, q: f64) -> f64 {
27     if !(0.0..=1.0).contains(&p) || p <= 0.0 || p >= 1.0 { return 0.0; }
28     let p_occ = p * q + (1.0 - p) * (1.0 - q);
29     let p_free = 1.0 - p_occ;
30     let post_occ = if p_occ > 0.0 { p * q / p_occ } else { 0.0 };
31     let post_free = if p_free > 0.0 { p * (1.0 - q) / p_free } else { 0.0 };
32     let expected = p_occ * binary_entropy(post_occ) + p_free * binary_entropy(post_free);
33     (binary_entropy(p) - expected).max(0.0)
34 }
35
36 pub struct InfoGainEstimator<'a> {
37     pub grid: &'a OccGrid,
38     pub sensor: SensingParams,
39     /// Count each cell at most once per scan. Without it, the estimator happily
40     /// rewards standing still with its nose against a wall.
41     pub dedupe: bool,
42 }
43
44 impl InfoGainEstimator<'_> {
45     /// `expected_info_gain` -- bits of map entropy one scan from `pose` should
46     /// remove. Equation (F.2); the reach probability is the running product.
47     pub fn expected_gain(&self, pose: &Isometry2<f64>) -> f64 {
48         let origin = self.grid.world_to_cell(pose.translation.vector.into());
49         let mut visited = FxHashSet::default();
50         let mut total = 0.0;
51
52         for phi in beam_angles(self.sensor.n_beams, self.sensor.fov) {
53             let theta = pose.rotation.angle() + phi;
54             let end = self.grid.world_to_cell(
55                 pose.translation.vector + self.sensor.max_range * Vector2::new(theta.cos
56                 (), theta.sin()),
57             );
58
59             let mut reach = 1.0_f64;
60             for cell in bresenham(origin, end) {
61                 if reach < 1e-3 { break; }
62                 let Some(flat) = self.grid.try_flat(cell) else { break };
63                 let p = log_odds_to_prob(self.grid.log_odds()[flat]);
64                 if !self.dedupe || visited.insert(flat) {
65                     total += reach * cell_mutual_information(p, self.sensor.z_hit);
66                 }
67                 // The beam is stopped by this cell whether or not we already
68                 // scored it: occlusion belongs to the map, not the bookkeeping.
69                 reach *= 1.0 - p;
70             }
71             total
72         }
73     }
74 }

```

24.7.3 D-optimality over the pose graph

RUST crates/ch24_explore/src/utility.rs

```

1 use ch13_occgrid::{GridIdx, OccGrid};
2 use ch15_graph::{BlockIndex, Ordering};
3 use ch16_slam2d::PoseGraph;
4 use faer::linalg::solvers::Llt;
5 use faer::sparse::SparseColMat;
6 use nalgebra::Isometry2;
7
8 use crate::frontier::{Frontier, NavField};
9 use crate::info_gain::InfoGainEstimator;
10
11 /// `graph_log_det` -- log det Omega in nats, from a factorization we were going to
12 /// compute anyway.
13 ///
14 /// The gauge node's rows and columns are *deleted*, not damped: with them in,
15 /// Omega is singular by construction and its determinant is zero however good the
16 /// map is. Cost is one sparse Cholesky -- about  $O(n^{1.5})$  for a planar graph,
17 /// which is what makes scoring a dozen candidates per decision affordable.
18 pub fn graph_log_det(graph: &PoseGraph) -> f64 {
19     let (omega, _index): (SparseColMat<usize, f64>, BlockIndex) = graph.information_free
20     ();
21     let symbolic = Ordering::Amd.symbolic(&omega);
22     let llt = Llt::try_new_with_symbolic(symbolic, omega.as_ref())
23     .expect("Omega is singular -- is the gauge node fixed?");
24     // log det Omega = 2 Sigma log L_kk.
25     2.0 * llt.L().diagonal().column_vector().iter().map(|d| d.ln()).sum::<f64>()
26 }
27
28 /// D-optimality in the form the field settled on (Carrillo et al. 2012;
29 /// Placed et al. 2023):  $D_{opt}(\Omega) = \exp( (1/n) \log \det \Omega )$ .
30 ///
31 /// The  $1/n$  is the whole point. Raw log det counts nodes, so it prefers the
32 /// longer trajectory for being longer; the geometric-mean form is what makes
33 /// two graphs of different sizes comparable at all.
34 pub fn d_optimality(graph: &PoseGraph) -> f64 {
35     let n = 3 * graph.free_node_count();
36     if n == 0 { return 0.0; }
37     (graph_log_det(graph) / n as f64).exp()
38 }
39
40 #[derive(Clone, Copy)]
41 pub struct UtilityWeights { pub w_i: f64, pub w_g: f64, pub w_c: f64 }
42
43 pub struct Candidate {
44     pub target: GridIdx,
45     pub pose: Isometry2<f64>,
46     ///  $I(a)$ , bits.
47     pub gain: f64,
48     ///  $C(a)$ , metres of known-free path.
49     pub cost: f64,
50     ///  $\Delta_a = \log D_{opt}(\Omega_{a??}) - \log D_{opt}(\Omega)$ , nats per degree of freedom.
51     pub graph_gain: f64,
52     pub utility: f64,
53 }
54
55 /// `score_candidates` -- one utility per frontier region.
56 ///

```

```

56 // The sensing pose faces from the reachable target toward the region's
57 // centroid: a range finder pointed back down the corridor it just drove
58 // learns nothing, and the estimator would happily tell you so if you asked it,
59 // but only if you ask at the right heading.
60 pub fn score_candidates(
61     grid: &OccGrid,
62     frontiers: &[Frontier],
63     nav: &NavField<'_>,
64     gain: &InfoGainEstimator<'_>,
65     w: UtilityWeights,
66     graph_gain: Option<&dyn Fn(GridIdx, f64) -> f64>,
67 ) -> Vec<Candidate> {
68     let mut out: Vec<Candidate> = frontiers
69         .iter()
70         .filter_map(|f| {
71             // Unreachable through known-free space: skip it, do not fail.
72             let (target, cost) = nav.nearest_reachable(f.representative, 4)?;
73             let t = grid.center(target);
74             let heading = (f.centroid - t).y.atan2((f.centroid - t).x);
75             let pose = Isometry2::new(t.coords, heading);
76             let i = gain.expected_gain(&pose);
77             let g = graph_gain.map_or(0.0, |f| f(target, cost));
78             Some(Candidate {
79                 target, pose, gain: i, cost, graph_gain: g,
80                 utility: w.w_i * i + w.w_g * g - w.w_c * cost,
81             })
82         })
83         .collect();
84     out.sort_by(|a, b| b.utility.partial_cmp(&a.utility).unwrap());
85     out
86 }

```

24.7.4 The loop

RUST crates/ch24_explore/src/explorer.rs

```

1 use ch16_slam2d::Slam2d;
2 use nalgebra::{Isometry2, Point2};
3
4 use crate::frontier::{detect_frontiers, ClassThresholds, NavField};
5 use crate::info_gain::{InfoGainEstimator, SensingParams};
6 use crate::utility::{d_optimality, score_candidates, Candidate, StopRule, UtilityWeights};
7
8 // What one decision can be. `Done` carries *why*, because "the run ended" and
9 // "the run ended for a good reason" are different claims and the ablation
10 // table needs to tell them apart.
11 pub enum Decision {
12     Goto { target: Isometry2<f64>, path: Vec<Point2<f64>>, candidates: Vec<Candidate> },
13     CloseLoop { node: ch16_slam2d::NodeIdx },
14     Done { reason: StopReason },
15 }
16
17 pub enum StopReason { NoFrontiers, GainRate, Budget }
18
19 pub struct Explorer {
20     pub weights: UtilityWeights,

```

```

21     pub sensing: SensingParams,
22     pub thresholds: ClassThresholds,
23     pub stop: StopRule,
24     pub min_frontier_size: usize,
25     pub inflate: usize,
26 }
27
28 impl Explorer {
29     /// `explore_step` -- identify, select, execute. One decision.
30     ///
31     /// Placed et al. (2023) name the three stages every active-SLAM system has,
32     /// whether or not its authors drew the boxes. Notice the proportions: the
33     /// policy is the two `let` bindings in the middle, and everything else in
34     /// this crate is keeping the estimate alive while it runs. That is the
35     /// honest ratio for a real system too.
36     pub fn decide(&mut self, slam: &Slam2d) -> Decision {
37         let grid = slam.grid();
38         let here = grid.world_to_cell(slam.pose().translation.vector.into());
39
40         // ---- identify -----
41         let frontiers = detect_frontiers(grid, self.thresholds, self.min_frontier_size);
42         let nav = NavField::new(grid, here, self.thresholds, self.inflate);
43         let gain = InfoGainEstimator { grid, sensor: self.sensing, dedupe: true };
44
45         // ---- select -----
46         let base = d_optimality(slam.graph()).ln();
47         let graph_gain = |target: GridIdx, cost: f64| {
48             // Predict the factors this trajectory would add: one odometry
49             // factor per step, plus a loop closure wherever it re-observes a
50             // mapped region. Then score the hypothetical graph.
51             let predicted = slam.graph().with_predicted_factors(target, cost, &self.
sensing);
52             d_optimality(&predicted).ln() - base
53         };
54         let candidates =
55             score_candidates(grid, &frontiers, &nav, &gain, self.weights, Some(&
graph_gain));
56
57         if let Some(reason) = self.stop.triggered(&candidates) {
58             return Decision::Done { reason };
59         }
60
61         // ---- execute -----
62         let best = &candidates[0];
63         Decision::Goto {
64             target: best.pose,
65             path: nav.path_to(best.target), // handed to ch23_mppi to track
66             candidates,
67         }
68     }
69 }

```

24.8 Putting it together

cargo run --release --example autonomous_explore drops Rusty into the apartment at the corridor midpoint with 4800 bits of ignorance and no instructions.

The example prints one row per policy over ten seeds; the TypeScript port that drives the widgets on this page produces the same numbers, because it is the same algorithm.

policy	runs reaching 4000 of 4800 bits	median distance to 4000 bits	total distance	final $H(m)$	cells resolved	bits per metre
lawnmower (scripted sweep)	10 / 10	46.0 m	92.6 m	586 b	96.0%	45.5
nearest frontier ($w_I = 0$)	2 / 10	75.2 m	42.5 m	1722 b	72.7%	83.2
utility ($w_I = 1$, $w_C = 0.35$)	9 / 10	33.6 m	71.3 m	370 b	94.9%	76.8

Read it carefully, because the interesting result is not the one you expected.

The utility explorer wins the column that matters: it reaches a fixed information target in 27% less distance than the scripted sweep, in nine runs out of ten, and it finishes with the lowest final entropy of the three.

The lawnmower is not embarrassing. It is *reliable* — 10/10 — because it does not depend on the map being right. That is exactly the trade a scripted policy makes, and it is why coverage path planning is still the correct answer when you already have the floorplan.

Nearest-frontier is the most efficient policy per metre travelled, and it is the worst policy on this table. It clears 83 bits for every metre it drives; it just stops driving. Eight runs out of ten end below the information target, most of them with frontiers still visible on the map, because the argmax of $-C(a)$ is whatever boundary is nearest — including the one under the robot's own wheels. This is not a strawman built to lose. It is the reason every deployed frontier explorer carries a minimum-target-distance guard, a hysteresis term, or a blacklist, and the sixth exercise asks you to add one and re-run the table.

i NOTE

What this chapter left out, and where it went. *Multi-robot exploration* and market-based task allocation were the centerpiece of the 2005 textbook's Chapter 17; the machinery above generalizes (the utility becomes an assignment problem over robots and frontiers) but the book's lab is single-robot, so it is a pointer, not a section — Placed et al. §VII surveys the state of it, and Asgharivaskasi et al. (2025) give the modern distributed formulation. *Coverage path planning* is one paragraph at the top of this chapter, and Choset's cellular decomposition is the reference. *RBPF-specific exploration utilities* (Stachniss et al., 2005) computed gain over a particle set of maps rather than one grid; that is a historical note now that the book's SLAM spine is the graph, but it is the right idea if your spine is Chapter

17's. *Learned exploration policies* and neural map-completion priors — which predict what is behind the frontier instead of assuming it behaves like the prior — belong to Chapter 25.

Everything the capstone needs now exists. Chapter 26 adds no new estimator, no new planner, and no new controller: what remains is orchestration and the failure modes that appear only when all of it runs at once.

24.9 Exercises

1 FOUNDATION •• Non-negative in expectation, and the case that is not

Prove $I(a) = I(m; z_a) \geq 0$ from the KL form, stating exactly where Gibbs' inequality is used and what equality would mean physically. Then construct an explicit two-cell, one-beam example in which a *specific* measurement increases the map entropy, and compute both the increase and the probability of that reading. Finally, confirm your example numerically with `cell_mutual_information` and explain in two sentences why an entropy log that never rises would be evidence of a bug rather than of a good filter.

2 FOUNDATION •• A worse sensor, sublinearly

Redo the five-cell worked example with $q = 0.7$ instead of 0.9. You should get 0.2300 bits against the chapter's 1.0288 — a factor of 4.5 for a sensor that is only 22% less reliable. Explain the nonlinearity in terms of the channel capacity $1 - H_b(q)$, and predict (before computing) whether the *ratio* between the unknown ray and the mapped ray grows or shrinks as $q \rightarrow 0.5$.

3 FOUNDATION ••• Where the tree identity breaks

The derivation shows $\det \Omega = \prod_e \det \Omega_e$ for a tree. Two of its hypotheses can fail in practice. (a) Show what happens to $\det A$ when a node is constrained by *two* odometry factors — for instance because the front-end fused wheel odometry and IMU into separate factors — and say whether D-optimality then depends on the trajectory. (b) The derivation drops the right-Jacobian factor $J_r^{-1}(e)$ that Chapter 16 also drops. Argue why that is harmless for the *ranking* of two candidates while being wrong for the absolute value of $\log \det \Omega$.

4 CONCEPTUAL •• Predict the flip, then check it

In the Disambiguation Detour, read the candidate bars — do not run the policy — and predict the sensor-noise level at which the *scored* argmax moves from the detour to

the direct route. Write down your prediction, then find it with the slider. Explain the flip in terms of $I(a)$ against $C(a) + T(a)$, and say what would have to change about the corridor (not the sensor) to move the flip point by 0.05.

5 CONCEPTUAL •• Make greed win

Using the Frontier Chaser, find a seed and a cost weight w_C for which nearest-frontier reaches a *higher* cells-resolved percentage than the utility policy at the same distance travelled. What structural property of that particular run makes greed the right answer? Then state the property of the apartment as a whole that makes greed usually wrong, and connect it to the shape of the bits-per-metre sparkline.

6 PRACTICAL •• A guard against the stall

Reproduce the stall: run the nearest-frontier policy and log, per decision, the chosen candidate's cost. You will find long stretches at $C(a) \approx 0$. Add a guard on the minimum target distance — reject candidates whose reachable target is within $d_{\min}$ of the robot — and re-run the ten-seed ablation table. Report every column for $d_{\min} \in \{0, 0.5, 1.5\}$ m, and say which column moves the most; then argue whether the guard is a fix or a hyperparameter hiding a modelling error. *Extension:* a frontier region that wraps around a large room gets one candidate at its centroid, which can be a bad place to stand — implement centroid-splitting along the region's principal axis and run the table a third time.

7 PRACTICAL ••• Lookahead, and whether it pays

Replace the depth-1 `active_localize` argmax with a POMCP search (Chapter 22) over a three-action macro-space: detour, direct, and wait-and-scan. Run both on the Disambiguation Detour world across 50 seeds and report the success rate, the mean decisions per trial, and the wall-clock time per decision. Then build the world where the lookahead is *necessary* — a corridor whose disambiguating feature lies two macro-actions away, so that the first step toward it scores exactly zero — and show the depth-1 policy failing on it.

24.10 References

Yamauchi, B. (1997). A Frontier-Based Approach for Autonomous Exploration *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, Monterey, CA, 146–151.

The origin of the frontier, and the source of this chapter’s epigraph. Older than the 2005 textbook baseline and still the structural backbone of every exploration stack in this chapter. doi:10.1109/CIRA.1997.613851

Fox, D., Burgard, W., and Thrun, S. (1998). Active Markov Localization for Mobile Robots *Robotics and Autonomous Systems* 25(3–4), 195–207.

The expected-entropy-reduction rule that widget w24.2 implements, derived over a grid belief. Reading it beside Chapter 12 shows how little the idea needs beyond a Bayes filter you can run forwards. doi:10.1016/S0921-8890(98)00049-9

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

Chapter 17 is this chapter’s conceptual ancestor: greedy information-gain exploration, active localization, and multi-robot coordination over occupancy grids. Note that the 1999–2000 draft this book otherwise works from has no exploration chapter at all. [Link](#)

Stachniss, C., Grisetti, G., and Burgard, W. (2005). Information Gain-based Exploration Using Rao-Blackwellized Particle Filters *Proceedings of Robotics: Science and Systems (RSS)*, Cambridge, MA.

The first system to score exploration actions by their effect on the joint map-and-trajectory posterior rather than the map alone — the idea this chapter re-expresses over a pose graph instead of a particle set. doi:10.15607/RSS.2005.I.009

Carrillo, H., Reid, I., and Castellanos, J. A. (2012). On the Comparison of Uncertainty Criteria for Active SLAM *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, St. Paul, MN, 2080–2087.

The paper that settled A- versus D- versus E-optimality for robotics, and the source of the normalized D-opt form used in this chapter’s utility. doi:10.1109/ICRA.2012.6224890

Placed, J. A., Strader, J., Carrillo, H., Atanasov, N., Indelman, V., Carlone, L., and Castellanos, J. A. (2023). A Survey on Active Simultaneous Localization and Mapping: State of the Art and New Frontiers *IEEE Transactions on Robotics* 39(3), 1686–1705.

The chapter’s map of the field: active SLAM as an intractable POMDP approached by an identify–select–execute pipeline, with a unified treatment of optimality criteria and an open-problems section that includes stopping. doi:10.1109/TR0.2023.3248510

Placed, J. A. and Castellanos, J. A. (2023). A General Relationship Between Optimality Criteria and Connectivity Indices for Active Graph-SLAM *IEEE Robotics and Automation Letters* 8(2), 816–823.

Relates the graph Laplacian’s connectivity indices to the optimality criteria of the information matrix, which is what makes the tree-versus-cycle argument of this chapter’s derivation into a usable surrogate on graphs too large to factor per candidate. doi:10.1109/LRA.2022.3233230

Zhou, B., Zhang, Y., Chen, X., and Shen, S. (2021). FUEL: Fast UAV Exploration Using Incremental Frontier Structure and Hierarchical Planning *IEEE Robotics and Automation Letters* 6(2), 779–786.

Frontier detection maintained incrementally rather than recomputed, plus a hierarchical tour over frontier clusters — the two engineering moves that take this chapter’s $O(\#cells)$ rescan to real-time on a drone. doi:10.1109/LRA.2021.3051563

Cao, C., Zhu, H., Choset, H., and Zhang, J. (2021). TARE: A Hierarchical Framework for Efficiently Exploring Complex 3D Environments *Proceedings of Robotics: Science and Systems (RSS), Virtual*.

The strongest counterargument to greedy selection: plan a detailed local path and a coarse global tour, and the myopia that produces this chapter’s nearest-frontier stall disappears. Best paper at RSS 2021. doi:10.15607/RSS.2021.XVII.018

Asgharivaskasi, A., Girke, F., and Atanasov, N. (2025). Riemannian Optimization for Active Mapping With Robot Teams *IEEE Transactions on Robotics* 41, 1077–1097.

The modern multi-robot form of this chapter’s utility: distributed optimization of a mutual-information objective over trajectories on a manifold, with consensus and optimality guarantees. The pointer for the multi-robot material this chapter drops. doi:10.1109/TR0.2025.3526295

Part VII

Frontiers and Integration

Learning in the Loop

FRONTIERS AND INTEGRATION · Advanced · 60 min

“

As this book is being written, the issue of learning inverse sensor models from data remains relatively unexplored.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 9 (§9.3.4)

Every model in this book so far was written by hand. The beam mixture's four densities, the six α parameters of the odometry model, the likelihood field's smoothing kernel — someone chose those functional forms, and someone tuned their constants. This chapter asks the 2026 question honestly: where does machine learning belong in a probabilistic robot?

The answer that organizes everything else is a restriction, not a permission. **Learning replaces the boxes inside the Bayes filter; it does not replace the filter.** A learned $p(z_t \mid x_t, m)$ is admissible exactly when it is a *calibrated* density, because the Bayes filter is a machine that turns stated confidence into decisions: it multiplies likelihoods together, normalizes, and acts on the result. Feed it a model that overstates its own certainty and it will not merely be wrong — it will be *confidently* wrong, and it will stop listening.

Two things follow, and they are the chapter's two "aha"s. First, a model can be more accurate and still poison a filter, because accuracy and calibration are different properties and only one of them is what the filter consumes. Calibration is measurable, it is repairable, and by the end of the chapter you will have measured and repaired it. Second, the Bayes filter is itself a differentiable program: "tuning R_t and Q_t " and "training a network" are the same act — gradient descent on a proper scoring rule, straight through the filter equations. The capstone (Chapter 26) ships a calibrated learned sensor model as a toggle, and cites the benchmark at the end of this chapter as its justification.

🎯 AFTER THIS CHAPTER YOU CAN

- Say precisely where learning belongs in a probabilistic robot: inside the boxes of the Bayes filter, never in place of the filter
- Define calibration for a density forecast, measure it with PIT histograms, reliability diagrams and ECE, and repair it with a single scalar
- Prove that the log score is the honest training loss, and that a sharper model is not automatically a better one
- Quantify what overconfidence costs: posterior precision that grows linearly in κ while effective sample size dies exponentially
- Train a Kalman filter's noise parameters by gradient descent on the evidence — with no ground truth anywhere in the loss
- Repair the one step of a particle filter that has no gradient, and state the price soft resampling charges
- Read the honesty ledger: which learned components of 2026 compose with Bayes rule, and which only look like they do

📖 ASSUMED

- Entropy, KL divergence, and expectation (Chapter 2)
- The Bayes filter and its measurement update (Chapter 5)
- The Kalman filter's five equations and the innovation covariance S_t (Chapter 6)
- Particle filters: importance weights, resampling, effective sample size (Chapters 8 and 12)
- The beam model, `learn_intrinsic_parameters`, and the lie of independence (Chapter 10)

25.1 The problem: an accurate model that lies

An engineer wires up Rusty's new LiDAR on a Tuesday afternoon. The datasheet says ranging accuracy ± 5 cm, so the beam model of Chapter 10 gets $\sigma_{hit} = 0.05$ m, a small spike at z_{max} for dropouts, and a thin uniform floor for whatever else happens. No unexpected-obstacle component: the room is empty, the model has no reason to expect anything in front of the wall.

The model is *accurate*. Its peak sits on the correct range. Ask it for the most likely reading and it is right more often than the model the robot shipped with. Then Monte Carlo localization is switched on and the robot loses itself inside twenty seconds — not by drifting, but by committing, hard, to a pose that is 30 cm wrong and refusing every subsequent scan that disagrees.

 INTERACTIVE · w25.1**The Calibration Clinic**

A sharper sensor model is not a better one. Calibration is a measurable property, and it is the one the filter downstream actually consumes.

Run this simulation in the web edition: [/chapters/ch25-learning #w25.1](/chapters/ch25-learning#w25.1)

Watch one full sweep of the slider before reading on. Three things are happening, and each one has a piece of mathematics later in this chapter.

The reliability curve sags below the diagonal. The diagonal is the promise: of all the observations for which the model claimed "at most this far out", that fraction really should land at most that far out. A curve below the diagonal means every interval the model quotes catches fewer observations than advertised. At $\sigma_{hit} = 5$ cm the expected calibration error is 0.107 — the model's 50% interval is closer to a 30% interval — and the χ^2 statistic of its PIT histogram is over 1400 on 14 degrees of freedom. That is not a borderline call; that is a model that is lying in a way you could detect from a hundred readings.

The PIT histogram is a U. Readings keep landing in the tails the model declared empty. The shape of the miscalibration tells you its cause: a U means too narrow, a hump means too wide, a tilt means biased. This is a *diagnostic instrument*, not a score, and it is the single most useful plot in this chapter.

The filter downstream cares, and it does not care about what you expected. The two right-hand tiles come from a real importance-weighting step over twelve particle clouds in the Apartment. As the claimed width shrinks, the effective sample size collapses — at $\sigma_{hit} = 3$ cm a 150-particle cloud is doing the work of one particle — and the posterior error *rises* even as the model looks sharper and more confident.

 THE ONE SENTENCE VERSION

Learning belongs inside the boxes of the Bayes filter, and a learned box earns its place by being calibrated, not by being sharp: the filter consumes stated confidence, so a model that overstates its confidence corrupts every decision downstream, no matter how accurate its point predictions are.

25.2 Building intuition

25.2.1 Learning, 1999 style — and what it cannot fix

Learning is not something modern practice grafted onto Thrun, Burgard and Fox. The 1999–2000 draft already fits the beam model's intrinsic parameters by expectation-maximization (`learn_intrinsic_parameters`, Table 6.2, derived in Chapter 10); it already trains a neural network to *invert* the measurement model

by sampling triplets from the forward model (§9.3.2); it learns maps with EM, and it learns POMDP value functions by nearest-neighbor function approximation. The epigraph to this chapter is that book noticing its own frontier.

So run it. Take 4000 range readings drawn from the true sensor in the Apartment and hand them to `learn_intrinsic_parameters`, initialized at the engineer's hand-tuned guess. EM does its job: the mean log-likelihood climbs from -0.923 to -0.224 nats per beam in forty iterations and σ_{hit} moves from 0.050 m to 0.112 m, essentially the truth. And yet the fitted model is *still* measurably miscalibrated, with an ECE of 0.029 against the true model's 0.021 — because EM inherited a defect from its initialization that no amount of iterating can repair.

The engineer's model set $z_{short} = 0$: no unexpected-obstacle component at all. In EM, a component's responsibility is proportional to its weight, so a weight of exactly zero produces zero responsibility, which produces a new weight of exactly zero. **A component initialized at zero can never come back.** EM did the only thing left to it and inflated z_{rand} from 0.05 to 0.142, smearing a uniform floor across the whole range to absorb readings that came from an exponential concentrated in front of the wall.

That is the first honest lesson about learning in a robot. Maximum likelihood inside a family only searches that family. When the family is wrong — and it always is, a little — the fit will spend its freedom compensating in whatever direction the parameterization allows, and the result can be an excellent likelihood attached to a badly shaped uncertainty.

25.2.2 The question is not "how accurate", it is "how honest"

Point accuracy asks: is the model's best guess close to the truth? Calibration asks something the filter cares about far more: **when the model says 70%, does it happen 70% of the time?**

Gneiting, Balabdaoui and Raftery gave the field the sentence that settles the priority. The goal of probabilistic forecasting, they argue, is to *maximize the sharpness of the predictive distributions subject to calibration* — sharpness being how concentrated the forecast is, a property of the forecast alone, and calibration being the statistical consistency between forecasts and what actually happens, a joint property of the forecast and the world. Sharpness is what you want. Calibration is what you are allowed. Take them in the wrong order and you get exactly the widget above: a model that wins on confidence and loses on reality.

25.2.3 Overconfidence is a filter-killer, not an aesthetic complaint

Here is the mechanism, before the algebra. A particle filter weights each particle by $w^{[i]} \propto p(z_t \mid x_t^{[i]}, m)$. Sharpen the likelihood and you sharpen the *weights*: one particle takes almost all the mass, the effective sample size collapses, and resampling clones that single particle four hundred times. The cloud now has one hypothesis. If the truth is not underneath it, there is nothing left to re-weight

and no recovery is possible — the particle deprivation of Chapter 12, arriving not from bad luck but from an overconfident model.

 INTERACTIVE · w25.3

The Tempering Lab

Multiplying in more confidence is not free: the effective sample size falls off a cliff, and a filter with one live particle cannot recover from anything.

Run this simulation in the web edition: `/chapters/ch25-learning #w25.3`

The knob in that widget, κ , is the honest way to talk about this. It does not change *what* the model says, only how loudly: $p(z \mid x, m)^\kappa$. And it is not an artificial device — Chapter 10 showed that treating K correlated beams as independent multiplies every log-odds by roughly K/K_{eff} , which is *exactly* a tempering exponent greater than one that nobody chose and nobody wrote down.

25.3 The mathematics

25.3.1 Notation for this chapter

θ	Learnable parameters: network weights, log r and log q , a temperature — anything gradient descent may move.
$\hat{p}_\theta(z_t \mid x_t, m)$	A learned observation model. Must be a proper density in z_t : non-negative and integrating to one.
$F_\theta(z \mid x)$	Its predictive CDF. $v = F_\theta(z \mid x)$ is the probability integral transform (PIT) of an observation.
$S(q, z)$	A scoring rule: the penalty a forecast density q pays when z materializes. Negatively oriented — lower is better.
ECE	Expected calibration error: mean absolute distance between claimed and observed coverage, over $B = 15$ bins.
κ	Likelihood tempering exponent. Deliberately not γ , which is reserved for the discount factor.
λ	Soft-resampling mixture weight. Deliberately not α , which is reserved for motion-model noise.
$\mathcal{L}(\theta)$	Training loss. The negative log score unless stated otherwise.
s	Spread (sample standard deviation) of the per-particle log-likelihoods within a cloud.

⚠ WHERE ROBOTS BREAK THIS

One bridging note on κ . Chapter 10 writes the tempered likelihood as $p^{1/\kappa}$, with $\kappa \geq 1$ counting how many raw beams it takes to make one independent beam — a *discount*. This chapter writes p^κ , so that a single slider can pass continuously through $\kappa = 1$ from underconfident to overconfident. The two are reciprocals of each other: Chapter 10's $\kappa = 10$ is this chapter's $\kappa = 0.1$. Nothing else changes.

25.3.2 Proper scoring rules

You cannot train a model without a loss, and not every loss is safe. A scoring rule that a forecaster can improve by *misreporting* its beliefs is a machine for producing liars.

💡 D25.1 — PROPER SCORING RULE

A scoring rule $S(q, z)$ assigns a penalty to a forecast density q when the value z materializes. It is **proper** if for every pair of densities p and q ,

$$\mathbb{E}_{z \sim p} [S(p, z)] \leq \mathbb{E}_{z \sim p} [S(q, z)],$$

that is: honestly reporting your true belief p is never worse than reporting anything else. It is **strictly proper** if equality holds only when $q = p$. The **log score** $S(q, z) = -\log q(z)$ is strictly proper.

Strict propriety is what makes the negative log-likelihood the default training loss for anything that will live inside a Bayes filter: it is the loss whose unique optimum is *the truth*, so a model that minimizes it has no incentive left to shade its uncertainty in either direction.

📐 DERIVATION F1 — Why the log score is strictly proper

$$E_p[-\log q] = H(p) + KL(p||q) \geq H(p)$$

Step 1 — write the expected score. For a forecast q evaluated on data drawn from the true density p ,

$$\mathbb{E}_{z \sim p} [-\log q(z)] = - \int p(z) \log q(z) dz.$$

Step 2 — add and subtract the entropy. Insert $\log p(z) - \log p(z)$:

$$-\int p \log q = \underbrace{-\int p \log p}_{H(p)} + \underbrace{\int p \log \frac{p}{q}}_{\text{KL}(p \parallel q)}.$$

Step 3 — Gibbs' inequality. $\text{KL}(p \parallel q) \geq 0$, by Jensen's inequality applied to the convex function $-\log$:

$$\text{KL}(p \parallel q) = \mathbb{E}_p \left[-\log \frac{q}{p} \right] \geq -\log \mathbb{E}_p \left[\frac{q}{p} \right] = -\log \int q = 0.$$

Step 4 — the equality condition. $-\log$ is *strictly* convex, so Jensen holds with equality only if q/p is constant p -almost everywhere; since both integrate to one, that constant is one and $q = p$ almost everywhere. Hence the expected log score is uniquely minimized at the truth, with minimum value the entropy $H(p)$ — the irreducible part, which no model can score below.

Two caveats worth stating. For continuous densities the score is a *differential* log score, so its absolute value carries an arbitrary unit convention (change metres to centimetres and every NLL in this chapter shifts by $\log 100$); only differences between models on the same data are meaningful. And the argument assumes $q > 0$ wherever $p > 0$ — a model that assigns zero density to something that happens takes an infinite penalty, which is the correct behavior and the reason every implementation in this book floors its densities before taking a logarithm. ■

25.3.3 Calibration, PIT, and ECE

💡 D25.2 — CALIBRATION OF A DENSITY FORECAST

Let $F_\theta(\cdot \mid x)$ be the model's predictive CDF and let (x_i, z_i) be observations from the world. The **probability integral transform** of observation i is $v_i = F_\theta(z_i \mid x_i)$. The model is **calibrated** if the PIT values are uniformly distributed on $[0, 1]$:

$$v_i \sim \mathcal{U}(0, 1).$$

Equivalently, for every level p , a fraction p of observations fall at or below the model's p -quantile. The binary special case is the familiar one: among

all events the model calls 70% likely, 70% occur.

The PIT is the whole diagnostic in one line, and it is worth seeing why it works. If z really is drawn from the density the model claims, then $F(z)$ is uniform — the classic inverse-transform argument run backwards. So any structure in the histogram of PIT values is *evidence of misspecification*, and its shape names the defect:

flat	Calibrated. Nothing to fix — improve it by making it sharper, if you can do so without bending this.
U-shaped	Forecast too narrow (overconfident). Observations keep landing in tails the model called empty.
hump-shaped	Forecast too wide (underconfident). The model hedges; it is honest but wasteful.
tilted	Biased. The predicted mean is systematically off — a modelling error, not an uncertainty error.

One technical repair matters for range sensors. The beam model has an **atom**: a lump of probability mass z_{max} sitting exactly on the maximum range. For a discontinuous CDF the plain PIT is not uniform even for a perfect model — every dropout beam would pile up at $v = 1$. The standard fix is the *randomized* PIT, which spreads each atom uniformly across the jump it makes:

$$v = F_{\theta}(z^-) + u \cdot [F_{\theta}(z) - F_{\theta}(z^-)], \quad u \sim \mathcal{U}(0, 1).$$

For a continuous forecast $F(z^-) = F(z)$ and this reduces to the plain PIT, so it costs nothing to always use it. Skip it and you will spend an afternoon "fixing" the calibration of a model whose only sin was reporting dropouts honestly.

💡 D25.3 — EXPECTED CALIBRATION ERROR

Bin the claims into B bins. With $\text{acc}(b)$ the observed frequency in bin b and $\text{conf}(b)$ the claimed probability,

$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|.$$

This book uses $B = 15$ everywhere, following Guo et al. For a density forecast the natural binning is by nominal quantile level, $p_b = b/B$: every bin then holds the same number of claims, $n_b/N = 1/B$, and the ECE collapses to the mean absolute distance between the reliability curve and the diagonal. The **reliability diagram** is the plot of $\text{acc}(b)$ against $\text{conf}(b)$; ECE is its average vertical gap.

ALGORITHM `calibrate(model, D, B)` COST $O(N \log N + B)$ -- one CDF evaluation per observation, then a sort

in a model with a predictive CDF, an evaluation set $D = \{(x_i, z_i)\}$, bin count B

out reliability bins, ECE, mean NLL, PIT histogram and its χ^2

```

1: for all  $(x_i, z_i) \in D$  do
2:    $v_i = F_\theta(z_i^- | x_i) + u_i [F_\theta(z_i | x_i) - F_\theta(z_i^- | x_i)]$ ,  $u_i \sim \mathcal{U}(0, 1)$ 
3:    $\ell_i = -\log \hat{p}_\theta(z_i | x_i)$ 
4: endfor
5: for  $b = 1$  to  $B$  do
6:    $\text{conf}(b) = b/B$ 
7:    $\text{acc}(b) = \frac{1}{N} |\{i : v_i \leq b/B\}|$ 
8: endfor
9:  $\text{ECE} = \frac{1}{B} \sum_b |\text{acc}(b) - \text{conf}(b)|$ 
10: return bins, ECE,  $\frac{1}{N} \sum_i \ell_i$ , histogram of  $v$ ,  $\chi^2$ 

```

The two numbers this returns are not redundant, and it is important to hold both: the **mean NLL** measures accuracy *and* sharpness together, and the **ECE** measures honesty alone. A model can win one and lose the other. The benchmark at the end of this chapter is a table in which exactly that happens.

25.3.4 Calibration and sharpness are different axes

DERIVATION F2 — Murphy's decomposition: reliability, resolution, uncertainty

$BS = \text{reliability} - \text{resolution} + \text{uncertainty}$

The clean version of the calibration/sharpness split lives on the Brier score, the squared-error scoring rule for a binary outcome: $BS = \frac{1}{N} \sum_i (f_i - y_i)^2$ with $f_i \in [0, 1]$ the forecast and $y_i \in \{0, 1\}$ the outcome. It is strictly proper, and it decomposes exactly.

Step 1 — bin the forecasts. Group the N forecasts into B bins, bin b holding n_b of them. Write $\bar{f}_b$ for the mean forecast in bin b , $\bar{y}_b$ for the observed frequency in bin b , and $\bar{y}$ for the overall base rate.

Step 2 — replace each forecast by its bin mean. Within a bin, $f_i \approx \bar{f}_b$ (this is the estimator's only approximation, and the source of its binning bias). Then

$$\text{BS} \approx \frac{1}{N} \sum_b \sum_{i \in b} (\bar{f}_b - y_i)^2.$$

Step 3 — add and subtract $\bar{y}_b$, then $\bar{y}$. Inside bin b ,

$$(\bar{f}_b - y_i)^2 = (\bar{f}_b - \bar{y}_b)^2 + 2(\bar{f}_b - \bar{y}_b)(\bar{y}_b - y_i) + (\bar{y}_b - y_i)^2,$$

and the cross term vanishes on summing over the bin, because $\sum_{i \in b} (\bar{y}_b - y_i) = 0$ by the definition of $\bar{y}_b$.

Step 4 — expand the last term the same way, now around the base rate: $\frac{1}{n_b} \sum_{i \in b} (\bar{y}_b - y_i)^2 = \bar{y}_b(1 - \bar{y}_b)$, and $\sum_b \frac{n_b}{N} \bar{y}_b(1 - \bar{y}_b) = \bar{y}(1 - \bar{y}) - \sum_b \frac{n_b}{N} (\bar{y}_b - \bar{y})^2$.

Step 5 — collect.

$$\text{BS} = \underbrace{\sum_b \frac{n_b}{N} (\bar{f}_b - \bar{y}_b)^2}_{\text{reliability}} - \underbrace{\sum_b \frac{n_b}{N} (\bar{y}_b - \bar{y})^2}_{\text{resolution}} + \underbrace{\bar{y}(1 - \bar{y})}_{\text{uncertainty}}.$$

Reliability is exactly what the reliability diagram plots — squared distance from the diagonal, lower is better. **Resolution** is sharpness that pays: how far the model's conditional frequencies move away from the base rate, higher is better. **Uncertainty** is the problem's own difficulty and no model changes it.

The moral is the one Gneiting states as a principle. You may buy a better score with sharpness, but only while reliability stays near zero; a model that buys score by shrinking its intervals past what the data supports pays for it in reliability, and — this is the part the score itself hides — pays again downstream, where a Bayes filter compounds the overstatement across every update. ■

25.3.5 The tempered update, and what it costs

💡 D25.4 — TEMPERED MEASUREMENT UPDATE

$$\text{bel}(x_t) = \eta p(z_t \mid x_t, m)^\kappa \overline{\text{bel}}(x_t)$$

$\kappa = 1$ is Bayes' rule. $\kappa > 1$ is overconfidence — the regime an independence-violating scan product and an overconfident learned model both land in. $\kappa < 1$ is the deliberate discount that buys honesty back.

DERIVATION F3 — Overconfidence poisons the filter, in two exact statements

$$\Omega_t = \bar{\Omega}_t + \kappa H^\top Q^{-1} \text{HandESS}/M \approx e^{(-\kappa^2 s^2)}$$

Step 1 — tempering a Gaussian scales its precision. For $p(z \mid x) = \mathcal{N}(z; Hx, Q)$, raising to the power κ gives

$$p(z \mid x)^\kappa \propto \exp\left(-\frac{\kappa}{2}(z - Hx)^\top Q^{-1}(z - Hx)\right),$$

which is the same Gaussian in x with Q replaced by Q/κ . So in the information form of Chapter 6, the update adds

$$\Omega_t = \bar{\Omega}_t + \kappa H_t^\top Q_t^{-1} H_t.$$

Step 2 — read what that means. Claimed information grows *linearly* in κ while the measurement carries exactly as much information as it did before. Ten correlated beams treated as independent produce a covariance $\sqrt{10}$ times too tight around an estimate that has not improved at all. Nothing in the filter's own output reveals this: the residuals shrink together with the covariance, which is precisely why Chapter 11 grades filters by NEES against ground truth and NIS against the innovations.

Step 3 — the particle-filter version is worse, because it is exponential. Let ℓ_i be the per-particle log-likelihood within a cloud, with mean μ_ℓ and spread s . Tempered weights are $w_i \propto e^{\kappa \ell_i}$, and

$$\frac{\text{ESS}}{M} = \frac{(\sum_i w_i)^2}{M \sum_i w_i^2} \xrightarrow{M \rightarrow \infty} \frac{\mathbb{E}[e^{\kappa \ell}]^2}{\mathbb{E}[e^{2\kappa \ell}]}.$$

Step 4 — evaluate with the Gaussian moment generating function. If $\ell \sim \mathcal{N}(\mu_\ell, s^2)$ then $\mathbb{E}[e^{t\ell}] = e^{t\mu_\ell + t^2 s^2/2}$, so the numerator is $e^{2\kappa\mu_\ell + \kappa^2 s^2}$, the denominator is $e^{2\kappa\mu_\ell + 2\kappa^2 s^2}$, and

$$\boxed{\frac{\text{ESS}}{M} \approx e^{-\kappa^2 s^2}}$$

The mean cancels — only the *spread* of the log-likelihoods matters — and κ enters squared, inside an exponential. With a spread of $s = 2$ nats across the cloud, $\kappa = 1$ leaves 1.8% of the population alive and $\kappa = 2$ leaves 3×10^{-5} of it. This is the closed form the Tempering Lab's "predicted ESS"

tile evaluates live, and the library’s self-checks confirm it to within 0.016 across $\kappa \in [0.5, 2]$.

Step 5 — connect it to the failure you can see. An overconfident model widens s just as surely as raising κ does, because both scale the log-likelihood. Once ESS approaches one, resampling clones a single hypothesis and the filter’s diversity is gone; the kidnapped-robot recovery of Chapter 12 becomes impossible, not unlikely. ■

The repair is one scalar, fitted on data the model has never seen.

ALGORITHM `fit_temperature(model, V)` COST $O(|V|)$ per objective evaluation; ~50 evaluations for golden-section search

in a trained model, a held-out validation set V

out the scale s minimizing validation loss

- 1: define $\mathcal{L}(s) = \frac{1}{|V|} \sum_{(x,z) \in V} -\log \hat{p}_{\theta,s}(z \mid x)$
- 2: where $\hat{p}_{\theta,s}$ is the model with its scale parameter multiplied by s
- 3: $s^\star = \arg \min_s \mathcal{L}(s)$ by golden-section search on $\log s$
- 4: return $s^\star$

Three properties make this the first thing to try and the last thing to skip. It has **one parameter**, so it cannot overfit a validation set of any reasonable size. It is fitted on **held-out** data, so it corrects the model rather than memorizing the training set. And for a density it does not reorder hypotheses at all — raising a Gaussian likelihood to the power κ is identical to scaling its width by $s = 1/\sqrt{\kappa}$, and neither operation moves the mode. It buys calibration without costing accuracy. That is why Guo et al.’s temperature scaling took over the classification literature, and it transposes to a range density without modification.

25.3.6 Learning the model from the model you already have

Thrun et al.’s §9.3.2 recipe is still the right one, and it is worth stating in its modern form because it explains where a learned sensor model’s training data comes from when the robot has no labels: **sample the forward model, fit the inverse**. You have a simulator, a map, and a physical model of the sensor; that is a generator of (x, z) pairs at any volume you like.

ALGORITHM `learn_observation_model(sim, cfg)` COST $O(N)$ per epoch; $O(N \log N)$ for the calibration pass

in a forward simulator, a map, a model family $\hat{p}_\theta$, sample count N

out θ fitted by NLL descent, plus a fitted temperature

```

1:  $D = \emptyset$ 
2: for  $k = 1$  to  $N$  do
3:   sample a pose  $x^{[k]} \sim p(x)$  over the free space of  $m$ 
4:    $z^{*[k]} = \text{ray\_cast}(m, x^{[k]})$  // the noise-free expected range
5:    $z^{[k]} \sim p(z | z^{*[k]})$  // the forward model, §9.3.2
6:    $D = D \cup \{(z^{*[k]}, z^{[k]})\}$ 
7: endfor
8: split  $D$  into  $D_{\text{train}}, D_{\text{val}}, D_{\text{test}}$ 
9:  $\theta = \arg \min_{\theta} \sum_{D_{\text{train}}} -\log \hat{p}_{\theta}(z | z^*)$  // strictly proper, by F1
10:  $s = \text{fit\_temperature}(\hat{p}_{\theta}, D_{\text{val}})$ 
11: return  $\theta, s$ , and  $\text{calibrate}(\hat{p}_{\theta, s}, D_{\text{test}})$ 

```

⚠ WHERE ROBOTS BREAK THIS

Line 5 is where the circularity lives, and this chapter will not pretend otherwise. A model trained on samples from a simulator and then evaluated against samples from the same simulator is calibrated *with respect to the simulator*. Everything measured in this chapter’s widgets is of that kind. The only honest transfer test is the one the 1999 draft already describes in §9.3.4 — localize a real robot in a known environment and assemble training pairs from its recorded measurements — and it is still the expensive, correct answer in 2026.

25.3.7 The filter is a differentiable program

💡 D25.5 — DIFFERENTIABLE FILTER

A Bayes filter whose motion model, measurement model, and noise parameters are differentiable functions of θ , trained by gradient descent on a loss over logged trajectories, with the gradient propagated *through the filter recursion itself*. The filter’s structure is kept as an algorithmic prior; only the contents of its boxes are learned.

The reason to care is not that gradient descent is fashionable. It is that hand-tuning R_t and Q_t is the single most common and least principled activity in applied estimation, and there is a loss function that does it properly — one that does not require ground truth.

DERIVATION F4 — Evidence via innovations: training without ground truth

$$\log p(z_{1:T} | u_{1:T}, \theta) = \sum_t \log N(z_t; H_t \bar{\mu}_t, S_t)$$

Step 1 — chain rule over time. The joint likelihood of the whole measurement sequence factorizes with no approximation:

$$p(z_{1:T} | u_{1:T}, \theta) = \prod_{t=1}^T p(z_t | z_{1:t-1}, u_{1:t}, \theta).$$

Step 2 — recognize each factor. $p(z_t | z_{1:t-1}, u_{1:t})$ is the one-step-ahead predictive density of the measurement — precisely the normalizer η^{-1} that Chapter 5 named the *evidence* and threw away.

Step 3 — in the linear-Gaussian case it is available in closed form. The filter has already computed the predicted state $\bar{\mu}_t, \bar{\Sigma}_t$; pushing it through the measurement model gives

$$z_t | z_{1:t-1} \sim \mathcal{N}(H_t \bar{\mu}_t, S_t), \quad S_t = H_t \bar{\Sigma}_t H_t^\top + Q_t.$$

Step 4 — sum the logs. With the innovation $v_t = z_t - H_t \bar{\mu}_t$,

$$\mathcal{L}(\theta) = -\log p(z_{1:T} | u_{1:T}, \theta) = \frac{1}{2} \sum_{t=1}^T [\log |2\pi S_t| + v_t^\top S_t^{-1} v_t].$$

Every quantity in that expression is something the filter already computes, and **not one of them is a state**. The loss is a function of measurements the robot actually took and predictions it actually made. Minimizing it is exactly *prediction-error identification* in the sense of classical system identification, and it is the reason a robot can learn its own noise parameters from an afternoon of driving with no motion-capture rig in sight.

Two failure modes are worth naming. The loss is invariant to anything that leaves the predictive distribution unchanged, so parameters that only ever appear inside S_t as a sum are not separately identifiable from a single measurement (see the worked example below). And a model that is *structurally* wrong can still minimize this loss by inflating S_t until nothing is surprising — evidence training will happily hand you an honest, useless filter, which is a much better failure than a dishonest, useless one. ■

DERIVATION F5 — The gradient of the Kalman filter

$$\partial K / \partial \theta = (\partial \bar{\sigma}^2 - K \partial S) / S$$

Take the scalar filter of Chapter 6, with $x_t = a x_{t-1} + b u_t + w_t$, $w_t \sim \mathcal{N}(0, r)$, and $z_t = x_t + v_t$, $v_t \sim \mathcal{N}(0, q)$. Parameterize by $\theta = (\log r, \log q)$: the logs are not cosmetic, they enforce positivity through the parameterization, so gradient descent can never propose a negative variance.

Step 1 — the filter, as a composed map. Five equations, all smooth in θ :

$$\bar{\mu}_t = a \mu_{t-1} + b u_t, \quad \bar{\sigma}_t^2 = a^2 \sigma_{t-1}^2 + r, \quad S_t = \bar{\sigma}_t^2 + q, \quad K_t = \frac{\bar{\sigma}_t^2}{S_t},$$

$$\mu_t = \bar{\mu}_t + K_t v_t, \quad \sigma_t^2 = (1 - K_t) \bar{\sigma}_t^2.$$

Step 2 — differentiate the prediction. Writing ∂ for $\partial / \partial \theta_p$ and using $\partial r / \partial \log r = r$:

$$\partial \bar{\mu}_t = a \partial \mu_{t-1}, \quad \partial \bar{\sigma}_t^2 = a^2 \partial \sigma_{t-1}^2 + r [\theta_p = \log r].$$

Step 3 — differentiate the innovation and the gain. $\partial v_t = -\partial \bar{\mu}_t$ and $\partial S_t = \partial \bar{\sigma}_t^2 + q [\theta_p = \log q]$, hence by the quotient rule

$$\partial K_t = \frac{\partial \bar{\sigma}_t^2 - K_t \partial S_t}{S_t}.$$

This is the scalar instance of the matrix identity $dS^{-1} = -S^{-1}(dS)S^{-1}$, which is where the gradient of any Kalman gain comes from.

Step 4 — differentiate the update, and notice the recursion.

$$\partial \mu_t = \partial \bar{\mu}_t + v_t \partial K_t + K_t \partial v_t, \quad \partial \sigma_t^2 = (1 - K_t) \partial \bar{\sigma}_t^2 - \bar{\sigma}_t^2 \partial K_t.$$

The right-hand sides depend on $\partial \mu_{t-1}, \partial \sigma_{t-1}^2$: the sensitivities are carried forward alongside the state, one extra pair of numbers per parameter. With two parameters, this *forward-mode* recursion is cheaper than reverse-mode backpropagation through time and gives bit-identical gradients; with a network in the loop, reverse mode wins and an autodiff framework does exactly this bookkeeping for you.

Step 5 — differentiate the loss. For the evidence loss of F4, each term contributes

$$\partial \mathcal{L}_t = \frac{1}{2} \left[\left(1 - \frac{v_t^2}{S_t} \right) \frac{\partial S_t}{S_t} + \frac{2v_t \partial v_t}{S_t} \right].$$

Read the first bracket: when the squared innovation exceeds the claimed innovation variance ($v_t^2 > S_t$, the filter was surprised), the coefficient is negative and descent *increases* S_t . When the filter is consistently unsurprised, it shrinks. Gradient descent on the evidence is a feedback controller whose set point is $\mathbb{E}[v_t^2] = S_t$ — the NIS consistency criterion of Chapter 11, arrived at from an entirely different direction. ■

25.3.8 A worked example you can check by hand

Take the shortest possible trajectory: one step, $a = 1$, $b = 0$, a known initial state ($\mu_0 = 0$, $\sigma_0 = 0$), and unit noise parameters $r = q = 1$, so $\theta = (\log r, \log q) = (0, 0)$. The robot reads $z_1 = 2$.

Run the filter:

$$\bar{\mu}_1 = 0, \quad \bar{\sigma}_1^2 = 0 + r = 1, \quad v_1 = 2 - 0 = 2, \quad S_1 = 1 + 1 = 2, \quad K_1 = \frac{1}{2}.$$

The evidence loss is one Gaussian term:

$$\mathcal{L} = \frac{1}{2} \left[\log(2\pi S_1) + \frac{v_1^2}{S_1} \right] = \frac{1}{2} [\log 4\pi + 2] = 2.265512 \text{ nats}.$$

Now the gradients. Since $\sigma_0 = 0$, the prediction $\bar{\mu}_1$ does not depend on θ at all, so $\partial v_1 = 0$ and only the first bracket of F5 Step 5 survives. With $\partial S_1 / \partial \log r = r = 1$ and $\partial S_1 / \partial \log q = q = 1$:

$$\frac{\partial \mathcal{L}}{\partial \log r} = \frac{1}{2} \left(1 - \frac{4}{2} \right) \cdot \frac{1}{2} = -\frac{1}{4}, \quad \frac{\partial \mathcal{L}}{\partial \log q} = -\frac{1}{4}.$$

Both gradients are negative, so descent grows both variances: the filter was surprised — a $v = 2$ innovation against a claimed $S = 2$ is a 1.41σ event whose square is twice the claimed variance — and the honest response is to admit more uncertainty.

And the two gradients are *equal*, which is the lesson hiding in the arithmetic. After one step from a known initial state, the loss depends on θ only through the sum $S_1 = r + q$: no single measurement can tell process noise from measurement noise. What separates them is **time**. Process noise accumulates across steps and measurement noise does not, so the autocorrelation of the innovation sequence carries the information that identifies the split — which is why the trainer below needs a few hundred steps of trajectory, not a few.

ALGORITHM `train_dkf(traj_log, loss, lr, epochs)` COST $O(T(n^3 + m^3))$ per epoch
 per trajectory; $O(T)$ for the scalar filter
in a logged trajectory (`u_{1:T}`, `z_{1:T}`), a loss $\in \{\text{evidence, state-NLL}\}$, learning rate, epochs
out $\theta = (\log r, \log q)$

```

1: initialize  $\theta$ 
2: for epoch = 1 to epochs do
3:    $\mathcal{L} = 0$ ,  $\partial\mu = 0$ ,  $\partial\sigma^2 = 0$ ,  $g = 0$ 
4:   for  $t = 1$  to  $T$  do
5:     run one filter step, propagating  $(\partial\mu, \partial\sigma^2)$  by F5
6:      $\mathcal{L} += \frac{1}{2} [\log 2\pi S_t + v_t^2/S_t]$  // evidence: no ground truth
7:      $g += \partial\mathcal{L}_t$  // F5, Step 5
8:   endfor
9:    $\theta \leftarrow \theta - \text{lr} \cdot g$ 
10: endfor
11: return  $\theta$ 
```

INTERACTIVE · w25.2

The Differentiable Filter Trainer

Tuning a filter and training a network are the same act: gradient descent on a proper scoring rule, straight through the five Kalman equations.

Run this simulation in the web edition: [/chapters/ch25-learning #w25.2](/chapters/ch25-learning#w25.2)

Two things in that widget deserve more than a glance. The trainer starts with a filter that believes the cart is wildly unpredictable and the sensor is nearly perfect — the classic tuning mistake, in both directions at once — and the purple band is far too tight for it. As the loss falls, the band breathes out to a width that *contains the truth about 95% of the time*, and the mean NEES walks to

1. Nobody told it what 1 was. The evidence loss never saw the gray dashed line.

And the "cheat" toggle is there to make the alternative concrete. With ground

truth you can minimize the state NLL directly, which is what a differentiable filter trained in simulation usually does. It converges faster and to a slightly different place, and it is unavailable on any robot that is not in a motion-capture room. When someone reports a learned filter that was trained on state supervision and deployed without it, this toggle is the question to ask them.

25.3.9 Where the gradient dies

Everything above differentiates cleanly because the Kalman filter is a chain of smooth functions. Particle filters are not: they contain a categorical draw, and a categorical draw has no useful derivative.

DERIVATION F6 — Soft resampling carries gradient, and λ is exactly what it costs

$$q(i) = \lambda w_i + (1 - \lambda)/M, w'_i = w_i/q(i)$$

Step 1 — why resampling has no gradient. Multinomial or low-variance resampling maps weights to a multiset of surviving indices, then sets every surviving weight to $1/M$. Nudge one weight by ϵ and, almost surely, the same indices survive with the same new weights: the map is piecewise constant, so its derivative is zero almost everywhere and undefined on a measure-zero set of ties. A gradient travelling backwards from the loss reaches the first resample node and stops.

Step 2 — why the reparameterization trick does not save you. For a Gaussian you can write $x = \mu + \sigma\epsilon$ and push the gradient through the sample. A categorical distribution has no such smooth reparameterization: the outcome is an index, and indices do not interpolate.

Step 3 — move the dependence into the weight instead. Draw indices from a *mixture* of the weights and the uniform,

$$q(i) = \lambda w^{[i]} + \frac{1 - \lambda}{M},$$

and correct with the importance ratio $w'^{[i]} = w^{[i]}/q(i)$. Unbiasedness is one line: for any test function f ,

$$\mathbb{E}_{i \sim q} [w'^{[i]} f(x^{[i]})] = \sum_i q(i) \frac{w^{[i]}}{q(i)} f(x^{[i]}) = \sum_i w^{[i]} f(x^{[i]}),$$

which is the pre-resampling estimator, exactly, for every $\lambda \in (0, 1]$.

Step 4 — measure the gradient that survives. Differentiating the log of w' with respect to the log of w ,

$$\frac{\partial \log w'^{[i]}}{\partial \log w^{[i]}} = 1 - \frac{\lambda w^{[i]}}{q(i)} = \frac{(1 - \lambda)/M}{q(i)},$$

and averaging under the proposal that actually draws the indices, the $q(i)$ cancels:

$$\mathbb{E}_{i \sim q} \left[\frac{\partial \log w'^{[i]}}{\partial \log w^{[i]}} \right] = \sum_i q(i) \cdot \frac{(1 - \lambda)/M}{q(i)} = 1 - \lambda.$$

So λ is not merely *related* to how much gradient crosses a resample node — in expectation it **is** the fraction that dies, whatever the weights are. At $\lambda = 1$ (the classical resampler) nothing survives; at $\lambda = 0$ everything does, and the "resampler" has become plain importance sampling that never concentrates its particles at all. A filter unrolled over T resamples transmits $(1 - \lambda)^T$: gradients from the distant past are gone regardless, which is one honest reason differentiable particle filters are trained on short windows. ■

ALGORITHM `soft_resample(X_t, λ, rng)` **COST** $O(M)$ -- the low-variance comb of Table 4.4, stepped through q instead of w
in a weighted particle set, mixture weight $\lambda \in [0,1]$, a seeded RNG
out M particles with importance weights w' that depend smoothly on w

```

1: normalize  $w^{[i]}$  so that  $\sum_i w^{[i]} = 1$ 
2:  $q(i) = \lambda w^{[i]} + (1 - \lambda)/M$  for all  $i$ 
3:  $r \sim \mathcal{U}(0; M^{-1})$ ,  $c = q(1)$ ,  $i = 1$ 
4: for  $m = 1$  to  $M$  do
5:    $U = r + (m - 1) \cdot M^{-1}$ 
6:   while  $U > c$  do  $i = i + 1$ ;  $c = c + q(i)$  endwhile
7:   add  $x^{[i]}$  to  $\bar{X}_t$  with weight  $w'^{[i]} = w^{[i]}/q(i)$ 
8: endfor
9: return  $\bar{X}_t$ 
```

INTERACTIVE · w25.4

The Resampling Gradient Microscope

You cannot differentiate through a categorical draw — but you can move the weight dependence into the importance weight, and λ buys exactly $1 - \lambda$ of the gradient back.

Run this simulation in the web edition: [/chapters/ch25-learning #w25.4](#)

25.3.10 Learning to act: distributions over actions

The chapter’s thesis has a control-side sibling. Chapter 23 already samples action sequences and weights them; a **diffusion policy** learns the sampler.

💡 D25.6 — DIFFUSION POLICY

An action model $\pi_\theta(u_{t:t+H} \mid o_t)$ represented as a learned reverse-diffusion process. Forward noising is fixed, $q(u^k \mid u^{k-1}) = \mathcal{N}(\sqrt{1 - \beta_k} u^{k-1}, \beta_k I)$; training minimizes the denoising objective

$$\mathcal{L}(\theta) = \mathbb{E}_{k, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(u^k, k, o_t) \right\|^2 \right],$$

a bound on the negative log-likelihood of the demonstrated actions. Sampling an action sequence means iteratively denoising Gaussian noise conditioned on the observation.

The argument for it is the argument for particle filters, transposed. Suppose an obstacle can be passed on the left or on the right, and the demonstrations show both. A policy trained to regress the mean action drives straight into the obstacle — the average of two good plans is not a plan. Learning a *sampler* preserves the two modes, and the robot commits to one of them.

That is genuinely in the spirit of this book: represent what you cannot summarize. But note the asymmetry, because it is the reason this section is short and has no widget. A diffusion policy is a sampler, not a density: you can draw actions from it, but you cannot cheaply evaluate the probability of an action, so it does not compose with Bayes’ rule the way a calibrated sensor model does. Everything else in this chapter can be multiplied into a belief. This cannot. The full derivation of the diffusion ELBO is out of scope here, and the honest reference is [Chi et al.](#), whose IJRR version is the standard treatment.

25.4 Implementation in Rust

Three modules carry the chapter: the instrument, the learned model, and the trainer.

25.4.1 The instrument

Calibration is model-agnostic on purpose. Grading consumes PIT values and log scores, never a model, so the same code grades a hand-tuned mixture, a network, and a Kalman filter’s innovation sequence.

RUST crates/ch25_learning/src/calib.rs

```

1 use rand::Rng;
2
3 /// Anything that can serve the Bayes filter as  $p(z \mid x, m)$ .
4 ///
5 /// Chapter 10's `BeamModel` and `LikelihoodField` implement it, and so does
6 /// this chapter's `LearnedBeamModel`. The second and third methods are what
7 /// make a model *auditable*: a density you cannot integrate is a density you
8 /// cannot grade, and an atom you do not declare will masquerade as a defect.
9 pub trait ObservationModel {
10     ///  $\log p(z \mid x, m)$  for a whole scan -- what the filter multiplies in.
11     fn log_likelihood(&self, z: &Scan, x: &Se2, m: &OccGrid) -> f64;
12
13     /// Per-beam predictive CDF  $F(z_k \mid z^*)$ , needed for the PIT.
14     fn beam_cdf(&self, z_k: f64, z_star: f64) -> f64;
15
16     /// Mass of the atom at `z_k`, if the model has one. The beam mixture's
17     /// max-range spike is an atom; a likelihood field has none.
18     fn beam_atom(&self, _z_k: f64) -> f64 {
19         0.0
20     }
21 }
22
23 pub const N_BINS: usize = 15;
24
25 #[derive(Debug, Clone, Copy)]
26 pub struct ReliabilityBin {
27     pub nominal: f64,
28     pub empirical: f64,
29     pub count: usize,
30 }
31
32 #[derive(Debug, Clone)]
33 pub struct CalibrationReport {
34     pub bins: [ReliabilityBin; N_BINS],
35     /// D25.3, in the quantile-binned form: mean |observed - claimed|.
36     pub ece: f64,
37     /// Largest gap from the diagonal -- the Kolmogorov statistic.
38     pub max_calibration_error: f64,
39     /// Mean negative log score, nats per beam. Accuracy *and* sharpness.
40     pub mean_nll: f64,
41     /// Pearson chi2 of the PIT histogram against uniform, N_BINS - 1 d.o.f.
42     pub pit_chi2: f64,
43 }
44
45 /// Randomized PIT: spread each atom uniformly across the jump it makes, so a
46 /// dropout beam does not pile up at  $v = 1$  and look like miscalibration.
47 pub fn randomized_pit<M: ObservationModel>(
48     model: &M,
49     z_k: f64,
50     z_star: f64,
51     rng: &mut impl Rng,
52 ) -> f64 {
53     let upper = model.beam_cdf(z_k, z_star);
54     let lower = upper - model.beam_atom(z_k);
55     (lower + rng.random:::<f64>()) * (upper - lower).clamp(0.0, 1.0)
56 }
57
58 pub fn calibrate<M: ObservationModel>(
59     model: &M,

```

```

66     eval: &[(f64, f64)], // (z*, z) pairs from a held-out set
67     log_density: impl Fn(f64, f64) -> f64,
68     rng: &mut impl Rng,
69 ) -> CalibrationReport {
70     let mut pit: Vec<f64> = eval
71         .iter()
72         .map(|&(z_star, z)| randomized_pit(model, z, z_star, rng))
73         .collect();
74     let mean_nll = -eval
75         .iter()
76         .map(|&(z_star, z)| log_density(z, z_star))
77         .sum::<f64>()
78         / eval.len() as f64;
79
80     // Sorting turns every empirical-CDF query into an O(1) index lookup.
81     pit.sort_by(|a, b| a.partial_cmp(b).unwrap());
82     let n = pit.len() as f64;
83
84     let mut bins = [ReliabilityBin { nominal: 0.0, empirical: 0.0, count: 0 }; N_BINS];
85     let (mut ece, mut worst) = (0.0, 0.0_f64);
86     for b in 0..N_BINS {
87         let nominal = (b + 1) as f64 / N_BINS as f64;
88         let count = pit.partition_point(|&v| v <= nominal);
89         let empirical = count as f64 / n;
90         let gap = (empirical - nominal).abs();
91         ece += gap / N_BINS as f64;
92         worst = worst.max(gap);
93         bins[b] = ReliabilityBin { nominal, empirical, count };
94     }
95
96     // chi2 of the histogram (not the CDF) against a uniform expectation.
97     let expected = n / N_BINS as f64;
98     let mut counts = [0usize; N_BINS];
99     for &v in &pit {
100         counts[((v * N_BINS as f64) as usize).min(N_BINS - 1)] += 1;
101     }
102     let pit_chi2 = counts
103         .iter()
104         .map(|&c| (c as f64 - expected).powi(2) / expected)
105         .sum();
106
107     CalibrationReport { bins, ece, max_calibration_error: worst, mean_nll, pit_chi2 }
108 }

```

25.4.2 The learned model

The hand-tuned beam model claims one σ_{hit} for every range, which is false on every real LiDAR: a beam returning from 7 m is not the beam returning from 70 cm. The learned model makes the mixture parameters *functions of the expected range* — a two-layer MLP that emits four mixture logits and a log-width, trained by the NLL that F1 licensed.

The design decision worth recording: training happens natively with candle; the browser never trains a net. Inference is a dependency-free forward pass — two matrix multiplies and a GELU — so the same weights run in the WASM

widget with no framework attached.

RUST crates/ch25_learning/src/learned_beam.rs

```

1 use candle_core::{DType, Device, Result, Tensor};
2 use candle_nn::{loss, AdamW, Linear, Module, Optimizer, ParamsAdamW, VarBuilder, VarMap};
3
4 // Learned per-beam range density: MLP(z*) -> (mixture logits, log sigma_hit).
5 //
6 // The *shape* of the mixture is kept from Chapter 10 -- hit, short, max, rand --
7 // because that structure is physics and the network has nothing better to
8 // offer. What is learned is how the four weights and the hit width vary with
9 // range, which is exactly the part the hand-tuned model got wrong.
10 pub struct BeamNet {
11     l1: Linear,
12     l2: Linear,
13     max_range: f64,
14 }
15
16 impl BeamNet {
17     pub fn new(vb: VarBuilder, hidden: usize) -> Result<Self> {
18         Ok(Self {
19             l1: candle_nn::linear(2, hidden, vb.pp("l1"))?,
20             l2: candle_nn::linear(hidden, 5, vb.pp("l2"))?,
21             max_range: 8.0,
22         })
23     }
24
25     // Features are deliberately dimensionless: a network trained on metres and
26     // deployed on a sensor with a different range would otherwise extrapolate.
27     fn features(&self, z_star: &Tensor) -> Result<Tensor> {
28         let u = (z_star / self.max_range)?;
29         Tensor::cat(&[u, &u.sqrt()], 1)
30     }
31
32     // Negative log-likelihood of a batch under the predicted mixture. This is
33     // the strictly proper log score of D25.1 -- nothing else is trained.
34     pub fn nll(&self, z_star: &Tensor, z: &Tensor) -> Result<Tensor> {
35         let out = self.l2.forward(&self.l1.forward(&self.features(z_star))?.gelu())?;
36         let logits = out.narrow(1, 0, 4)?;
37         let log_w = candle_nn::ops::log_softmax(&logits, 1)?; // weights on the simplex
38         let sigma = (out.narrow(1, 4, 1)?.exp()? + 1e-3)?; // positivity by construction
39
40         // log of each component density, then log-sum-exp against the weights.
41         let comps = Tensor::cat(
42             &[
43                 self.log_hit(z, z_star, &sigma)?,
44                 self.log_short(z, z_star)?,
45                 self.log_max(z)?,
46                 self.log_rand(z)?,
47             ],
48             1,
49         )?;
50         let log_mix = (comps + log_w)?.log_sum_exp(1)?;
51         log_mix.neg()?.mean_all()
52     }
53 }
54
55 // One epoch of `learn_observation_model`, lines 8-9.
56 pub fn train(pairs: &[(f64, f64)], epochs: usize, dev: &Device) -> Result<VarMap> {
57     let varmap = VarMap::new();

```

```

58     let vb = VarBuilder::from_varmap(&varmap, DType::F64, dev);
59     let net = BeamNet::new(vb, 64)?;
60     let mut opt = AdamW::new(varmap.all_vars(), ParamsAdamW { lr: 1e-3, ..Default::
default() })?;
61
62     let z_star = Tensor::from_iter(pairs.iter().map(|p| p.0), dev)?.reshape((pairs.len(),
1))?;
63     let z = Tensor::from_iter(pairs.iter().map(|p| p.1), dev)?.reshape((pairs.len(), 1))
?;
64
65     for _ in 0..epochs {
66         let loss = net.nll(&z_star, &z)?;
67         opt.backward_step(&loss)?;
68     }
69     Ok(varmap)
70 }

```

25.4.3 The trainer

DiffKf1d is the whole of F5, transcribed. It is about forty lines, it has no dependencies, and it is the engine the trainer widget runs in the browser — the derivation *is* the source code.

RUST crates/ch25_learning/src/diff_kf.rs

```

1  /// A scalar Kalman filter with  $\theta = (\log r, \log q)$  exposed and differentiated.
2  ///
3  /// Logs, not variances, so that descent cannot propose a negative variance:
4  /// the constraint lives in the parameterization rather than in a clamp.
5  pub struct DiffKf1d {
6      pub log_r: f64,
7      pub log_q: f64,
8  }
9
10 pub enum DkfLoss {
11     /// F4. Scored on one-step-ahead predictions of measurements the robot took.
12     Evidence,
13     /// The supervised alternative: available in simulation and motion capture,
14     /// and nowhere else.
15     StateNll,
16 }
17
18 impl DiffKf1d {
19     /// Loss and its exact gradient in a single forward pass (F5).
20     pub fn loss_and_grad(&self, log: &TrajLog1d, loss: DkfLoss) -> (f64, f64, f64) {
21         let (r, q) = (self.log_r.exp(), self.log_q.exp());
22         let (a, b) = (log.a, log.b);
23
24         let (mut mu, mut sigma2) = (log.mu0, log.sigma0 * log.sigma0);
25         // Sensitivities  $\partial(\mu, \sigma^2)/\partial\theta$ . The initial belief is given, not learned.
26         let (mut d_mu, mut d_sig) = ([0.0_f64; 2], [0.0_f64; 2]);
27         let (mut total, mut grad) = (0.0, [0.0_f64; 2]);
28
29         for t in 0..log.z.len() {
30             let u = log.u.get(t).copied().unwrap_or(0.0);

```

```

32     let mu_bar = a * mu + b * u;
33     let sigma2_bar = a * a * sigma2 + r;
34     let d_mu_bar = [a * d_mu[0], a * d_mu[1]];
35     // ?r/?log r = r, ?r/?log q = 0.
36     let d_sig_bar = [a * a * d_sig[0] + r, a * a * d_sig[1]];
37
38     let nu = log.z[t] - mu_bar;
39     let s = sigma2_bar + q;
40     let d_nu = [-d_mu_bar[0], -d_mu_bar[1]];
41     let d_s = [d_sig_bar[0], d_sig_bar[1] + q];
42
43     let k = sigma2_bar / s;
44     let d_k = [
45         (d_sig_bar[0] - k * d_s[0]) / s,
46         (d_sig_bar[1] - k * d_s[1]) / s,
47     ];
48
49     let mu_new = mu_bar + k * nu;
50     let sigma2_new = (1.0 - k) * sigma2_bar;
51
52     match loss {
53         DkfLoss::Evidence => {
54             total += 0.5 * ((core::f64::consts::TAU * s).ln() + nu * nu / s);
55             for p in 0..2 {
56                 // Negative when nu2 > S: descent widens a surprised filter.
57                 grad[p] += 0.5 * ((1.0 - nu * nu / s) * (d_s[p] / s) + 2.0 * nu *
d_nu[p] / s);
58             }
59         }
60         DkfLoss::StateNll => {
61             let e = log.x[t] - mu_new;
62             total += 0.5 * ((core::f64::consts::TAU * sigma2_new).ln() + e * e /
sigma2_new);
63             for p in 0..2 {
64                 let d_mu_new = d_mu_bar[p] + d_k[p] * nu + k * d_nu[p];
65                 let d_sig_new = (1.0 - k) * d_sig_bar[p] - d_k[p] * sigma2_bar;
66                 grad[p] += 0.5
67                     * ((1.0 - e * e / sigma2_new) * (d_sig_new / sigma2_new)
68                       - 2.0 * e * d_mu_new / sigma2_new);
69             }
70         }
71     }
72
73     d_mu = [
74         d_mu_bar[0] + d_k[0] * nu + k * d_nu[0],
75         d_mu_bar[1] + d_k[1] * nu + k * d_nu[1],
76     ];
77     d_sig = [
78         (1.0 - k) * d_sig_bar[0] - d_k[0] * sigma2_bar,
79         (1.0 - k) * d_sig_bar[1] - d_k[1] * sigma2_bar,
80     ];
81     mu = mu_new;
82     sigma2 = sigma2_new;
83 }
84
85 (total, grad[0], grad[1])
86 }
87 }

```

Soft resampling is the same low-variance comb as Thrun’s Table 4.4, walked through q instead of w , with the importance ratio attached to each survivor:

RUST crates/ch25_learning/src/soft_resample.rs

```

1  use rand::Rng;
2
3  /// `soft_resample(X_t, lambda, rng)` -- F6. Additive: `ParticleFilter` opts in by
4  /// calling this instead of `low_variance_sampler`, and nothing else changes.
5  pub fn soft_resample<S: Clone>(
6      particles: &mut Vec<Particle<S>>,
7      lambda: f64,
8      rng: &mut impl Rng,
9  ) -> f64 {
10     let m = particles.len();
11     let total: f64 = particles.iter().map(|p| p.weight).sum();
12     let w: Vec<f64> = particles.iter().map(|p| p.weight / total).collect();
13     let lambda = lambda.clamp(0.0, 1.0);
14     let q: Vec<f64> = w.iter().map(|wi| lambda * wi + (1.0 - lambda) / m as f64).collect
15     ();
16
17     let step = 1.0 / m as f64;
18     let start = rng.random_range(0.0..step);
19     let (mut out, mut acc, mut i, mut transmission) = (Vec::with_capacity(m), q[0], 0
20     usize, 0.0);
21
22     for k in 0..m {
23         let u = start + k as f64 * step;
24         while u > acc && i < m - 1 {
25             i += 1;
26             acc += q[i];
27         }
28         out.push(Particle { state: particles[i].state.clone(), weight: w[i] / q[i] });
29         // ? log w' / ? log w? -- the fraction of the gradient this survivor passes.
30         transmission += ((1.0 - lambda) / m as f64) / q[i];
31     }
32
33     *particles = out;
34     transmission / m as f64 // ~= 1 - lambda, exactly, in expectation
35 }

```

25.4.4 The tests that pin it

RUST crates/ch25_learning/src/diff_kf.rs (tests)

```

1  #[test]
2  fn worked_example_ch25_one_step_evidence() {
3      // One step, r = q = 1, sigma_0 = 0, z_1 = 2 -- the example derived in the text.
4      let log = TrajLog1d { a: 1.0, b: 0.0, u: vec![0.0], z: vec![2.0], x: vec![0.0],
5                          mu0: 0.0, sigma0: 0.0 };
6      let kf = DiffKf1d { log_r: 0.0, log_q: 0.0 };
7
8      let (loss, d_log_r, d_log_q) = kf.loss_and_grad(&log, DkfLoss::Evidence);
9
10     // L = ?(ln 4pi + 2) = 2.265512...
11     assert_relative_eq!(loss, 0.5 * ((4.0 * PI).ln() + 2.0), epsilon = 1e-12);

```



```

12 // Equal and negative: the filter was surprised, and one measurement cannot
13 // separate process noise from measurement noise.
14 assert_relative_eq!(d_log_r, -0.25, epsilon = 1e-12);
15 assert_relative_eq!(d_log_q, -0.25, epsilon = 1e-12);
16 }
17
18 #[test]
19 fn analytic_gradient_matches_central_differences() {
20     let log = simulate_traj_1d(120, 0.96, 0.1, 0.05, 0.4, &mut SmallRng::seed_from_u64
21     (2025));
22     for loss in [DkfLoss::Evidence, DkfLoss::StateNll] {
23         for theta in [(-1.5, 0.4), (0.3, -0.8), (-3.0, -3.0)] {
24             let kf = DiffKf1d { log_r: theta.0, log_q: theta.1 };
25             let (_, g_r, g_q) = kf.loss_and_grad(&log, loss);
26             let (fd_r, fd_q) = finite_diff_grad(&kf, &log, loss, 1e-5);
27             assert_relative_eq!(g_r, fd_r, max_relative = 1e-5);
28             assert_relative_eq!(g_q, fd_q, max_relative = 1e-5);
29         }
30     }
31
32     #[test]
33     fn evidence_training_recovers_the_noises_without_ground_truth() {
34         // 1500 steps generated with  $r = 0.08$ ,  $q = 0.25$ ; start deliberately wrong
35         // in both directions and never look at log.x.
36         let log = simulate_traj_1d(1500, 1.0, 0.05, 0.08, 0.25, &mut SmallRng::seed_from_u64
37         (1234));
38         let mut kf = DiffKf1d { log_r: 0.4_f64.ln(), log_q: 0.02_f64.ln() };
39         for _ in 0..12_000 {
40             kf.sgd_step(&log, DkfLoss::Evidence, 0.05 / log.z.len() as f64);
41         }
42         assert!((kf.log_r.exp() / 0.08).ln().abs() < 0.2); //  $r \rightarrow 0.085$ 
43         assert!((kf.log_q.exp() / 0.25).ln().abs() < 0.1); //  $q \rightarrow 0.250$ 
44     }
45 }

```

NOTE

All three tests have twins in the TypeScript port that drives the widgets on this page (`lib/learn/__checks_ch25__.ts`, eleven invariants, all passing). The last one is the chapter's real claim in executable form: starting from r five times too large and q twelve times too small, and using nothing but the measurements, the trainer lands on $r = 0.0848$ against a true 0.08 and $q = 0.2497$ against a true 0.25.

25.5 Putting it together: the clinic benchmark

The chapter's empirical thesis needs a table, and the table is generated by cargo run --example clinic — the same computation the Calibration Clinic runs live in your browser, so every number here is one you can reproduce by clicking a button in the widget above.

The setup: the true sensor is a beam mixture with $\sigma_{hit} = 0.11$ m and a real

unexpected-obstacle component ($z_{short} = 0.08$). The model family available to the engineer is mis-specified — it has no z_{short} at all. Held-out evaluation is 1200 beams from random poses in the Apartment; downstream is one importance-weighting step over twelve 150-particle clouds, each displaced about 42 cm from the truth.

model	claimed σ_{hit}	held-out NLL	ECE	ESS after one update	posterior error
hand-tuned (the Tuesday afternoon)	0.050 m	0.937	0.1074	0.99%	0.292 m
EM, learn_intrinsic_parameters	0.112 m	0.290	0.0286	1.20%	0.170 m
best held-out NLL, width only	0.113 m	0.352	0.0248	1.20%	0.172 m
+ fitted temperature (held-out NLL)	0.115 m	0.352	0.0226	1.23%	0.171 m
+ temperature fitted for calibration	0.142 m	0.381	0.0143	1.62%	0.162 m
the true sensor (unavailable in practice)	0.110 m	0.223	0.0209	1.18%	0.157 m

Read the last three rows against each other, because that is the entire chapter in six numbers.

The best likelihood is not the best filter. EM wins the NLL column decisively — it is the only model here that fits the mixture weights as well as the width — and it still loses the posterior-error column to a model with a *worse* NLL by 0.09 nats. The widened, best-calibrated model gives up score in exchange for honesty, and honesty is what the filter downstream converts into accuracy: a 60% larger effective sample size and 5% less error, from a model that any leaderboard would rank below the one it beats.

Overconfidence is expensive and calibration is cheap. The hand-tuned model is not subtly worse. It is 4.5× the ECE of anything fitted, and it leaves the posterior 0.292 m from the truth when the prior was already only 0.424 m away — it converts a scan into a third of the correction it should have bought. Fixing it required no retraining and no new data: one scalar, fitted on 800 held-out beams, in about fifty function evaluations.

And the honest model still wins on score. The true sensor’s NLL of 0.223 is far below everything else, and no amount of temperature scaling closes that gap, because tempering cannot invent the z_{short} component the family is missing. Calibration is a repair, not a substitute for getting the structure right. That is worth saying loudly in a chapter about learning: the largest single improvement available in this table comes from *the model family*, not from the fitting procedure.

25.5.1 The honesty ledger

The chapter closes where it should — with a line drawn between what is principled and what is alchemy, as of 2026.

Principled. A learned component that emits a *calibrated density* and is dropped into an unchanged Bayes filter: this is just a better $p(z_t | x_t, m)$, and every theorem in Parts II through V still holds. Training noise parameters by maximizing evidence, which is prediction-error identification with a fifty-year pedigree and needs no ground truth. Post-hoc calibration on held-out data, which has one parameter and cannot lie to you in any interesting way. Differentiable filters that keep the filter’s structure as an algorithmic prior and learn only its contents.

Still alchemy. End-to-end “filters” whose belief is a vector of activations that nobody has graded — if you cannot draw its reliability diagram, do not call it a posterior. Learned resampling heuristics published without a variance analysis. Sim-to-real transfer of learned sensor models: our own learned model was trained in the simulator that also evaluates it, which is exactly the circularity flagged after `learn_observation_model`, and no reliability diagram computed inside the simulator says anything about the building. And diffusion policies’ uncertainty: they are samplers, not densities, so they do not compose with Bayes’ rule, and a policy that produces diverse actions is not thereby a policy that knows what it does not know.

A useful sanity check before adopting any of this: Greenberg, Yannay and Mannor (NeurIPS 2023) showed that a Kalman filter *optimized as carefully as the network it is compared against* closes much of the reported gap in the differentiable-filtering literature. The technique in this chapter — train the classical filter’s parameters properly — is also the baseline that most learned filters should have been compared against.

Chapter 26 puts this to work: its stack exposes a calibrated learned sensor model as a toggle, and its retrospective quotes this table.

25.6 Exercises

1 FOUNDATION •• Propriety, and a score you can game

Prove D25.1 for the log score without looking at derivation F1 — expected score equals entropy plus KL, and Gibbs’ inequality does the rest. Then construct a scoring rule that a beam model could game: find S and two densities $p \neq q$ with $\mathbb{E}_{z \sim p}[S(q, z)] < \mathbb{E}_{z \sim p}[S(p, z)]$, and say in one sentence what a model trained on it would do to σ_{hit} . (Hint: reward the density at the observation’s *mode* rather than at the observation.)

2 FOUNDATION ••• Differentiate the filter yourself

Derive $\partial \mathcal{L} / \partial \log q$ for the scalar Kalman filter under the evidence loss, carrying the sensitivity recursion of F5 over two steps rather than one. Verify your expression against `DiffKf1d::loss_and_grad` by central differences at $\theta = (-1.5, 0.4)$; the test

skeleton is `analytic_gradient_matches_central_differences` above. Then explain why the one-step worked example has $\partial \mathcal{L} / \partial \log r = \partial \mathcal{L} / \partial \log q$ and the two-step version does not.

3 FOUNDATION ••• The ESS lemma

Derive $\text{ESS}/M \approx e^{-\kappa^2 s^2}$ from the Gaussian moment generating function (F3, Steps 3–4), stating precisely where you used the assumption that the per-particle log-likelihoods are Gaussian. Then compute how large a spread s your particle filter can tolerate before $\kappa = 1$ leaves fewer than 5% of $M = 1000$ particles effective, and compare with the spread the Tempering Lab reports for a 16-beam likelihood-field scan.

4 CONCEPTUAL •• Predict, then temper

Before touching the Tempering Lab: predict what happens to the kidnap-recovery percentage at $\kappa = 3$, and predict whether $\kappa = 0.5$ helps or hurts *tracking* accuracy between kidnaps. Run both for at least three kidnap cycles each. Explain both outcomes using F3, and say which of the two effects — precision growth or ESS collapse — dominates the failure you saw.

5 CONCEPTUAL •• Find the two optima

In the Calibration Clinic, use the slider to locate the width that minimizes the mean NLL and the width that minimizes the ECE. They are different. Before reading the two right-hand tiles, predict which of those two models gives the lower posterior error — then check. Now explain the gap using the reliability diagram: which end of the PIT histogram is the NLL optimum trading away, and what is it buying with it?

6 PRACTICAL •• Grade the likelihood field

Implement `ObservationModel` and calibrate for Chapter 10's `LikelihoodField`. Its per-beam density is a Gaussian on the distance to the nearest occupied cell plus a uniform floor, so its CDF is available in closed form. Is it calibrated? Produce its PIT histogram over 1200 beams in the Apartment and identify where it bulges — then explain the bulge from the model's structure (hint: what does the likelihood field do with a max-range beam, and with a beam that hits an unmapped obstacle?).

7 PRACTICAL ••• Train through the resampler

Wire `soft_resample` into Chapter 8's `ParticleFilter` and train a single parameter — the sensor model's width — end-to-end through three steps of 1-D hallway MCL, using the state NLL of the filter mean as the loss. Sweep $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ and plot both the gradient magnitude reaching the parameter and the variance of the resulting

estimate. Does the learned width agree with the post-hoc temperature from `fit_temperature`? If it does not, say which loss each one is optimizing and which of the two you would ship.

25.7 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

The ancestor of this chapter, twice over: `learn_intrinsic_parameters` (Table 6.2) fits the beam model by EM, and the section on learning inverse measurement models trains a function approximator on triplets sampled from the forward model — self-supervised sim-to-model learning, in 1999. The epigraph is that section noticing its own frontier. [link](#)

Gneiting, T. and Raftery, A. E. (2007). Strictly Proper Scoring Rules, Prediction, and Estimation *Journal of the American Statistical Association* 102(477), 359–378.

The reference treatment of D25.1. Everything this chapter trains, it trains on a strictly proper score, and this paper is why that phrase has content. doi:10.1198/016214506000001437

Gneiting, T., Balabdaoui, F., and Raftery, A. E. (2007). Probabilistic Forecasts, Calibration and Sharpness *Journal of the Royal Statistical Society Series B* 69(2), 243–268.

The source of the principle this chapter is organized around — maximize sharpness subject to calibration — and of the PIT-histogram diagnostic that the Calibration Clinic plots. doi:10.1111/j.1467-9868.2007.00587.x

Abbeel, P., Coates, A., Montemerlo, M., Ng, A. Y., and Thrun, S. (2005). Discriminative Training of Kalman Filters *Robotics: Science and Systems I*.

The robotics ancestor of the Differentiable Filter Trainer: learn the noise covariances instead of hand-designing them, and beat a carefully hand-tuned filter on a real rover. doi:10.15607/RSS.2005.I.038

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks *ICML 2017*.

Established that accuracy and calibration diverge in modern networks, and that a single temperature fixes most of it. This chapter's $B = 15$ bins, its ECE definition, and its post-hoc scaling all come from here. [link](#)

Karkus, P., Hsu, D., and Lee, W. S. (2018). Particle Filter Networks with Application to Visual Localization *Conference on Robot Learning (CoRL) 2018*.

Section 3 introduces soft resampling — the mixture proposal and importance correction of derivation F6, with the trade-off parameter written α where this book writes λ . [link](#)

Chen, X. and Li, Y. (2024). An Overview of Differentiable Particle Filters for Data-Adaptive Sequential Bayesian Inference *Foundations of Data Science (AIMS)*; *arXiv:2302.09639*.

The current map of the differentiable-filtering field, from Jonschkowski et al.'s RSS 2018 differentiable particle filters onward: which components get learned, which resampling relaxations exist, and what each objective actually optimizes. [link](#)

Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. (2025). Diffusion Policy: Visuomotor Policy Learning via Action Diffusion *The International Journal of Robotics Research* 44(10–11), 1684–1704.

The extended version of the RSS 2023 paper, and the standard treatment of D25.6. Read it for the multimodality argument: a learned sampler keeps the two ways around an obstacle that a regressed mean averages into the obstacle. doi:10.1177/02783649241273668

Capstone: A Complete Autonomous Robot

FRONTIERS AND INTEGRATION · Advanced · 70 min



Sound mathematical theory, clear assumptions, therefore it's easier to predict failure modes.

Sebastian Thrun, Wolfram Burgard, and Dieter Fox

Probabilistic Robotics, Chapter 1 (1999–2000 draft)

Rusty is about to be dropped into an apartment it has never seen, with no map, no starting pose beyond the origin of its own coordinate frame, and ninety seconds of patience. It will come out the other side with a floorplan.

That is not a new algorithm. Every piece of it — the scan matcher, the log-odds map, the distance transform, the frontier scorer, the lattice planner, the sampling controller, the particle filter — you have already built. What you have not built is the thing that makes them one robot rather than seven demos, and that thing turns out to have mathematics of its own: interface contracts, staleness bounds, chance constraints, and detection statistics with thresholds someone has to defend.

The argument of this chapter is that an autonomy stack is not an architecture diagram. It is a *tower of stated approximations* to a single intractable problem, and each layer's assumption is precisely a failure mode waiting to happen. So we will break it on purpose: kidnap the robot, walk a person through the LiDAR, cut the sensor. Each time, watch a named statistic cross a named threshold and a mode with a plan take over. Recovery in a well-built system is never luck.

🕒 AFTER THIS CHAPTER YOU CAN

- Write the exploration mission as a POMDP, and see precisely why solving

it directly is hopeless

- Read an autonomy stack as a tower of named approximations, each with a failure mode you can predict from its assumption
- Derive the chance-constrained safety margin that lets a certainty-equivalent planner stay safe under pose uncertainty
- Derive the latency budget that decides whether a moving obstacle is handled or hit
- Build three failure detectors out of statistics this book already derived, and choose their thresholds defensibly
- Compose the whole thing in Rust as typed tasks on a bus, with a supervisor the compiler forces you to keep exhaustive

ASSUMED

- The whole book, honestly — but the load-bearing chapters are 12 (recovery), 13 (occupancy grids), 16 (scan matching), 19 (distance fields), 20 (planning), 23 (MPPI), and 24 (exploration)
- Chapter 22 for the belief-space vocabulary the mission POMDP is written in
- Rust traits, enums with data, and pattern matching — the supervisor is one big match

26.1 Ninety seconds

INTERACTIVE · w26.1

The Grand Demo

An autonomy stack is not a magic monolith: every internal is inspectable, and each recovery is a named statistic crossing a threshold.

Run this simulation in the web edition: `/chapters/ch26-capstone #w26.1`

It is running the real thing. There is no recorded trace, no scripted trajectory, no hidden ground truth feeding the estimator: what you are watching is a scan matcher registering sweeps against a map it is simultaneously building, a distance transform being recomputed five times a second, a frontier detector proposing goals, A* routing through cells that are mostly still unknown, and 56 sampled rollouts being reweighted twenty times a second. Every number in the side panels is read straight out of the running stack.

Three things are worth watching for before we start taking it apart.

The map is worse than the world, and everything downstream inherits that. Turn on the ground-truth overlay and the gray dashed walls will not sit exactly on the dark cells. The planner does not know this. It routes through the map as though the map were the world — an assumption with a name (certainty equivalence) and a price we will compute exactly.

The orange ring around Rusty breathes. That is the safety margin $r_{\text{robot}} + k_{\sigma} \sigma_{\text{pose}}$, and it grows whenever the pose belief loosens: during a sensor dropout, immediately after a mode switch, in a featureless stretch of corridor. A controller that ignored it would be brave in exactly the moments it should be timid.

Press Kidnap. The fitness ratio ρ on the left rail dives, the supervisor switches to Relocalize, six thousand particles scatter across the known-free space, and the cloud condenses over a few seconds of creeping. Nothing about that sequence is scripted. It is a threshold, a mode, and a behavior — and by the end of this chapter you will be able to derive every one of them.

💡 THE ONE SENTENCE VERSION

An autonomy stack is the mission POMDP under a stack of named approximations; each approximation buys tractability and sells you a specific failure, and the job of the supervisor is to hold a statistic against each sale.

26.2 Why not one big filter?

The obvious question, having built all these parts, is why they should not be one optimization. Write down what the robot actually wants and the answer becomes clear quickly.

The robot's belief is joint over its pose and the map, $b_t = p(x_t, m \mid z_{1:t}, u_{1:t})$. It wants a policy π mapping beliefs to controls that gathers map information as fast as possible without hitting anything:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_t \gamma^t r(b_t, u_t) \right], \quad r(b_t, u_t) = \underbrace{-\Delta H(m_t)}_{\text{information gained}} - \lambda c(u_t), \quad \text{s.t. } P(\text{collision}) \leq \delta$$

This is a POMDP — Chapter 22's object, at building scale — and it is hopeless three times over.

The **state space** is the pose crossed with the map: three continuous dimensions plus one binary variable per cell. The apartment at 15 cm resolution is $80 \times 60 = 4800$ cells, so the map alone lives in a space of size 2^{4800} . The **belief space** is the space of distributions over that, which is worse by an exponential. And the **horizon** is the whole mission: a thousand decisions, each of which changes what the robot will be able to see later, so the value function does not decompose over time.

Even the tractable-looking pieces are not. Exact information gain for one candidate viewpoint requires integrating over every measurement the robot might receive there, which requires ray-casting a distribution over maps. Point-based POMDP solvers get you to a few dozen states, not 2^{4800} .

So nobody solves it. What everyone does instead — SLAM Toolbox and Nav2, the Cartographer stack, every warehouse robot you have ever seen — is factor the problem into layers, approximate each layer independently, and then *engineer around* the approximations they made. This chapter is about doing that on purpose, with the approximations written down.

26.3 The stack, as measured dataflow

INTERACTIVE · w26.2

Stack Anatomy

Every arrow in this diagram carries measured traffic, and every box has a switch — architecture is causal, not decorative.

Run this simulation in the web edition: `/chapters/ch26-capstone #w26.2`

Eight tasks. Each publishes a typed message at its own rate and consumes the messages of its neighbors, and nothing else — no task reaches into another’s internals. On native Rust each runs on its own thread over a `crossbeam-channel`; in the browser a deterministic round-robin scheduler ticks whichever tasks are due. Same task code, same message types, same seed, same mission. The diagram above is not a drawing of that arrangement, it is an instrument attached to it: the hertz on each edge is the publisher’s own counter, and the switches really switch tasks off.

Play with the switches for a minute, because the failures are the argument. Turn off the **SLAM front end** and the pose belief degenerates to raw wheel odometry; the map does not explode, it *shears* — corridors drawn twice, at an angle, exactly the picture Chapter 16 opened with. Turn off the **frontier explorer** and something more unsettling happens: nothing. Every task is healthy, every rate is nominal, the robot sits still beside a room it has never entered. That is the failure mode of a layer whose job is to *want* something.

WHAT A MESSAGE IS

Every value crossing a task boundary is a ‘Stamped’: payload, timestamp, frame id. Estimator tasks publish *beliefs* — mean and covariance, or particles — never bare states. A pose without a covariance cannot be turned into a safety margin, and a pose without a frame is the

single most expensive bug in mobile robotics.

26.4 The mathematics

There are no new estimators in this chapter. What is new is precise statements about *composition*: what each layer assumes, what that assumption costs, and how you notice when it has been violated.

b_t	Mission belief: the joint posterior over pose and map, $p(x_t, m \mid z_{1:t}, u_{1:t})$.
τ_i	Period of task i , in seconds. The stack's rate table is {LiDAR 10, SLAM 10, map 10, ESDF 5, frontier 1, plan 1, control 20} Hz.
ς_i	Staleness of task i : the age of its most recent publication. Bounded by τ_i when the task is healthy.
$\rho_t = w_{fast}/w_{slow}$	Dual-EMA fitness ratio — Chapter 12's recovery statistic, applied to scan-match fitness instead of particle weight.
$\epsilon_t = v_t^T S_t^{-1} v_t$	Normalized innovation squared (NIS) of the SLAM front end; χ^2_3 under a correctly specified filter.
$H(m_t)$	Occupancy-map entropy in bits, $\sum_i H(p_i)$; its rate. The mission objective and half the stopping criterion.
$\delta, k_\sigma = \Phi^{-1}(1 - \delta)$	Collision-chance bound and the corresponding inflation gain on the pose standard deviation.

26.4.1 Four definitions

D26.1 — Autonomy stack. A set of tasks $\{T_i\}$ with periods τ_i , exchanging typed messages. Every message is a 'Stamped

: payload, timestamp, frame identifier. An estimator task publishes a *belief*, never a point.

D26.2 — Mission. The POMDP written above: maximize expected discounted map information subject to $P(\text{collision}) \leq \delta$.

D26.3 — Stopping criterion. Terminate when *both* hold: no frontier of area $\geq A_{\min}$ remains reachable, **and** $|\dot{H}(m_t)| < h_{\min}$ over a trailing window. Either condition alone is gameable — a single unreachable speck of unknown keeps a frontier count above zero forever, and a robot standing still has a perfectly flat entropy curve.

D26.4 — Mode. The supervisor's discrete state, an exhaustive enum by design:

$\text{Mode} \in \{ \text{Explore}, \text{Navigate}\{\text{goal}\}, \text{Relocalize}, \text{Recover}(\text{kind}), \text{Done} \}$

26.4.2 F1 — The layer tower

Statement. Each subsystem in the stack is the mission POMDP of D26.2 under exactly one named approximation, and each approximation induces exactly one characteristic failure.

Layer	Approximation of D26.2	What it buys	What it sells you
Exploration (Ch. 24 (Chapter 24))	one-step greedy on information gain, ignoring the future	a scored list instead of a tree search	oscillating targets; the robot ping-pongs between two equally good frontiers
Planning (Ch. 20 (Chapter 20))	certainty equivalence in the map: plan on the MAP map as if it were the world	a graph search instead of planning in belief space	routes straight through walls that have not been seen yet
Control (Ch. 23 (Chapter 23))	certainty equivalence in the pose: roll out from μ_t as if it were x_t	56 rollouts instead of a belief-space integral	confident driving while the filter is lost
SLAM (Ch. 16 (Chapter 16))	MAP point estimate of the map marginal of b_t	one map instead of a distribution over maps	a wrong loop closure is accepted with total confidence and is permanent
Mapping (Ch. 13 (Chapter 13))	cell independence given the pose	a += per cell instead of a joint posterior	thin structures dissolve; a doorway can be carved open by two beams that disagree

DERIVATION The chain of substitutions

$\pi^* \approx \text{approx}; \text{control} \circ \text{plan} \circ \text{explore} \circ \text{slam}$

Start from D26.2 and substitute one approximation at a time. Each step is a *choice*, and naming it is the whole point of the exercise.

Step 1 — factor the belief. Write $b_t(x, m) = p(x_t | z_{1:t}, u_{1:t}, \hat{m}) \delta(m - \hat{m}_t)$: replace the map marginal by a point mass at its MAP estimate $\hat{m}_t$. This is what a pose-graph SLAM system publishes and it is the reason a false loop closure is unrecoverable — there is no probability mass left anywhere else to recover *to*.

Step 2 — separate the objective. With m fixed at $\hat{m}_t$, the reward $-\Delta H(m)$ depends on the robot's *trajectory* only through which cells it observes. Choose a goal now and worry about the path later:

$$u_{\text{mission}}^* \approx \text{plan}(\hat{m}_t, g^*), \quad g^* = \arg \max_g \underbrace{I(g)}_{\text{cells revealed}} e^{-\lambda d(g)}$$

The exponential is the greedy discount of Chapter 24. It is a one-step lookahead over a set of candidate goals, which is why two frontiers of

nearly equal utility can make a robot oscillate: nothing in this expression knows that committing has value.

Step 3 — certainty equivalence in the map. Planning to g^* over $\hat{m}_t$ treats unknown cells as traversable with a penalty. That is deliberately optimistic: a pessimistic planner refuses to enter unknown space, and since every frontier is by definition adjacent to unknown space, a pessimistic explorer never explores. The price is paid whenever the optimism is wrong, and it is paid as a replan.

Step 4 — certainty equivalence in the pose. MPPI rolls out from μ_t , dropping Σ_t entirely:

$$u_t = \text{mppi}(\mu_t, \text{path}, d_{\text{esdf}}) \quad \text{instead of} \quad \arg \min_u \mathbb{E}_{x \sim \mathcal{N}(\mu_t, \Sigma_t)} [J(x, u)]$$

This is the substitution F2 repairs. Dropping Σ_t is exactly right when Σ_t is small and exactly catastrophic when it is not, so instead of restoring the expectation we restore a *bound*: keep the nominal trajectory far enough from obstacles that the true one is inside with probability $1 - \delta$.

Step 5 — read off the failures. Each substitution has a signature. Step 1 fails loudly (a sheared map) or silently (a confidently wrong loop). Step 2 fails as indecision. Step 3 fails as a replan. Step 4 fails as a collision — which is why it is the one we patch rather than merely detect. ■

26.4.3 F2 — Safety under pose uncertainty

Statement. Let $d_{\text{esdf}}(x)$ be the distance from x to the nearest obstacle in the map, r_{robot} the robot radius, and σ_{pose}^2 the largest eigenvalue of the position block of Σ_t . If every point of the planned trajectory satisfies

$$d_{\text{esdf}}(\mu) \geq r_{\text{robot}} + k_{\sigma} \sigma_{\text{pose}}, \quad k_{\sigma} = \Phi^{-1}(1 - \delta)$$

then each point of the *true* trajectory collides with probability at most δ .

DERIVATION From a Gaussian tail to a clearance in metres

$$d_{\text{esdf}}(\mu) \geq r_{\text{robot}} + \Phi^{-1}(1 - \delta) \sigma_{\text{pose}}$$

Step 1 — what a collision is. The robot is a disc of radius r centred at the true position x . It collides iff the true clearance is negative: $d_{\text{esdf}}(x) - r < 0$.

Step 2 — the error is Gaussian in the tangent plane. Write $x = \mu + e$ with $e \sim \mathcal{N}(0, \Sigma_{xy})$, Σ_{xy} the 2×2 position block. The distance field is 1-Lipschitz

— it is a distance — so for any unit vector $\hat{n}$,

$$d_{\text{esdf}}(\mu + e) \geq d_{\text{esdf}}(\mu) - \hat{n}^\top e$$

with equality in the worst case, when $\hat{n}$ points at the nearest obstacle.

Step 3 — project. The scalar $\hat{n}^\top e$ is Gaussian with variance $\hat{n}^\top \Sigma_{xy} \hat{n} \leq \lambda_{\max}(\Sigma_{xy}) = \sigma_{\text{pose}}^2$. We do not know $\hat{n}$ — the obstacle can be in any direction — so we take the worst case, which is the largest eigenvalue. This is why the margin uses $\lambda_{\max}$ and not, say, the trace: the nearest wall is free to lie along the belief's long axis, and in a corridor it usually does.

Step 4 — the tail bound. Collision requires $\hat{n}^\top e > d_{\text{esdf}}(\mu) - r$, so

$$P(\text{collision}) \leq 1 - \Phi\left(\frac{d_{\text{esdf}}(\mu) - r}{\sigma_{\text{pose}}}\right)$$

Setting the right-hand side to δ and solving gives the margin. ■

Two honest caveats. First, this bounds the collision probability *per point*, not per path. A path of n waypoints each safe at δ is safe at $n\delta$ by a union bound, so a genuine path-level guarantee needs δ/n per point — the Bonferroni correction of Exercise 1. Second, the bound assumes the pose error is actually Gaussian. It is not, particularly after a loop closure, and the steepness of the Gaussian tail that makes this margin so cheap is exactly what makes it fragile: a tenfold tightening of δ , from 1% to 0.1%, costs only 3.8 cm at $\sigma = 5$ cm, which should make you suspicious of how much safety you are really buying.

Worked example, checkable by hand. Take $\delta = 0.01$, so $k_\sigma = \Phi^{-1}(0.99) = 2.3263$ — that is the standard normal table, one entry. With Rusty's radius $r = 0.19$ m and a well-localized $\sigma_{\text{pose}} = 0.05$ m:

$$\text{margin} = 0.19 + 2.3263 \times 0.05 = \boxed{0.3063 \text{ m}}$$

Now cut the LiDAR while Rusty is at cruise. Each 0.1 s of open-loop prediction adds about $(0.11 \times 0.062)^2 \approx 4.9 \times 10^{-5} \text{ m}^2$ of along-track variance, so 2.5 s of driving *blind* would take σ_{pose} from 0.025 m to roughly 0.043 m and the margin from 0.248 m to $0.19 + 2.3263 \times 0.043 = 0.290$ m — a four-centimetre squeeze on every corridor.

It never gets that far, and the reason is worth stating plainly. The watchdog fires after 0.3 s, Rusty decelerates to rest in about half a second, and a robot that is not moving accumulates no process noise: σ_{pose} flattens at 0.027 m and the margin at 0.255 m and neither moves again until the scans come back. Stopping is not merely the cautious response to losing your sensor. It is the action that *bounds* σ , and that is why the margin and the watchdog are not redundant — the watchdog buys the stop, and the stop is what keeps the margin finite.

26.4.4 F3 — The latency budget

Statement. A dynamic obstacle approaching at v_{obs} is avoided rather than hit iff

$$\underbrace{v_{\text{obs}} (\varsigma_{\text{map}} + \tau_{\text{plan}} + \tau_{\text{ctrl}})}_{\text{how far it moves while we react}} + \underbrace{\frac{v_{\text{rusty}}^2}{2a_{\text{max}}}}_{\text{braking distance}} < d_{\text{detect}}$$

DERIVATION Chaining the delays

$$v_{\text{obs}} (\varsigma_{\text{map}} + \tau_{\text{plan}} + \tau_{\text{ctrl}}) + \frac{v_{\text{rusty}}^2}{2a_{\text{max}}} < d_{\text{detect}}$$

Step 1 — the pipeline delay. A measurement is only acted on after it has traversed every task between the sensor and the wheels. In the worst case each task has just published when the measurement arrives, so it waits a full period: the novelty flag waits ς_{map} for the next costmap, the costmap waits τ_{plan} for the next replan, and the path waits τ_{ctrl} for the next control tick. Delays in series add.

Step 2 — the obstacle keeps moving. During that delay the obstacle covers v_{obs} times the total.

Step 3 — and then you still have to stop. Once the command changes, the robot needs $v^2/2a_{\text{max}}$ to come to rest. Compare the sum with the range at which the obstacle is reliably detected. ■

The demo's numbers. The rate table gives $\varsigma_{\text{map}} = 0.1$ s (mapping at 10 Hz), $\tau_{\text{plan}} = 1.0$ s (replanning at 1 Hz), $\tau_{\text{ctrl}} = 0.05$ s (MPPI at 20 Hz), for a reaction delay of 1.15 s. Rusty cruises at 0.62 m/s and brakes at about 1.2 m/s², so braking costs $0.62^2/(2 \times 1.2) = 0.16$ m. Against the walker at 0.8 m/s:

$$0.8 \times 1.15 + 0.16 = 1.08 \text{ m} < d_{\text{detect}} \approx 2.5 \text{ m}$$

Comfortable — but the detection range on the right deserves justifying rather than asserting, because it is the term everyone fudges. A walker is flagged when at least three beams land in cells the map calls confidently free. Measured across six seeds, the range at which that first happens ran from 2.45 m to over 5 m, depending on how much of the surrounding map had been confidently cleared when the person arrived. Budget with the worst of them, not the median.

Now solve for the speed at which the bound stops holding: $v_{\text{obs}} \times 1.15 + 0.16 = 2.5$ gives $v_{\text{obs}} \approx 2.0$ m/s. That is Exercise 3, and the Grand Demo has a walker-speed slider so you can go looking for it.

Notice which term dominates. The replanning period alone is 1.0 of the 1.15 s reaction delay — 87% of it. Doubling the LiDAR rate would buy essentially nothing; moving the replan from 1 Hz to 5 Hz would take the reaction delay to 0.35 s and more than *triple* the safe obstacle speed. This is what a latency budget is for: it tells you which knob is worth turning, and the answer is very rarely the sensor.

26.4.5 F4 — Three detectors

Every approximation above is a claim about the world. The supervisor's job is to hold a statistic against each claim, and — this is the part people skip — to pick the threshold for a reason.

(a) Mislocalization, from a fitness ratio. Chapter 12 tracks the average particle weight at two timescales and compares them. Transplant the same detector onto scan-match fitness f_t — the fraction of beam endpoints landing within 25 cm of something the map already knows about:

$$w_{\text{fast}} \leftarrow w_{\text{fast}} + \alpha_{\text{fast}}(f_t - w_{\text{fast}}), \quad w_{\text{slow}} \leftarrow w_{\text{slow}} + \alpha_{\text{slow}}(f_t - w_{\text{slow}}), \quad \rho_t = \frac{w_{\text{fast}}}{w_{\text{slow}}}$$

with $\alpha_{\text{fast}} = 0.5 \gg \alpha_{\text{slow}} = 0.05$. The justification transfers verbatim: ρ is a *ratio*, so it is invariant to the absolute scale of the score, and the slow average supplies a baseline that no fixed threshold on f_t could — a fitness of 0.6 is excellent in a cluttered room and alarming in a corridor.

Note the statistic being fed in is deliberately stricter than ICP's own inlier count. ICP always converges to *something*; the question is not "did it converge?" but "does this sweep belong to this part of the map?"

(b) Divergence, from the innovation. The SLAM front end's correction has an innovation $v_t = \log(\hat{x}_t^{-1} x_t^{\text{icp}})$ and an innovation covariance $S_t = \bar{\Sigma}_t + R_t^{\text{icp}}$. Under a correctly specified filter $\epsilon_t = v_t^T S_t^{-1} v_t \sim \chi_3^2$ (Chapter 11's gate, reused). One excursion past the 95% point happens once every twenty scans by construction and means nothing; k in a row has probability 0.05^k under the null, which is one in 160 000 by $k = 4$. That is the entire design of the test, and it is why the gate reports a *streak* rather than a flag.

The same statistic gets a second, grosser threshold: past the 99.9% point of χ_3^2 twice running is not a filter that needs damping, it is a filter tracking the wrong hypothesis, and it routes to Relocalize rather than to Recover.

(c) Silence, from a watchdog. A message that never arrives produces no statistic to test, so nothing upstream can notice it. The watchdog notices the *absence*: scan age $\varsigma_{\text{scan}} > 3\tau_{\text{scan}}$. Three periods is long enough to ride out one dropped sweep and short enough that the covariance has not yet grown past the F2 margin.

DERIVATION Choosing $\rho_{\min}$, and the worked example the widget reproduces

$$\rho_1 = 0.7450, \quad \rho_2 = 0.6200$$

A threshold nobody can defend is a threshold that will be tuned until the alarm stops going off, which is how safety systems die. So measure the null distribution first.

Over four nominal missions with different seeds — 4232 scans in total, sampled after the first five seconds — the *smallest* ρ_t ever observed was 0.919 and the fifth percentile was 0.983. Setting $\rho_{\min} = 0.80$ therefore leaves twelve percentage points of headroom below anything a healthy filter has ever produced, and requiring two consecutive violations makes a false alarm from measurement noise alone essentially impossible.

Now the alarm, by hand. Suppose fitness has been steady at 0.98, so $w_{\text{fast}} = w_{\text{slow}} = 0.98$, and a kidnapping drops it to 0.44.

After the first bad scan:

$$w_{\text{fast}} = 0.98 + 0.5(0.44 - 0.98) = 0.71, \quad w_{\text{slow}} = 0.98 + 0.05(0.44 - 0.98) = 0.953$$

$$\rho_1 = \frac{0.71}{0.953} = 0.7450 < 0.80 \quad (\text{one violation})$$

After the second:

$$w_{\text{fast}} = 0.71 + 0.5(0.44 - 0.71) = 0.575, \quad w_{\text{slow}} = 0.953 + 0.05(0.44 - 0.953) = 0.92735$$

$$\rho_2 = \frac{0.575}{0.92735} = 0.6200 < 0.80 \quad (\text{alarm})$$

Two scans at 10 Hz is 0.2 s. Press Kidnap in the Grand Demo and read the event log: the KidnapSuspected entry lands within two tenths of a second of the injection, with $\rho \approx 0.60$. The test at the end of this chapter pins both numbers.

26.5 The algorithms

ALGORITHM `stack_tick(bus, tasks, now)` COST sum of the due tasks; ≈ 1.1 ms per quantum for the default apartment

in the message bus, the task set, the current time

out one scheduler quantum executed

```

1: for each task  $T_i$  in topological order do
2:   if  $\lfloor \text{now}/\tau_i \rfloor > \lfloor \text{last}_i/\tau_i \rfloor$  and  $T_i$  is enabled then
3:      $T_i.\text{tick}(\text{bus}, \text{now})$ 
4:      $\text{last}_i \leftarrow \text{now}$ 
5:   endif
6: endfor
7: for each  $T_i$  do  $\varsigma_i \leftarrow \text{now} - \text{last}_i$ 
```

Topological order matters and is worth dwelling on. Running tasks in dataflow order means a message published this quantum is consumed this quantum, so *all* observed staleness comes from task periods and none from the scheduler. That is precisely the property that makes the browser’s cooperative scheduler and the native build’s threads agree run-for-run — and it is also the property that lets F3 be an arithmetic statement about τ_i rather than a distribution over scheduling outcomes.

ALGORITHM supervisor_step(mode, events)

COST 0(1)

in the current mode, this quantum’s detector readings

out the next mode

```

1: if  $\varsigma_{\text{scan}} > 3\tau_{\text{scan}}$  then return Recover(SensorDropout)
2: if scans have resumed and mode is Recover(SensorDropout) then
   return the remembered mode
3: if a scan was matched this quantum then
4:   if  $\rho_t < \rho_{\min}$  for  $k_\rho$  consecutive scans then return Relocalize
5:   if  $\epsilon_t > \chi_{3,0.999}^2$  for 2 consecutive scans then return Relocalize
6:   if  $\epsilon_t > \chi_{3,0.95}^2$  for 4 consecutive scans then inflate  $\Sigma_t$ ; return
   Recover(Divergence)
7: match mode:
8:   Explore  $\rightarrow$  pick arg max utility frontier; Navigate if a path exists,
   else Done when D26.3 holds
9:   Navigate{g}  $\rightarrow$  Explore when g is reached, is no longer a frontier,
   is unreachable, or no progress for 4 s
10:  Relocalize  $\rightarrow$  Explore once the cloud is unimodal, tight, and
   verified against the live sweep
11:  Recover(Divergence)  $\rightarrow$  the remembered mode after a settling
```

```

    period
12:   Done → Done

```

Line 10 carries more weight than it looks. A converged particle cloud is a *hypothesis*, and accepting one without checking is how a robot ends up confidently in the wrong room. So the same discipline Chapter 16 applies to loop closure applies here: detection is cheap and often wrong, verification is expensive and has to be right. The verification is a direct one — re-score the live sweep against the map at the proposed pose and demand 75% agreement — and when it fails, the recovery scatters again rather than committing.

26.6 Implementation in Rust

Three listings: the bus, the supervisor, and the mission with its regression test.

26.6.1 The bus

Everything the stack sends is stamped and frame-tagged, and the frame is a *type*, not a string. That single decision is what makes an entire family of bugs — the family that puts a goal expressed in the map frame into a controller expecting the base frame — impossible rather than merely unlikely.

```

RUST crates/capstone/src/bus.rs

1 use crossbeam_channel::{bounded, Receiver, Sender};
2 use localize::GaussianBelief;           // Ch. 11: { mean: SE2, cov: Matrix3<f64> }
3 use nalgebra::Vector2;
4 use std::marker::PhantomData;
5
6 // A coordinate frame, as a zero-sized type. `Pose<Map>` and `Pose<Base>` are
7 // different types and cannot be mixed; the tag costs nothing at run time.
8 pub trait Frame: Copy + 'static {
9     const NAME: &'static str;
10 }
11 #[derive(Clone, Copy)] pub struct Map;
12 #[derive(Clone, Copy)] pub struct Odom;
13 #[derive(Clone, Copy)] pub struct Base;
14 impl Frame for Map { const NAME: &'static str = "map"; }
15 impl Frame for Odom { const NAME: &'static str = "odom"; }
16 impl Frame for Base { const NAME: &'static str = "base"; }
17
18 // D26.1: nothing crosses a task boundary without a time and a frame on it.
19 #[derive(Clone, Copy, Debug)]
20 pub struct Stamped<T, F: Frame> {
21     pub t: SimTime,
22     pub v: T,
23     _frame: PhantomData<F>,
24 }
25
26 impl<T, F: Frame> Stamped<T, F> {
27     pub fn new(t: SimTime, v: T) -> Self {
28         Self { t, v, _frame: PhantomData }

```

```

29     }
30     /// Age in seconds. This is ?_i, the quantity Derivation F3 budgets.
31     pub fn staleness(&self, now: SimTime) -> f64 {
32         now - self.t
33     }
34 }
35
36 #[derive(Clone)]
37 pub enum Msg {
38     Scan(Stamped<Scan, Base>),
39     Odom(Stamped<Twist2, Odom>),
40     /// Estimators publish *beliefs*. A pose alone cannot produce an F2 margin.
41     PoseBelief(Stamped<GaussianBelief, Map>),
42     MapPatch(Stamped<OccGridPatch, Map>),
43     Frontiers(Stamped<Vec<ScoredFrontier>, Map>),
44     Path(Stamped<Vec<Vector2<f64>>, Map>),
45     Cmd(Stamped<Cmd, Base>),
46     Event(StackEvent),
47 }
48
49 /// One capstone subsystem. The *same* impl runs threaded on native and
50 /// cooperatively on wasm; `Task: Send` is what the native runner needs, and it
51 /// is also what stops a task from smuggling an `Rc<RefCell<SmallRng>>` inside.
52 pub trait Task: Send {
53     fn name(&self) -> &'static str;
54     fn period(&self) -> f64;
55     fn tick(&mut self, bus: &mut Bus, now: SimTime);
56 }
57
58 /// Bounded channels, on purpose: an unbounded queue turns a slow consumer into
59 /// a memory leak and hides the very staleness F3 is trying to bound. When a
60 /// costmap consumer falls behind we would rather drop the stale patch than
61 /// deliver it late.
62 pub struct Bus {
63     tx: Sender<Msg>,
64     rx: Receiver<Msg>,
65 }
66
67 impl Bus {
68     pub fn new(capacity: usize) -> Self {
69         let (tx, rx) = bounded(capacity);
70         Self { tx, rx }
71     }
72     pub fn publish(&self, m: Msg) {
73         // A full bus is a scheduling bug, not a reason to block the producer.
74         let _ = self.tx.try_send(m);
75     }
76 }

```

26.6.2 The supervisor

The mode is an enum with data, the detectors are three small structs, and supervisor_step is one match. That last point is not stylistic: when Recover was added to Mode late in the writing of this book, the compiler produced an E0004 at every decision site and refused to build until each had been considered. A stringly-typed mode would have let the new state fall silently through to "do

nothing", which is what a robot in an unhandled state does off a loading dock.

RUST crates/capstone/src/tasks/supervisor.rs

```

1 use nalgebra::{Matrix3, Vector3};
2 use stats::distribution::{ChiSquared, ContinuousCDF};
3
4 #[derive(Clone, Copy, Debug, PartialEq)]
5 pub enum RecoverKind { SensorDropout, Divergence }
6
7 #[derive(Clone, Copy, Debug, PartialEq)]
8 pub enum Mode {
9     Explore,
10    Navigate { goal: FrontierId },
11    Relocalize,
12    Recover(RecoverKind),
13    Done,
14 }
15
16 /// F4(a). Chapter 12's dual EMA, fed scan-match fitness instead of particle
17 /// weight. The ratio is scale-free, which is why one pair of gains works in a
18 /// corridor and in a warehouse.
19 pub struct DualEmaDetector {
20     w_fast: f64,
21     w_slow: f64,
22     alpha_fast: f64,
23     alpha_slow: f64,
24     seeded: bool,
25     streak: u32,
26     rho_min: f64,
27     patience: u32,
28 }
29
30 impl DualEmaDetector {
31     pub fn rho(&self) -> f64 {
32         if self.w_slow > 0.0 { self.w_fast / self.w_slow } else { 1.0 }
33     }
34
35     /// Returns true when the alarm fires. Seeding both averages with the first
36     /// sample avoids a spurious spike on step two that is an artefact of
37     /// initialisation rather than a property of the data.
38     pub fn update(&mut self, fitness: f64) -> bool {
39         if !self.seeded {
40             self.w_fast = fitness;
41             self.w_slow = fitness;
42             self.seeded = true;
43             return false;
44         }
45         self.w_fast += self.alpha_fast * (fitness - self.w_fast);
46         self.w_slow += self.alpha_slow * (fitness - self.w_slow);
47         self.streak = if self.rho() < self.rho_min { self.streak + 1 } else { 0 };
48         self.streak >= self.patience
49     }
50 }
51
52 /// F4(b). A chi2 gate that reports a *streak*: one excursion past the 95% point
53 /// is expected once every twenty scans and means nothing.
54 pub struct ChiSquareGate {
55     threshold: f64,
56     patience: u32,
57     streak: u32,

```

```

58 }
59
60 impl ChiSquareGate {
61   pub fn at(dof: f64, quantile: f64, patience: u32) -> Self {
62     let chi = ChiSquared::new(dof).expect("dof > 0");
63     Self { threshold: chi.inverse_cdf(quantile), patience, streak: 0 }
64   }
65
66   pub fn update(&mut self, nis: f64) -> bool {
67     self.streak = if nis > self.threshold { self.streak + 1 } else { 0 };
68     self.streak >= self.patience
69   }
70 }
71
72 pub struct Supervisor {
73   mode: Mode,
74   resume: Mode,
75   fitness: DualEmaDetector,      // F4(a)
76   gross: ChiSquareGate,         // chi2_3 at 99.9%, patience 2
77   divergence: ChiSquareGate,    // chi2_3 at 95%,  patience 4
78   scan_watchdog: Watchdog,      // F4(c)
79 }
80
81 impl Supervisor {
82   pub fn step(&mut self, ev: &Telemetry, now: SimTime) -> Mode {
83     // A missing message produces no statistic, so absence is checked first.
84     if self.scan_watchdog.expired(now) {
85       self.resume = self.mode;
86       self.mode = Mode::Recover(RecoverKind::SensorDropout);
87       return self.mode;
88     }
89     if matches!(self.mode, Mode::Recover(RecoverKind::SensorDropout)) {
90       self.mode = self.resume;
91       return self.mode;
92     }
93
94     // Both remaining detectors are statistics *of a scan match*, so they may
95     // only be fed once per scan -- never once per scheduler quantum, which
96     // would silently halve their effective thresholds.
97     if ev.matched_this_tick {
98       if self.fitness.update(ev.icp_fitness) || self.gross.update(ev.nis) {
99         self.mode = Mode::Relocalize;
100        return self.mode;
101      }
102      if self.divergence.update(ev.nis) {
103        self.resume = self.mode;
104        self.mode = Mode::Recover(RecoverKind::Divergence);
105        return self.mode;
106      }
107    }
108
109     // The exhaustive match. Adding a variant to `Mode` breaks this build,
110     // which is the entire reason `Mode` is an enum.
111     self.mode = match self.mode {
112       Mode::Explore => match ev.best_frontier {
113         Some(g) if ev.path_exists => Mode::Navigate { goal: g },
114         _ if ev.stopping_criterion_met() => Mode::Done,
115         _ => Mode::Explore,
116       },
117       Mode::Navigate { goal } if ev.goal_finished(goal) => Mode::Explore,
118       Mode::Relocalize if ev.reloc_converged && ev.reloc_verified => Mode::Explore,

```

```

119         Mode::Recover(RecoverKind::Divergence) if now > self.settle_until => self.
        resume,
120         other => other,
121     };
122     self.mode
123 }
124 }

```

26.6.3 The margin, and the mission

The F2 margin is four lines, and it is the only place in the stack where the pose covariance is allowed to change what the robot does.

RUST crates/capstone/src/tasks/control.rs

```

1 use nalgebra::Matrix3;
2 use stats::distribution::{ContinuousCDF, Normal};
3
4 /// Largest position standard deviation: the square root of the larger
5 /// eigenvalue of the translation block, in closed form for 2*2.
6 pub fn position_sigma(cov: &Matrix3<f64>) -> f64 {
7     let (a, b, c) = (cov[(0, 0)], cov[(0, 1)], cov[(1, 1)]);
8     let mean = 0.5 * (a + c);
9     let disc = (0.25 * (a - c).powi(2) + b * b).max(0.0).sqrt();
10    (mean + disc).max(0.0).sqrt()
11 }
12
13 /// Derivation F2: d_esdf(x) >= r_robot + k_sigma sigma_pose, k_sigma = Phi-1(1 - delta).
14
15 ///
16 /// Note what happens as the filter loses confidence: the margin grows, the
17 /// planner finds fewer admissible cells, and eventually every MPPI rollout is
18 /// infeasible and the robot stops. Nobody wrote "stop when lost".
19 pub fn safety_margin(r_robot: f64, sigma_pose: f64, delta: f64) -> f64 {
20     let k_sigma = Normal::standard().inverse_cdf(1.0 - delta);
21     r_robot + k_sigma * sigma_pose
22 }

```

And the mission entry point, with the regression that CI runs on every commit:

RUST crates/capstone/src/mission.rs

```

1 pub struct MissionCfg {
2     pub seed: u64,
3     pub delta: f64,
4     pub rates: RateTable,
5     pub chaos: Vec<ChaosEvent>,
6 }
7
8 pub struct MissionReport {
9     pub coverage: f64, // fraction of *reachable* cells known
10    pub traj_rmse: f64, // against ground truth -- a simulator-only luxury

```

```

11     pub odom_error: f64,           // where dead reckoning ended up, for contrast
12     pub contacts: usize,
13     pub entropy_curve: Vec<(SimTime, f64)>,
14     pub events: Vec<(SimTime, StackEvent)>,
15 }
16
17 /// Deterministic under `cfg.seed`: same trajectory, same events, same numbers,
18 /// in `cargo test` and in the browser. That is why the wasm scheduler is
19 /// round-robin rather than preemptive.
20 pub fn run_mission(cfg: &MissionCfg) -> MissionReport { /* ... */ }
21
22 #[cfg(test)]
23 mod tests {
24     use super::*;
25     use approx::assert_relative_eq;
26
27     /// The worked example of Derivation F2, to the digit printed in the text.
28     #[test]
29     fn f2_margin_worked_example() {
30         assert_relative_eq!(
31             Normal::standard().inverse_cdf(0.99), 2.3263479, epsilon = 1e-6);
32         assert_relative_eq!(
33             safety_margin(0.19, 0.05, 0.01), 0.3063174, epsilon = 1e-6);
34         // A tenfold tighter bound costs under four centimetres. The Gaussian
35         // tail is steep, which is why this margin is cheap -- and why it is a
36         // poor defence against error that is not really Gaussian.
37         assert_relative_eq!(
38             safety_margin(0.19, 0.05, 0.001) - safety_margin(0.19, 0.05, 0.01),
39             0.03819, epsilon = 1e-4);
40     }
41
42     /// The worked example of Derivation F4: fitness steady at 0.98, then a
43     /// kidnapping drops it to 0.44. The alarm must fire on the second scan.
44     #[test]
45     fn f4a_dual_ema_worked_example() {
46         let mut d = DualEmaDetector::new(0.5, 0.05, 0.80, 2);
47         assert!(d.update(0.98)); // seeds w_fast = w_slow = 0.98
48         assert!(d.update(0.44));
49         assert_relative_eq!(d.rho(), 0.745016, epsilon = 1e-5);
50         assert!(d.update(0.44)); // second violation => alarm
51         assert_relative_eq!(d.rho(), 0.620047, epsilon = 1e-5);
52     }
53
54     /// The book's end-to-end regression. CI fails if any of these regress.
55     #[test]
56     fn seed_42_maps_the_apartment() {
57         let r = run_mission(&MissionCfg::seed(42));
58         assert!(r.coverage >= 0.99, "coverage {}", r.coverage);
59         assert!(r.traj_rmse <= 0.20, "rmse {}", r.traj_rmse);
60         assert_eq!(r.contacts, 0);
61         // SLAM must beat dead reckoning by an order of magnitude, or the scan
62         // matcher is not earning its place in the stack.
63         assert!(r.odom_error / r.traj_rmse >= 10.0);
64     }
65 }

```


i NOTE

Those three tests pass in the TypeScript port that runs the widgets on this page, in `lib/capstone/checks.ts`. Seed 42 maps the apartment in **87.4 s of simulated time**, reaching **99.7% coverage** with a trajectory RMSE of **0.124 m** and **zero** wall contacts, while dead reckoning over the same run ends up **2.90 m** from the truth. If the prose and the code ever disagree, the test settles it.

26.7 Breaking it on purpose

Three sabotages, one per row of F1's table, each with the statistic that is supposed to notice.

🖥 INTERACTIVE · w26.3**Failure Theater**

Recovery is not luck and not magic: it is a named statistic crossing a threshold, followed by a mode that has a plan.

Run this simulation in the web edition: [/chapters/ch26-capstone #w26.3](/chapters/ch26-capstone/#w26.3)

Kidnap attacks the assumption that the belief brackets the truth. Watch ρ : it is flat and near 1 for the whole warm-up, dives within two scans of the teleport, and the supervisor switches to `Relocalize`. Then watch what the recovery *does* — it stops mapping (integrating scans at a pose you have just admitted is wrong is the fastest way to destroy a map that was fine), abandons the goal, drops MPPI in favour of a reactive creep that steers by raw range data, and only commits when the cloud is unimodal, tight, *and* verified. That creep is not decoration: a stationary robot in a rectangular room can drive a particle cloud into one confident mode in half a second — in the wrong room, because from a single viewpoint the two rooms are the same measurement. Motion is what separates them. It is the cheapest possible taste of belief-space planning: act to disambiguate, then commit.

The recovery does not always succeed, and the widget does not hide it. The apartment's south rooms are deliberate mirror images of each other (Chapter 12 built that symmetry on purpose), so on some seeds the cloud settles in the mirrored room and the verification step accepts it — because at that pose the sweep really does match the map. That is Exercise 4.

Walker attacks the static-world assumption. A person crosses in front of the LiDAR; several beams stop early. The novelty test is a single line — a beam whose endpoint lands in a cell the map calls *confidently free* is a beam the map cannot explain — and it has two consequences, which are opposite on purpose. Those

beams are **withheld from mapping**, so the map never learns the person and does not need to unlearn them. And their endpoints are **injected into the controller's distance field as transient obstacles**, so MPPI steers around a person who is not in the map at all. Same measurement, opposite treatment, decided by which downstream consumer is asking.

Dropout attacks the assumption that messages arrive. It is the quietest failure and the most instructive: nothing errors. The estimator simply predicts without correcting, Σ_t grows, and the F2 margin grows with it. The chart plots both. Watch them rise for a few tenths of a second — and then watch them go *flat*, because by then the watchdog has fired and Rusty has stopped, and a stationary robot accumulates no process noise. The two mechanisms are doing different jobs: the watchdog is the fast detector, and the margin is the graceful degradation that would have stopped the robot anyway, a few seconds later, if the watchdog had not existed.

The interesting engineering question is what happens if the dropout lasts thirty seconds instead of two and a half, and the answer is that the stack behaves correctly and uselessly: it sits still, perfectly safe, indefinitely. Deciding what to do then — call for help, drive home on odometry alone, retry the sensor — is a product question, not a probability question, and this is the chapter to be honest about where that boundary is.

And one sabotage that is not a chaos button. The Grand Demo's *calibrated sensor model* toggle is Chapter 25's contribution to the stack, and switching it off is the most realistic failure on this page, because it is the one you inflict on yourself. An uncalibrated model claims a smaller σ than the sensor has and treats consecutive LiDAR beams as five times more independent than they are. Over five seeds, that takes the mean pose σ from 0.0176 m to 0.0127 m — a filter looking 28% more confident while being no more accurate — shrinks the F2 margin, and raises the mean NIS from 0.76 to 2.48. Three of the five runs raised a false alarm, and two of them failed to finish.

That is worth sitting with. Overconfidence did not show up as a bad estimate. It showed up as a *consistency* failure: the filter's own χ^2 test started rejecting the filter's own updates, and the supervisor spent the mission responding to alarms that were correct about the model and wrong about the world. Calibration is not a nicety at the bottom of the stack; it is what makes every detector above it meaningful.

26.8 What Rust cost, and what it bought

INTERACTIVE · w26.4

Retrospective Scorecard

A language choice is a trade, not a win: name what the type system caught and what it charged, at the same size.

Run this simulation in the web edition: [/chapters/ch26-capstone #w26.4](#)

Three honesty items that the panels above cannot express on their own.

The simulator grades its own homework. Trajectory RMSE against ground truth exists only because we own the world. On hardware there is no truth column, and evaluation becomes: held-out maps, loop-closure precision and recall, repeatability across repeated runs, and the map-consistency number Chapter 16 defined — how far the second pass over a corridor lands from the first. That last one is the most useful metric in this book precisely because a robot can compute it about itself.

The browser proves throughput, not scheduling. WASM is single-threaded here, so the in-page stack is cooperative, not preemptive. The identical-semantics claim holds because the scheduler is deterministic and topological, and that is a real and useful property — but it is not a proof that the native, threaded build meets its deadlines. Real-time scheduling is a claim about worst cases under contention, and nothing on this page tests that.

Every ecosystem statement here is dated. The crate versions in this book were pinned in August 2026. Sparse solvers, factor-graph libraries, and Lie-group crates in Rust are all younger than their C++ counterparts and all moving. Check before you trust.

26.9 Where to go next

Onto ROS 2. The architecture above was deliberately shaped like the modern ROS 2 navigation stack, so the mapping is one-to-one: our SLAM task is `slam_toolbox`, our ESDF layer is a Nav2 costmap layer, our planner is the planner server, our MPPI is the controller server (Nav2 ships one), and our supervisor is a behavior tree. Rust bindings exist — `rclrs` from the `ros2-rust` project, with `r2r` as an alternative — and porting a single task is the natural first step, because the message boundary is already exactly where the ROS topic would go. That is Exercise 6.

Onto hardware. The three things that break first are, in order: time (your sensor stamps and your clock disagree, and F3 becomes a distribution rather than an arithmetic statement), extrinsics (the LiDAR is not where the URDF says it is, and every scan match inherits the error), and the motion model (real wheels slip

in ways Chapter 9's α 's do not describe). None of these is a new algorithm. All of them are calibration, which is Chapter 25's subject.

Into three dimensions. Everything here generalizes, and most of it gets harder in one specific way: the map. Occupancy grids do not scale to 3-D at useful resolution, which is why Chapter 19 spent its time on octrees and TSDFs; the ESDF, the planner, and MPPI all carry over almost unchanged on top of them.

To more than one robot. Multi-robot SLAM is out of scope for a single-browser demo and is genuinely different: the hard part is not the estimation but deciding which map is whose and merging them without a shared frame. It is one of the liveliest areas in the field and a good place to read next.

💡 THE CLOSING CLAIM, STATED PLAINLY

Everything you studied in this book is running on this page, unmodified, at real time, with every internal inspectable. Not a simplified version of it; the same code, seeded so that your run and mine are the same run. That is what it means for a book to be honest about its algorithms.

26.10 Exercises

1 🏠 FOUNDATION •• From a point guarantee to a path guarantee

Derivation F2 bounds the collision probability at a single point of the trajectory. Show that a path of n waypoints, each individually safe at level δ , is only guaranteed safe at level $n\delta$, and derive the Bonferroni-corrected per-point level needed for a path-level guarantee at δ .

Then compute the cost: for a 30-waypoint path at $\delta = 0.01$ and $\sigma_{\text{pose}} = 0.05$ m, how much wider is the corrected margin than the naive one? Finally, argue in two sentences why the union bound is loose here — what is the relationship between the collision events at consecutive waypoints?

2 🏠 FOUNDATION ••• Write the mission down, then take it apart

Write the exploration mission of D26.2 as a formal POMDP tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma \rangle$, being explicit about what $\mathcal{S}$ contains and how large it is for the apartment at 15 cm resolution.

Then, from memory, name the approximation each of the five layers in F1's table makes and the failure it induces. Finally: if you had one graduate student and one year, which single assumption would you spend the effort removing, and what evidence from the widgets on this page supports that choice?

3 FOUNDATION •• A threshold you can defend

The chapter sets $\rho_{\min} = 0.80$ with patience 2 by measuring the null distribution over four nominal missions. Suppose instead you measure a mean of $\bar{\rho} = 1.00$ with standard deviation 0.03 and decide to set the threshold at $\bar{\rho} - 3s$.

(a) What false-alarm rate per scan does that imply if ρ were Gaussian, and what rate per hour at 10 Hz? (b) With patience k , how does that rate change, and what does patience cost you in detection latency? (c) ρ is a ratio of two correlated EMAs and is not Gaussian. Which direction does that error most likely go, and what would you measure instead of assuming?

4 CONCEPTUAL •• Predict the walker speed that wins

Using F3 and the numbers in the Timing tab of the Grand Demo, predict the walker speed at which the stack starts failing to avoid the person. Write your prediction down before you test it.

Now test it: raise the walker-speed slider and press Walker repeatedly, watching the novelty count and the trajectory. Then change one rate at a time in your head — LiDAR to 20 Hz, replanning to 5 Hz, control to 50 Hz — and rank them by how much each raises the safe speed. Which term in F3 dominated, and by how much?

One honest complication to account for in your answer: the corridor is 1.2 m wide, so a fast walker crosses the robot's path in well under a second and the geometry, not just the latency, decides the outcome. Does that make your prediction optimistic or pessimistic?

5 CONCEPTUAL ••• Find a seed where recovery is confidently wrong

In the Failure Theater's Kidnap tab, re-roll the seed until you find a run where the particle cloud converges, passes the verification gate, and is nevertheless in the wrong place. (Turn on the ground-truth overlay in the Grand Demo to confirm.)

Explain the symmetry that caused it in terms of the apartment's floorplan. Then propose two fixes and say what each costs: one that changes the *sensor* and one that changes the *behavior*. Which would you ship?

6 PRACTICAL •• Add a ReturnHome mode

Add `Mode::ReturnHome` to the supervisor: after Done, plan a path back to the starting pose and drive it, then stop. The change should touch only `supervisor.rs` and `mission.rs`.

Do it by adding the variant *first* and building before writing any other code, so you experience the E0004 cascade from the Retrospective Scorecard firsthand. Count the sites the compiler flags. Then ask: how many of those would a `_ => {}` arm have hidden, and what would each have done at run time?

7 PRACTICAL ●●● Make the margin honest about non-Gaussian error

F2 assumes the pose error is Gaussian. Replace that assumption with a sampled one: draw N poses from the current belief, evaluate d_{esdf} at each, and set the margin so that at most δN samples are in collision — a sample-average approximation of the chance constraint.

Implement it in the control task, compare margins against the closed form under a Gaussian belief (they should agree as N grows), and then break the agreement: run the comparison immediately after a `Relocalize` completes, when the belief is a freshly-collapsed particle cloud rather than a Gaussian. Report the disagreement in centimetres, and say whether the closed form was optimistic or pessimistic.

26.11 References

Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics* MIT Press.

The book this one modernizes. It has no integration chapter — the reader finishes knowing filters, maps, and planners but never sees them composed with rates, interfaces, and failure handling. That gap is what Chapter 26 exists to fill. [link](#)

Yamauchi, B. (1997). A Frontier-Based Approach for Autonomous Exploration *Proceedings of the 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, 146–151.

The original frontier idea, and still the one the capstone’s explorer implements: drive to the boundary between known-free and unknown, and when none remains the map is done. doi:10.1109/CIRA.1997.613851

Blackmore, L., Ono, M., and Williams, B. C. (2011). Chance-Constrained Optimal Path Planning With Obstacles *IEEE Transactions on Robotics* 27(6), 1080–1094.

The rigorous version of Derivation F2, including the risk-allocation machinery that replaces this chapter’s crude Bonferroni correction when you need a path-level guarantee that is not wasteful. doi:10.1109/TR0.2011.2161160

Williams, G., Drews, P., Goldfain, B., Rehg, J. M., and Theodorou, E. A. (2018). Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving *IEEE Transactions on Robotics* 34(6), 1603–1622.

The MPPI formulation the control task uses, with the free-energy derivation of the exponential weighting that Chapter 23 follows. doi:10.1109/TR0.2018.2865891

Macenski, S. and Jambrecic, I. (2021). SLAM Toolbox: SLAM for the Dynamic World *Journal of Open Source Software* 6(61), 2783.

The production counterpart of the capstone’s SLAM task: scan-to-map matching against a persistent map with a pose-graph back end. Read it to see which of this chapter’s simplifications a deployed system does not make. doi:10.21105/joss.02783

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W. (2022). Robot Operating System 2: Design, Architecture, and Uses in the Wild *Science Robotics* 7(66), eabm6074.

What the bus in this chapter becomes at production scale — typed topics, QoS policies, lifecycle-managed nodes. The design rationale is the best available argument for why stamped, frame-tagged messages are not pedantry. doi:10.1126/scirobotics.abm6074

Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A., and Ferguson, M. (2023). From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2 *Robotics and Autonomous Systems* 168, 104493.

Written by the Nav2 maintainers, and the fastest way to map every task in this chapter onto a production counterpart by name — including the costmap layers, the planner server, and the MPPI controller. doi:10.1016/j.robot.2023.104493

Placed, J. A., Strader, J., Carrillo, H., Atanasov, N., Indelman, V., Carlone, L., and Castellanos, J. A. (2023). A Survey on Active Simultaneous Localization and Mapping: State of the Art and New Frontiers *IEEE Transactions on Robotics* 39(3), 1686–1705.

Where to go after the greedy frontier scorer: this is the modern treatment of the exploration objective in D26.2, including the belief-space formulations that would remove the first approximation in F1’s table. doi:10.1109/TR0.2023.3248510